



ADVANCES IN DEEP LEARNING AND COMPUTER VISION

Dr. Jagadeesh Kumar

Xoffencer

ADVANCES IN DEEP LEARNING AND COMPUTER VISION

First Edition 2024

Author:

- Dr. Jagadeesh Kumar

Xoffencer

www.xoffencerpublication.in

Copyright © 2024 Dr. Jagadeesh Kumar

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through Rights Link at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law.

ISBN-13: 978-93-48116-19-2 (Paperback)

Publication Date: 18 December 2024

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

MRP: ₹ 550/-



Published by:

***Xoffencer International Book Publication House
Behind shyam vihar vatika, laxmi colony
Dabra, Gwalior, M.P. – 475110***

Cover Page Designed by:

Satyam soni

Contact us:

Email: mr.xoffencer@gmail.com

Visit us: www.xofferncerpublishing.in

Copyright © 2024 Dr. Jagadeesh Kumar

Author Details



Dr. Jagadeesh Kumar

Professor

Marik Institute of Computing, Artificial Intelligence, and Machine Learning

NIMS Institute of Engineering and Technology

NIMS University Rajasthan, Jaipur - 303121, India

Dr. Jagadeesh Kumar, Professor in the Department of Robotics and AI, NIMS University Rajasthan, is a renowned Scientist, Researcher, and Academician. He received his educational credentials in Engineering, Technology, and Management from the Top Universities and a notable Alumnus of Harvard University, United States and University of Oxford, United Kingdom. He authored more than 100 journal, conference, and magazine articles indexed in Scopus, Web of Science and SCI. Also, he is an author of more than 50 international books.

**This book is dedicated to
NIMS University, Rajasthan, Jaipur**

ACKNOWLEDGMENT

**Stanford University
United States**

**University of Oxford
United Kingdom**

**Harvard University
United States**

Preface

The text has been written in simple language and style in well organized and systematic way and utmost care has been taken to cover the entire prescribed procedures for Science Students.

We express our sincere gratitude to the authors not only for their effort in preparing the procedures for the present volume, but also their patience in waiting to see their work in print. Finally, we are also thankful to our publishers **Xoffencer Publishers, Gwalior, Madhya Pradesh** for taking all the efforts in bringing out this volume in short span time.

Preview

Computer vision has enormous progress in modern times. Deep learning has driven and inferred a range of computer vision problems, such as object detection and recognition, face detection and recognition, motion tracking and estimation, transfer learning, action recognition, image segmentation, semantic segmentation, robotic vision.

The chapters in this book are persuaded towards the applications of advanced computer vision using modern deep learning techniques.

The author trust in making the readers with more interesting illustrations in understanding the concepts of deep learning and computer vision at a simpler perspective approach.

Contents

		Page No.
Chapter 1	FUNDAMENTALS OF COMPUTER VISION	1-72
	1.1 What Motivates Us?	2
	1.2 What is Computer Vision?	3
	1.3 Why is Computer Vision Important?	5
	1.4 What is Image Formation?	7
	1.5 What is Low-Level Vision?	9
	1.6 What is Middle-Level Vision?	12
	1.7 What is High-Level Vision?	13
	1.8 What are Computer Vision Algorithms?	15
	1.9 Why is Computer Vision Difficult?	25
	1.10 What is This Area About?	31
	1.11 What are the Application Areas?	32
	1.12 What are the Related Fields?	42
Chapter 2	MACHINE LEARNING AND COMPUTER VISION	73-184
	2.1 What is Machine Learning?	75
	2.2 Machine Learning in Our Everyday Life	83
	2.3 Where to Apply Machine Learning?	96
	2.4 Is Machine Learning A Computer Program?	99
	2.5 What are the Skills Required for Machine Learning?	101
	2.6 Its Connection with Artificial Intelligence	
	2.7 Artificial Neural Networks and Machine Learning	103
	2.8 What is Statistical Learning?	107
	2.9 Role of Machine Learning in Computer Vision	109
	2.10 What are the Categories of Machine Learning?	112
	2.11 What are Machine Learning Algorithms?	114
		133
Chapter 3	MODERN DEEP LEARNING APPROACHES	185-253
	3.1 What is Deep Learning?	187
	3.2 Where to Apply Deep Learning?	190
	3.3 Its Connection with Machine Learning	193
	3.4 Neural Networks and Deep Learning	198
	3.5 Role of Deep Learning in Computer Vision	201
	3.6 What are Modern Deep Learning Architectures?	202

	3.7 What are Recent Deep Learning Algorithms?	228
Chapter 4	RECENT TRENDS IN COMPUTER VISION	254-338
	4.1 Introduction	254
	4.2 What is Visual Question Answering?	255
	4.3 What is 3D Object Understanding?	261
	4.4 What is Object Recognition with Point Cloud?	263
	4.5 What are Semantic and Instance Segmentation?	265
	4.6 What is Transfer Learning?	273
	4.7 What is Deep Domain Adaptation?	283
	4.8 What is Simultaneous Location and Mapping?	289
	4.9 What is Augmented Reality?	297
	4.10 What is 3D Reconstruction?	304
	4.11 What is 3D Motion Tracking?	311
	4.12 What is Medical Image Analysis?	330
Chapter 5	ADVANCED COMPUTER VISION TECHNIQUES	339-439
	5.1 What is Advanced Computer Vision?	339
	5.2 What is Pose Estimation?	339
	5.3 What are Generative Adversarial Networks?	349
	5.4 What are Style GAN and Image Synthesis?	363
	5.5 What is Language And Vision?	369
	5.6 What are Smart Mirrors?	376
	5.7 What is Merged Reality?	383
	5.8 What are Video Analytics?	392
	5.9 What are Graph Neural Networks?	399
	5.10 What are Visual and Facial Recognition?	411
	5.11 What is Data Extraction and Analysis?	421
	5.12 What is Detection and Recognition?	432

CHAPTER 1

FUNDAMENTALS OF COMPUTER VISION

Computer Vision (CV) can be defined as “the hypothesis and innovation for building artificial frameworks that acquire data from pictures or multi-dimensional information.” A more straightforward clarification is that computer vision endeavors to take care of similar issues you can unravel with your own one of kind eyes. For instance, in case you're driving and you see a kid run into the street, your mind will rapidly translate the kid in the street in front of you, that it's perilous, and that you ought to quickly brake to abstain from hitting the kid. That is one of the issues self-driving vehicle engineers are presently striving to comprehend by the methods of computer vision. The method requires being competent of realizing object recognition, which can be subdivided into three varieties: object classification, identification, and detection.

Object Classification is everywhere you have a little recently learned objects that you need to have the option to perceive in a picture. Characterizing a representation photograph as having individual's face in it is a model object classification, arranged that this photograph contains a face in it.

Object Identification is the recognition of a specific instance of an object. For example, being able to identify that there are two faces in an image and that one is John and the other is Sarah is an example of object identification.

Object Detection is the ability to identify that there's an object in an image. This is typically used for things like automatic toll roads where you want to know when a new object has entered the frame so you can take a scan the license plate.

Connecting this to the self-driving car problem, if you think to how the human brain would solve this problem, it would have to answer the same questions: In order for the situation to be dangerous, we would have to both identify that there is a child (object) in or approaching the road. Identify that the child in the road is something that we should avoid. You would also want to identify other objects, like trash, soccer ball, bike, etc., where you don't necessarily need evasive action. In other words, Computer Vision is the field of study that seeks to develop techniques to help computers “see” and understand the content of digital images such as photographs and videos. The problem of computer vision appears simple because it is trivially solved by people, even very young children. Nevertheless, it largely remains an unsolved problem based both on the limited understanding of biological vision and because of the complexity of vision perception in a dynamic and nearly infinitely varying physical world.

Smartphones have cameras, and taking a photo or video and sharing it has never been easier, resulting in the incredible growth of modern social networks like Instagram.

YouTube might be the second largest search engine and hundreds of hours of video are uploaded every minute and billions of videos are watched every day. The internet is comprised of text and images. It is relatively straightforward to index and search text, but in order to index and search images, algorithms need to know what the images contain. For the longest time, the content of images and video has remained opaque; best described using the meta descriptions provided by the person that uploaded them. To get the most out of image data, we need computers to “see” an image and understand the content. This is a trivial problem for human, even young children.

-  A person can describe the content of a photograph they have seen once.
-  A person can summarize a video that they have only seen once.
-  A person can recognize a face that they have only seen once before.

We require at least the same capabilities from computers in order to unlock the images and videos. “Computer vision as a field is a scholarly wilderness. Like any boondocks, it is energizing and scattered, and there is frequently no solid power to speak to. Numerous helpful thoughts have no hypothetical establishing, and a few hypotheses are pointless by and by; created zones are generally dispersed, and regularly one looks totally difficult to reach from the other.

1.1 WHAT MOTIVATES US?

An interesting question we always ask is what the next generation of computers is going to be like. To answer this question, let’s recall our first touch of computer. At least, our experience was that we waved our hands and said “how are you” to a machinery. Obviously, no answer at all. It was a dream that computers would be able to see and think like a human, which has been driving us to explore various research issues to make this dream come true as shown in Figure 1.1. Although computers become faster and faster, they are still quite dull, since they can neither see nor even perform simple reasoning. Obviously, we are not satisfied to just use our computers as a calculator, a word processor, a CD player, or a game station; instead, we expect computers to do more intelligent things like our human beings. “Computer vision is the extraction of data from pictures. Data can mean anything from 3D models, camera position, object location and answer to gathering and looking through picture content.

For example,

-  Can computers identify me by looking at my face or even my gait?

- ✚ Can computers know where I am looking at and what I am doing?
- ✚ Can computers tell what a car is and what is not a car?
- ✚ Can computers learn something by themselves?
- ✚ Can computers summarize a video for me?

1.2 WHAT IS COMPUTER VISION?

Computer vision is basically to infer different factors such as camera model, lighting, color, texture, shape and motion that affect images and videos, from visual inputs. A rough structure of machine vision could be illustrated by Figure 1.2. In a word, computer vision is an inverse processing of the forward process of image formation and graphics. In this sense, as many people agree, vision is a much more challenging problem than computer graphics, because it is full of uncertainties.



Fig. 1.1: Computer Vision and Self-Driving Car

“The objective of computer vision is to extricate helpful data from pictures. This has demonstrated a shockingly testing assignment; it has involved a huge number of savvy and inventive personalities in the course of the most recent four decades, and notwithstanding this we are still a long way from having the option in building a broadly useful computer vision.

Computer vision is a field of study focused on the problem of helping computers to see. It is a multidisciplinary field that could broadly be called a subfield of artificial intelligence and machine learning, which may involve the use of specialized methods and make use of general learning algorithms as illustrated in Figure 1.3. One particular problem in vision may be easily addressed with a hand-crafted statistical method, whereas another may necessitate a large and complex ensemble of generalized machine learning algorithms.

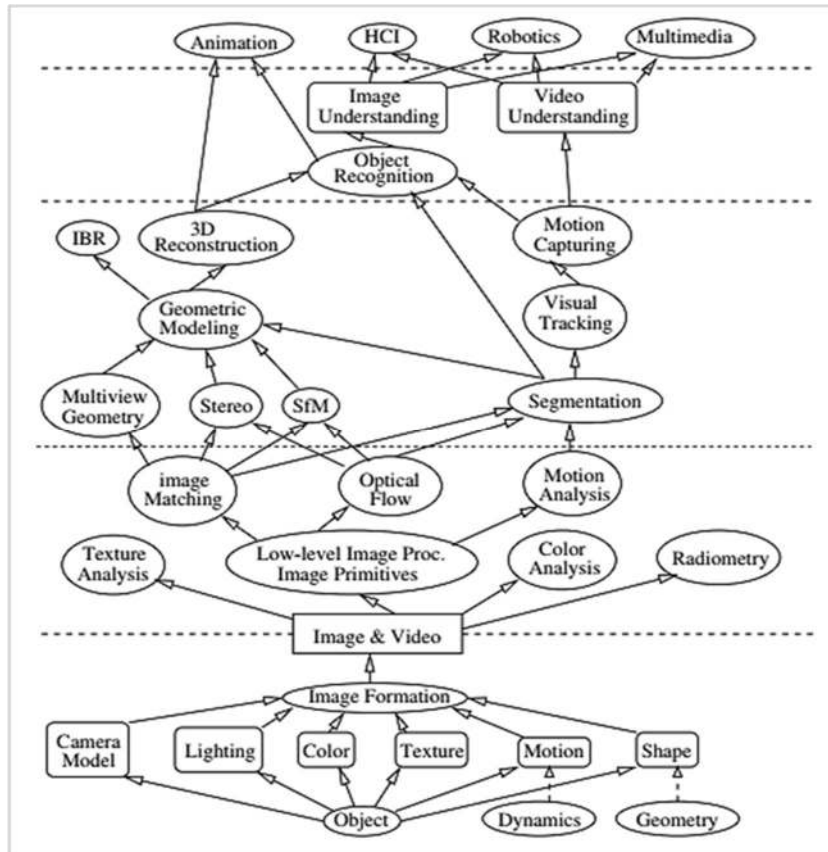


Fig. 1.2: What Is Computer Vision?

As a multidisciplinary area of study, it can look messy, with techniques borrowed and reused from a range of disparate engineering and computer science fields. One particular problem in vision may be easily addressed with a hand-crafted statistical method, whereas another may require a large and complex ensemble of generalized machine learning algorithms. One particular problem in vision may be easily addressed with a hand-crafted statistical method, whereas another may require a large and complex ensemble of generalized machine learning algorithms. The goal of computer vision is to understand the content of digital images. Typically, this involves developing methods that attempt to reproduce the capability of human vision. Understanding the content of digital images may involve extracting a description from the image, which may be an object, a text description, a three-dimensional model, and so on.

Computer vision is distinct from image processing. Image processing is the process of creating a new image from an existing image, usually simplifying or enhancing the content in some way. It is a category of digital signal processing and is not concerned with understanding the

content of an image. A given computer vision system may require image processing to be applied to raw input, e.g. pre-processing images. Image Processing is the subset of Computer Vision. At this time, transformations applied to an input image include sharpening, smoothing, stretching, and the resultant output image are returned. Helping computers to see turns out to be very hard. Computer vision seems easy, maybe as it is so effortless for humans.

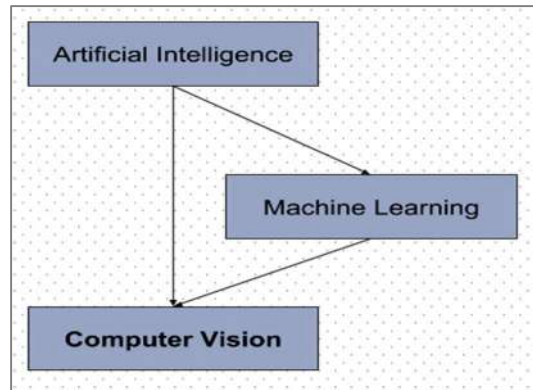


Fig. 1.3: Computer Vision as a Multidisciplinary Field.

“Building a computer vision was something that driving specialists in the field of Artificial Intelligence thought to be at the degree of trouble of a late spring understudy's undertaking, harking back to the sixties. After sixty years the assignment is as yet unsolved and appears to be imposing.

Examples of image processing include:

- ✚ Normalizing photometric properties of the image, such as brightness or color.
- ✚ Cropping the bounds of the image, such as centering an object in a photograph.
- ✚ Removing digital noise from an image, such as digital artifacts from low light levels.

1.3 WHY IS COMPUTER VISION IMPORTANT?

Initially, it was believed to be a trivially simple problem that could be solved by a student connecting a camera to a computer. After decades of research, “computer vision” remains unsolved, at least in terms of meeting the capabilities of human vision (Figure 1.4). One reason is that we don’t have a strong grasp of how human vision works. Studying biological vision requires an understanding of the perception organs like the eyes, as well as the interpretation of the perception within the brain. Much progress has been made, both in charting the process and in terms of discovering the tricks and shortcuts used by the system, although like any study that involves the brain, there is a long way to go.

“Perceptual clinicians have gone through decades attempting to see how the visual framework functions and, despite the fact that they can devise optical fantasies to prod separated a portion of its standards; a total answer for this riddle stays subtle. Computer vision is being used in the real world for things like self-driving cars and pedestrian detection, but in plenty of other situations as well: face recognition, gesture recognition, optical character recognition, augmented reality, digital video fingerprinting, iris recognition, people counting, reverse image search, and more. Another reason why it is such a challenging problem is because of the complexity inherent in the visual world. A given object may be seen from any orientation, in any lighting conditions, with any type of occlusion from other objects, and so on.

A true vision system must be able to “see” in any of an infinite number of scenes and still extract something meaningful. Computers work well for tightly inhibited problems, not open unbounded problems like visual perception. Nevertheless, there has been progress in the field, especially in recent years with commodity systems for optical character recognition and face detection in cameras and smartphones. Computer vision is also useful in an industrial context, where CV can be used to detect product defects. Toll bridges are another area that has been disrupted by CV. Rather than stopping to pay a toll; a camera reads the license plate as you drive by and charges your account therefore. “Computer vision is at a phenomenal point in its advance. The subject itself has been around since the 1960s, yet as of late has it been plausible to fabricate helpful computer frameworks utilizing thoughts from Machine Learning, Signal and Image processing.

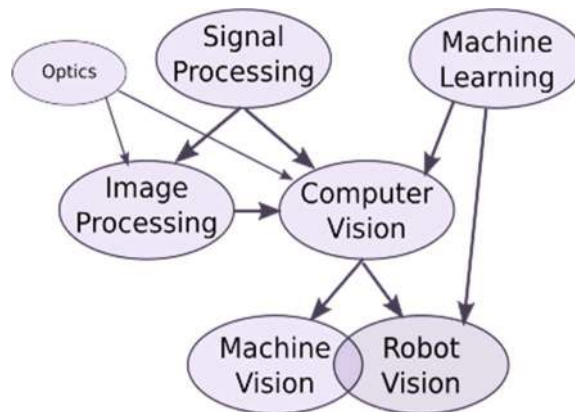


Fig. 1.4: Capability of Computer Vision in Signal and Image Processing

“Computer vision has a wide assortment of utilizations, both old (e.g., portable robot routing, modern assessment, and military knowledge) and new (e.g., human computer connection, picture recovery in computerized libraries, medical picture investigation, and the sensible

rendering of engineered scenes in computer illustrations). It may be helpful to zoom in on some of the simpler computer vision tasks that are likely to encounter or be interested in solving given the vast number of publicly available digital photographs and videos available. Many popular computer vision applications involve trying to recognize things in photographs; for example:

- 📷 **Object Classification:** What broad category of object is in this photograph?
- 📷 **Object Identification:** Which type of a given object is in this photograph?
- 📷 **Object Verification:** Is the object in the photograph?
- 📷 **Object Detection:** Where are the objects in the photograph?
- 📷 **Object Landmark Detection:** What are the key points for the object in the photograph?
- 📷 **Object Segmentation:** What pixels belong to the object in the image?
- 📷 **Object Recognition:** What objects are in this photograph and where are they?

Other common examples are related to information retrieval; for example: finding images like an image or images that contain an object.

1.4 WHAT IS IMAGE FORMATION?

Image formation studies the forward process of producing images and videos. It is an important research topic for both vision and graphics. To produce a real image, the nature of the visual sensors, i.e., cameras should be studied. In terms of geometrical aspects of camera, people have been looking into pinhole cameras, cameras with lenses and even omnidirectional cameras. In terms of physical aspects, factors such as focal lengths and dynamic ranges of CCD and CMOS cameras have been investigated. Besides the imaging device, it is also important to study the factors from objects and scenes themselves, such as lighting, color, texture, motion and shape, which largely affect the appearance of images and video. Before discussing the image formation on analog and digital cameras, we have to first discuss the **image formation on human eye** (Figure 1.5).

Because the basic principle that is followed by the cameras has been taken from the way, the human eye works. When light falls upon the particular object, it is reflected back after striking through the object. The rays of light when passed through the lens of eye form a particular angle, and the image is formed on the retina which is the back side of the wall. The image that is formed is inverted. This image is then interpreted by the brain and that makes us able to understand things. Due to angle formation, we are able to perceive the height and depth of the object we are seeing. This has been more explained in the tutorial of perspective transformation. When sun light falls on the object (in this case the object is a tree), it is reflected back and

different rays form different angle when they are passed through the lens and an invert image of the object has been formed on the back wall.

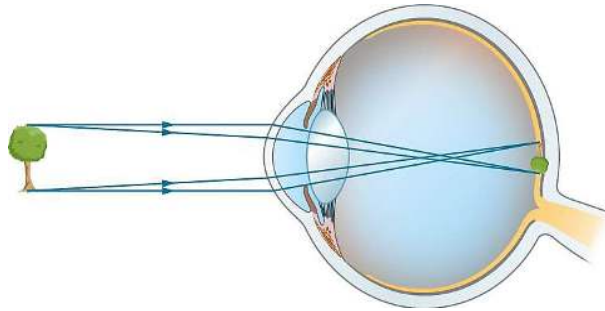


Fig. 1.5: Image Formation on Human Eye

In **analog cameras**, the image formation is due to the chemical reaction that takes place on the strip that is used for image formation. A 35mm strip is used in analog camera. This strip is coated with silver halide (a chemical substance). Light is nothing but just the small particles known as photon particles. So when these photon particles are passed through the camera, it reacts with the silver halide particles on the strip and it results in the silver which is the negative of the image. This is just the basics, although image formation involves many other concepts regarding the passing of light inside, and the concepts of shutter and shutter speed and aperture and its opening. In the digital cameras, the image formation is not due to the chemical reaction that takes place; rather it is a bit more complex than this.

In the **digital camera**, a CCD array of sensors is used for the image formation. CCD stands for charge-coupled device. It is an image sensor, and like other sensors it senses the values and converts them into an electric signal. In case of CCD it senses the image and converts it into electric signal etc. This CCD is actually in the shape of array or a rectangular grid. It is like a matrix with each cell in the matrix contains a sensor that senses the intensity of photon. Like analog cameras, in the case of digital too, when light falls on the object, the light reflects back after striking the object and allowed to enter inside the camera.

Each sensor of the CCD array itself is an analog sensor. When photons of light strike on the chip, it is held as a small electrical charge in each photo sensor. The response of each sensor is directly equal to the amount of light or (photon) energy strike on the surface of the sensor. An image as a two dimensional signal and due to the two dimensional formation of the CCD array, a complete image can be achieved from this CCD array. It has limited number of sensors, and it means a limited detail can be captured by it. Also each sensor can have only one value against each photon particle that strike on it. So the number of photons striking (current) are counted and stored. In order to measure accurately these, external CMOS sensors are also attached with

CCD array. The value of each sensor of the CCD array refers to each the value of the individual pixel.

The number of sensors is equal to the number of pixels. It also means that each sensor could have only one and only one value (Figure 1.6). The charges stored by the CCD array are converted to voltage one pixel at a time. With the help of additional circuits, this voltage is converted into digital information and then it is stored. Each company that manufactures digital camera, make their own CCD sensors. That includes Sony, Mistubishi, Nikon, Samsung, Toshiba, FujiFilm, Canon etc. Apart from the other factors, the quality of the image captured also depends on the type and quality of the CCD array that has been used.

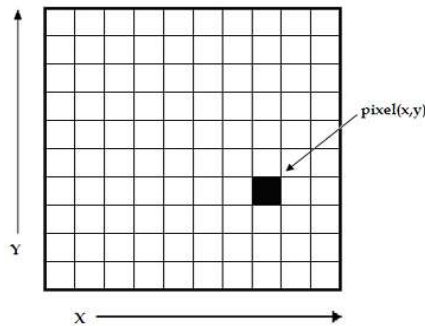


Fig. 1.6: 2D Array of Image Pixels

1.5 WHAT IS LOW-LEVEL VISION?

Low-level image processing is not vision, but the pre-processing steps for vision. The basic task is to extract fundamental image primitives for further processing, including edge detection, corner detection, filtering, and morphology, etc. Based on low-level image processing, low-level vision tasks could be achieved, such as image matching, optical flow computation and motion analysis. The field of computer vision has seen a meteoric rise in the recent past, with the development of a wide variety of techniques to accomplish certain tasks.

These tasks include motion analysis, scene reconstruction, and image matching. Image matching basically is to find correspondences between two or more images. These images could be the same scene taken from different view points, or a moving scene taken by a fixed camera, or both. Constructing image correspondences is a essentially important problem in vision for both geometry recovery and motion recovery. Without exaggeration, image matching is part of the base for vision. Optical flow is a kind of image observation of motion, but it is not the true motion. Since it only measures the optical changes in images, an aperture problem is unavoidable. But based on optical flows, camera motion or object motion could be estimated.

1.5.1 Deep Image Matching

- ✚ Establishment of the correspondence between each pair of visible homologous image points on a given pair of images, aiming at the evaluation of novelties.
- ✚ The act of checking a distance functions between two sets of points which belong to some images taken from the same content.
- ✚ Establishment of the correspondence between each pair of visible homologous image points on a given pair of images, aiming at the evaluation of novelties.
- ✚ Deep Image Matching is a matching algorithm used to compute dense correspondences amid two images. Deep Image Matching relies on a deep, multi-layer, convolutional architecture designed for matching images (Figure 1.7). It can hold non-rigid distortions and repetitive textures, and can thus efficiently determine dense correspondences in the presence of major changes between images.

1.5.2 Optical Flow Computation and Motion Analysis

Estimating the motion of every pixel in a sequence of images is a problem with many applications in computer vision, such as image segmentation, object classification, visual odometry, and driver assistance. In general, optical flow describes a sparse or dense vector field, where a displacement vector is assigned to certain pixel position that points to where that pixel can be found in another image. In the context of scene flow estimation, which is performed on images with additional depth values, every pixel is assigned a depth displacement as well. Since much of the structural information of a 3D scene gets lost in the imaging process, so does the motion information. The estimation of the "correct" projected motion in an image sequence is therefore highly ill-posed and has to be aided by additional priors such as the regularity of the motion.

Our group mainly focuses on optical flow estimation by means of variational methods that allow a clear formulation of the assumptions incorporated into the estimation process and generally produce dense vector fields. Optical flow is defined as the apparent motion of individual pixels on the image plane. It often serves as a good approximation of the true physical motion projected onto the image plane. Most methods that compute optical flow assume that the color and or intensity of a pixel are invariant under the displacement from one video frame to the next. Optical flow provides a concise description of both the regions of the image undergoing motion and the velocity of motion. In practice, computation of optical flow is susceptible to noise and illumination changes. Applications include which used optical flow to detect and track vehicles in an automated traffic surveillance practice. Optical flow, or motion estimation, is a fundamental method of calculating the motion of image intensities, which may be ascribed to the motion of objects in the scene.

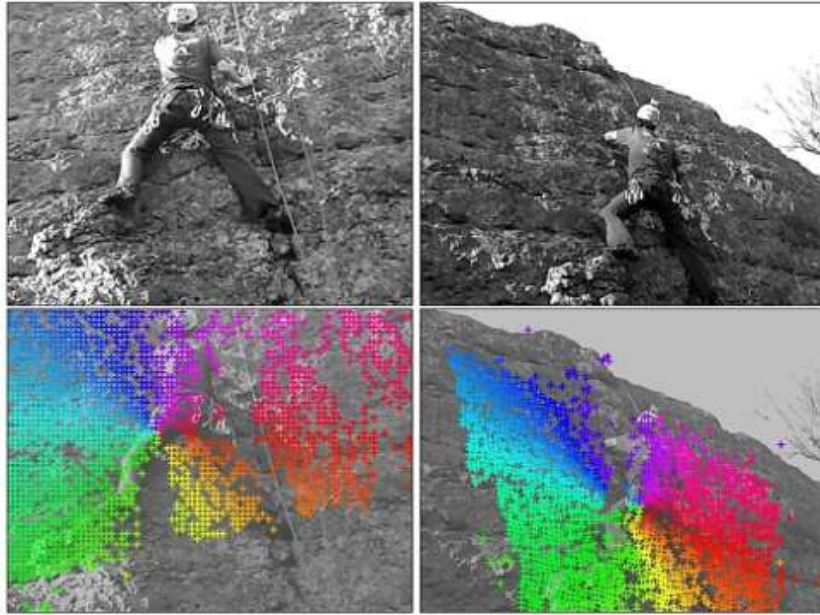


Fig. 1.7: Deep Image Matching

Optical flow is an extremely fundamental concept that is utilized in one form or another in most video-processing algorithms. Optical-flow methods are based on computing estimates of the motion of the image intensities over time in a video. The flow fields can then be analyzed to produce segmentations into regions, which might be associated with moving objects. Optical-flow or motion-estimation algorithms can be used to detect and delineate independently moving objects, even in the presence of camera motion. Of course, optical-flow-based techniques are computationally complex, and hence require fast hardware and software solutions to implement. Since optical flow is fundamentally a differential quantity, estimation of it is highly susceptible to noise; ameliorating the noise sensitivity can imply increases in complexity. Therefore, smart camera-based video surveillance systems that use optical-flow calculations of some type must be equipped with substantial computational resources, for example, a devoted DSP chip, FPGA, or other special-purpose acceleration device.

Indeed, it is because such technologies are being introduced to smart camera technology (along with wireless-networking capabilities) that video surveillance systems having significant intelligent capability are being envisioned and realized. While the accuracy of optical flow estimation has increased markedly, a number of problems remain, most notably the treatment of motion and image boundaries, the tracking of fast but small/thin objects, and the computational complexity of current methods. Once the optical flow is computed, it can be used to learn video-level representation for gesture recognition.

1.6 WHAT IS MIDDLE-LEVEL VISION?

There are two major characteristics in middle-level vision: (1) inferring the geometry and (2) inferring the motion. These two aspects are not sovereign but highly related. A simple question is can we estimate geometry based on just one image? The answer is obvious. We need at least two images. They could be taken from two cameras or come from the motion of the scene. Some fundamental parts of geometric vision include multi-view geometry, stereo and structure from motion, which fulfill the step of from 2D to 3D by inferring 3D scene information from 2D images. Based on that, geometric modelling is to construct 3D models for objects and scenes, such that 3D reconstruction and image-based rendering could be made possible.

Another task of middle-level vision is to answer the question how the object moves. Firstly, we should know which areas in the images belong to the object, which is the task of image segmentation. Image segmentation has been a challenging fundamental problem in computer vision for decades. Segmentation could be constructed on spatial similarities and continuities. However, uncertainty cannot be overcome for static image. When considering motion continuities, we hope the uncertainty of segmentation could be alleviated. On top of that is visual tracking and visual motion capturing, which estimate 2D and 3D motions, including deformable motions and articulated motions. In contrast to low-level vision, which recovers information from the sensors, mid-level vision concerns how this information gets organized into what we experience as objects and surfaces. "Mid-level vision" is nearly coextensive with what is called "perceptual organization." Mid-level perception provides the policy with a more processed state of the world compared to raw images.



Fig. 1.8: Object Tracking through Deep Sort Algorithm

Visual Object Tracking is one of the principal challenges in Computer Vision, where the task is to locate a certain object in all frames of a video, given only its location in the first frame. Object Tracking is another important domain in computer vision. It involves the process of

tracking an object which could be a person, ball or a car across a series of frames. For people tracking we would start with all possible detections in a frame and give them an ID. In subsequent frames we try to carry forward a person's ID. If the person has moved away from the frame then that ID is dropped. If a new person appears then they start off with a fresh ID. This is a difficult task since people could look similar causing the model to switch IDs, people may get occluded as in when a pedestrian or player gets hidden behind someone else or objects may disappear and reappear in later frames. Deep Learning has allowed getting a phenomenal performance on tracking. Tracking is an important problem in Computer Vision with tons of applications. Deep Sort algorithm is quite powerful and fast to run and is a good starting point for many cases.

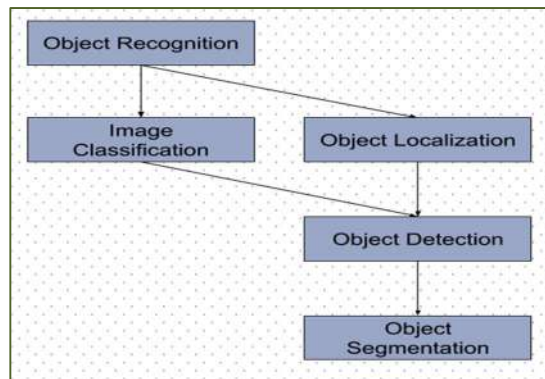


Fig. 1.9: Object Recognition.

1.7 WHAT IS HIGH-LEVEL VISION?

High-level vision is to infer the semantics, for example, object recognition (Figure 1.9) and scene understanding. An inspiring query in many decades is that how to achieve invariant recognition, i.e., recognize 3D object from different view directions. There have been two approaches for recognition: model-based recognition and learning-based recognition. It is noticed that there was a spiral development of these two approaches in history. Even higher level vision is image understanding and video understanding. We are interested in answering questions like ‘Is there a car in the image?’ or ‘Is this video a drama or an action?’, or ‘Is the person in the video jumping?’ Based on the answers of these questions, we should be able to fulfill different tasks in intelligent human-computer interaction, intelligent robots, smart environment and content-based multimedia.

Object recognition and classification, involves recognizing objects despite large variations in appearance caused by changes in viewing position, illumination, occlusion, and object shape. The second, visual cognition, involves the extraction of shape properties and spatial relations

in the course of performing visual tasks such as object manipulation, planning movements in the environment, or interpreting graphical material such as diagrams, graphs and maps. Understanding vision is a key problem in the brain sciences, human cognition, and artificial intelligence. Because of the interdisciplinary nature of the theories, High-Level Vision will be of interest in all three of these fields. Object recognition concerns the identification of an object as a specific entity i.e., semantic recognition or the ability to tell that one has seen the object before i.e., episodic recognition. Interest in object recognition is at least partly caused by the development of a new theory of human object recognition. Building on Marr's recognition ideas, where perceptually derived 3-D volumetric descriptions are matched to those stored in memory, Biederman in addition incorporated Gestalt principles of perceptual grouping and some newer ideas from computer vision into his theory.

In object recognition, the object as a whole must be segmented as a part, the shapes of the parts and their interrelations must then be represented in a way that is suitable for indexing a catalogue of visual categories. Hoffman and Richards showed that objects naturally can be segmented into parts prior to describing the shape of the parts. The Transversality principle applied that that when an object consisting of two parts is stucked together, the joint between the two parts forms discontinuity that is concave or has its greatest curvature to the surrounding area. But with the extension of the principle that points where the curvature is greatest divides images into parts we do not have to precede segmentation into parts. The approach to represent shapes is widely shared. Instead of representing the shape of the entire object, the problem is split into representing the spatial locations of the parts and their shapes. The technique of generalized cylinders plays a key role in the theories of shape.

The development of a cylinder can be done by moving a circle along a straight line that is perpendicular to its center. We can generalize this notion to be valid for any two-dimensional shape along an axis, be able to contract or expand or let the axis be curved. Another simplification to the technique is made if we restrict ourselves to describe parts separated by local concavities. Further research by Biederman successfully verified the theory of cylinder representation and discovered that rapid object identification can be made by four parameters, edges, symmetry and size of the cross-section and the curvature of axis. This scheme, consisting of four parameters is able to generate 36 different types of parts. If we assume that objects typically have more than one part we can build several million objects combining the different types of parts. The next step is to provide a process describing the arrangements of parts into entire objects.

This is one of the most difficult problems of high-level vision. One approach is the hierarchical representation, where some parts are represented relative to or as part of other parts. For example, an arm consists of the upper arm, the forearm and the hand, also the arm is a part of

the body. Some representations in LTM are called canonical because they consist of one single description which characterizes the object independent of viewpoint. If the LTM catalogue now is build up on basis of hierarchical and cylindral representation, the remaining process is matching an object-centered representation built on low-level information with the knowledgebase. One problem that can occur is that some of the needed parameters are distorted by the viewpoint or parts of the object may be hidden. These two problems in the best case only delay the retrieval. Mental rotation may also be needed in some cases when the object is asymmetric or complex.

1.8 WHAT ARE COMPUTER VISION ALGORITHMS?

An algorithm that makes computer vision possible and best for many tasks is a computer vision algorithm. e.g. convolutional neural networks (CNNs). CNNs are a form of deep learning that attempts to mimic how the brain understands objects in images. One application of CNNs is the ability to identify the saliency of an image automatically or detecting emotions in photos and to apply artsy filters to images and video. As Computer Vision represents a relative understanding of visual environments and their contexts, many scientists believe the field paves the way towards Artificial General Intelligence due to its cross-domain mastery. Computer Vision is one of the hottest research fields within Deep Learning at the moment. It sits at the intersection of many academic subjects, such as

-  Computer Science (Graphics, Algorithm, Theory, Systems, Design)
-  Mathematics (Information Retrieval, Machine Learning)
-  Engineering (Robotics, Speech, NLP, Image Processing)
-  Physics (Optics)
-  Biology (Neuroscience)
-  Psychology (Cognitive Science)

Computer Vision Algorithms is the most obvious answer in solving a fast growing collection of useful applications derived from this field of study. Here are just a handful of them:

-  **Face Detection Algorithms:** Snapchat, Facebook use face detection algorithms to apply filters and recognize you in pictures.
-  **Image Retrieval Algorithms:** Google Images uses content-based queries to search relevant images. The algorithms analyze the data in the query image and return results based on best-matched content.
-  **Gaming and Control Algorithms:** A great commercial product in gaming that uses stereo vision is Microsoft Kinect.
-  **Surveillance Algorithms:** Surveillance cameras are ubiquitous at public locations and are used to detect suspicious behaviors.

- 📌 **Biometric Algorithms:** Fingerprint, iris and face matching remains some common methods in biometric identification.
- 📌 **Smart vision Algorithms:** Vision remains the main source of data to detect traffic signs and lights and other visual features.

Recent developments in neural networks and deep learning approaches have greatly advanced the performance of these state-of-the-art visual recognition systems.

Case Study 1:

Given a set of images that are all labeled with a single category, predict the categories for a novel set of test images and measure the accuracy of the predictions.

There are a variety of challenges associated with this task, including viewpoint variation, scale variation, intra-class variation, image deformation, image occlusion, illumination conditions, and background clutter. How might one go about writing an algorithm that can classify images into distinct categories? Computer Vision researchers have come up with a data-driven approach to solve this. Instead of trying to specify what every one of the image categories of interest look like directly in code, they provide the computer with many examples of each image class and then develop learning algorithms that look at these examples and learn about the visual appearance of each class. In other words, they first accumulate a training dataset of labeled images, and then feed it to the computer to process the data. Given that fact, the complete **image classification** pipeline can be formalized as follows:

- 📌 The input is a training dataset that consists of N images, each labeled with one of K different classes.
- 📌 Then, use this training set to train a classifier to learn what every one of the classes looks like.
- 📌 In the end, evaluate the quality of the classifier by predicting the labels for a new set of images that it's never seen before. Then compare the true labels of these images to the ones predicted by the classifier.

The most popular algorithm used for image classification is Convolutional Neural Networks (CNNs). A typical use case for CNNs is where you feed the network images and the network classifies the data. CNNs tend to start with an input “scanner” which isn't intended to parse all the training data at once. For example, to input an image of 100×100 pixels, you wouldn't want a layer with 10,000 nodes. Rather, you create a scanning input layer of say 10×10 which you feed the first 10×10 pixels of the image. Once you passed that input, you feed it the next 10×10 pixels by moving the scanner one pixel to the right. This technique is known as sliding windows.

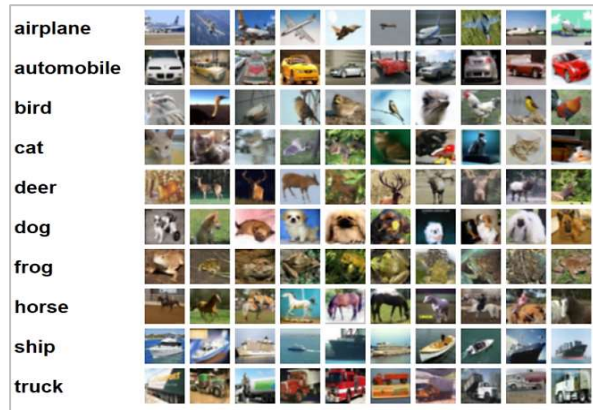


Fig. 1.10: Image Classification using Convolutional Neural Networks.

This input data is then fed through convolutional layers instead of normal layers. Each node only concerns itself with close neighboring cells. These convolutional layers also tend to shrink as they become deeper, mostly by easily divisible factors of the input. Besides these convolutional layers, they also often feature pooling layers. Pooling is a way to filter out details: a commonly found pooling technique is max pooling, say a, 2x2 pixels and passes on the pixel with the most amount of a certain attribute. Most image classification techniques nowadays are trained on ImageNet, a dataset with approximately 1.2 million high-resolution training images.

Test images will be presented with no initial annotation (no segmentation or labels), and algorithms will have to produce labelings specifying what objects are present in the images. Some of the best existing computer vision methods were tried on this dataset by leading computer vision groups from Oxford, INRIA, and XRCE. Typically, computer vision systems use complicated multi-stage pipelines, and the early stages are typically hand-tuned by optimizing a few parameters. A deep convolutional neural network with seven hidden layers, not counting some max pooling layers can be realized. The early layers were convolutional, while the last two layers were globally connected. The activation functions were rectified linear units in every hidden layer. These train much faster and are more expressive than logistic units. In addition to that, it also uses competitive normalization to suppress hidden activities when nearby units have stronger activities. This helps with variations in intensity.

Case Study 2:

Given an image, detect the cars with their bounding boxes.

The task to define objects within images usually involves outputting bounding boxes and labels for individual objects. This differs from the classification and localization task by applying

classification and localization to many objects instead of just a single dominant object. There are only two classes of object classification, which means object bounding boxes and non-object bounding boxes. For example, in car detection, you have to detect all cars in a given image with their bounding boxes. If we use the Sliding Window technique like the way we classify and localize images, we need to apply a CNN to many different crops of the image. Because CNN classifies each crop as object or background, we need to apply CNN to huge numbers of locations and scales, which is very computationally expensive!

In order to cope with this, neural network researchers have proposed to use regions instead, where we find “blobby” image regions that are likely to contain objects. This is relatively fast to run. The first model that kicked things off was R-CNN (Region-based Convolutional Neural Network). In R-CNN, we first scan the input image for possible objects using an algorithm called Selective Search, generating ~2,000 region proposals. Then we run a CNN on top of each of these region proposals. Finally, we take the output of each CNN and feed it into an SVM to classify the region and a linear regression to tighten the bounding box of the object. Essentially, we turned object detection into an image classification problem.

However, there are some problems; the training is slow, a lot of disk space is required, and inference is also slow. An immediate descendant to R-CNN is Fast R-CNN, which improves the detection speed through two augmentations, one by performing feature extraction before proposing regions, thus only running one CNN over the entire image, and another by replacing SVM with a softmax layer, thus extending the neural network for predictions instead of creating a new model. Fast R-CNN performed much better in terms of speed, because it trains just one CNN for the entire image. However, the selective search algorithm is still taking a lot of time to generate region proposals. Thus comes the invention of Faster R-CNN, which now is a canonical model for deep learning-based object detection. It replaces the slow selective search algorithm with a fast neural network by inserting a Region Proposal Network (RPN) to predict proposals from features.

The RPN is used to decide “where” to look in order to reduce the computational requirements of the overall inference process. The RPN quickly and efficiently scans every location in order to assess whether further processing needs to be carried out in a given region. It does that by outputting k bounding box proposals each with two scores representing the probability of object or not at each location. RPNs for efficient and accurate region proposal generation. By sharing convolutional features with the down-stream detection network, the region proposal step is nearly cost-free and improves the overall object detection quality and accuracy.

Once we have our region proposals, we feed them straight into what is essentially a Fast R-CNN. We add a pooling layer, some fully-connected layers, and finally a SoftMax

classification layer and bounding box regressor. Altogether, Faster R-CNN achieved much better speeds and higher accuracy. It's worth noting that although future models did a lot to increase detection speeds, few models managed to outperform Faster R-CNN by a significant margin. In other words, Faster R-CNN may not be the simplest or fastest method for object detection, but it's still one of the best performing. Faster R-CNN is composed of two modules. The first module is a deep fully convolutional network that proposes regions, and the second module is the Fast R-CNN detector that uses the proposed regions. The entire system is a single, unified network for **object detection**.

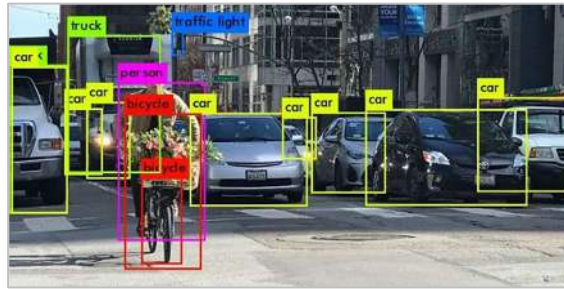


Fig. 1.11: Object Detection using Region-based Convolutional Neural Network (R-CNN).

Major Object Detection trends in recent years have shifted towards quicker, more efficient detection systems. This was visible in approaches like You Only Look Once (YOLO), Single Shot MultiBox Detector (SSD), and Region-Based Fully Convolutional Networks (R-FCN) as a move towards sharing computation on a whole image. Hence, these methods differentiate themselves from the costly subnetworks associated with the three R-CNN techniques. The main rationale behind these trends is to avoid having separate algorithms focus on their respective subproblems in isolation, as this typically increases training time and can lower network accuracy.



Fig. 1.12: Object Detection using Fast Region-based Convolutional Neural Network (Fast R-CNN)

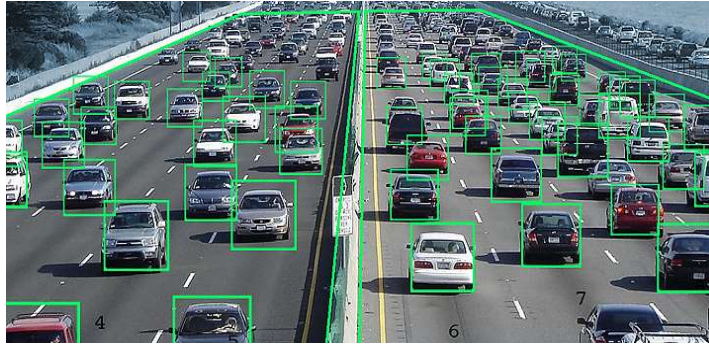


Fig. 1.13: Object Detection using Fast Region-based Convolutional Neural Network (Faster R-CNN).

Case Study 3:

Given an image, track a specific object using deep learning.

Object Tracking refers to the process of following a specific object of interest, or multiple objects, in a given prospect. It traditionally has applications in video and real-world interactions where observations are made following initial object detection. Now, it's crucial to autonomous driving systems such as self-driving vehicles from companies like Uber and Tesla. Object tracking methods can be divided into two categories according to the observation model: generative method and discriminative method. The generative method uses the generative model to describe the apparent characteristics and minimizes the reconstruction error to search the object, such as Principal Component Analysis (PCA).

The discriminative method can be used to distinguish between the object and the background, its performance is more robust, and it gradually becomes the main method in tracking. The discriminative method is also referred to as Tracking-by-Detection, and deep learning belongs to this category. To achieve tracking-by-detection, we detect candidate objects for all frames and use deep learning to recognize the wanted object from the candidates. There are two kinds of basic network models that can be used: stacked auto encoders (SAE) and convolutional neural network (CNN). The most popular deep network for tracking tasks using SAE is Deep Learning Tracker (DLT), which proposes offline pre-training and online fine-tuning the net. The process works like this:

- Off-line unsupervised pre-train the stacked denoising auto-encoder using large-scale natural image datasets to obtain the general object representation. Stacked denoising auto-encoder can obtain more robust feature expression ability by adding noise in input images and reconstructing the original images.

- ✚ Combine the coding part of the pre-trained network with a classifier to get the classification network, then use the positive and negative samples obtained from the initial frame to fine-tune the network, which can distinguish the current object and background.
- ✚ DLT uses particle filter as the motion model to produce candidate patches of the current frame.
- ✚ The classification network outputs the probability scores for these patches, meaning the confidence of their classifications, and then chooses the highest of these patches as the object.
- ✚ In the model updating, DLT uses the way of limited threshold.

Because of its superiority in image classification and object detection, CNN has become the mainstream deep model in computer vision and in visual tracking. Generally speaking, a large-scale CNN can be trained both as a classifier and as a tracker. Two representative CNN-based tracking algorithms are fully-convolutional network tracker (FCNT) and multi-domain CNN (MD Net). FCNT analyzes and takes advantage of the feature maps of the Visual Geometry Group (VGG) model successfully, which is a pre-trained ImageNet, and results in the following observations:

- ✚ CNN feature maps can be used for localization and tracking.
- ✚ Many CNN feature maps are noisy or un-related for the task of discriminating a particular object from its background.
- ✚ Higher layers capture semantic concepts on object categories
- ✚ Lower layers encode more discriminative features to capture intra-class variation.

Because of these observations, FCNT designs the feature selection network to select the most relevant feature maps on the conv4-3 and conv5-3 layers of the Visual Geometry Group network. Then in order to avoid overfitting on noisy ones, it also designs extra two channels (called SNet and GNet) for the selected feature maps from two layers' separately. The GNet captures the category information of the object, while the SNet classifies the object from a background with a similar appearance. Both of the networks are initialized with the given bounding-box in the first frame to get heat maps of the object, and for new frames, a region of interest (ROI) centered at the object location in the last frame is cropped and propagated. At last, through SNet and GNet, the classifier gets two heat maps for prediction, and the tracker decides which heat map will be used to generate the final tracking result according to whether there are distractors.

Different from the idea of FCNT, MD Net uses all the sequences of a video to track movements in them. The networks mentioned above use irrelevant image data to reduce the training

demand of tracking data, and this idea has some deviation from tracking. The object of one class in this video can be the background in another video, so MD Net proposes the idea of multi-domain to distinguish the object and background in every domain independently. And a domain indicates a set of videos that contain the same kind of object. MD Net is divided into 2 parts: the shared layers and the K branches of domain-specific layers. Each branch contains a binary classification layer with softmax loss, which is used to distinguish the object and background in each domain, and the shared layers sharing with all domains to ensure the general representation.

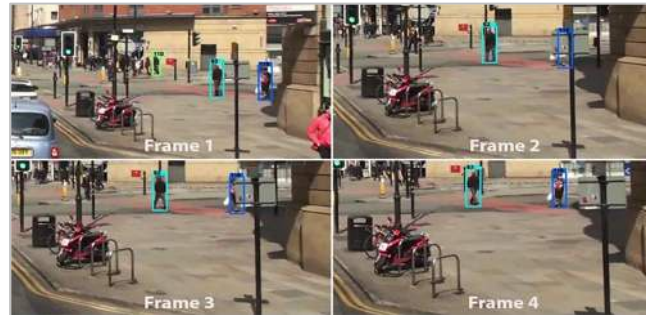


Fig. 1.14: Object Tracking using Fully-Convolutional Network Tracker

In recent years, deep learning researchers have tried different ways to adapt to features of the visual tracking task. There are many directions that have been explored: applying other network models such as

- ✚ Recurrent Neural Net
- ✚ Deep Belief Net

Designing the network structure to adapt to video processing and end-to-end learning, optimizing the process, structure, and parameters, or even combining deep learning with traditional methods of computer vision or approaches in other fields such as Language Processing and Speech Recognition gives better result.

Case Study 4:

Given an image, delineate the boundaries of each object.

Central to Computer Vision is the process of segmentation, which divides whole images into pixel groupings which can then be labelled and classified. Particularly, **Semantic Segmentation** tries to semantically understand the role of each pixel in the image (e.g. is it a car, a motorbike, or some other type of class?). For any given image, apart from recognizing

the person, the road, the cars, the trees, etc., one also has to delineate the boundaries of each object. Therefore, unlike classification, we need dense pixel-wise predictions from our models. As with other computer vision tasks, CNNs have had enormous success on segmentation problems. One of the popular initial approaches was patch classification through a sliding window, where each pixel was separately classified into classes using a patch of images around it. This, however, is very inefficient computationally because we don't reuse the shared features between overlapping patches.

The solution, instead, is UC Berkeley's Fully Convolutional Networks (FCN), which popularized end-to-end CNN architectures for dense predictions without any fully connected layers. This allowed segmentation maps to be generated for images of any size and was also much faster compared to the patch classification approach. Almost all subsequent approaches to semantic segmentation adopted this paradigm. However, one problem remains: convolutions at original image resolution will be very expensive. To deal with this, FCN uses down-sampling and up-sampling inside the network. The down-sampling layer is known as striped convolution, while the up-sampling layer is known as transposed convolution.

Despite the up-sampling and down-sampling layers, FCN produces coarse segmentation maps because of information loss during pooling. SegNet is a more memory efficient architecture than FCN that uses max pooling and an encoder-decoder framework. In SegNet, shortcut/ skip connections are introduced from higher resolution feature maps to improve the coarseness of up-sampling and down-sampling. Recent research in Semantic Segmentation all relies heavily on fully convolutional networks, such as Dilated Convolutions, DeepLab, and RefineNet.

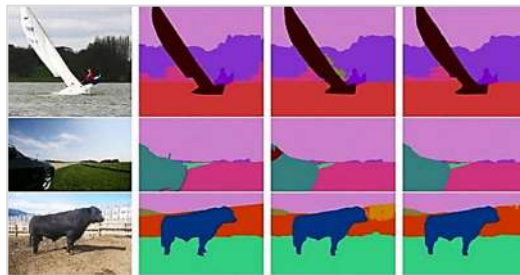


Fig. 1.15: Semantic Segmentation using Fully Convolutional Networks

Case Study 5:

Given an image, segment diverse instance of classes.

Beyond Semantic Segmentation, **Instance Segmentation** segments diverse instances of classes, such as labelling 5 cars with 5 different colors. In classification, there's generally an

image with a single object as the focus and the task is to say what that image is. But in order to segment instances, we need to carry out far more complex tasks. We see complicated sights with multiple overlapping objects and different backgrounds, and we not only classify these different objects but also identify their boundaries, differences, and relations to one another! So far, we've seen how to use CNN features in many interesting ways to effectively locate different objects in an image with bounding boxes. Can we extend such techniques to locate exact pixels of each object instead of just bounding boxes? This instance segmentation problem is explored at Facebook AI using an architecture known as Mask R-CNN.

Much like Fast R-CNN, and Faster R-CNN, Mask R-CNN's underlying intuition is straightforward given that Faster R-CNN works so well for object detection, could we extend it to also carry out pixel-level segmentation? Mask R-CNN does this by adding a branch to Faster R-CNN that outputs a binary mask that says whether or not a given pixel is part of an object. The branch is a Fully Convolutional Network on top of a CNN-based feature map. Given the CNN Feature Map as the input, the network outputs a matrix with 1s on all locations where the pixel belongs to the object and 0s elsewhere, this is known as a binary mask. Additionally, when run without modifications on the original Faster R-CNN architecture, the regions of the feature map selected by RoIPool (Region of Interests Pool) were slightly misaligned from the regions of the original image. Since image segmentation requires pixel-level specificity, unlike bounding boxes, this naturally led to inaccuracies. Mask R-CNN solves this problem by adjusting RoIPool to be more precisely aligned using a method known as RoIAlign (Region of Interests Align). Essentially, RoIAlign uses bilinear interpolation to avoid error in rounding, which causes inaccuracies in detection and segmentation. Once these masks are generated, Mask R-CNN combines them with the classifications and bounding boxes from Faster R-CNN to generate such wonderfully precise segmentations.

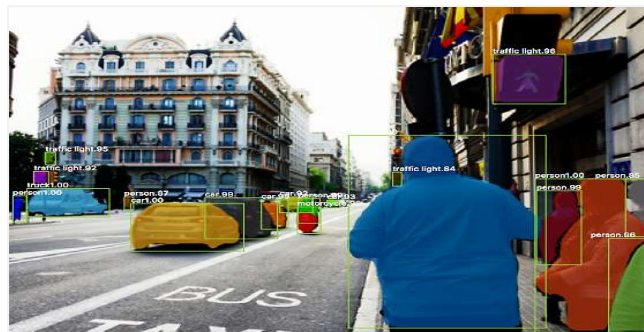


Fig. 1.16: Instance Segmentation using Mask R-CNN

The above case studies in the five major computer vision techniques such as Image Classification, Object Detection, Object Tracking, Semantic Segmentation, and Instance

Segmentation can help a computer extract, analyze, and understand useful information from a single or a sequence of images. There are many other advanced techniques including style transfer, colorization, action recognition, 3D objects, human pose estimation, and more. Indeed, the field of Computer Vision is too expensive to implement, train, and debug your own neural networks but not limited too.

Packages and Frameworks

OpenCV – OpenCV was designed for computational efficiency and with a strong focus on real-time applications. Adopted all around the world, OpenCV has more than 47 thousand people of user community and estimated number of downloads exceeding 14 million. Usage ranges from interactive art, to mines inspection, stitching maps on the web or through advanced robotics. OpenCV is the most popular free and open source solutions for computer vision. OpenCV algorithms range from being able to pixelate faces in images, to being able to smartly crop images automatically, to finding objects in images.

SimpleCV – SimpleCV is an open source framework for building computer vision applications. With it, you get access to several high-powered computer vision libraries such as OpenCV – without having to first learn about bit depths, file formats, color spaces, buffer management, eigenvalues, or matrix versus bitmap storage.

Mahotas – Mahotas is a computer vision and image processing library for Python. It includes many algorithms implemented in C++ for speed while operating in numpy arrays and with a very clean Python interface. Mahotas currently has over 100 functions for image processing and computer vision and it keeps growing.

Openface – OpenFace is a Python and Torch implementation of face recognition with deep neural networks and is based on the CVPR 2015 paper FaceNet: A Unified Embedding for Face Recognition and Clustering by Florian Schroff, Dmitry Kalenichenko, and James Philbin at Google. Torch allows the network to be executed on a CPU or with CUDA.

Ilastik – Ilastik is a simple, user-friendly tool for interactive image classification, segmentation and analysis. It is built as a modular software framework, which currently has workflows for automated (supervised) pixel- and object-level classification, automated and semi-automated object tracking, semi-automated segmentation and object counting without detection. Using it requires no experience in image processing.

1.9 WHY IS COMPUTER VISION DIFFICULT?

Computer Vision has a dual goal. From the biological science point of view, computer vision aims to come up with computational models of the human visual system. From the engineering

point of view, computer vision aims to build autonomous systems which could perform some of the tasks which the human visual system can perform (and even surpass it in many cases). Many vision tasks are related to the extraction of 3D and temporal information from time-varying 2D data such as obtained by one or more television cameras, and more generally the understanding of such dynamic scenes. Of course, the two goals are intimately related.

The properties and characteristics of the human visual system often give inspiration to engineers who are designing computer vision systems. Conversely, computer vision algorithms can offer insights into how the human visual system works. In this section, we shall adopt the engineering point of view. Computer Vision as a field of research is notoriously difficult. Almost no research problem has been satisfactorily solved. One main reason for this difficulty is that the human visual system is simply too good for many tasks (e.g., face recognition), so that computer vision methodologies suffer by comparison. A human can recognize faces under all kinds of variations in illumination, viewpoint, expression, etc. In most cases we have no difficulty in recognizing a friend in a photograph taken many years ago. Also, there appears to be no limit on how many faces we can store in our brains for future recognition. There appears no hope in building an autonomous system with such stellar performance. Two major related difficulties in computer vision can be identified:

- 📚 How do we distil and represent the vast amount of human knowledge in a computer in such a way that retrieval is easy?
- 📚 How do we carry out (in both hardware and software) the vast amount of computation that is often required in such a way that the task (such as face recognition) can be done in real time?

The most difficult things to understand are the ones that are fundamental. We live in a visual world. We see things and instantly understand what they are and what we can do with them. We cannot only identify them but also understand their particular attributes and the classification they belong in. Without thinking too deeply about it we can look at a book and understand the process of the idea, the thinking, the writing, the writer, the agent, the editor, the publishing house, the printer, the marketer, the salesman, the bookshop, the economy. And this is a far from exhaustive list of association that revolves around a book. Even more to the point we can each look at a book and understand what it is and what it does even if we have not all got the same nuanced understanding of what goes into its making and what happens around it.

Our depth of knowledge (or lack of it) will only play a role in specific contexts (like a book convention or a forum on the economy) but for most everyday purposes a number of us that is sufficiently large to be called the majority, will be able to ascribe most of the key attributes that

make the book into an entity. So vision, really, is a knowledge thing as opposed to an eye thing and this is where things get very interesting. Knowledge is based on what is accepted of the real world and that includes both factual and imaginary things. We can all agree, for instance, on whom Harry Potter is what he did and why he did it while we all also agree that he is an imaginary being. This means that in order to understand what we see we do not just use deductive reasoning whereby what we reach as a conclusion is 100% true, we also use inductive reasoning where we extrapolate from premises that are probably true and reach conclusions that are likely: “A book may be a great read if it is well written but in the cases where this is not the case it also makes a great doorstep.”

In that last sentence we imagine not only instances of success and failure but also a world where wit and sarcasm play a part in describing quality. Computers can be equipped with hardware that captures data (as in light in this case) in a far broader spectrum than our organic eyes, and they can also be equipped with algorithms that do such a marvelous job of interpreting this data that just by studying patterns of light they can learn to see around corners. Computers can also be equipped to perform deductive reasoning. So, now it sounds like we have the problem virtually solved. Yes and no. Yes in that search has become very good at understanding objects in images (and sometimes, video) and building an index which when coupled to its knowledge base for context can create a pretty good sense of search being intelligent. Two Google patents in particular point out just how this is done in a way that makes it appear seamless. Both of these highlight the issue that is the major stumbling block: namely context and, by association, inductive reasoning.

Because knowledge constantly evolves and morphs into analogies that contain meaning but make no sense (like the example above of using a badly written book as a doorstep) computer logic stumbles on inductive reasoning when it is unsupervised and this, also affects, computer vision, at least, where specific contexts are involved. So, it may be OK to have a self-driving vehicle where the on-board telemetry provides a system of vision that is superior to anything a human could bring to bear (think of a car that could ‘see’ through fog, know what’s coming around corners and can even tap into traffic sensors as well as traffic reports to optimize the journey) but a robot nanny is more problematic (because children are inherently unpredictable and in their unpredictability lie risks to their safety).

Computer vision is a field that deals with trying to get computers to understand digital images and videos. It’s very easy for us to see a picture and figure it out, but it turns out that trying to get a computer to figure that out is an extremely difficult task. To explain why this is, we will first see how an image is represented on a computer. Every pixel in an image contains a color. Every color can be represented by 3 numbers ranging from 0–255. We can write these numbers in the form (n1, n2, n3), which represent red, green, and blue values, respectively. For example,

(255,0,0) represents the color red, (0,255,0) represents green, and (0,0,255) represents blue. Using different values for these numbers, you can create whatever color you want. So, if every pixel contains a color that means every pixel is described by 3 numbers. So if we have an image that is 100x100 px, which means that, this image is represented by $100 \times 100 \times 3 = 30000$ numbers.

Imagine being given 30000 numbers ranging from 0–255 and being told to figure out whether these numbers represent an image of a dog or a cat. Now you have an idea about why this is such a difficult task. Over the past few years, we have actually made incredible advancements in solving this problem. More specifically, we have achieved very high success rates for image classification, which is the process of classifying an image into one of several pre-determined categories, through the use of convolutional neural networks (CNN's). Basically, CNN's are extremely large and complex mathematical models that we 'train' by feeding those thousands of labelled images, and allowing the computer to learn the labels. For example, if we wanted to classify images as either images of dogs or cats, we would train a CNN by feeding it tons of images of dogs and cats, along with their respective labels. Eventually, with the proper CNN architecture and training techniques, the CNN should be able to classify images it has never seen before as either a dog or a cat, with fairly high accuracy rates.

However, there is still a lot of room for improvement with our current computer vision techniques. Image segmentation, for example, is something that is still an extremely difficult task. Image segmentation is the process of locating and segmenting out objects in an image. See the image below for an example of this. There are a number of different strategies to do this, but our current methods are nowhere near perfect. Currently, computer vision is a very large area of research. You may have already noticed the impact that our advancements in this field of research have had. For example, most smartphones nowadays have automatic face detection so that when you take photos of people, your phone can automatically figure out who is in the picture. And for all you Android users, you can now search through your photos using keywords such as "mountains" or "beaches", since Google now analyzes all your photos with state-of-the-art computer vision techniques (as an iPhone user, I must say that I am quite jealous of and amazed at this feature).

The more progress we make in this field, the more changes we will see in our lives. Tesla's autonomous vehicles currently use incredibly advanced computer vision techniques in real-time to successfully navigate the streets using several different cameras on the car. While their current cars available to the public are not fully autonomous, once they perfect their algorithms (and after driverless car laws are passed), we will see completely autonomous cars all over the place. There are so many different industries that can and will benefit from computer vision. Thanks to this amazing area of research, we already have face detection and partially driverless cars, and soon we will see fully driverless cars, stores without checkout lines, and so much

more. There are so many amazing potential applications of computer vision, and what people are going to come up with.

In computer vision, we demand a higher standard than we would with a human. Mistakes made by machines undermine our trust in them because; unlike with humans we cannot usually see how they failed. As a result their failure becomes generalized and we perceive a machine (whether it is a search algorithm or a robot) to be fundamentally flawed. With humans we usually understand failure because we can model their performance on our own parameters of knowledge, memory and skill. Failures then become acceptable because we fully understand the limits of human capability. The development of artificial intelligence in both the specific and the general sense is closely tied to cracking the computer vision challenge. Vision, after all, truly is mostly mental.

Case Study 6:

What makes it difficult with computer vision, to see or to describe or to understand the given instance of an image?

Someone across the room throws you a ball and you catch it. Simple, right? Actually, this is one of the most complex processes we've ever attempted to comprehend, let alone recreate. Inventing a machine that sees like we do is a deceptively difficult task, not just because it's hard to make computers do it, but because we're not entirely sure how we do it in the first place. What actually happens is roughly this: the image of the ball passes through your eye and strikes your retina, which does some elementary analysis and sends it along to the brain, where the visual cortex more thoroughly analyzes the image. It then sends it out to the rest of the cortex, which compares it to everything it already knows, classifies the objects and dimensions, and finally decides on something to do: raise your hand and catch the ball (having predicted its path).

This takes place in a tiny fraction of a second, with almost no conscious effort, and almost never fails. So recreating human vision isn't just a hard problem, it's a set of them, each of which relies on the other. Well, no one ever said this would be easy. Except, perhaps, AI pioneer Marvin Minsky, who famously instructed a graduate student in 1966 to "connect a camera to a computer and have it described what it sees." Pity the kid: 50 years later, we're still working on it. Serious research began in the 50s and started along three distinctive lines: replicating the eye (**difficult**); replicating the visual cortex (**very difficult**); and replicating the rest of the brain (**arguably the most difficult problem ever attempted**).

To see: Reinventing the eye is the area where we've had the most success. Over the past few decades, we have created sensors and image processors that match and in some ways exceed

the human eye's capabilities. With larger, more optically perfect lenses and semiconductor subpixels fabricated at nanometer scales, the precision and sensitivity of modern cameras is nothing short of incredible. Cameras can also record thousands of images per second and detect distances with great precision. Yet despite the high fidelity of their outputs, these devices are in many ways no better than a pinhole camera from the 19th century: They merely record the distribution of photons coming in a given direction. The best camera sensor ever made couldn't recognize a ball; much less be able to catch it. The hardware, in other words, is severely limited without the software; which, it turns out, is by far the greater problem to solve. But modern camera technology does provide a rich and flexible platform on which to work.

To describe: This isn't the place for a course on visual neuroanatomical, but suffice it to say that our brains are built from the ground up with seeing in mind, so to speak. More of the brain is dedicated to vision than any other task, and that specialization goes all the way down to the cells themselves. Billions of them work together to extract patterns from the noisy, disorganized signal from the retina. Sets of neurons excite one another if there's contrast along a line at a certain angle, say, or rapid motion in a certain direction. Higher-level networks aggregate these patterns into meta-patterns: a circle, moving upwards. Another network chimes in: the circle is white, with red lines. Another: it is growing in size. A picture begins to emerge from these crude but complementary descriptions. Early research into computer vision, considering these networks as being unfathomably complex, took a different approach: "top-down" reasoning; a book looks like /this/, so watch for /this/ pattern, unless it's on its side, in which case it looks more like /this/.

A car looks like /this/ and moves like /this/. For a few objects in controlled situations, this worked well, but imagine trying to describe every object around you, from every angle, with variations for lighting and motion and a hundred other things. It became clear that to achieve even toddler-like levels of recognition would require impractically large sets of data. A "bottom-up" approach mimicking what is found in the brain is more promising. "We can scarcely think of a working definition of how our brains work, substantially less how to mimic it.

A computer can apply a series of transformations to an image and discover edges, the objects they imply, perspective and movement when presented with multiple pictures, and so on. The processes involve a great deal of math and statistics, but they amount to the computer trying to match the shapes it sees with shapes it has been trained to recognize; trained on other images, the way our brains were. What an image like the shown in the computer displaying that, by its calculations, the objects highlighted look and act like other examples of that object, to a certain level of statistical certainty. Proponents of bottom-up architecture might have said "I told you so." Except that until recent years, the creation and operation of artificial neural networks was

impractical because of the immense amount of computation they require. Advances in parallel computing have eroded those barriers, and the last few years have seen an explosion of research into and using systems that imitate; still very approximately, the ones in our brain. The process of pattern recognition has been sped up by orders of magnitude, and we're making more progress every day.

To understand: Of course, you could build a system that recognizes every variety of apple, from every angle, in any situation, at rest or in motion, with bites taken out of it, anything and it wouldn't be able to recognize an orange. For that matter, it couldn't even tell you what an apple is, whether it's edible, how big it is or what they're used for. The problem is that even good hardware and software aren't much use without an operating system. For us, that's the rest of our minds: short and long term memory, input from our other senses, attention and cognition, a billion lessons learned from a trillion interactions with the world, written with methods we barely understand to a network of interconnected neurons more complex than anything we've ever encountered. This is where the frontiers of computer science and more general artificial intelligence converge and where we're currently spinning our wheels.

Between computer scientists, engineers, psychologists, neuroscientists and philosophers, we can barely come up with a working definition of how our minds work, much less how to simulate it. That doesn't mean we're at a dead end. The future of computer vision is focused on concepts that are a bit harder to pin down: context, attention, intention. That said, computer vision even in its nascent stage is still incredibly useful. It's in our cameras, recognizing faces and smiles. It's in self-driving cars, reading traffic signs and watching for pedestrians. It's in factory robots, monitoring for problems and navigating around human co-workers. There's still a long way to go before they see like we do, if it's even possible but considering the scale of the task at hand, it's amazing that they see at all. "The future of computer vision is in incorporating the incredible yet explicit frameworks we've made with more extensive ones.

1.10 WHAT IS THIS AREA ABOUT?

Computer vision has been around for more than 50 years, but recently, we see a major resurgence of interest in how machines 'see' and how computer vision can be used to build products for consumers and businesses. Few examples of such applications are- Amazon Go, Google Lens, Autonomous Vehicles, and Face Recognition. The key driving factor behind all these is Computer Vision. In the simplest terms, Computer Vision is the discipline under a broad area of Artificial Intelligence which teaches machines to see. Its goal is to extract meaning from pixels. From the biological science point of view, its aims are to come up with computational models of the human visual system. From the engineering point of view, computer vision aims to build autonomous systems which could perform some of the tasks which the human visual system can perform (and even surpass it in many cases).

In the summer of the year 1966, Seymour Papert and Marvin Minsky at MIT Artificial Intelligence group started a project titled Summer Vision Project. The aim of the project was to build a system that can analyze a scene and identify objects in the scene. So the vast, puzzling area of computer vision that researchers and tech giants are still trying to decode was first thought to be simple enough for an undergraduate summer project by the very people who pioneered artificial intelligence. In the 70s, taking ideas from studies of the cerebellum, hippocampus and cortex for human perception, David Marr, a neuroscientist at MIT, set up the building blocks for the modern Computer Vision and thus is known as the father of the modern Computer Vision.

Obviously, with an interdisciplinary nature, this area involves fundamental research in image processing, computer graphics, machine learning, pattern recognition, biomechanics and even psychology. On top of it are several major application areas such as human-computer interaction, robotics, virtual environments, and multimedia. The common foundation for such applications includes computer vision, image processing and speech processing. Instead of taking some ad hoc approaches to audio and visual processing when the area was in its infantile stage, we are currently pursuing some intelligent ways by machine learning and pattern recognition, trying to achieve a kind of artificial intelligence.

Computer vision technology that allows computers to understand and label images is now used in convenience stores, driverless car testing, daily medical diagnostics, and in monitoring the health of crops and livestock. Computers are proficient at recognizing images. Today, top technology companies such as Amazon, Google, Microsoft, and Facebook are investing billions of dollars in computer vision research and product development. With this in mind, the top global tech firms are making use of computer vision and explore what kind of new technology and media could appear over the next few years. Deep learning is a subset of machine learning where artificial neural networks, algorithms inspired by the human brain, learn from large amounts of data. Powering recommender systems, identify and tags friends in photos, translate your voice to text, translate text into different languages; Deep Learning has transformed Computer vision leading towards superior performance.

1.11 WHAT ARE THE APPLICATION AREAS?

We can imagine what a visually-capable and intelligent computer can do! We expect a revolution in next generation of computer: we do not use mice and keyboards anymore. Computers could understand our actions and our languages, they could think and feedback to us some kind of smart results in response to our commands, and they could even perform some missions on behalf of our human beings. Least but not last, we expect a rapid progress in the near future in such areas as intelligent human-computer interaction, robotics, virtual

environments, intelligent environments, and multimedia. Many of the use cases of computer vision descent into the following clusters:

1.11.1 Retail and Retail Security

The research of human-computer interaction is no longer the design of devices and psychological experiments of windows layouts, but evolutes to a new stage: intelligent interaction. One aspect is that computers should be able to accept audio and visual sensory inputs, and then make some kind of analysis and interpretation, and then make available intuitive feedbacks by synthesizing speech, video or actions. Basically, besides speech recognition, computers should be able to recognize, interpret and understand human actions and behaviors from visual inputs.

Amazon: Amazon recently opened to the public the Amazon Go store where shoppers need not wait in line at the checkout counter to pay for their purchases. Located in Seattle, Washington, the Go store is fitted with cameras specialized in computer vision. Initially allowed Amazon employee shoppers, but welcomed the public beginning in early 2018. The technology that runs behind the Go store is called Just Walk Out. Shoppers activate the IOS or Android mobile phone app before entering the gates of the store. Also, cameras are placed in the ceiling above the aisles and on shelves, Using computer vision technology, the company website claims that these cameras have the capability to determine when an object is taken from a shelf and who has taken it. If an item is returned to the shelf, the system is also able to remove that item from a customer's virtual basket.

The network of cameras allows the app to track people in the store at all times, ensuring it bills the right items to the right shopper when they walk out, without having to use facial recognition. As the name suggests, shoppers are free to walk out of the store once they have their products. The app will then send them an online receipt and charge the cost of the products to their Amazon account. While the store has eliminated cashiers, The New York Times reports that store employees are still available to check IDs in the store's alcohol section restock shelves and assist customers with finding products or aisles.

An Amazon rep also confirmed to recode that human employees work behind screens at the Go store to help train the algorithms and correct them if they wrongly detect that items are pulled off the shelf. Amazon has not revealed plans for the Go store in the longer term, but the company has registered Go trademarks in the UK. While Amazon in 2017 purchased Whole Foods, Gianna Puerini, Vice President at Amazon Go, said the company has no plans to implement the Just Walk Out Technology in the supermarket chain. In retail fashion, Amazon has applied for a patent for a virtual mirror. In the patent, the company said, "For entertainment purposes, unique visual displays can enhance the experiences of users." The virtual mirror

technology, sketched in the patent is described as a blended-reality display that puts a shopper's images into an augmented scene, and puts the individual in a virtual dress.

According to the patent, the virtual mirror will use enhanced facial detection, a subset of computer vision, whose algorithms will locate the eyes. Catching the user's eye position will let the system know what objects the user is seeing in the mirror. The algorithms will then use this data to control the projectors. Amazon has not made any announcement on this development and the virtual mirror has not been deployed, but the sketches released by the patent office show how a user could see illuminated objects reflected on the mirror combined with images transmitted from the display device to create a scene. For example, based on the patent descriptions, the transmitted image could show a scene, say a mountain trail, as software would put the shopper into the scene and potentially superimpose virtual clothes onto the reflection of his body. Face-tracking sensors and software would show a realistic image from all angles.

The shopper would be able to try on several outfits without needing to put them on. There is no available demo of Amazon's virtual mirror but a two minute video sample of how Kinect's on-the-market Windows virtual mirror works, showing a shopper "trying on" outfits, superimposing it on the image of her body in the mirror, following her movements, and even changing the color of items at her voice command: Amazon has previously released Echo Look, a voice-activated camera that takes pictures and six second videos of an individual's wardrobe and recommends combinations of outfits. The 2-minute review of the app shows how it claims to use Amazon's virtual assistant Alexa to help users compile images of clothes and can even recommend which outfit looks better on the individual. As the video shows, a user can speak to the gadget and instruct it to take full-body photos or a six second video. The content is collated to create an inventory of the user's wardrobe, according to Amazon.

Alexa compares two photos of the user in different outfits and recommends which looks better. According to Amazon, Echo Look is equipped with a depth-sensing camera and computer vision-based background blur that focuses on the image of the user. Part of the company's home automation product line, it is intended for consumers and priced at \$200. It is not clear if any retail businesses have used it.

Stop Lift: In retail security specific to groceries, Massachusetts-based Stop Lift claims to have developed a computer-vision system that could reduce theft and other losses at store chains. The company's product, called Scan It All, is a system that detects checkout errors or cashiers who avoid scanning, also called "sweet hearting." Sweet hearting is the cashier's act of fake scanning a product at the checkout in collusion with a customer who could be a friend, family or fellow employee. Scan It All's computer vision technology works with the grocery store's

existing ceiling-installed video cameras and point-of-sale (POS) systems. Through the camera, the software “watches” the cashier scan all products at the checkout counter. Any product that is not scanned at the POS is labeled as a “loss” by the software.

After being notified of the loss, the company says it is up to management to take the next step to accost the staff and take measures to prevent similar incidents from happening in the future. Using algorithms, Stop lift claims that Scan It All can identify sweet hearting behaviors such as covering the barcode, stacking items on top of one another, skipping the scanner and directly bagging the merchandise. The three minute video released shows how the Scan It All detects the many ways items are skipped at checkout, such as pass around, random weight abuse, cover-up, among others, and how grocery owners can potentially stop the behavior. In a Stop Lift case study of Piggly Wiggly, Stop Lift claims that incidences of sweet hearting at two of its grocery outlets have diminished since the computer vision system was deployed. From losses amounting to a total of \$10,000 monthly due to suspected failure to scan items at checkout, losses have dropped to \$1,000 monthly.

These losses are mostly identified as errors, rather than suspicious behavior, according to Stop Lift. Piggly Wiggly representatives said that cashiers and staff were notified that the system had been implemented, but it is unclear of whether that impacted the end result. Based on news stories, the company claims to have the technology installed in some supermarkets in Rhode Island, Massachusetts, and Australia, although no case studies have been formally released. Prior to this, the development of real-time facial recognition systems that identify terrorists in airports for Facia Reco Associates (licensor to facial recognition company Viisage), and delivered the first such system ever to the Army Research Laboratory.

1.11.2 Automotive

Intelligent environments, or smart environments, refer to some physical spaces that could automatically or intelligently react according to human activities. For example, when a person enters, the system could tell a people comes in and even identify who s/he is, and then turn on the lights. When the people sits on a sofa and points to a TV, the TV will be turned on. When s/he says “I want some news”, the TV will be switched to a channel that is broadcasting news at that moment. According to the World Health Organization, more than 1.25 million people die each year as a result of traffic incidents. The WHO adds that this trend is predicted to become the seventh leading cause of death by 2030 if no sustained action is taken. Nearly half of road casualties are “vulnerable road users:” pedestrians, cyclists and motorcyclists. According to this research, there’s a clear theme to the vast majority of these incidents: human error and inattention.

Waymo: One company that claims to make driving safer is Waymo. Formerly known as the Google self-driving car project, Waymo is working to improve transportation for people,

building on self-driving car and sensor technology developed in Google Labs. The company website reports that Waymo cars are equipped with sensors and software that can detect 360 degrees of movements of pedestrians, cyclists, vehicles, road work and other objects from up to three football fields away. The company also reports that the software has been tested on 7 million miles of public roads to train its vehicles to navigate safely through daily traffic. The 3 minute video released demonstrates how the Waymo car navigates through the streets autonomously. According to the video, it is able to follow traffic flow and regulations, and detects obstacles in its way. For example, when a cyclist extends his left hand, the software will detect the hand signal and predict if that cyclist will move to another lane. The software can also instruct the vehicle to slow down to let the cyclist to pass safely.

The company claims to use deep networks for prediction, planning, mapping and simulation to train the vehicles to maneuver through different situations such as construction sites, give way to emergency vehicles, make room to cars that are parking, and stop for crossing pedestrians. “Raindrops and snowflakes can create a lot of noise in sensor data for a self-driving car. Machine learning helps us filter out that noise and correctly identify pedestrians, vehicles and more”. There were reports that a Waymo van was involved in an accident in Arizona while on autopilot with the driver behind the wheel, but police said the software was not believed to be at fault. On its website, the company said it intends to apply the technology to a variety of transportation types from ride-hailing and logistics, to public transport and personal vehicles. According to Waymo press releases, the company has partnered with Chrysler and Jaguar on self-driving car technology.

Tesla: Another company that claims it has developed self-driving cars is Tesla, which claims that all its three Autopilot car models are equipped for full self-driving capability. Each vehicle, the website reports, is fitted with eight cameras for 360-degree visibility around the car with a viewing distance of 250 meters around. Twelve ultrasonic sensors enable the car to detect both hard and soft objects. The company claims that forward-facing radar enables the car to see through heavy rain, fog, dust and even the car ahead. Its camera system, called Tesla Vision, works with vision processing tools which the company claims are built on a deep neural network and able to deconstruct the environment to enable the car to navigate complex roads.

The three minute video released shows a driver with his hands off the wheel and feet off the pedals as they move through rush hour traffic in a Tesla Autopilot car. Last March 2018, a Tesla self-driving car was involved in a fatal accident while autopilot was engaged. The report said that the driver did not have his hands on the wheel despite the platforms warnings, telling him to do so. In the six seconds that hands were not on the wheel, his SUV hit a concrete divider, killing the driver. It was later discovered that neither the driver nor the car activated the brakes before the crash. Recent software improvements introduced to Tesla cars include

more warnings to drivers to keep their hands on the steering wheel. After three warnings, the software prevents the car from running until the driver restarts, according to the company.

1.11.3 Healthcare

Multimedia is a vague term. Different people have different emphasis. We are particularly interested in the analysis of the content of multimedia. An interesting question we ask is what is inside this picture or what this video means, which involves a relatively challenging assignment of image/video understanding. Many appealing applications have been proposed, but yet to be accomplished. When given just a photo of Sophie Marceau, without knowing her name, computers could search the Internet and get tones of her photos and movies. When you get tired of watching a long movie, computers could automatically summarize the movie in maybe five minutes. In healthcare industry, computer vision technology is helping healthcare professionals to precisely classify conditions or illnesses that may potentially save patients' lives by reducing or eliminating inaccurate diagnoses and incorrect treatment.

Gauss Surgical: Gauss Surgical has developed blood monitoring solutions that are described to estimate in real-time blood loss during medical situations. This solution, the website reports, maximizes transfusions and recognizes hemorrhage better than the human eye. Gauss Surgical's Triton line of blood monitoring solutions includes Triton OR which uses an iPad-based app to capture images of blood on surgical sponges and suction canisters. These images are processed by cloud-based computer vision and machine learning algorithms to estimate blood loss. The company says the application is currently used by medical professionals in hospital operating rooms during surgical operations or Caesarian deliveries. The six minute video released shows Triton how captures images of sponges or cloths that absorb blood during a medical procedure, works as a real-time scanner and estimates the potential of blood loss in the patient.

CEO of Gauss Surgical Siddharth Satish explains how the solution uses computer vision technology to predict the onset of hemorrhage. The Triton OR app underwent clinical studies in childbirth settings to validate accuracy and precision, and was cleared by the US Food and Drug Administration in 2017. Gauss Surgical in 2017 conducted an Accuracy of Blood Loss Measurement during Cesarean Delivery study at the Santa Clara Valley Medical Center. By the computer vision (colorimetric image analysis), the application determined amounts of blood on surgical sponges and in suction canisters collected from 50 patients during cesarean deliveries. When the application's estimates were compared with obstetricians' visual estimates, it was discovered that the application's estimates were more accurate, presenting the hospital with an opportunity to potentially improve clinical outcomes.

Deep Lens and DermLens: Amazon Web Services (AWS) also developed Deep Lens, a programmable deep learning-enabled camera that can be integrated with open source software

in any industry. In a video released, Deep Lens is described as a kit that programmers from various industries can use to develop their own computer-vision application. One healthcare application that uses the DeepLens camera is DermLens, which was developed by an independent startup. DermLens aims to assist patients to monitor and manage a skin condition called psoriasis. Created by digital health startup Predictably Well's Terje Norderhaug, the DermLens app is intended as a continuous care service where the reported data is available for the physician and care team. The four minute video released instructs developers on how to create and deploy an object-detection project using the DeepLens kit.

According to the video, developers will need to enter the Amazon Web Services (AWS) management console with their own username and password. From within the console, developers must choose a pre-populated object-detection project template, and then enter the project description and other values in appropriate fields. The console has a method in which the project will deploy to the developer's target device, and enable developers to view the output from their own screen. For DermLens, the short video explains how the application's algorithms were trained to recognize psoriasis, by feeding it with 45 images of skin that showed typically red and scaly segments. Each image in the set comes with a mask indicating the abnormal skin.

The computer vision device then sends data to the app, which in turn presents the user with an estimate of the severity of the psoriasis. The DermLens team also created a mobile app for self-reporting of additional symptoms such as itching and fatigue. In a case study published in the Journal of American Dermatology, DermLens claims to have been tested on 92 patients, 72 percent of whom said they preferred using the DermLens camera compared with using a smartphone. The study also revealed that 98 percent of patients surveyed said they would use the device to send images to the health care provider if the device was available. At the moment, a search of the company website and the web does not reveal any clients.

1.11.4 Agriculture

Robots have been giving quite good mechanical ability, but they are still machinery because they are neither able to see nor able to think. Honda has built a humanoid robot, ASIMO, which can walk like a human being. However, he is blind, dumb and dull. We expect to see that ASIMO moves by itself. Some farms are beginning to adopt computer vision technology to improve their operations. Recent research suggests that these technologies aim to help farmers adopt more efficient growth methods, increase yields, and sooner or later increase the profit. This section covers agricultural applications in great depth with a more general interest in that area.

Slantrange: Slantrange claims to offer computer vision-equipped drones that are connected to what the company calls an "intelligence system" consisting of sensors, processors, storage

devices, networks, artificial intelligence analytics software and other user interfaces to measure and monitor the condition of crops. At 120 meters above ground level, the camera has a resolution of 4.8 cm/pixel. However, its website notes that flying lower provides better resolution. The company claims that the drone captures images of the fields to show the different signatures of healthy crops compared with “stressed” crops. These stressors include pest infestations, nutrient deficiencies and dehydration; and metrics to estimate potential yield at harvest, and others. These signatures are passed on to the SlantView analytics system which interprets the data and ultimately helps farmers make decisions related to treatment for stress conditions.

The five minute video released provides a tutorial on how to use the basic functions of the SlantView app, starting with how a user can use the solution to identify stressed areas through drone-captured images. According to case study, Alex Petersen, owner of On Target Imaging, believed that changing their farming approach from analog to digital agriculture could bring more crops with less inputs and translating to more efficient farming. Using the Slantrange 3p sensor drone, the team flew and mapped the first 250 acres of corn fields to determine areas of crop stress from above. This enabled them to collect data about areas with high nitrate levels in the soil, which would negatively affect the sugar beet crops. The team then processed the data in SlantView Analytics, whose algorithms have been trained to determine if a plant is a crop or weed for accurate counts.

Slantrange claims that its drones only requires 17 minutes of flight time to cover 40 acres of field, equivalent to about 8 minutes of data processing. Alex reports that it took about 3 hours of combined flying and data processing to produce stress maps of the area covered. The resulting data enabled the team to decide using 15% to 20% less nitrogen fertilizer on 500 acres, which translated to savings of about 30-40 pounds per acre or \$9 to \$13 per acre. They also anticipated healthier corn and estimate an increase in crop yield of about 10 bushels per acre. The combined savings, higher yields and increased profits have allowed the team to recoup the investment of implementing precision farming.

Cainthus: Animal facial recognition is one feature that Dublin-based Cainthus claims to offer. Cainthus uses predictive imaging analysis to monitor the health and well-being of crops and livestock. Cainthus uses predictive imaging analysis to monitor the health and well-being of crops and livestock. The two minute video released shows how the software could use imaging technology to identify individual cows in seconds, based on hide patterns and facial recognition, and tracks key data such as food and water intake, heat detection and behavior patterns. These pieces of information are taken by the AI-powered algorithms and sends health alerts to farmers who make decisions about milk production, reproduction management and overall animal health.

Cainthus also claims to provide features like all-weather crop analysis in rates of growth, general plant health, stressor identification, fruit ripeness and crop maturity, among others. Cargill, a producer and distributor of agricultural products such as sugar, refined oil, cotton, chocolate and salt, recently partnered with Cainthus to bring facial recognition technology to dairy farms worldwide. The deal includes a minority equity investment from Cargill although terms were not disclosed. According to news reports in February 2018, Cargill and Cainthus are working on trials using pigs, and aim to release the application commercially by the end of the year. There are also plans to expand the application to poultry and aquaculture.

1.11.5 Banking

While most of our previous coverage of AI in banking has involved fraud detection and natural language processing, computer vision technology has also found its way into the banking industry as well.

Mitek Systems: Mitek Systems offers image recognition applications that use machine learning to classify, extract data, and authenticate documents such as passports, ID cards, driver's licenses, and checks. The applications work by having customers take a photo of an ID or a paper check using their mobile device and send to the user's bank where computer vision software on the bank's side verifies authenticity. Once verified and accepted by the user's bank, the application or check is processed. For deposits, funds typically become available to the customer within a business day, according to the Mitek company website. The two minute demo shows how the Mitek software works on mobile phones to capture the image of a check to be deposited to an account: To start the process, the user enters his mobile phone number into the bank's application form. A text message will be sent to his phone with a link the user can click to open an image-capture experience.

The customer can choose from a driver license, ID card or passport. Mitek's technology recognizes thousands of ID documents from around the world. Front and back images of the ID or document are required. Once the user has submitted the images, the applications will real-time feedback to ensure that high-quality images are captured. The company claims that its algorithms correct images; dewarp, deskew, distortion, and poor lighting conditions. Mobile Verify detected that this user started his session at his desktop, so once it's done processing; it will automatically deliver the data extracted to the user's channel of choice, desktop in this case.

A client, Mercantile Bank of Michigan, wanted to expand its retail bank portfolio and associated core deposits by providing retail customers with digital services. The bank chose Mitek's application called Mobile Deposit that allows customers to deposit checks using their camera smartphones. Mitek claims that it took just over 30 days to implement the application.

Within four months, 20 percent of the bank's online banking users started using mobile banking, according to the study. In the same period, the monthly enrollment into the bank's Consumer Deposit Capture program increased by 400 percent, due to the shift in mobile banking usage from the flat-bed scanner solution.

1.11.6 Industry

Osprey Informatics: In the industrial sector, computer vision applications such as Osprey Informatics are being used to monitor the status of critical infrastructure, such as remote wells, industrial facilities, work activity and site security. In its website, the company lists Shell and Chevron as among its clients. In a case study, a client claims that Osprey's online visual monitoring system for remote oil wells helped it reduce site visits and the equivalent cost. The client was seeking ways to make oil production more efficient in the face of depressed commodity prices. The study notes that it turned to Osprey to deploy virtual monitoring systems at several facilities for operations monitoring and security, and to identify new applications to improve productivity.

The Osprey Reach computer vision system was deployed to the client's high-priority well sites to provide 15-minute time-lapse images of specific areas of the well, with an option for on-demand images and a live video. Osprey also deployed at a remote tank battery, enabling operators to read tank levels and view the containment area. According to the case study, the client was able to reduce routine site visits by 50 percent since deploying the Osprey solution. The average cost for an in-person well site inspection was also reduced from \$20 to \$1. The 3-minute video below shows how the Osprey Reach solution claims to allow operators to remotely monitor oil wells, zooming in and out of images to ensure there are no leaks in the surrounding area. The on-demand video also shows the pump jack operating at a normal cadence. According to the company website, the list of industrial facilities that could potentially use computer vision includes oil and gas platforms, chemical factories, petroleum refineries, and even nuclear power plants.

Information gathered by sensors and cameras are passed on to AI software which alerts the maintenance department to take safety measures at even the slightest stress detected by the application. Computer vision applications have emerged in more industries, although some have adopted the technology faster than others. Whatever computer vision technology exists continues to rely on the human element, to monitor, analyze, interpret, control, decide and take action. In the automotive industry, global companies such as Google and Tesla are moving forward in improving self-driving cars equipped with computer vision cameras. However, with reported fatal accidents, it is clear that these cars are not completely ready to be commercially available and cannot be entirely autonomous.

In retail stores such as the Amazon Go store, human employees continue to work in customer service, and behind the screens to train the algorithms and confirm that the machine learning capability is on track. In terms of retail security, the technology assists to capture videos of the theft incidents, but human resources must step in to correct erring employees. The application of computer vision in agriculture has been slow to take off. But companies such as Cainthus have entered this market aiming to take the technology from other industries and apply it to agriculture. These applications claim to offer farmers the opportunity to conduct precision farming, to raise production at lower cost. The partnership of Cainthus and Cargill could potentially open other forms of artificial intelligence to the rest of the industry. Apart from the above major areas, a list of some high-level problems has seen success with computer vision but not limited as well;

- ✚ Optical character recognition
- ✚ Machine inspection
- ✚ 3D model building
- ✚ Medical imaging
- ✚ Match move (e.g. merging CGI with live actors in movies)
- ✚ Motion capture
- ✚ Surveillance
- ✚ Fingerprint recognition and biometrics
- ✚ Augmented reality
- ✚ Handwriting recognition
- ✚ Remote sensing
- ✚ Gesture recognition
- ✚ Automated Lip Reading
- ✚ Automatic image annotation
- ✚ Human image synthesis
- ✚ Digital video fingerprinting
- ✚ Visual sensor network
- ✚ Traffic enforcement camera
- ✚ Image stitching
- ✚ Document mosaicking
- ✚ Eye tracking
- ✚ Geometric feature learning

1.12 WHAT ARE THE RELATED FIELDS?

Within computer science, computer vision is tightly connected with numerous fields including: artificial intelligence, machine learning, machine vision, robotics, computer graphics, image

and signal processing. Machine learning is very important for computer vision. Machine learning is about learning algorithms, relations between the units in the training dataset and how to generalize the findings. But this area is separated from computer vision. Although computer vision relies heavily on machine learning approaches, the latter has many other applications. Computer graphics and computer vision intersect at several points. One such intersection would be that the computer graphics use already existing images to make their models more realistic, to be able to animate models of realistic images. People from computer vision now use these graphic models for algorithms training, to yield things like interfaces, to teach a machine how to create these images.

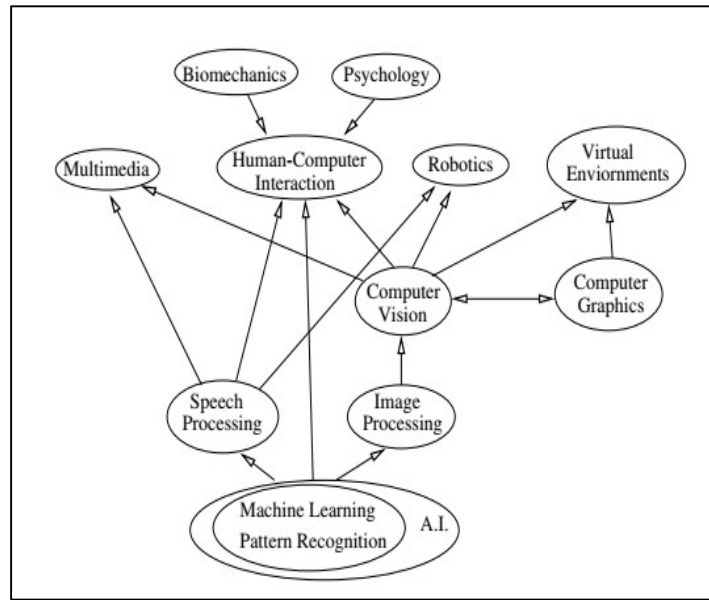


Fig. 1.17: Computer Vision and Related Areas

1.12.1 Artificial Intelligence

Areas of artificial intelligence deal with autonomous planning for robotical systems to navigate through an environment. A detailed understanding of these environments is required to navigate through them. Information about the environment could be provided by a computer vision system, acting as a vision sensor and providing high-level information about the environment and robots. Artificial intelligence and computer vision share other topics such as pattern recognition and learning techniques. Therefore, computer vision is sometimes seen as a part of the artificial intelligence field or the computer science field in general. As a scientific discipline, computer vision is concerned with the theory and technology for building artificial systems that obtain information from images or multi-dimensional data.

A significant part of artificial intelligence deals with planning or planning for system which can perform mechanical actions such as moving a robot through some environment. This type of processing typically needs input data provided by a computer vision system, acting as a vision sensor and providing high-level information about the environment and the robot. Other parts which sometimes are described as belonging to artificial intelligence and which are used in relation to computer vision is pattern recognition and learning techniques. Computer vision is a scholastic term that depicts the capability of a machine to get and analyze visual information all alone and afterward settles on decisions about it. That can consist of photographs and videos, yet more comprehensively may consist of “pictures” from thermal or infrared sensor, indicators and different sources.

Computer vision is now being used for various purposes, however, on the customer level; it is now depended upon by remote control drones to keep away from impediments, and via vehicles from Tesla and Volvo, among others. Computer vision permits computers, and in this manner robots, other computer-controlled vehicles, and the whole thing from processing plants and farm kit to semi-independent cars and drones, to run all the more productively and shrewdly and even securely. In any case, computer vision’s significance has turned out to be considerably increasingly evident in a world deluged with digital pictures. Since the advent of camera-prepared smartphones, we’ve been accruing amazing measures of visual symbolism that, without somebody or rather to process everything, is far less valuable and usable than it ought to be. We’re now witnessing computer vision enable purchasers to compose and get access to their photograph gallery without expecting to include labels in, say, Google Photos, yet how to remain over the billions of pictures shared online consistently which is around 3 billion.

To get a thought of the amount discussed here, a year ago photograph printing service Photo world did the math and discovered it would take an individual 10 whole years to try and take a look at all the photographs shared on Snapchat, in simply the most recent hour. What’s more, clearly, in those 10 years, an additional 880,000 years’ worth of photographs would have been as of now been produced if things proceed at a similar rate. Basically, our reality has turned out to be progressively loaded up with digital pictures and we require computers to understand everything. It’s as of now well past human capacities to keep up. The determined computer vision ventures we have seen in 2018 connote that the innovation is finally making up for lost time with the applications that developers have since a long time ago longed to make.

It additionally implies that it will further be less expensive to create customized computer vision applications. ModiFace gives clients a chance to attempt on cosmetics utilizing just their cell phones. Topology does likewise for eyewear. MTailor make specially custom fitted pants and shirts utilizing a similar procedure. Outside of fashion, Pottery Barn gives clients a chance to perceive what new furniture may look like in their homes, and Hover transforms clients’

photos of their homes into completely estimated 3D models. None of these tasks is as muddled as self-driving vehicles and cashier less supermarkets, yet that is the thing that qualifies the present era of computer vision products as a harbinger for monstrous deployment in the following couple of years: Once it ends up being feasible for little organizations to create working computer vision products for a mass audience, the innovation will start invading pretty much all aspects of our lives.

Microsoft as of late made an algorithm that inaccurately identified what was in pictures simply 3.5% of the time. That implies it was right 96.5% of the time. Luckily, a portion of the masters at Google brainstormed another option: Back in 2012, they bolstered a computer loads and heaps of pictures and let it make sense of patterns all alone and see what occurred, a procedure named deep learning. Turns out that, with adequate algorithms, computers can discover patterns individually and start to deal with pictures without expecting people to handhold en route. Today, some deep learning calculations are shockingly exact. There is no end of uses for computer vision. Think about any modern circumstance, and there's possible a computer vision-related solution that can or will sometime be deployed.

Take those extravagant Tesla cars we've heard so much about: They depend on a large group of cameras and in addition sonar, that not just keep your vehicle from floating out of a path, however, can perceive what different objects and vehicles are around you and furthermore pursue signs and traffic signals. Truth be told, Tesla's vehicles really look under the vehicle before you to the vehicle ahead to consider traffic patterns. So also, as dependent on innovation as the present medicinal services as of now seems to be, computer vision will empower better approaches for doing diagnostics that are nearer to Star Trek to analyze X-rays, MRI, CAT, mammography, and different outputs. All things considered, nearly 90% of every medical data is picture based.

The recent leaps forward that Amazon, Microsoft, and Google displayed in 2018 have been the impetus that will drive computer vision over the tipping point. Product developers and Artificial Intelligence (AI) engineers are now chipping away at new solutions that make use of computer vision and augmented reality. Hardware makers are enhancing component execution and expanding cost efficiencies to improve this innovation and make it more accessible. One of the greatest not so distant future advancements will be tied in with training data. Now, people still need to prepare computer vision AI with manually named pictures. If you've at any point filled out a web form that expected you to pick a couple of pictures from a grid appearing basic articles like a storefront or a vehicle, you've really taken an interest in making labeled data for computer vision ventures.

Be that as it may, as the innovation enhances, AI will figure out how to prepare AI, further streamlining the procedure and hastening the rate of enhancement. The market for computer

vision is developing nearly as fast as the capacities. It's anticipated to reach \$26.2 billion by 2025, developing more than 30% for every year. Artificial intelligence is the future, and computer vision is the most amazing appearance of that future. Before long, it will be anywhere and all over the place, to such an extent that you won't even notice it. Computer vision involves capturing, processing and analyzing real-world images and video to allow machines to extract meaningful, contextual information from the physical world. Today, computer vision is the foundation and a key means of testing and exploiting deep-learning models that are propelling the evolution of artificial intelligence toward ubiquitous, useful and practical applications. A lot of advancements are expected to occur in the decade to come. The primary contributions of artificial intelligence and computer vision are;

- 📌 **Image Classification:** Image classification is also referred to as object classification and perhaps more generally as image recognition, although this latter task may apply to a much broader set of tasks related to classifying the content of images.
- 📌 **Image Classification with Localization:** Image classification with localization involves assigning a class label to an image and showing the location of the object in the image by a bounding box, drawing a box around the object. This is a more challenging version of image classification.
- 📌 **Object Detection:** Object detection is the assignment of image classification with localization, although an image may contain multiple objects that require localization and classification. This is a more challenging task than simple image classification or image classification with localization, as often there are multiple objects in the image of different types. Often, techniques developed for image classification with localization are used and demonstrated for object detection.
- 📌 **Object Segmentation:** Object segmentation is the task of object detection where a line is drawn around each object detected in the image. Image segmentation is a more general problem of spitting an image into segments. Object detection is sometimes referred to as object segmentation. Unlike object detection that involves using a bounding box to identify objects, object segmentation identifies the specific pixels in the image that belong to the object. It is like a fine-grained localization. More generally, image segmentation might refer to segmenting all pixels in an image into different categories of object. Object segmentation is known as semantic segmentation.
- 📌 **Style Transfer:** Style transfer or neural style transfer is the task of learning style from one or more images and applying that style to a new image. This task can be thought of as a type of photo filter or transform that may not have an objective evaluation.

- 🖼️ **Image Colorization:** Image colorization or neural colorization involves converting a grayscale image to a full color image. This task can be thought of as a type of photo filter or transform that may not have an objective evaluation.
- 🖼️ **Image Reconstruction:** Image reconstruction and image inpainting is the task of filling in missing or corrupt parts of an image. This task can be thought of as a type of photo filter or transform that may not have an objective evaluation.
- 🖼️ **Image Super-Resolution:** Image super-resolution is the task of generating a new version of an image with a higher resolution and detail than the original image. Often models developed for image super-resolution can be used for image restoration and inpainting as they solve related problems. Datasets often involve consuming existing photo datasets and creating down-scaled versions of photos for which models must learn to create super-resolution versions.
- 🖼️ **Image Synthesis:** Image synthesis is the task of generating targeted modifications of existing images or entirely new images. This is a very broad area that is rapidly advancing.

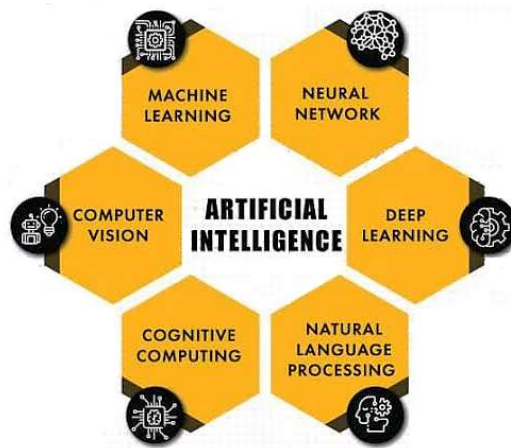


Fig. 1.18: Artificial Intelligence and Computer Vision

1.12.2 Machine Learning

In the present era of machines, three key interlocking factors has begun to come together since 2012, the concepts of “context, attention, intention” are deliberately evolving into computer vision, a new branch of Artificial Intelligence. Machine learning is the study of algorithms and statistical models, which is a subset of artificial intelligence. Systems use it to perform a task

without explicit instructions and instead rely on patterns and inference. Thus, it applies to computer vision, software engineering, and pattern recognition. Machine learning is done by computers with minimal assistance from software programmers. It uses data to make decisions and allows it to be used in interesting ways in a wide variety of industries.

It can be classified as supervised learning, semi-supervised learning, and unsupervised learning. Computer vision is the process of understanding digital images and videos using computers. It seeks to automate tasks that human vision can achieve. This involves methods of acquiring, processing, analyzing, and understanding digital images, and extraction of data from the real world to produce information. It also has sub-domains such as object recognition, video tracking, and motion estimation, thus having applications in medicine, navigation, and object modeling.

Supervised learning is a machine learning task that maps each input object to the desired output value. The computer is trained to associate an object with the desired output. It has a wide range of algorithms for different supervised learning problems. Applications in computer vision with machine learning grow exponentially over the years, wherein the society is the sole beneficiary. This endeavor is made possible by our so-called heroes in the technology sector, the developers and entrepreneurs working together enamored with the features of these technologies. Technology never ceases to mimic the human brain; thus AI gains a lot of interest for decades. To show the roadmap of these breakthroughs, let's discuss the relationship between AI, machine learning, and computer vision.

AI is the umbrella of these fields; machine learning is a subset of AI, wherein computer vision is also the subset of machine learning. However, computer vision can be considered as a direct subset of AI. Machine learning and computer vision are two fields that have become closely related to one another. Machine learning has improved computer vision about recognition and tracking. It offers effective methods for acquisition, image processing, and object focus which are used in computer vision. In turn, computer vision has broadened the scope of machine learning. It involves a digital image or video, a sensing device, an inferring device, and the interpretation stage. Machine learning is used in computer vision in the interpreting device and interpretation stage.

Relatively, machine learning is the broader field, and this is evident in the algorithms that can be applied to other fields. An example is the analysis of a digital recording, which is done with the use of machine learning principles. Computer vision, on the other hand, primarily deals with digital images and videos. Also, it has relationships in the fields of information engineering, physics, neurobiology, and signal processing. The obstacle faced by developers and entrepreneurs is the huge gap between computer vision and biological vision. The fields

most closely related to computer vision are image processing and image analysis. However, it deserves another interesting article to cite its relationship and differences. Also, the lack of knowledge about the main goal of machine learning in a particular project is a huge disruption among entrepreneurs.

Recognition in computer vision involves object recognition, identification, and detection. Some specialized tasks of recognition are optical character recognition, image retrieval, and facial recognition. Object recognition involves finding and identifying objects in a digital image or video. It is most commonly applied in face detection and recognition. Object recognition can be approached through the use of either machine learning or deep learning. Object recognition using machine learning requires the features to be defined first before being classified. A common approach that uses machine learning is the scale-invariant feature transform (SIFT). SIFT uses key points of objects and stores them in a database.

When categorizing an image, SIFT checks the key points of the image, which matches those found in the database. Object recognition using deep learning does not need specifically defined features. The common approaches that use deep learning are based on convolutional neural networks. A convolutional neural network is a type of deep neural network which is an artificial neural network with multiple layers between the input and output. An artificial neural network is a computing system inspired by the biological neural network in the brain. The best example of this is the ImageNet. It is a visual database designed for object recognition in which the performance is said to be almost similar to that of humans.

Motion Analysis in computer vision involves a digital video that is processed to produce information. Simple processing can detect the motion of an object. More complex processing tracks an object over time and can determine the direction of the motion. It has applications in motion capture, sports, and gait analysis. Motion capturing involves recording the movement of objects. Markers are worn near joints to identify motion. It has applications in animation, sports, computer vision, and gait analysis. Typically, only the movements of the actors are recorded and the visual appearance is not included. Gait analysis is the study of locomotion and the activity of muscles by means of instruments. It involves quantifying and interpreting the gait pattern. Several cameras linked to a computer are required. The subject wears markers at various reference points of the body. As the subject moves, the computer calculates the trajectory of each marker in three dimensions. It can be applied to sports biomechanics.

Machine learning is building solutions to real-life problems using computer vision. Here are some of the applications that work to assess the exciting aspects of machine learning:

 **Video tracking:** It is a process of locating a moving object over time. Object recognition is used to aid in video tracking. Video tracking can be used in sports. Sports

involve a lot of movement, and these technologies are ideal for tracking the movement of players.

-  **Autonomous vehicles:** Computer vision is used in autonomous vehicles such as a self-driving car. Cameras are placed on top of the car providing 360 degrees field of vision up to 250 meters of range. The cameras aid in lane finding, road curvature estimation, obstacle detection, traffic sign detection, and many more. Computer vision has to implement object detection and classification.
-  **Sports:** Computer vision is used in sports to improve the broadcast experience, athlete training, analysis and interpretation, and decision making. Sports biomechanics is a quantitative study and analysis of athletes and sports. For broadcast improvement, virtual markers can be drawn across the field or court. As for athlete training, creating a skeleton model of an acrobat and estimating the center of mass allows for improvement in form and posture. Finally, for sports analysis and interpretation, players are tracked in live games allowing for real-time information.
-  **Facial Authorization:** Banks are also using computer vision and machine learning to quickly authenticate documents such as IDs, checks, and passports. A customer can just take a photo of themselves or their ID using a mobile device to authorize transactions, but liveness detection and anti-spoofing can be acquired through machine learning and then detected by computer vision featuring facial authorization for banking.
-  **Liveness Detection:** Liveness detection foils a large number of attack vectors with a very high degree of accuracy to ensure that the person being photographed for recognition is a live human.
-  **Geometric Correction:** Detecting and rectifying document IDs like driver's license, credit cards, passports. Imaging these documents present major challenges because of arbitrary camera orientation, glare, and bad lighting conditions.
-  **Real-Time CCTV Video Analysis:** Machine learning based video analysis/monitoring/security system dramatically decreased false alarm rate. The training system uses state of the art object detection and tracking algorithms and is trained using a large dataset.
-  **Robust Feature Extraction:** Several custom algorithms are being used in a medical device for accurately measuring the power of the corrective lens. Medical devices with a major imaging component have stringent algorithm requirements when it comes to

accuracy and robustness. These algorithms involved object detection, real-time calibration and robust feature extraction.

 **Spoof detection:** Spoof detection using machine learning accurately detects spoof attacks based on a person's picture. This significantly improved their recognition system that was based on extremely challenging examples under arbitrary lighting conditions.

Computer vision is used to acquire the data to achieve basketball analytics. These analytics are retrieved using video tracking and object recognition by tracking the movement of the players. Motion analysis methods are also used to assist in motion tracking. Deep learning using convolutional neural networks is used to analyze the data. Despite the clamor of AI, machine learning, and computer vision, it is clear, albeit accurate, that the computer vision is still behind the human biological vision. This is the reality faced by both entrepreneurs and developers. Aside from the fact that engaging in this kind of venture introduced tantamount of expenses, the limitations of general learning algorithms, and resource scarcity. Like machine learning, computer vision is seen as a key factor for automation in various fields like autonomous cars. Computer intelligence is a relatively new field at the intersection of computer science and mathematics.

The aim is to develop computer models that can, after being fed with training data, generalize and realize patterns and relations, making computers to learn independently. Visual computing on the other hand aims at the automated perception and understanding of visual data. So is state-of-the-art computer vision only possible with machine learning technology behind it? Not really. The task of gathering and processing optical input in such a way that it might assist or even replace human vision is complex. So computer vision methods nowadays leverage intelligent algorithms and systems. Even to an extent that visual computing has become one of the main fields of this technology's successful application. Computer vision, yet, is more than machine learning applied.

It involves tasks as 3D scene modeling, multi-view camera geometry, structure from motion, stereo correspondence, point cloud processing, motion estimation and more, where machine learning is not a key element. The world is about to undergo the biggest technological revolution in history with Artificial Intelligence, Machine Learning, Deep Learning, and Computer Vision. However, although there is a lot of talk about these four technologies, the terms are often used interchangeably without any attempt to clearly define their precise meaning. To best understand how these topics are related to each other, it's best to take an outside-in look at how they are connected.

 Artificial Intelligence is a subset of Computer Science where machines can appear to

be intelligent by running programs. It is a very broad topic and covers everything from the automatic doors at the shopping mall to the most intelligent systems built today. From this set of programs, we can refine the set even further.

- ✚ Machine Learning is the practice of giving a computer a set of rules and tasks, then letting it figure out a way to complete those tasks. The machine in essence starts out with no knowledge, and through trial and error comes up with a suitable solution. The work horse of Machine Learning is the Neural Network.
- ✚ Neural Networks are algorithms and data structures designed to let machines classify and predict outputs based on a series of inputs. The neural network is an analogous structure to the brain. It consists of Nodes (brain cells), connections, and weights and works on the principle of Gradient Descent. The network has two modes of operation: Training and Inference. In Training mode, lots of data sets are fed into the input nodes and the weights are adjusted. In Inference mode, the unknown data is fed into the input nodes and the system suggests an output. There is a lot more to understanding Neural Networks but this a very broad overview. Neural Networks are usually very complicated and take a lot of computing power to train.
- ✚ Deep Learning use Artificial Neural Networks within them. Deep Learning Networks and Neural Networks Architectures have a lot of things in common. They both have an input and output layer and Training and Inference modes. But there are some new twists usually implemented in Deep Learning Networks like Convolution and Max Pooling to make the algorithms run faster and allow for computation at great depths. In a nutshell, one can think of a Deep Learning Network as a network of Neural Networks.
- ✚ Computer Vision is the practice of giving machines knowledge of their physical surrounding world through sensors. In the past this was a very fragile and complicated task requiring a specific tailored algorithm to analyze pixels. These algorithms were not flexible and had to be used in a specific case and were very susceptible to rotation and lighting. Recent developments in the speed and number of hardware GPUs have allowed Computer Vision to take advantage of Deep Learning Networks that help to mitigate the issues with experienced with standard computer vision algorithms.

One of the goals of computer vision is for machines to “see” and process images in the same way, a crucial capability for artificial intelligence. But it’s what computer vision can then do with these images that’s really exciting. Computers can see things that we can’t, with multiple channels of insight. Simply put, machines have greater visual capabilities than we do. Given

their superhuman seeing power, the potential impact computer vision will have on our lives is profound.

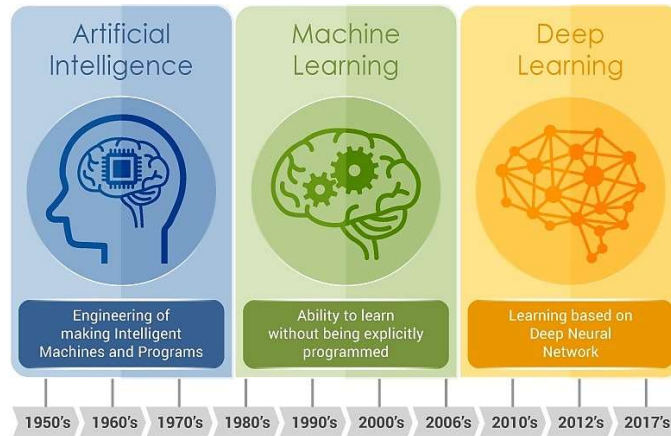


Fig. 1.19: Evolution of Artificial Intelligence

1.12.3 Natural Language Processing

Integration and interdisciplinarity are the cornerstones of modern science and industry. One of examples of recent attempts to combine everything is integration of computer vision and natural language processing (NLP). Both these fields are one of the most actively developing machine learning research areas. Yet, until recently, they have been treated as separate areas without many ways to benefit from each other. It is now, with expansion of multimedia, researchers have started exploring the possibilities of applying both approaches to achieve one result. The most natural way for humans is to extract and analyze information from diverse sources. This conforms to the theory of semiotics, the study of the relations between signs and their meanings at different levels. Semiotic studies the relationship between signs and meaning, the formal relations between signs roughly equivalent to syntax and the way humans interpret signs depending on the context, pragmatics in linguistic theory. Considering purely visual signs, then this leads to the conclusion that semiotics can also be approached by computer vision, extracting interesting signs for natural language processing to realize the corresponding meanings.

Computer Vision tasks might be briefed in 3Rs: reconstruction, recognition and reorganization. Reconstruction refers to estimation of a 3D scene that gave rise to a particular visual image by incorporating information from multiple views, shading, texture, or direct depth sensors. The process results in a 3D model, such as point clouds or depth images. Recognition involves assigning labels to objects in the image. For 2D objects, examples of

recognition are handwriting or face recognition, and 3D tasks tackle such problems as object recognition from point clouds which assists in robotics manipulation. Reorganization means bottom-up vision when raw pixels are segmented into groups that represent the structure of an image. Low-level vision tasks include edge, contour, and corner detection, while high-level tasks involve semantic segmentation, which partly overlaps with recognition tasks. It is recognition that is most closely connected to language because it has the output that can be interpreted as words. For example, objects can be represented by nouns, activities by verbs, and object attributes by adjectives. In this sense, vision and language are connected by means of semantic representations.

NLP tasks are more diverse as compared to Computer Vision and range from syntax, including morphology and compositionality, semantics as a study of meaning, including relations between words, phrases, sentences and discourses, to pragmatics, a study of shades of meaning, at the level of natural communication. Some complex tasks in NLP include machine translation, dialog interface, information extraction, and summarization. It is believed that switching from images to words is the closest to machine translation. Still, such “translation” between low-level pixels or contours of an image and a high-level description in words or sentences, the task known as bridging the Semantic Gap, remains a wide gap to cross. The integration of vision and language was not going smoothly in a top-down deliberate manner, where researchers came up with a set of principles. Integrated techniques were rather developed bottom-up, as some pioneers identified certain rather specific and narrow problems, attempted multiple solutions, and found a satisfactory outcome.

The new trajectory started with accepting that most present-day files are multimedia, that they contain interrelated images, videos, and natural language texts. For example, a typical news article contains a written by a journalist and a photo related to the news content. Furthermore, there may be a clip video that contains a reporter or a snapshot of the scene where the event in the news occurred. Language and visual data provide two sets of information that are combined into a single story, making the basis for fitting and unambiguous communication. This understanding gave rise to multiple applications of integrated approach to visual and textual content not only in working with multimedia files, but also in the fields of robotics, visual translations and distributional semantics. The multimedia-related tasks for NLP and computer vision fall into three main categories: visual properties description, visual description, and visual retrieval.

Visual properties description is a step beyond classification; the descriptive approach summarizes object properties by assigning attributes. Such attributes may be both binary values for easily recognizable properties or relative attributes describing a property with the help of a learning-to-rank framework. The key is that the attributes will provide a set of contexts as a

knowledge source for recognizing a specific object by its properties. The attribute words become an intermediate representation that helps bridge the semantic gap between the visual space and the label space. Visual description in the real life, the task of visual description is to provide image or video capturing. It is believed that sentences would provide a more informative description of an image than a bag of unordered words.

To generate a sentence that would describe an image, a certain amount of low-level visual information should be extracted that would provide the basic information “who did what to whom, and where and how they did it”. From the part-of-speech perspective, the quadruplets of “Nouns, Verbs, Scenes, and Prepositions” can represent meaning extracted from visual detectors. Visual modules extract objects that are either a subject or an object in the sentence. Then a Hidden Markov Model is used to decode the most probable sentence from a finite set of quadruplets along with some corpus-guided priors for verb and scene (preposition) predictions. The meaning is represented using objects (nouns), visual attributes (adjectives), and spatial relationships (prepositions). Then the sentence is generated with the help of the phrase fusion technique using web-scale n-grams for determining probabilities.

Content-based Image Retrieval (CBIR) is another field in multimedia that utilizes language in the form of query strings or concepts. As a rule, images are indexed by low-level vision features like color, shape, and texture. CBIR systems try to annotate an image region with a word, similarly to semantic segmentation, so the keyword tags are close to human interpretation. CBIR systems use keywords to describe an image for image retrieval but visual attributes describe an image for image understanding. Nevertheless, visual attributes provide a suitable middle layer for CBIR with an adaptation to the target domain.

Healthcare companies are using machine learning algorithms, computer vision and NLP in their healthcare technologies to understand everything from drug chemistry to genetic markers. They’re offering online consultations using predictive analytics, and they’re incorporating test results and sensor data to give real-time patient status updates to medical practitioners. A thirst field which plays an important role is neurobiology, specifically the study of the biological vision system. Over the last century, there has been an extensive study of eyes, neurons, and the brain structures devoted to processing of visual stimuli in both humans and various animals. This has led to a coarse, yet complicated, description of how "real" vision systems operate in order to solve certain vision related tasks. These results have led to a subfield within computer vision where artificial systems are designed to mimic the processing and behaviour of biological systems, at different levels of complexity.

Also, some of the learning-based methods developed within computer vision (*e.g.* neural net and deep learning based image and feature analysis and classification) have their

background in biology. Some strands of computer vision research are closely related to the study of biological vision – indeed, just as many strands of AI research are closely tied with research into human consciousness, and the use of stored knowledge to interpret, integrate and utilize visual information. The field of biological vision studies and models the physiological processes behind visual perception in humans and other animals. Computer vision, on the other hand, studies and describes the processes implemented in software and hardware behind artificial vision systems. Interdisciplinary exchange between biological and computer vision has proven fruitful for both fields.

The most exciting areas for AI in healthcare are around computer vision and natural language processing (NLP). AI is good at identifying patterns, making predictions, and analyzing complex situations. Healthcare is perhaps the ultimate combination of those three disciplines. Well implemented AI algorithms can literally save lives when they help a doctor notice something, point out a mistake, improve drug delivery, or help train medical experts. Natural language processing and computer vision are the cutting edge of AI with the greatest potential in healthcare. NLP helps computers interpret and respond to human language. Aside from visual observation, one of the key inputs a doctor relies on to make a diagnosis or narrow down promises is the patient's description of their symptoms, therefore Natural Language Processing in Healthcare can have major benefits. If NLP algorithms can help with initial screening questions, doctors can spend less time triaging and asking background information. Instead, they can get right to ordering tests and investigating specific concerns.

Healthcare also relies heavily on various types of images and scans for everything from diagnosis to new drug discovery; this is where Computer Vision in Healthcare comes into its own. Frequently, these images are grainy, hard to distinguish, or require recognizing very small, specific patterns. Computers can assist and often exceed human capabilities in these types of image analysis tasks. Using computer vision in healthcare, this artificial intelligence technology can help doctors and researchers get faster, more accurate results from tests, scans, and screenings.

Cancer Screening: Computer vision and NLP has shown major promise is in identifying cancerous cells and tumors from images and biopsy results. So far, the biggest breakthroughs have come in dermatology, where a computer can analyze an image of a person's skin much more quickly and thoroughly than a dermatologist doing an in-person exam. Recently, computer vision algorithms have proven themselves more effective at identifying potential skin cancer tumors than doctors. Similar advances have come in the field of breast cancer screenings. Computer vision can be applied to mammogram images to accurately identify tumors in the breast. Moreover, lung CT scan images processed through computer vision algorithms have shown promise at identifying lung cancer, as well. One of the presenters say

at ReWork, from DeepMind Health, shared some of the success they've had identifying head and neck cancer in collaboration with the Radiotherapy Department at University College London Hospitals. Initial testing shows DeepMind's algorithm can identify head and neck cancer with the same accuracy as a trained doctor in a fraction of the time.

Testing to Treatment: Another promising application of computer vision and natural language processing in healthcare is for remote diagnosis and faster test results. If patients can get seen and tested more quickly, preventative medicine is more effective in mitigating the consequences of disease. Babylon Health is one British startup working on the area of rapid diagnosis. They've developed an app and NLP algorithms to help a chatbot ask you the same questions a doctor would ask you at an in-person examination. The app does not provide an official diagnosis but uses speech and language processing to pull out symptoms and then forwards your profile information to a doctor. The doctor uses the processed information from the app to provide a fast diagnosis and can even chat with the patient via video call in the app. Another company, Medopad, has been working on similar issues but with a focus on providers. Their intelligent apps provide doctors with supplemental information during the diagnosis process.

Lately, both Babylon Health and Medopad have partnered with Chinese company Tencent to use and improve its machine learning algorithms alongside Tencent's other computer vision applications that can identify symptoms from user photos. The major promise of computer vision is triage, easily weeding out obvious non-symptomatic cases so that doctors can focus on reviewing images, and ultimately seeing patients, that are symptomatic. Even after a visit to the doctor, NLP can help patients understand their diagnosis and options for treatment and prevention of future problems. NLP algorithms can provide research-backed advice tailored to the patient's education level in much greater depth than a doctor ever could bedside.

Accelerated Research: Another highly-promising application of computer vision and NLP in healthcare is for research. Using anonymized scans of past patients, researchers, medical device manufacturers, and drug firms can identify trends and save time and money in the clinical trials phases of research. Computer vision promises to accelerate the identification of inclinations in patient images, making connections that would be time-consuming, if not impossible, for human researchers to discover on their own. Identifying patterns in injuries and disease progression is key to discovering solutions and learning how to prevent diseases in the first place. On this front, Benevolent AI is one company leading the charge into a new AI-powered world of medical research. Their natural language processing algorithms analyze the world's research papers and link common papers together for researchers with a reach and depth that wasn't feasible before AI. The next step is applying this linking philosophy to research images, drug molecules, and other visual models to accelerate and contextualize healthcare research even further.

Surgical Simulation: At the intersection of computer vision and augmented reality is surgical simulation and surgical assistance technology. These rapidly developing fields help surgeons train for and make decisions during complicated surgeries, including laparoscopic surgeries where surgeons only have camera images to rely on. One company paving the way in this space is Touch Surgery. Their mobile app allows anyone in the world to learn and prepare for surgery based on cutting-edge best practices with more than 100 surgical simulations across fourteen specialties. Touch Surgery's app already has over 1.5 million users, and new hires and partnerships in computer vision and augmented reality will allow Touch Surgery's training to become even more immersive. As computer vision improves in its recognition capacity, surgeons might be able to use augmented reality in real-life surgeries. They could receive guidance, warnings, and updates in real time based on what the computer vision algorithm sees in the operating room.

Computer vision and natural language processing in healthcare clearly hold great potential for improving the quality and standard of healthcare around the world. Doctors rely on images, scans, in-person vision, the patient's responses, and medical research to make their diagnoses. With the help of computer vision and NLP, those diagnoses can come more quickly and comprehensively, leading to faster, higher quality healthcare for everyone. Unfortunately, while there are lots of breakthroughs and technological developments in healthcare, due to the way the industry works it is likely to be another decade before the majority of these applications of AI in healthcare are widespread. It is predicted that more widespread use of autonomous and human assisting robots will be around in the decade to come in healthcare industry using computer vision and NLP. Since the integration of vision and language is a fundamentally cognitive problem, research in this field should take account of cognitive sciences that may provide insights into how humans process visual and textual content as a whole and create stories based on it.

Neural models have surpassed many traditional methods in both vision and language by learning better distributed representation from the data. For instance, Multimodal Deep Boltzmann Machines can model joint visual and textual features better than topic models. In addition, neural models can model some cognitively plausible phenomena such as attention and memory. For attention, an image can initially give an image embedding representation using CNNs and RNNs. An LSTM network can be placed on top and act like a state machine that simultaneously generates outputs, such as image captions or look at relevant regions of interest in an image one at a time. For memory, commonsense knowledge is integrated into visual question answering.

The idea lying behind Distributional Semantics Models is that words in similar contexts should have similar meaning; therefore, word meaning can be recovered from co-occurrence statistics

between words and contexts in which they appear. This approach is believed to be beneficial in computer vision and natural language processing as image embedding and word embedding. DSMs are applied to jointly model semantics based on both visual features like colors, shape or texture and textual features like words. The common pipeline is to blueprint visual data to words and apply distributional semantics models like LSA or topic models on top of them. Visual attributes can approximate the linguistic features for a distributional semantics model.

Future of Integrating NLP and Computer Vision: Integrating computer vision and natural language processing is a novel interdisciplinary field that has established a lot of attention. It marks as an analog of distributional semantics in computer vision and as image and word embedding in natural language processing, respectively. If combined, two tasks can solve a number of long-standing problems in multiple fields, including:

- ✚ **Designing:** In the sphere of designing of homes, clothes, jewelry or similar items, the customer can explain the requirements verbally or in written form and this description can be automatically converted to images for better visualization.
- ✚ **Description of medical images:** computer vision can be trained to identify subtler problems and see the image in more details comparing to human specialists.
- ✚ **Converting sign language to speech or text:** To help hearing-impaired people and ensure their better integration into society.
- ✚ **Spoken Description:** Making a system which sees the surrounding and gives a spoken description of the same can be used by blind people.
- ✚ **Spoken contents to images:** Making systems which can convert spoken content in form of some image which may assist to an extent to people which do not possess ability of speaking and hearing.

1.12.4 Image and Signal Processing

The terms computer vision and image processing are used interchangeably in many contexts. They both involve doing some computations on images. But are they really the same thing? What they are, how they are different, and how they are linked to each other. Image processing focuses on, well, processing images. What this means is that the input and the output are both images. An image processing algorithm can transform images in many ways: smoothing, sharpening, and changing the brightness and contrast, highlighting the edges, and so on. Computer vision, on the other hand, focuses on making sense of what a machine sees. A computer vision system inputs an image and outputs task-specific knowledge, such as object labels and coordinates. Computer vision and image processing work together in many cases. Many computer vision systems rely on image processing algorithms. For example, computer vision systems rarely use raw imaging data that comes directly from a sensor.

Instead, they use images that are processed by an image signal processor. The opposite is also possible. It wasn't common for an image processing algorithm to rely on computer vision systems in the past but more and more advanced image processing methods have started to use computer vision to enhance images. Face beautification filters, for example, use computer vision techniques to detect faces and apply smoothing filters such as a bilateral filter selectively. They can do more advanced stuff such as enhancing eye clarity or emulating a spotlight by detecting facial landmarks and tuning the images locally. I know the result you see on the right doesn't really look good. It looks very artificial. I did it on purpose though to show the difference better. Another key characteristic of computer vision is the use of machine learning. Machine learning is the field of study that focuses on teaching machines how to perform a certain task given a set of examples. For example, we can build a model that can tell the difference between a cat and a dog after being trained on pictures of cats and dogs. It's true that computer vision heavily relies on machine learning but that's no longer a differentiator.

Many advanced image processing methods also use machine learning models to transform images to accomplish a variety of tasks, such as applying artistic filters to an image, tuning an image for optimal perceptual image quality, or enhancing details to maximize the performance for computer vision tasks. It's worth mentioning that there isn't really a hard line between these two fields. The line between computer vision and image processing gets blurry when you do pixel to pixel transformation. Consider semantic segmentation as an example. If a model produces per-pixel labels for an input image, then its output can be considered as an image. In that sense, the model would be doing some sort of image processing. On the other hand, since such a transformation involves image understanding, trying to understand what's in the input, it would also be considered computer vision. Overall, semantic segmentation might be considered more of computer vision than image processing but you get the idea.

Another example of the interplay between image processing and computer vision would be the use of Convolutional Neural Networks or CNNs for short. CNNs typically take pixel intensity values as inputs and learn to process them in a way that makes it possible to accomplish a certain computer vision task, such as image recognition. The output of such a model can, for example, be a label that describes what's in the input image. Internal layers of CNNs can be considered as image filters with tunable parameters. Therefore, what a CNN does can be considered as some sort of adaptive image processing. The use of CNNs is not limited to image processing though. They can be used to process and analyze other types of data as well. CNNs do a great job at vision, audio, and even natural language processing applications. Researchers and engineers have built amazing applications using CNNs.

Yet another field related to computer vision is signal processing. Many methods for processing of one-variable signals, typically temporal signals, can be extended in a natural way to

processing of two-variable signals or multi-variable signals in computer vision. However, because of the specific nature of images there are many methods developed within computer vision which have no counterpart in processing of one-variable signals. Together with the multi-dimensionality of the signal, this defines a subfield in signal processing as a part of computer vision.

Computer vision, image processing, signal processing, machine learning – you’ve heard the terms but what’s the difference between them? Each of these fields is based on the input of an image or signal. They process the signal and then give us altered output in return. So what distinguishes these fields from each other? The boundaries between these domains may seem obvious since their names already imply their goals and methodologies. However, these fields draw heavily from the methodologies of one another, which can make the boundaries between them blurry.

In this section we’ll draw the distinction between the fields according to the type of input used, and more importantly, the methodologies and outputs that characterize each one. Let’s start by defining the input used in each field. Many, if not all, inputs can be thought of as a type of a signal. We favor the engineering definition of a signal, that is, a signal is a sequence of discrete measurable observations obtained using a capturing device, be it a camera, radar, ultrasound, a microphone, et cetera. The dimensionality of the input signal gives us the first distinction between the fields. Mono-channel sound waves can be thought of as a one-dimensional signal of amplitude over time, whereas pictures are a two-dimensional signal, made up of rows and columns of pixels. Recording consecutive images over time produces video which can be thought of as a three-dimensional signal.

Input of one form can sometimes be transformed to another. For example, ultrasound images are recorded using the reflection of sound waves from the object observed, and then transformed to a visual modality. X-ray can be considered similarly to ultrasound; only that radioactive absorption is transformed into an image. Magnetic Resonance Imaging (MRI) records the excitation of ions and transforms it into a visual image. In this sense, signal processing might be understood actually as image processing. Let’s look at the x-ray as a prototypical example. Let’s assume we have acquired a single image from an x-ray machine.

- 🧩 Image processing engineers (or software) would often have to improve the quality of the image before it passes to the physician’s display. Hence, the input is an image and the output is an image. Image processing is, as its name implies, all about the processing of images. Both the input and the output are images. Methods often processed in image processing are: filtering, noise removal, edge detection, color processing and so forth. Software packages loyal to image processing are, for example, Photoshop and Gimp.

- 🌈 In computer vision, we anticipate to receive quantitative and qualitative information from visual data. Much like the process of visual reasoning of human vision; we can distinguish between objects, classify them, sort them according to their size, and so forth. Computer vision, like image processing, takes images as input. However, it returns another type of output, namely information on size, color, number, et cetera. Image processing methods are harnessed for achieving tasks of computer vision. Extending beyond a single image, in computer vision we try to extract information from video. For example, we may want to count the number of cars passing by a certain point in the street as recorded by a video camera. Or, we may want to measure the distance run by a soccer player during the game and extract other statistics. Therefore, temporal information plays a major role in computer vision, much like it is with our own way of understanding the world. But not all processes are understood to their fullest, which hinders our ability to construct a reliable and well-defined algorithm for our tasks.
- 🌈 Machine learning methods then come to our rescue. Methodologies like Support Vector Machine (SVM) and Neural Networks are intended at mimicking our way of reasoning without having full knowledge of how we do this. For example, a sonar machine placed to alert for intruders in oil drill facilities at sea needs to be able to detect a single diver in the vicinity of the facility. By sonar alone it is not possible to detect the difference between a big fish and a diver – more in-depth analysis is needed.

Characterizing the difference between the motions of the diver compared to a fish by sonar would be a good start. Features related to this motion, such as frequency, speed and so on are fed into the Support Vector Machine or neural network classifier. With training, the classifier learns to distinguish a diver from a fish. After the training set is completed, the classifier is intended to repeat the same observation as the human expert will make in a new situation. Thus, machine learning is quite a general framework in terms of input and output (Table 1.1). Like humans, it can receive any signal as an input and give almost any type of output.

Beside the above-mentioned views on computer vision, many of the related research topics can also be studied from a purely mathematical point of view. For example, many methods in computer vision are constructed on statistics, optimization or geometry. Finally, a significant part of the field is devoted to the implementation aspect of computer vision; how existing methods can be realized in various combinations of software and hardware, or how these methods can be modified in order to gain processing speed without losing too much performance. The fields most closely related to computer vision are image processing, image analysis and machine vision. There is a significant overlap in the range of techniques and applications that these cover. This implies that the basic techniques that are used and developed

in these fields are similar, something which can be interpreted as there is only one field with different names. On the other hand, it appears to be necessary for research groups, scientific journals, conferences and companies to present or market themselves as belonging specifically to one of these fields and, hence, various characterizations which distinguish each of the fields from the others have been presented.

Table 1.1. Understanding Image Processing, Signal Processing, Computer Vision, Machine Learning

Domain	Input	Output
Image processing	Image	Image
Signal processing	Signal	Signal, quantitative information e.g. Peak location
Computer vision	Image/video	Image, quantitative/qualitative information, e.g. size, color, shape, classification, etc.
Machine learning	Any feature signal, from e.g. image, video, sound, etc.	Signal, quantitative/qualitative information, image

Computer graphics produces image data from 3D models; computer vision often produces 3D models from image data. There is also a trend towards a combination of the two disciplines, *e.g.*, as explored in augmented reality. The following characterizations appear relevant but should not be taken as universally accepted:

- ✚ Image processing and Image Analysis have a tendency to focus on 2D images, how to transform one image to other, *e.g.*, by pixel-wise operations such as contrast enhancement, local operations like edge extraction or noise removal, or geometrical transformations such as rotating the image. This characterization implies that image processing/analysis neither necessitate assumptions nor produce interpretations about the image content.
- ✚ Computer Vision includes 3D analysis from 2D images. This analyses the 3D scene projected onto one or several images, *e.g.*, how to reconstruct structure or other information about the 3D scene from one or several images. Computer vision often relies on more or less complex assumptions about the scene represented in an image.
- ✚ Machine Vision is the process of applying a range of technologies to provide imaging-based automatic inspection, process control and robot guidance in industrial

applications. Machine vision tends to focus on applications, mainly in manufacturing, *e.g.*, vision based robots and systems for vision based inspection, measurement, or picking (such as Bin Picking). This implies that image sensor technologies and control theory often are integrated with the processing of image data to control a robot and that real-time processing is emphasized by means of efficient implementations in hardware and software. It also implies that the external conditions such as lighting can be and are often more controlled in machine vision than they are in general computer vision, which can enable the use of different algorithms.

- 🔗 Imaging primarily focus on the process of producing images, but sometimes also deals with processing and analysis of images. For example, medical imaging includes substantial work on the analysis of image data in medical applications.
- 🔗 Pattern Recognition is a field which uses various methods to extract information from signals, mainly established on statistical approaches and artificial neural networks. A significant part of this field is devoted to applying these methods to image data.

1.12.5 Machine Vision and Robot Vision

The idea of machines that are able to see and act instead of humans isn't a fresh one. Since the times of Herbert Wells people haven't stop dreaming of them. Being once the main theme of sci-fi stories, the idea turned into reality nowadays. The terms "computer vision" and "machine vision" have already become usual for both pros and amateurs in the AI field. However, there's one problem. They are not always used correctly. Do you personally see the difference between these two terms? If not, machine vision versus computer vision topic may arouse your deep interest. This section aims at throwing the light on machine vision versus computer vision issue. It will help you take a closer look at what these terms mean. What's more, we'll also focus on the practical use of both technologies and their benefits. Learning about machine vision versus computer vision is just impossible without learning the basics. Therefore, let's find out what the two terms mean in general.

'Computer vision' is a field that embraces methods used to acquire process, analyze, and understand images. It interprets high-dimensional data from the real world producing numerical or symbolic information. Ballard and Brown's textbook defines computer vision as the "construction of explicit, meaningful descriptions of physical objects from images". Whereas Trucco and Verri state that computer vision lies in "computing properties of the 3-D world from one or more digital images". Whatever definition you go for, the quintessence stays the same. Computer vision is all about extracting information about an object from a scene via computer analysis of its image or sequence of images. It employs optical character recognition, image recognition, video recognition, video tracking, and other algorithms to make the most of

the digital visual data. As usual, a computer vision system consists of the following components:

- ✚ An image capture device (mostly a camera with an image sensor and a lens);
- ✚ Lighting appropriate for the specific application;
- ✚ An image capture board (frame grabber or digitizer);
- ✚ Image processing software.

In fact, the computer vision system approximately resembles the human vision. An image apprehension device serves as human eyes while image processing software works like a human brain. As a result, we get the precious information simply irreplaceable for many business fields. Actually, the applications of computer vision are more than numerous. They include agriculture, geoscience, biometrics, augmented reality, medical image analysis, robotics, industrial quality inspection, security and surveillance to name just a few.

‘Machine Vision’ refers to the industrial usage of computer vision for automatic inspection, process control, and robot guidance. In other words, it’s the application of computer vision to factory automation. We turn to machine vision when we need to execute a certain function or outcome on the basis of the image analysis performed by the vision system. The software the machine vision system uses helps identify the pre-programmed features. Such a system triggers a variety of set actions according to the findings. The machine vision process begins with imaging. Afterward, the automated analysis of the image and extraction of the required information come. Finally, the solution is created. To put it simply, a machine vision system resembles a human inspector who visually controls the quality of the products on assembly lines. Its “eyes” (digital cameras) and its “brain” (image processing software) are able to perform similar inspections. As a result, it makes decisions on the basis of the digital images analysis. A machine vision system includes:

- ✚ An image capture device (a camera with an optical sensor);
- ✚ Lighting appropriate for the specific application;
- ✚ Camera interface card for a computer (frame grabber);
- ✚ Computer software for processing images;
- ✚ Digital signal hardware for reporting the results.

Traditionally, machine vision systems are programmed for performing narrowly defined tasks. For instance, they can count objects on the conveyor, search for defects, or read serial numbers. Unlike humans, they don’t possess the intelligence or learning capability. However, they are really helpful in many ways due to their high speed, accuracy, and ability to operate 24/7. Primarily, machine vision systems are used for image-based automatic inspection, sorting, and robot guidance. Perhaps, you might have already realized that computer vision and machine

vision differ not only in their names. Though they are two overlapping technologies and the boundaries between them are often blurred, they aren't the same thing.

To begin with, computer vision doesn't depend on machine vision. It can be used separately for a wide range of fields. On the contrary, machine vision can't exist without computer vision because it employs computer vision algorithms. It goes without saying that all of you have drawn a family tree at the primary school. If you decide to place computer and machine vision on such a tree, machine vision will be, probably, the child of computer vision. Next, computer vision is more a technique, whereas machine vision is more about specific industrial applications. In other words, computer vision is a scientific domain while machine vision is an engineering one. Finally, used in industrial settings, machine vision deals with light and motion that are controlled. Besides, the viewed objects are already known and the observed events are predictable. Computer vision often deals with the objects of the "outside world" and their activities which are uncontrolled and sometimes quite unpredictable.

Using Machine Vision: As mentioned, machine vision has found its major implementation in the manufacturing process. Here are just a few examples of the industrial machine vision everyday usage.

 **Guidance:** It's common knowledge that many manufacturing processes require high accuracy to within a millimeter. Thanks to machine vision, it's not a problem today. Machine vision software is capable of identifying and recognizing a variety of characteristics of the item. That's why it's irreplaceable for robotic guidance. Machine vision systems are able to locate the position and angle of a part and evaluate its accuracy concerning a specified tolerance, angle, etc. Such information is helpful for measuring and verifying assembly. Besides, they can report the location and orientation of a part in 2D or 3D space to the robot locating the part or machine controller aligning the part.

 **Gauging:** Though human vision is flexible, it's not capable of making fast, accurate, monotonous measurements. That's when machine vision is of great help. A machine vision system can calculate the distances between two or more points or geometrical locations on an object with pixel accuracy. It can accomplish thousands of measurements per second. What's more, it does a good job even with such tricky calculations as circularity. A machine vision system defines if the measurements meet hopes.

 **Defect Detection:** It's quite obvious that the reputation of the company fully depends on the quality of the products. That's why it's so important to detect defective items before they leave your factory. Machine vision systems can check products for defects

by automatically detecting and classifying them. Defects can be either cosmetic or affect the product functionality. Whatever the defect is, the system can react with a sound alarm or shut down the production.

 **Packaging Inspection:** It doesn't matter whether your products come out in paper boxes, cans, containers or glass bottles. In any case, you have to be sure that they are packed in an appropriate way. That's when machine vision systems are highly helpful. While dealing with bottles they can verify the fill level as well as detect open, cocked, and improperly torqued caps. When it comes to the box package, they can detect damaged or incorrectly closed boxes. What's more, they can check that all edges are straight and parallel and nothing protrudes from the box.

Using Computer Vision: Compared to machine vision, computer vision has found implementation in a broader range of fields. Truly, it is successfully employed in each and every sphere that employs various cameras, drones, remote sensing, etc. Here are a few of its numerous usages.

 **Scientific Research:** Nowadays thousands of laboratories globally employ computer vision systems as the core of their scientific experiments and studies. Chemical and physical labs get access to the processes not visible for a human naked eye. Scientists can effortlessly examine and track particles or droplets, for instance. This means that they have a unique opportunity of getting the exhaustive information they need. With computer vision, biologists are capable of studying animal behavior. It is done either at the lab, or in the wild. Medical specialists can benefit from computer vision as well while conducting tests or working on new medicines.

 **Protection and Security:** Perhaps, today it's rather difficult to find a public place without cameras. It's not surprising. The security issue is of vital importance for businesses. Moreover, it is useful for individuals who care of their personal safety. Computer vision technologies are widely used to protect both offices and homes from intruders. Computer vision systems enable to track moving objects and detect suspicious activities requiring immediate action. Whether you want to protect perimeter to prevent intrusions, identify abandoned objects or recognize suspicious activities. It's all possible with computer vision software. Do you want to identify your staff to prevent unauthorized access or authenticate clients to avoid fraud? It's also not a problem thanks to computer vision.

 **Natural Resources Management:** Undoubtedly, natural resources are crucial for the economic development of almost any country. It doesn't matter whether the economy is based on mining, agriculture or forestry. In any case, computer vision is a great

solution for remote inspection, monitoring, and fast decision making. With accurate geospatial data available due to computer vision software, many businesses can move to the new level. Survey drone technology is a reliable way to indicate the likely presence of oil, gas, and mineral natural resources. Moreover, it's helpful to determine how to better use land in forestry and agriculture. Furthermore, it's useful not only for planning harvests and mining but also for preserving ecosystems.

 **Marketing:** It's quite evident that taking right marketing decisions requires careful data collection and analysis. That's why a lot of retail businesses rely on computer vision technologies in improving their marketing strategy. In order to increase sales, retail stores examine the flow of their visitors. Firstly, they determine the busiest hours of their shops. Secondly, they define the dwell time their customers spend in certain departments. Thirdly, they study the emotions that every product evokes and find out those that are in demand. Finally, they identify VIPs in a timely manner and, of course, authenticate known shoplifters to circumvent thefts. Machine vision and computer vision are different AI technologies. However, the benefits they give are alike. They empower to reduce cost, save time and effort, and significantly increase the efficiency of any business.

Both computer vision and machine vision use image capture and analysis to perform tasks with speed and accuracy human eyes can't match. In the next section, we will have a glance about robotic vision and computer vision.

Robotic Vision: Artificial Intelligence has linked the gap between machines and humans by emphasizing some amazing aspects and some innovative expertise like Machine Learning, Computer Vision and Image Processing. It might never have occurred to you but your day-to-day life is very much influenced by Artificial Intelligence. For instance, if you're frequently visiting an e-commerce site, it automatically keeps a track of your search preferences. So the next time you visit that site, you'll start seeing results based on your search preferences. This technique of learning user behavior is called pattern recognition which is an integral part of Machine Learning and Artificial Intelligence.

Robots need to perceive their surrounding from more than one way of interaction. Similar to humans processing perceptual inputs by using their knowledge about things in the form of words, phrases, and sentences, robots also need to integrate their perceived picture with the language to obtain the relevant knowledge about objects, scenes, actions, or events in the real world, make sense of them and perform a corresponding action. For example, if an object is far away, a human operator may verbally request an action to reach a clearer viewpoint. Robotics Vision tasks relate to how a robot can perform sequences of actions on objects to manipulate

the real-world environment using hardware sensors like depth camera or motion camera and having a verbalized image of their surrounds to respond to verbal commands.

Robots use languages to describe the physical world and understand their environment. Moreover, spoken language and natural gestures are more convenient way of interacting with a robot for a human being, if at all robot is trained to understand this mode of interaction. From the human point of view this is more natural way of interaction. Therefore, a robot should be able to see and transform the information from its contextual perception into a language using semantic structures. The most well-known approach to represent meaning is Semantic Parsing, which transforms words into logic predicates. Semantic Parsing tries to map a natural language sentence to a corresponding meaning representation that can be a logical form like λ -calculus using Combinatorial Categorical Grammar (CCG) as rules to compositionally construct a parse tree. Robot Vision and Machine Vision are used interchangeably. However, there are a few subtle differences. Some machine vision applications, such as part inspection, have nothing to do with robotics; it is placed in front of a vision sensor for fault detection.

Robot Vision and Computer Vision: They are the two most astounding inventions in the field of AI but as it turns out, there's quite a lot of misconception about these two terms or entities. More often than usual, the two entities are perceived to be the same. But the thing is, there are numerous major differences between the two. While the two entities can never be the same, it is a common mistake that many people make owing to the striking similarities between Computer Vision and Robot Vision. In this section, you'll know how these two technologies are different from one another and where do we have to draw the line.

Robot Vision as the name suggests, is a set of algorithms which renders vision to the robotic components. At its core, Robot Vision is a blend of computer algorithms, cameras and other hardware components that work unanimously in order to provide visual insights to the robot or machine. This helps the robot to accomplish complex tasks that require visual understanding. For instance, if it wasn't for the Robot Vision techniques, a machine or say a robotic arm won't be able to pick an object and place it somewhere else if need be. You can imagine this scenario where sensors and cameras are used to detect an object placed on a wooden plank which is then lifted up by the robotic arm using complex Robot Vision algorithms. For object detection as discussed in above mentioned case, the Robot is equipped with the ordinary 2D cameras while this can be problematic if it were to mount its wheels on a moving vehicle. In that case, you'll need advanced 3D Stereo Cameras instead. So that's what Robot Vision is all about and seeing its use, one can easily conclude that a Robot would apparently be blind if it wasn't for Robot Vision.

Computer Vision is a subfield of Artificial Intelligence which aims on rendering advanced visual capabilities to the computers by means of complex algorithms and camera hardware. It

mainly deals with image recognition. The computer vision methodologies initially extract useful information from the digital images and videos. This information is then processed and analyzed. Finally with help of cameras (usually webcam), the computer acquires high-dimensional data from the real world. This data is further processed to generate symbolic information which enables the computer to take important decisions. There are several other subfields of Computer Vision including object recognition, video tracking, object pose estimation, image restoration and event detection.

Robot Vision and Machine Vision: While the above explanation contrasts between Computer Vision and Robot Vision, sometimes people still relate Robot Vision with Machine Vision. The two terms are apparently the same but there are certain points where these two contradict. Machine Vision mainly finds its use in industrial domain. Some notable applications of Machine Vision are automatic inspection, robot guidance and process control. But it must be taken into account that although Machine Vision methods are used to guide robots but it's still different from Robot Vision. First off, Machine Vision is comparatively a broader field and many of the Machine Vision applications have nothing to do with Robotics. On the contrary to this, Robot Vision is primarily meant to be implemented inside the robotic landscape alone and that's where we draw the line.

In a Nutshell: Robot Vision is closely related to Machine Vision. They both are in turn, closely related to Computer Vision. If we were talking about a family, Computer Vision could be seen as their "parent." However, to understand where they all fit into the world, we have to take a step higher to introduce the "grandparent", the Signal Processing (Figure 1.4). Signal Processing encompasses processing electronic signals to clean them up (e.g. removing noise), extract information, and prepare them to output to a display or prepare them for further processing. Anything can be a signal, more or less. There are various types of signals which can be processed, e.g. analog electrical signals, digital electronic signals, frequency signals etc. Images are basically just a two dimensional signal.

For Robot Vision, we're involved in the processing of images. Computer Vision and Image Processing are like cousins, but they have quite different aims. Image Processing techniques are primarily used to improve the quality of an image, convert it into another format (like a histogram) or otherwise change it for further processing. Computer Vision, on the other hand, is more about extracting information from images to make sense of them. So, you might use Image Processing to convert a color image to grayscale and then use Computer Vision to detect objects within that image. If we look even further, both of these domains are heavily influenced by the domain of Physics, specifically Optics. Where it starts to get a little more complex is when we include Pattern Recognition, or more broadly Machine Learning. This branch of the family is focused on recognizing patterns in data, which is quite important for many of the

more advanced functions required of Robot Vision. For example, to be able to recognize an object from its image, the software must be able to detect if the object it sees is similar to previous objects.

Machine Learning, therefore, is another parent of Computer Vision alongside Signal Processing. But, not all Computer Vision techniques require Machine Learning. You can also use Machine Learning on signals which are not images. In practice, the two domains are often combined like this: Computer Vision detects features and information from an image, which are then used as an input to the Machine Learning algorithms. For example, Computer Vision detects the size and color of parts on a conveyor belt, and then Machine Learning decides if those parts are faulty based on its learned knowledge about what a good part should look like. Now we get to Machine Vision and everything changes. This is because Machine Vision is quite different to all the previous terms. It is more about specific applications than it is about techniques. Machine Vision refers to the industrial use of vision for automatic inspection, process control and robot guidance.

The rest are scientific domains, whereas Machine Vision is an engineering domain. In some ways, you could think of it as a child of Computer Vision because it uses techniques and algorithms for Computer Vision and Image Processing. But, although it's used to guide robots, it's not exactly the same thing as Robot Vision. Finally, we arrive at Robot Vision. If you've been following the section up until now, you will realize that Robot Vision incorporates techniques from all of the previous terms. In many cases, Robot Vision and Machine Vision are used interchangeably. However, there are a few subtle differences. Some Machine Vision applications, such as part inspection, have nothing to do with robotics - the part is merely placed in front of a vision sensor which looks for faults.

Also, Robot Vision is not only an engineering domain. It is a science with its own specific areas of research. Unlike pure Computer Vision research, Robot Vision must incorporate aspects of robotics into its techniques and algorithms, such as kinematics, reference frame calibration and the robot's ability to physically affect the environment. Visual Servoing is a perfect example of a technique which can only be termed Robot Vision, not Computer Vision. It involves controlling the motion of a robot by using the feedback of the robot's position as detected by a vision sensor. Robot Vision involves using a combination of camera hardware and computer algorithms to allow robots to process visual data from the world. For example, your system could have a 2D camera which detects an object for the robot to pick up.

A more complex example might be to use a 3D stereo camera to guide a robot to mount wheels onto a moving vehicle. Without Robot Vision, your robot is essentially blind. This is not a problem for many robotic tasks, but for some applications Robot Vision is useful or even

essential. One useful way of understanding the differences comes from defining some of the domains in terms of their input. So, to finish up this chapter, here are the basic inputs for each of the domains introduced above.

Table 1.2. Computer Vision and Related Techniques in a Nutshell

Technique	Input	Output
Signal Processing	Electrical signals	Electrical signals
Image Processing	Images	Images
Computer Vision	Images	Information/features
Pattern Recognition/ Machine Learning	Information/ features	Information
Machine Vision	Images	Information
Robot Vision	Images	Physical Action
Signal Processing	Electrical signals	Electrical signals

CHAPTER 2

MACHINE LEARNING AND COMPUTER VISION

Machine Learning is undeniably one of the most influential and powerful technologies in today's world. More importantly, we are far from seeing its full potential. There's no doubt, it will continue to be making headlines for the foreseeable future. This chapter is designed as an introduction to the Machine Learning concepts, covering all the fundamental ideas within the perspective of computer vision. "Machine learning is a tool for turning information into knowledge. In the past 50 years, there has been an explosion of data. This mass of data is useless unless we analyze it and find the patterns hidden within. Machine learning techniques are used to automatically find the valuable underlying patterns within complex data that we would otherwise struggle to discover. The hidden patterns and knowledge about a problem can be used to predict future events and perform all kinds of complex decision making.

Most of us are unaware that we already interacting with Machine Learning every single day. Every time we Google something, listen to a song or even take a photo, Machine Learning is becoming part of the engine behind it, constantly learning and improving from every interaction. It's also behind world-changing advances like detecting cancer, creating new drugs and self-driving cars. The reason that Machine Learning is so exciting is because it is a step away from all our previous rule-based systems. Traditionally, software engineering combined human created rules with data to create answers to a problem. Instead, machine learning uses data and answers to discover the rules behind a problem. To learn the rules governing a phenomenon, machines have to go through a learning process, trying different rules and learning from how well they perform, thus known as Machine Learning. There are multiple forms of Machine Learning; supervised, unsupervised, semi-supervised and reinforcement learning. Each form of Machine Learning has differing approaches, but they all follow the same underlying process and theory. This explanation covers the general Machine Learning concept and then focuses in on each approach in the forthcoming sections.

Machine learning (ML) is a group of tool that allows software applications to become more accurate in predicting outcomes without being explicitly programmed. The basic premise of machine learning is to build algorithms that can receive input data and use statistical analysis to predict an output while updating outputs as new data becomes available. The processes involved in machine learning are analogous to that of data mining and predictive modeling. Both require searching through data to look for patterns and adjusting program actions accordingly. Many people are familiar with machine learning from shopping on the internet and being served ads related to their purchase.

This happens because recommendation engines use machine learning to personalize online ad delivery in almost real time. Beyond personalized marketing, other common machine learning use cases take account of fraud detection, spam filtering, network security threat detection, predictive maintenance and building news feeds. Many services that we use every day rely on machine learning, a field of science and a powerful technology that allows machines to learn from data and self-improve. Machine learning is used in internet search engines, email filters to sort out spam, websites to make personalized recommendations, banking software to detect unusual transactions, and mobile apps voice recognition.

The technology has many more potential applications, some with higher stakes than others. Future developments could support the UK economy and will have a significant impact upon society. For example, machine learning could provide us with readily available ‘personal assistants’ to help manage our lives, it could dramatically improve the transport system through the use of autonomous vehicles; and the healthcare system, by improving disease diagnoses or personalizing treatment. Machine learning could also be used for security applications, such as analyzing email communications or internet usage. The implications of these and other applications of the technology need to be considered now and action taken to ensure uses will be beneficial to society.

Machine learning is distinct from, but overlaps with, some aspects of robotics (robots are an example of the hardware that can use machine learning algorithms, for instance to make robots autonomous) and artificial intelligence (AI) (a concept that doesn’t have an agreed definition; however machine learning is a way of achieving a degree of AI). “There are both opportunities and challenges around this transformative technology and it raises social, legal, and ethical questions. Machine learning aims to stimulate, to increase and to demonstrate the potential of huge data around us; highlights the opportunities and challenges it presents.

Have you ever shopped online? So while checking for a product, did you notice when it recommends for a product similar to what you are looking for? or did you notice “the person bought this product also bought this” combination of products. How are they doing this recommendation? This is machine learning. Did you ever get a call from any bank or finance concern asking you to take a loan or an insurance policy? What do you think, do they call everyone? No, they call only a few selected customers who they think will purchase their product. How do they select? This is target marketing and can be applied using Clustering. This is machine learning. Machine Learning is a subset of artificial intelligence which focuses mainly on machine learning from their experience and making predictions based on its experience. It enables the computers or the machines to make data-driven decisions rather than being explicitly programmed for carrying out a certain task. These programs or algorithms are designed in a way that they learn and improve over time when are exposed to new data.

2.1 WHAT IS MACHINE LEARNING?

Machine learning is the science of getting computers to act without being explicitly programmed. In the past decade, machine learning has given us self-driving cars, practical speech recognition, effective web search, and a vastly improved understanding of the human genome. Machine learning is so pervasive today that you probably use it dozens of times a day without knowing it. Many researchers also think it is the best way to make progress towards human-level Artificial Intelligence. Machine learning is an application of artificial intelligence (AI) that provides systems the ability to automatically learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can access data and use it learn for themselves. The process of learning begins with observations or data, such as examples, direct experience, or instruction, in order to look for patterns in data and make better decisions in the future based on the examples that we provide.

The primary aim is to allow the computers learn automatically without human intervention or assistance and adjust actions accordingly. Machine learning enables analysis of massive quantities of data. While it generally delivers faster, more accurate results in order to identify profitable opportunities or dangerous risks, it may also require additional time and resources to train it properly. Combining machine learning with AI and cognitive technologies can make it even more effective in processing large volumes of information. “Machine Learning is the science of getting computers to learn and act like humans do, and improve their learning over time in autonomous fashion, by feeding those data and information in the form of observations and real-world interactions.

The above definition encapsulates the ideal objective or ultimate aim of machine learning, as expressed by many researchers in the field. The purpose of this section is to provide a business-minded reader with expert perspective on how machine learning is defined, and how it works. Machine learning and artificial intelligence share the same definition in the minds of many however; there are some distinct differences readers should recognize as well. References and related researcher views are given below this section for further understanding. As with any concept, machine learning may have a slightly diverse definition, depending on whom you ask. We combed to find five practical definitions from reputable sources:

-  Machine learning is the practice of using algorithms to parse data, learn from it, and then make a determination or prediction about something in the world. – Nvidia
-  Machine learning is the science of getting computers to act without being explicitly programmed. – Stanford
-  Machine learning is based on algorithms that can learn from data without relying on rules-based programming. – McKinsey & Co.

- 🚦 Machine learning algorithms can figure out how to perform important tasks by generalizing from examples. – University of Washington
- 🚦 The field of Machine Learning seeks to answer the questions, How can we build computer systems that automatically improve with experience, and what are the fundamental laws that govern all learning processes? – Carnegie Mellon University

While machine learning algorithms have been around for decades, they've attained new popularity as artificial intelligence (AI) has grown in prominence. Deep learning models in particular power today's most advanced AI applications.

Machine learning platforms are among enterprise technology's most competitive realms, with most major vendors, including Amazon, Google, Microsoft, IBM and others, racing to sign customers up for platform services that cover the spectrum of machine learning activities, including data collection, data preparation, model building, training and application deployment.

As machine learning continues to increase in importance to business operations and AI becomes ever more practical in enterprise settings, the machine learning platform wars will only intensify. Continued research into deep learning and AI is increasingly focused on developing more general applications. Today's AI models require extensive training in order to produce an algorithm that is highly optimized to perform one task. But some researchers are exploring ways to make models more flexible and able to apply context learned from one task to future, different tasks.

There are many different types of machine learning algorithms, with hundreds published each day, and they're typically grouped by either learning style (i.e. supervised learning, unsupervised learning, semi-supervised learning) or by similarity in form or function (i.e. classification, regression, decision tree, clustering, deep learning, etc.). Regardless of learning style or function, all combinations of machine learning algorithms consist of the following:

- 🚦 Representation (a set of classifiers or the language that a computer understands)
- 🚦 Evaluation (aka objective/scoring function)
- 🚦 Optimization (search method; often the highest-scoring classifier, for example; there are both off-the-shelf and custom optimization methods used)

The fundamental goal of machine learning algorithms is to simplify beyond the training samples i.e. successfully interpret data that it has never 'seen' before. Just as there are nearly limitless uses of machine learning, there is no shortage of machine learning algorithms. They range from the fairly simple to the highly complex, which will be prophesied in the coming sections.

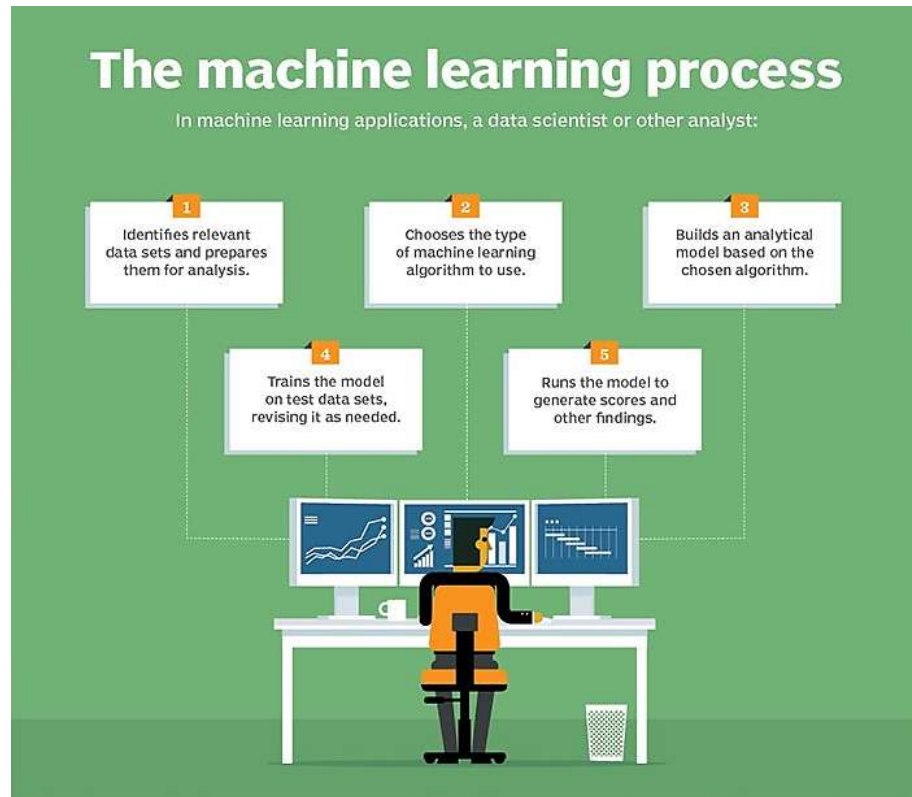


Fig. 2.1: The Machine Learning Process

2.1.1 How We Get Machines to Learn

There are different approaches to getting machines to learn, from using basic decision trees to clustering to layers of artificial neural networks (the latter of which has given way to deep learning), depending on what task you're trying to accomplish and the type and amount of data that you have available. This dynamic sees itself played out in applications as varying as medical diagnostics or self-driving cars. While emphasis is often placed on choosing the best learning algorithm, researchers have found that some of the most interesting questions arise out of none of the available machine learning algorithms performing to par. Most of the time this is a problem with training data, but this also occurs when working with machine learning in new domains. Research done when working on real applications often drives progress in the field, and reasons are twofold:

- ✚ Tendency to discover boundaries and limits of existing methods.
- ✚ Developers working with domain experts, leveraging time and expertise to improve system performance.

We might consider model ensembles, or combinations of many learning algorithms to improve accuracy, to be one example. Teams competing for the 2009 Netflix Prize found that they got their best results when combining their learners with other team's learners, resulting in an improved recommendation algorithm. One important point, in terms of application within business and elsewhere, is that machine learning is not just, or even about, automation, an often misunderstood concept. If you think this way, you're bound to miss the valuable insights that machines can provide and the resulting opportunities (rethinking an entire business model, for example, as has been in industries like manufacturing and agriculture).

Machines that learn are useful to humans because, with all of their processing power, they're able to more quickly highlight or find patterns in big (or other) data that would have otherwise been missed by human beings. Machine learning is a tool that can be used to enhance humans' abilities to solve problems and make informed inferences on a wide range of problems, from helping diagnose diseases to coming up with solutions for global climate change. Machine Learning algorithm is trained using a training data set to create a model. When new input data is introduced to the ML algorithm, it makes a prediction on the basis of the model. The prediction is evaluated for accuracy and if the accuracy is acceptable, the Machine Learning algorithm is deployed. If the accuracy is not acceptable, the Machine Learning algorithm is trained again and again with an augmented training data set. This is just a very high-level example as there are many factors and other steps involved.

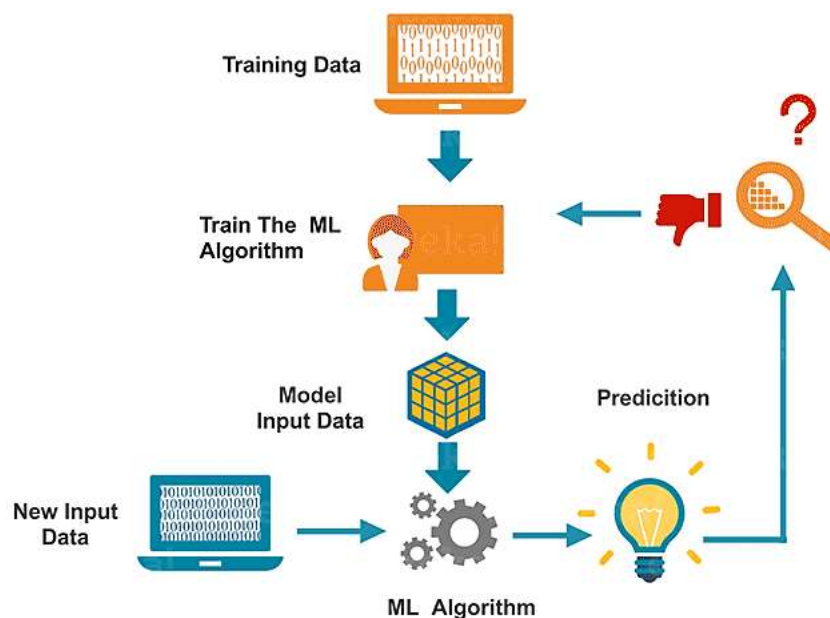


Fig. 2.2: How Machine Learning Works?

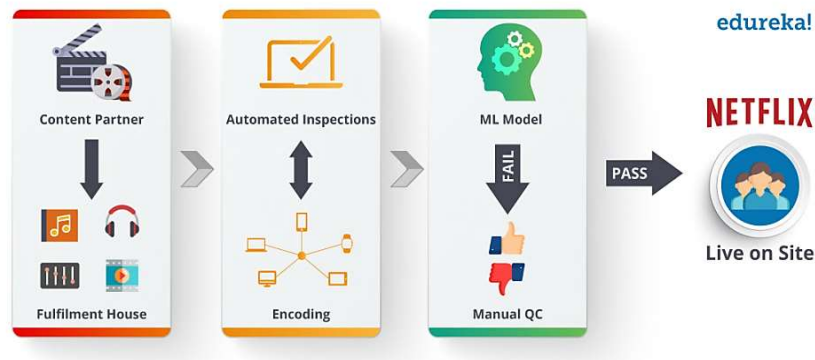


Fig. 2.3: Netflix Recommendation System using ML Model

2.1.2 Challenges and Limitations

As you know, we are living in the world of humans and machines. The Humans have been evolving and learning from their past experience since millions of years. On the other hand, the era of machines and robots have just begun. You can consider it in a way that currently we are living in the primitive age of machines, while the future of machine is enormous and is beyond our scope of imagination. In today's world, these machines or the robots have to be programmed before they start following your guidelines. But what if the machine started learning on their own from their practice, work like us, feel like us, do things more accurately than us? These things sound fascinating, Right? Well, just remember this is just the beginning of the new era.

Astounding technological advances in the field of Artificial Intelligence (AI) and its sub-field Machine Learning (ML) have been made in the last couple of years. Machines can now be trained to behave like humans enabling them to mimic complex cognitive functions like informed decision-making, deductive reasoning, and inferences. Robots behaving like humans are no longer science fiction, but a reality in multiple industry practices today. As a matter of fact, human society is gradually becoming more reliant on smart machines to solve day to day challenges and make decisions.

A good example of a simple use case for machine learning that has completely permeated our day-to-day lives is spam filters, which intrinsically determine whether a message is junk based on how closely it matches emails with a similar tag. However, these basic applications have evolved into 'deep learning' enabling software to complete complex tasks with significant implications for the way business is conducted. In all the hype surrounding these game-changing technologies, the reality that often times gets lost amidst both the fears and the headline victories like Cortana, Alexa, Google Duplex, Waymo, and AlphaGo, is that AI

technologies have several limitations that will still need a substantial amount of effort to overcome.

Algorithms Require Massive Stores of Training Data: AI systems are ‘trained’, not programmed. This means that they require enormous amounts of data to perform complex tasks at the level of humans. Despite the fact that data is being created at an accelerated pace and the robust computing power needed to efficiently process it is available; massive data sets are not simple to create or obtain for most business use cases. Deep learning utilizes an algorithm called back-propagation that adjusts the weights between nodes, to ensure an input translates to the right output. Supervised learning occurs when neural nets are trained to recognize photographs, for example, using millions or billions of previous labeled examples. And every slight variation in an assigned task calls for another large data set to conduct additional training. The major limitation is that neural networks simply require too much ‘brute force’ to function at a level similar to human intellect. This limitation can be overcome by coupling deep learning with ‘unsupervised’ learning techniques that don’t heavily rely on labeled training data.

For example, deep reinforcement learning models ideally learn via trial and error as opposed to via example. The model is optimized over multiple steps by penalizing unfavorable steps and incentivizing effective steps. Labeling Training Data Is a Tedious Process: Supervised machine learning using deep neural networks forms the basis for AI. Labeling is a requisite stage of data processing in supervised learning. This model training style utilizes predefined target attributes from historical data. Data labeling is simply the process of cleaning up raw data and organizing it for cognitive systems (machines) to ingest. Deep learning requires lots of labeled data, and while labeling is not rocket science, it is still a complex task to complete. If unlabeled data is fed into the AI, it is not going to get smart over time.

An algorithm can only develop the ability to make decisions, perceive, and behave in a way that is consistent with the environment within which it is required to navigate in the future if a human mapped target attributes for it. To establish what is in the data, a time-consuming process of manually spotting and labeling items is required. However, promising new techniques are coming up, like in-stream supervision, where data is labeled during natural usage. App designers can accomplish this by ‘sneaking in’ features in the design that inherently grow training data. High-quality data collection from users can be used to enhance machine learning over time.

Machines Cannot Explain Themselves: Researchers at MIT hypothesize that the human brain has an intuitive physics engine. This basically means that the information we are able to collect via our sense is noisy and imprecise; however, we make conclusions about what we think will likely happen. For decades, common sense has been the most difficult challenge in the field of

Artificial Intelligence. A large majority of AI-based models currently deployed is based on statistical machine learning that relies on tons of training data to build a statistical model. This is the main reason why adoption of some AI tools is still low in areas where explain ability is crucial. A good example is in regulations such as General Data Protection Regulation (GDPR), which requires a ‘right to explanation’. Whether the decision is good or bad, having visibility into how/why it was made is crucial, so that the human expectation can be brought in line with how the algorithm actually behaves. There are techniques that can be used to interpret complicated machine learning models like neural networks. A nascent approach is Local Interpretable Model-Agnostic Explanations (LIME), which attempts to pinpoint the parts of input data a trained ML model depends on most to create predictions, by feeding inputs similar to the initial ones and observing how these predictions vary.

There is Bias in the Data: As AI and machine learning algorithms are deployed, there will likely be more instances in which potential bias finds its way into algorithms and data sets. In some instances, models that are seemingly performing well maybe actually picking up noise in the data. As much as transparency is important, unbiased decision making builds trust. The infallibility of an AI solution is based on the quality of its inputs. For example, facial recognition has had a large impact on social media, human resources, law-enforcement and other applications. But biases in the data sets provided by facial recognition applications can lead to inexact outcomes. If the training data is not neutral the outcomes will inherently amplify the discrimination and bias that lies in the data set. The most ideal way to mitigate such risks is by collecting data from multiple random sources. A heterogeneous dataset limits the exposure to bias and results in higher quality ML solutions.

Algorithms Don’t Collaborate: Despite the multiple breakthroughs in ML, AI models still lack the ability to generalize conditions that vary from the ones they encountered in training. AI models have difficulty transferring their experiences from one set of circumstances to the other. This means that anything a model has achieved for a specific use case will only be applicable to that use case. As a result, organizations are forced to endlessly commit resources to train other models, even when the use cases are fairly similar. A solution to this scenario comes in the form of transfer learning. Knowledge obtained from one task can be used in situations where little labeled data is available. As this and other generalized methods mature, organizations will have the ability to build new applications more rapidly. The two biggest, historical (and ongoing) challenges of machine learning have involved are;

- 🧩 Overfitting (in which the model exhibits bias towards the training data and does not generalize to new data, and/or variance i.e. learns random things when trained on new data)
- 🧩 Dimensionality (algorithms with more features work in higher/ multiple dimensions, to understand the data is more difficult).

Having access to a large enough data set has in some cases also been a primary problem. One of the most common mistakes among machine learning beginners is testing training data successfully and having the illusion of success; Domingo (and others) emphasize the importance of keeping some of the data set separate when testing models, and only using that reserved data to test a chosen model, followed by learning on the whole data set.

When a learning algorithm (i.e. learner) is not working, often the quicker path to success is to feed the machine more data, the availability of which is by now well-known as a primary driver of progress in machine and deep learning algorithms in recent years; however, this can lead to issues with scalability, in which we have more data but time to learn that data remains an issue. In terms of purpose, machine learning is not an end or a solution in and of itself. Furthermore, attempting to use it as a blanket solution is not a useful exercise; instead, coming to the table with a problem or objective is often best driven by a more specific question.

“Machine learning can’t get something from nothing, what it does is get more from less.

It is undeniable that AI has opened up a wealth of promising opportunities, it has also led to the emergence of a mindset that can be best described as “AI solutionism”. This is the philosophy that, given enough data, machine learning algorithms can solve all of humanity’s problems. There are limitations that, at least for the time being, prevent that from being the case. A neural network can never tell us how to be a good person, and, at least for now. There are also fundamental limitations grounded in the underlying theory of machine learning, called computational learning theory, and the interpretability of results.

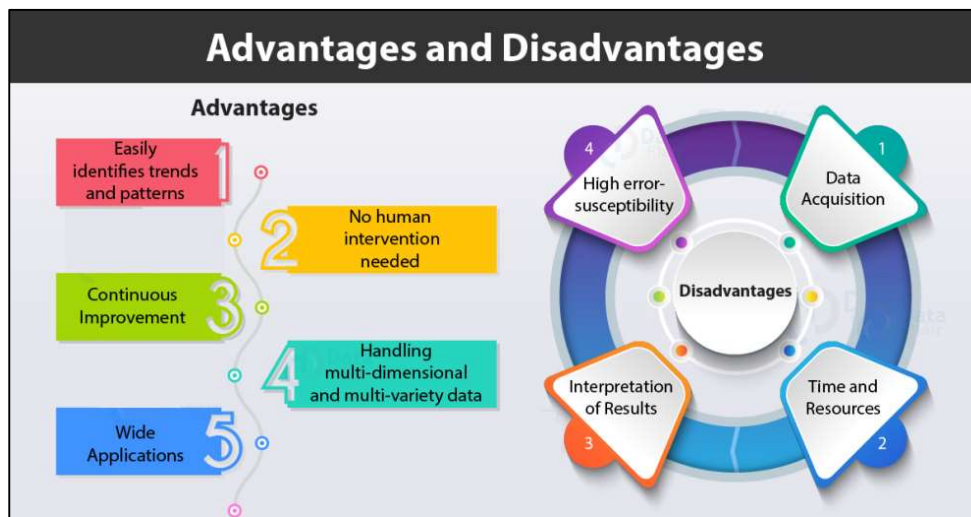


Fig. 2.4: Advantages and Disadvantages of ML

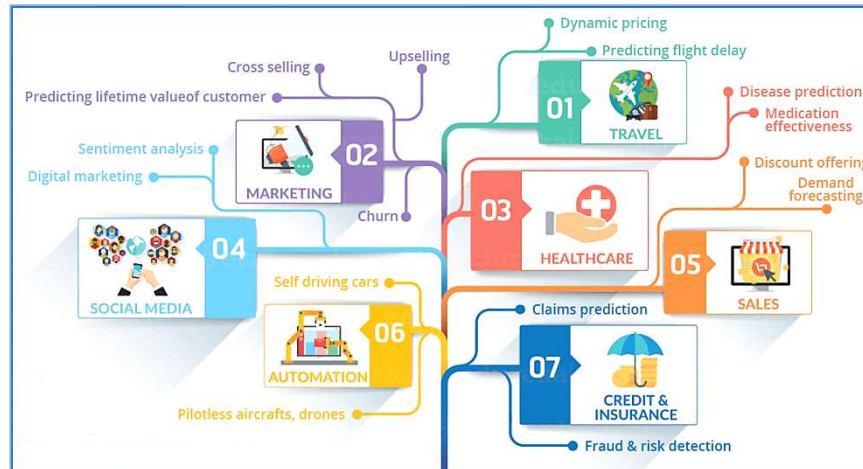


Fig. 2.5: Machine Learning in our Everyday Life

2.2 MACHINE LEARNING IN OUR EVERYDAY LIFE:

Machine learning is being used in a wide range of applications today. One of the most well-known examples is Facebook's News Feed. The News Feed uses machine learning to personalize each member's feed. If a member frequently stops scrolling to read or like a particular friend's posts, the News Feed will start to show more of that friend's activity earlier in the feed. Behind the scenes, the software is simply using statistical analysis and predictive analytics to identify patterns in the user's data and use those patterns to populate the News Feed. The members no longer stop to read, like or comment on the friend's posts, that new data will be included in the data set and thus, the News Feed will adjust accordingly.

Machine learning is also entering an array of enterprise applications. Customer relationship management (CRM) systems use learning models to analyze email and prompt sales team members to respond to the most important messages first. More advanced systems can even recommend potentially effective responses. Business intelligence (BI) and analytics vendors use machine learning in their software to help users automatically identify potentially important data points. Human resource (HR) systems use learning models to identify characteristics of effective employees and rely on this knowledge to find the best applicants for open positions.

Machine learning also plays an important role in self-driving cars. Deep learning neural networks are used to identify objects and determine optimal actions for safely steering a vehicle down the road. Virtual assistant technology is also powered through machine learning. Smart assistants combine several deep learning models to interpret natural speech, bring in relevant context like a user's personal schedule or previously defined preferences and take an action,

like booking a flight or pulling up driving directions. Below is few machine learning in our daily lives;

Creative Arts: Culinary arts require the human touch, right? Yes and no. AI-enabled Chef Watson from IBM offers a glimpse of how artificial intelligence can become a sous-chef in the kitchen to help develop recipes and advise their human counterparts on food combinations to create completely unique flavors. Working together, AI and humans can create more in the kitchen than working alone. Another way AI and big data can augment creativity is in the world of art and design. In one example, IBM's machine learning system, Watson, was fed hundreds of images of artist Gaudi's work along with other complementary material to help the machine learn possible influences for his work including Barcelona, its culture, biographies, historical articles and song lyrics. Watson analyzed all the information and delivered inspiration to the human artists who were charged with the creating a sculpture "informed" by Watson and in the style of Gaudi.

Music: Music-generating algorithms are now inspiring new songs. Given enough input, millions of conversations, newspaper headlines and speeches, insights are gleaned that can help create a theme for lyrics. There are machines such as Watson BEAT that can come up with different musical elements to inspire composers. AI helps musicians understand what their audiences want and to help determine more accurately what songs might ultimately be hits.

Energy: Global energy leader, BP is at the forefront of realizing the opportunities big data and artificial intelligence has for the energy industry. They use the technology to drive new levels of performance, improve the use of resources and safety and reliability of oil and gas production and refining. From sensors that relay the conditions at each site to using AI technology to improve operations, BP puts data at the fingertips of engineers, scientists and decision-makers to help drive high performance. In an attempt to deliver energy into the 21st century, GE Power uses big data, machine learning and Internet of Things (IoT) technology to build an "internet of energy." Advanced analytics and machine learning enable predictive maintenance and power, operations and business optimization to help GE Power work toward its vision of a "digital power plant."

Financial Services: With approximately 3.6 petabytes of data (and growing) about individuals around the world, credit reference agency Experian gets its extraordinary amount of data from marketing databases, transactional records and public information records. They are actively embedding machine learning into their products to allow for quicker and more effective decision-making. Over time, the machines can learn to distinguish what data points are important from those that aren't. Insight extracted from the machines will allow Experian to optimize its processes. American Express processes \$1 trillion in transaction and has 110

million AmEx cards in operation. They rely heavily on data analytics and machine learning algorithms to help detect fraud in near real time, therefore saving millions in losses. Additionally, AmEx is leveraging its data flows to develop apps that can connect a cardholder with products or services and special offers. They are also giving merchants online business trend analysis and industry peer benchmarking.

Healthcare: AI and deep learning is being put to use to save lives by Infervision. In China, where there aren't enough radiologists to keep up with the demand of reviewing 1.4 billion CT scans each year to look for early signs of lung cancer. Radiologists need to review hundreds of scans each day which is not only tedious, but human fatigue can lead to errors. Infervision trained and taught algorithms to augment the work of radiologists to allow them to diagnose cancer more accurately and efficiently. Neuroscience is the inspiration and foundation for Google's DeepMind, creating a machine that can mimic the thought processes of our own brains. While DeepMind has successfully beaten humans at games, what's really intriguing are the possibilities for healthcare applications such as reducing the time it takes to plan treatments and using machines to help diagnose ailments.

Manufacturing: Cars are increasingly connected and generate data that can be used in a number of ways. Volvo uses data to help predict when parts would fail or when vehicles need servicing, uphold its impressive safety record by monitoring vehicle performance during hazardous situations and to improve driver and passenger convenience. Volvo is also conducting its own research and development on autonomous vehicles. BMW has big data-related technology at the heart of its business model and data guides decisions throughout the business from design and engineering to sales and aftercare. The company is also a leader in driverless technology and plans for its cars to deliver Level 5 autonomy, the vehicle can drive itself without any human intervention by 2021. The AI tech revolution has hit farming as well, and John Deere is getting data-driven analytical tools and automation into the hands of farmers. They acquired Blue River Technology for its solution to use advanced machine learning algorithms to allow robots to make decisions based on visual data about whether or not a plant is a pest to treat it with a pesticide. The company already offers automated farm vehicles to plough and sow with pinpoint-accurate GPS systems and its Farmsight system is designed to help agricultural decision-making.

Media: The BBC project, 'Talking with Machines' is an audio drama that allows listeners to join in and have a two-way conversation via their smart speaker. Listeners get to be a part of the story as it prompts them to answer questions and insert their own lines into the story. Created specifically for smart speakers Amazon Echo and Google Home, the BBC expects to expand to other voice-activated devices in the future. UK news agency Press Association (PA) is hoping robots and artificial intelligence might be able to save local news. They partnered

with news automation specialist Urbs Media to have robots write 30,000 local news stories each month in a project called RADAR (Reporters and Data and Robots).

Fed with a variety of data from government, public services and local authorities, the machine uses natural language generation technology to write local news stories. These robots are filling a gap in news coverage that wasn't being filled by humans. Big data analytics is helping Netflix predict what its customers will enjoy watching. They are also increasingly a content creator, not just a distributor, and use data to drive what content it will invest in creating. Due to the confidence, they have in the data findings, they are willing to buck convention and commission multiple seasons of a new show rather than just a pilot episode.

Retail: When you first think of Burberry, you likely consider its luxury fashion and not first consider them a digital business. However, they have been busy reinventing themselves and use big data and AI to combat counterfeit products and improve sales and customer relationships. The company's strategy for increasing sales is to nurture deep, personal connections with its customers. As part of that, they have reward and loyalty programs that create data to help them personalize the shopping experience for each customer. In fact, they are making the shopping experience at their brick-and-mortar stores just as innovative as an online experience. As the world's second-largest retailer, Walmart is on the cutting edge of finding ways to transform retail and provide better service to its customers. They use big data, machine learning, AI and the IoT to ensure a seamless experience between the online customer experience and the in-store experience (with 11,000 brick-and-mortar stores, something rival Amazon isn't able to do. Enhancements include using the Scan and Go feature on the app, Pick-up Towers and they are experimenting with facial recognition technology to determine if customers are happy or sad.

Service: Central to everything Microsoft does is leveraging smart machines. Microsoft has Cortana, a virtual assistant; chatbots that run Skype and answer customer service queries or deliver info such as weather or travel updates and the company has rolled out intelligent features within its Office enterprise. Other companies can use the Microsoft AI Platform to create their own intelligent tools. In the future, Microsoft wants to see intelligent machines with generalized AI capabilities that allow them to complete any task. When you bring together cloud computing, geo-mapping and machine learning, some really interesting things can happen. Google is using AI and satellite data to prevent illegal fishing. On any given day, 22 million data points are created that show where ships are in the world's waterways.

Google engineers found that when they applied machine learning to the data, they could identify why a vessel was at sea. They ultimately created Global Fishing Watch that shows where fishing is happening and could then identify when fishing was happening illegally.

Always at the top of delivery extraordinary service, Disney is getting even better thanks to big data. Every visitor gets their own MagicBand wristband that serves as ID, hotel room key, tickets, FastPasses and payment system. While guest enough the convenience, Disney gets a lot of data that helps them anticipate guests' needs and deliver an amazing, personalized experience. They can resolve traffic jams, give extra services to guests who may have been inconvenienced by a closed attraction and data even allows the company to schedule staff more efficiently.

Google is one of the pioneers of deep learning from its initial foray with the Google Brain project in 2011. Google first used deep learning for image recognition and now is able to use it for image enhancement. Google has also applied deep learning to language processing and to provide better video recommendations on YouTube, because it studies viewers' habits and preferences when they stream content. Next up, Google's self-driving car division also leverages deep learning. Google also used machine learning to help it figure out the right configuration of hardware and coolers in their data centers to reduce the amount of energy expended to keep them operational. AI and machine learning has helped Google unlock new ways of sustainability.

Social Media: From what tweets to recommend to fighting inappropriate or racist content and enhancing the user experience, Twitter has begun to use artificial intelligence behind the scenes to enhance their product. They process lots of data through deep neural networks to learn over time what users' preferences are. Deep learning is helping Facebook draw value from a larger portion of its unstructured datasets created by almost 2 billion people updating their statuses 293,000 times per minute. Most of its deep learning technology is built on the Torch platform that focuses on deep learning technologies and neural networks. Instagram also uses big data and artificial intelligence to target advertising and fight cyberbullying and delete offensive comments. As the amount of content grows in the platform, artificial intelligence is critical to be able to show users of the platform information they might like, fight spam and enhance the user experience.

Virtual Personal Assistants: Siri, Alexa, Google Now are some of the popular examples of virtual personal assistants. As the name suggests, they assist in finding information, when asked over voice. All you need to do is activate them and ask "What is my schedule for today?", "What are the flights from Germany to London", or similar questions. For answering, your personal assistant looks out for the information, recalls your related queries, or send a command to other resources (like phone apps) to collect info. You can even instruct assistants for certain tasks like "Set an alarm for 6 AM next morning", "Recap me to visit Visa Office day after tomorrow".

Machine learning is an important part of these personal assistants as they collect and refine the information on the basis of your previous involvement with them. Later, this set of data is

utilized to render results that are tailored to your preferences. Virtual Assistants are integrated to a variety of platforms. For example:

-  **Smart Speakers:** Amazon Echo and Google Home
-  **Smartphones:** Samsung Bixby on Samsung S8
-  **Mobile Apps:** Google Allo

Predictions while Commuting:

-  **Traffic Predictions:** We all have been using the GPS navigation services. While we do that, our current locations and velocities are being saved at a central server for managing traffic. This data is then used to build a map of current traffic. While this helps in preventing the traffic and does congestion analysis, the underlying problem is that there is less number of cars that are well-found with the Global Positioning System (GPS).
-  **Online Transportation Networks:** When booking a cab, the app estimates the price of the ride. When sharing these services, how do they minimize the detours? The answer is machine learning. Jeff Schneider, the engineering lead at Uber ATC make known that they practice ML to define price surge hours by predicting the rider demand. In the entire cycle of the services, Machine Learning is playing a major role.

Videos Surveillance: Imagine a single person monitoring multiple video cameras! Certainly, a difficult job to do and boring as well. This is why the idea of training computers to do this job makes sense. The video surveillance systems nowadays are powered by AI that makes it possible to detect crime before they happen. They track unusual behavior of people like standing motionless for a long time, stumbling, or napping on benches etc. The system can thus give an alert to human attendants, which can ultimately help to avoid mishaps. And when such activities are reported and counted to be true, they help to improve the surveillance services. This happens with machine learning doing its job at the backend.

Social Media Services: From personalizing your news feed to better ads targeting, social media platforms are utilizing machine learning for their own and user benefits. Here are a few examples that you must be noticing, using, and loving in your social media accounts, without realizing that these wonderful features are nothing but the applications of ML.

-  **People You May Know:** Machine learning works on the concept understanding with experiences. Facebook continuously notices the friends that you connect with, the profiles that you visit very often, your interests, workplace, or a group that you share

with someone etc. On the basis of continuous learning, a list of Facebook users is suggested that you can become friends with.

 **Face Recognition:** You upload a picture of you with a friend and Facebook instantly recognizes that friend. Facebook checks the poses and projections in the picture, notice the unique features, and then match them with the people in your friend list. The entire process at the backend is complicated and takes care of the precision factor but seems to be a simple application of ML at the front end.

 **Similar Pins:** Machine learning is the core element of Computer Vision, which is a technique to extract useful information from images and videos. Pinterest uses computer vision to identify the objects (or pins) in the images and recommend similar pins accordingly.

Email Spam and Malware Filtering:

 There are a number of spam filtering approaches that email clients use. To ascertain that these spam filters are continuously updated, they are powered by machine learning. When rule-based spam filtering is done, it fails to track the latest tricks adopted by spammers. Multi-Layer Perceptron, C 4.5 Decision Tree Induction are some of the spam filtering techniques that are powered by ML.

 Over 325, 000 malwares are detected every day and each piece of code is 90–98% similar to its previous versions. The system security programs that are powered by machine learning understand the coding pattern. Thus, they detect new malware with 2–10% variation easily and offer protection against them.

Online Customer Support: A number of websites nowadays offer the option to chat with customer support representative while they are navigating within the site. However, not every website has a live executive to answer your queries. In most of the cases, you talk to a chatbot. These bots tend to extract information from the website and present it to the customers. Meanwhile, the chatbots advances with time. They tend to understand the user queries better and serve them with better answers, which is possible due to its machine learning algorithms.

Search Engine Result Refining: Google and other search engines use machine learning to improve the search results for you. Every time you execute a search, the algorithms at the backend keep a watch at how you respond to the results. If you open the top results and stay on the web page for long, the search engine assumes that the results it displayed were in accordance to the query. Similarly, if you reach the second or third page of the search results but do not open any of the results, the search engine estimates that the results served did not

match requirement. This way, the algorithms working at the backend improve the search results.

Product Recommendations: You shopped for a product online few days back and then you keep receiving emails for shopping suggestions. If not this, then you might have noticed that the shopping website or the app recommends you some items that somehow match with your taste. Certainly, this refines the shopping experience but did you know that it's machine learning doing the magic for you? On the basis of your behavior with the website/app, past purchases, items liked or added to cart, brand preferences etc., the product recommendations are made.

Online Fraud Detection: Machine learning is proving its potential to make cyberspace a secure place and tracking monetary frauds online is one of its examples. For example: Paypal is using ML for protection against money laundering. The company uses a set of tools that helps them to compare millions of transactions taking place and distinguish between legitimate or illegitimate transactions taking place between the buyers and sellers.

Intelligent Gaming: Some of you may remember 1997 when IBM's Deep Blue defeated Gary Kasparov in chess. But if you weren't old enough then, you might reminisce when another computer program, Google DeepMind's AlphaGo, defeated Lee Sedol, the Go world champion, in 2016.

Cyborg Technology: Obviously, our bodies and our brains have built in limitations and weaknesses. According to Oxford University, technology will improve to such an extent that we will be able to augment some of our weaknesses and limitations with computers, thereby enhancing many of our natural abilities. But wait - before you start picturing dystopian worlds of steel and flesh, consider for a moment that most people walking around are already "cyborgs" in a sense. How many people do you know who could survive the day without their trusty smartphone? We already rely on these handheld computers for communication, navigation, acquiring knowledge, receiving important news, and a host of other activities. Yoky Matsuoka of Nest also believes that AI will become useful for people with amputated limbs. One day, the brain will be able to communicate with a robotic limb. This technology will give amputees more control and reduce the daily limitations they deal with.

Taking Over Dangerous Jobs: One of the most dangerous jobs is bomb disposal. Today, robots (or more technically, drones) are taking over these risky jobs, among others. Right now, most of these drones require a human to control them. But as machine learning technology improves in the future, these tasks would be done completely by robots with AI. This technology alone has already saved thousands of lives. Another job being outsourced to robots is welding. This kind of work produces noise, intense heat, and toxic substances found in the

fumes. Without machine learning, these robot welders would need to be pre-programmed to weld in a certain location. However, advancements in computer vision and deep learning have enabled more flexibility and greater accuracy.

Environmental Protection: Machines can store and access more data than any one person could include mind-boggling statistics. Using big data, AI could one day identify trends and use that information to arrive at solutions to previously untenable problems. For example, IBM's Green Horizon Project analyzes environmental data from thousands of sensors and sources to produce accurate, evolving weather and pollution forecasts. It allows city planners to run "what-if" scenarios and conventional ways to mitigate environmental impact. And that's just beginning. Exciting environment-oriented innovations are entering the market every day, from self-adjusting smart thermostats to distributed energy grids.

Digital Empathy and Robots as Friends: Most robots are still emotionless. But a firm in Japan has made the first big steps toward a robot companion, one that can understand and feel emotions. Introduced in 2014, Pepper the companion robot went on sale in 2015, with all 1,000 initial units selling out within a minute. The robot was programmed to read human emotions, develop its own, and help its human friends stay happy. The day that one can literally "buy a friend" is not too far away. "We believe practical advancements in machine learning will start to enable a more contextual form of computing with some of our devices, particularly smartphones and smart speakers, part of the way this development will likely occur is by learning more about people and how they think, essentially building a form of digital empathy.

Improved Elder Care: For many seniors, everyday tasks can be a struggle. Many have to hire outside help or rely on family members. Elder care is a growing concern for many families. AI is at a stage where replacing this need isn't too far off, says Matthew Taylor, computer scientist at Washington State University. Elderly relatives who don't want to leave their homes could be assisted by in-home robots. That solution offers family members more flexibility in managing a loved one's care. These robots could help seniors with everyday tasks and allow them to stay independent and living in their homes for as long as possible, improving their overall well-being. Medical and AI researchers have even piloted systems based on infrared cameras that can detect when an elderly person falls. Researchers and medical specialists can also monitor alcohol and food consumption, fevers, restlessness, urinary frequency, chair and bed comfort, fluid intake, eating, sleeping, declining mobility, and more.

Enhanced Health Care: Hospitals may soon put your wellbeing in the hands of an AI, and that's good news. Hospitals that develop machine learning to aid in treating patients see fewer accidents and fewer cases of hospital-related illnesses, like sepsis. AI is also tackling some of medicine's most intractable problems, such as allowing researchers to better appreciate genetic

diseases through the use of predictive models. Previously, health professionals must review reams of data manually before they diagnose or treat a patient. Today, high-performance computing GPUs have become key tools for deep learning and AI platforms. Deep learning models quickly provide real-time insights and, combined with the explosion of computing power, are helping healthcare professionals diagnose patients faster and more accurately, develop innovative new drugs and treatments, reduce medical and diagnostic errors, predict adverse reactions, and lower the costs of healthcare for providers and patients.

Innovations in Banking: Consider how many people have a bank account. Now, on top of that, consider the number of credit cards that are in circulation. How many man hours would it take for employees to sift through the thousands of transactions that take place every day? By the time they noticed an anomaly, your bank account could be empty or your credit card maxed out. Using location data and purchase patterns, AI can also help banks and credit issuers identify fraudulent behavior while it is happening. These machine learning based anomaly detection models monitor transaction requests. They can spot patterns in your transactions and alert users to suspicious activity. They can even confirm with you that the purchase was indeed yours before they process the payment. It may seem inconvenient if it was just you eating at a restaurant while traveling on holiday, but it could end up saving you thousands of dollars someday.

Personalized Digital Media: Machine learning has massive potential in the entertainment industry, and the technology has already found a home in streaming services such as Netflix, Amazon Prime, Spotify, and Google Play. Some algorithms are already being used to eliminate buffering and low-quality playback, getting you the best quality from your internet service provider. ML algorithms are also making use of the almost endless stream of data about consumers' viewing habits, helping streaming services offer more useful recommendations. "Given the rapid pace of research, we expect AI to be able to create new personalized media, such as music according to your taste. Imagine a future music service that doesn't just play existing songs you might like, but continually generates new songs just for you.

They will help more and more with the production of media too. NLP (Natural Language Processing) algorithms help write trending news stories to decrease production time, and a new MIT-developed AI named Shelley is helps users write horror stories through deep learning algorithms and a bank of user-generated fiction. At this rate, the next great content creators may not be human at all.

Home Security and Smart Homes: For the best tech in home security, many homeowners look toward AI-integrated cameras and alarm systems. These cutting-edge systems use facial recognition software and machine learning to build a catalog of your home's frequent visitors,

allowing these systems to detect uninvited guests in an instant. AI-powered smart homes also provide many other useful features, like tracking when you last walked the dog or notifying you when your kids come home from school. The newest systems can even call for emergency services autonomously, making it an attractive alternative to subscription-based services that provide similar benefits.

 **Smart Home Diagram:** Consumer AI will enable wave after wave of convenient automations in the home. When combined with appliances, AI could make housework and household management seamless. AI-powered apps which allow the oven to communicate with the refrigerator and the pantry robot would act like home chefs. Instant replenishment of food and supplies would mean never running out of anything again. Cleaning could be scheduled through sensor-to-appliance connections, after which robotic cleaners would work almost completely independently of humans.

 **Jetsons Smart Home:** Another advantage of smart homes would be a reduction of household waste and automated recycling, putting the household in better balance with the ecosystem. Releasing humans from housework could deliver major benefits in terms of improving sustainability, saving time, and reducing stress.

Streamlined Logistics and Distribution: Imagine getting a package in just a few hours and at a very low shipping cost. That's the promise of AI in logistics and distribution, with its promise to tame the massive amounts of data and decisions in the trillion-dollar shipping and logistics industry. Amazon has already started experimenting with autonomous drones that blow their already-quite-fast two-day shipping out of the water. Currently, shipping costs are still quite expensive. Improving efficiency through AI integration and automation will mean big reductions in shipping costs and increases in delivery speed. Optimization opportunities in supply chain management, vehicle maintenance, and inventory will also make shipping faster, easier, and more environmentally friendly.

Digital Personal Assistants: Imagine never needing to worry about preparing dinner, as your personal assistant knows what you like, what you have in your pantry and which days of the week you like to cook at home. Imagine that when you get back from work, all your groceries are waiting at your doorstep, ready for you to prepare that delicious meal you've been craving. You even have a bonus recipe for a new dessert you've been meaning to try. Digital assistants are getting smarter by the year. Companies such as Amazon and Google are pouring billions of dollars into making digital assistants even better at speech recognition and learning about our daily routines, opening the door to more and more complex tasks.

Brick and Mortar and AI: It might be anticipated that a time when people will no longer need to wait in line at a store. Observing how tech and retail are merging, like Amazon and Whole

Foods. While some people claim that e-commerce and the Internet will completely eat away the traditional retail market, the more likely scenario is that they will arrive at some sort of equilibrium. However, it's undeniable that even the biggest traditional retail giants are starting to adopt AI-powered tools to gain a competitive edge. "Thanks to AI and ML, the face will be the new credit card, the new driver's license and new barcode. Facial recognition is already completely transforming security with biometric capabilities being adopted.

Customized News and Market Reports: Technologies are adjacent to providing customized news and market reports, and newsrooms are starting to embrace the possibilities. Can you imagine getting market reports that were written on demand for you and not just when the market closed? Instead of a generic recap of market performance, your customized report compares how your portfolio performed against the broader market, citing key reasons why. For example: "It's 3:14 pm. The market is currently up 2%, but your portfolio is down 3%. This is attributed in part to the purchase of XYZ stock last week, which has fallen sharply since ..." While the most obvious application of this technology would be in the finance and investing space, there are plenty of other domains that would benefit as well, including ad tech, agriculture, sports, and more.

Flight Simulation and Transport: Have you flown on an airplane lately? If so, then you've already experienced transportation automation at work. These modern commercial aircraft use FMS (Flight Management System), a combination of GPS, motion sensors, and computer systems to track its position during flight. So an average Boeing 777 pilot spends just seven minutes actually flying the plane manually, and many of those minutes are spent during take-off and landing. The leap into self-driving cars is more complicated. There are more cars on the road, obstacles to avoid, and limitations to account for in terms of traffic patterns and rules. Even so, self-driving cars are already a reality. These AI-powered cars have even surpassed human-driven cars in safety, according to a study with 55 Google vehicles that have driven over 1.3 million miles altogether. The navigation question has already been solved long ago. Google Maps already source's location data from your smartphone.

By comparing the location of a device from one point in time to another, it can determine how fast the device is traveling. Put simply, it can determine how slow traffic is in real time. It can combine that data with incidents reported by users to build a picture of the traffic at any given moment. Maps can recommend the fastest route for you based on traffic jams, construction work or accidents between you and your destination. But what about the skill of actually driving a car? Well, machine learning allows self-driving cars to instantaneously adapt to changing road conditions, while at the same time learning from new road situations. By continuously parsing through a stream of visual and sensor data, onboard computers can make split-second decisions even faster than well-trained drivers.

It's not magic. It's based on the exact same fundamentals of machine learning used in other industries. You have input features (i.e. the real-time visual and sensor data) and an output (i.e. a decision among the universe of possible next "actions" for a car). So, sure these self-driving cars already exist, but are they ready for prime-time? Perhaps not yet, since the vehicles are currently required to have a driver present for safety. So despite exciting developments in this new field of automated transportation, the technology isn't perfect yet. But give it a few months or years, and you'll probably want to have one of these cars yourself.

Online Video Streaming (Netflix): With over 100 million subscribers, there is no doubt that Netflix is the daddy of the online streaming world. Netflix's speedy rise has all movie industrialists taken aback – forcing them to ask, "How on earth could one single website take on Hollywood?" The answer is Machine Learning. The Netflix algorithm constantly gathers massive amounts of data about users' activities like:

- 📊 When you pause, rewind, or fast forward
- 📊 What day you watch content (TV Shows on Weekdays and Movies on Weekends)
- 📊 The Date and Time you watch
- 📊 When you pause and leave content (and if you ever come back)
- 📊 The ratings Given (about 4 million per day), Searches (about 3 million per day)
- 📊 Browsing and Scrolling Behavior

And a lot more. They collect this data for each subscriber they have and use their Recommender System and a lot of Machine Learning Applications. That's why they have such a huge customer retention rate.

Crop Disease and Weed Detection: ML-driven image agriculture allows farmers to rely upon digital tools to recognize weed species and to determine which crops are healthy and which ones are infested with disease caused by fungi, bacteria, or viruses. The ability to identify weeds with digital tools makes it possible to train mechanical devices (robots) to pull weeds from fields, protecting the environment from damage caused by pesticide use and saving farmers time, effort, and money. Additionally, digital applications that can evaluate crops for disease can also provide an accurate disease diagnosis and recommend an optimal treatment plan. This technology helps farmers avoid settling on a one-size-fits-all solution that not only fails to address a specific disease, but may inadvertently cause undesirable side effects, such as pollution or bee-population reduction. It also lets the companies that manufacture crop disease treatment products to better serve their customers.

As many people have wisely observed, the dream of artificial intelligence is not new. It has been around since the very earliest days of computing. Pioneers have always imagined ways to build intelligent learning machines. Currently, most promising approach of AI is the use of

applied machine learning. Rather than trying to encode machines with everything they need to know up front (which is impossible), we want to enable them to learn, and then to learn how to learn. Machine learning's time has come, and it is in the process of revolutionizing all of our lives. Artificial intelligence (AI) and machine learning is now measured to be one of the biggest innovations since the microchip. AI used to be a fanciful concept from science fiction, but now it's becoming a daily reality. Neural networks (imitating the process of real neurons in the brain) are paving the way toward breakthroughs in machine learning, called "deep learning."

Machine learning can help us live happier, healthier, and more productive lives, if we know how to harness its power. Some say that AI is ushering in another "industrial revolution." Whereas the previous Industrial Revolution harnessed physical and mechanical strength, this new revolution will harness mental and cognitive ability. One day, computers will not only replace manual labor, but also mental labor. But how exactly will this happen? And is it already happening?

2.3 WHERE TO APPLY MACHINE LEARNING?

Machine Learning is being used to solve many problems, which problems can you use it for? In the last 5 years there has been growing success using machine learning. Rapidly increasing processor speed and access to large scale data sets are allowing for many new problems to utilize machine learning successfully. Today machine learning is being applied by innovative companies in almost every field.

Using machine learning to solve problems is becoming central to many company's core points of differentiation. At every, step in the development of machine learning, there will be huge economic payoff for the companies involved. Machine Learning is the development of computer programs that use the information in datasets to decide outputs. Traditional computer programs explicitly define the steps to transform an input to an output. For example, when a sale is made for \$2 and paid for with \$5, give back \$3 change. This is based on mathematical rules which can be executed without knowing anything about other transactions that have occurred.

Machine learning is useful for problems where we want to decide the output based on data. The model that determines the output creates patterns based on variations in the data; it is then trained by the data that passes through and 'learning' occurs which creates outputs closer to the goal. The type of problems that we want to use data to make decisions on (i.e. machine learning) are problems where rules of deciding the output are not clearly defined. These types of problems include human interaction and natural systems. These types of interactions are good candidates because there is always some degree of unknown and highly complex rules that will determine the outcome of a natural interaction.

When thinking specifically about where machine learning can be applied it is often valuable to identify what the natural impactor in the problem is. For example, in advertising we want to serve the ad that is most likely to get clicked. However, a human will make the decision to click on the ad or not, which is the natural impactor. Natural impactors do not have a clear set of rules, which makes them a good candidate to learn from the data. In this case, we want to learn about how people have acted with the previous ads to make fortitude about which ad to show next.

The thinking pattern can be generalized as the following:

- ✚ What output do we want to reach?
- ✚ Is calculating this output dependent on a human perception or natural system?

A few examples using this pattern:

- ✚ We want to determine which emails are spams. Yes this is based on human perception because the email is only spam if the user would determine it as spam.
- ✚ We want to determine the amount of change to give back. No, this is a math equation; it is based on formal math rules and only needs the input of cost and payment amount to calculate change returned.
- ✚ What animal is in the picture? Yes, an animal will exist in a natural system. Unless the same pictures are used, the pictures taken of an animal in a natural system will continue to have different variables.
- ✚ Which driving maneuver will provide safety and follow laws? Mostly yes. While traffic laws may be a well-defined set of rules, the best actions for providing safety in a natural system are not defined.

Once we have framed our problem, understanding the goal and the decision we are trying to make using data, we can begin to explore the data/feedback that will be provided to the system. Machine learning problems are classified into three buckets depending on the nature of the learning “feedback” available to the system. These are:

- ✚ **Supervised learning:** Looking to predict a future value based on existing data, known as a Training Set.
- ✚ **Unsupervised Learning:** Problems, when given a data set with no rights and wrongs will look for patterns in the data.
- ✚ **Reinforcement learning:** A computer program interacts with a dynamic environment in which it must perform a certain goal (such as driving a vehicle), without a teacher explicitly telling it whether it has come close to its goal. Another example is learning to play a game by playing against an opponent.

Often times the most difficult part of effectively using machine learning is creating a suitable data set. A good data set will be standardized and large. Supervised problems require a training data set, which is a collection of data points that have a label defining the correct answer. This training data is used to ‘train’ the system for data where the answer is not defined. Examples of training datasets are the MINST and ImageNet public datasets. Once we have our standardized data, we can begin to train our model. There are many machine learning libraries that exist; the most popular lately is TensorFlow, which has been open sourced by Google. TensorFlow offers examples that span many common machine learning problem types.

Many of these models can be reused to solve different but structurally similar problems. These libraries will allow you to quickly experiment and apply machine learning with near to state-of-the-art algorithms. It is important to remember that ML is not a solution for every type of problem. There are certain cases where robust solutions can be developed without using ML techniques. For example, you don’t need ML if you can determine a target value by using simple rules, computations, or predetermined steps that can be programmed without needing any data-driven learning.

Use machine learning for the following situations:

- 📌 **You cannot code the rules:** Many human tasks (such as recognizing whether an email is spam or not spam) cannot be adequately solved using a simple (deterministic), rule-based solution. A large number of factors could influence the answer. When rules depend on too many factors and many of these rules overlap or need to be tuned very finely, it soon becomes difficult for a human to accurately code the rules. You can use ML to effectively solve this problem.
- 📌 **You cannot scale:** You might be able to manually recognize a few hundred emails and decide whether they are spam or not. However, this task becomes tedious for millions of emails. ML solutions are effective at handling large-scale problems.

Building ML applications is an iterative process that involves a sequence of steps. To build an ML application, follow these general steps:

- 📌 Frame the core ML problem(s) in terms of what is observed and what answer you want the model to predict.
- 📌 Collect, clean, and prepare data to make it suitable for depletion by ML model training algorithms. Visualize and analyze the data to run sanity checks to validate the quality of the data and to realize the data.
- 📌 Often, the raw data (input variables) and answer (target) are not represented in a way that can be used to train a highly predictive model. Therefore, you typically should

attempt to construct more predictive input representations or features from the raw variables.

- ✚ Feed the resulting features to the learning algorithm to build models and evaluate the quality of the models on data that was held out from model building.

Below is a selection of best-practices and concepts of applying machine learning. Some of these principles will clarify how ML is used, and how to avoid some of the common pitfalls that companies and researchers might be vulnerable to in starting off on an ML-related project.

- ✚ Arguably the most important factor in successful machine learning projects is the features used to describe the data, and having adequate data to train your models in the first place.
- ✚ Most of the time when algorithms don't perform well, it's due to a problem with the training data (i.e. insufficient data; noisy data; or insufficient features describing the data for making decisions).
- ✚ There is no given connection between number of parameters of a model and tendency to overfit.
- ✚ Obtaining experimental data (as opposed to observational data, over which we have no control) should be done if possible (for example, data gleaned from sending different variations of an email to a random audience sampling).
- ✚ Whether or not we label data causal or correlative, the more important point is to predict the effects of our actions.
- ✚ Always set aside a portion of your training data set for cross validation; you want your chosen classifier or learning algorithm to perform well on fresh data.

2.4 IS MACHINE LEARNING A COMPUTER PROGRAM?

Traditional computer programming has been around for more than a century, with the first known computer program dating back to the mid-1800s. Traditional Programming refers to any manually created program that uses input data and runs on a computer to produce the output. But for decades now, an advanced type of programming has revolutionized business, particularly in the areas of intelligence and embedded analytics. In Machine Learning, also known as augmented analytics, the input data and output are fed to an algorithm to create a program. This yields powerful insights that can be used to predict future outcomes. Here's a closer comparison of traditional programming versus machine learning:

Traditional Programming

Traditional programming is a manual process, meaning a programmer creates the program. But without anyone programming the logic, one has to manually formulate or code rules.



In machine learning method, on the other hand, the algorithm automatically formulates the rules from the data.

Machine Learning

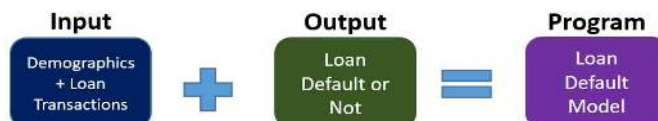
Unlike traditional programming, machine learning is an automated process. It can increase the value of your embedded analytics in many expanses, including data prep, natural language interfaces, automatic outlier detection, recommendations, and causality and significance detection. All of these features help speed user insights and reduce decision bias.



For example, if you feed in customer demographics and transactions as input data and use historical customer churn rates as your output data, the algorithm will formulate a program that can predict if a customer will churn or not. That program is called a model.



Customers, in general, have a lot of such input and output data and they can feed that to the algorithms to create predictive models. For example, feed-in customer information/loan transactions (input) and how many defaulted on the loan (observed output), and it will create a model to predict who will default on the loan.

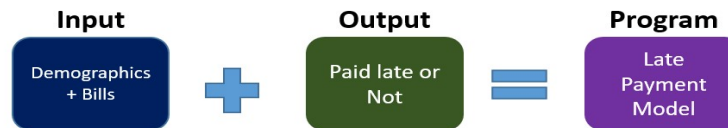


You can use this model to predict business outcomes in any situation where you have input and historical output data:

- ✚ Identify the business question you would like to ask.
- ✚ Identify the historical input.

- 📊 Identify the historically observed output (i.e., data samples for when the condition is true and for when it's false).

For instance, if you want to predict who will pay the bills late, identify the input (customer demographics, bills) and the output (pay late or not), and let the machine learning use this data to create your model.



Machine learning can turn your business data into a financial asset. You can point the algorithm at your data so it can learn powerful rules that can be used to predict future outcomes. It's no wonder predictive analytics is now the number one capability on product roadmaps.

2.5 WHAT ARE THE SKILLS REQUIRED FOR MACHINE LEARNING?

It's critical to understand the differences between a Data Analyst and a Machine Learning engineer. In simplest form, the key distinction has to do with the end goal. A Data Analyst will be analysing data in order to tell a story, and to produce actionable insights. The emphasis is on dissemination; charts, models, visualizations. The analysis is performed and presented by human beings, to other human beings who may then go on to make business decisions based on what's been presented.

This is especially important to note, the "audience" for your output is human. A Machine Learning engineer, on the other hand, will anticipate the final "output" as working software (not the analyses or visualizations that you may have to create along the way), and your "audience" for this output often consists of other software components that run autonomously with minimal human supervision. The intelligence is still meant to be actionable, but in the Machine Learning model, the decisions are being made by machines and they affect how a product or service behaves. This is why the software engineering skill set is so important in Machine Learning.

Before getting into specific skills, there is one more concept to address. Being a Machine Learning engineer necessitates understanding the entire ecosystem that you're designing for. Let's say you're working for a grocery chain, and the company wants to start issuing targeted coupons based on things like the past purchase history of customers, with a goal of generating coupons that shoppers will actually use. In a Data Analysis model, you could collect the purchase data, do the analysis to figure out trends, and then propose strategies. The Machine

Learning approach would be to write an automated coupon generation system. But what does it take to write that system, and have it work? You have to understand the whole ecosystem inventory, catalog, pricing, purchase orders, bill generation, Point of Sale software, CRM software, etc.

Ultimately, the process is less about understanding Machine Learning algorithms or when and how to apply them and more about understanding the systemic interrelationships and writing working software that will successfully integrate and interface. Remember, Machine Learning output is actually working software! Now, let's get into the real details of what it takes to be a Machine Learning engineer. We're going to break this into two primary sections: Summary of Skills, and Languages and Libraries.

Computer Science Fundamentals and Programming: Computer science fundamentals important for Machine Learning engineers include data structures (stacks, queues, multi-dimensional arrays, trees, graphs, etc.), algorithms (searching, sorting, optimization, dynamic programming, etc.), computability and complexity (P vs. NP, NP-complete problems, big-O notation, approximate algorithms, etc.), and computer architecture (cache, memory, bandwidth, deadlocks, distributed processing, etc.). You must be able to apply, implement, adapt or address them (as appropriate) when programming.

Probability and Statistics: A formal characterization of probability (Bayes rule, conditional probability, likelihood, independence, etc.) and techniques derived from it (Bayes Nets, Markov Decision Processes, Hidden Markov Models, etc.) are at the heart of many Machine Learning algorithms; these are a means to deal with uncertainty in the real world. Closely related to this is the field of statistics, which provides various measures (mean, median, variance, etc.), distributions (uniform, normal, binomial, Poisson, etc.) and analysis methods (ANOVA, hypothesis testing, etc.) that are necessary for building and validating models from observed data. Numerous Machine Learning algorithms are fundamentally extensions of statistical modelling procedures.

Data Modelling and Evaluation: Data modelling is the process of assessing the underlying structure of a given dataset, with the goal of finding useful patterns (correlations, clusters, eigenvectors, etc.) and predicting properties of previously unseen instances (classification, regression, anomaly detection, etc.). A key part of this estimation process is continually evaluating how good a given model is. Depending on the task at hand, you will need to choose an appropriate accuracy and the error measure (e.g. log-loss for classification, sum-of-squared-errors for regression, etc.) and an evaluation strategy (training-testing split, sequential vs. randomized cross-validation, etc.). Iterative learning algorithms often directly utilize resulting errors to tweak the model (e.g. back-propagation for neural networks), therefore understanding these measures are very important even for just applying standard algorithms.

Applying Machine Learning Algorithms and their Libraries: Standard implementations of Machine Learning algorithms are extensively available through libraries/packages/APIs (e.g. scikit-learn, Theano, Spark MLlib, H2O, TensorFlow etc.), but applying them effectively involves choosing a suitable model (decision tree, nearest neighbor, neural net, support vector machine, ensemble of multiple models, etc.), a learning procedure to fit the data (linear regression, gradient descent, genetic algorithms, bagging, boosting, and other model-specific methods), as well as understanding how hyperparameters affect learning. You also need to be aware of the relative advantages and disadvantages of different approaches, and the numerous gotchas that can trip you (bias and variance, overfitting and underfitting, missing data, data leakage, etc.).

Software Engineering and System Design: At the end of the day, Machine Learning engineer's typical output or deliverable is software. And often it is a small component that fits into a larger ecosystem of products and services. You need to understand how these different pieces work together, communicate with them (using library calls, REST APIs, database queries, etc.) and build appropriate interfaces for your component that others will depend on. Careful system design may be necessary to avoid bottlenecks and let your algorithms scale well with increasing volumes of data. Software engineering best practices (including requirements analysis, system design, modularity, version control, testing, documentation, etc.) are invaluable for productivity, collaboration, quality and maintainability. What is perhaps most compelling about Machine Learning is its seemingly limitless applicability. There are already so many fields being impacted by Machine Learning, including education, finance, computer science, and more.

There are also virtually no fields to which Machine Learning doesn't apply. In some cases, Machine Learning techniques are in fact desperately needed. Healthcare is an obvious example. Machine Learning techniques are already being applied to critical arenas within the Healthcare sphere, impacting everything from care variation reduction efforts to medical scan analysis. Machine learning has the potential to change health care across the industry, from enabling the next-generation electronic health record to population-level risk stratification from health insurance claims. The world is unquestionably changing in rapid and dramatic ways, and the demand for Machine Learning engineers is going to keep increasing exponentially. The world's challenges are complex, and they will require complex systems to solve them. Machine Learning engineers are building these systems. If this is your future, then there's no time like the present to start mastering the skills and developing the mindset you're going to need to succeed.

2.6 ITS CONNECTION WITH ARTIFICIAL INTELLIGENCE?

In the modern technology landscape, artificial intelligence is divided into two key areas. The first one is general AI, which is based on the concept that a system can handle tasks like

speaking and translating, recognizing sounds and objects, performing business or social transactions etc. The other one is applied AI that refers to concepts like driverless cars. Data science uses key skills of a wide range of fields including machine learning, statistics, visualization etc. It enables us to identify meaning and appropriate information from huge volumes of data to make informed decisions in technology, science, business etc. For a simpler view on the relation amid these technologies; “Artificial intelligence is applied based on machine learning. And machine learning is a part of data science that draws features from algorithms and statistics to work on the data extracted from and produced by multiple resources.

Therefore, you can say data science merges together a bunch of algorithms obtained from machine learning to develop a solution, and during the process, lots of concepts from traditional domain expertise, statistics and mathematics are borrowed. In other words, data science stands for an all-inclusive term that consists of aspects of ML for functionality. Interestingly, ML is also an element of artificial intelligence, where a diverse set of purpose is achieved on a whole new level. ML and AI are a part and parcel of data science. You can think of deep learning, machine learning and artificial intelligence as a set of Russian dolls nested within each other, beginning with the smallest and working out. Deep learning is a subset of machine learning, and machine learning is a subset of AI, which is an umbrella term for any computer program that does something smart. In other words, all machine learning is AI, but not all AI is machine learning, and so forth. John McCarthy, widely recognized as one of the godfathers of AI, defined it as “the science and engineering of making intelligent machines.” AI can be a pile of if-then statements, or a complex statistical model mapping raw sensory data to symbolic categories.

The if-then statements are simply rules explicitly programmed by a human hand. Taken together, these if-then statements are sometimes called rules engines, expert systems, knowledge graphs or symbolic AI. Collectively, these are known as Good, Old-Fashioned AI (GOFAI). The intelligence that rules engines mimic could be that of an accountant with knowledge of the tax code, who takes information you feed it, runs the information through a set of static rules, and gives you the amount of taxes you owe as a result. In the US, we call that TurboTax. Usually, when a computer program designed by AI researchers actually succeeds at something, like winning at chess, many people say it’s “not really intelligent”, because the algorithm’s internals are well understood. The critics think intelligence must be rather intangible, and exclusively human. A wag would say that true AI is whatever computers can’t do yet. Here are a few other definitions of artificial intelligence:

-  A branch of computer science dealing with the simulation of intelligent behavior in computers.
-  The capability of a machine to imitate intelligent human behavior.

- 🧩 A computer system able to perform tasks that normally require human intelligence, such as visual perception, speech recognition, decision-making, and translation between languages.

There are a lot of ways to simulate human intelligence, and some methods are more intelligent than others. Machine learning is a subset of AI. That is, all machine learning counts as AI, but not all AI counts as machine learning. For example, symbolic logic; rules engines, expert systems and knowledge graphs, could all be described as AI, and none of them are machine learning. One aspect that separates machine learning from the knowledge graphs and expert systems is its ability to modify itself when exposed to more data; i.e. machine learning is dynamic and does not require human intervention to make certain changes. That makes it less brittle, and less reliant on human experts. “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E .”

In 1959, Arthur Samuel, one of the pioneers of machine learning, defined machine learning as a “field of study that gives computers the ability to learn without being explicitly programmed.” That is, machine-learning programs have not been explicitly entered into a computer, like the if-then statements above. Machine-learning programs, in a sense, adjust themselves in response to the data they’re exposed to (like a child that is born knowing nothing adjusts its understanding of the world in response to experience). Samuel taught a computer program to play checkers. His goal was to teach it to play checkers better than himself, which is obviously not something he could program explicitly. He succeeded, and in 1962 his program beat the checkers champion of the state of Connecticut. The “learning” part of machine learning means that ML algorithms attempt to optimize along a certain dimension; i.e. they usually try to minimize error or maximize the likelihood of their predictions being true.

This has three names: an error function, a loss function, or an objective functions, because the algorithm has an objective. When someone says they are working with a machine-learning algorithm, you can get to the gist of its value by asking: What’s the objective function? How does one minimize error? Well, one way is to build a framework that multiplies inputs in order to make guesses as to the inputs’ nature. Different outputs/guesses are the product of the inputs and the algorithm. Usually, the initial guesses are quite wrong, and if you are lucky enough to have ground-truth labels pertaining to the input, you can measure how wrong your guesses are by contrasting them with the truth, and then use that error to modify your algorithm. That’s what neural networks do. They keep on measuring the error and modifying their parameters until they can’t achieve any less error. They are, in short, an optimization algorithm. If you tune them right, they minimize their error by guessing and guessing and guessing again. Deep learning is a subset of machine learning.

Usually, when people use the term deep learning, they are referring to deep artificial neural networks, and somewhat less frequently to deep reinforcement learning. Deep artificial neural networks are a set of algorithms that have set new records in accuracy for many important problems, such as image recognition, sound recognition, recommender systems, natural language processing etc. For example, deep learning is part of DeepMind's well-known AlphaGo algorithm, which beat the former world champion Lee Sedol at Go in early 2016, and the current world champion Ke Jie in early 2017. A more complete explanation of neural networks is here. Deep is a technical term. It refers to the number of layers in a neural network. A shallow network has one so-called hidden layer, and a deep network has more than one. Multiple hidden layers allow deep neural networks to learn features of the data in a so-called feature hierarchy, because simple features (e.g. two pixels) recombine from one layer to the next, to form more complex features (e.g. a line).

Nets with many layers pass input data (features) through more mathematical operations than nets with few layers, and are therefore more computationally intensive to train. Computational intensity is one of the hallmarks of deep learning, and it is one reason why a new kind of chip called GPUs is in demand to train deep-learning models. So you could apply the same definition to deep learning that Arthur Samuel did to machine learning, a "field of study that gives computers the ability to learn without being explicitly programmed", while adding that it tends to result in higher accuracy, require more hardware or training time, and perform extremely well on machine perception tasks that involved unstructured data such as blobs of pixels or text.

The advances made by researchers at DeepMind, Google Brain, OpenAI and various universities are accelerating. AI is capable of solving harder and harder problems better than humans can. This means that AI is changing faster than its history can be written, so predictions about its future quickly become obsolete as well. Are we chasing a breakthrough like nuclear fission (possible), or are attempts to wring intelligence from silicon more like trying to turn lead into gold? Those who believe that AI progress will continue apace tend to think a lot about strong AI, and whether or not it is good for humanity. Among those who forecast continued progress, one camp emphasizes the benefits of more intelligent software, which may save humanity from its current stupidities; the other camp worries about the existential risk of a super intelligence. Given that the power of AI advances hand in hand with the power of computational hardware, advances in computational capacity, such as better chips or quantum computing, will set the stage for advances in AI.

On a purely algorithmic level, most of the astonishing results produced by labs such as DeepMind come from merging different approaches to AI, much as AlphaGo combines deep learning and reinforcement learning. Relating deep learning with symbolic reasoning,

analogical reasoning, Bayesian and evolutionary methods all show promise. Those who do not believe that AI is making that much progress relative to human intelligence are forecasting another AI winter, during which funding will dry up due to generally disappointing results, as has happened in the past. Many of those people have a pet algorithm or approach that plays with deep learning. There are the pragmatists, plugging along at the math, struggling with messy data, scarce AI talent and user acceptance.

2.7 ARTIFICIAL NEURAL NETWORKS AND MACHINE LEARNING?

We need machine learning for tasks that are too complex for humans to code directly, i.e. tasks that are so complex that it is impractical, if not difficult, for us to work out all of the nuances and code for them explicitly. So instead, we provide a machine learning algorithm with a large amount of data and let it explore and search for a model that will work out what the programmers have set out to achieve. Let's look at these two examples:

-  It's very hard to write programs to solve problems like recognizing a 3D object, from a novel viewpoint, in new lighting conditions, in a cluttered scene. We don't know what program to write because we don't know how it's done in our brain. Even if we had a good idea for how to do it, the program might be terribly complicated.
-  It's hard to write a program to compute the probability that a credit card transaction is fraudulent. There may not be any rules that are both simple and reliable. We need to combine a very large number of weak rules. Fraud is a moving target, but the program needs to keep changing.

Then comes the Machine Learning Approach: instead of writing a program by hand for each specific task, we collect lots of examples that specify the correct output for a given input. A machine learning algorithm then takes these examples and produces a program that does the job. The program produced by the learning algorithm may look very different from a typical hand-written program; it may contain millions of numbers. If we do it right, the program works for new cases, as well as the ones we trained it on. If the data changes, the program can change too by training from the new data. You should note that conducting massive amounts of computation is now cheaper than paying someone to write a task-specific program. Some examples of tasks best solved by machine learning include:

-  **Recognizing patterns:** Objects in real scenes, facial identities or facial expressions, and/or spoken words.
-  **Recognizing anomalies:** Unusual series of credit card transactions, unusual patterns of sensor readings in a nuclear power plant.
-  **Prediction:** Future stock prices or currency exchange rates, which movies a person will like.

What are Neural Networks?

Neural Networks are a class of models within the general machine learning literature. Neural networks are a specific set of algorithms that have revolutionized machine learning. They are inspired by biological neural networks and the current so-called deep neural networks have proven to work moderately well. Neural Networks are themselves general function approximations, which is why they can be applied to almost any machine learning problem about learning a complex mapping from the input to the output space. Here are the three reasons you should know neural networks:

- 📖 To understand how the brain actually works: it's very big and very complicated and made of stuff that dies when you poke it, so we need to use computer simulations.
- 📖 To understand a style of parallel computation inspired by neurons and their adaptive connections: it's a very different style from sequential computation.
- 📖 To solve practical problems by using novel learning algorithms inspired by the brain: learning algorithms can be very useful even if they are not how the brain actually works.

Here are the most frequently used neural networks in Machine Learning and Computer Vision;

- 📖 Perceptrons
- 📖 Convolutional Neural Networks
- 📖 Recurrent Neural Networks
- 📖 Long/Short Term Memory Networks
- 📖 Gated Recurrent Unit
- 📖 Hopfield Network
- 📖 Boltzmann Machine
- 📖 Deep Belief Networks
- 📖 Autoencoders
- 📖 Generative Adversarial Network

Artificial Neural networks (ANN) or neural networks are computational algorithms. It intended to simulate the behavior of biological systems composed of “neurons”. ANNs are computational models inspired by an animal’s central nervous systems. It is capable of machine learning as well as pattern recognition. These open as systems of interconnected “neurons” which can compute values from inputs. A neural network is an oriented graph. It consists of nodes which in the biological analogy represent neurons, connected by arcs. It corresponds to dendrites and synapses. Each arc associated with a weight while at each node. Apply the values received as input by the node and define Activation function along the incoming arcs, adjusted by the weights of the arcs.

A neural network is a machine learning algorithm based on the model of a human neuron. The human brain consists of millions of neurons. It sends and process signals in the form of electrical and chemical signals. These neurons are connected with a special structure known as synapses. Synapses allow neurons to pass signals. An Artificial Neural Network is an information processing method. It works like the way human brain processes information. ANN includes a large number of connected processing units that work together to process information. They also generate meaningful results from it. The structure of a neural network consists of the number of layers, elementary units. It also consists of interconnected weight adjustment mechanism. The choice of the structure determines the results which are going to obtain. It is the most critical part of the implementation of a neural network. We can apply neural network not only for classification.

It can also apply for regression of continuous target attributes. Neural networks find great application in data mining used in sectors, e.g. economics, forensics, etc. and for pattern recognition. It can be also used for data classification in a large amount of data after careful training. ANNs are considered as simple mathematical models to enhance existing data analysis tools. Although it is not comparable with the power of the human brain, still it is the basic building block of the Artificial intelligence. Neural networks are one of the most beautiful programming paradigms ever invented. In the conventional approach to programming, we tell the computer what to do and break big problems up into many small, precisely defined tasks that the computer can easily perform. In contrast, we don't tell the computer how to solve our problems for a neural network. Instead, it learns from observational data and figures out its own solution to the problem. Nowadays, deep neural networks and deep learning achieve outstanding performance for many important problems in computer vision, speech recognition, and natural language processing. They're being arranged on a large scale by companies such as Google, Microsoft, and Facebook.

2.8 WHAT IS STATISTICAL LEARNING?

What is the difference between Machine Learning and Statistical Learning?

Even though you would think that the answer is obvious, there are a lot of novice data scientists still confused about those two approaches. It is hard to see the differences between the two, and it is probably due to how we learn Data Science. To become a data scientist, you are required to develop knowledge in multiple subjects such as Statistics, Programming, SQL, Linear Algebra and have the domain expertise. Hopefully, you will start your journey with Statistics, and most of the data scientists believe that this is the foundation in Data Science. After, when you get comfortable with Statistics, then, eventually, expand your horizons within Data Science, by sailing away from all too familiar, small datasets such as Titanic, Iris, Cars, Diamonds, etc. to more uncharted territories, to a new world of Big Data.

Nevertheless, with your confidence in Statistical Learning, you will probably take on a big data challenge and hope to generate insight from your data by applying Statistical Learning techniques. We don't want to disappoint you, but not much value will be formed from this method. This is because you incorrectly approached the situation; you applied a statistical learning solution to a machine learning problem. We cannot stress enough the importance of understanding the differences between those two. To prevent novice data scientists from the future disappointments, we have composed a list of differences between Statistical Learning and Machine Learning to aid you on your journey to success. Here are some of the differences:

- ✚ Both methods are data dependent. However, Statistical Learning relies on rule-based programming; it is formalized in the form of relationship between variables, where Machine Learning learns from data without explicitly programmed instructions.
- ✚ Statistical Learning is based on a smaller dataset with a few attributes, compared to Machine Learning where it can learn from billions of observations and attributes.
- ✚ Statistical Learning operates on assumptions, such as normality, no multicollinearity, homoscedasticity, etc. when Machine Learning is not as assumptions dependent and in most of the cases overlooks them.
- ✚ Statistical Learning is mostly about inferences; most of the idea is generated from the sample, population, and hypothesis, in comparison to Machine Learning which emphasizes predictions, supervised learning, unsupervised learning, and semi-supervised learning.
- ✚ Statistical Learning is math intensive which is based on the coefficient estimator and requires a good understanding of your data. On the other hand, Machine Learning identifies patterns from your dataset through the iterations which require a way less of human effort.

Uncertainty is a key element of many artificial intelligence environments in the real world. By uncertainly, we refer to the characteristics that prevent an AI agent from knowing the precise outcome of a specific state-action combination in a given scenario. Uncertainty is classically the result of nondeterministic and partially observable environments. Statistical learning has become a powerful weapon to overcome uncertainty in AI scenarios and, consequently, it has been widely implemented in many modern AI frameworks.

When we talk about statistical learning, there is a name that comes to mind: Bayes. Even though most of modern statistic theory was the result of the work of French mathematician Pierre-Simon de Laplace who lived five decades after Bayes, it is Bayes who got all the credit in a theorem that bears his name. Thomas Bayes was an eighteen century British clergyman that described new insights to think about chance and uncertainty. Laplace codified Bayes ideas into a single theorem that helps us reason about almost anything in the world with uncertainty:

$$P(\text{cause}|\text{effect}) = P(\text{effect}) \times P(\text{effect}|\text{cause})/P(\text{cause})$$

By $P(A|B)$ we denote the probability and A occurs given B. Replacing cause and effect in the previous equation with the probabilities of any state-action combination in an AI environment we arrive to the fundamentals of Bayesian learning. Essentially, Bayesian or statistical learning focuses on calculating the probabilities of each hypothesis and make predictions accordingly. Although Bayesian learning seems theoretically trivial, it runs into many roadblocks in real world AI solutions. Specifically, Bayesian learning models frequently result impractical in environments in which the number of hypothesis is very large or infinite. A very well-known AI algorithm that tries to address those limitations is the maximum a posteriori (MAP) model that simply makes predictions based on the single most probable hypothesis. Maybe the most notorious algorithm in statistical learning is the Naive Bayes model (also referred to as the Bayesian classifier) which uses networks to model environments in which the effects are independent given the cause. The model is “naive” precisely because it assumes that attributes are independent of each other given the class.

AI models like Native Bayes are only appropriate in fully-observable environments. Many AI don't resemble the real world AI environments. For instance, many AI environments contain hidden variables that are not available in the training data set. Let's take an example from the health care world, electronical medical records typically include observations about the symptoms is a disease rather than about the disease itself. In those scenarios algorithms such as unsupervised clustering using Mixture of Gaussians, Learning Bayesian Networks and Learning Hidden Markov Models are typically a good choice. As it turns out, statistical learning is not a solution to every AI problem. Laplace's original thought that any form of human knowledge can be codified in a statistical network failed to account for many characteristics of human reasoning. A very well-known limitation of statistical learning models is the absence of logic which results key in many forms of knowledge. As a result, statistical learning techniques are not applicable in many AI scenarios in the real world.

Statistical learning mostly has been used to investigate whether statistical regularities within a language can explain some characteristics of language acquisition with experientially developed expectancies. Simple frequency of an item is not enough for statistical learning to occur. Instead, a transitional probability must be used to examine whether statistical patterns could serve as a cue to word boundaries. Statistical learning employs some method of probability of a given outcome for a given set of input stimuli. The system matches a set of active input stimuli to one or more sets of statistical functions having the same input parameters, and then applies the function to the input values, thus generating an expected outcome, course of action, or classification assignment. The probabilities applied to the calculation affect the result. Even though most will argue that Machine Learning is superior,

and to some extent, it is true. On the contrary, with the use of Statistical Learning, you familiarize yourself better with your data which help you to build that needed confidence in your modeling.

2.9 ROLE OF MACHINE LEARNING IN COMPUTER VISION?

Learning and computation be responsible for machine, the ability to better understand the context of images and build visual systems which truly understand intelligence. The huge amount of image and video content urges the scientific community to make sense and identify patterns amongst it to reveal particulars which we aren't aware of. Computer Vision generates mathematical models from images; Computer Graphics draws in images from models and lastly image processing takes image as an input and gives an image at the output. Computer Vision is an overlapping field drawing on concepts from areas such as artificial intelligence, digital image processing, machine learning, deep learning, pattern recognition, probabilistic graphical models, scientific computing and a lot of mathematics. Computer vision focuses on replicating parts of the complexity of the human vision system and enabling computers to identify and process objects in images and videos in the same way that humans do. Until recently, computer vision only worked in limited capacity.

“Thanks to advances in artificial intelligence and innovations in machine learning, deep learning and neural networks, the field has been able to take great leaps in recent years and has been able to surpass humans in some tasks related to detecting and labeling objects. Machine learning provided a different approach to solving computer vision problems. With machine learning, developers no longer needed to manually code every single rule into their vision applications. As an alternative, they programmed “features,” smaller applications that could detect specific patterns in images. They then used a statistical learning algorithm such as linear regression, logistic regression, decision trees or support vector machines (SVM) to detect patterns and classify images and detect objects in them. Machine learning helped solve many problems that were historically challenging for classical software development tools and approaches. For instance, years ago, machine learning engineers were able to create software that could predict breast cancer survival windows superior than human experts. However, building the features of the software required the efforts of dozens of engineers and breast cancer experts and took a lot of time develop.

Deep learning provided a basically different approach to doing machine learning. Deep learning relies on neural networks, a general-purpose function that can solve any problem representable through examples. When you provide a neural network with many labeled examples of a specific kind of data, it'll be able to extract common patterns between those examples and transform it into a mathematical equation that will help classify future pieces of

information. For instance, creating a facial recognition application with deep learning only requires you to develop or choose a preconstructed algorithm and train it with examples of the faces of the people it must detect. Given enough examples (lots of examples), the neural network will be able to detect faces without further instructions on features or measurements. Deep learning is a very effective method to do computer vision. In most cases, creating a good deep learning algorithm comes down to gathering a large amount of labeled training data and tuning the parameters such as the type and number of layers of neural networks and training epochs. Compared to previous types of machine learning, deep learning is both easier and faster to develop and deploy.

Most of current computer vision applications such as cancer detection, self-driving cars and facial recognition make use of deep learning. Deep learning and deep neural networks have moved from the conceptual realm into practical applications thanks to availability and advances in hardware and cloud computing resources. However, deep learning algorithms have their own limits, most notable among them being lack of transparency and interpretability. Thanks to deep learning, computer vision has been able to solve the first of the two problems mentioned at the beginning of this section, meaning the detecting and classifying of objects in images and video. In fact, deep learning has been able to exceed human performance in image classification. However, despite the nomenclature that is reminiscent of human intelligence, neural networks function in a way that is fundamentally different from the human mind.

The human visual system relies on identifying objects based on a 3D model that we build in our minds. We are also able to transfer knowledge from one domain to another. For instance, if we see a new animal for the first time, we can quickly identify some of the body parts found in most animals such as nose, ears, tail, and legs. Deep neural networks have no notion of such concepts and they develop their knowledge of each class of data individually. At their heart, neural networks are statistical models that compare batches of pixels, though in very intricate ways. That's why they need to see many examples before they can develop the necessary foundations to recognize every object. Accordingly, neural networks can make stupid and dangerous mistakes when not trained properly.

But where computer vision really struggling is in understanding the context of images and the relation between the objects they see. We humans can quickly tell without a second thought about a picture, because we have an understanding of abstract concepts it represents. All of that and countless other little experiences we've had in our lives quickly go through our minds when we see a picture. Likewise, if told about something unusual, like a "winter picnic" or a "volcano picnic" you can quickly put together a mental image of what such an exotic event would look like. For a computer vision algorithm, pictures are still arrays of color pixels that can be statistically mapped to a certain descriptions. Unless you specifically train a neural network

about a picture, it won't be able to make the connection between the different objects it sees in a photo. Even when trained, the network will only have a statistical model that will probably label any picture that has a lot of grass, several people and tablecloths as a "family picnic."

It won't know what a picnic is contextually. Accordingly, it might mistakenly classify a picture of a poor family with sad looks and sooty faces eating in the outdoors as a happy family picnic. And it probably won't be able to tell the following picture is a drawing of an animal picnic. "True computer vision can only be achieved when we crack the code of general artificial intelligence and machine learning that has the abstract and commonsense capabilities of the human mind. We don't know when or if that will ever happen. Until then, or until we find some other way to represent concepts in a way that can also leverage the strengths of neural networks, we'll have to throw more and more data at our computer vision algorithms, hoping that we can account for every possible type of object and context they should be able to recognize. Before the advent of machine learning, the tasks that computer vision could perform were very limited and required a lot of manual coding and effort by developers and human operators. Computer vision has flagged as abnormal. This will help us to save tons of storage space in cloud and computer vision at the edge on the security camera itself.

2.10 WHAT ARE THE CATEGORIES OF MACHINE LEARNING?

Machine learning is a large field of study that overlaps with and inherits ideas from many related fields such as artificial intelligence. The focus of the field is learning, that is, acquiring skills or knowledge from experience. Most commonly, this means synthesizing useful concepts from historical data. As such, there are many different types of learning that you may encounter as a practitioner in the field of machine learning: from whole fields of study to specific techniques. At a high-level, machine learning is simply the study of teaching a computer program or algorithm how to progressively improve upon a set task that it is given. On the research-side of things, machine learning can be viewed through the lens of theoretical and mathematical modeling of how this process works. However, more practically it is the study of how to build applications that exhibit this iterative improvement. There are many ways to frame this idea, but largely there are three major recognized categories:

-  Supervised learning
-  Unsupervised learning
-  Reinforcement learning

In a world saturated by artificial intelligence, machine learning, and over-zealous talk about both, it is interesting to learn to understand and identify the types of machine learning we may encounter. For the average computer user, this can take the form of understanding the types of machine learning and how they may exhibit themselves in applications we use. And for the

practitioners creating these applications, it's essential to know the types of machine learning so that for any given task you may encounter, you can craft the proper learning environment and understand why, what you did worked.

2.10.1 Supervised Learning

Supervised learning is the most popular model for machine learning. It is the easiest to understand and the simplest to implement. It is very similar to teaching a child with the use of flash cards. Given data in the form of examples with labels, we can feed a learning algorithm these example-label pairs one by one, allowing the algorithm to predict the label for each example, and giving it feedback as to whether it predicted the right answer or not. Over time, the algorithm will learn to approximate the exact nature of the relationship between examples and their labels. When fully-trained, the supervised learning algorithm will be able to observe a new, never-before-seen example and predict a good label for it. Supervised learning is often described as task-oriented because of this. It is highly focused on a singular task, feeding more and more examples to the algorithm until it can accurately perform on that task.

Supervised learning is a type of system in which both input and desired output data are provided. Input and output data are labelled for classification to provide a learning basis for future data processing. The term supervised learning comes from the idea that an algorithm is learning from a training dataset, which can be thought of as the teacher. Supervised machine learning systems provide the learning algorithms with known quantities to support future judgments. Chatbots, self-driving cars, facial recognition programs, expert systems and robots are among the systems that may use either supervised or unsupervised learning. Supervised learning systems are mostly associated with retrieval-based AI but they may also be capable of using a generative learning model. In general, supervised learning occurs when a system is given input and output variables with the intentions of learning how they are mapped together, or related.

The goal is to produce an accurate enough mapping functions that when new input is given, the algorithm can predict the output. This is an iterative process, and each time the algorithm makes a prediction, it is corrected or given feedback until it achieves an acceptable level of performance. Training data for supervised learning includes a set of examples with paired input subjects and desired output (which is also referred to as the supervisory signal). For example, in an application of supervised learning for image processing, an AI system might be provided with labeled pictures of vehicles in categories such as cars or trucks. After a sufficient amount of observation, the system should be able to distinguish between and categorize unlabeled images, at which time the training is complete. Applications of supervised learning are typically broken down into two categories, classification and regression.

Classification is similar to the example above, when the output value is a category such as car or truck and true or false. A regression problem is when the output is a real, computed value such as the price or weight. A supervised learning algorithm learns from labeled training data, helps you to predict outcomes for unforeseen data. Successfully building, scaling, and deploying accurate supervised machine learning models takes time and technical expertise from a team of highly skilled data scientists. Moreover, Data scientist must rebuild models to make sure the insights given remains true until its data changes. The majority of practical machine learning practices supervised learning. Mathematically, supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm to learn the mapping function from the input to the output.

$$Y = f(X)$$

The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data. It is called supervised learning because the process of algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. This is the learning type that you will most likely encounter, as it is exhibited in many of the following common applications:

- 📌 **Advertisement Popularity:** Selecting advertisements that will perform well is often a supervised learning task. Many of the ads you see as you browse the internet are placed there because a learning algorithm said that they were of reasonable popularity. Furthermore, its placement associated on a certain site or with a certain query is largely due to a learned algorithm saying that the matching between ad and placement will be effective.
- 📌 **Spam Classification:** If you use a modern email system, chances are you've encountered a spam filter. That spam filter is a supervised learning system. Fed email examples and labels (spam/not spam), these systems learn how to preemptively filter out malicious emails so that their user is not harassed by them. Many of these also behave in such a way that a user can provide new labels to the system and it can learn user preference.
- 📌 **Face Recognition:** Do you use Facebook? Most likely your face has been used in a supervised learning algorithm that is trained to recognize your face. Having a system that takes a photo, finds faces and guesses who that is in the photo (suggesting a tag) is a supervised process. It has multiple layers to it, finding faces and then identifying them, but is still supervised nonetheless.

Supervised learning with the concept of function approximation, where basically we train an algorithm and in the end of the process we pick the function that best describes the input data,

the one that for a given X makes the best estimation of y ($X \rightarrow y$). Most of the times we are not able to figure out the true function that always make the correct predictions and other reason is that the algorithm relies upon an assumption made by humans about how the computer should learn and these assumptions introduce a bias. Here the human experts' acts as the teacher where we feed the computer with training data containing the input/predictors and we show it the correct answers (output) and from the data the computer should be able to learn the patterns.

Supervised learning algorithms try to model relationships and dependencies between the target prediction output and the input features such that we can predict the output values for new data based on those relationships which it learned from the previous data sets. Supervised learning is analogous to training a child to walk. You will hold the child's hand, show him how to take his foot forward, walk yourself for a demonstration and so on, until the child learns to walk on his own. We know the correct answers; the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance. The different types of supervised learning are;

Regression: Similarly, in the case of supervised learning, you give concrete known examples to the computer. You say that for given feature value x_1 the output is y_1 , for x_2 it is y_2 , for x_3 it is y_3 , and so on. Based on this data, you let the computer figure out an empirical relationship between x and y . Once the machine is trained in this way with a sufficient number of data points, now you would ask the machine to predict Y for a given X .

Assuming that you know the real value of Y for this given X , you will be able to deduce whether the machine's prediction is correct. Thus, you will test whether the machine has learned by using the known test data. Once you are satisfied that the machine is able to do the predictions with a desired level of accuracy (say 80 to 90%) you can stop further training the machine. Now, you can safely use the machine to do the predictions on unknown data points, or ask the machine to predict Y for a given X for which you do not know the real value of Y . This training comes under the regression that we talked about earlier.

Classification: Machine learning techniques can be used for classification problems. In classification problems, you classify objects of similar nature into a single group. For example, in a set of 100 students say, you may like to group them into three groups based on their heights - short, medium and long. Measuring the height of each student, you will place them in a proper group. Now, when a new student comes in, you will put him in an appropriate group by measuring his height.

By following the principles in regression training, you will train the machine to classify a student based on his feature, the height. When the machine learns how the groups are formed, it will be able to classify any unknown new student correctly. Once again, you would use the

test data to verify that the machine has learned your technique of classification before putting the developed model in production. Supervised Learning is where the AI really began its journey. This technique was applied successfully in several cases. You have used this model while doing the hand-written recognition on your machine. Several algorithms have been developed for supervised learning. In classification problems, we start with a data set containing training examples with associated correct labels.

List of Common Algorithms of Supervised Learning

When choosing a supervised learning algorithm, there are a few things that should be considered. The first is the bias and variance that exists within the algorithm as there is a fine line between being flexible enough and too flexible. Another is the complexity of the model or function that the system is trying to learn. Additionally, the heterogeneity, accuracy, redundancy and linearity of the data should be analyzed before choosing an algorithm.

- ✚ Nearest Neighbor
- ✚ Gaussian Naive Bayes
- ✚ Decision Trees
- ✚ Linear and Logistic Regression
- ✚ Support Vector Machines (SVM)
- ✚ Neural Networks
- ✚ Random Forest
- ✚ Linear Discriminant Analysis

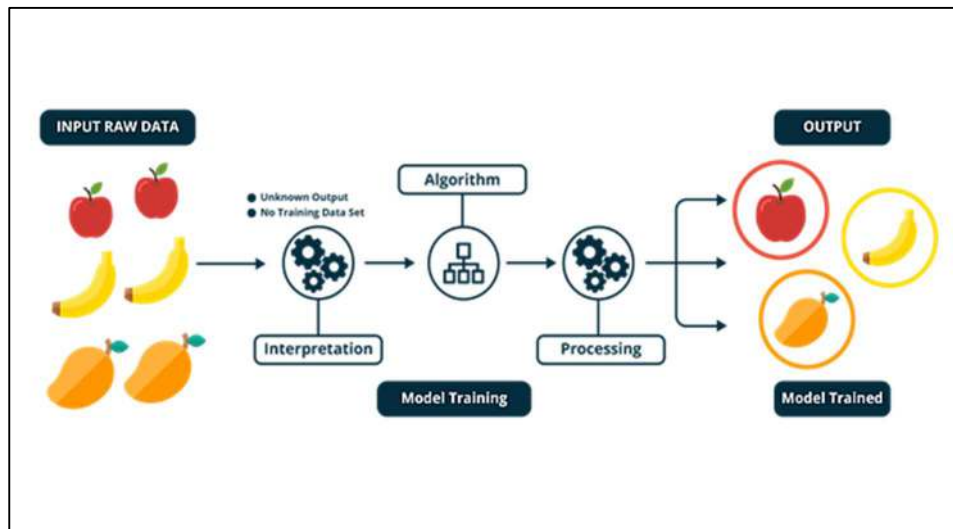


Fig. 2.6: Supervised Learning (Classification)

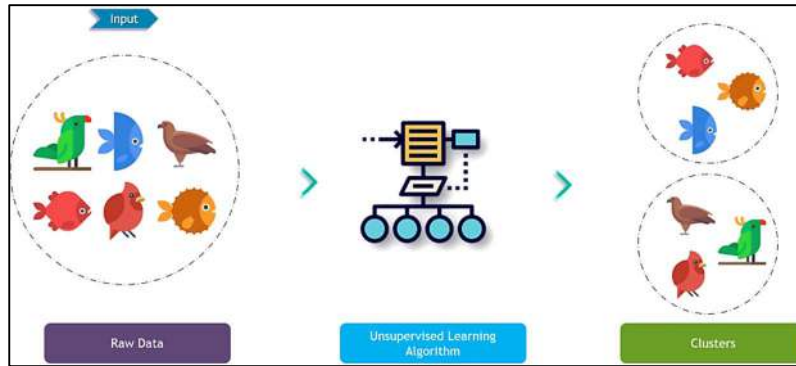


Fig. 2.7: Unsupervised Learning (Clustering)

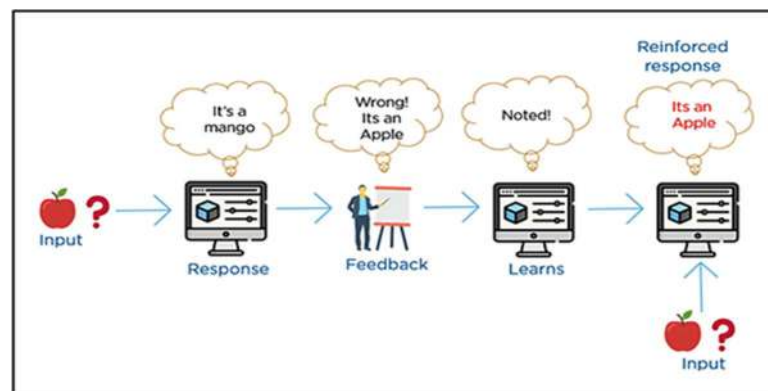


Fig. 2.8: Reinforcement Learning (Trial and Error Feedback)

2.10.2 Unsupervised Learning

Unsupervised learning is very much the opposite of supervised learning. It features no labels. Instead, our algorithm would be fed a lot of data and given the tools to understand the properties of the data. From there, it can learn to group, cluster, and/or organize the data in a way such that a human (or other intelligent algorithm) can come in and make sense of the newly organized data. What makes unsupervised learning such an interesting area is that an overwhelming majority of data in this world is unlabeled. Having intelligent algorithms that can take our terabytes and terabytes of unlabeled data and make sense of it is a huge source of potential profit for many industries.

That alone could help boost productivity in a number of fields. For example, what if we had a large database of every research paper ever published and we had unsupervised learning algorithms that knew how to group these in such a way so that you were always aware of the

current progression within a particular domain of research. Now, you begin to start a research project yourself, hooking your work into this network that the algorithm can see. As you write your work up and take notes, the algorithm makes suggestions to you about related works, works you may wish to cite, and works that may even help you push that domain of research forward. With such a tool, your productivity can be extremely boosted. Because unsupervised learning is based upon the data and its properties, we can say that unsupervised learning is data-driven. The results from an unsupervised learning task are controlled by the data and the way it's formatted.

Unsupervised learning is the training of an artificial intelligence algorithm using information that is neither classified nor labeled and allowing the algorithm to act on that information without guidance. In unsupervised learning, an AI system may group unsorted information according to similarities and differences even though there are no categories provided. AI systems capable of unsupervised learning are often associated with generative learning models, although they may also use a retrieval-based approach (which is most often associated with supervised learning). Chatbots, self-driving cars, facial recognition programs, expert systems and robots are among the systems that might use either supervised or unsupervised learning approaches.

In unsupervised learning, an AI system is presented with unlabeled, uncategorized data and the system's algorithms act on the data without prior training. The output is dependent upon the coded algorithms. Subjecting a system to unsupervised learning is one way of testing AI. Unsupervised learning algorithms can perform more complex processing tasks than supervised learning systems. However, unsupervised learning can be more unpredictable than the alternate model. While an unsupervised learning AI system might, for example, figure out on its own how to sort cats from dogs, it might also add unforeseen and undesired categories to deal with unusual breeds, creating clutter instead of order.

“Performance is often subjective and domain-specific.

What do you do when your dataset doesn't have any labels? Unsupervised learning is a group of machine learning algorithms and approaches that work with this kind of “no-ground-truth” data. Unsupervised learning is a machine learning technique, where you do not need to supervise the model. Instead, you need to allow the model to work on its own to discover information. It mainly deals with the unlabelled data. Unsupervised learning algorithms allow you to perform more complex processing tasks compared to supervised learning. Unsupervised learning can be more unpredictable compared with other natural learning methods. Here, are prime reasons for using Unsupervised Learning:



Unsupervised machine learning finds all kind of unknown patterns in data.

- ✚ Unsupervised methods help you to find features which can be useful for categorization.
- ✚ It is taken place in real time, so all the input data to be analyzed and labeled in the presence of learners.
- ✚ It is easier to get unlabeled data from a computer than labeled data, which needs manual intervention.

The computer is trained with unlabeled data. Here there's no teacher at all, actually the computer might be able to teach you new things after it learns patterns in data, these algorithms are particularly useful in cases where the human expert doesn't know what to look for in the data. Unsupervised learning is the family of machine learning algorithms which are mainly used in pattern detection and descriptive modeling. However, there are no output categories or labels here based on which the algorithm can try to model relationships. These algorithms try to use techniques on the input data to mine for rules, detect patterns, and summarize and group the data points which help in deriving meaningful insights and describe the data better to the users. The two unsupervised learning tasks usually explored are clustering the data into groups by similarity and reducing dimensionality to compress the data while maintaining its structure and usefulness. Some areas you might see unsupervised learning crop up are:

- ✚ **Recommender Systems:** If you've ever used YouTube or Netflix, you've most likely encountered a video recommendation system. These systems are often times placed in the unsupervised domain. We know things about videos, maybe their length, their genre, etc. We also know the watch history of many users. Taking into account users that have watched similar videos as you and then enjoyed other videos that you have yet to see, a recommender system can see this relationship in the data and prompt you with such a suggestion.
- ✚ **Buying Habits:** It is likely that your buying habits are contained in a database somewhere and that data is being bought and sold actively at this time. These buying habits can be used in unsupervised learning algorithms to group customers into similar purchasing segments. This helps companies' market to these grouped segments and can even resemble recommender systems.
- ✚ **Grouping User Logs:** Less user facing, but still very relevant, we can use unsupervised learning to group user logs and issues. This can help companies identify central themes to issues their customers face and rectify these issues, through improving a product or designing an FAQ to handle common issues. Either way, it is something that is actively done and if you've ever submitted an issue with a product or submitted a bug report, it is likely that it was fed to an unsupervised learning algorithm to cluster it with other similar issues.

In unsupervised learning, we do not specify a target variable to the machine; rather we ask machine “What can you tell me about X?” More specifically, we may ask questions such as given a huge data set X, “What are the five best groups we can make out of X?” or “What features occur together most frequently in X?” To arrive at the answers to such questions, you can understand that the number of data points that the machine would require to deduce a strategy would be very large. In case of supervised learning, the machine can be trained with even about few thousands of data points. However, in case of unsupervised learning, the number of data points that is reasonably accepted for learning starts in a few millions. These days, the data is generally abundantly available.

The data ideally requires curating. However, the amount of data that is continuously flowing in a social area network, in most cases data curation is an impossible task. How do you find the underlying structure of a dataset? How do you summarize it and group it most usefully? How do you effectively represent data in a compressed format? These are the goals of unsupervised learning, which is called “unsupervised” because you start with unlabeled data (there’s no Y). In contrast to supervised learning, it’s not always easy to come up with metrics for how well an unsupervised learning algorithm is doing. The common disadvantages of using unsupervised learning are;

- ✚ You cannot get precise information regarding data sorting, and the output as data used in unsupervised learning is labeled and not known.
- ✚ Less accuracy of the results is because the input data is not known and not labeled by people in advance. This means that the machine requires doing this itself.
- ✚ The spectral classes do not always relate to informational classes.
- ✚ The user needs to spend time interpreting and label the classes which follow that classification.
- ✚ Spectral properties of classes can also change over time so you can't have the same class information while moving from one image to another.

List of Common Algorithms of Unsupervised Learning

- ✚ K-means Clustering
- ✚ Association Rules
- ✚ Cluster Identification
- ✚ Density Estimation
- ✚ Principal Component Analysis
- ✚ Hierarchical clustering
- ✚ Singular Value Decomposition
- ✚ Independent Component Analysis

-  K-NN (k nearest neighbors)
-  Dendrogram
-  Agglomerative clustering

2.10.3 Reinforcement Learning

Reinforcement Learning is defined as a Machine Learning method that is concerned with how software agents should take actions in an environment. Reinforcement Learning is a part of the deep learning method that helps you to maximize some portion of the cumulative reward. This neural network learning method helps you to learn how to attain a complex objective or maximize a specific dimension over many steps. Reinforcement Learning enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experiences. Though both supervised and reinforcement learning use mapping between input and output, unlike supervised learning where feedback provided to the agent is correct set of actions for performing a task, reinforcement learning uses rewards and punishment as signals for positive and negative behavior. As compared to unsupervised learning, reinforcement learning is different in terms of goals.

While the goal in unsupervised learning is to find similarities and differences between data points, in reinforcement learning the goal is to find a suitable action model that would maximize the total cumulative reward of the agent. The figure below represents the basic idea and elements involved in a reinforcement learning model. Reinforcement learning is fairly different when compared to supervised and unsupervised learning. Where we can easily see the relationship between supervised and unsupervised (the presence or absence of labels), the relationship to reinforcement learning is a bit murkier. Some people try to tie reinforcement learning closer to the two by describing it as a type of learning that relies on a time-dependent sequence of labels; however, my opinion is that that simply makes things more confusing. We prefer to look at reinforcement learning as learning from mistakes. Place a reinforcement learning algorithm into any environment and it will make a lot of mistakes in the beginning.

So long as we provide some sort of signal to the algorithm that associates good behaviors with a positive signal and bad behaviors with a negative one, we can reinforce our algorithm to prefer good behaviors over bad ones. Over time, our learning algorithm learns to make fewer mistakes than it used to. Reinforcement learning is very behavior driven. It has influences from the fields of neuroscience and psychology. If you've heard of Pavlov's dog, then you may already be familiar with the idea of reinforcing an agent, albeit a biological one. However, to truly understand reinforcement learning, let's break down a concrete example. Let's look at teaching an agent to play the game Mario. For any reinforcement learning problem, we need an agent and an environment as well as a way to connect the two through a feedback loop.

To connect the agent to the environment, we give it a set of actions that it can take that affect the environment. To connect the environment to the agent, we have it continually issue two signals to the agent: an updated state and a reward (our reinforcement signal for behavior). In the game of Mario, our agent is our learning algorithm and our environment is the game (most likely a specific level). Our agent has a set of actions. These will be our button states. Our updated state will be each game frame as time passes and our reward signal will be the change in score. So long as we connect all these components together, we will have set up a reinforcement learning scenario to play the game Mario.

While neural networks are responsible for recent advances in problems like computer vision, machine translation and time series prediction – they can also combine with reinforcement learning algorithms to create something astounding like AlphaGo. Reinforcement learning refers to goal-oriented algorithms, which learn how to attain a complex objective (goal) or maximize along a particular dimension over many steps; for example, maximize the points won in a game over many moves. They can start from a blank slate, and under the right conditions they achieve superhuman performance. Like a child incentivized by spankings and candy, these algorithms are penalized when they make the wrong decisions and rewarded when they make the right ones – this is reinforcement. Reinforcement algorithms that incorporate deep learning can beat world champions at the game of Go as well as human experts playing numerous Atari video games.

While that may sound trivial, it's a vast improvement over their previous activities, and the state of the art is progressing rapidly. Reinforcement learning solves the difficult problem of correlating immediate actions with the delayed returns they produce. Like humans, reinforcement learning algorithms sometimes have to wait a while to see the fruit of their decisions. They operate in a delayed return environment, where it can be difficult to understand which action leads to which outcome over many time steps. Reinforcement learning algorithms can be expected to perform better and better in more ambiguous, real-life environments while choosing from an arbitrary number of possible actions, rather than from the limited options of a video game. That is, with time we expect them to be valuable to achieve goals in the real world. Deep reinforcement learning can be applied in the simulations of real-world use cases to help businesses optimize how they build factories, staff call centers, set up warehouses and supply chains, and manage traffic flows. Where is reinforcement learning in the real world?

 **Video Games:** The most common places to look at reinforcement learning are in learning to play games. Google's reinforcement learning application, AlphaZero and AlphaGo which learned to play the game Go. Mario example is also a common example. Currently, we don't know any production-grade game that has a

reinforcement learning agent deployed as its game AI, but we can imagine that this will soon be an interesting option for game developers to employ.

 **Industrial Simulation:** In robotic applications (consider assembly lines), it is useful to have our machines learn to complete their tasks without having to hardcode their processes. This can be a cheaper and safer option; it can even be less prone to failure. We can also incentivize our machines to use less electricity, so as to save us money. More than that, we can start this all within a simulation so as to not waste money if we potentially break our machine.

 **Resource Management:** Reinforcement learning is good for navigating complex environments. It can handle the need to balance certain requirements. Take, for example, Google's data centers. They used reinforcement learning to balance the need to satisfy our power requirements, but do it as efficiently as possible, cutting major costs. How does this affect us and the average person? Cheaper data storage costs for us as well and less of an impact on the environment we all share.

Reinforcement Learning aims at using observations gathered from the interaction with the environment to take actions that would maximize the reward or minimize the risk. Reinforcement learning algorithm (called the agent) continuously learns from the environment in an iterative fashion. In the process, the agent learns from its experiences of the environment until it explores the full range of possible states. Reinforcement Learning is a type of Machine Learning, and thereby also a branch of Artificial Intelligence. It allows machines and software agents to inevitably determine the ideal behavior within a specific context, in order to maximize its performance. Simple reward feedback is required for the agent to learn its behavior; this is known as the reinforcement signal.

There are many diverse algorithms that tackle this issue. As a matter of fact, Reinforcement Learning is defined by a specific type of problem, and all its solutions are classed as Reinforcement Learning algorithms. In the problem, an agent is supposed decide the best action to select based on his current state. When this step is repeated, the problem is known as a Markov Decision Process. In order to produce intelligent programs (also called agents), reinforcement learning goes through the following steps:

-  Input state is observed by the agent.
-  Decision making function is used to make the agent perform an action.
-  After the action is performed, the agent receives reward or reinforcement from the environment.
-  The state-action pair information about the reward is stored.

List of Common Algorithms of Reinforcement Learning

- ✚ Q-Learning
- ✚ Temporal Difference (TD)
- ✚ Deep Adversarial Networks
- ✚ Markov Decision Process

Reinforcement learning algorithms have a different relationship to time than humans do. An algorithm can run through the same states over and over again while experimenting with different actions, until it can infer which actions are best from which states. Effectively, algorithms enjoy their very own Groundhog Day, where they start out as dumb jerks and slowly get wise. Since humans never experience Groundhog Day outside the movie, reinforcement learning algorithms have the potential to learn more, and better, than humans. Indeed, the true advantage of these algorithms over humans stems not so much from their inherent nature, but from their ability to live in parallel on many chips at once, to train night and day without fatigue, and therefore to learn more. An algorithm trained on the game of Go, such as AlphaGo, will have played many more games of Go than any human could hope to complete in 100 lifetimes. Reinforcement learning is no doubt a cutting-edge technology that has the potential to transform our world.

The categories of machine learning in a broad sense can be classified as; Learning Problems

- ✚ Supervised Learning
- ✚ Unsupervised Learning
- ✚ Reinforcement Learning

Hybrid Learning Problems

- ✚ Semi-Supervised Learning
- ✚ Self-Supervised Learning
- ✚ Multi-Instance Learning

Statistical Inference

- ✚ Inductive Learning
- ✚ Deductive Inference
- ✚ Transductive Learning

Learning Techniques

- ✚ Multi-Task Learning

-  Active Learning
-  Online Learning
-  Transfer Learning
-  Ensemble Learning

Since the perspective of this book is towards advanced computer vision, the author do not want to discuss in detail about applied machine learning.

2.10.4 In A Nutshell

Supervised Learning: Supervised learning describes a class of problem that involves using a model to learn a mapping between input examples and the target variable. Models are fit on training data comprised of inputs and outputs and used to make predictions on test sets where only the inputs are provided and the outputs from the model are compared to the withheld target variables and used to estimate the skill of the model. There are two main types of supervised learning problems: they are classification that involves predicting a class label and regression that involves predicting a numerical value. Both classification and regression problems may have one and more input variables and input variables may be any data type.

Unsupervised Learning: Unsupervised learning defines a class of problems that involves using a model to describe or extract relationships in data. Compared to supervised learning, unsupervised learning operates upon only the input data without outputs or target variables. As such, unsupervised learning does not have a teacher correcting the model, as in the case of supervised learning. There are many types of unsupervised learning, although there are two main problems that are often encountered by a practitioner: they are clustering that involves finding groups in the data and density estimation that involves summarizing the distribution of data. Clustering and density estimation may be performed to learn about the patterns in the data. Additional unsupervised methods may also be used, such as visualization that involves graphing or plotting data in different ways and projection methods that involves reducing the dimensionality of the data.

Reinforcement Learning: Reinforcement learning designates a class of problems where an agent operates in an environment and must learn to operate using feedback. The use of an environment means that there is no fixed training dataset, rather a goal or set of goals that an agent is required to achieve, actions they may perform, and feedback about performance toward the goal. It is similar to supervised learning in that the model has some response from which to learn, although the feedback may be delayed and statistically noisy, making it challenging for the agent or model to connect cause and effect. An example of a reinforcement problem is playing a game where the agent has the goal of getting a high score and can make moves in the game and received feedback in terms of punishments or rewards.

Semi-Supervised Learning: Semi-supervised learning is supervised learning where the training data contains very few labeled examples and a large number of unlabeled examples. The goal of a semi-supervised learning model is to make effective use of all of the available data, not just the labelled data like in supervised learning. Making effective use of unlabelled data may require the use of or inspiration from unsupervised methods such as clustering and density estimation. Once groups or patterns are exposed, supervised methods or ideas from supervised learning may be used to label the unlabeled examples or apply labels to unlabeled representations later used for prediction. It is common for many real-world supervised learning problems to be examples of semi-supervised learning problems given the expense or computational cost for labeling examples. For example, classifying photographs requires a dataset of photographs that have already been labeled by human operators. Many problems from the fields of computer vision (image data), natural language processing (text data), and automatic speech recognition (audio data) fall into this category and cannot be easily addressed using standard supervised learning methods.

Self-Supervised Learning: Self-supervised learning algorithms refers to an unsupervised learning problem that is framed as a supervised learning problem in order to apply supervised learning algorithms to solve it. Supervised learning algorithms are used to solve an alternate or pretext task, the result of which is a model or representation that can be used in the solution of the original (actual) modeling problem. A common example of self-supervised learning is computer vision where a corpus of unlabeled images is available and can be used to train a supervised model, such as making images grayscale and having a model predict a color representation (colorization) or removing blocks of the image and have a model predict the missing parts (inpainting). A general example of self-supervised learning algorithms is autoencoders. These are a type of neural network that is used to create a compact or compressed representation of an input sample. They achieve this via a model that has an encoder and a decoder element separated by a bottleneck that signifies the internal compact representation of the input.

These autoencoder models are trained by providing the input to the model as both input and the target output, requiring that the model reproduce the input by first encoding it to a compressed representation then decoding it back to the original. Once trained, the decoder is discarded and the encoder is used as needed to create compact representations of input. Although autoencoders are trained using a supervised learning method, they solve an unsupervised learning problem, namely, they are a type of projection method for reducing the dimensionality of input data. Another example of self-supervised learning is generative adversarial networks, or GANs. These are generative models that are most commonly used for creating synthetic photographs using only a collection of unlabeled examples from the target domain. GAN models are trained indirectly via a separate discriminator model that classifies

examples of photos from the domain as real or fake (generated), the result of which is fed back to update the GAN model and encourage it to generate more realistic photos on the next iteration.

Multi-Instance Learning: Multi-instance learning is a supervised learning problem where individual examples are unlabeled; instead, bags or groups of samples are labeled. In multi-instance learning, an entire collection of examples is labeled as containing or not containing an example of a class, but the individual members of the collection are not labeled. Instances are in “bags” rather than set because a given instance may be present one or more times, e.g. duplicates. Modeling involves using knowledge that one or some of the instances in a bag are associated with a target label, and to predict the label for new bags in the future given their composition of multiple unlabeled examples. In supervised multi-instance learning, a class label is associated with each bag, and the goal of learning is to determine how the class can be inferred from the instances that make up the bag. Simple methods, such as assigning class labels to individual instances and using standard supervised learning, often work as a good first step.

Inductive Learning: Inductive learning encompasses using evidence to determine the outcome. Inductive reasoning refers to using specific cases to determine general outcomes, e.g. specific to general. Most machine learning models learn using a type of inductive inference or inductive reasoning where general rules (the model) are learned from specific historical examples (the data). The problem of induction is the problem of how to draw general conclusions about the future from specific observations from the past. Fitting a machine learning model is a process of induction. The model is a generalization of the specific examples in the training dataset. A model or hypothesis is made about the problem using the training data, and it is believed to hold over new unseen data later when the model is used. Lacking any further information, our assumption is that the best hypothesis regarding unseen instances is the hypothesis that best fits the observed training data. This is the fundamental assumption of inductive learning.

Deductive Inference: Inference refers to reaching an outcome or decision. In machine learning, fitting a model and making a prediction are both types of inference. There are different paradigms for inference that may be used as a framework for understanding how some machine learning algorithms work or how some learning problems may be approached. Some examples of approaches to learning are inductive, deductive, and transductive learning and inference. Deduction or deductive inference refers to using general rules to determine specific outcomes. We can better understand induction by contrasting it with deduction. Deduction is the reverse of induction. If induction is going from the specific to the general, deduction is going from the general to the specific. The simple observation that induction is just the inverse of deduction! Deduction is a top-down type of reasoning that seeks for all premises to be met before

determining the conclusion, whereas induction is a bottom-up type of reasoning that uses available data as evidence for an outcome. In the context of machine learning, once we use induction to fit a model on a training dataset, the model can be used to make predictions. The use of the model is a type of deduction or deductive inference.

Transductive Learning: Transduction or transductive learning is used in the field of statistical learning theory to refer to predicting specific examples given specific examples from a domain. It is different from induction that involves learning general rules from specific examples, e.g. specific to specific. Induction is deriving the function from the given data. Deduction is deriving the values of the given function for points of interest. Transduction is deriving the values of the unknown function for points of interest from the given data. Unlike induction, no generalization is required; instead, specific examples are used directly. This may, in fact, be a simpler problem than induction to solve. The model of estimating the value of a function at a given point of interest describes a new concept of inference: moving from the particular to the particular. We call this type of inference transductive inference. Note that this concept of inference appears when one would like to get the best result from a restricted amount of data. A classic example of a transductive algorithm is the k-Nearest Neighbors algorithm that does not model the training data, but instead uses it directly each time a prediction is required.

Multi-Task Learning: Multi-task learning is a type of supervised learning that involves fitting a model on one dataset that addresses multiple related problems. It involves devising a model that can be trained on multiple related tasks in such a way that the performance of the model is improved by training across the tasks as compared to being trained on any single task. Multi-task learning is a way to rally generalization by pooling the examples (which can be seen as soft constraints imposed on the parameters) arising out of several tasks. Multi-task learning can be a useful approach to problem-solving when there is an abundance of input data labeled for one task that can be shared with another task with much less labeled data. We may want to learn multiple related models at the same time, which is known as multi-task learning. For example, it is common for a multi-task learning problem to involve the same input patterns that may be used for multiple different outputs or supervised learning problems.

In this setup, each output may be predicted by a diverse part of the model, allowing the core of the model to generalize across each task for the same inputs. In the same way that additional training examples put more pressure on the parameters of the model towards values that generalize well, when part of a model is shared across tasks, that part of the model is more constrained towards good values (assuming the sharing is justified), often yielding better generalization. A popular example of multi-task learning is where the same word embedding is used to learn a distributed representation of words in text that is then shared across multiple different natural language processing supervised learning tasks.

Active Learning: Active learning is a technique where the model is able to query a human user operator during the learning process in order to resolve opacity during the learning process. The learner adaptively or interactively collects training examples, typically by querying an oracle to request labels for new points. Active learning is a type of supervised learning and seeks to achieve the same or better performance of so-called “passive” supervised learning, although by being more efficient about what data is collected or used by the model. The key idea behind active learning is that a machine learning algorithm can achieve greater accuracy with fewer training labels if it is allowed to choose the data from which it learns. An active learner may pose queries, usually in the form of unlabeled data instances to be labeled by an oracle. It is not unreasonable to view active learning as an approach to solving semi-supervised learning problems, or an alternative paradigm for the same types of problems.

We see that active learning and semi-supervised learning attack the same problem from opposite directions. While semi-supervised methods adventure what the learner thinks it knows about the unlabeled data, active methods attempt to explore the unknown aspects. It is thus natural to think about combining the two. Active learning is a useful approach when there is not much data available and new data is expensive to collect or label. The active learning process allows the sampling of the domain to be directed in a way that minimizes the number of samples and maximizes the effectiveness of the model. Active learning is often used in applications where labels are expensive to obtain, for example computational biology applications.

Online Learning: Online learning involves using the data available and updating the model directly before a prediction is required or after the last observation was made. Online learning is appropriate for those problems where observations are provided over time and where the probability distribution of observations is expected to also change over time. Hence, the model is expected to change just as frequently in order to capture and harness those changes. Traditionally machine learning is performed offline, which means we have a batch of data, and we optimize an equation. However, if we have streaming data, we need to perform online learning, so we can update our estimates as each new data point arrives rather than waiting until “the end” (which may never occur). This approach is also used by algorithms where there may be more observations than can reasonably fit into memory; therefore, learning is performed incrementally over observations, such as a stream of data. Online learning is helpful when the data may be changing rapidly over time.

It is also useful for applications that involve a large collection of data that is constantly growing, even if changes are gradual. Generally, online learning seeks to minimize “regret,” which is how well the model performed compared to how well it might have performed if all the available information was available as a batch. In the theoretical machine learning community,

the objective used in online learning is the regret, which the averaged loss is incurred relative to the best we could have gotten in hindsight using a single fixed parameter value. One example of online learning is so-called stochastic or online gradient descent used to fit an artificial neural network. The fact that stochastic gradient descent minimizes generalization error is easiest to see in the online learning case, where examples or minibatches are drawn from a stream of data.

Transfer Learning: Transfer learning is a type of learning where a model is first trained on one task, and then some or the entire model is used as the starting point for a related task. In transfer learning, the learner must perform two or more different tasks, but we assume that many of the factors that explain the variations in P1 are relevant to the variations that need to be captured for learning P2. It is a useful approach on problems where there is a task related to the main task of interest and the related task has a large amount of data. It is different from multi-task learning as the tasks are learned sequentially in transfer learning, whereas multi-task learning seeks good performance on all considered tasks by a single model at the same time in parallel. Pretrain a deep convolutional net with 8 layers of weights on a set of tasks (a subset of the 1000 ImageNet object classes) and then initialize a same-size network with the first k layers of the first net. All the layers of the second network (with the upper layers initialized randomly) are then jointly trained to perform a different set of tasks (another subset of the 1000 ImageNet object classes), with fewer training examples than for the first set of tasks.

An example is image classification, where a predictive model, such as an artificial neural network, can be trained on a large corpus of general images, and the weights of the model can be used as a starting point when training on a smaller more specific dataset, such as dogs and cats. The features already learned by the model on the broader task, such as extracting lines and patterns, will be helpful on the new related task. If there is significantly more data in the first setting (sampled from P1), then that may help to learn representations that are useful to quickly generalize from only very few examples drawn from P2. Many visual categories share low-level notions of edges and visual shapes, the effects of geometric changes, changes in lighting, etc. As noted, transfer learning is particularly useful with models that are incrementally trained and an existing model can be used as a starting point for continued training, such as deep learning networks.

Ensemble Learning: Ensemble learning is an approach where two or more models are fit on the same data and the predictions from each model are combined. The field of ensemble learning runs many ways of combining the ensemble members' predictions, including uniform weighting and weights chosen on a validation set. The objective of ensemble learning is to achieve better performance with the ensemble of models as compared to any individual model. This involves both deciding how to create models used in the ensemble and how to best

combine the predictions from the ensemble members. Ensemble learning can be broken down into two tasks: developing a population of base learners from the training data, and then combining them to form the composite predictor. Ensemble learning is a useful approach for improving the predictive skill on a problem domain and to reduce the variance of stochastic learning algorithms, such as artificial neural networks. Some examples of popular ensemble learning algorithms include: weighted average, stacked generalization, bootstrap aggregation.

2.11 WHAT ARE MACHINE LEARNING ALGORITHMS?

Machine learning algorithms are programs (math and logic) that regulate themselves to perform better as they are exposed to more data. The “learning” part of machine learning means that those programs change how they process data over time, much as humans change how they process data by learning. So a machine-learning algorithm is a program with a specific way to adjusting its own parameters, given feedback on its previous performance making predictions about a dataset. We are probably living in the most defining period of human history. The period when computing moved from large mainframes to PCs to cloud. But what makes it defining is not what has happened, but what is coming our way in years to come. What makes this period exciting and enthralling is the democratization of the various tools and techniques, which followed the boost in computing.

Human race has stepped into the era of Intelligent Revolution. This wave of Intelligent Revolution is going to be powered by Artificial Intelligence (AI) and Machine Learning (ML). Automating paper-based processes are soon becoming history as Artificial Intelligence and Machine Learning have started opening doors to new opportunities enabling computers to tackle tasks that have until now been carried out only by people. Even though the hype or the buzz around the words ‘Machine Learning’ and ‘Artificial Intelligence’ has increased pervasively in the past few years, the first historic timeline for Artificial Intelligence came into existence during ‘World War II’ when the Computer Scientist Alan Turing created the Bombe machine to crack the impossible German force’s Enigma Code. Bombe was ‘intelligent’ and was able to learn, and eventually it cracked the code.

In a way, that machine has laid the foundation of what Machine Learning and Artificial Intelligence are today. Now that we have access to a huge amount of data, Machine Learning researchers are constantly working on making use of this data in order to push the boundaries of computer intelligence. From driving cars to translating speeches, from impacting business growth to detecting fraud attacks, Machine Learning is driving an explosion in the capabilities of Artificial Intelligence helping software make sense of the unpredictable real world. Even though there are various Machine Learning algorithms, people still get confused about their choice of algorithm.

In this section, we will cook up the brief introduction of different Machine Learning algorithms that are commonly used for computer vision applications. In a world where nearly all manual tasks are being automated, the definition of manual is changing. Machine Learning algorithms can help computers play chess, perform surgeries, and get smarter and more personal. We are living in an era of constant technological progress, and looking at how computing has advanced over the years, we can predict what's to come in the days ahead. One of the main features of this revolution that stands out is how computing tools and techniques have been democratized. In the past five years, data scientists have built sophisticated data-crunching machines by seamlessly executing advanced methods. The results have been amazing. Figure 2.9 illustrates the functional working of machine learning algorithms.

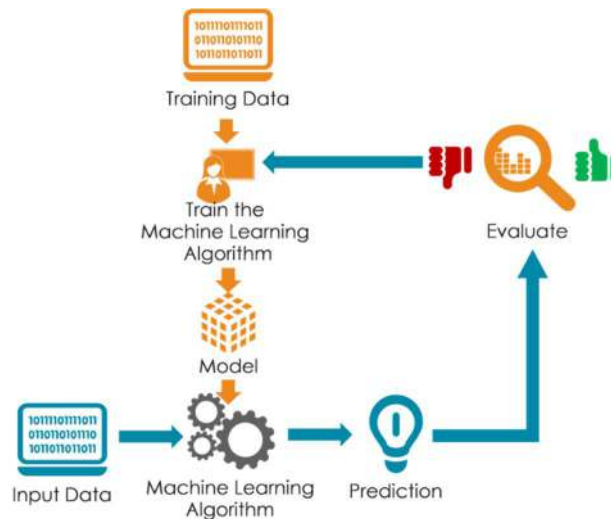


Fig. 2.9: A Machine Learning Algorithm

Machine learning is a class of methods for automatically creating models from data. Machine learning algorithms are the engines of machine learning, meaning it is the algorithms that turn a data set into a model. Which kind of algorithm works best (supervised, unsupervised, reinforcement, etc.) hinges on the kind of problem you're solving, the computing resources available, and the nature of the data? Let us instigate with the some of the most common machine learning algorithms, and explain how those algorithms relate to the other pieces of the puzzle of creating predictive models from historical data in computer vision applications.

2.11.1 Linear Regression

Linear regression is perhaps one of the most well-known and well agreed algorithms in statistics and machine learning. Machine learning, more specifically the field of predictive

modeling is primarily concerned with minimizing the error of a model or making the most accurate predictions possible, at the expense of explain ability. In applied machine learning we will borrow, reuse and steal algorithms from many different fields, including statistics and use them towards these ends. As such, linear regression was radical in the field of statistics and is studied as a model for understanding the relationship between input and output numerical variables, but has been borrowed by machine learning. It is both a statistical algorithm and a machine learning algorithm. When you start looking into linear regression, things can get very confusing. The reason is because linear regression has been around for so long (more than 200 years). It has been studied from every possible angle and often each angle has a new and different name.

Linear regression is a linear model, e.g. a model that assumes a linear relationship between the input variables (x) and the single output variable (y). More specifically, that y can be calculated from a linear combination of the input variables (x). When there is a single input variable (x), the method is referred to as simple linear regression. When there are multiple input variables, literature from statistics often refers to the method as multiple linear regression. Different techniques can be used to prepare or train the linear regression equation from data, the most common of which is called Ordinary Least Squares. It is common to therefore refer to a model prepared this way as Ordinary Least Squares Linear Regression or just Least Squares Regression.

Linear regression is an attractive model because the representation is so simple. The representation is a linear equation that combines a specific set of input values (x) the solution to which is the predicted output for that set of input values (y). As such, both the input values (x) and the output value are numeric. The linear equation assigns one scale factor to each input value or column, called a coefficient and represented by the capital Greek letter Beta (B). One additional coefficient is also added, giving the line an additional degree of freedom (e.g. moving up and down on a 2D) and is often called the intercept or the bias coefficient.

For example, in a simple regression problem (a single x and a single y), the form of the model would be:

$$y = B_0 + B_1 * x$$

In higher dimensions when we have more than one input (x), the line is called a plane or a hyper-plane. The representation therefore is the form of the equation and the specific values used for the coefficients (e.g. B₀ and B₁ in the above example). It is common to talk about the complexity of a regression model like linear regression. This refers to the number of coefficients used in the model. When a coefficient becomes zero, it effectively removes the influence of the input variable on the model and therefore from the prediction made from the

model ($0 * x = 0$). This becomes relevant if you look at regularization methods that change the learning algorithm to reduce the complexity of regression models by putting pressure on the absolute size of the coefficients, driving some to zero.

Now that we understand the representation used for a linear regression model, let's review some ways that we can learn this representation from data. Learning a linear regression model means estimating the values of the coefficients used in the representation with the data that we have available. We will take a brief look at four techniques to prepare a linear regression model. This is not enough information to implement them from scratch, but enough to get a flavor of the computation and trade-offs involved. There are many more techniques because the model is so well studied. Take note of Ordinary Least Squares because it is the most common method used in general. Also take note of Gradient Descent as it is the most common technique taught in machine learning classes.

Simple Linear Regression: With simple linear regression when we have a single input, we can use statistics to estimate the coefficients. This requires that you calculate statistical properties from the data such as means, standard deviations, correlations and covariance. All of the data must be available to traverse and calculate statistics. This is fun as an exercise in excel, but not really useful in practice.

Ordinary Least Squares: When we have more than one input we can use Ordinary Least Squares to estimate the values of the coefficients. This procedure seeks to minimize the sum of the squared residuals. This means that given a regression line through the data we calculate the distance from each data point to the regression line, square it, and sum all of the squared errors together. This is the quantity that ordinary least squares seek to minimize. This approach treats the data as a matrix and uses linear algebra operations to estimate the optimal values for the coefficients. It means that all of the data must be available and you must have enough memory to fit the data and perform matrix operations. It is unusual to implement the Ordinary Least Squares procedure yourself unless as an exercise in linear algebra. It is more likely that you will call a procedure in a linear algebra library. This procedure is very fast to calculate.

Gradient Descent: When there are one or more inputs you can use a process of optimizing the values of the coefficients by iteratively minimizing the error of the model on your training data. This operation is called Gradient Descent and works by starting with random values for each coefficient. The sum of the squared errors is calculated for each pair of input and output values. A learning rate is used as a scale factor and the coefficients are updated in the direction towards minimizing the error. The process is repeated until a minimum sum squared error is achieved or no further improvement is possible. When using this method, you must select a learning rate (alpha) parameter that determines the size of the improvement step to take on each iteration of

the procedure. Gradient descent is often taught using a linear regression model because it is relatively straightforward to understand. In practice, it is useful when you have a very large dataset either in the number of rows or the number of columns that may not fit into memory.

Regularization: There are extensions of the training of the linear model called regularization methods. These seek to both minimize the sum of the squared error of the model on the training data (using ordinary least squares) but also to reduce the complexity of the model (like the number or absolute size of the sum of all coefficients in the model). Two popular examples of regularization procedures for linear regression are:

📊 **Lasso Regression:** where Ordinary Least Squares is modified to also minimize the absolute sum of the coefficients (called L1 regularization).

📊 **Ridge Regression:** where Ordinary Least Squares is modified to also minimize the squared absolute sum of the coefficients (called L2 regularization).

These methods are effective to use when there is collinearity in your input values and ordinary least squares would overfit the training data. Now that you know some techniques to learn the coefficients in a linear regression model, let's look at how to make predictions on new data.

Making Predictions with Linear Regression: Given the representation is a linear equation, making predictions is as simple as solving the equation for a specific set of inputs. Let's make this concrete with an example. Imagine we are predicting weight (y) from height (x). Our linear regression model representation for this problem would be:

$$y = B_0 + B_1 * x_1$$

or

$$\text{weight} = B_0 + B_1 * \text{height}$$

Where B_0 is the bias coefficient and B_1 is the coefficient for the height column. We use a learning technique to find a good set of coefficient values. Once found, we can plug in different height values to predict the weight. For example, let's use $B_0 = 0.1$ and $B_1 = 0.5$. Let's plug them in and calculate the weight (in kilograms) for a person with the height of 182 centimeters.

$$\text{weight} = 0.1 + 0.5 * 182$$

$$\text{weight} = 91.1$$

You can see that the above equation could be plotted as a line in two-dimensions. The B_0 is our starting point regardless of what height we have. We can run through a bunch of heights

from 100 to 250 centimeters and plug them to the equation and get weight values, creating our line. Now that we know how to make predictions given a learned linear regression model, let's look at some rules of thumb for preparing our data to make the most of this type of model.

Preparing Data For Linear Regression: Linear regression is been used and studied at great length, and there is a lot of literature on how your data must be structured to make best use of the model. As such, there is a lot of sophistication when talking about these requirements and expectations which can be intimidating. In practice, you can use these rules more as rules of thumb when using Ordinary Least Squares Regression, the most common implementation of linear regression.

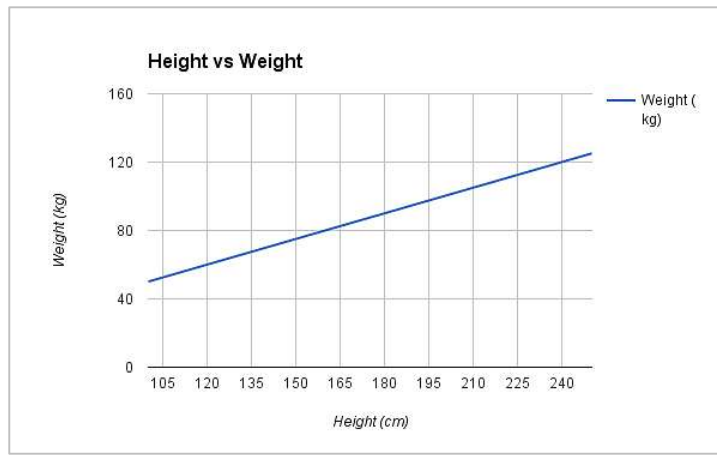


Fig. 2.10: Sample Height Versus Weight Linear Regression

Try different preparations of your data using these heuristics and see what works best for your problem.

-  **Linear Assumption:** Linear regression assumes that the relationship between your input and output is linear. It does not support anything else. This may be obvious, but it is good to remember when you have a lot of attributes. You may need to transform data to make the relationship linear (e.g. log transform for an exponential relationship).
-  **Remove Noise:** Linear regression assumes that your input and output variables are not noisy. Consider using data cleaning operations that let you better expose and clarify the signal in your data. This is most important for the output variable and you want to remove outliers in the output variable (y) if possible.
-  **Remove Collinearity:** Linear regression will over-fit your data when you have highly correlated input variables. Consider calculating pairwise correlations for your input data and removing the most correlated.

📊 **Gaussian Distributions:** Linear regression will make more reliable predictions if your input and output variables have a Gaussian distribution. You may get some benefit using transforms (e.g. log or BoxCox) on your variables to make their distribution more Gaussian looking.

📊 **Rescale Inputs:** Linear regression will often make more reliable predictions if you rescale input variables using standardization or normalization.

2.11.2 Logistic Regression

Logistic regression is another method borrowed by machine learning from the field of statistics. Logistic regression is named for the function used at the core of the method, the logistic function. The logistic function, also called the sigmoid function was developed by statisticians to describe properties of population growth in ecology, rising quickly and maxing out at the carrying capacity of the environment. It's an S-shaped curve that can take any real-valued number and map it into a value between 0 and 1, but never exactly at those limits.

$$1 / (1 + e^{-\text{value}})$$

Where e is the base of the natural logarithms (Euler's number or the EXP() function in your spreadsheet) and value is the actual numerical value that you want to transform. Below is a plot of the numbers between -5 and 5 transformed into the range 0 and 1 using the logistic function. Logistic regression uses an equation as the representation, very much like linear regression. Input values (x) are combined linearly using weights or coefficient values (referred to as the Greek capital letter Beta) to predict an output value (y). A key difference from linear regression is that the output value being modeled is binary values (0 or 1) rather than a numeric value.

Below is an example logistic regression equation:

$$y = e^{(b_0 + b_1 * x)} / (1 + e^{(b_0 + b_1 * x)})$$

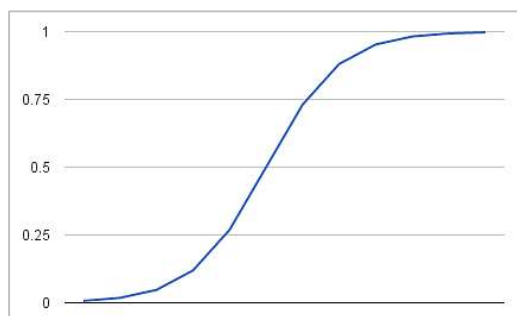


Fig. 2.11: Logistic Function

Where y is the predicted output, b_0 is the bias or intercept term and b_1 is the coefficient for the single input value (x). Each column in your input data has an associated b coefficient (a constant real value) that must be learned from your training data. The actual representation of the model that you would store in memory or in a file is the coefficients in the equation (the beta value or b 's).

Logistic Regression Predicts Probabilities: Logistic regression models the probability of the default class (e.g. the first class). For example, if we are modeling people's sex as male or female from their height, then the first class could be male and the logistic regression model could be written as the probability of male given a person's height, or more formally:

$$P(\text{sex}=\text{male} \mid \text{height})$$

Written another way, we are modeling the probability that an input (X) belongs to the default class ($Y=1$), we can write this formally as:

$$P(X) = P(Y=1 \mid X)$$

We're predicting probabilities? Note that the probability prediction must be transformed into binary values (0 or 1) in order to essentially make a probability prediction. Logistic regression is a linear method, but the predictions are transformed using the logistic function. The impact of this is that we can no longer understand the predictions as a linear combination of the inputs as we can with linear regression, for example, continuing on from above, the model can be stated as:

$$P(X) = e^{(b_0 + b_1 * X)} / (1 + e^{(b_0 + b_1 * X)})$$

We don't want to dive into the math too much, but we can turn around the above equation as follows (remember we can remove the e from one side by adding a natural logarithm ($\ln$) to the other):

$$\ln(P(X) / 1 - P(X)) = b_0 + b_1 * X$$

This is useful because we can see that the calculation of the output on the right is linear again (just like linear regression), and the input on the left is a log of the probability of the default class. This ratio on the left is called the odds of the default class (it's historical that we use odds, for example, odds are used in horse racing rather than probabilities). Odds are calculated as a ratio of the probability of the event divided by the probability of not the event, e.g. $0.8/(1-0.8)$ which has the odds of 4. So we could instead write:

$$\ln(\text{odds}) = b_0 + b_1 * X$$

Because the odds are log transformed, we call this left hand side the log-odds or the probit. It is possible to use other types of functions for the transform (which is out of scope, but as such it is common to refer to the transform that relates the linear regression equation to the probabilities as the link function, e.g. the probit link function. We can move the exponent back to the right and write it as:

$$\text{odds} = e^{(b_0 + b_1 * X)}$$

All of this helps us understand that indeed the model is still a linear combination of the inputs, but that this linear combination relates to the log-odds of the default class.

Learning the Logistic Regression Model: The coefficients of the logistic regression algorithm must be estimated from your training data. This is done using maximum-likelihood estimation. It is a learning algorithm used by a variety of machine learning algorithms, even though it does make assumptions about the distribution of your data (more on this when we talk about preparing your data). The best coefficients would result in a model that would predict a value very close to 1 (e.g. male) for the default class and a value very close to 0 (e.g. female) for the other class. The intuition for maximum-likelihood for logistic regression is that a search procedure seeks values for the coefficients (Beta values) that minimize the error in the probabilities predicted by the model to those in the data (e.g. probability of 1 if the data is the primary class). We are not going to go into the math of maximum likelihood. It is enough to say that a minimization algorithm is used to optimize the best values for the coefficients for your training data. This is often implemented in practice using efficient numerical optimization algorithm (like the Quasi-newton method). When you are learning logistic, you can implement it yourself from scratch using the much simpler gradient descent algorithm.

Making Predictions with Logistic Regression: Making predictions with a logistic regression model is as simple as plugging in numbers into the logistic regression equation and calculating a result. Let's make this concrete with a specific example. Let's say we have a model that can predict whether a person is male or female based on their height (completely fictitious). Given a height of 150cm is the person male or female. We have learned the coefficients of $b_0 = -100$ and $b_1 = 0.6$. Using the equation above we can calculate the probability of male given a height of 150cm or more formally $P(\text{male}|\text{height}=150)$. We will use $\text{EXP}()$ for e , because that is what you can use if you type this example into your spreadsheet:

$$y = e^{(b_0 + b_1 * X)} / (1 + e^{(b_0 + b_1 * X)})$$

$$y = \exp(-100 + 0.6 * 150) / (1 + \text{EXP}(-100 + 0.6 * X))$$

$$y = 0.0000453978687$$

Or a probability of near zero that the person is a male.

In practice we can use the probabilities directly. Because this is classification and we want a crisp answer, we can snap the probabilities to a binary class value, for example:

0 if $p(\text{male}) < 0.5$

1 if $p(\text{male}) \geq 0.5$

Now that we know how to make predictions using logistic regression, let's look at how we can prepare our data to get the most from the technique.

Prepare Data for Logistic Regression: The assumptions prepared by logistic regression about the distribution and relationships in your data are much the same as the assumptions made in linear regression. Much study has gone into defining these assumptions and precise probabilistic and statistical language is used. My advice is to use these as guidelines or rules of thumb and experiment with different data preparation schemes. Ultimately in predictive modeling machine learning projects you are laser focused on making accurate predictions rather than interpreting the results. As such, you can break some assumptions as long as the model is robust and performs well.

- ✚ **Binary Output Variable:** This might be obvious as we have already mentioned it, but logistic regression is intended for binary (two-class) classification problems. It will predict the probability of an instance belonging to the default class, which can be snapped into a 0 or 1 classification.
- ✚ **Remove Noise:** Logistic regression assumes no error in the output variable (y), consider removing outliers and possibly misclassified instances from your training data.
- ✚ **Gaussian Distribution:** Logistic regression is a linear algorithm (with a non-linear transform on output). It does assume a linear relationship between the input variables with the output. Data transforms of your input variables that better expose this linear relationship can result in a more accurate model. For example, you can use log, root, Box-Cox and other univariate transforms to better expose this relationship.
- ✚ **Remove Correlated Inputs:** Like linear regression, the model can overfit if you have multiple highly-correlated inputs. Consider calculating the pairwise correlations between all inputs and removing highly correlated inputs.
- ✚ **Fail to Converge:** It is probable for the expected likelihood estimation process that learns the coefficients to fail to converge. This can happen if there are many highly correlated inputs in your data or the data is very sparse (e.g. lots of zeros in your input data).

2.11.3 Decision Trees

A tree has many analogies in real life, and turns out that it has influenced a wide area of machine learning, covering both classification and regression. In decision analysis, a decision tree can be used to visually and explicitly represent decisions and decision making. As the name goes, it uses a tree-like model of decisions. Though a commonly used tool in data mining for deriving a strategy to reach a particular goal, it's also widely used in machine learning. How can an algorithm be represented as a tree? For this let's consider a very basic example that uses titanic data set for predicting whether a passenger will survive or not. Below model uses three features/attributes/columns from the data set, namely sex, age and sibsp (number of spouses or children along).

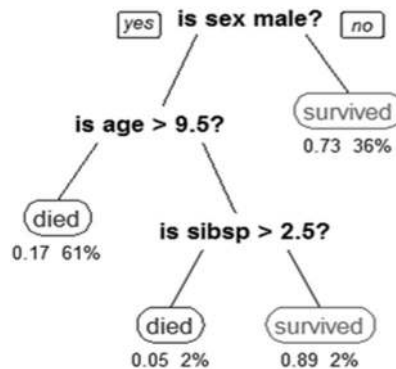


Fig. 2.12: Decision Tree for Predicting Survival of a Titanic Passenger

A decision tree is drawn upside down with its root at the top. In Figure 2.12, the bold text represents a condition/internal node, based on which the tree splits into branches/edges. The end of the branch that doesn't split anymore is the decision/leaf, in this case, whether the passenger died or survived. Although, a real dataset will have a lot more features and this will just be a branch in a much bigger tree, but you can't ignore the simplicity of this algorithm. The feature importance is clear and relations can be viewed easily. This methodology is more commonly known as learning decision tree from data and above tree is called Classification tree as the target is to classify passenger as survived or died.

Regression trees are represented in the same manner; just they predict continuous values like price of a house. In general, Decision Tree algorithms are referred to as Classification and Regression Trees. So, what is actually going on in the background? Growing a tree involves deciding on which features to choose and what conditions to use for splitting, along with knowing when to stop. As a tree generally grows arbitrarily, you will need to trim it down for it to look beautiful. Let's start with a common technique used for splitting.

Recursive Binary Splitting: In this method all the features are considered and different split points are tried and tested using a cost function. The split with the best cost (or lowest cost) is selected. Consider the earlier example of tree learned from titanic dataset. In the first split or the root, all attributes/features are considered and the training data is divided into groups based on this split. We have three features, so will have three candidate splits. Now we will calculate how much accuracy each split will cost us, using a function. The split that costs least is chosen, which in our example is sex of the passenger. This algorithm is recursive in nature as the groups formed can be sub-divided using same strategy. Due to this procedure, this algorithm is also known as the greedy algorithm, as we have an excessive desire of lowering the cost. This makes the root node as best predictor/ classifier.

Cost of a split: Let's come first with a closer look at cost functions salvaged for classification and regression. In both cases the cost functions try to find most homogeneous branches, or branches having groups with analogous retorts. This makes sense when a test data input will follow a definite path.

Regression : $\text{sum}(y - \text{prediction})^2$

Let's say, we are predicting the price of houses. Now the decision tree will start splitting by considering each feature in training data. The mean of responses of the training data inputs of particular group is considered as prediction for that group. The above function is applied to all data points and cost is calculated for all candidate splits. Again the split with lowest cost is chosen. Another cost function involves reduction of standard deviation, more about it can be found here.

Classification : $G = \text{sum}(pk * (1 - pk))$

A Gini score springs an idea of how good a split is by how mixed the response classes are in the groups fashioned by the split. Here, pk is proportion of same class inputs present in a particular group. A perfect class purity occurs when a group contains all inputs from the same class, in which case pk is either 1 or 0 and $G = 0$, where as a node having a 50–50 split of classes in a group has the worst purity, so for a binary classification it will have $pk = 0.5$ and $G = 0.5$.

When to stop splitting? You might ask when to stop growing a tree? As a problem usually has a large set of features, it results in large number of split, which in turn gives a huge tree. Such trees are complex and can lead to overfitting. So, we need to know when to stop? One way of doing this is to set a minimum number of training inputs to use on each leaf. For example we can use a minimum of 10 passengers to reach a decision (died or survived), and ignore any leaf

that takes less than 10 passengers. Another way is to set maximum depth of your model. Maximum depth refers to the length of the longest path from a root to a leaf.

Pruning: The performance of a tree can be further increased by pruning. It involves removing the branches that make use of features having low importance. This way, we reduce the complexity of tree, and thus increasing its predictive power by reducing overfitting. Pruning can start at either root or the leaves. The simplest method of pruning starts at leaves and removes each node with most popular class in that leaf, this change is kept if it doesn't deteriorate accuracy. It's also called reduced error pruning. More sophisticated pruning methods can be used such as cost complexity pruning where a learning parameter (alpha) is used to weigh whether nodes can be removed based on the size of the sub-tree. This is also known as weakest link pruning.

Advantages of Classification and Regression Trees

- ✚ Simple to understand, interpret, visualize.
- ✚ Decision trees implicitly perform variable screening or feature selection.
- ✚ Can handle both numerical and categorical data. Can also handle multi-output problems.
- ✚ Decision trees require relatively little effort from users for data preparation.
- ✚ Nonlinear relationships between parameters do not affect tree performance.

Disadvantages of Classification and Regression Trees

- ✚ Decision-tree learners can create over-complex trees that do not generalize the data well. This is called overfitting.
- ✚ Decision trees can be unstable because small variations in the data might result in a completely different tree being generated. This is called variance, which needs to be lowered by methods like bagging and boosting.
- ✚ Greedy algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees, where the features and samples are randomly sampled with replacement.
- ✚ Decision tree learners create biased trees if some classes dominate. It is therefore recommended to balance the data set prior to fitting with the decision tree.

An improvement over decision tree learning is made using technique of boosting. A popular library for implementing these algorithms is Scikit-Learn. It has a wonderful API that can get your model up and running with just a few lines of code in python. Dividing efficiently based on maximum information gain is the key to decision tree classifier. However, in real world with millions of data dividing into pure class is practically not feasible (it may take longer

training time) and so we stop at points in nodes of tree when fulfilled with certain parameters (for example impurity percentage). Therefore, Decision Tree repetitively divides the working area (plot) into sub part by identifying lines.

2.11.4 Random Forest

Random Forest is one of the most versatile machine learning algorithms available today. With its built-in ensembling capacity, the task of building a decent generalized model (on any dataset) gets much easier. In fact, the easiest part of machine learning is coding. Its capability to solve, both regression and classification problems along with robustness to correlated features and variable importance plot gives us enough head start to solve various problems. Random forest is a tree-based algorithm which involves building several trees (decision trees), then combining their output to improve generalization ability of the model. The method of combining trees is known as an ensemble method. Ensembling is nothing but a combination of weak learners (individual trees) to produce a strong learner.

Say, you want to watch a movie. But you are uncertain of its reviews. You ask 10 people who have watched the movie. 8 of them said “the movie is fantastic.” Since the majority is in favor, you decide to watch the movie. This is how we use ensemble techniques in our daily life too. Random Forest can be used to solve regression and classification problems. In regression problems, the dependent variable is continuous. In classification problems, the dependent variable is categorical. Random Forest is a learning method that operates by constructing multiple decision trees. The final decision is made based on the majority of the trees and is chosen by the random forest as shown in Figure 2.13. A decision tree is a tree-shaped diagram used to determine a course of action as shown in Figure 2.14. Each branch of the tree represents a possible decision, occurrence, or reaction.

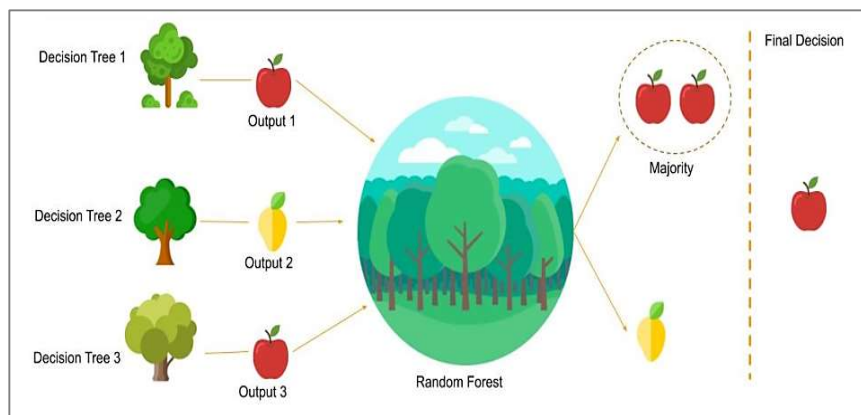


Fig. 2.13: Random Forest Algorithm

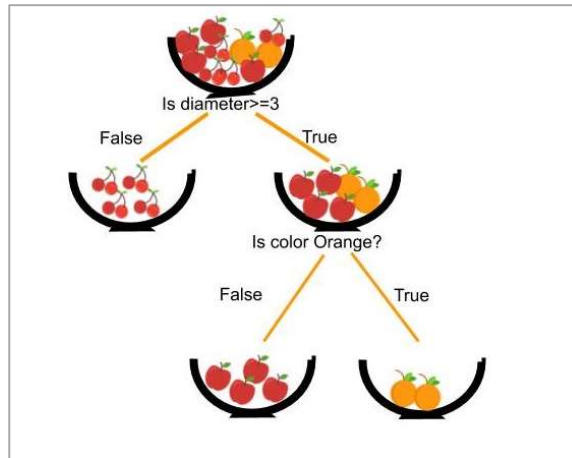


Fig. 2.14: A Decision Tree in Random Forest Algorithm

A few of the practices for random forests currently used in remote sensing consist of:

- ✚ The enhanced thematic mapper used on satellites, which see far outside the human spectrum for looking at land masses and acquiring mages of the Earth's surface.
- ✚ Object detection and multi-class object detection: for example, sorting out different vehicles, such as cars and buses, in traffic.
- ✚ Kinect, which uses random forest algorithms as part of game consoles by tracking body movements, and then recreating it in the game (Figure 2.15).

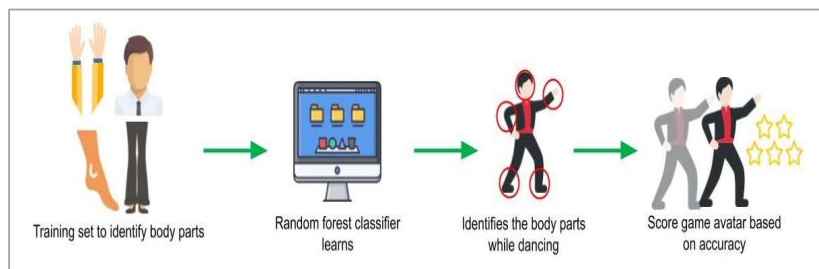


Fig. 2.15: Kinect using Random Forest Algorithm

There are a lot of benefits to using Random Forest, but one of the main advantages is that it reduces the risk of overfitting and the required training time. Additionally, it offers a high level of accuracy. Random Forest runs efficiently in large databases and produces highly accurate predictions by estimating missing data. There are diverse ways that Random Forest makes data decisions, and consequently, there are some important related terms to know. Some of these terms include:

- 📊 **Entropy:** It is a measure of randomness or unpredictability in the data set.
- 📊 **Information Gain:** A measure of the decrease in the entropy after the data set is split is the information gain.
- 📊 **Leaf Node:** A leaf node is a node which carries the classification or the decision.
- 📊 **Decision Node:** A node that has two or more branches.
- 📊 **Root Node:** The root node is the topmost decision node, which is where you have all of your data.

Case Study 7:

Classify the different types of fruits in a bowl based on various features using Random Forest Algorithm

Let's say we want to classify the different types of fruits in a bowl based on various features, but the bowl is cluttered with a lot of options. You would create a training dataset that contains information about the fruit, including colors, diameters, and specific labels (i.e., apple, grapes, etc.) You would then need to split the data by sorting out the smallest piece so that you can split it in the biggest way possible. You might want to start by splitting your fruits by diameter and then by color. You would want to keep splitting until that particular node no longer needs it, and you can predict a specific fruit with 100 percent accuracy.

Now let's say you have some flowers and you're trying to figure out what species of iris they belong to. In this case example, you can use Python coding to determine the species. First, you'll load the different modules into Python in an editor. If you're going to do a Random Forest classifier, you'll also need to import a Random Forest classifier from the scikit module. You'll also need to import two other modules: pandas (which will create a data frame) and numpy (which are the arrays in Python).

These allow the user to perform different mathematical sets. You'll then need to assign your data to the variable "iris" in this specific example. After the iris data is imported, you'll need to look at the target and put that section of code in your notebook. As you explore this data, you'll need to split it into different parts, called training and testing. You'll also want to make your data legible to humans. Nevertheless, you'll also need to create something that the computer understands, which you'll do in your final step. Before you've finished, you need to take care of the prediction and create a Random Forest classifier, which is the code that does everything. It's crucial to limit the process as much as possible to not overwhelm the system.

When you run the code, it's going to come out with a bunch of zeros, ones, and twos, which represent the three types of flowers based on the test features and other imported data. There are also other methods for running this data, which could yield slightly different results. Once

you run your code, you'll get a prediction. You may also rerun the code based on different variables. In our case example, the image below shows how likely the flowers you're trying to identify fall under a specific species. Random Forest will give you your prediction, but it needs to match the actual data to validate the accuracy. What you'll need to do is combine these with a single line of code, which will create a chart. You may end up with a set of precise predictions, as well as a set of inaccurate ones. A great quality of the random forest algorithm is that it is very tranquil to measure the relative importance of each feature on the prediction.

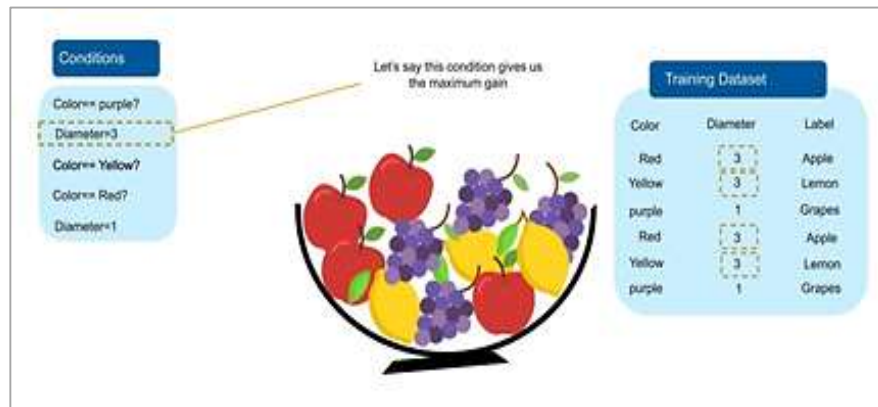


Fig. 2.16: Classifying Fruits in a bowl using Random Forest Algorithm

Advantages and Disadvantages of Random Forest Algorithm

- ✚ One of the biggest advantages of random forest is its versatility. It can be used for both regression and classification tasks, and it's also easy to view the relative importance it assigns to the input features.
- ✚ Random forest is also a very handy algorithm because the default hyperparameters it uses often produce a good prediction result. Understanding the hyperparameters is pretty straightforward, and there's also not that many of them.
- ✚ One of the biggest problems in machine learning is overfitting, but most of the time this won't happen thanks to the random forest classifier. If there are enough trees in the forest, the classifier won't overfit the model.
- ✚ The main limitation of random forest is that a large number of trees can make the algorithm too slow and ineffective for real-time predictions. In general, these algorithms are fast to train, but quite slow to create predictions once they are trained. A more accurate prediction requires more trees, which results in a slower model. In most real-world applications, the random forest algorithm is fast enough but there can certainly be situations where run-time performance is vital and other approaches might be preferred.

- And, of course, random forest is a predictive modeling tool and not a descriptive tool, meaning if you're looking for a description of the relationships in your data, other approaches would be better.

Random forest is a great algorithm to train early in the model development process, to see how it performs. Its simplicity makes building a “bad” random forest a tough proposition. The algorithm is also a great choice for anyone who needs to develop a model quickly. On top of that, it provides a pretty good indicator of the importance it assigns to your features. Random forests are also very hard to beat performance wise. Of course, you can probably always find a model that can perform better, like a neural network for example, but these usually take more time to develop, though they can handle a lot of different feature types, like binary, categorical and numerical. Overall, random forest is a (mostly) fast, simple and flexible tool, but not without some limitations.

2.11.5 Support Vector Machines

Support Vector Machine (SVM) is conceivably one of the most popular and talked about machine learning algorithms. They were extremely popular around the time they were developed in the 1990s and continue to be the go-to method for a high-performing algorithm with little tuning. SVM is an exciting algorithm and the concepts are relatively simple.

The Maximal-Margin Classifier is a hypothetical classifier that best explains how SVM works in practice. The numeric input variables (x) in your data (the columns) form an n-dimensional space. For example, if you had two input variables, this would form a two-dimensional space. A hyperplane is a line that splits the input variable space. In SVM, a hyperplane is selected to best separate the points in the input variable space by their class, either class 0 or class 1. In two-dimensions you can visualize this as a line and let's assume that all of our input points can be completely separated by this line. For example:

$$B_0 + (B_1 * X_1) + (B_2 * X_2) = 0$$

Where the coefficients (B1 and B2) that determine the slope of the line and the intercept (B0) are found by the learning algorithm, and X1 and X2 are the two input variables. You can make classifications using this line. By plugging in input values into the line equation, you can calculate whether a new point is above or below the line.

- Above the line, the equation returns a value greater than 0 and the point belongs to the first class (class 0).
- Below the line, the equation returns a value less than 0 and the point belongs to the second class (class 1).

- ✚ A value close to the line returns a value close to zero and the point may be difficult to classify.
- ✚ If the magnitude of the value is large, the model may have more confidence in the prediction.

The distance between the line and the closest data points is referred to as the margin. The best or optimal line that can separate the two classes is the line that has the prime margin. This is called the Maximal-Margin hyperplane. The margin is calculated as the perpendicular distance from the line to only the closest points. Only these points are relevant in defining the line and in the construction of the classifier. These points are called the support vectors. They support or define the hyperplane. The hyperplane is learned from training data using an optimization procedure that maximizes the margin. In practice, real data is messy and cannot be separated perfectly with a hyperplane.

The constraint of maximizing the margin of the line that splits the classes must be relaxed. This is often called the soft margin classifier. This change allows some points in the training data to violate the separating line. An additional set of coefficients are introduced that give the margin wiggle room in each dimension. These coefficients are sometimes called slack variables. This increases the complexity of the model as there are more parameters for the model to fit to the data to provide this complexity.

A tuning parameter is presented called C that defines the magnitude of the wiggle allowed across all dimensions. The C parameters define the amount of violation of the margin allowed. A $C=0$ is no violation and we are back to the inflexible Maximal-Margin Classifier described above. The larger the value of C the more violations of the hyperplane are permitted. During the learning of the hyperplane from data, all training instances that lie within the distance of the margin will affect the placement of the hyperplane and are referred to as support vectors. And as C affects the number of instances that are allowed to fall within the margin, C influences the number of support vectors used by the model.

- ✚ The smaller the value of C , the more sensitive the algorithm is to the training data (higher variance and lower bias).
- ✚ The larger the value of C , the less sensitive the algorithm is to the training data (lower variance and higher bias).

The SVM algorithm is implemented in practice using a kernel. The learning of the hyperplane in linear SVM is done by transforming the problem using some linear algebra, which is out of the scope of this introduction to SVM. A powerful insight is that the linear SVM can be rephrased using the inner product of any two given observations, rather than the observations themselves. The inner product between two vectors is the sum of the multiplication of each pair

of input values. For example, the inner product of the vectors [2, 3] and [5, 6] is $2*5 + 3*6$ or 28. The equation for making a prediction for a new input using the dot product between the input (x) and each support vector (xi) is calculated as follows:

$$f(x) = B_0 + \sum (a_i * (x, x_i))$$

This is an equation that involves calculating the inner products of a new input vector (x) with all support vectors in training data. The coefficients B_0 and a_i (for each input) must be estimated from the training data by the learning algorithm.

Linear Kernel SVM: The dot-product is called the kernel and can be re-written as:

$$K(x, x_i) = \sum (x * x_i)$$

The kernel defines the similarity or a distance measure between new data and the support vectors. The dot product is the similarity measure used for linear SVM or a linear kernel because the distance is a linear combination of the inputs. Other kernels can be used that transform the input space into higher dimensions such as a Polynomial Kernel and a Radial Kernel. This is called the Kernel Trick. It is desirable to use more complex kernels as it allows lines to separate the classes that are curved or even more complex. This in turn can lead to more accurate classifiers.

Polynomial Kernel SVM: Instead of the dot-product, we can practice a polynomial kernel, for example:

$$K(x, x_i) = 1 + \sum (x * x_i)^d$$

Where the degree of the polynomial must be specified by hand to the learning algorithm. When $d=1$ this is the same as the linear kernel. The polynomial kernel allows for curved lines in the input space.

Radial Kernel SVM: Finally, we can also have a more complex radial kernel. For example:

$$K(x, x_i) = \exp(-\gamma * \sum ((x - x_i)^2))$$

Where γ is a parameter that must be stated to the learning algorithm. A good default value for γ is 0.1, where γ is $0 < \gamma < 1$. The radial kernel is very local and can create complex regions within the feature space, like closed polygons in two-dimensional space. The objective of the SVM is to find a hyperplane in an N-dimensional space (N — the number of features) that distinctly classifies the data points.

How to Learn a SVM Model: The SVM model needs to be solved using an optimization procedure. You can use a numerical optimization procedure to search for the coefficients of the hyperplane. This is inefficient and is not the approach used in widely used SVM implementations like LIBSVM. If implementing the algorithm as an exercise, you could use stochastic gradient descent. There are specialized optimization procedures that re-formulate the optimization problem to be a Quadratic Programming problem. The most popular method for fitting SVM is the Sequential Minimal Optimization (SMO) method that is very efficient. It breaks the problem down into sub-problems that can be solved analytically (by calculating) rather than numerically (by searching or optimizing).

Data Preparation for SVM: Here are some suggestions for how to best prepare your training data when learning an SVM model.

- ✚ **Numerical Inputs:** SVM assumes that your inputs are numeric. If you have categorical inputs you may need to convert them to binary dummy variables (one variable for each category).
- ✚ **Binary Classification:** Basic SVM as described is intended for binary (two-class) classification problems although extensions have been advanced for regression and multi-class classification.

Advantages and Disadvantages of Support Vector Machine

- ✚ It works really well with clear margin of separation.
- ✚ It is effective in high dimensional spaces.
- ✚ It is effective in cases where number of dimensions is greater than the number of samples.
- ✚ It uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.
- ✚ It doesn't perform well, when we have large data set because the required training time is higher.
- ✚ It also doesn't perform very well, when the data set has more noise i.e. target classes are overlapping.
- ✚ SVM doesn't directly provide probability estimates, these are calculated using an expensive five-fold cross-validation. It is related SVC method of Python scikit-learn library.

2.11.6 Singular Value Decomposition

Matrix decomposition, also known as matrix factorization, includes relating a given matrix using its constituent elements. Perhaps the most known and widely employed matrix

decomposition technique is the Singular-Value Decomposition, or SVD. All matrices have an SVD, which makes it more stable than other methods, such as the Eigen decomposition. As such, it is often used in a wide array of applications including compressing, denoising, and data reduction. The Singular-Value Decomposition, or SVD for short, is a matrix decomposition method for reducing a matrix to its constituent parts in order to make certain subsequent matrix calculations simpler. For the case of simplicity we will focus on the SVD for real-valued matrices and ignore the case for complex numbers.

$$A = U \cdot \text{Sigma} \cdot V^T$$

Where A is the real m x n matrix that we wish to decompose, U is an m x m matrix, Sigma (often represented by the uppercase Greek letter Sigma) is an m x n diagonal matrix, and V^T is the transpose of an n x n matrix where T is a superscript. The diagonal values in the Sigma matrix are known as the singular values of the original matrix A. The columns of the U matrix are called the left-singular vectors of A, and the columns of V are called the right-singular vectors of A. The SVD is calculated via iterative numerical methods. Every rectangular matrix has singular value decomposition, but the resulting matrices may contain complex numbers and the limitations of floating point arithmetic may cause some matrices to fail to decompose neatly. The SVD is used widely both in the calculation of other matrix operations, such as matrix inverse, but also as a data reduction method in machine learning. SVD can also be used in least squares linear regression, image compression, and denoising data.

The SVD can be calculated by calling the `svd()` function. The function takes a matrix and returns the U, Sigma and V^T elements. The Sigma diagonal matrix is returned as a vector of singular values. The V matrix is returned in a transposed form. The original matrix can be reconstructed from the U, Sigma, and V^T elements. The U, s, and V elements returned from the `svd()` cannot be multiplied directly. The s vector must be converted into a diagonal matrix using the `diag()` function. By default, this function will create a square matrix that is n x n, relative to our original matrix. This cause a problem as the size of the matrices does not fit the rules of matrix multiplication, where the number of columns in a matrix must match the number of rows in the subsequent matrix. After creating the square Sigma diagonal matrix, the sizes of the matrices are relative to the original m x n matrix that we are decomposing, as follows:

$$U (m \times m) \cdot \text{Sigma} (n \times n) \cdot V^T (n \times n)$$

Where, in fact, we require:

$$U (m \times m) \cdot \text{Sigma} (m \times n) \cdot V^T (n \times n)$$

We can achieve this by creating a new Sigma matrix of all zero values that is m x n (e.g. more rows) and populate the first n x n part of the matrix with the square diagonal matrix calculated

via `diag()`. The pseudoinverse is the generalization of the matrix inverse for square matrices to rectangular matrices where the number of rows and columns are not equal. It is also known as the Moore-Penrose Inverse after two independent discoverers of the method or the Generalized Inverse. The pseudoinverse is denoted as A^+ , where A is the matrix that is being inverted and $+$ is a superscript. The pseudoinverse is calculated using the singular value decomposition of A :

$$A^+ = V \cdot D^+ \cdot U^T$$

Or, without the dot notation:

$$A^+ = VD^+U^T$$

Where A^+ is the pseudoinverse, D^+ is the pseudoinverse of the diagonal matrix Σ and U^T is the transpose of U . We can get U and V from the SVD operation.

$$A = U \cdot \Sigma \cdot V^T$$

The D^+ can be calculated by creating a diagonal matrix from Σ , calculating the reciprocal of each non-zero element in Σ , and taking the transpose if the original matrix was rectangular. The pseudoinverse affords one way of solving the linear regression equation, specifically when there are more rows than there are columns, which is often the case. NumPy runs the function `pinv()` for scheming the pseudoinverse of a rectangular matrix. We can calculate the pseudoinverse manually via the SVD and compare the results to the `pinv()` function. First we must calculate the SVD. Next we must calculate the reciprocal of each value in the s array. Then the s array can be transformed into a diagonal matrix with an added row of zeros to make it rectangular. Finally, we can calculate the pseudoinverse from the elements.

SVD for Dimensionality Reduction: A popular application of SVD is for dimensionality reduction. Data with a large number of features, such as more features (columns) than observations (rows) may be reduced to a smaller subset of features that are most relevant to the prediction problem. The result is a matrix with a lower rank that is said to approximate the original matrix. To do this we can perform an SVD operation on the original data and select the top k largest singular values in Σ . These columns can be selected from Σ and the rows selected from V^T . An approximate B of the original vector A can then be reconstructed.

$$B = U \cdot \Sigma_k \cdot V^T_k$$

In natural language processing, this approach can be used on matrices of word occurrences or word frequencies in documents and is called Latent Semantic Analysis or Latent Semantic

Indexing. In practice, we can retain and work with a descriptive subset of the data called T . This is a dense summary of the matrix or a projection.

$$T = U \cdot \Sigma_k$$

Further, this transform can be calculated and applied to the original matrix A as well as other similar matrices.

$$T = V^k \cdot A$$

First a 3×10 matrix is defined, with more columns than rows. The SVD is calculated and only the first two features are selected. The elements are recombined to give an accurate reproduction of the original matrix. Finally the transform is calculated two different ways. The scikit-learn provides a Truncated SVD class that implements this capability directly.

The Truncated SVD class can be created in which you must specify the number of desirable features or components to select, e.g. 2. Once created, you can fit the transform (e.g. calculate V^k) by calling the `fit()` function, then apply it to the original matrix by calling the `transform()` function. The result is the transform of A called T above. We can see that the values match those calculated manually above, except for the sign on some values. We can expect there to be some instability when it comes to the sign given the nature of the calculations involved and the differences in the underlying libraries and methods used. This instability of sign should not be a problem in practice as long as the transform is trained for reuse.

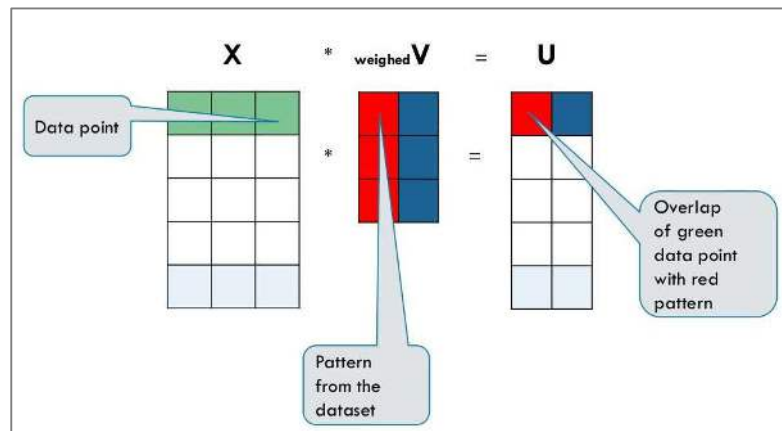


Fig. 2.17: Singular Value Decomposition

The singular value decomposition is called the "billion-dollar algorithm" since it provides the mathematical basis for many modern algorithms in data science, including text mining,

recommender systems (think Netflix and Amazon), image processing, and classification problems. Although the SVD was mathematically discovered in the late 1800s, computers have made the SVD an indispensable tool in computational statistics and data science. Singular Value Decomposition and the closely-related Principal Component Analysis (PCA) are well established feature extraction methods that have a wide range of applications. Oracle Data Mining implements SVD as a feature extraction algorithm and PCA as a special scoring method for SVD models.

2.11.7 Gaussian Naive Bayes

Naive Bayes is a simple but surprisingly powerful algorithm for predictive modeling. In machine learning we are often interested in selecting the best hypothesis (h) given data (d). In a classification problem, our hypothesis (h) may be the class to assign for a new data instance (d). One of the easiest ways of selecting the most probable hypothesis given the data we can use as our prior knowledge about the problem. Bayes' Theorem provides a way that we can calculate the probability of a hypothesis given our prior knowledge. Bayes' Theorem is stated as:

$$P(h|d) = (P(d|h) * P(h)) / P(d)$$

Where

$P(h|d)$ is the probability of hypothesis h given the data d. This is called the posterior probability.

$P(d|h)$ is the probability of data d given that the hypothesis h was true.

$P(h)$ is the probability of hypothesis h being true (regardless of the data). This is called the prior probability of h.

$P(d)$ is the probability of the data (regardless of the hypothesis).

You can see that we are interested in calculating the posterior probability of $P(h|d)$ from the prior probability $p(h)$ with $P(D)$ and $P(d|h)$. After calculating the posterior probability for a number of different hypotheses, you can select the hypothesis with the highest probability. This is the maximum probable hypothesis and may formally be called the maximum a posteriori (MAP) hypothesis. This can be written as:

$$\text{MAP}(h) = \max(P(h|d))$$

or

$$\text{MAP}(h) = \max((P(d|h) * P(h)) / P(d))$$

or

$$\text{MAP}(h) = \max(P(d|h) * P(h))$$

The $P(d)$ is a normalizing term which allows us to calculate the probability. We can drop it when we are interested in the most probable hypothesis as it is constant and only used to normalize. Back to classification, if we have an even number of instances in each class in our training data, then the probability of each class (e.g. $P(h)$) will be equal. Again, this would be a constant term in our equation and we could drop it so that we end up with:

$$\text{MAP}(h) = \max(P(d|h))$$

This is a useful exercise, because when reading up further on Naive Bayes you may see all of these forms of the theorem. Naive Bayes is a classification algorithm for binary (two-class) and multi-class classification problems. The technique is easiest to understand when described using binary or categorical input values. It is called naive Bayes or idiot Bayes because the calculation of the probabilities for each hypothesis is simplified to make their calculation tractable. Rather than attempting to calculate the values of each attribute value $P(d_1, d_2, d_3|h)$, they are assumed to be conditionally independent given the target value and calculated as $P(d_1|h) * P(d_2|h)$ and so on. This is a very strong assumption that is most unlikely in real data, i.e. that the attributes do not interact. Nevertheless, the approach performs surprisingly well on data where this assumption does not hold. Naive Bayes can be extended to real-valued attributes, most commonly by assuming a Gaussian distribution. This extension of naive Bayes is called Gaussian Naive Bayes. Other functions can be used to estimate the distribution of the data, but the Gaussian (or Normal distribution) is the easiest to work with because you only need to estimate the mean and the standard deviation from your training data.

Representation for Gaussian Naive Bayes: Above, we calculated the probabilities for input values for each class using a frequency. With real-valued inputs, we can calculate the mean and standard deviation of input values (x) for each class to summarize the distribution. This means that in addition to the probabilities for each class, we must also store the mean and standard deviations for each input variable for each class. Learn a Gaussian Naive Bayes Model From Data: This is as simple as calculating the mean and standard deviation values of each input variable (x) for each class value.

$$\text{mean}(x) = 1/n * \text{sum}(x)$$

Where n is the number of instances and x are the values for an input variable in your training data.

We can calculate the standard deviation using the following equation:

$$\text{standard deviation}(x) = \sqrt{1/n * \sum(x_i - \text{mean}(x))^2)}$$

This is the square root of the average squared difference of each value of x from the mean value of x , where n is the number of instances, $\sqrt{}$ is the square root function, $\sum()$ is the sum function, x_i is a specific value of the x variable for the i 'th instance and $\text{mean}(x)$ is described above, and 2 is the square. Make Predictions With a Gaussian Naive Bayes Model: Probabilities of new x values are calculated using the Gaussian Probability Density Function (PDF). When making predictions these parameters can be plugged into the Gaussian PDF with a new input for the variable, and in return the Gaussian PDF will provide an estimate of the probability of that new input value for that class.

$$\text{pdf}(x, \text{mean}, \text{sd}) = (1 / (\sqrt{2 * \text{PI}} * \text{sd})) * \exp(-((x - \text{mean})^2 / (2 * \text{sd}^2)))$$

Where $\text{pdf}(x)$ is the Gaussian PDF, $\sqrt{}$ is the square root, mean and sd are the mean and standard deviation calculated above, PI is the numerical constant, $\exp()$ is the numerical constant e or Euler's number raised to power and x is the input value for the input variable. We can then plug in the probabilities into the equation above to make predictions with real-valued inputs. For example, adapting one of the above calculations with numerical values for weather and car:

$$\text{go-out} = P(\text{pdf}(\text{weather})|\text{class}=\text{go-out}) * P(\text{pdf}(\text{car})|\text{class}=\text{go-out}) * P(\text{class}=\text{go-out})$$

Best Prepare Your Data For Naive Bayes:

-  **Categorical Inputs:** Naive Bayes assumes label attributes such as binary, categorical or nominal.
-  **Gaussian Inputs:** If the input variables are real-valued, a Gaussian distribution is assumed. In which case the algorithm will perform better if the univariate distributions of your data are Gaussian or near-Gaussian. This may require removing outliers (e.g. values that are more than 3 or 4 standard deviations from the mean).
-  **Classification Problems:** Naive Bayes is a classification algorithm suitable for binary and multiclass classification.
-  **Log Probabilities:** The calculation of the likelihood of different class values involves multiplying a lot of small numbers together. This can lead to an underflow of numerical precision. As such it is good practice to use a log transform of the probabilities to avoid this underflow.
-  **Kernel Functions:** Rather than assuming a Gaussian distribution for numerical input values, more complex distributions can be used such as a variety of kernel density functions.

- 📌 **Update Probabilities:** When new data becomes available, you can simply update the probabilities of your model. This can be helpful if the data changes frequently.

2.11.8 Markov Decision Process

The goal of reinforcement learning is to tweak the system in order to perform right or wrong in certain ways. We do that by attaching rewards and punishments to different outcomes, which ultimately drive the machine to find the “right” priorities. An example of it is self-driving cars. There the system prefers one optimality versus another, taking into concern rewards and punishments along the way. A mathematical representation of a complex decision making process is “Markov Decision Processes” (MDP). MDP is defined by:

- 📌 A state S , which represents every state that one, could be in, within a defined world.
- 📌 A model or transition function T is a function of the current state, the action taken and the state where we end up. This transition produces a certain probability of ending up in state S' , starting from the state S and taking the action A .
- 📌 Actions are things; one can do in a particular state.
- 📌 A reward is a scalar value for being in the state. It tells us the usefulness of entering the state.

Supervised learning is characterized by x, y pairs and the goal is to find function approximation that defines them and that predicts future data. In Unsupervised learning, instead, we are given a bunch of x 's and the goal is to find a function that describes the unrolling clustering description. Instead, in reinforcement learning we are given a pair of data (x, z) and we need to learn the function that generates y . The final goal of the MDP is to find a policy that can tell us, for any state, which action to take. The optimal policy is the one that maximizes the long-term expected reward. There are two properties on which the Markovian process is based on:

- 📌 Only the present matters; which means that the transition function only depends on the current state S and not any of the previous states.
- 📌 Things are stationary; therefore, rules do no change over time.

Going back to the reinforcement learning definition above, when compared to the rest of the machine learning tools, we can see better what are the inputs and the outputs. We are given a bunch of state/action pairs and we learn a function, i.e. the policy, which maps the states into actions. Therefore in the MDP world the x is state “ S ”, z is the reward “ R ” and y is the action “ A ” and finally the policy is the function we are aiming to. As mentioned before, the MDP process is defined by two characteristics, one of which is stationarity. This property plays a role in terms of assumptions we take. The actions taken during a transition between different states take us to different rewards.

This mapping of states to actions is our policy. Well, this policy remains the same only if we are considering an infinite time horizon, while, instead, for a finite time horizon the policy might change, even if I'm in the same state. Another concept related to stationarity is the "stationarity of sequences". This claims that if we prefer one sequence of states today over another sequence of states, then we prefer that sequence of states over the same sequence of states even tomorrow.

What defines a sequence of states is the "utility" function. Basically a reward gives us immediate feedback, present gratification, while utility gives a long-term feedback, since it accounts for all the delayed rewards. Having introduced this concept of utility now helps us understand better what a policy and, in particular, what an optimal policy is. The optimal policy is the one that maximizes the expected rewards. The utility does indeed depend upon the policy. It is the expected set of states that we see from that point on, given that we follow a policy. This finally takes us to the definition given by the Bellman equation, which mathematically states the following:

"The utility of a certain state is equal to the reward in the current state, plus the discount from all the rewards we get from that point on, transitioning from the current state to a future state". The "discount" mentioned above is a value between zero and one, and it allows us to treat an infinite sequence with finite value. Otherwise the utility in an infinite time horizon will always be the same, of an infinite magnitude. To solve the Bellman equation we normally start with arbitrary utilities and then update them based on the neighbors (which are all the states that it can reach from the current state). Finally, we repeat that until convergence. This process is called "value iteration". As we can infer, the utility is similar to a regression, mapping through continuous values, while the policy is more like a classifier. Besides the above-mentioned value-based function (from which we observe the values as outputs, find the utility and finally infer the policy), there are two other methods.

One is the "policy-search" algorithm, with which we observe the actions from different states and deduct the policy, which we can directly use. The other one is a "model-based" algorithm, from which we get state/action pairs as input and we receive rewards/states as output. It's a direct learning approach, similar to a supervised learning one, but with an indirect use. In fact, from this one we will need to solve the Bellman equation in order to deduct the utility and then finally derive the policy. MDPs are the tools for modelling decision problems, but how we solve them? In order to solve MDPs we need Dynamic Programming, more specifically the Bellman equation. But first what is dynamic programming? Basically it's a method that divides a problem into simpler sub-problems easier to solve, it's just really a divide and conquer strategy. Dynamic programming is both a mathematical optimization method and a computer programming method, but both of them follow this divide and conquer mechanism.

But on mathematics it's often used as an optimization tool. On programming is often implemented with recursion and is used on problems like find the shortest path on a graph and generation sequences. Also you may find a term called memorization which is something that computer people use to improve the performance of those divide-and-conquer algorithms by memorizing sub problems that were already calculated. MDP is somehow more powerful than simple planning, because your policy will allow you to do optimal actions even if something went wrong along the way. Simple planning just follows the plan after you find the best strategy.

2.11.9 Principal Component Analysis

There are two basic approaches to factor analysis: principal component analysis (PCA) and common factor analysis. Commonly, factor analysis includes methods to help produce a smaller number of linear combinations on variables so that the reduced variables account for and explain most the variance in correlation matrix pattern. Principal component analysis is an approach to factor analysis that considers the total variance in the data, which is unlike common factor analysis, transforms the original variables into a smaller set of linear combinations. The diagonal of the correlation matrix consists of unities and the full variance is brought into the factor matrix. The term factor matrix is the matrix that comprises the factor loadings of all the variables on all the factors extracted.

The term 'factor loadings' are the simple correlations within the factors and the variables. Principal component analysis is suggested when the researcher's primary concern is to determine the minimum number of factors that will account for the maximum variance in the data in use in the particular multivariate analysis, like in Delphi studies. While conducting principal component analysis, the researcher can get well versed with terms such as standard deviations and eigenvalues. The eigenvalues refer to the total variance explained by each factor. The standard deviation measures the variability of the data. The task of principal component analysis is to identify the patterns in the data and to direct the data by highlighting their similarities and differences.

PCA is a tool for finding patterns in high-dimensional data such as images. Machine-learning practitioners sometimes use PCA to preprocess data for their neural networks. By centering, rotating and scaling data, PCA arranges dimensionality (allowing you to drop some low-variance dimensions) and can improve the neural network's convergence speed and the overall quality of results. To get to PCA, we're going to quickly designate certain basic statistical ideas – mean, standard deviation, variance and covariance – so we can weave them together later. Their equations are closely related. Mean is simply the average value of all x 's in the set X , which is found by dividing the sum of all data points by the number of data points, n .

$$\bar{X} = \frac{\sum_{i=1}^n X_i}{n}$$

Standard deviation, as fun as that sounds, is simply the square root of the average square distance of data points to the mean. In the equation below, the numerator contains the sum of the differences between each datapoint and the mean, and the denominator is simply the number of data points (minus one), producing the average distance.

$$s = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{(n - 1)}}$$

Variance is the measure of the data's spread. If we take a team of Dutch basketball players and measure their height, those measurements won't have a lot of variance. They'll all be grouped above six feet. But if we throw the Dutch basketball team into a classroom of psychotic kindergartners, then the combined group's height measurements will have a lot of variance. Variance is the spread, or the amount of difference that data expresses. Variance is simply standard deviation squared, and is often stated as s^2 .

$$var(X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{(n - 1)}$$

For both variance and standard deviation, squaring the differences between data points and the mean makes them positive, so that values above and below the mean don't cancel each other out. Let's assume to plot the age (x axis) and height (y axis) of those individuals (setting the mean to zero) and came up with an oblong scatterplot:

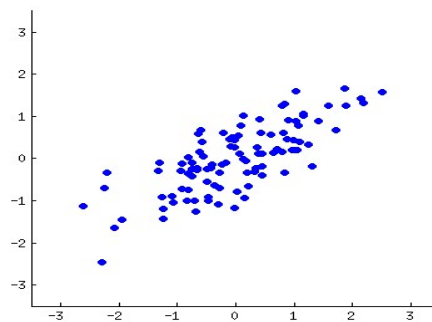


Fig. 2.18: Plot between the age (x axis) and height (y axis)

PCA attempts to draw straight, explanatory lines through data, like linear regression. Each straight line represents a “principal component,” or a relationship between an independent and dependent variable. While there are as many principal components as there are dimensions in the data, PCA’s role is to prioritize them. The first principal component bisects a scatterplot with a straight line in a way that explains the most variance; that is, it follows the longest dimension of the data. (This happens to coincide with the least error, as expressed by the red lines). In the graph below, it slices down the length of the “baguette.”

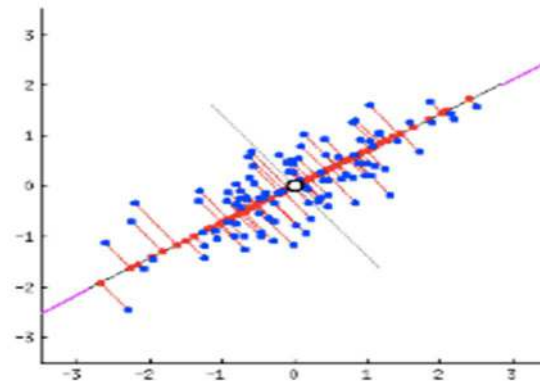


Fig. 2.19: Principal Component Analysis Graph

The second principal component cuts through the data perpendicular to the first, fitting the errors produced by the first. There are only two principal components in the graph above, but if it were three-dimensional, the third component would fit the inaccuracies from the first and second principal components, and so forth.

Covariance Matrix: While we introduced matrices as something that transformed one set of vectors into another, another way to think about them is as a description of data that captures the forces at work upon it, the forces by which two variables might relate to each other as expressed by their variance and covariance. Imagine that we compose a square matrix of numbers that describe the variance of the data, and the covariance among variables. This is the covariance matrix. It is an empirical description of data we observe. Finding the eigenvectors and eigenvalues of the covariance matrix is the equivalent of fitting those straight, principal-component lines to the variance of the data. Why? Because eigenvectors trace the principal lines of force, and the axes of greatest variance and covariance illustrate where the data is most susceptible to change.

If a variable changes, it is being acted upon by a force known or unknown. If two variables change together, in all likelihood that is either because one is acting upon the other, or they are

both subject to the same hidden and unnamed force. When a matrix performs a linear transformation, eigenvectors trace the lines of force it applies to input; when a matrix is populated with the variance and covariance of the data, eigenvectors reflect the forces that have been applied to the given. One applies force and the other reflects it. Eigenvalues are simply the coefficients attached to eigenvectors, which give the axes magnitude. In this case, they are the measure of the data's covariance. By ranking your eigenvectors in order of their eigenvalues, highest to lowest, you get the principal components in order of significance. For a 2 x 2 matrix, a covariance matrix might look like this:

$$\begin{bmatrix} 1.07 & 0.63 \\ 0.63 & 0.64 \end{bmatrix}$$

The numbers on the upper left and lower right represent the variance of the x and y variables, respectively, while the identical numbers on the lower left and upper right represent the covariance between x and y. Because of that identity, such matrices are known as symmetrical. As you can see, the covariance is positive, since the graph near the top of the PCA section points up and to the right. If two variables increase and decrease together (a line going up and to the right), they have a positive covariance, and if one decreases while the other increases, they have a negative covariance (a line going down and to the right). Notice that when one variable or the other doesn't move at all, and the graph shows no diagonal motion, there is no covariance whatsoever (Figure 2.20). Covariance answers the question; do these two variables dance together? If one remains null while the other moves, the answer is no. Likewise, in the equations below, you'll notice that there is only a small difference between covariance and variance.

$$cov(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{(n - 1)}$$

$$var(X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{(n - 1)}$$

The great thing about calculating covariance is that, in a high-dimensional space where you can't eyeball intervariable relationships, you can know how two variables move together by the positive, negative or non-existent character of their covariance. (Correlation is a compassionate of normalized covariance, with a value between -1 and 1.) To sum up, the covariance matrix defines the shape of the data. Diagonal spread along eigenvectors is expressed by the covariance, while x-and-y-axis-aligned spread is expressed by the variance.

Causality has a bad name in statistics, so take this with a grain of salt. While not entirely accurate, it may help to think of each component as a causal force in the Dutch basketball player example above, with the first principal component being age; the second possibly gender; the third nationality, and each of those occupying its own dimension in relation to height. Each acts on height to different degrees. You can read covariance as traces of possible cause.

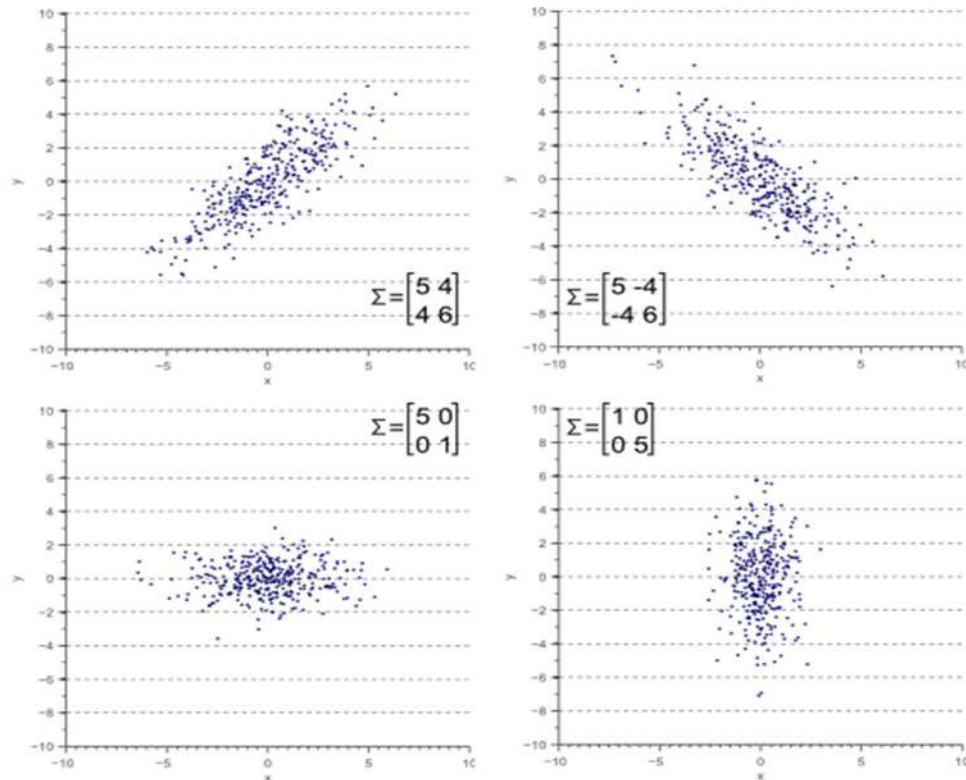


Fig. 2.20: Covariance Graph of Principal Component Analysis

Change of Basis: Because the eigenvectors of the covariance matrix are orthogonal to each other, they can be used to reorient the data from the x and y axes to the axes represented by the principal components. You re-base the coordinate system for the dataset in a new space defined by its lines of greatest variance. The x and y axes we've shown above are what's called the basis of a matrix; that is, they provide the points of the matrix with x, y coordinates. But it is possible to recast a matrix along other axes; for example, the eigenvectors of a matrix can serve as the foundation of a new set of coordinates for the same matrix. Matrices and vectors are animals in themselves, independent of the numbers linked to a specific coordinate system like x and y.

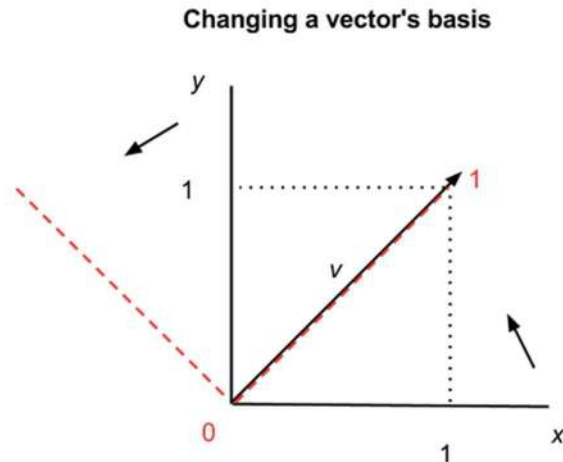


Fig. 2.21: Change of Basis

In the graph above, we see how the same vector v can be situated contrarily in two coordinate systems, the x - y axes in black, and the two other axes shown by the red dashes. In the first coordinate system, $v = (1,1)$, and in the second, $v = (1,0)$, but v itself has not changed. Vectors and matrices can therefore be abstracted from the numbers that appear inside the brackets. This has profound and almost spiritual implications, one of which is that there exists no natural coordinate system, and mathematical objects in n -dimensional space are subject to multiple descriptions. Changing matrices' bases also makes them easier to manipulate. A change of basis for vectors is roughly analogous to changing the base for numbers; i.e. the quantity nine can be described as 9 in base ten, as 1001 in binary, and as 100 in base three (i.e. 1, 2, 10, 11, 12, 20, 21, 22, 100 $\leftarrow$ that is "nine"), which means Same quantity, different symbols; same vector, different coordinates.

Entropy and Information Gain: In information theory, the term entropy refers to information we don't have (normally people define "information" as what they know, and jargon has triumphed once again in turning plain language on its head to the detriment of the uninitiated). The information we don't have about a system, its entropy, is related to its unpredictability: how much it can surprise us. If you know that a certain coin has heads embossed on both sides, then flipping the coin gives you absolutely no information, because it will be heads every time. You don't have to flip it to know. We would say that two-headed coin contains no information, because it has no way to surprise you. A balanced, two-sided coin does contain an element of surprise with each coin toss. And a six-sided die, by the same argument, contains even more surprise with each roll, which could produce any one of six results with equal frequency. Both those objects contain information in the technical sense.

Now let's imagine the die is loaded, it comes up "three" on five out of six rolls, and we figure out the game is rigged. Suddenly the amount of surprise produced with each roll by this die is greatly reduced. We understand a trend in the die's behavior that gives us greater predictive capacity. Understanding the die is loaded is similar to finding a principal component in a dataset. You simply identify an underlying pattern. That transfer of information, from what we don't know about the system to what we know, represents a change in entropy. Insight decreases the entropy of the system. Get information, reduce entropy.

This is information gain. And yes, this type of entropy is subjective, in that it depends on what we know about the system at hand. Successively, each principal component cutting through the scatterplot means a decrease in the system's entropy, in its unpredictability. It so happens that clarifying the shape of the data one principal component at a time, beginning with the component that accounts for the maximum variance, is analogous to walking data through a decision tree. The first component of PCA, like the first if-then-else split in a suitably formed decision tree, will be along the dimension that moderates unpredictability the most.

Advantages of Principal Component Analysis

- ✚ **Removes Correlated Features:** In a real world scenario, this is very common that you get thousands of features in your dataset. You cannot run your algorithm on all the features as it will reduce the performance of your algorithm and it will not be easy to visualize that many features in any kind of graph. So, you must reduce the number of features in your dataset. You need to find out the correlation among the features. Finding correlation manually in thousands of features is nearly impossible, frustrating and time-consuming. PCA does this for you efficiently. After realizing the PCA on your dataset, all the Principal Components are sovereign of one another. There is no correlation among them.
- ✚ **Improves Algorithm Performance:** With so many features, the performance of your algorithm will drastically degrade. PCA is a very common way to speed up your Machine Learning algorithm by getting rid of correlated variables which don't contribute in any decision making. The training time of the algorithms reduces significantly with less number of features. Therefore, if the input dimensions are too high, then using PCA to speed up the algorithm is a reasonable choice.
- ✚ **Reduces Overfitting:** Overfitting mainly occurs when there are too many variables in the dataset. So, PCA helps in overcoming the overfitting issue by reducing the number of features.
- ✚ **Improves Visualization:** It is very hard to visualize and understand the data in high dimensions. PCA transforms a high dimensional data to low dimensional data (2D) so that it can be visualized easily. We can use 2D Scree Plot to see which Principal

Components result in high variance and have more impact as compared to other Principal Components. The trivial IRIS dataset is 4 dimensional which is hard to visualize. We can use PCA to reduce it to 2 dimensions for better visualization.

- ✚ **Maintains Information Lose:** When reducing the dimensions of data, it's important not to lose more information than is necessary. The variation in a dataset can be seen as signifying the information. PCA is a well-established mathematical technique for reducing the dimensionality of data, while keeping as much variation as possible.

Disadvantages of Principal Component Analysis

- ✚ **Independent variables become less interpretable:** After realizing PCA on the dataset, your original features will turn into Principal Components. Principal Components are the linear combination of your original features. Principal Components are not as readable and interpretable as original features.
- ✚ **Data standardization is must before PCA:** You must standardize your data before implementing PCA; otherwise PCA will not be able to find the optimal Principal Components. For instance, if a feature set has data expressed in units of Kilograms, Light years, or Millions, the variance scale is huge in the training set. If PCA is applied on such a feature set, the resultant loadings for features with high variance will also be large. Hence, principal components will be biased towards features with high variance, leading to false results. Also, for standardization, all the categorical features are required to be converted into numerical features before PCA can be applied. PCA is exaggerated by scale, so you need to scale the features in your data before applying PCA. Custom Standard Scaler from Scikit Learn to standardize the dataset features onto unit scale (mean = 0 and standard deviation = 1) which is a requirement for the optimal performance of many Machine Learning algorithms.
- ✚ **Information Loss:** Although Principal Components try to cover maximum variance among the features in a dataset, if we don't select the number of Principal Components with care, it may miss some information as compared to the original list of features.

Accordingly, Principal Component Analysis is a statistical techniques used to reduce the dimensionality of the data (reduce the number of features in the dataset) by selecting the most important features that capture maximum information about the dataset. The features are selected on the basis of variance that they cause in the output. Original features of the dataset are converted to the Principal Components which are the linear combinations of the existing features. The feature that causes highest variance is the first Principal Component. The feature that is responsible for second highest variance is considered the second Principal Component, and so on. In simple words, Principal Component Analysis is a method of extracting important features from a large set of features available in a dataset.

2.11.10 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a very common technique used for supervised classification problems. Let's understand together what is LDA and how does it work. Linear Discriminant Analysis is a dimensionality reduction technique used as a preprocessing step in Machine Learning and pattern classification applications. The principal objective of dimensionality reduction methods is to reduce the dimensions by removing the redundant and reliant features by transforming the features from higher dimensional space to a space with lower dimensions. Linear Discriminant Analysis is a supervised classification technique which takes labels into consideration.

This sort of dimensionality reduction is used in biometrics, bioinformatics and chemistry. The goal of Linear Discriminant Analysis is to project the features in higher dimension space onto a lower dimensional space. Linear Discriminant Analysis is most commonly used as dimensionality reduction technique in the pre-processing step for pattern-classification and machine learning applications. The objective is to project a dataset onto a lower-dimensional space with good class-separability in order avoid overfitting ("curse of dimensionality") and also reduce computational costs.

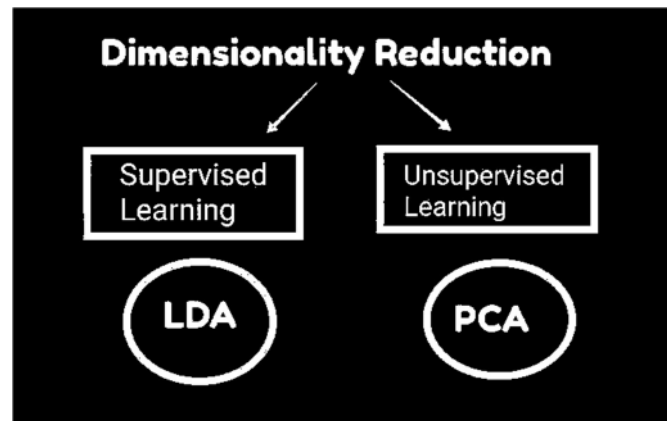


Fig. 2.22: Dimensionality Reduction using LDA and PDA

Logistic regression is a classification algorithm traditionally limited to only two-class classification problems. If you have more than two classes then Linear Discriminant Analysis is the preferred linear classification technique. LDA is a simple model in both preparation and application. There is some interesting statistics behind how the model is setup and how the prediction equation is derived. Logistic regression is a modest and powerful linear classification algorithm. It also has limitations that suggest at the need for alternate linear classification algorithms.

- 📊 **Two-Class Problems:** Logistic regression is intended for two-class or binary classification problems. It can be extended for multi-class classification, but is rarely used for this purpose.
- 📊 **Unstable with Well Separated Classes:** Logistic regression can become unstable when the classes are well separated.
- 📊 **Unstable With Few Examples:** Logistic regression can become unstable when there are few examples from which to estimate the parameters.

Linear Discriminant Analysis does address each of these points and is the go-to linear method for multi-class classification problems. Even with binary-classification problems, it is a good impression to try both logistic regression and linear discriminant analysis.

Representation of LDA Models: The representation of LDA is straight forward. It consists of statistical properties of your data, calculated for each class. For a single input variable (x) this are the mean and the variance of the variable for each class. For multiple variables, this is the same properties calculated over the multivariate Gaussian, namely the means and the covariance matrix. These statistical properties are estimated from your data and plug into the LDA equation to make predictions. These are the model values that you would save to file for your model. Let's look at how these parameters are estimated.

Learning LDA Models: LDA makes some simplifying assumptions about your data: That your data is Gaussian that each variable is shaped like a bell curve when plotted. That each attribute has the same variance that values of each variable vary around the mean by the same amount on average. With these assumptions, the LDA model estimates the mean and variance from your data for each class. It is easy to think about this in the univariate (single input variable) case with two classes. The mean (μ) value of each input (x) for each class (k) can be estimated in the normal way by dividing the sum of values by the total number of values.

$$\mu_k = 1/n_k * \sum(x)$$

Where μ_k is the mean value of x for the class k, n_k is the number of instances with class k. The variance is calculated across all classes as the average squared difference of each value from the mean.

$$\sigma^2 = 1 / (n-K) * \sum((x - \mu)^2)$$

Where σ^2 is the variance across all inputs (x), n is the number of instances, K is the number of classes and μ is the mean for input x.

Making Predictions with LDA: LDA makes predictions by estimating the probability that a new set of inputs belongs to each class. The class that gets the highest probability is the output

class and a prediction is made. The model uses Bayes Theorem to estimate the probabilities. Briefly Bayes' Theorem can be used to estimate the probability of the output class (k) given the input (x) using the probability of each class and the probability of the data belonging to each class:

$$P(Y=x|X=x) = (PI_k * f_k(x)) / \sum(PI_l * f_l(x))$$

Where PI_k refers to the base probability of each class (k) observed in your training data (e.g. 0.5 for a 50-50 split in a two class problem). In Bayes' Theorem this is called the prior probability.

$$PI_k = n_k/n$$

The $f(x)$ above is the estimated probability of x belonging to the class. A Gaussian distribution function is used for $f(x)$. Plugging the Gaussian into the above equation and simplifying we end up with the equation below. This is called a discriminate function and the class is calculated as having the largest value will be the output classification (y):

$$D_k(x) = x * (\mu_k/\sigma^2) - (\mu_k^2/(2*\sigma^2)) + \ln(PI_k)$$

$D_k(x)$ is the discriminate function for class k given input x, the μ_k , σ^2 and PI_k are all estimated from your data. The general LDA approach is very similar to a Principal Component Analysis but in addition to finding the component axes that maximize the variance of our data (PCA), we are additionally interested in the axes that maximize the separation between multiple classes (LDA).

How to Prepare Data for LDA: This section lists some suggestions you may consider when preparing your data for use with LDA.

-  **Classification Problems:** This might go without saying, but LDA is intended for classification problems where the output variable is categorical. LDA chains both binary and multi-class classification.
-  **Gaussian Distribution:** The standard implementation of the model assumes a Gaussian distribution of the input variables. Consider reviewing the univariate distributions of each attribute and using transforms to make them more Gaussian-looking (e.g. log and root for exponential allocations and Box-Cox for skewed distributions).
-  **Remove Outliers:** Consider removing outliers from your data. These can skew the basic statistics used to separate classes in LDA such the mean and the standard deviation.

- 📌 **Same Variance:** LDA assumes that each input variable has the same variance. It is almost always a good idea to standardize your data before using LDA so that it has a mean of 0 and a standard deviation of 1.

Extensions to LDA: Linear Discriminant Analysis is a simple and effective method for classification. Because it is simple and so well understood, there are numerous extensions and variations to the technique. Some popular extensions include:

- 📌 **Quadratic Discriminant Analysis (QDA):** Each class uses its own estimate of variance (or covariance when there are multiple input variables).
- 📌 **Flexible Discriminant Analysis (FDA):** Where non-linear mixtures of inputs is used such as splines.
- 📌 **Regularized Discriminant Analysis (RDA):** Presents regularization into the estimate of the variance (actually covariance), moderating the influence of different variables on LDA.

The original development was called the Linear Discriminant or Fisher's Discriminant Analysis. In general, dimensionality reduction does not only help reducing computational costs for a given classification task, but it can also be helpful to avoid overfitting by minimizing the error in parameter estimation.

2.11.11 K-Nearest Neighbour

With lots of Machine Learning algorithms out there it is often hard and confusing as to which to choose for our model especially for those who are just preparing for a big dive into data science. And no, I'm not talking about Model Selection here. Of all the algorithms there is, there has to be a simple algorithm that is well equipped and has the capability to overcome most of the complex of algorithms. KNN is one such. KNN which stands for K-Nearest Neighbors is a simple algorithm that is used for classification and regression problems in Machine Learning. KNN is also non-parametric which means the algorithm does not rely on strong assumptions instead tries to learn any functional form from the training data.

Unlike most of the algorithms with complex names, which are often confusing as to what they really mean, KNN is pretty straight forward. The algorithm considers the k nearest neighbors to predict the class or value of a data point. Let's try to understand it in a simple way. Consider a hypothetical situation in which there is a new student joining your class today say Class 5. None of your friends or class teachers knows him yet. Only the new student knows to which class he belongs. So he shows up during the assembly and finds his class and stands in line. Since he is standing in the line that has only class 5 students in it, he is recognized as a Class 5 student. KNN predicts in a very similar fashion. For example, if it was KNN predicting that

which class the student belongs to, it would simply consider the closest distance between the new student and the k number of students standing next to him. The k can be any number greater than or equal to 1. If it finds out that more number of students from class 5 are closer to him than any other classes, he is recognized as a Class 5 student by KNN.

While the above example depicts a classification problem, what if we were to predict a continuous value using KNN. Well, in this case, KNN simply finds the average of all the values of the neighboring data points and assign it to the new data point. Say for example if KNN is to predict the age of the new student in the above example, it calculates the average age of all its closest neighbors and assigns it to the new student. 'k' in KNN algorithm is based on feature similarity choosing the right value of K is a process called parameter tuning and is important for better accuracy. Finding the value of k is not easy.

Few ideas on picking a value for 'K'

- ✚ Firstly, there is no physical or biological way to determine the best value for "K", so we have to try out a few values before settling on one. We can do this by pretending part of the training data.
- ✚ Small values for K can be noisy and focus to the effects of outliers.
- ✚ Larger values of K will have smoother decision boundaries which mean lower variance but increased bias.
- ✚ Another way to choose K is through cross-validation. One way to select the cross-validation dataset from the training dataset. Take the small portion from the training dataset and call it a validation dataset, and then use the same to evaluate different possible values of K. This way we are going to predict the label for every instance in the validation set using with K equals to 1, K equals to 2, K equals to 3 and then we look at what value of K gives us the best performance on the validation set and then we can take that value and use that as the final setting of our algorithm so we are reducing the validation error.
- ✚ In general, practice, choosing the value of k is $k = \sqrt{N}$ where N stands for the number of samples in your training dataset.
- ✚ Try and keep the value of k odd in order to avoid confusion amid two classes of data

The Intuitive Steps For KNN: The K Nearest Neighbor Algorithm can be performed in 4 simple steps.

- ✚ Identify the problem as either falling to classification or regression.
- ✚ Fix a value for k which can be any number greater than zero.
- ✚ Now find k data points that are closest to the uncategorized/ unknown data point constructed on distance (Euclidean Distance, Manhattan Distance etc.)

- Find the solution in either of the following steps: If classification, assign the uncategorized datapoint to the class where the maximum number of neighbors belonged to or if regression, find the average value of all the closest neighbors and assign it as the value for the unknown data point.

2.11.12 K-means Clustering

K-means clustering is a type of unsupervised learning, which is used when you have unlabeled data (i.e., data without defined categories or groups). The goal of this algorithm is to find groups in the data, with the number of groups represented by the variable K. The algorithm works iteratively to assign each data point to one of K groups based on the features that are provided. Data points are clustered based on feature similarity. The results of the K-means clustering algorithm are:

- The centroids of the K clusters, which can be used to label new data.
- Labels for the training data (each data point is assigned to a single cluster).

Kmeans algorithm is an iterative algorithm that tries to partition the dataset into Kpre-defined distinct non-overlapping subgroups (clusters) where each data point belongs to only one group. It tries to make the inter-cluster data points as similar as possible while also keeping the clusters as different (far) as possible. It assigns data points to a cluster such that the sum of the squared distance between the data points and the cluster's centroid (arithmetic mean of all the data points that belong to that cluster) is at the least. The less variation we have within clusters, the more homogeneous (similar) the data points are within the same cluster.

The way kmeans algorithm works is as follows:

- Specify number of clusters K.
- Initialize centroids by first shuffling the dataset and then randomly selecting K data points for the centroids without replacement.
- Keep iterating until there is no change to the centroids. i.e., assignment of data points to clusters isn't changing.
- Compute the sum of the squared distance between data points and all centroids.
- Assign each data point to the closest cluster (centroid).
- Compute the centroids for the clusters by taking the average of the all-data points that belong to each cluster.

The approach kmeans follows to solve the problem is called Expectation-Maximization. The E-step is assigning the data points to the closest cluster. The M-step is computing the centroid of each cluster. Underneath is a breakdown of how we can solve it mathematically.

Choosing K:

- ✚ The algorithm described above finds the clusters and data set labels for a particular pre-chosen K. To find the number of clusters in the data, the user needs to run the K-means clustering algorithm for a range of K values and compare the results. In general, there is no method for determining exact value of K, but an accurate estimate can be obtained using the following techniques.
- ✚ One of the metrics that is commonly used to compare results across different values of K is the mean distance between data points and their cluster centroid. Since increasing the number of clusters will always reduce the distance to data points, increasing K will always decrease this metric, to the extreme of reaching zero when K is the same as the number of data points. Thus, this metric cannot be used as the sole target. Instead, mean distance to the centroid as a function of K is plotted and the "elbow point," where the rate of decrease sharply shifts, can be secondhand to roughly determine K.
- ✚ A number of other techniques exist for validating K, comprising cross-validation, information criteria, the information theoretic jump method, the silhouette method, and the G-means algorithm. In addition, monitoring the distribution of data points across groups provides insight into how the algorithm is splitting the data for each K.

Kmeans clustering is one of the most popular clustering algorithms and usually the first thing practitioners apply when solving clustering tasks to get an idea of the structure of the dataset. The goal of kmeans is to group data points into distinct non-overlapping subgroups. It does a very good job when the clusters have a kind of spherical shapes. However, it suffers as the geometric shapes of clusters deviates from spherical shapes. Moreover, it also doesn't learn the number of clusters from the data and requires it to be pre-defined. To be a good practitioner, it's good to know the norms behind algorithms/methods so that you would have a pretty good idea about the strength and weakness of each method. This will help you decide when to use each method and under what circumstances. In this section, we covered strength, weaknesses, and some evaluation methods related to kmeans. Below are the main takeaways:

- ✚ Scale/standardize the data when applying kmeans algorithm.
- ✚ Elbow method in selecting number of clusters doesn't usually work because the error function is monotonically decreasing for all k.
- ✚ Kmeans gives more weight to the bigger clusters.
- ✚ Kmeans assumes spherical shapes of clusters (with radius equal to the distance between the centroid and the furthest data point) and doesn't work well when clusters are in different shapes such as elliptical clusters.
- ✚ If there is overlapping between clusters, kmeans doesn't have an intrinsic measure for uncertainty for the examples belong to the overlapping region in order to determine for which cluster to assign each data point.

- ✚ Kmeans may still cluster the data even if it can't be clustered such as data that comes from uniform distributions.

Advantages of Principal Component Analysis

- ✚ Relatively simple to implement.
- ✚ Scales to large data sets.
- ✚ Guarantees convergence.
- ✚ Can warm-start the positions of centroids.
- ✚ Easily adapts to new examples.
- ✚ Generalizes to clusters of different shapes and sizes, such as elliptical clusters.

Disadvantages of Principal Component Analysis

- ✚ Choosing K manually.
- ✚ Being dependent on initial values.
- ✚ Clustering data of varying sizes and density.
- ✚ Clustering outliers.
- ✚ Scaling with number of dimensions.

2.11.13 Association Rules

Association Rules is one of the very vital concepts of machine learning being used in market basket analysis. In a store, all vegetables are placed in the same aisle, all dairy items are placed together and cosmetics form another set of such groups. Investing time and resources on deliberate product placements like this not only reduces a customer's shopping time, but also reminds the customer of what relevant items (s)he might be interested in purchasing, thus helping stores cross-sell in the process. Association rules help uncover all such relationships between items from huge databases. Rules do not extract an individual's preference, rather find relationships between set of elements of every distinct transaction. This is what makes them different from collaborative filtering.

To elaborate on this idea, rules do not tie back users' different transactions over time to identify relationships. List of items with unique transaction IDs (from all users) are studied as one group. This is helpful in placement of products on aisles. On the other hand, collaborative filtering ties back all transactions analogous to a user ID to identify similarity between users' preferences. This is helpful in recommending items on e-commerce websites, recommending songs on spotify, etc. Let's now see what an association rule exactly looks like. It consists of an antecedent and a consequent, both of which are a list of items. Note that implication here is co-occurrence and not causality. For a given rule, itemset is the list of all the items in the antecedent and the consequent.

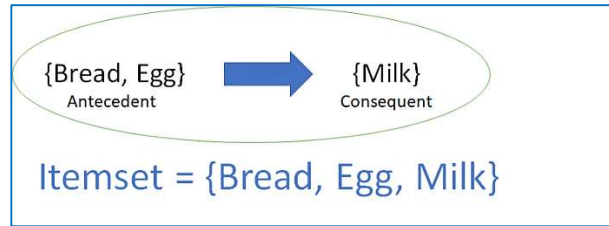


Fig. 2.23: Association Rule

Various metrics are in place to help us realize the strength of association between these two.

Let us go through them all.

Support: This measure gives an idea of how frequent an itemset is in all the transactions. Consider itemset1 = {bread} and itemset2 = {shampoo}. There will be far more transactions containing bread than those containing shampoo. So as you rightly guessed, itemset1 will generally have a higher support than itemset2. Now consider itemset1 = {bread, butter} and itemset2 = {bread, shampoo}. Many transactions will have both bread and butter on the cart but bread and shampoo? Not so much. So in this case, itemset1 will generally have a higher support than itemset2. Mathematically, support is the fraction of the total number of transactions in which the itemset occurs.

$$\text{Support}(\{X\} \rightarrow \{Y\}) = \frac{\text{Transactions containing both } X \text{ and } Y}{\text{Total number of transactions}}$$

Value of support helps us identify the rules worth considering for further analysis. For example, one might want to consider only the itemsets which occur at least 50 times out of a total of 10,000 transactions i.e. support = 0.005. If an itemset happens to have a very low support, we do not have enough information on the relationship between its items and hence no conclusions can be drawn from such a rule.

Confidence: This measure defines the likeliness of occurrence of resultant on the cart given that the cart already has the antecedents. That is to answer the question; of all the transactions containing say, {Captain Crunch}, how many also had {Milk} on them? We can say by common knowledge that {Captain Crunch} → {Milk} ought to be a high confidence rule. Precisely, confidence is the conditional probability of occurrence of consequent given the antecedent.

$$\text{Confidence}(\{X\} \rightarrow \{Y\}) = \frac{\text{Transactions containing both } X \text{ and } Y}{\text{Transactions containing } X}$$

Let us consider few more examples before moving ahead. What do you think would be the confidence for {Butter} → {Bread}? That is, what fraction of transactions having butter also had bread? Very high i.e. a value closes to 1? That's right. What about {Yogurt} → {Milk}? High again. {Toothbrush} → {Milk}? Not so sure? Confidence for this rule will also be high since {Milk} is such a frequent itemset and would be present in every other transaction. It does not matter what you have in the antecedent for such a frequent consequent. The confidence for an association rule having a very frequent consequent will always be high.

Lift: Lift controls for the support (frequency) of resultant while calculating the conditional probability of occurrence of {Y} given {X}. Lift is a very literal term given to this measure. Think of it as the *lift* that {X} provides to our confidence for having {Y} on the cart. To rephrase, lift is the rise in probability of having {Y} on the cart with the knowledge of {X} being present over the probability of having {Y} on the cart without any knowledge about presence of {X}. Mathematically,

$$Lift(\{X\} \rightarrow \{Y\}) = \frac{(Transactions\ containing\ both\ X\ and\ Y) / (Transactions\ containing\ X)}{Fraction\ of\ transactions\ containing\ Y}$$

In cases where {X} actually leads to {Y} on the cart, value of lift will be greater than 1. Let us understand this with an example which will be continuation of the {Toothbrush} → {Milk} rule. Probability of having milk on the cart with the knowledge that toothbrush is present (i.e. confidence): $10/(10+4) = 0.7$. Now to put this number in perspective, consider the probability of having milk on the cart without any knowledge about toothbrush: $80/100 = 0.8$. These numbers show that having toothbrush on the cart actually reduces the probability of having milk on the cart to 0.7 from 0.8! This will be a lift of $0.7/0.8 = 0.87$. Now that's more like the real picture. A value of lift less than 1 shows that having toothbrush on the cart does not increase the chances of occurrence of milk on the cart in spite of the rule showing a high confidence value. A value of lift greater than 1 vouches for high association between {Y} and {X}. More the value of lift, greater are the chances of preference to buy {Y} if the customer has already bought {X}. Lift is the measure that will help store managers to decide product placements on aisle.

Association Rule Mining: Now that we understand how to quantify the importance of association of products within an itemset, the next step is to generate rules from the entire list of items and identify the most important ones. This is not as simple as it might sound. Supermarkets will have thousands of different products in store. After some simple calculations, it can be shown that just 10 products will lead to 57000 rules!! And this number increases exponentially with the increase in number of items. Finding lift values for each of these will get computationally very expensive. How to deal with this problem? How to come

up with a set of most important association rules to be considered? Apriori algorithm comes to our rescue for this.

It is very essential for effective Market Basket Analysis and it helps in understanding which items customers will buy together. It has also been used in the field of healthcare for the discovery of adverse drug reactions. It generates association rules that show what all combinations of medications and patient characteristics which help in effective drug delivering. The main applications of Association Rules are in data analysis, classification, cross-marketing, clustering, catalogue design, loss-leader analysis, etc.

2.11.14 Hierarchical Clustering

The hierarchical clustering technique is one of the widespread clustering techniques in Machine Learning. Hierarchical clustering, also known as hierarchical cluster analysis, is an algorithm that groups similar objects into groups called clusters. The endpoint is a set of clusters, where each cluster is distinct from each other cluster, and the objects within each cluster are broadly similar to each other. Hierarchical clustering is one of the popular and easy to understand clustering technique. This clustering technique is divided into two types:

- ✚ Agglomerative
- ✚ Divisive

Agglomerative Hierarchical clustering Technique: In this method, initially each data point is considered as an individual cluster. At each iteration, the similar clusters merge with other clusters until one cluster or K clusters are formed. The basic algorithm of Agglomerative is straight forward.

- ✚ Compute the proximity matrix.
- ✚ Let each data point be a cluster.
- ✚ **Repeat:** Merge the two closest clusters and update the proximity matrix.
- ✚ Until only a single cluster remains.

The key operation is the computation of the proximity of two clusters. To understand better let's see a pictorial representation of the Agglomerative Hierarchical clustering Technique. Let's approximately have six data points {A,B,C,D,E,F}.

- ✚ In the initial step, we calculate the proximity of individual points and consider all the six data points as individual clusters as shown in the image below.
- ✚ In step two, similar clusters are merged together and formed as a single cluster. Let's consider B,C, and D,E are similar clusters that are merged in step two. Now, we're left with four clusters which are A, BC, DE, F.

- ✚ We again calculate the proximity of new clusters and merge the similar clusters to form new clusters A, BC, DEF.
- ✚ Calculate the proximity of the new clusters. The clusters DEF and BC are similar and merged together to form a new cluster. We're now left with two clusters A, BCDEF.
- ✚ To end with, all the clusters are merged together and form a single cluster (Figure 2.24).
- ✚ The Hierarchical clustering Technique can be visualized using a Dendrogram. A Dendrogram is a tree-like diagram that records the sequences of merges or splits (Figure 2.25).

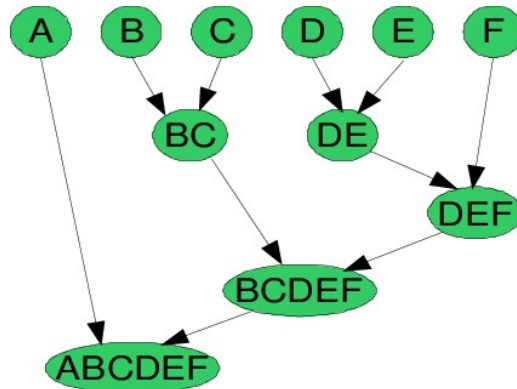


Fig. 2.24: Agglomerative Hierarchical Clustering Technique

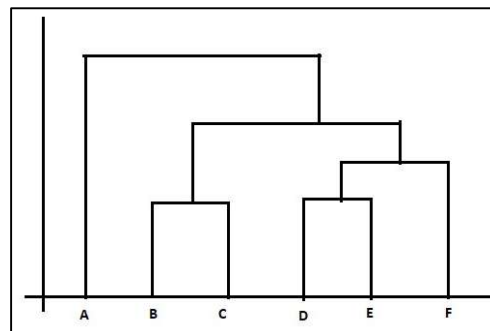


Fig. 2.25: Dendrogram Representation

Divisive Hierarchical clustering Technique: Since the Divisive Hierarchical clustering Technique is not much used in the real world; we will brief of the Divisive Hierarchical clustering Technique. In simple words, we can say that the Divisive Hierarchical clustering is exactly the opposite of the Agglomerative Hierarchical clustering. In Divisive Hierarchical clustering, we consider all the data points as a single cluster and in each iteration, we separate the data points from the cluster which are not similar. Each data point which is separated is

considered as an individual cluster. In the end, we'll be left with n clusters. As we're dividing the single clusters into n clusters, it is named as Divisive Hierarchical clustering.

Also known as top-down approach. This algorithm also does not require prespecifying the number of clusters. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been splitted into singleton cluster. Hierarchical clustering typically works by sequentially merging similar clusters. This is known as agglomerative hierarchical clustering. In theory, it can also be done by initially grouping all the observations into one cluster, and then successively splitting these clusters. This is known as divisive hierarchical clustering. Divisive clustering is rarely done in practice. Hierarchical clustering is a super useful way of segmenting observations. The advantage of not having to pre-define the number of clusters gives it quite an edge over k-Means.

Hierarchical Agglomerative Versus Divisive clustering

- 📊 Divisive clustering is more complex as compared to agglomerative clustering, as in case of divisive clustering we need a flat clustering method as “subroutine” to split each cluster until we have each data having its own singleton cluster.
- 📊 Divisive clustering is more efficient if we do not generate a complete hierarchy all the way down to individual data leaves. Time complexity of a naive agglomerative clustering is $O(n^3)$ because we exhaustively scan the $N \times N$ matrix for the lowest distance in each of $N-1$ iterations. Using priority queue data structure we can reduce this complexity to $O(n^2 \log n)$. By using some more optimizations it can be brought down to $O(n^2)$. While for divisive clustering given a fixed number of top levels, using an efficient flat algorithm like K-Means, divisive algorithms are linear in the number of patterns and clusters.
- 📊 Divisive algorithm is also more accurate. Agglomerative clustering makes decisions by considering the local patterns or neighbor points without initially taking into account the global distribution of data. These early decisions cannot be undone. whereas divisive clustering takes into consideration the global distribution of data when making top-level partitioning decisions.

2.11.15 Q-Learning

Q-learning is an off policy reinforcement learning algorithm that seeks to find the best action to take given the current state. It's considered off-policy because the q-learning function learns from actions that are outside the current policy, like taking random actions, and therefore a policy isn't needed. More specifically, q-learning seeks to learn a policy that maximizes the total reward. The 'q' in q-learning stands for quality. Quality in this case represents how useful

a given action is in gaining some future reward. When q-learning is performed we create what's called a q-table or matrix that follows the shape of [state, action] and we initialize our values to zero. We then update and store our q-values after an episode. This q-table becomes a reference table for our agent to select the best action based on the q-value.

Q-learning and making updates: The next step is simply for the agent to interact with the environment and make updates to the state action pairs in our q-table Q [state, action].

Taking Action: Explore or Exploit

An agent interacts with the environment in 1 of 2 ways. The first is to use the q-table as a reference and view all possible actions for a given state. The agent then selects the action based on the max value of those actions. This is known as exploiting since we use the information we have available to us to make a decision. The second way to take action is to act randomly. This is called exploring. Instead of selecting actions based on the max future reward we select an action at random. Acting randomly is important because it allows the agent to explore and discover new states that otherwise may not be selected during the exploitation process. You can balance exploration/exploitation using epsilon (ϵ) and setting the value of how often you want to explore versus exploit. Here's some rough code that will depend on how the state and action space are setup.

Updating the q-table: The updates occur after each step or action and ends when an episode is done. Done in this case means reaching some terminal point by the agent. A terminal state for example can be anything like landing on a checkout page, reaching the end of some game, finishing some desired objective, etc. The agent will not learn much after a single episode, but eventually with enough exploring (steps and episodes) it will converge and learn the optimal q-values or q-star (Q^*). Here are the three basic steps:

- 🧩 Agent starts in a state (s_1) takes an action (a_1) and receives a reward (r_1)
- 🧩 Agent selects action by referencing Q-table with highest value (max) OR by random (epsilon, ϵ)
- 🧩 Update q-values

Here is the basic update rule for q-learning: In the update above there are a couple variables that we haven't mentioned yet. What's happening here is we adjust our q-values based on the difference between the discounted new values and the old values. We discount the new values using gamma and we adjust our step size using learning rate (lr). Below are some references.

- 🧩 **Learning Rate:** lr or learning rate, often referred to as alpha or α , can simply be defined as how much you accept the new value vs the old value. Above we are taking

the difference between new and old and then multiplying that value by the learning rate. This value then gets added to our previous q-value which essentially moves it in the direction of our latest update.

- ✚ **Gamma:** gamma or γ is a discount factor. It's used to balance immediate and future reward. From our update rule above you can see that we apply the discount to the future reward. Typically, this value can range anywhere from 0.8 to 0.99.
- ✚ **Reward:** reward is the value received after completing a certain action at a given state. A reward can happen at any given time step or only at the terminal time step.
- ✚ **Max:** `np.max()` uses the numpy library and is taking the maximum of the future reward and applying it to the reward for the current state. What this does is impact the current action by the possible future reward. This is the beauty of q-learning. We're allocating future reward to current actions to help the agent select the highest return action at any given state.

At last...let us recap

- ✚ Q-Learning is a value-based reinforcement learning algorithm used to find the optimal action-selection policy using a Q function.
- ✚ Our goal is to maximize the value function Q.
- ✚ The Q table helps us to find the best action for each state.
- ✚ It helps to maximize the expected reward by selecting the best of all possible actions.
- ✚ Q (state, action) returns the expected future reward of that action at that state.
- ✚ This function can be estimated using Q-Learning, which iteratively updates $Q(s,a)$ using the Bellman equation.
- ✚ Initially, we reconnoiter the environment and update the Q-Table. When the Q-Table is ready, the agent will begin to exploit the environment and start taking better actions.

CHAPTER 3

MODERN DEEP LEARNING APPROACHES

Deep Learning is at the cutting edge of what machines can do, and developers and business leaders absolutely need to understand what it is and how it works. This unique type of algorithm has far surpassed any previous benchmarks for classification of images, text, and voice. It also powers numerous of the most interesting applications in the world, like autonomous vehicles and real-time translation. There was certainly a bunch of excitement around Google's Deep Learning based AlphaGo beating the best Go player in the world, but the business applications for this expertise are more immediate and potentially more impactful. This chapter will break down where Deep Learning fits into the ecosystem, how it works, and why it matters.

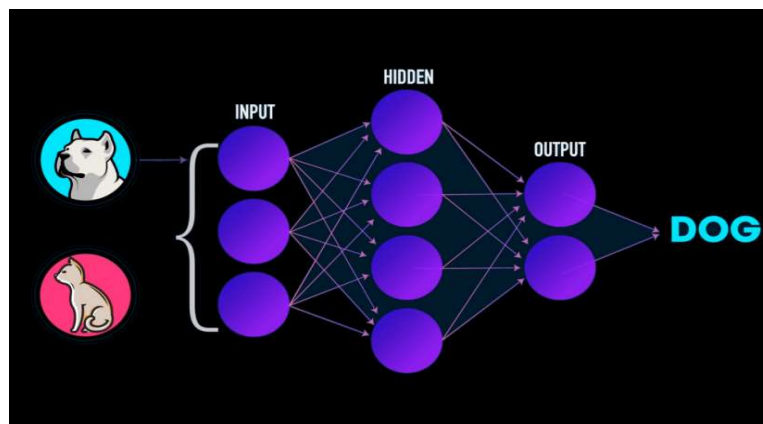


Fig. 3.1: A Simple Deep Learning Network

During recent years, deep learning has become somewhat of a buzzword in the tech community. We always seem to hear about it in news regarding AI, and yet most people don't actually know what it is! In this chapter, we'll be demystifying the buzzword that is deep learning, and providing an intuition of how it works. Deep learning is a machine learning method that takes in an input X , and uses it to predict an output of Y . As an example, given the stock prices of the past week as input, deep learning algorithm will try to predict the stock price of the next day. Given a large dataset of input and output pairs, a deep learning algorithm will try to minimize the difference between its prediction and expected output.

By doing this, it tries to learn the association/pattern between given inputs and outputs, this in turn allows a deep learning model to generalize to inputs that it hasn't seen before. As another example, let's say that inputs are images of dogs and cats, and outputs are labels for those

images (i.e. is the input picture a dog or a cat). If an input has a label of a dog, but the deep learning algorithm predicts a cat, then the deep learning algorithm will learn that the features of the given image (e.g. sharp teeth, facial features) are going to be linked with a dog (Figure 3.1). Deep learning is a specific subset of Machine Learning, which is a specific subset of Artificial Intelligence. For individual definitions:

- 🧩 Artificial Intelligence is the broad mandate of creating machines that can think intelligently.
- 🧩 Machine Learning is one way of doing that, by using algorithms to glean insights from data.
- 🧩 Deep Learning is one way of doing that, using a specific algorithm called a Neural Network.

Deep Learning is just a type of algorithm that seems to work really well for predicting things. Deep Learning and Neural Nets, for most purposes, are effectively synonymous. If people try to confuse you and argue about technical definitions, don't worry about it: like Neural Nets, labels can have many layers of meaning. Neural networks are inspired by the structure of the cerebral cortex. At the basic level is the perceptron, the mathematical representation of a biological neuron. Like in the cerebral cortex, there can be several layers of interconnected perceptrons. Input values, or in other words our underlying data, get passed through this "network" of hidden layers until they eventually converge to the output layer. The output layer is our prediction: it might be one node if the model just outputs a number or a few nodes if it's a multiclass classification problem. The hidden layers of a Neural Net perform modifications on the data to eventually feel out what its relationship with the target variable is. Each node has a weight, and it multiplies its input value by that weight. Do that over a few different layers and the Net is able to essentially manipulate the data into something meaningful.

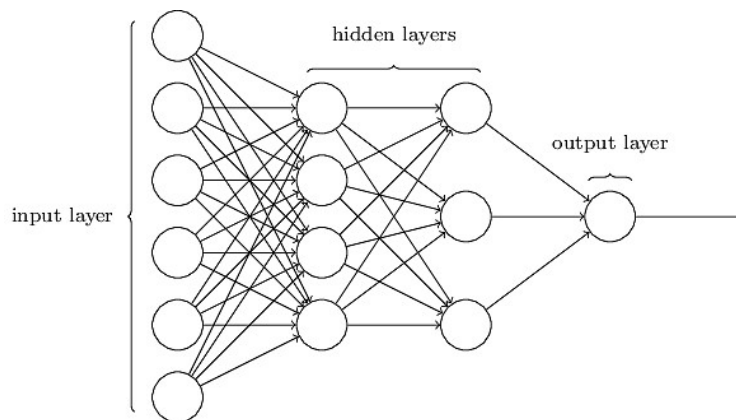


Fig. 3.2: Neural Networks and Deep Learning

In the 1980s, most neural networks were a single layer due to the cost of computation and availability of data. Nowadays we can afford to have more hidden layers in our Neural Nets, hence the moniker “Deep” Learning. The different types of Neural Networks available for use have also proliferated. Models like Convolutional Neural Nets, Recurrent Neural Nets, and Long Short-Term Memory are finding compelling use cases across the board.

3.1 WHAT IS DEEP LEARNING?

The field of artificial intelligence is essentially when machines can do tasks that typically require human intelligence. It encompasses machine learning, where machines can learn by experience and acquire skills without human involvement. Deep learning is a subset of machine learning where artificial neural networks, algorithms inspired by the human brain, learn from large amounts of data. Likewise to how we learn from experience, the deep learning algorithm would perform a task repeatedly, each time tweaking it a little to improve the outcome. We refer to ‘deep learning’ because the neural networks have various (deep) layers that enable learning. Just about any problem that requires “thought” to figure out is a problem deep learning can learn to solve. The amount of data we generate every day is staggering, currently estimated at 2.6 quintillion bytes and it’s the resource that makes deep learning possible.

Since deep-learning algorithms require a ton of data to learn from, this increase in data creation is one reason that deep learning capabilities have grown in recent years. In addition to more data creation, deep learning algorithms benefit from the stronger computing power that’s available today as well as the increase of Artificial Intelligence (AI) as a Service. AI as a Service has given smaller organizations access to artificial intelligence technology and specifically the AI algorithms required for deep learning without a large initial investment. Deep learning allows machines to solve complex problems even when using a data set that is very diverse, unstructured and inter-connected. The advanced deep learning algorithms learnt, the better they perform.

Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: learn by example. Deep learning is a key technology behind driverless cars, enabling them to recognize a stop sign, or to distinguish a pedestrian from a lamppost. It is the key to voice control in consumer devices like phones, tablets, TVs, and hands-free speakers. Deep learning is getting lots of attention lately and for good reason. It’s realizing results that were not possible before. In deep learning, a computer model learns to perform classification tasks directly from images, text, or sound. Deep learning models can achieve state-of-the-art accuracy, every so often exceeding human-level performance.

Models are trained by using a large set of labeled data and neural network architectures that contain many layers. Deep learning achieves recognition accuracy at higher levels than ever

before. This helps consumer electronics meet user expectations, and it is crucial for safety-critical applications like driverless cars. Recent advances in deep learning have improved to the point where deep learning outpaces humans in some tasks like classifying objects in images. While deep learning was first theorized in the 1980s, there are two main reasons it has only recently become useful:

- 🧩 **Deep learning requires large amounts of labeled data.** For example, driverless car development necessitates millions of images and thousands of hours of video.
- 🧩 **Deep learning requires large computing power.** High-performance GPUs have a parallel architecture that is efficient for deep learning. When combined with clusters or cloud computing, this enables development teams to moderate training time for a deep learning network from weeks to hours or less.

Computer vision is a prodigious example of a task that Deep Learning has transformed into something realistic for business applications. Using Deep Learning to classify and label images isn't only better than any other algorithms: it's starting to be better than actual humans. Deep learning don't require a human programmer to tell them what to do with the data. This is made possible by the extraordinary amount of data we collect and consume; data is the fuel for deep-learning models. Deep learning applications are castoff in industries from automated driving to medical devices.

- 🧩 **Automated Driving:** Automotive researchers are using deep learning to automatically detect objects such as stop signs and traffic lights. In addition, deep learning is employed to detect pedestrians, which helps decrease accidents.
- 🧩 **Aerospace and Defense:** Deep learning is used to identify objects from satellites that locate areas of interest, and identify safe or unsafe zones for troops.
- 🧩 **Medical Research:** Cancer researchers are using deep learning to automatically detect cancer cells. Teams at UCLA built an radical microscope that yields a high-dimensional data set used to train a deep learning application to accurately identify cancer cells.
- 🧩 **Industrial Automation:** Deep learning is helping to mend worker safety around heavy machinery by automatically detecting when people or objects are within an unsafe distance of machines.
- 🧩 **Electronics:** Deep learning is being used in automated hearing and speech translation. For example, home assistance devices that respond to your voice and know your preferences are powered by deep learning applications.

Deep learning algorithms run data through several “layers” of neural network algorithms, each of which passes a simplified representation of the data to the next layer. Most machine learning

algorithms work well on datasets that have up to a few hundred features, or columns. However, an unstructured dataset, like one from an image, has such a large number of features that this process becomes cumbersome or completely unfeasible. A single 800-by-1000-pixel image in RGB color has 2.4 million features, far too many for traditional machine learning algorithms to handle. Deep learning algorithms learn progressively more about the image as it goes through each neural network layer. Early layers learn how to detect low-level features like edges, and subsequent layers combine features from earlier layers into a more holistic representation. For example, a middle layer might identify edges to detect parts of an object in the photo such as a leg or a branch, while a deep layer will detect the full object such as a dog or a tree.

The ability to process large numbers of features makes deep learning very powerful when dealing with unstructured data. However, deep learning algorithms can be overkill for less complex problems because they require access to a vast amount of data to be effective. For instance, ImageNet, the common benchmark for training deep learning models for comprehensive image recognition, has access to over 14 million images. If the data is too simple or incomplete, it is very easy for a deep learning model to become overfitted and fail to generalize well to new data. As a result, deep learning models are not as effective as other techniques (such as boosted decision trees or linear models) for supreme practical business problems such as understanding customer churn, detecting fraudulent transactions and other cases with smaller datasets and fewer features. In convinced cases like multiclass classification, deep learning work for smaller, structured datasets.

Data Robot's automated machine learning platform includes support for deep learning and neural networks with technologies like TensorFlow. Additionally, Data Robot employs several cutting-edge techniques that make deep learning more effective on smaller, less complex datasets. Data Robot develops dozens of different types of models, giving businesses the ability to compare deep learning with various other techniques and decide for themselves which models perform the best for the specific problem at hand. Computer programs that use deep learning go through much the same process as the toddler learning to identify the dog. Each algorithm in the hierarchy applies a nonlinear transformation to its input and uses what it learns to create a statistical model as output. Iterations continue until the output has reached a suitable level of accuracy.

The number of processing layers through which data must pass is what inspired the label 'deep'. Deep learning programs require access to immense amounts of training data and processing power, neither of which were easily available to programmers until the era of big data and cloud computing. Because deep learning programming can create complex statistical models directly from its own iterative output, it is able to create accurate predictive models from large

quantities of unlabeled, unstructured data. This is important as the internet of things (IoT) continues to become more pervasive, because most of the data humans and machines create is unstructured and is not labeled.

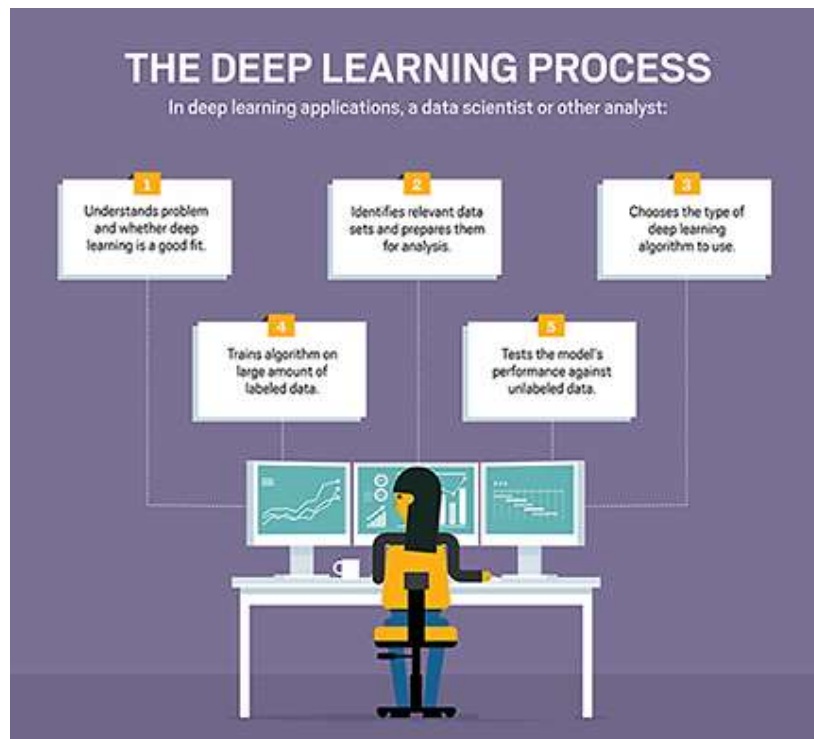


Fig. 3.3: The Deep Learning Process

3.2 WHERE TO APPLY DEEP LEARNING?

Deep Learning is gaining much popularity due to its supremacy in terms of accuracy when trained with huge amount of data. Deep learning technique learn categories incrementally through its hidden layer architecture, defining low-level categories like letters first then little higher level categories like words and then higher level categories like sentences. In the example of image recognition it means identifying light/dark areas before categorizing lines and then shapes to allow face recognition. Each neuron or node in the network represents one aspect of the whole and together they provide a full representation of the image. Each node or hidden layer is given a weight that represents the strength of its relationship with the output and as the model develops the weights are adjusted.

A big advantage with deep learning, and a key part in understanding why it's becoming popular, is that it's powered by massive amounts of data. The "Big Data Era" of machinery will provide

huge amounts of opportunities for new innovations in deep learning. As per Andrew Ng, the chief scientist of China's major search engine Baidu and one of the leaders of the Google Brain Project, "The analogy to deep learning is that the rocket engine is the deep learning models and the fuel is the huge amounts of data we can feed to these algorithms." Deep Learning requires high-end machines contrary to traditional Machine Learning algorithms. GPU has become an integral part now to execute any Deep Learning algorithm.

In traditional Machine learning techniques, most of the applied features need to be identified by a domain expert in order to reduce the complexity of the data and make patterns more visible to learning algorithms to work. The biggest advantage Deep Learning algorithms as discussed before are that they try to learn high-level features from data in an incremental manner. This eliminates the need of domain expertise and hard core feature extraction. Another major difference between Deep Learning and Machine Learning technique is the problem solving approach. Deep Learning techniques tend to solve the problem end to end, whereas Machine learning techniques need the problem statements to break down to different parts to be solved first and then their results to be combine at final stage. For example for a multiple object detection problem, Deep Learning techniques like Yolo net take the image as input and provide the location and name of objects at output.

But in usual Machine Learning algorithms like SVM, a bounding box object detection algorithm is required first to identify all possible objects to have the HOG as input to the learning algorithm in order to recognize relevant objects. Usually, a Deep Learning algorithm takes a long time to train due to large number of parameters. Popular ResNet algorithm takes about two weeks to train completely from scratch. However, traditional Machine Learning algorithms take few seconds to few hours to train. The scenario is entirely reverse in testing phase. At test time, Deep Learning algorithm takes much less time to run. Whereas, if you compare it with k-nearest neighbors (a type of machine learning algorithm), test time increases on increasing the size of data. Although this is not applicable on all machine learning algorithms, as some of them have small testing times too. Interpretability is the main issue why many sectors using other Machine Learning techniques over Deep Learning.

Let's take an example. Suppose we use deep learning to calculate the relevance score of a document. The performance it gives is quite excellent and is near human performance. But there's is an issue. It does not reveal why it has given that score. Indeed mathematically you can find out which nodes of a deep neural network were activated, but we don't know what there neurons were supposed to model and what these layers of neurons were doing collectively. So we fail to interpret the results. What is not the case of Machine Learning algorithms like decision trees, logistic regression etc. When to use Deep Learning or not over others?

- 📌 Deep Learning outperforms other techniques if the data size is large. However, with small data size, traditional Machine Learning algorithms are preferable.
- 📌 Deep Learning techniques need to have high end infrastructure to train in reasonable time.
- 📌 When there is lack of domain understanding for feature reflection, Deep Learning techniques outshines others as you have to worry less about feature engineering.
- 📌 Deep Learning really shines when it comes to complex problems such as image classification, natural language processing, and speech recognition.

Because deep learning models process information in ways similar to the human brain, they can be applied to many tasks people do. Deep learning is currently used in most common image recognition tools, natural language processing and speech recognition software. These tools are starting to appear in applications as diverse as self-driving cars and language translation services. Use cases today for deep learning include all types of big data analytics applications, especially those focused on natural language processing, language translation, medical diagnosis, stock-market trading signals, network security and image recognition. Specific fields in which deep learning is currently being used include the following:

- 📌 **Customer experience:** Deep learning models are already being used for chatbots. And, as it continues to mature, deep learning is expected to be implemented in various businesses to improve the customer experiences and increase customer satisfaction.
- 📌 **Text generation:** Machines are being taught the grammar and style of a piece of text and are then using this model to automatically create a completely new text matching the proper spelling, grammar and style of the original text.
- 📌 **Aerospace and military:** Deep learning is being used to detect objects from satellites that identify areas of interest, as well as safe or unsafe zones for troops.
- 📌 **Industrial automation:** Deep learning is improving worker safety in environments like factories and warehouses by providing services that automatically detect when a worker or object is getting too close to a machine.
- 📌 **Adding color:** Color can be added to black and white photos and videos using deep learning models. In the past, this was an extremely time-consuming, manual process.
- 📌 **Medical research:** Cancer researchers have started implementing deep learning into their practice as a way to automatically detect cancer cells.
- 📌 **Computer vision:** Deep learning has greatly enhanced computer vision, providing computers with extreme accuracy for object detection and image classification, restoration and segmentation.

Machine learning algorithms created by human programmers are liable for parsing and learning from the data. They make decisions based on what they learn from the data. Deep learning

learns through an artificial neural network that acts very much like a human brain and allows the machine to analyze data in a structure very much as humans do.

3.3 ITS CONNECTION WITH MACHINE LEARNING?

Deep learning is a specialized form of machine learning. A machine learning workflow starts with relevant features being manually extracted from images. The features are then used to create a model that categorizes the objects in the image. With a deep learning workflow, relevant features are automatically extracted from images. In addition, deep learning performs “end-to-end learning”, where a network is given raw data and a task to perform, such as classification, and it learns how to do this automatically. Another key difference is deep learning algorithms scale with data, whereas shallow learning converges. Shallow learning refers to machine learning methods that plateau at a certain level of performance when you add more examples and training data to the network. A key benefit of deep learning networks is that they often continue to improve as the size of your data increases. In machine learning, you manually choose features and a classifier to sort images. With deep learning, feature extraction and modeling steps are automatic.

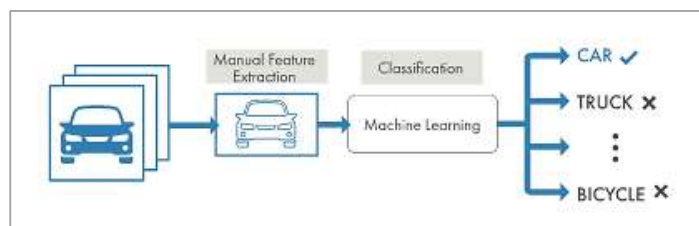


Fig. 3.4: Machine Learning Approach

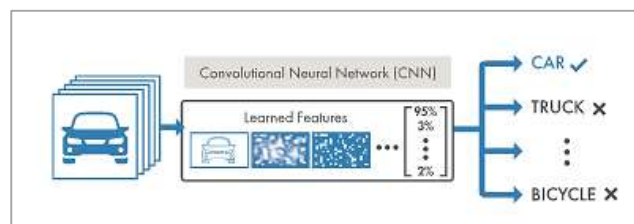


Fig. 3.5: Deep Learning Approach

Machine learning offers a variety of techniques and models you can choose based on your application, the size of data you're processing and the type of problem you want to solve. A successful deep learning application requires a very large amount of data (thousands of images) to train the model, as well as GPUs, or graphics processing units, to rapidly process your data.

When choosing between machine learning and deep learning, consider whether you have a high-performance GPU and lots of labeled data. If you don't have either of those things, it may make more sense to use machine learning instead of deep learning. Deep learning is generally more complex, so you'll need at least a few thousand images to get reliable results. Having a high-performance GPU means the model will take less time to analyze all those images.

Deep learning differentiates itself from machine learning through the way it solves problems. Machine learning requires a domain expert to identify most applied features. On the other hand, deep learning learns features incrementally, thus eliminating the need for domain expertise. This makes deep learning algorithms take much longer to train than machine learning algorithms, which only need a few seconds to a few hours. However, the reverse is true during testing. Deep learning algorithms take much less time to run tests than machine learning algorithms, whose test time increases along with the size of the data. Furthermore, machine learning does not require the same costly, high-end machines and high-performing GPUs that deep learning does. In the end, many data scientists choose traditional machine learning over deep learning due to its superior interpretability, or the ability to make sense of the solutions.

Machine learning algorithms are also favored when the data is small. Instances where deep learning becomes preferable include situations where there is a large amount of data, a lack of domain understanding for feature introspection or complex problems, such as speech recognition and natural language processing. Deep learning can trace its roots back to 1943 when Warren McCulloch and Walter Pitts fashioned a computational model for neural networks using mathematics and algorithms. However, it was not until the mid-2000s that the term deep learning started to appear. It gained popularity following the publication of a paper by Geoffrey Hinton and Ruslan Salakhutdinov that showed how a neural network with many layers could be trained one layer at a time. In 2012, Google made a huge impression on deep learning when its algorithm revealed the ability to recognize cats.

Two years later, in 2014, Google bought DeepMind, an artificial intelligence startup from the U.K. Two years after that, in 2016, Google DeepMind's algorithm, AlphaGo, mastered the complicated board game Go, beating professional player Lee Sedol at a tournament in Seoul. Recently, deep learning models have generated the majority of advances in the field of artificial intelligence. Deep reinforcement learning has emerged as a way to integrate AI with complex applications, such as robotics, video games and self-driving cars. The primary difference between deep learning and reinforcement learning is, while deep learning learns from a training set and then applies what is learned to a new data set, deep reinforcement learning learns dynamically by adjusting actions using continuous feedback in order to optimize the reward. A reinforcement learning agent has the ability to provide fast and strong control of generative adversarial networks (GANs).

The Adversarial Threshold Neural Computer (ATNC) blends deep reinforcement learning with GANs in order to design small organic molecules with a specific, desired set of pharmacological properties. GANs are also being used to generate artificial training data for machine learning tasks, which can be used in situations with imbalanced data sets or when data contains sensitive information. The biggest limitation of deep learning models is they learn through observations. This means they only know what was in the data on which they were trained. If a user has a small amount of data or it comes from one specific source that is not necessarily representative of the broader functional area, the models will not learn in a way that is generalizable. The issue of biases is also a major problem for deep learning models. If a model trains on data that contains biases, the model will reproduce those biases in its predictions. This has been a vexing problem for deep learning systems analyst, because models learn to differentiate based on subtle variations in data elements.

Often, the factors it determines are important are not made explicitly clear to the programmer. This means, for example, a facial recognition model might make determinations about people's characteristics based on things like race or gender without the programmer being aware. The learning rate can also become a major challenge to deep learning models. If the rate is too high, then the model will converge too quickly, producing a less-than-optimal solution. If the rate is too low, then the process may get stuck, and it will be even harder to reach a solution. The hardware requirements for deep learning models can also create limitations. Multicore high-performing graphics processing units (GPUs) and other similar processing units are required to ensure improved efficiency and decreased time consumption. However, these units are expensive and use large amounts of energy. Other hardware requirements include random access memory (RAM) and a hard drive or RAM-based solid-state drive (SSD). Other **limitations and challenges** include the following:

- 🧩 Deep learning requires large amounts of data. Furthermore, the more powerful and accurate models will need more parameters, which, in turn, requires more data.
- 🧩 Once trained, deep learning models become inflexible and cannot handle multitasking. They can deliver efficient and accurate solutions, but only to one specific problem. Even solving a similar problem would require retraining the system.
- 🧩 Any application that requires reasoning such as programming or applying the scientific method long-term planning and algorithmic like data manipulation is completely beyond what current deep learning techniques can do, even with large data.

In both machine learning and deep learning, engineers use software tools, such as MATLAB, to enable computers to identify trends and features in data by learning from an example data set. In the case of machine learning, training data is used to build a model that the computer can use to classify test data, and ultimately real-world data. Traditionally, an important step in

this workflow is the development of features, the additional metrics derived from the raw data which help the model be more accurate. Deep learning is a subset of machine learning, where engineers and scientists skip the manual step of creating features. Instead, the data are fed into the deep learning algorithm and it automatically learns what features are most useful to determine the output. Technically,

- 🧩 **Machine Learning:** A branch of artificial intelligence (AI) where engineers and scientists manually select features within the data and train the model. Common machine learning algorithms include decision trees, support vector machines, neural networks, and ensemble methods.
- 🧩 **Deep Learning:** A branch of machine learning modeled loosely on the neural pathways of the human brain where the algorithm automatically learns what features are useful. Common deep learning algorithms include convolutional neural networks (CNNs), recurrent neural networks, and deep Q networks.

Machine learning is typically used for projects that involve predicting an output or uncovering trends. In these examples, a limited body of data is used to help the machines learn patterns that they can later use to make a correct determination on new input data. Deep learning is more complex and is typically used for projects that involve classifying images, identifying objects in images, and enhancing images and signals. In these instances, a deep neural network can be applied, as they are designed to automatically extract features from spatially and temporally organized data such as images and signals. Machine learning algorithms may be more desirable if you need quicker results. They are faster to train and require less computational power. The number of features and observations will be the key factors that affect training time.

Engineers applying machine learning should expect to spend a majority of their time developing and evaluating features to improve model accuracy. Deep learning models will take time to train. Pretrained networks and public datasets can shorten training through transfer learning, but sometimes these can be complicated to implement. In general, deep learning algorithms can take anywhere from a minute to a few weeks to train depending on your hardware and computing power. Engineers applying deep learning should expect to spend a majority of their time training models and making modifications to the architecture of their deep neural network.

Consideration for Choosing Machine Learning vs. Deep Learning

(1) Data Considerations:

- 🧩 Understanding the available dataset can help determine whether machine learning or deep learning should be applied for a given task.

- ✚ Generally, machine learning is used when there is more limited, structured data available. Utmost machine learning algorithms are designed to train models to tabular data (organized into independent rows and columns). If the data are non-tabular, machine learning can be applied, but it does require some data manipulation i.e. sensor data can be converted into a tabular representation by extracting windowed features using common statistical metrics (mean, median, standard deviation, skewness, kurtosis, etc.), and then used with traditional machine learning techniques.
- ✚ Deep Learning typically requires a large quantity of training data to ensure that the network, which may very well have tens of millions of parameters and does not overfit the training data. Convolutional neural networks are designed to operate on image data, although they can be used on sensor data as well by performing a time-frequency calculation such as a spectrogram on the signal. Recurrent neural networks such as LSTM (Long Short-Term Memory) networks are designed to operate on sequential data such as signals and text.

(2) Available Hardware and Deployment:

- ✚ Determining which AI approach should be applied is also contingent on available hardware.
- ✚ Machine learning algorithms require less computational power. For example, desktop CPUs are sufficient for training these models.
- ✚ For deep learning models, specialized hardware is typically required due to the higher memory and compute requirements. Specialized hardware is also appropriate because the operations performed within a deep neural network, such as convolutions, lend themselves well to the parallel architecture of the GPU.
- ✚ Deep learning models take significant computing power. They should be considered if GPUs are available, or if there is time to run trainings on a CPU (which will take significantly longer).
- ✚ Training deep learning models on clusters or the cloud has gained popularity with deep learning due to the high costs associated with obtaining the GPUs. This option lets the hardware be shared by several researchers.

Deployment to embedded GPUs has also gained popularity, as it can provide fast inference speed in the deployed environment. GPU Coder enables code generation from deep learning models in MATLAB that leverages optimized libraries from Intel, NVIDIA and Arm. With GPU Coder Support Package for NVIDIA GPUs, you can cross-compile and deploy the generated CUDA code as a standalone application on an embedded GPU. While there will always be trial and error, the above can help guide decision making and accelerate the overall design process for engineers and scientists new to machine learning and deep learning. By

understanding the differences between machine learning and deep learning, knowing the end application of their project and factoring in data and hardware availability, design teams will gain faster insight into which approach fits best for their respective projects.

The rise of artificial intelligence in recent years is grounded in the success of deep learning. Three major drivers caused the breakthrough of deep neural networks: the availability of huge amounts of training data, powerful computational infrastructure, and advances in academia. Thereby deep learning systems start to outperform not only classical methods, but also human benchmarks in various tasks like image classification or face recognition. This creates the potential for many disruptive new businesses leveraging deep learning to solve real-world problems. Deep learning excels at identifying patterns in unstructured data, which most people know as media such as images, sound, video and text. Traditional machine learning has the advantage of feature introspection, that is, it knows why it is classifying an input in one way or another, which is important for analytics. But that very advantage is what excludes it from working with unlabeled and unstructured data, as well as attaining the record-breaking accuracy of the latest deep learning models.

Feature engineering is one of the major chokepoints of traditional machine learning, since so few people are able to do it well and quickly enough to adapt to changing data. For cases where feature introspection is necessary (e.g. the law requires that you justify a decision to, say, deny a loan due to predicted credit risk), we recommend using a deep net in an ensemble with machine-learning algorithms, allowing each one to vote and relying on each for its strength. Alternatively, one can perform various analyses on the results of deep nets to form hypotheses about their decision-making. Advances in reality capture and reality computing are making virtual and real worlds converge. One application of deep learning to this newly available data is to recognize and label objects in 3D environments, and in real life. From there, it's a short step to simulated semantics, in which machines learn the nature and constraints of objects in the world, through their virtual representations, and then bring that understanding to the language they generate and ingest. We believe that is one of many futures in store for neural nets.

3.4 NEURAL NETWORKS AND DEEP LEARNING?

A type of advanced machine learning algorithm, known as artificial neural networks, underpins most deep learning models. As a result, deep learning may sometimes be referred to as deep neural learning or deep neural networking. Neural networks come in several different forms, including recurrent neural networks, convolutional neural networks, artificial neural networks and feedforward neural networks and each has benefits for specific use cases. However, they all function in somewhat similar ways, by feeding data in and letting the model figure out for itself whether it has made the right interpretation or decision about a given data element.

Neural networks involve a trial-and-error process, so they need massive amounts of data on which to train. It's no coincidence neural networks became popular only after most enterprises embraced big data analytics and accumulated large stores of data. Because the model's first few iterations involve somewhat-educated guesses on the contents of an image or parts of speech, the data used during the training stage must be labeled so the model can see if its guess was accurate. This means, though many enterprises that use big data have large amounts of data, unstructured data is less helpful. Unstructured data can only be analyzed by a deep learning model once it has been trained and reaches an acceptable level of accuracy, but deep learning models can't train on unstructured data. Most deep learning methods use neural network architectures, which is why deep learning models are often referred to as deep neural networks. The term "deep" usually refers to the number of hidden layers in the neural network. Traditional neural networks only contain 2-3 hidden layers, while deep networks can have as many as 150.

Deep learning models are trained by using large sets of labeled data and neural network architectures that learn features directly from the data without the need for manual feature extraction. Neural networks are biologically-inspired programming model which enables a computer to learn from observational data. Deep learning is a powerful set of techniques for learning in neural networks. AI may have come on in leaps and bounds in the last few years, but we're still some way from truthfully intelligent machine, machines that can reason and make decisions like humans. Artificial neural networks (ANNs for short) may provide the answer to this.

Human brains are made up of connected networks of neurons. ANNs seek to simulate these networks and get computers to act like interconnected brain cells, so that they can learn and make decisions in a more humanlike manner. Different parts of the human brain are responsible for processing different pieces of information, and these parts of the brain are arranged hierarchically, or in layers. In this way, as information comes into the brain, each level of neurons processes the information, provides insight, and passes the information to the next, more senior layer. For example, your brain may process the delicious smell of pizza wafting from a street café in multiple stages: 'I smell pizza,' (that's your data input) ... 'I love pizza!' (thought) ... 'I'm going to get me some of that pizza' (decision making) ... 'Oh, but I promised to cut out junk food' (memory) ... 'Surely one slice won't hurt?' (reasoning) 'I'm doing it!' (action). It's this layered approach to processing information and making decisions that ANNs are trying to simulate.

In its simplest form, an ANN can have only three layers of neurons: the input layer (where the data enters the system), the hidden layer (where the information is processed) and the output layer (where the system decides what to do based on the data). But ANNs can get much more complex than that, and include multiple hidden layers. Whether it's three layers or more,

information flows from one layer to another, just like in the human brain. Deep learning on the other hand, represents the very cutting edge of artificial intelligence (AI). Instead of teaching computers to process and learn from data (which is how machine learning works), with deep learning, the computer trains itself to process and learn from data. This is all possible thanks to layers of ANNs. Remember that an ANN in its simplest form has only three layers? Well an ANN that is made up of more than three layers, i.e. an input layer, an output layer and multiple hidden layers – is called a ‘deep neural network’, and this is what underpins deep learning. A deep learning system is self-teaching, learning as it goes by filtering information through multiple hidden layers, in a similar way to humans. As you can see, the two are closely connected in that one relies on the other to function.

“Without neural networks, there would be no deep learning.

Ever since the advent of computers in particular and technology in general, the idea of creating intelligent systems has always fascinated us. The idea of something that can learn and adapt to changing circumstances and produce information and data relevant in solving real world problems has always been one of the top-most researched areas in the field of computer science. It’s not just because it would be awesome to have such a program that will adapt to changing parameters (though that is indeed a noble cause) but also because of how immense the real-world implications of such a system would be. The good news is that we have already made an amazing amount of progress in just the last couple years. This has been evident to everyone as given the amount of news coverage Artificial Intelligence and particularly Deep Learning has gotten over the last couple of years, even the most disinterested in the field would have read about it just to know what the hype is all about.

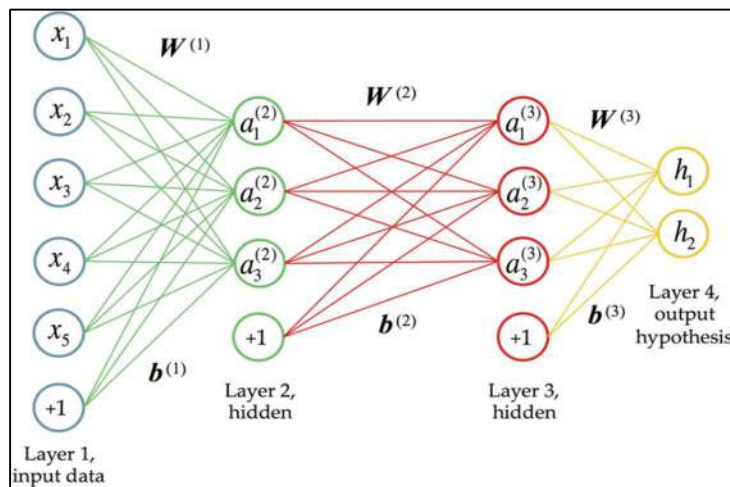


Fig. 3.6: A Deep Neural Network with Two Hidden Layers

One of the supreme popular types of deep neural networks is known as convolutional neural networks (CNN or ConvNet). A CNN convolves learned features with input data, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images. CNNs eliminate the need for manual feature extraction, so you do not need to identify features used to classify images. The CNN works by extracting features directly from images. The relevant features are not pretrained; they are learned while the network trains on a collection of images. This automated feature extraction makes deep learning models highly accurate for computer vision tasks such as object classification. CNNs learn to detect different features of an image using tens or hundreds of hidden layers. Every hidden layer increases the complexity of the learned image features. For example, the first hidden layer could learn how to detect edges, and the last learns how to detect more complex shapes specifically catered to the shape of the object we are trying to recognize.

Deep Learning (DL) is a powerful tool to make prediction an actionable result. Deep learning excels in pattern discovery (unsupervised learning) and knowledge-based prediction. Big data is the fuel for deep learning. When both are combined, an organization can reap unprecedented results in term of productivity, sales, management, and innovation. Deep learning can outperform traditional method. For instance, deep learning algorithms are 41% more accurate than machine learning algorithm in image classification, 27 % more accurate in facial recognition and 25% in voice recognition. Each Hidden layer in deep neural network is composed of neurons. The neurons are connected to each other. The neuron will process and then propagate the input signal it receives the layer above it. The strength of the signal given the neuron in the next layer depends on the weight, bias and activation function. The network consumes large amounts of input data and operates them through multiple layers; the network can learn increasingly complex features of the data at each layer. With deep neural networks, extremely complex problems of prediction and classification can be solved in very much the same way.

“This is the beauty of how such simple constructs can do such amazing jobs!

Deep learning is really good at learning, particularly in situations where the data is complex. In fact, artificial neural networks are known as universal function approximators because they’re able to learn any function, no matter how wiggly, with just a single hidden layer. The idea behind a deep neural network is to mimic a similar structure with layers of artificial neurons.

3.5 ROLE OF DEEP LEARNING IN COMPUTER VISION?

Computer vision can be succinctly described as finding and telling features from images to help discriminate objects and/or classes of objects. Computer vision has become one of the vital

research areas and the commercial applications bounded with the use of computer vision practices are becoming a huge portion in industry. The accuracy and the speed of processing and identifying images captured from cameras are has developed through decades. Being the well-known boy in town, deep learning is playing a major role as a computer vision tool. Deep learning came to the scene of computer vision couple of years back. Back then, computer vision was mainly based with image processing algorithms and methods.

The main process of computer vision was extracting the features of the image. Detecting the color, edges, corners and objects were the first step to do when performing a computer vision task. These features are human engineered and accuracy and the reliability of the models directly depend on the extracted features and on the methods used for feature extraction. The difficulty with this approach of feature extraction in image classification is that you have to choose which features to look for in each given image. When the number of classes of the classification goes high or the image clarity goes down it's really hard to cope up with traditional computer vision algorithms. Computer vision applications integrated with deep learning provide advanced algorithms with deep learning accuracy. This section will be dealt in detail in the forthcoming chapters.

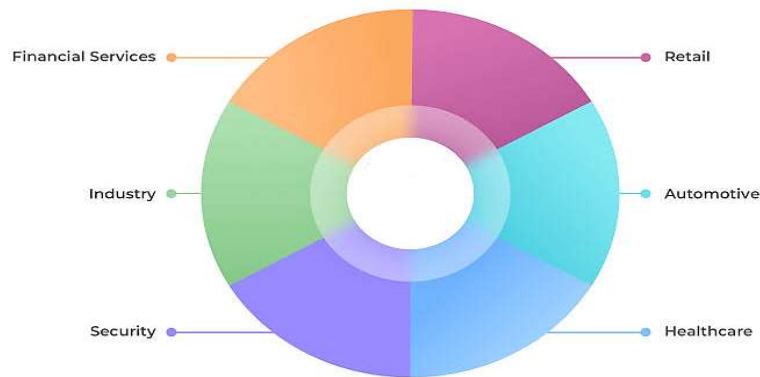


Fig. 3.7: Role of Deep Learning in Computer Vision Applications

3.6 WHAT ARE MODERN DEEP LEARNING ARCHITECTURE?

Modern Deep Learning Architecture comprises of such a diverse set of models in comparison to a single traditional machine learning algorithm. This is because of the flexibility that neural network provides when building a full-fledged end-to-end model. Neural network can sometimes be related with lego blocks, where you can build practically any simple to complex structure. It is becoming very hard to stay up to date with recent advances happening in deep learning. Hardly a day goes by without a new innovation or a new application of deep learning

coming by. We can describe an advanced architecture as one that has a proven track record of being a successful model. This is mainly seen in challenges like ImageNet, where your task is to solve a problem, say image recognition, using the data given. Those who don't know what ImageNet is, it is the dataset which is provided in ILSVR (ImageNet Large Scale Visual Recognition).

Also as described in the below mentioned architectures, each of them has a nuance which sets them apart from the usual models; giving them an edge when they are used to solve a problem. These architectures also fall in the category of “deep” models, so they are likely to perform better than their shallow counterparts. This section is mainly focused on Computer Vision, so it is natural to describe the horizon of computer vision tasks. Computer Vision; as the name suggests is simply creating artificial models which can replicate the visual tasks performed by a human. This essentially means what we can see and what we perceive is a process which can be implicit and implemented in an artificial system. Now that we have understood what an advanced architecture is and explored the tasks of computer vision, let us list down the most important architectures and their descriptions:

3.6.1 Google Net

The Google Net or Inception network was an important milestone in the development of CNN classifiers by 2014. Prior to its inception, most popular CNNs just stacked convolution layers deeper and deeper, hoping to get better performance. The Inception network on the other hand, was complex (heavily engineered). It used a lot of tricks to push performance; both in terms of speed and accuracy. Its constant evolution leads to the creation of several versions of the network. Each version is an iterative improvement over the previous one. Understanding the upgrades can help us to build custom classifiers that are optimized both in speed and accuracy. Also, depending on your data, a lower version may actually work better. This section aims to elucidate the evolution of the inception network.

Inception v1: Let us analyze what problem it was purported to solve, and how it was solved. The inceptor of GoogleNet or Inception network was expected to solve the following;

-  Salient parts in the image can have extremely large variation in size.
 -  Because of this huge variation in the location of the information, choosing the right kernel size for the convolution operation becomes tough. A larger kernel is preferred for information that is distributed more globally, and a smaller kernel is preferred for information that is distributed more locally.
 -  Very deep networks are prone to overfitting. It is also hard to pass gradient updates through the entire network.
 -  Naively stacking large convolution operations is computationally expensive.
-

For instance, consider the image with a dog can be either of the following, as shown in Figure 3.8. The area occupied by the dog is different in each image.



Fig. 3.8: Image of Dog with Large Variation in Size

Why not have filters with multiple sizes operate on the same level? The network essentially would get a bit “wider” rather than “deeper”. The authors designed the inception module to reflect the same. The below image is the “naive” inception module. It performs convolution on an input, with 3 different sizes of filters (1x1, 3x3, 5x5). Additionally, max pooling is also performed. The outputs are concatenated and sent to the next inception module. Deep neural networks are computationally affluent. To make it cheaper, limit the number of input channels by adding an extra 1x1 convolution before the 3x3 and 5x5 convolutions. Though adding an extra operation may seem counterintuitive, 1x1 convolutions are far cheaper than 5x5 convolutions, and the reduced number of input channels also helps. Do note that however, the 1x1 convolution is introduced after the max pooling layer, rather than before.

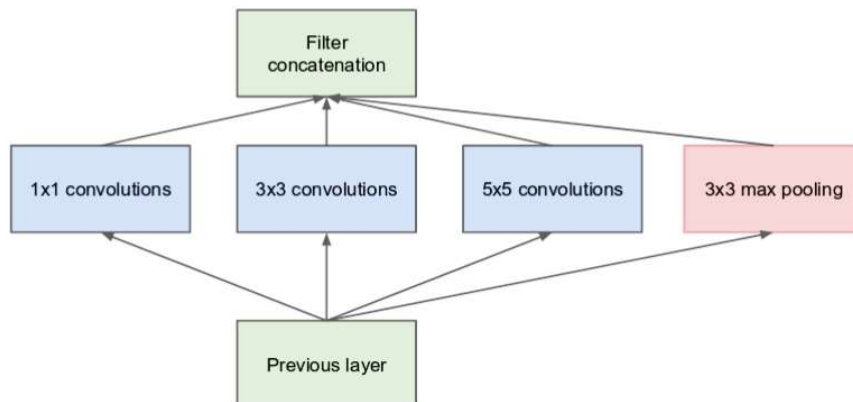


Fig. 3.9: The Naive Inception Module

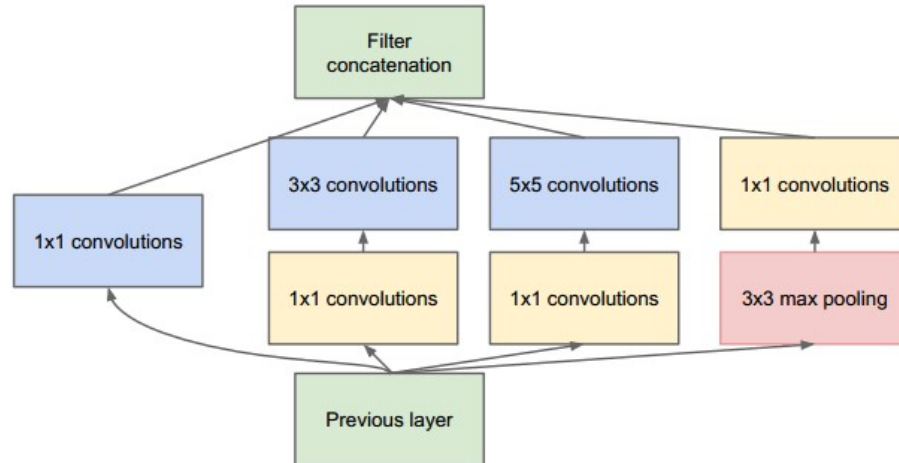


Fig. 3.10: Inception Module with Dimension Reduction

Using the dimension reduced inception module, neural network scheme was built. This was widely known as GoogleNet (Inception v1). GoogleNet has 9 such inception modules stacked linearly. It is 22 layers deep (27, including the pooling layers). It uses global average pooling at the end of the last inception module. Needless to say, it is a pretty deep classifier. As with any very deep network, it is subject to the vanishing gradient problem. To prevent the middle part of the network from “dying out”, two auxiliary classifiers were introduced. They essentially applied softmax to the outputs of two of the inception modules, and computed an auxiliary loss over the same labels. The total loss function is a weighted sum of the auxiliary loss and the real loss. Weight value used was 0.3 for each auxiliary loss.

Inception v2: A number of upgrades which increased the accuracy and reduced the computational complexity. Inception v2 explores the following:

- 🧩 **Reduce representational bottleneck:** The intuition was that, neural networks perform better when convolutions didn’t change the dimensions of the input drastically. Reducing the dimensions too much may cause loss of information, known as a “representational bottleneck”
- 🧩 Using smart factorization methods, convolutions can be made more efficient in terms of computational complexity.

Factorize 5x5 convolution to two 3x3 convolution operations to improve computational speed. Although this may seem counterintuitive, a 5x5 convolution is 2.78 times more expensive than a 3x3 convolution. So stacking two 3x3 convolutions infact leads to a boost in performance. Moreover, they factorize convolutions of filter size $n \times n$ to a combination of $1 \times n$ and $n \times 1$

convolutions. For example, a 3x3 convolution is equivalent to first performing a 1x3 convolution, and then performing a 3x1 convolution on its output. They found this method to be 33% more cheaply than the single 3x3 convolution. The filter banks in the module were expanded (made wider instead of deeper) to remove the representational bottleneck. If the module was made deeper instead, there would be excessive reduction in dimensions, and hence loss of information. The above three principles were used to build three different types of inception modules with better accuracy and reduced the computational complexity.

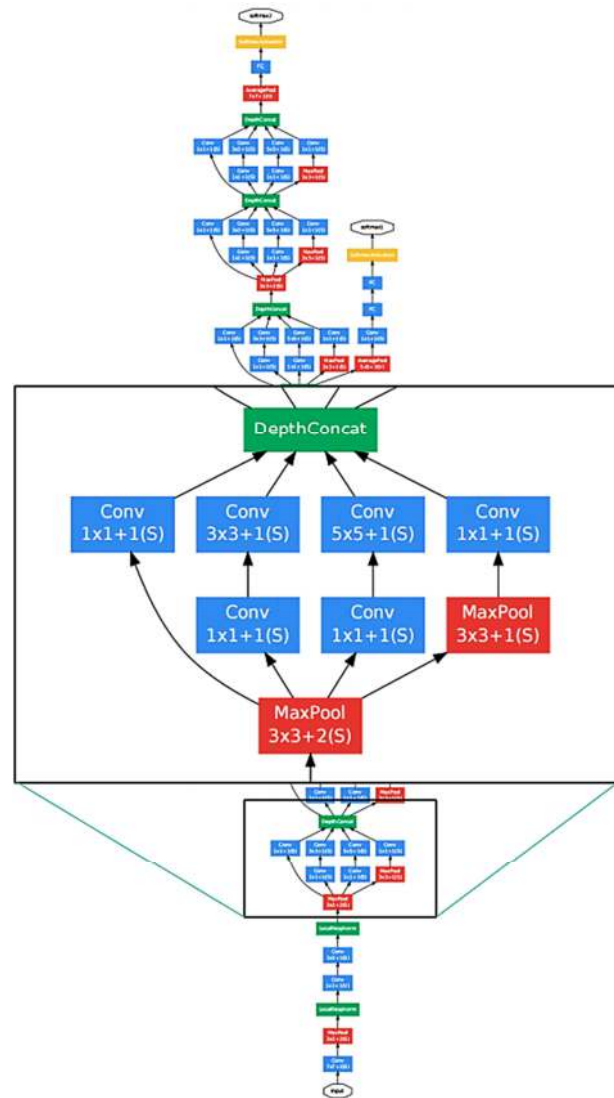


Fig. 3.11: Google Net (Inception v1)

Inception v3: The auxiliary classifiers didn't contribute much until near the end of the training process, when accuracies were nearing saturation. It can be argued that they function as regularizers, chiefly if they have BatchNorm or Dropout operations. The possibilities available to evolution on the Inception v2 without drastically changing the modules were to be inspected. Inception Net v3 assimilated all of the above upgrades stated for Inception v2, and in addition used the following:

- ✚ RMSProp Optimizer.
- ✚ Factorized 7x7 convolutions.
- ✚ BatchNorm in the Auxiliary Classifiers.
- ✚ Label Smoothing (A type of regularizing component added to the loss formula that averts the network from becoming too confident about a class. Prevents over fitting).

Inception v4: Make the modules more uniform. It was identified that some of the modules were more complicated than necessary. This can enable us to boost performance by adding more of these uniform modules. The following solutions were encountered;

- ✚ The “stem” of Inception v4 was modified. The stem here refers to the initial set of operations performed before introducing the Inception blocks.
- ✚ Three main inception components those look very similar to the Inception v2 (or v3) counterparts.
- ✚ Inception v4 introduced specialized “Reduction Blocks” which are used to change the width and height of the grid. The earlier versions didn't clearly have reduction blocks, but the functionality was implemented.

Inception ResNet v1 and v2: Inspired by the performance of the ResNet, a hybrid inception module was designed. There are two sub-versions of Inception ResNet, namely v1 and v2. Before checking out the prominent features, let us look at the minor variances between these two sub-versions.

- ✚ Inception-ResNet v1 has a computational cost that is similar to that of Inception v3.
- ✚ Inception-ResNet v2 has a computational cost that is similar to that of Inception v4.
- ✚ They have different stems, as illustrated in the Inception v4.
- ✚ Both sub-versions have the same structure for the three modules and the reduction blocks. Only difference is the hyper-parameter settings. In this section, we'll only focus on the structure.

These versions aim at introducing residual connections that adds the output of the convolution operation of the inception module to the input. It was found that Inception-ResNet models were able to achieve higher accuracies at a lower epoch. To fit the entire model on a single GPU (without BatchNorm after summation), the following solutions were struck;

- ✚ For residual addition to work, the input and the output after convolution must have the identical dimensions. Therefore, 1x1 convolutions after the original convolutions was used to match the depth sizes (Depth is increased after convolution).
- ✚ The pooling operation inside the main inception modules were replaced in favor of the residual connections. However, you can still find those operations in the reduction blocks. Reduction block is same as that of Inception v4.
- ✚ Networks with residual units deeper in the architecture caused the network to “die” if the number of filters exceeded 1000. Hence, to increase stability, the residual activations were scaled by a value around 0.1 to 0.3.

GoogLeNet achieved a top-5 error rate of 6.67%! This was very close to human level performance. As it turns out, this was actually rather hard to do and needs some human training in order to beat GoogLeNets accuracy.

3.6.2 Alex Net

Alex Net introduced in 2012, solves the problem of image classification where the input is an image of one of 1000 different classes (e.g. cats, dogs etc.) and the output is a vector of 1000 numbers. The i th element of the output vector is inferred as the probability that the input image belongs to the i th class. Therefore, the sum of all elements of the output vector is 1. The input to Alex Net is an RGB image of size 256×256 . This means all images in the training set and all test images need to be of size 256×256 . If the input image is not 256×256 , it needs to be converted to 256×256 before using it for training the network. To achieve this, the smaller dimension is resized to 256 and then the resulting image is cropped to obtain a 256×256 image. If the input image is grayscale, it is converted to an RGB image by replicating the single channel to obtain a 3-channel RGB image. Random crops of size 227×227 were generated from inside the 256×256 images to feed the first layer of Alex Net. Alex Net was much larger than previous CNNs used for computer vision tasks.

It has 60 million parameters and 650,000 neurons and took five to six days to train on two GTX 580 3GB GPUs. Today there are much more complex CNNs that can run on faster GPUs very efficiently even on very large datasets. But back in 2012, this was huge! Alex Net consists of 5 Convolutional Layers and 3 Fully Connected Layers. Multiple Convolutional Kernels (filters) extract interesting features in an image. In a single convolutional layer, there are usually many kernels of the same size. For example, the first Conv Layer of Alex Net contains 96 kernels of size $11 \times 11 \times 3$. Note the width and height of the kernel are usually the same and the depth is the same as the number of channels. The first two Convolutional layers are followed by the Overlapping Max Pooling layers. The third, fourth and fifth convolutional layers are coupled directly. The fifth convolutional layer is followed by an Overlapping Max Pooling

layer, the output of which goes into a series of two fully connected layers. The second fully connected layer feeds into a softmax classifier with 1000 class labels. ReLU nonlinearity is applied after the entire convolution and fully connected. The ReLU nonlinearity of the first and second convolution layers are followed by a local normalization step before doing pooling.

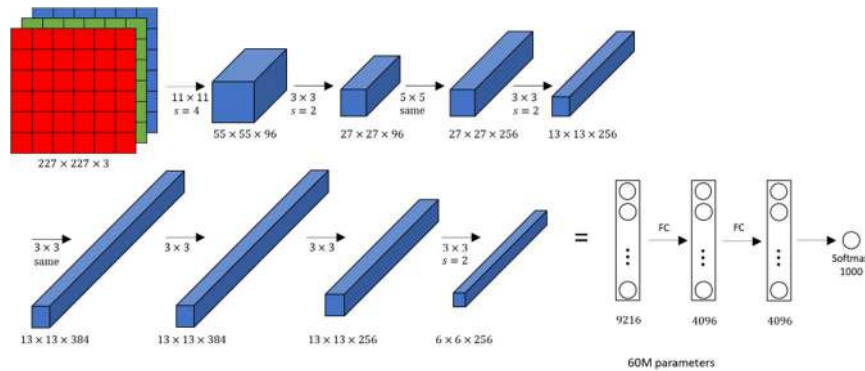


Fig. 3.12: Alex Net Architecture

Max Pooling layers are usually used to down sample the width and height of the tensors, keeping the depth same. Overlapping Max Pool layers are similar to the Max Pool layers, except the adjacent windows over which the max is computed overlap each other. The authors used pooling windows of size 3×3 with a stride of 2 between the adjacent windows. This overlapping nature of pooling helped reduce the top-1 error rate by 0.4% and top-5 error rate by 0.3% respectively when compared to using non-overlapping pooling windows of size 2×2 with a stride of 2 that would give same output dimensions. A significant characteristic of the AlexNet is the practice of ReLU (Rectified Linear Unit) Nonlinearity. Tanh or sigmoid activation functions used to be the standard way to train a neural network model.

AlexNet revealed that using ReLU nonlinearity, deep CNNs could be trained much faster than using the saturating activation functions like tanh or sigmoid. Remember the kid from your middle school class who did very well in tests, but did poorly whenever the questions on the test required original thinking and were not covered in the class? Why did he do so poorly when confronted with a problem he had never seen before? Because he had memorized the answers to questions covered in the class without understanding the underlying concepts. Similarly, the size of the Neural Network is its capacity to learn, but if you are not careful, it will try to memorize the examples in the training data without understanding the concept. As a result, the Neural Network will work exceptionally well on the training data, but they fail to learn the real concept. It will fail to work well on new and unseen test data. This is called **overfitting**. AlexNet reduced overfitting using a couple of different methods.

📦 **Data Augmentation:** Showing a Neural Net different variation of the same image helps prevent overfitting. You are forcing it to not memorize! Often it is possible to generate additional data from existing data for free! Here are few tricks used by the AlexNet like Mirroring and Random Crops.

📦 **Dropout:** In dropout, a neuron is dropped from the network with a probability of 0.5. When a neuron is dropped, it does not contribute to either forward or backward propagation. So every input goes through different network architecture. As a result, the learnt weight parameters are more robust and do not get overfitted easily.

The recent availability of large datasets like ImageNet, which consist of hundreds of thousands to millions of labeled images, have pushed the need for an extremely capable deep learning model like AlexNet.

3.6.3 VGG Net

VGG16 is a convolutional neural network model proposed by K. Simonyan and A. Zisserman in 2014 from the University of Oxford in the paper “Very Deep Convolutional Networks for Large-Scale Image Recognition”. The model achieves 92.7% top-5 test accuracy in ImageNet, which is a dataset of over 14 million images belonging to 1000 classes. It was one of the famous models submitted to ILSVRC-2014. It makes the improvement over AlexNet by replacing large kernel-sized filters (11 and 5 in the first and second convolutional layer, respectively) with multiple 3×3 kernel-sized filters one after another. VGG16 was trained for weeks and was using NVIDIA Titan Black GPU’s. ImageNet is a dataset of over 15 million labelled high-resolution images belonging to roughly 22,000 categories.

The images were collected from the web and labelled by human labellers using Amazon’s Mechanical Turk crowd-sourcing tool. Starting in 2010, as part of the Pascal Visual Object Challenge, an annual competition called the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) has been held. ILSVRC uses a subset of ImageNet with roughly 1000 images in each of 1000 categories. In all, there are roughly 1.2 million training images, 50,000 validation images, and 150,000 testing images. ImageNet consists of variable-resolution images. Therefore, the images have been down-sampled to a fixed resolution of 256×256. Given a rectangular image, the image is rescaled and cropped out the central 256×256 patch from the resulting image. The architecture depicted below is VGG16.

The input to cov1 layer is of fixed size 224 x 224 RGB image. The image is passed through a stack of convolutional (conv.) layers, where the filters were used with a very small receptive field: 3×3 (which is the smallest size to capture the notion of left/right, up/down, center). In one of the configurations, it also utilizes 1×1 convolution filters, which can be seen as a linear transformation of the input channels (followed by non-linearity). The convolution stride is

fixed to 1 pixel; the spatial padding of conv. layer input is such that the spatial resolution is preserved after convolution, i.e. the padding is 1-pixel for 3×3 conv. layers. Spatial pooling is carried out by five max-pooling layers, which follow some of the conv-layers (not all the conv-layers are followed by max-pooling). Max-pooling is performed over a 2×2 pixel window, with stride 2. Three Fully-Connected (FC) layers follow a stack of convolutional layers (which has a different depth in different architectures): the first two have 4096 channels each, the third performs 1000-way ILSVRC classification and thus contains 1000 channels (one for each class). The final layer is the soft-max layer.

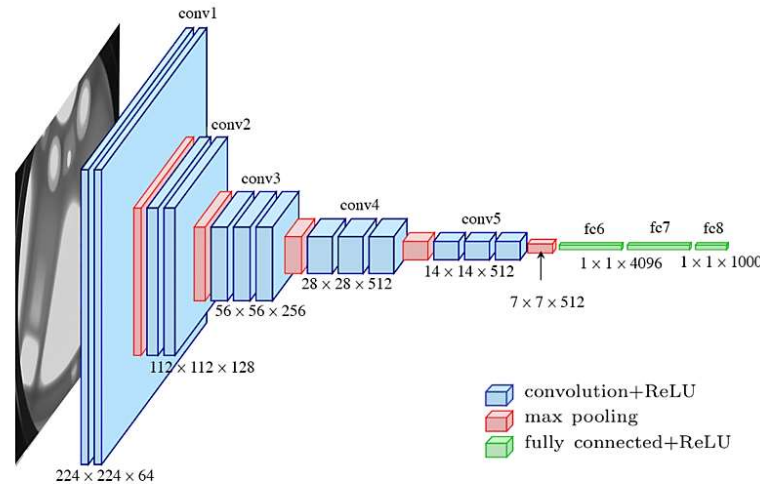


Fig. 3.13: VGG16 Architecture

The configuration of the fully connected layers is the same in all networks. The VGG16 and VGG19 stand for the number of weight layers in the network. All hidden layers are armed with the rectification (ReLU) non-linearity. It is also noted that none of the networks (except for one) contain Local Response Normalisation (LRN); such normalization does not improve the performance on the ILSVRC dataset, but leads to increased memory consumption and computation time.

The ConvNet configurations are outlined in Table 3.1. The nets are referred to their names (A-E). All configurations follow the generic design present in architecture and differ only in the depth: from 11 weight layers in the network A (8 conv. and 3 FC layers) to 19 weight layers in the network E (16 conv. and 3 FC layers). The width of conv. layers (the number of channels) is rather small, starting from 64 in the first layer and then increasing by a factor of 2 after each max-pooling layer, until it reaches 512. Unfortunately, there are two major drawbacks with VGGNet:

- It is painfully slow to train.
- The network architecture weights are quite large (concerning disk/ bandwidth).

Due to its depth and number of fully-connected nodes, VGG16 is over 533MB. This makes deploying VGG a tiresome task. VGG16 is used in many deep learning image classification problems; but, smaller network architectures are often more desirable (such as SqueezeNet, GoogLeNet, etc.). But it is a great building block for learning purpose as it is easy to implement. VGG16 significantly outperforms the previous generation of models. The VGG16 result is also competing for the classification task winner (GoogLeNet with 6.7% error) and substantially outperforms the ILSVRC-2013 winning submission Clarifai, which achieved 11.2% with external training data and 11.7% without it.

Table 3.1. Conv Net Configuration of VGG Net

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Concerning the single-net performance, VGG16 architecture achieves the best result (7.0% test error), outperforming a single GoogLeNet by 0.9%. It was demonstrated that the representation

depth is beneficial for the classification accuracy, and that state-of-the-art performance on the ImageNet challenge dataset can be achieved using a conventional ConvNet architecture with substantially increased depth. The smaller networks converged and were then used as initializations for the larger, deeper networks, this process is called pre-training. While making logical sense, pre-training is a very time consuming, tedious task, requiring an entire network to be trained before it can serve as an initialization for a deeper network. Due to its depth and the number of fully-connected nodes, VGG is over 533MB for VGG16 and 574MB for VGG19. This makes deploying VGG a tiresome task.

3.6.4 ZFNet

ZFNet introduced a way to visualize learned convolutional feature in 2013. Prior, convolutional neural networks were pretty much black boxes, whose internal workings were unknown. ZFNet offered essential insights into how convolutional neural networks are learning internal representations. To achieve this goal, ZFNet presents a way to map learned features into input pixel space by using a specially aimed Deconvolutional Network. Not to confuse with deconvolutions in deep learning and deconvolutions in signal processing, they are very different. In deep learning, deconvolutions are also named as the transposed convolution and partially strided convolutions. Convolution allows going from a specified input dimension to some output dimension. Note that the shape of the output dimension is defined by parameters of a given convolution: the input size, kernel size, strides, and padding. Deconvolution (transposed convolution) countenances going in the opposite direction: from the output dimension go back to the input dimension. Why would such operation be necessary? There are two reasons:

- 📦 Deconvolution is used during back-propagation of error through the convolutional layer
- 📦 Deconvolution is used for up-scaling of input in specific deep learning applications such as super resolution and hourglass networks, to name a few.

Backpropagation for the convolutional layer is a deconvolution operation applied to the incoming gradient of the convolutional layer. If you look at convolution task as the matrix multiplication of input by a convolution matrix C (defined by the convolution kernel weights), then backpropagating the error is equivalent to multiplying the incoming gradient by the transpose of the convolution matrix C . This is why deconvolution is also known as transposed convolution. Remember that multiplying by the transposed convolution matrix, C^T , is equivalent to deconvolution with filters rotated by 180 degrees. As for using deconvolutions in upscaling, you perform deconvolution but learn new filters, so that convolutional filters from preceding layers don't need to be reused. Visualizations produced are reconstructed patterns

from an input image coming from the validation set that causes high activations in a given feature map.

To produce input images that result in maximum activations, a separate Deconvnet is attached to the output of each layer whose features we want to visualize. Each Deconvnet itself is a neural network whose input is activations of a particular layer, and the output is an image depicting pixels that are responsible for maximum activation of a convolutional kernel of our interest for a given convolutional layer. Deconvnet is described as a sequence of transposed convolutions, de-pooling and a special modified ReLU. The sequence of operations is the reverse of the steps that were used in the original neural network to produce a particular layer's output. The output of the Deconvnet has the same dimensions as the input image in the convnet we are visualizing.

Deconvnet is a slightly modified backward pass through a convnet that you are trying to visualize whose output is an image that produces the maximum activation of a selected convolutional kernel for a selected convolutional layer. That image lets you understand what kind of feature this kernel learned to recognize. The visualizations above are fascinating as they offer insights into the internal representation of knowledge learned by the network. It is essential to understand what precisely they tell us. In other words, visualization of activations is produced for each image separately. Another critical thing to note is that the size of each of these projections is equal to each neuron's receptive field upon the input image. Note that neurons in layers closer to the image have a smaller receptive field on the input image, and the further you go away from the image, the larger the receptive field becomes. Receptive fields recall that each convolution kernel slides across the input space and for each position it produces 1 activation. As a result, we have a 2D feature map for this kernel, consisting of numbers representing neuron activations.

It is possible to find the highest activation, and then trace back receptive fields through the network and find at which patch of the input image this particular highly activated neuron is looking. These patches are on the right side of the panel of each layer. Lastly, it is worth mentioning that the higher you go in the network (further away from the input), the more complex representations become. They increase complexity by combining more straightforward features from layers below. For example, if layer 2 detects circles, then layer 5 use those detected circles when detecting dog faces because dog eyes look like circles. Overall, the architecture of the network is an optimized version of AlexNet. ZFNet overcomes the negatives of AlexNet in realizing superior performance.

In particular, they reduced filter size in the first convolutional layer from 11x11 to 7x7, which resulted in fewer dead features learned in the first layer. A dead feature is a situation where a

convolutional kernel fails to learn any significant representation. Visually it looks like a monotonic single-color image, where all the values are close to each other. In addition to changing the filter size, ZFNet have doubled the number of filters in all convolutional layers and the number of neurons in the fully connected layers as compared to the AlexNet. In the AlexNet, there were 48-128-192-192-128-2048-2048 kernels/neurons, and in the ZFNet all these doubled to 96-256-384-384-256-4096-4096. This modification allowed the network to increase the complexity of internal representations and as a result, decrease the error rate from 15.4%.

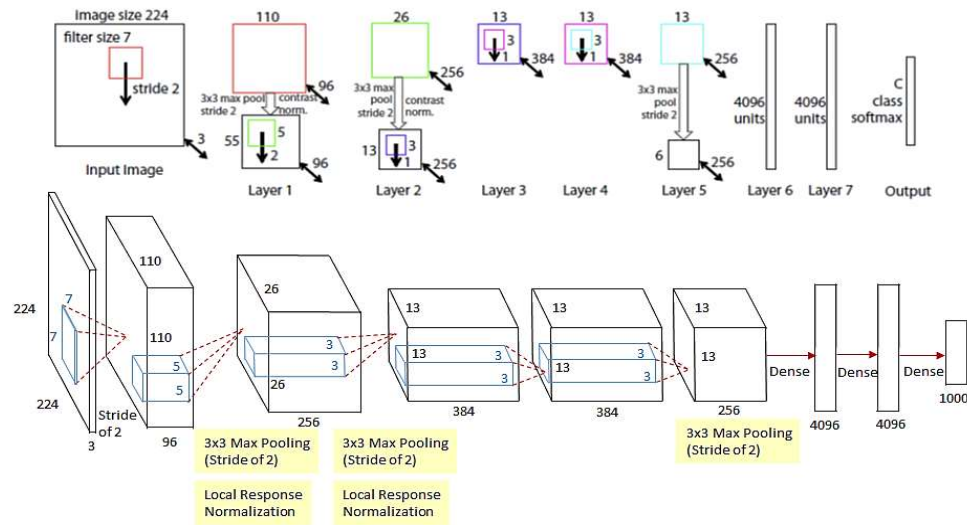


Fig. 3.14: ZFNet Architecture

The foremost notion behind ZFNet is deconvolution, transfer learning (supervised pretraining), learning rate annealing with momentum. ZFNet won the ImageNet competition in 2013, achieving the error rate of 14.8% (compared to 15.4% in the previous year). It was mostly an achievement by tweaking the hyper-parameters of AlexNet while maintaining the same structure with additional Deep Learning elements. The intuition behind this is that by using bigger filters we were losing a lot of pixel information, which we can retain by having smaller filter sizes in the earlier conv-layers. The number of filters increases as we go deeper. This network also used ReLUs for their activation and was trained using batch stochastic gradient descent.

3.6.5 ResNet

Researchers observed that it makes sense to affirm that “the deeper the better” when it comes to convolutional neural networks. This makes sense, since the models should be more capable

(their flexibility to adapt to any space increase because they have a bigger parameter space to explore). However, it has been noticed that after some depth, the performance degrades. This was one of the bottlenecks of VGG. They couldn't go as deep as wanted, because they started to lose generalization capability. One of the problems ResNets solve is the famous known vanishing gradient. This is because when the network is too deep, the gradients from where the loss function is calculated easily shrink to zero after several applications of the chain rule. This result on the weights never updating its values and therefore, no learning is being performed. In 2015, ResNets showed that the gradients can flow directly through the skip connections backwards from later layers to initial filters.

ResNet consists of one convolution and pooling step followed by 4 layers of similar behavior. Each of the layers follows the same pattern. They perform 3x3 convolution with a fixed feature map dimension (F) [64, 128, 256, 512] respectively, bypassing the input every 2 convolutions. Besides, the width (W) and height (H) dimensions remain constant during the entire layer. The dotted line is there, precisely because there has been a change in the dimension of the input volume (of course a reduction because of the convolution). Note that this reduction between layers is achieved by an increase on the stride, from 1 to 2, at the first convolution of each layer; instead of by a pooling operation, which we are used to see as down samplers. The first step on the ResNet before entering the common layer behavior is a block, called as Conv1 consists of a convolution + batch normalization + max pooling operation.

Therefore, initially a convolution operation is performed. In the Figure 3.15 we can see that they use a kernel size of 7, and a feature map size of 64. You need to infer that they have padded with zeros 3 times on each dimension. Taking this into account, the output size of that operation will be a (112x122) volume. Since each convolution filter (of the 64) provides one channel in the output volume, we end up with a (112x122x64) output volume; note this is free of the batch dimension to simplify the details. The next step is the batch normalization, which is an element-wise operation and therefore, it does not change the size of our volume. Finally, we have the (3x3) Max Pooling operation with a stride of 2. We can also infer that they first pad the input volume, so the final volume has the desired dimensions.

Every layer of a ResNet is composed of several blocks. This is because when ResNets go deeper, they normally do it by increasing the number of operations within a block, but the number of total layers remains the same. An operation here refers to convolution batch normalization and ReLU activation to an input, except the last operation of a block that does not have the ReLU. The next step is to escalate from the entire block to the entire layer. However, if we look at the first operation of each layer, we see that the stride used at that first one is 2, instead of 1 like for the rest of them. This means that the down sampling of the volume though the network is achieved by increasing the stride instead of a pooling operation like

normally CNNs do. In fact, only one max pooling operation is performed in our Conv1 layer, and one average pooling layer at the end of the ResNet, right before the fully connected dense layer.

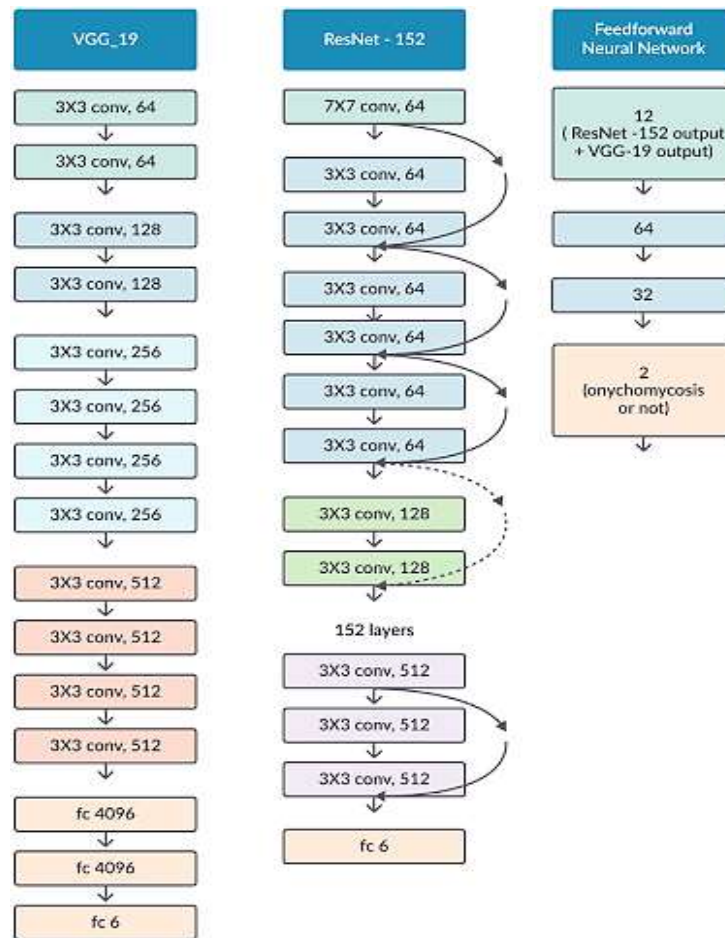


Fig. 3.15: ResNet Architecture

We can also see another repeating pattern over the layers of the ResNet, the dot layer representing the change of the dimensionality. The first operation of each layer is reducing the dimension, so we also need to resize the volume that goes through the skip connection. This difference on the skip connections are the so called as Identity Shortcut and Projection Shortcut. The identity shortcut is simply bypassing the input volume to the addition operator. The projection shortcut accomplishes a convolution operation to ensure the volumes at this addition operation are the same size. There are two options for matching the output size. Either padding the input volume or perform 1x1 convolutions. Down sampling is performed by increasing the

stride to 2. The number of filters is duplicated in an attempt to preserve the time complexity for every operation ($56 \times 64 = 28 \times 128$). Also, note that now the addition operation cannot be performed since the volume got modified. In the shortcut we need to apply one of our down sampling strategies. The final picture has the 2 output volumes of each thread has the same size and can be added.

After celebrating the victory of AlexNet, deep Residual Network was arguably the most ground breaking work in the computer vision and deep learning community in the last few years. ResNet makes it possible to train up to hundreds or even thousands of layers and still achieves compelling performance. Taking advantage of its powerful representational ability, the performance of many computer vision applications other than image classification has been progressive, such as object detection and face recognition. Since ResNet blew people's mind in 2015, many in the research community have dived into the secrets of its success; many refinements have been made in the architecture. In Conclusion, the Skip Connection is a very interesting extension to Deep Convolutional Networks that have empirically shown to increase performance in ImageNet classification. These layers can be used in other tasks requiring Deep networks as well such as Localization, Semantic Segmentation, Generative Adversarial Networks, Super-Resolution, and others.

3.6.6 ResNeXt

ResNeXt won 2nd place in ILSVRC 2016 classification task and also showed performance improvements in Coco detection and ImageNet-5k set than their ResNet counterpart. This is a very simple architecture which introduces a new term called “cardinality”. The architecture makes use of it in ResNet networks and does various ablation studies. ResNeXt made several attempts to describe the complexity of Inception networks and why it is simpler.

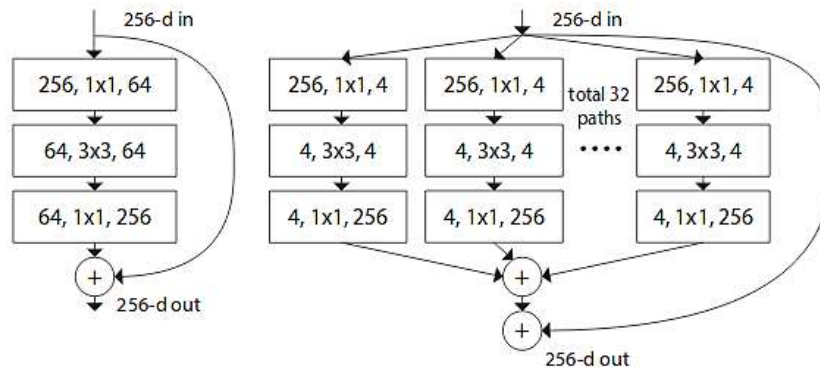


Fig. 3.16: ResNet (left) and ResNeXt (right) Architecture

The above architecture distinguishes between a simple ResNet block and ResNeXt block;

- ✚ ResNeXt follows a split-transform-aggregate strategy.
- ✚ The number of paths inside the ResNeXt block is defined as cardinality ($C=32$).
- ✚ All the paths contain the same topology.
- ✚ Instead of having high depth and width, having high cardinality helps in decreasing validation error.
- ✚ ResNeXt tries embedding more subspaces compared to its ResNet counterpart.
- ✚ Both the architectures have different width. Layer-1 in ResNet has one conv layer with 64 width, while layer-1 in ResNeXt has 32 different conv layers with 4 width (32×4 width). Despite the larger overall width in ResNeXt, both the architectures have the same number of parameters ($\sim 70k$).

With Cardinality C increasing from 1 to 32, we can clearly realize a decrease in top-1 % error rate. Therefore, increasing C by decreasing the width has improved the performance of the model. Basically three cases were studied.

- ✚ Increasing the number of layers to 200 from 101.
- ✚ Going wider by increasing the bottleneck width.
- ✚ Increasing cardinality by doubling C .

It was observed that increasing C gave better performance boosts. The model name, ResNeXt, contains Next. It means the next dimension, on top of ResNet. This next dimension is called the “cardinality” dimension.

Below are the results.

Table 3.2. Cardinality Vs. Width

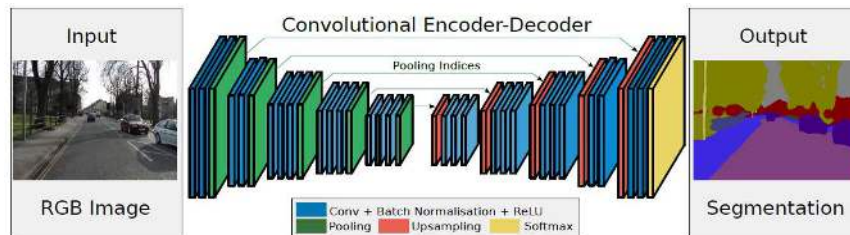
	setting	top-1 error (%)
ResNet-50	$1 \times 64d$	23.9
ResNeXt-50	$2 \times 40d$	23.0
ResNeXt-50	$4 \times 24d$	22.6
ResNeXt-50	$8 \times 14d$	22.3
ResNeXt-50	$32 \times 4d$	22.2
ResNet-101	$1 \times 64d$	22.0
ResNeXt-101	$2 \times 40d$	21.7
ResNeXt-101	$4 \times 24d$	21.4
ResNeXt-101	$8 \times 14d$	21.3
ResNeXt-101	$32 \times 4d$	21.2

Table 3.3. Cardinality Vs. Deeper/Wider Networks

	setting	top-1 err (%)	top-5 err (%)
<i>1× complexity references:</i>			
ResNet-101	1 × 64d	22.0	6.0
ResNeXt-101	32 × 4d	21.2	5.6
<i>2× complexity models follow:</i>			
ResNet-200 [15]	1 × 64d	21.7	5.8
ResNet-101, wider	1 × 100d	21.3	5.7
ResNeXt-101	2 × 64d	20.7	5.5
ResNeXt-101	64 × 4d	20.4	5.3

3.6.7 Seg Net

SegNet is deep encoder-decoder architecture for multi-class pixel wise segmentation researched and developed by members of the Computer Vision and Robotics Group at the University of Cambridge, UK. SegNet is also effective for indoor scene understanding. The architecture consists of a sequence of non-linear processing layers (encoders) and a corresponding set of decoders followed by a pixel wise classifier. Typically, each encoder consists of one or more convolutional layers with batch normalisation and a ReLU non-linearity, followed by non-overlapping maxpooling and sub-sampling. The sparse encoding due to the pooling process is upsampled in the decoder using the maxpooling indices in the encoding sequence. One key ingredient of the SegNet is the use of max-pooling indices in the decoders to perform upsampling of low-resolution feature maps. This has the vital advantages of retaining high frequency details in the segmented images and also reducing the total number of trainable parameters in the decoders. The entire architecture can be trained end-to-end using stochastic gradient descent. The raw SegNet predictions tend to be smooth even without a CRF based post-processing.

**Fig. 3.17: Seg Net Architecture**

SegNet has an encoder network and a corresponding decoder network, trailed by a final pixel wise classification layer. At the encoder, convolutions and max pooling are performed. There

are 13 convolutional layers from VGG-16 (The original fully connected layers are discarded). While doing 2×2 max pooling, the corresponding max pooling indices (locations) are stored. At the decoder, upsampling and convolutions are performed. At the end, there is softmax classifier for each pixel. During upsampling, the max pooling indices at the corresponding encoder layer are recalled to upsample.

K-class softmax classifier is used to predict the class for each pixel. SegNet uses a novel technique to upsample encoder output which involves storing the max-pooling indices used in pooling layer. This gives reasonably good performance and is space efficient. VGG16 with only forward connections and non-trainable layers is employed as encoder. This results to very less parameters. Semantic pixel-wise labelling i.e. labelling each pixel of an image to belong to some class (tree, road, sky, etc.) as shown in Figure 3.17. Some applications include autonomous driving, scene understanding, etc.

Direct adoption of classification networks for pixel wise segmentation yields poor results mainly because max-pooling and subsampling reduce feature map resolution and hence output resolution is reduced. Even if extrapolated to original resolution, lossy image is generated. Trained on road scene datasets hence, classes represent macro objects; hence segmentations are desired to be smooth. Boundary information is critical for objects like road markings and other small objects (Boundary delineation). Major use cases will be embedded systems hence it must be Computationally Efficient.

The novelty is in the subsampling stage, Max-pooling is used to achieve translation invariance over small spatial shifts in the image, combine that with Subsampling and it leads to each pixel governing a larger input image context (spatial window). These methods achieve better classification accuracy but reduce the feature map size, this leads to lossy image representation with blurred boundaries which is not ideal for segmentation purpose. It is desired that output image resolution is same as input image; to achieve this SegNet does Upsampling in its decoder, to do that it needs to store some information. It is necessary to capture and store boundary information in the encoder feature maps before sub-sampling. In order to that space efficiently, SegNet stores only the max-pooling indices i.e. the locations of maximum feature value in each pooling window is memorized for each encoder map.

Only 2 bits are needed for each window of 2×2 , slight loss of precision, but tradeoff. SegNet provides improved boundary delineation with less number of parameters. SegNet is slower than FCN and DeepLabv1 because SegNet contains the decoder architecture. And it is faster than DeconvNet because it does not have fully connected layers. SegNet has low memory requirement during both training and testing. And the model size is much smaller than FCN and DeconvNet. Perceptibly, SegNet outperforms FCN, DeconvNet, and DeepLabv1.

3.6.8 SqueezeNet

SqueezeNet provides a smart architecture as well as a quantitative analysis. For the same accuracy of AlexNet, SqueezeNet can be 3 times faster and 500 times smaller. The main ideas of SqueezeNet are:

- Using 1x1 (point-wise) filters to replace 3x3 filters, as the former only 1/9 of computation.
- Using 1x1 filters as a bottleneck layer to reduce depth to reduce computation of the following 3x3 filters.
- Downsample late to keep a big feature map.

The building brick of Squeeze Net is called fire module, which contains two layers: a squeeze layer and an expand layer. A Squeeze Net stacks a bunch of fire modules and a few pooling layers. The squeeze layer and expand layer keep the same feature map size, while the former reduce the depth to a smaller number, the later increase it. The squeezing (bottleneck layer) and expansion behavior is common in neural architectures.

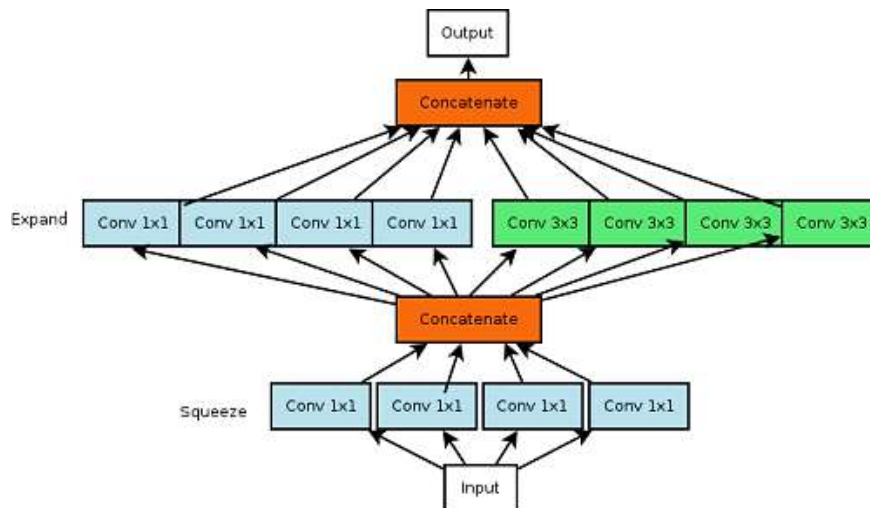


Fig. 3.18: Squeeze Net Architecture

Another common pattern is increasing depth while reducing feature map size to get high level abstract. The squeeze module only contains 1x1 filters, which means it works like a fully-connected layer working on feature points in the same position. In other words, it doesn't have the ability of spatial abstract. As its name says, one of its benefits is to reduce the depth of feature map. Reducing depth means the following 3x3 filters in the expand layer has fewer computation to do. It boosts the speed as a 3x3 filter need as 9 times computation as a 1x1

filter. By intuition, too much squeezing limits information flow; too few 3x3 filters limit space resolution. With the use of Fire Module, model size can be reduced while maintaining the prediction accuracy. Interestingly, the simple bypass enabled a sophisticated accuracy improvement than complex bypass. Adding the simple bypass connections yielded an increase of 2.9 percentage-points in top-1 accuracy and 2.2 percentage-points in top-5 accuracy without increasing model size.

3.6.9 R-CNN (Region Based CNN)

Region-based convolutional neural networks or regions with CNN features (R-CNNs) are a pioneering approach that applies deep models to object detection. R-CNN models first select several proposed regions from an image (for example, anchor boxes are one type of selection method) and then label their categories and bounding boxes (e.g., offsets). Then, they use a CNN to perform forward computation to extract features from each proposed area. Afterwards, we use the features of each proposed region to predict their categories and bounding boxes.

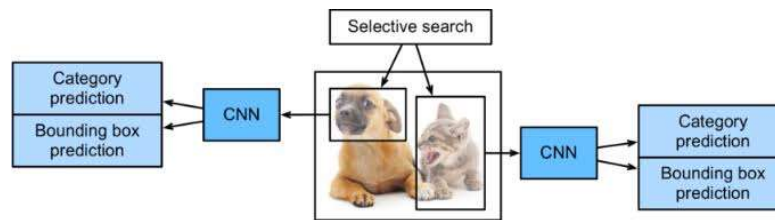


Fig. 3.19: R-CNN Architecture

R-CNNs are composed of four main parts:

- ✚ Selective search is performed on the input image to select multiple high-quality proposed regions. These proposed regions are usually selected on multiple scales and have different shapes and sizes. The category and ground-truth bounding box of each proposed region is labelled.
- ✚ A pre-trained CNN is selected and placed, in truncated form, before the output layer. It transforms each proposed region into the input dimensions required by the network and uses forward computation to output the features extracted from the proposed regions.
- ✚ The features and labeled category of each proposed region are combined as an example to train multiple support vector machines for object classification. Here, each support vector machine is used to determine whether an example belongs to a certain category.
- ✚ The features and labeled bounding box of each proposed region are combined as an example to train a linear regression model for ground-truth bounding box prediction.

Although R-CNN models use pre-trained CNNs to effectively extract image features, the main downside is the slow speed. As you can imagine, we can select thousands of proposed regions from a single image, requiring thousands of forward computations from the CNN to perform object detection. This massive computing load means that R-CNNs are not widely used in actual applications.

Fast R-CNN: The main performance bottleneck of an R-CNN model is the need to independently extract features for each proposed region. As these regions have a high degree of overlap, independent feature extraction results in a high volume of repetitive computations. Fast R-CNN improves on the R-CNN by only performing CNN forward computation on the image as a whole. Its primary computation steps are described below:

- ✚ Compared to an R-CNN model, a Fast R-CNN model uses the entire image as the CNN input for feature extraction, rather than each proposed region. Moreover, this network is generally trained to update the model parameters. As the input is an entire image, the CNN output shape is $1 \times c \times h1 \times w1$.
- ✚ Assuming selective search generates n proposed regions; their different shapes indicate regions of interests (RoIs) of different shapes on the CNN output. Features of the same shapes must be extracted from these RoIs (here we assume that the height is $h2$ and the width is $w2$). Fast R-CNN introduces RoI pooling, which uses the CNN output and RoIs as input to output a concatenation of the features extracted from each proposed region $n \times c \times h2 \times w2$.
- ✚ A fully connected layer is used to transform the output shape to $n \times d$, where d is determined by the model design.
- ✚ During category prediction, the shape of the fully connected layer output is again transformed to $n \times q$ and we use softmax regression (q is the number of categories). During bounding box prediction, the shape of the fully connected layer output is again transformed to $n \times 4$. This means that we predict the category and bounding box for each proposed region.

The RoI pooling layer in Fast R-CNN is somewhat different from the pooling layers we have discussed before. In a normal pooling layer, we set the pooling window, padding, and stride to control the output shape. In a RoI pooling layer, we can directly specify the output shape of each region, such as specifying the height and width of each region as $h2$, $w2$. Assuming that the height and width of the RoI window are h and w , this window is divided into a grid of sub-windows with the shape $h2 \times w2$. The size of each sub-window is about $(h/h2) \times (w/w2)$. The sub-window height and width must always be integers and the largest element is used as the output for a given sub-window. This allows the RoI pooling layer to extract features of the same shape from RoIs of different shapes.

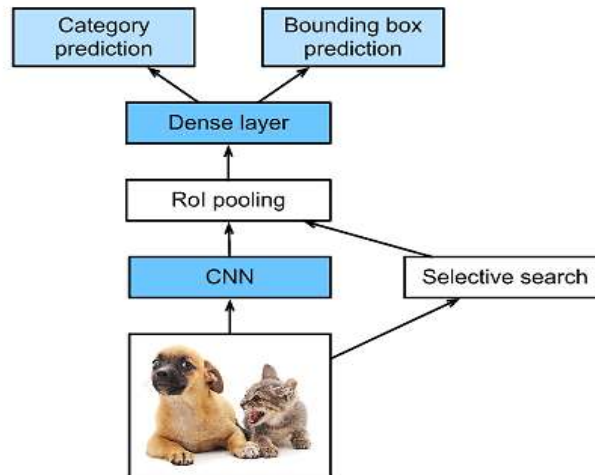


Fig. 3.20: Fast R-CNN Architecture

Faster R-CNN: In order to obtain precise object detection results, Fast R-CNN generally requires that many proposed regions be generated in selective search. Faster R-CNN replaces selective search with a region proposal network. This reduces the number of proposed regions generated, while ensuring precise object detection. Compared to Fast R-CNN, Faster R-CNN only changes the method for generating proposed regions from selective search to region proposal network. The other parts of the model remain unchanged. The detailed region proposal network computation process is described below:

- 📌 We use a 3×3 convolutional layer with a padding of 1 to transform the CNN output and set the number of output channels to c . This way, each element in the feature map the CNN extracts from the image is a new feature with a length of c .
- 📌 We use each element in the feature map as a center to generate multiple anchor boxes of different sizes and aspect ratios and then label them.
- 📌 We use the features of the elements of length c at the center on the anchor boxes to predict the binary category (object or background) and bounding box for their respective anchor boxes.
- 📌 Then, we use non-maximum suppression to remove similar bounding box results that correspond to category predictions of “object”. Finally, we output the predicted bounding boxes as the proposed regions required by the RoI pooling layer.

It is worth noting that, as a part of the Faster R-CNN model, the region proposal network is trained together with the rest of the model. In addition, the Faster R-CNN object functions include the category and bounding box predictions in object detection, as well as the binary

category and bounding box predictions for the anchor boxes in the region proposal network. Finally, the region proposal network can learn how to generate high-quality proposed regions, which reduces the number of proposed regions while maintaining the precision of object detection.

Mask R-CNN: If training data is labeled with the pixel-level positions of each object in an image, a Mask R-CNN model can effectively use these detailed labels to further improve the precision of object detection. Mask R-CNN is a modification to the Faster R-CNN model. Mask R-CNN models replace the RoI pooling layer with a RoI alignment layer. This allows the use of bilinear interpolation to retain spatial information on feature maps, making Mask R-CNN better suited for pixel-level predictions. The RoI alignment layer outputs feature maps of the same shape for all RoIs. This not only predicts the categories and bounding boxes of RoIs, but allows us to use an additional fully convolutional network to predict the pixel-level positions of objects.

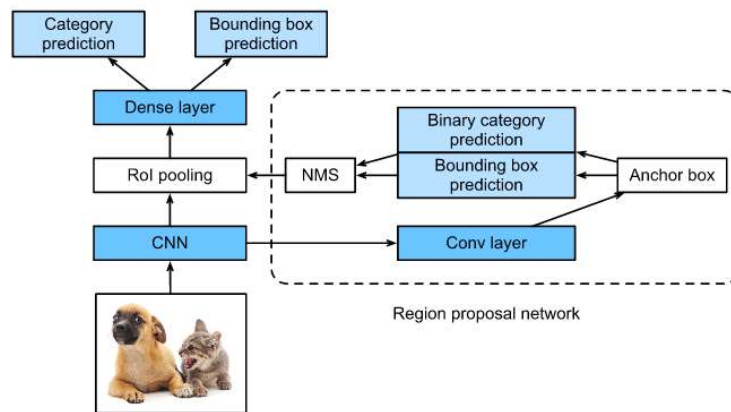


Fig. 3.21: Faster R-CNN Architecture

To conclude, An R-CNN model selects several proposed regions and uses a CNN to perform forward computation and extract the features from each proposed region. It then uses these features to predict the categories and bounding boxes of proposed regions. Fast R-CNN improves on the R-CNN by only performing CNN forward computation on the image as a whole. It introduces a RoI pooling layer to extract features of the same shape from RoIs of different shapes. Faster R-CNN replaces the selective search used in Fast R-CNN with a region proposal network. This reduces the number of proposed regions generated, while ensuring precise object detection. Mask R-CNN uses the same basic structure as Faster R-CNN, but adds a fully convolution layer to help locate objects at the pixel level and further improve the precision of object detection.

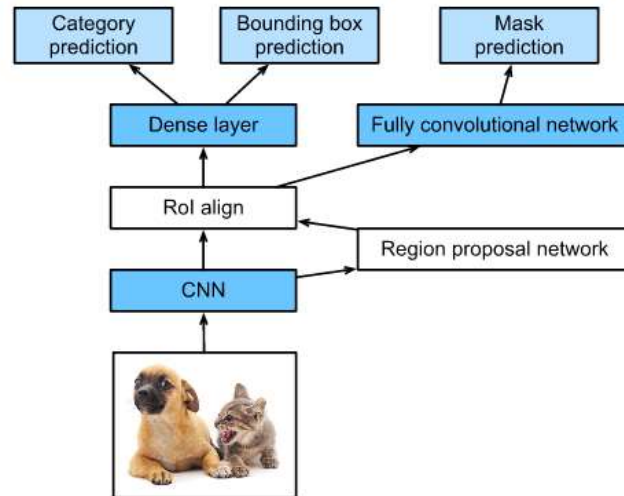


Fig. 3.22: Mask R-CNN Architecture

3.6.10 Creating Deep Learning Models

Various different methods can be used to create strong deep learning models. These techniques include learning rate decay, transfer learning, training from scratch and dropout.

- 📚 **Learning rate decay:** The learning rate is a hyperparameter, a factor that defines the system or sets conditions for its operation prior to the learning process that controls how much change the model experiences in response to the estimated error every time the model weights are altered. Learning rates that are too high may result in unstable training processes or the learning of a suboptimal set of weights. Learning rates that are too small may produce a lengthy training process that has the potential to get stuck.
- 📚 **Learning rate decay:** Also called as learning rate annealing or adaptive learning rates, it is the process of adapting the learning rate to increase performance and reduce training time. The easiest and most common adaptations of learning rate during training include techniques to reduce the learning rate over time.
- 📚 **Transfer learning:** This process involves perfecting a previously trained model; it requires an interface to the internals of a previous network. First, users feed the existing network new data containing previously unknown classifications. Once adjustments are made to the network, new tasks can be performed with more specific categorizing abilities thus reducing computation time.
- 📚 **Training from scratch:** This method requires a developer to collect a large labeled data set and configure a network architecture that can learn the features and model. This technique is especially useful for new applications, as well as applications with a

large number of output categories. However, overall, it is a less common approach, as it requires inordinate amounts of data, causing training to take days or weeks.

 **Dropout:** This method attempts to solve the problem of overfitting in networks with large amounts of parameters by randomly dropping units and their connections from the neural network during training. It has been proven that the dropout method can improve the performance of neural networks on supervised learning tasks in areas such as speech recognition, document classification and computational biology.

3.7 WHAT ARE RECENT DEEP LEARNING ALGORITHMS?

The computer vision community was fairly skeptical about deep learning until AlexNet demolished all its competitors on ImageNet in 2012. In the coming years, vision researchers would propose a variety of neural network architectures with increasingly better performance. Deep Learning was also rapidly adapted to other visual tasks such as object detection, where the image contains one or more objects and the background is much larger. For this task, neural networks were augmented by an initial stage which made proposals for possible positions and sizes of the objects. These methods outperformed the previous best methods, the Deformable Part Models, for the PASCAL object detection challenge, which was the foremost object detection and classification challenge before ImageNet.

Other Deep Net architectures also gave enormous performance jumps in other classic tasks. This is because of the flexibility that neural network provides when building a full-fledged end-to-end model. The successes of Deep Net architectures have been extraordinary and have helped vision become quite popular, dramatically increased the interaction between academia and industry, led to applications of vision techniques to a large range of disciplines, and have many other important consequences. In this section, we will briefly look into some of the most recent deep learning algorithms that are widely used for computer vision applications.

3.7.1 Deep Convolutional Neural Networks

In deep learning, Convolutional neural network (ConvNets or CNNs) is one of the main categories to do images recognition, images classifications. Objects detections, face recognition, etc., are some of the areas where CNNs are widely used. A CNN image classification takes an input image, process it and classify it under certain categories (Eg. Dog, Cat, Tiger, Lion). A computer sees an input image as array of pixels and it depends on the image resolution. Based on the image resolution, it will see $h \times w \times d$ (h = Height, w = Width, d = Dimension), in an image of $6 \times 6 \times 3$ array of matrix of RGB and an image of $4 \times 4 \times 1$ array of matrix of grayscale image. Technically, deep learning CNN models to train and test, each input image will pass it through a series of convolution layers with filters (Kernels), Pooling, fully connected layers (FC) and apply Softmax function to classify an object with

probabilistic values between 0 and 1. The below figure is a complete flow of CNN to process an input image and classifies the objects based on values.

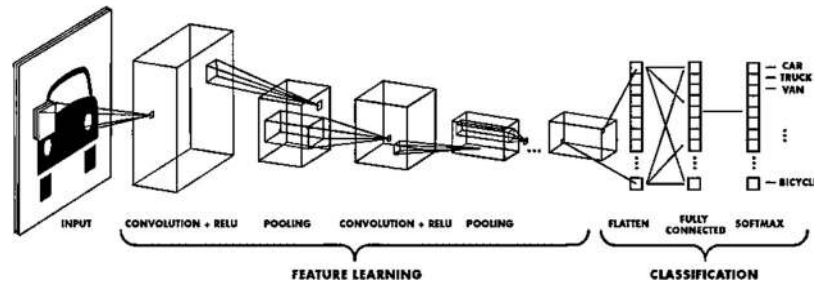


Fig. 3.23: Neural Network with Many Convolutional Layers

Convolution Layer: Convolution is the first layer to extract features from an input image. Convolution preserves the relationship between pixels by learning image features expending small squares of input data. It is a mathematical operation that takes two inputs such as image matrix and a filter or kernel. Convolution of an image with different filters can perform operations such as edge detection, blur and sharpen by applying filters. Stride is the number of pixels shifts over the input matrix. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on. Convolutional layers are formed by filters, feature maps, and activation functions. These Convol layers can be full, same or valid.

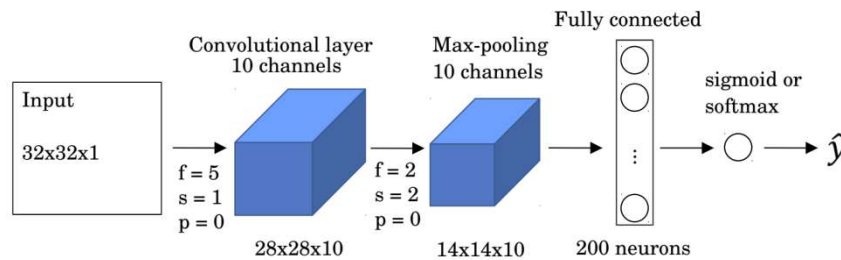


Fig. 3.24: Fully Connected Convolutional Neural Network

We can determine the number of output layers of a given convolutional block if the number of layers in the input is known, n_i , the number of filters in that stage, f , the size of the stride, s , and the pixel dimension of the image, p (assuming it is square). Pooling layers are used to reduce overfitting. Fully connected layers are used to mix spacial and channel features together. Each of the filter layers corresponds to the image after a feature map has been drawn across the image, which is how features are extracted. It is important to know the number of input and

output layers as this determines the number of weights and biases that make up the parameters of the neural network. The more parameters in the network, the more parameters need to be trained which results in longer training time. Training time is very important for deep learning as it is a limiting factor unless you have access to powerful computing resources such as a computing cluster.

In the above network, we have 250 weights on the convolutional filter and 10 bias terms. We have no weights on the max-pooling layer. We have $13 \times 13 \times 10 = 1,690$ output elements after the max-pooling layer. We have a 200 node fully connected layer, which results in a total of $1,690 \times 200 = 338,000$ weights and 200 bias terms in the fully connected layer. Thus, we have a total of 338,460 parameters to be trained in the network. We can see that the majority of the trained parameters occur at the fully connected output layer. Each CNN layer learns filters of increasing complexity. The first layers learn basic feature detection filters such as edges and corners. The middle layers learn filters that detect parts of objects for faces; they might learn to respond to eyes and noses. The last layers have higher representations: they learn to recognize full objects, in different shapes and their positions.

Receptive Field and Dilated Convolutions: The receptive field is defined as the region in the input space that a particular CNN's feature is looking at (i.e. be affected by). Applying a convolution C with kernel size $k = 3 \times 3$, padding size $p = 1 \times 1$, and stride $s = 2 \times 2$ on a 5×5 input map, we will get a 3×3 output feature map. Applying the same convolution on top of the 3×3 feature map, we will get a 2×2 feature map. Dilated convolution works in a similar way to a normal convolution, the major difference being that the receptive field no longer consists of contiguous pixels, but of individual pixels separated by other pixels. The way in which a dilated convolutional layer is applied to an image is shown in Figure 3.25.

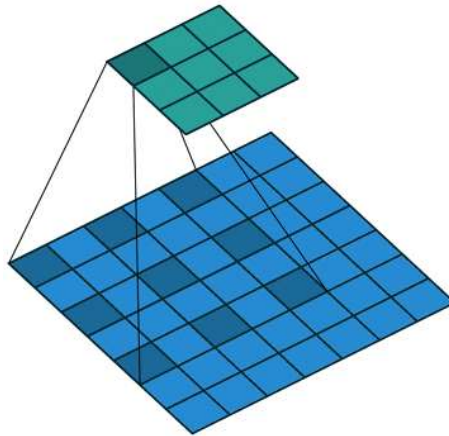


Fig. 3.25: Dilated Convolution

The receptive field is the implicit area captured on the initial input by each input (unit) to the next layer. The motivation behind expanding dilated convolutions are: to detect the fine details by processing inputs in higher resolutions, a broader view of the input to capture more contextual information and to achieve faster run-time with fewer parameters.

Transposed Convolution: So far, the convolutions we have looked at either maintain the size of their input or make it smaller. We can use the same practice to make the input tensor larger. This process is called upsampling. When we do it inside of a convolution step, it is called transposed convolution or fractional striding. A typical convolutional layer with no padding, acting on an image of size 5×5 , after the convolution, we end up with a 3×3 image as shown in Figure 3.26.

Non Linearity (ReLU): ReLU stands for Rectified Linear Unit for a non-linear operation. The output is $f(x) = \max(0, x)$. ReLU's purpose is to introduce non-linearity in our ConvNet. Since, the real world data would want our ConvNet to learn would be non-negative linear values. There are other nonlinear functions such as tanh or sigmoid that can also be used instead of ReLU. Most of the data scientists use ReLU since performance wise ReLU is better than the other two.

Fully Connected Layer: FC layers, flattens our matrix into vector and feeds it into a fully connected layer like a neural network. The feature map matrix will be converted as vector ($x_1, x_2, x_3, \dots$). With the fully connected layers, we combine these features together to create a model. Finally, we have an activation function such as softmax or sigmoid to classify the outputs as cat, dog, car, truck etc.

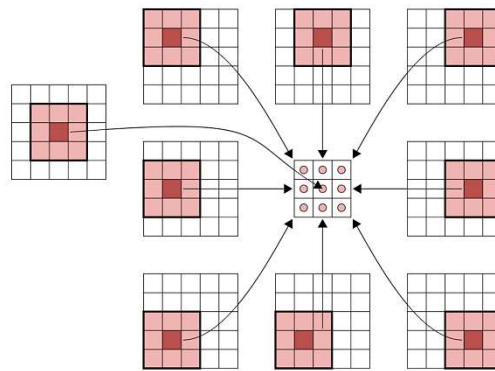


Fig. 3.26: Convolutional Layer with No Padding

Classic Networks: These networks were utilized in some of the seminal work done in the field of deep learning, and are often used for transfer learning purposes as listed below;

- 📁 LeNet-5
- 📁 AlexNet
- 📁 ZFNet
- 📁 VGG16 and VGG19
- 📁 GoogLeNet
- 📁 Residual Networks
- 📁 Dense Networks

3.7.2 Deep Recurrent Neural Networks

Humans don't start their thinking from scratch every second. By reading this passage, you understand each word based on your understanding of previous words. You don't throw everything away and start thinking from scratch again. Your thoughts have persistence. Traditional neural networks can't do this, and it seems like a major shortcoming. For example, imagine you want to classify what kind of event is happening at every point in a movie. It's unclear how a traditional neural network could use its reasoning about previous events in the film to inform later ones.

A glaring limitation of Vanilla Neural Networks (and also Convolutional Networks) is that their API is too constrained: they accept a fixed-sized vector as input (e.g. an image) and produce a fixed-sized vector as output (e.g. probabilities of different classes). Not only that: These models perform this mapping using a fixed amount of computational steps (e.g. the number of layers in the model). The core reason that recurrent nets are more exciting is that they allow us to operate over sequences of vectors: Sequences in the input, the output, or in the most general case both. Recurrent neural networks address this issue. They are networks with loops in them, letting data to persist.

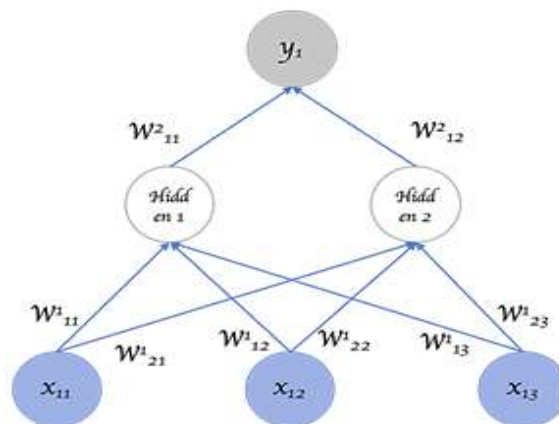


Fig. 3.27: A Vanilla Network Representation

Recurrent Neural Networks (RNNs) add an interesting twist to basic neural networks. A vanilla neural network takes in a fixed size vector as input which limits its usage in situations that involve a ‘series’ type input with no predetermined size. Recurrent Neural Network remembers the past and its decisions are influenced by what it has learnt from the past. Note: Basic feed forward networks “remember” things too, but they remember things they learnt during training. For example, an image classifier learns what a “1” looks like during training and then uses that knowledge to classify things in production. While RNNs learn similarly while training, in addition, they remember things learnt from prior input(s) while generating output(s). It’s part of the network. RNNs can take one or more input vectors and produce one or more output vectors and the output(s) are influenced not just by weights applied on inputs like a regular NN, but also by a “hidden” state vector representing the context based on prior input(s)/output(s). So, the same input could produce a different output depending on previous inputs in the series.

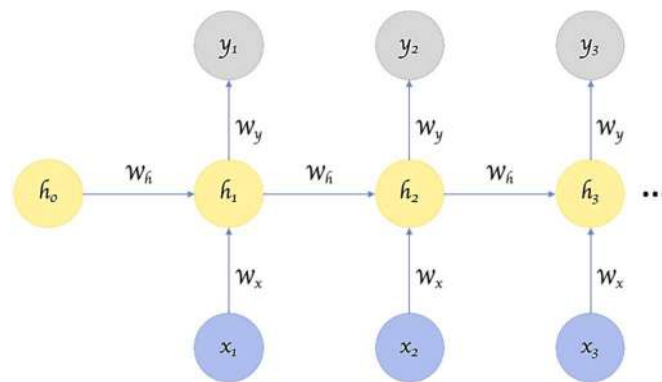


Fig. 3.28: A Recurrent Neural Network

To summarize, in a vanilla neural network, a fixed size input vector is transformed into a fixed size output vector. Such a network becomes “recurrent” when you repeatedly apply the transformations to a series of given input and produces a series of output vectors. There is no pre-set limitation to the size of the vector. And, in addition to generating the output which is a function of the input and hidden state, we update the hidden state itself based on the input and use it in processing the next input. Unlike in vanilla neural networks, in RNN, we apply the same weights over and over again to different items in the input series. If we don’t share parameters across inputs, then it becomes like a vanilla neural network where each input node requires weights of their own.

This introduces the constraint that the length of the input has to be fixed and that makes it impossible to leverage a series type input where the lengths differ and is not always known. But what we seemingly lose in value here, we gain back by introducing the “hidden state” that

links one input to the next. The hidden state captures the relationship that neighbors might have with each other in a serial input and it keeps changing in every step, and thus effectively every input undergoes a different transition! Image classifying CNNs have become so successful because the 2D convolutions are an effective form of parameter sharing where each convolutional filter basically extracts the presence or absence of a feature in an image which is a function of not just one pixel but also of its surrounding neighbor pixels. In other words, the success of CNNs and RNNs can be attributed to the concept of “parameter sharing” which is fundamentally an effective way of leveraging the relationship between one input item and its surrounding neighbors in a more intrinsic fashion compared to a vanilla neural network.

Deep RNNs: While it’s good that the introduction of hidden state enabled us to effectively identify the relationship between the inputs, is there a way we can make a RNN “deep” and gain the multi-level abstractions and representations we gain through “depth” in a typical neural network? Here are four possible ways to add depth.

- ✚ Perhaps the most obvious of all, is to add hidden states, one on top of another, feeding the output of one to the next.
- ✚ We can also add additional nonlinear hidden layers between input to hidden state.
- ✚ We can increase depth in the hidden to hidden transition.
- ✚ We can increase depth in the hidden to output transition.

Bidirectional RNNs: Sometimes it’s not just about learning from the past to predict the future, but we also need to look into the future to fix the past. In speech recognition and handwriting recognition tasks, where there could be considerable ambiguity given just one part of the input, we often need to know what’s coming next to better understand the context and detect the present.

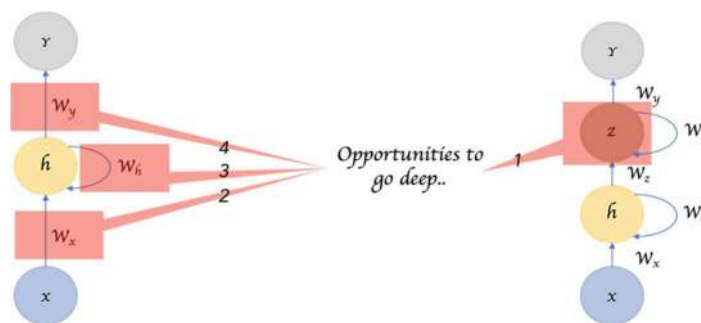


Fig. 3.29: A Typical Deep Recurrent Neural Network

This does introduce the obvious challenge of how much into the future we need to look into, because if we have to wait to see all inputs then the entire operation will become costly. And

in cases like speech recognition, waiting till an entire sentence is spoken might make for a less compelling use case. Whereas for NLP tasks, where the inputs tend to be available, we can likely consider entire sentences all at once. Likewise, depending on the application, if the sensitivity to immediate and closer neighbors is higher than inputs that come further away, a variant that looks only into a limited future/past can be modeled.

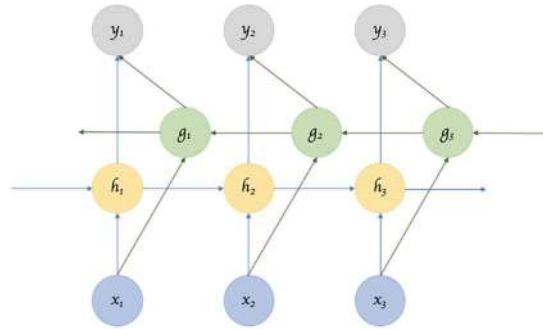


Fig. 3.30: A Bidirectional RNN

Recursive Neural Networks: A recurrent neural network parses the inputs in a sequential fashion. A recursive neural network is similar to the extent that the transitions are repeatedly applied to inputs, but not necessarily in a sequential fashion. Recursive Neural Networks are a more general form of Recurrent Neural Networks. It can work on any hierarchical tree structure. Parsing through input nodes, combining child nodes into parent nodes and linking them with other child/parent nodes to create a tree like structure. Recurrent Neural Networks do the same, but the structure there is strictly linear. i.e. weights are applied on the first input node, then the second, third and so on.

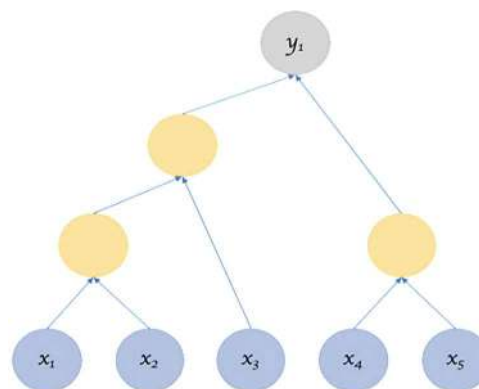


Fig. 3.31: A Recursive Neural Net

But this raises questions pertaining to the structure. How do we decide that? If the structure is fixed like in Recurrent Neural Networks then the process of training, backprop etc. makes sense in that they are similar to a regular neural network. But if the structure isn't fixed, is that learnt as well? But given the complexity involved, both computationally and even more importantly, in getting the requisite training data, recursive neural networks seem to be lagging in popularity to their recurrent cousin.

Encoder Decoder Sequence to Sequence RNNs: Encoder Decoder or Sequence to Sequence RNNs are used a lot in translation services. The basic idea is that there are two RNNs, one an encoder that keeps updating its hidden state and produces a final single “Context” output. This is then fed to the decoder, which translates this context to a sequence of outputs. Another key difference in this arrangement is that the length of the input sequence and the length of the output sequence need not necessarily be the same.

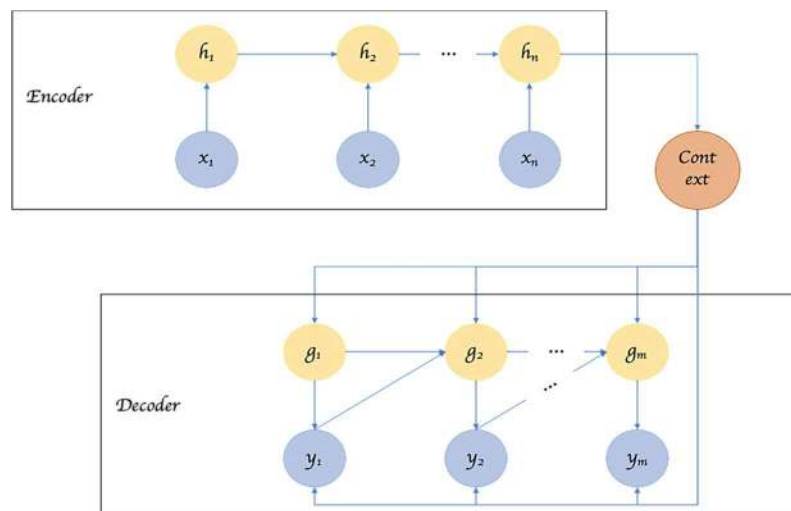


Fig. 3.32: Encoder Decoder or Sequence to Sequence RNNs

LSTMs: We cannot close any section that tries to look at what RNNs and related architectures are without mentioning LSTMs. This is not a different variant of RNN architecture, but rather it introduces changes to how we compute outputs and hidden state using the inputs. In a vanilla RNN, the input and the hidden state are simply passed through a single tanh layer. LSTM (Long Short Term Memory) networks improve on this simple transformation and introduce additional gates and a cell state, such that it fundamentally addresses the problem of keeping or resetting context, across sentences and regardless of the distance between such context resets. There are variants of LSTMs including GRUs that utilize the gates in different manners to address the problem of long term dependencies.

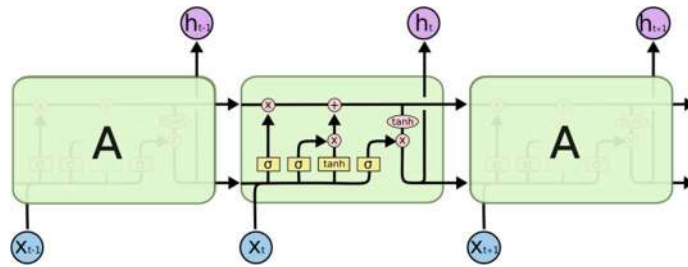


Fig. 3.33: Long Short-Term Memory with Four Interacting Layers

3.7.3 Deep Belief Networks

A deep-belief network can be defined as a stack of restricted Boltzmann machines (RBM), in which each RBM layer communicates with both the previous and subsequent layers. The nodes of any single layer don't connect with each other laterally. This stack of RBMs might end with a Softmax layer to create a classifier, or it may simply help cluster unlabeled data in an unsupervised learning scenario. With the exception of the first and final layers, each layer in a deep-belief network has a double role: it serves as the hidden layer to the nodes that come before it, and as the input (or "visible") layer to the nodes that come after. It is a network built of single-layer networks. Deep-belief networks (DBNs) are used to recognize, cluster and generate images, video sequences and motion-capture data. A continuous deep-belief network is simply an extension of a deep-belief network that accepts a continuum of decimals, rather than binary data.

- ✚ DBN is Unsupervised Probabilistic Deep learning algorithm.
- ✚ DBN is poised of multi-layer of stochastic latent variables. Latent variables are binary, also called as feature detectors or hidden units.
- ✚ DBN is a generative hybrid graphical model. Top two layers are undirected. Lower layers have directed connections from layers above.

DBN is a stack of Restricted Boltzmann Machine. Top two layers of DBN are undirected, symmetric connection between them that form associative memory. The connections between all lower layers are directed, with the arrows pointed toward the layer that is closest to the data. Lower Layers have directed acyclic connections that convert associative memory to observed variables. The lowest layer or the visible units receives the input data. Input data can be binary or real. There are no intra layer connections likes RBM. Hidden units represent features that capture the correlations present in the data. Two layers are connected by a matrix of symmetrical weights W . Every unit in each layer is connected to every unit in each neighboring layer. DBN are pre trained using Greedy learning algorithm. Greedy learning algorithm uses layer-by-layer approach for learning the top-down, generative weights.

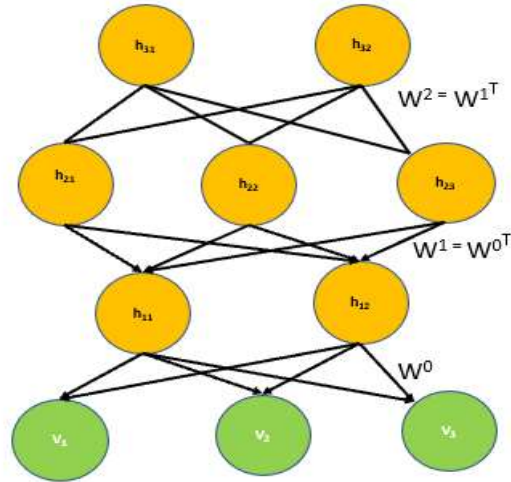


Fig. 3.34: Deep Belief Network

These generative weights govern how variables in one layer depend on the variables in the layer above. In DBN, we run several steps of Gibbs sampling on the top two hidden layers. This stage is essentially drawing a sample from the RBM defined by the top two hidden layers. Then use a single pass of ancestral sampling through the rest of the model to draw a sample from the visible units. Learning, the values of the latent variables in every layer can be inferred by a single, bottom-up pass. Greedy pretraining starts with an observed data vector in the bottom layer. It then uses the generative weights in the reverse direction using fine tuning.

Greedy Layer wise training algorithm was proposed by Geoffrey Hinton where we train a DBN one layer at a time in an unsupervised manner. Easy way to learn anything complex is to divide the complex problem into easy manageable chunks. We take a multi-layer DBN; divide into simpler models that are learned sequentially. It is easier to train a shallow network than training a deeper network. The idea behind our greedy algorithm is to allow each model in the sequence to receive a different representation of the data. How does Greedy Layer wise training algorithm work? First layer is trained from the training data greedily, while all other layers are frozen. We derive the individual activation probabilities for the first hidden layer. All the hidden units of the first hidden layer are updated in parallel. This is called as the positive phase. Final step in Greedy layer wise learning is to update all associated weights. L is the learning rate that we multiply by the difference between the positive and negative phase values and add to the initial value of the weight.

This process will be repeated till we get required threshold values. We then take the first hidden layer which now acts an input for the second hidden layer and so on. Each layer takes output

of the previous layer as an input to produce an output. Output generated is a new representation of data where distribution is simpler. Weights for the second RBM are the transpose of the weights for the first RBM. We again use the Contrastive Divergence method using Gibbs sampling just like we did for the first RBM. We calculate the positive phase, negative phase and update all the associated weights. This process will be repeated till we get required threshold values. We can add another RBM and calculate the contrastive divergence using the Gibbs sampling.

Why does DBM use Greedy Layer wise learning for pre training?

Pretraining helps in optimization by better initializing the weights of all the layers. Greedy learning algorithm is fast, efficient and learns one layer at a time. Trains layer sequentially starting from bottom layer. Each layer learns a higher data depiction of the lower layer. Greedy layer wise pretraining identifies feature detector. Fine tuning modifies the features slightly to get the category boundaries right. Adding fine tuning helps to discriminate between different classes better. Adjusting the weights during fine tuning process provides an optimal value. This helps increase the accuracy of the DBN model. Backward propagation works better with greedy layer wise training for fine tuning process. We do not start backward propagation till we have known sensible feature detectors that will be useful for discrimination task.

The objective of fine tuning is not discovering new features. Objective of DBM is to improve the accuracy of the model by finding the optimal values of the weights between layers. Once we have the sensible feature detectors identified then backward propagation only needs to perform a local search. Unlabelled data helps discover good features. We may also get features that are not very helpful for discriminative task but that is not an issue. We still get useful features from the raw input. Input vectors generally contain a lot more information than the labels. Precious information is the label is used only for fine tuning. Labelled dataset help associate patterns and features to the dataset. A small labelled dataset is used for fine tuning using backward propagation. Back Propagation fine tunes the model to be superior at discrimination. Fine tuning makes it easier to learn a deep network. To fine tune further we do a stochastic top down pass and adjust the bottom up weights. DBN is used for Image recognition, Video sequences, Motion capture data and Speech recognition.

3.7.4 Deep Boltzmann Machine

Deep Boltzmann Machine (DBM) is unsupervised, probabilistic, generative model with entirely undirected connections between different layers holding the following properties:

-  Contains visible units and multiple layers of hidden units.
-  Like Restricted Boltzmann Machine RBM, no intralayer connection exists in DBM.

- Connections exist only between the units of the neighboring layers.
- Network of symmetrically connected stochastic binary units.
- DBM can be organized as bipartite graph with odd layers on one side and even layers on one side.
- Units within the layers are independent of each other but are dependent on neighboring layers
- Learning is made efficient by - layer by layer pre training, Greedy layer wise pre training slightly different than done in DBM.
- After learning the binary features in each layer, DBM is fine-tuned by back propagation.

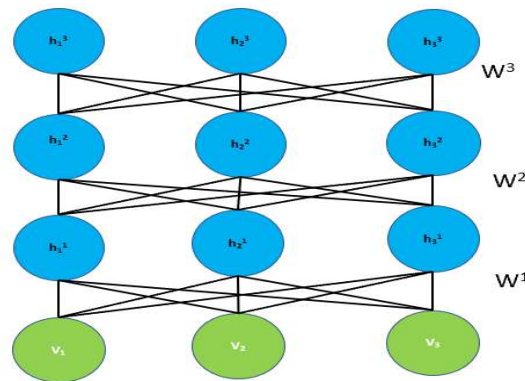


Fig. 3.35: Deep Boltzmann Machine

Boltzmann machine uses randomly initialized Markov chains to imprecise the gradient of the likelihood function which is too slow to be practical. DBM uses greedy layer by layer pre training to speed up learning the weights. It relies on learning stacks of Restricted Boltzmann Machine with a small modification using contrastive divergence. The key intuition for greedy layer wise training for DBM is that we double the input for the lower-level RBM and the top level RBM. Lower level RBM inputs are doubled to compensate for the lack of top-down input into first hidden layer. Similarly for top-level RBM, we double the hidden units to compensate for the lack of bottom-up input. For the intermediate layers, the RBM weights are simply doubled. The three RBM's are then combined to form a single model. Greedily pretraining the weights of a DBM initializes the weights to reasonable values helping subsequent joint learning of all layers.

This is expensive compared to a single bottom up inference used in DBN. In order to learn using large dataset we need to accelerate inference in a DBM. In order to accelerate inference of DBM, we use a set of recognition weights, which are initialized to the weights found by the

greedy pre training. We take an input vector and apply the recognition weights to reconstruct the input v of fully factorized approximation posterior distribution. Each layer of hidden units is activated in a single deterministic bottom-up pass. We double the weights of the recognition model at each layer to compensate for the lack of top-down feedback. However we do not double the top layer as it does not have a top-down input. We apply K iterations of mean-field to obtain the mean-field parameters that will be used in the training update for DBM's.

What are the similarities between Deep Belief networks (DBN) and Deep Boltzmann Machine (DBM)?

- ✚ Both DBN and DBM is unsupervised, probabilistic, generative, graphical model consisting of stacked layers of RBM.
- ✚ DBN and DBM both are used to identify latent feature present in the data.
- ✚ Both DBN and DBM performs inference and parameter learning efficiently using greedy layer-wise training.
- ✚ Both DBN and DBM apply discriminative fine tuning after greedy layer wise pre training.
- ✚ Both DBN and DBM use a large set of unlabeled data for pre training in an unsupervised manner to find good set of parameter for the model and then apply discriminative fine tuning on a small labelled dataset.

What are the differences between Deep Belief networks (DBN) and Deep Boltzmann Machine (DBM)?

- ✚ Deep Belief Network (DBN) have top two layers with undirected connections and lower layers have directed connections
- ✚ Deep Boltzmann Machine have entirely undirected connections.
- ✚ Approximate inference procedure for DBM uses a top-down feedback in addition to the usual bottom-up pass, allowing Deep Boltzmann Machines to better incorporate uncertainty about ambiguous inputs.
- ✚ A disadvantage of DBN is the approximate inference based on mean field approach is slower compared to a single bottom-up pass as in Deep Belief Networks. Mean field inference needs to be performed for every new test input.

Finally, we update the recognition weights for an initial guess of the input v close to the result μ . μ is the result of the mean field inference which is your target. By updating the recognition weights we want to minimize the KL divergence between the mean-field posterior ($h|v; \mu$) and the recognition model. To achieve classification, we need a separate multi-layer perceptrons (MLP) on top of the hidden features extracted from greedy layer pre training just as fine tuning is performed in DBN.

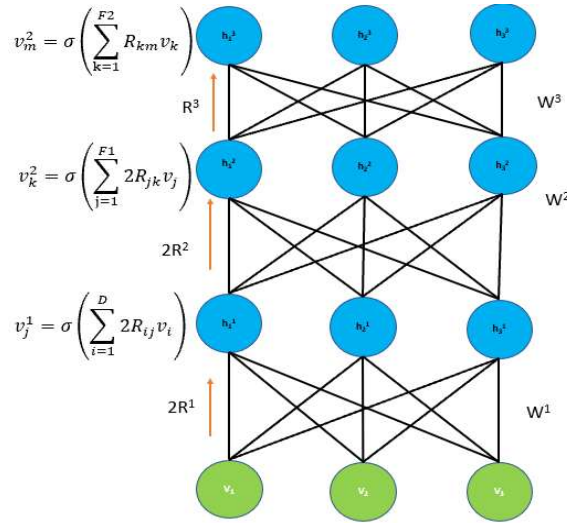


Fig. 3.36: R1, R2, R3 Recognition Weights of DBM

3.7.5 Deep Reinforcement Learning

While neural networks are responsible for recent innovations in problems like computer vision, machine translation and time series prediction, they can also combine with reinforcement learning algorithms to create rather astounding like AlphaGo. Reinforcement learning refers to goal-oriented algorithms, which learn how to attain a complex objective (goal) or exploit along a particular dimension over many steps; for example, maximize the points won in a game over many moves. They can start from a blank slate, and under the right conditions they achieve superhuman performance. Like a child incentivized by spankings and candy, these algorithms are penalized when they make the wrong decisions and rewarded when they make the right ones, this is reinforcement. Reinforcement algorithms that incorporate deep learning can beat world champions at the game of Go as well as human experts playing numerous Atari video games.

While that may sound trivial, it's a vast improvement over their previous accomplishments, and the state of the art is progressing rapidly. Reinforcement learning solves the difficult problem of correlating immediate actions with the delayed returns they produce. Like humans, reinforcement learning algorithms sometimes have to wait a while to see the fruit of their decisions. They operate in a delayed return environment, where it can be difficult to understand which action leads to which outcome over many time steps. Reinforcement learning algorithms can be expected to perform better and better in more ambiguous, real-life environments while choosing from an arbitrary number of possible actions, rather than from the limited options of a video game. That is, with time we expect them to achieve goals in the real world.

Deep reinforcement learning is about taking the best actions from what we see and hear. Unfortunately, reinforcement learning (RL) has a high barrier in learning the concepts and the lingos. In most AI topics, we create mathematical frameworks to tackle problems. For RL, the answer is the Markov Decision Process (MDP). It sounds complicated but it produces an easy framework to model a complex problem. An agent (e.g. a human) observes the environment and takes actions. Rewards are given out but they may be infrequent and delayed. Very often, the long-delayed rewards make it extremely hard to untangle the information and trace back what sequence of actions contributed to the rewards. State in MDP can be represented as raw images. An action can be a move in a chess game or moving a robotic arm or a joystick. For a GO game, the reward is very sparse: 1 if we win or -1 if we lose. At times, we get rewards more regularly.

In the Atari Seaquest game, we score whenever we hit the sharks. The discount factor discounts future rewards if it is smaller than one. Money earned in the future often has a smaller current value, and we may need it for a purely technical reason to converge the solution better. We can rollout actions forever or limit the experience to N time steps. This is called the horizon. The transition function is the system dynamics. It predicts the next state after taking action. It is called the model which plays a major role called Model-based RL. The concepts in RL come from many research fields including the control theory. Different notations may be used in a different context. Let's get this out first before any confusion.

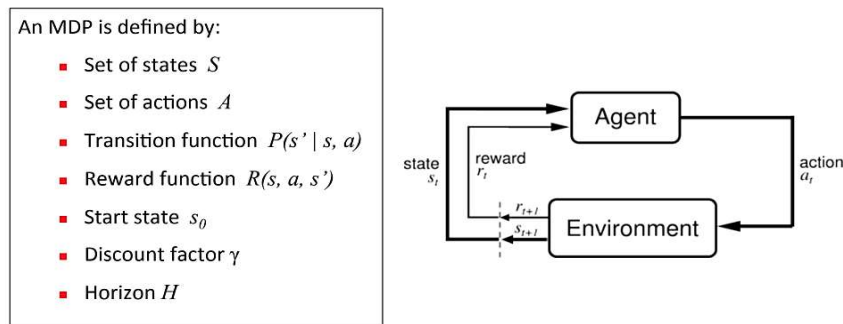


Fig. 3.37: Markov Decision Process

The state can be written as S and action as A . An action is the same as a control. We can maximize the rewards or minimize the costs which are simply the negative of each other. Notations can be in upper or lower case. So environments are functions that transform an action taken in the current state into the next state and a reward; agents are functions that transform the new state and reward into the next action. We can know and set the agent's function, but in most situations where it is useful and interesting to apply RL, we do not know the function of the environment. It is a black box where we only see the inputs and outputs. It's like most

people's relationship with technology: we know what it does, but we don't know how it works. Reinforcement learning represents an agent's attempt to approximate the environment's function, such that we can send actions into the black-box environment that maximize the rewards it spits out.

The feedback loop in the Figure 3.37, subscripts denote the time steps t and $t+1$, each of which refer to different states: the state at moment t , and the state at moment $t+1$. Unlike other forms of machine learning such as supervised and unsupervised learning, reinforcement learning can only be thought about sequentially in terms of state-action pairs that occur one after the other. Reinforcement learning judges actions by the results they yield. It is goal oriented, and its aim is to learn sequences of actions that will lead an agent to achieve its goal, or maximize its objective function. Here are some examples:

- ✚ In video games, the goal is to finish the game with the most points, so each additional point obtained throughout the game will affect the agent's ensuing behavior; i.e. the agent may learn that it should shoot battleships, touch coins or dodge meteors to maximize its score.
- ✚ In the real world, the goal might be for a robot to travel from point A to point B, and every inch the robot is able to move closer to point B could be counted like points.

Reinforcement learning differs from both supervised and unsupervised learning by how it interprets inputs. Reinforcement learning relies on the environment to send it a scalar number in response to each new action. The rewards returned by the environment can be varied, delayed or affected by unknown variables, introducing noise to the feedback loop. This leads us to a more complete expression, which takes into account not only the direct rewards produced by an action, but also the delayed rewards that may be returned several time steps deeper in the sequence. We can illustrate their difference by describing what they learn about a "thing."

- ✚ **Unsupervised learning:** That thing is like this other thing. (The algorithms learn similarities w/o names, and by extension they can spot the inverse and perform anomaly detection by recognizing what is unusual or dissimilar)
- ✚ **Supervised learning:** That thing is a "double bacon cheese burger". (Labels, putting names to faces). These algorithms learn the correlations between data instances and their labels; that is, they require a labelled dataset. Those labels are used to "supervise" and correct the algorithm as it make wrong guesses predicting labels.
- ✚ **Reinforcement learning:** Eat that thing because it tastes good and will keep you alive longer. (Actions based on short- and long-term rewards, such as the amount of calories you ingest, or the length of time you survive.) Reinforcement learning can be thought of as supervised learning in an environment of sparse feedback.

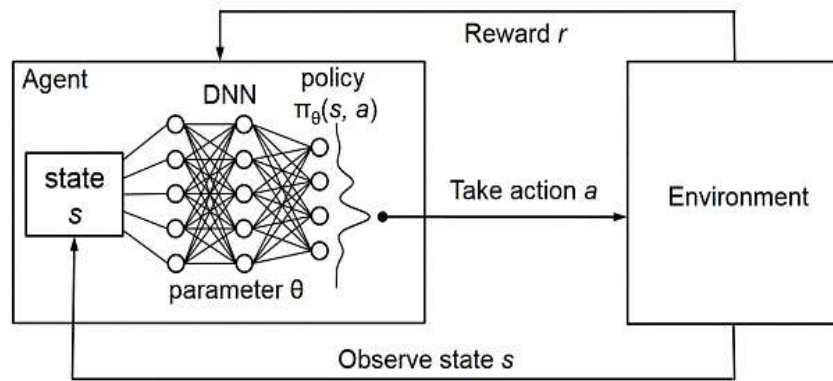


Fig. 3.38: Deep Reinforcement learning

3.7.6 Deep Residual Networks

Deep residual networks (DRN); called Deep Res Nets, are very deep Feed Forward Neural Networks (FFNNs) with extra connections; called 'skip connections', passing input from one layer to a later layer (often 2 to 5 layers) as well as the next layer. Instead of trying to find a solution for mapping some input to some output across say 5 layers, the network is enforced to learn to map some input to some output + some input. Basically, it adds an identity to the solution, carrying the older input over and serving it freshly to a later layer. It has been shown that these networks are very effective at learning patterns up to 150 layers deep, much more than the regular 2 to 5 layers one could expect to train. However, it has been proven that these networks are in essence just Recurrent Neural Network (RNNs) without the explicit time based construction and they're often compared to Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Recurrent Neural Network (RNN) without gates.

Deep residual network is a type of specialized neural network that helps to handle more sophisticated deep learning tasks and models. It has received quite a bit of attention at recent IT conventions, and is being considered for helping with the training of deep networks. In deep learning networks, a residual learning framework helps to preserve good results through a network with many layers. One problem commonly cited by professionals is that with deep networks composed of many dozens of layers, accuracy can become saturated, and some degradation can occur. Some talk about a different problem called "vanishing gradient" in which the gradient fluctuations become too small to be immediately useful. The deep residual network deals with some of these problems by using residual blocks, which take advantage of residual mapping to preserve inputs. By utilizing deep residual learning frameworks, engineers can experiment with deeper networks that have specific training challenges.

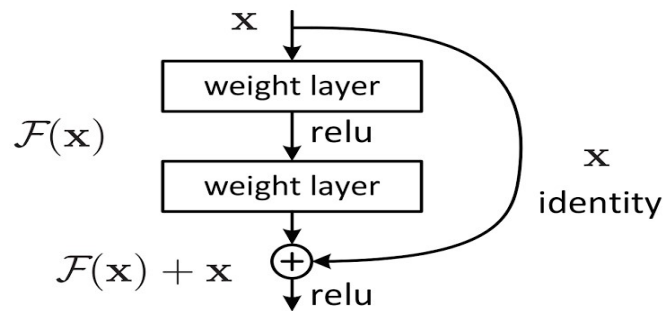


Fig. 3.39: Residual Learning

The depth of neural networks is absolutely important in obtaining better performance. When neural network layers are very deep, they exhibit higher training and validation loss due to information loss leading to vanishing gradients. To guarantee that deeper networks would always yield better accuracy than shallow networks, ResNet offered the use of residual blocks, bringing in information from the past to compensate for information loss. This technique enables the training of very deep networks resulting in state of the art accuracy on standard benchmarks. To reduce computation cost due to very deep layers, bottleneck layers were used with 1 x 1 convolutions that reduce the number of feature maps of the input. Since ResNet has been discussed in detail in the earlier sections, we had made it brief. Applying this Deep ResNet, might reach new state of the art results on all standard computer vision benchmarks. The idea has since been expanded into all other dominions of deep learning including speech and natural language processing. Image Recognition has advanced in recent years due to availability of large datasets and powerful GPUs that has enabled training of very deep architectures like Deep ResNet.

3.7.7 Deep Neural SVM

Recently, fully-connected and convolutional neural networks have been trained to achieve state-of-the-art performance on a wide variety of tasks such as speech recognition, image classification, bioinformatics, and natural language processing. For classification tasks, most of these “deep learning” models employ the softmax activation function for prediction and minimize cross-entropy loss. Deep Neural SVM demonstrates a small but consistent advantage of replacing the softmax layer with a linear support vector machine. Learning minimizes a margin-based loss instead of the cross-entropy loss. While there have been various combinations of neural nets and SVMs in prior art, using L2-SVMs shows that by simply replacing softmax with linear SVMs gives significant gains on popular deep learning datasets MNIST, CIFAR-10, and the ICML 2013 Representation Learning Workshop’s face expression recognition challenge. Lower layer weights are learned by back-propagating the gradients from

the top layer linear SVM. To do this, we need to differentiate the SVM objective with respect to the activation of the penultimate layer. Deep Neural SVM works better than softmax on 2 standard datasets and a recent dataset. Switching from softmax to SVMs is incredibly simple and appears to be useful for classification tasks. Further research is needed to explore other multiclass SVM formulations and better understand where and how much the gain is obtained.

As a rule of thumb, SVMs are great for relatively small data sets with fewer outliers. And deep learning algorithms... well, they require "relatively" large datasets to work well, and you also need the infrastructure to train them in reasonable time. Similarly, deep learning algorithms require much more experience: Setting up a neural network using deep learning algorithms is much more tedious than using off-the-shelf classifiers. On the other hand, deep learning really shines when it comes to complex problems such as image classification, natural language processing, and speech recognition. Another benefit is that you have to worry less about the feature engineering part. Yet again, in practice, the decision which classifier to choose really depends on your dataset and the general complexity of the problem, that's where your experience as deep learning practitioner kicks in. Deep Neural SVM shows better performance for Facial Expression Recognition and Image Classification applications.

3.7.8 Deep Autoencoders

It is frequently said that in machine learning (and more specifically deep learning) – it's not the person with the best algorithm that wins, but the one with the most data. We can always try and collect or generate more labelled data but it's an expensive and time-consuming task. This is where the promise and potential of unsupervised deep learning algorithms comes into the picture. They are designed to derive insights from the data without any supervision. Autoencoders are for unsupervised learning. It is designed to study a low-dimensional interpretation of a high-dimensional data set, analogous to Principal Components Analysis (PCA). The autoencoder model attempts to learn an approximation function $f(x) \approx x$ to reproduce the input data. However, it is restricted by a bottleneck layer in the middle with a very small number of nodes. With limited capacity, the model is forced to form a very efficient encoding of the data, which is essentially the low-dimensional code.

Autoencoders are hand-me-down to compress images. Also, both PCA and autoencoder can be applied to reduce the documents onto two dimensions; autoencoder verified a much better outcome. With the help of autoencoder, one can do efficient data compression to speed up the information retrieval including both documents and images. Autoencoders are a specific type of feed-forward neural networks where the input is the same as the output. They compress the input into a lower-dimensional code and then rebuild the output from this depiction. The code is a compact "summary" or "compression" of the input, also called the latent-space

representation. An autoencoder consists of three modules: encoder, code and decoder. With computer vision, deep autoencoders integrates the following applications;

- 📷 Image Reconstruction
- 📷 Image Compression
- 📷 Image Denoising
- 📷 Image Generation
- 📷 Apparel Recognition in Fashion

A simple autoencoder can be logically divided into two parts: an encoder part and a decoder part. The task of the encoder is to convert the input to a lower dimensional representation, while the task of the decoder is to recreate the input from this lower dimensional representation. Both the encoder and decoder are fully-connected feed-forward neural networks, essentially the Artificial Neural Networks (ANNs). Code is a single layer of an ANN with the dimensionality of our choice. The number of nodes in the code layer i.e. code size is a hyperparameter that is set before training the autoencoder. The input passes through the encoder, which is a fully-connected ANN, to produce the code. The decoder, which has the similar ANN structure, then produces the output only using the code. The goal is to get an output identical with the input. Note that the decoder architecture is the mirror image of the encoder.

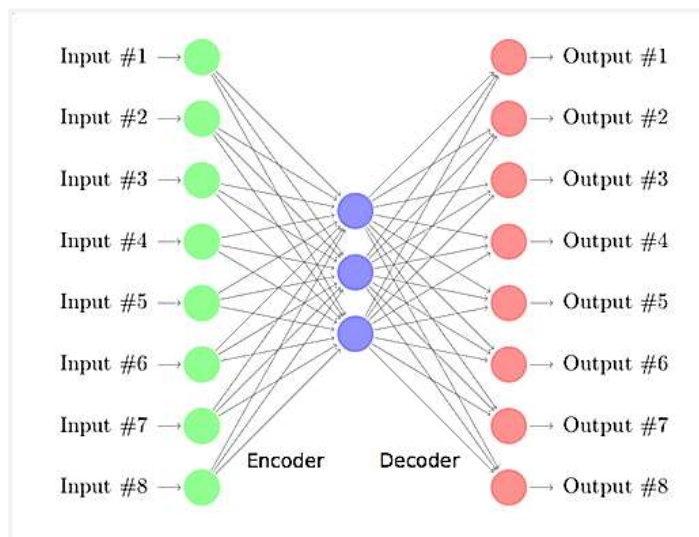


Fig. 3.40: Deep Autoencoders

This is not a requirement but it's typically the case. The only requirement is the dimensionality of the input and output needs to be the same. Anything in the middle can be played with. The four hyper-parameters that are supposed to be set before training are code size, number of

layers, number of nodes per layer, loss function. Keeping the code layer small, the autoencoder learns the intelligent representation of the data. There is another way to force the autoencoder to learn useful features, which is adding random noise to its inputs and making it recover the original noise-free data. This way the autoencoder can't simply copy the input to its output because the input also contains random noise forcing it to subtract the noise and produce the underlying meaningful data. This is called a denoising autoencoder.

Variational autoencoder (VAE) is one of the most popular approaches to learn the complicated data distribution such as images using neural networks in an unsupervised fashion. It is a probabilistic graphical model rooted in Bayesian inference i.e., the model aims to learn the underlying probability distribution of the training data so that it could easily sample new data from that learned distribution. The idea is to learn a low-dimensional latent representation of the training data called latent variables, which are not directly observed but are rather inferred through a mathematical model is assumed to have generated the actual training data. These latent variables can store useful information about the type of output the model needs to generate. The probability distribution of latent variables z is denoted by $P(z)$.

A Gaussian distribution is selected as a prior to learn the distribution $P(z)$ so as to easily sample new data points during inference time. Now the primary objective is to model the data with some parameters which maximizes the likelihood of training data X . In short, it is assumed that a low-dimensional latent vector has generated the data x ($x \in X$) and this latent vector can be mapped to data x using a deterministic function $f(z, \theta)$ parameterized by θ . Under this generative process, the aim is to maximize the probability of each data in X which is given as,

$$P_{\theta}(X) = \int P_{\theta}(X, z) dz = \int P_{\theta}(X|z) P_{\theta}(z) dz$$

At this point, $f(z, \theta)$ has been replaced by a distribution $P_{\theta}(X|z)$. The intuition behind this maximum likelihood estimation is that if the model can generate training samples from these latent variables then it can also generate similar samples with some variations. In other words, if a large number of latent variables from $P(z)$ is sampled and generates x from these variables then the generated x should match the data distribution $P_{data}(x)$. Now two questions are to be answered; how to capture the distribution of latent variables and how to integrate the above equation over all the dimensions of z ? Obviously, it is a tedious task to manually specify the relevant information to encode in latent vector to generate the output image.

Rather rely on neural networks to compute z just with a conjecture that this latent vector can be well approximated as a normal distribution so as to sample easily at inference time. Considering a normal distribution of z in n dimensional space then it is always possible to generate any kind of distribution using a sufficiently complicated function and the inverse of this function can be used to learn the latent variables itself. In the above equation, integration

is carried over all the dimensions of z and is therefore intractable. However, it can be calculated using methods of Monte-Carlo integration which is something not easy to implement. Therefore, another approach to approximately maximize $P_\theta(X)$ is accounted. The idea of VAE is to infer $P(z)$ using $P(z|X)$. $P(z|X)$ is inferred using a method called variational inference which is basically an optimization problem in Bayesian statistics. $P(z|X)$ is modelled using simpler distribution $Q(z|X)$ which is easy to find and tried to minimize the difference between $P(z|X)$ and $Q(z|X)$ using KL-divergence metric approach so that the hypothesis is close to the true distribution. The final objective function of VAE is;

$$\log P(X) - D_{KL}[Q(z|X)||P(z|X)] = E[\log P(X|z)] - D_{KL}[Q(z|X)||P(z)]$$

The above equation has a very nice interpretation. The term $Q(z|X)$ is basically the encoder net, z is the encoded representation of data $x(x \in X)$ and $P(X|z)$ is the decoder net. So in the above equation the goal is to maximize the log-likelihood of the data distribution under some error given by $D_{KL}[Q(z|X)||P(z|X)]$. It can be seen that VAE is trying to minimize the lower bound of $\log(P(X))$ since $P(z|X)$ is not tractable but the KL divergence term is ≥ 0 . This is same as maximizing $E[\log P(X|z)]$ and minimizing $D_{KL}[Q(z|X)||P(z|X)]$.

Maximizing $E[\log P(X|z)]$ is maximum likelihood estimation and is modelled using a decoder net. As said earlier that we want our latent representation to be close to Gaussian and hence we assume $P(z)$ as $N(0,1)$. Following this assumption, $Q(z|X)$ should also be close to this distribution. If we assume that it is a Gaussian with parameters $\mu(X)$ and $\Sigma(X)$, the error due to the difference between these two distributions i.e., $P(z)$ and $Q(z|X)$ given by KL divergence results in a closed form solution given below;

$$D_{KL}[N(\mu(X), \Sigma(X)) || N(0,1)] = \frac{1}{2} \sum_k (\exp(\sum_k \Sigma(X)) + \mu^2(X) - 1 - \sum_k \mu(X))$$

Consider optimizing the lower variational bound, optimization function is;

$$\log(P(X|z)) - D_{KL}[Q(z|X)||P(z)]$$

Hence, the loss function will contain two terms; firstly, reconstruction loss of the input to output and secondly, KL divergence loss. Now the network is trained using back-propagation algorithm. The first term doesn't only depend on the parameters of P but also on the parameters of Q . In-order to back-propagate through the layer, sampling z randomly from the distribution $Q(z|X)$ or $N[\mu(X), \Sigma(X)]$ so that P can be decoded. Gradients can't flow through random nodes. Reparameterization trick is used to make the network differentiable. Sample from $N(\mu(X), \Sigma(X))$ by first sampling $\epsilon \sim N(0, I)$, then computing $z = \mu(X) + \Sigma^{1/2}(X) * \epsilon$.

3.7.9 Deep Lambertian Networks

Deep Lambertian Networks is a combination of Deep belief network and Lambertian reflectance assumption which deals with the challenges posed by illumination variation in visual perception. Lambertian Reflectance model gives an illumination invariant representation which can be used for recognition. The Lambertian reflectance model is widely used for modelling illumination variations and is a good approximation for diffuse object surfaces. The Deep Lambertian Networks (DLN) is a hybrid undirected-directed generative model that combines DBNs with the Lambertian reflectance model. In the DLN, the visible layer consists of image pixel intensities $v \in \mathbb{R}^N$, where N is the number of pixels in the image. For every pixel i there are two latent variables namely the albedo and surface normal. Gaussian Restricted Boltzmann Machines (GRBMs) are used to model the albedo and surface normals. Deep Belief Network priors can be obtained by stacking additional binary RBM layers on top of the g and h layers as shown in Figure 3.41.

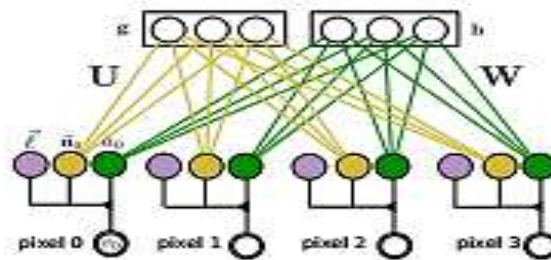


Fig. 3.41: A Deep Lambertian Network

Combining Deep Belief Nets with the Lambertian reflectance assumption, the model can accomplish good priors over the albedo from 2D images. Illumination variations can be explained by changing only the lighting latent variable. By transferring learned knowledge from similar objects, albedo and surface normals estimation from a single image is also possible. Experiments demonstrate that this model is able to generalize as well as improve over standard baselines in one-shot face recognition. The model has been successfully applied in reconstruction of shadows facial images, given any set of lighting conditions. The model has also been tested on non-living objects. The method outperforms most other methods and is faster than them.

3.7.10 You Only Look Once (YOLO)

YOLO (You Only Look Once) is a network for object detection. The object detection task consists in determining the location on the image where certain objects are present, as well as

classifying those objects. Previous methods for this, like R-CNN and its variations, used a pipeline to perform this task in multiple steps. This can be slow to run and also hard to optimize, because each individual component must be trained separately. YOLO, does it all with a single neural network. The first step to considerate YOLO is how it encodes its output. The input image is divided into an $S \times S$ grid of cells. For each object that is present on the image, one grid cell is said to be “responsible” for predicting it. That is the cell where the center of the object falls into. Each grid cell predicts B bounding boxes as well as C class probabilities.

The bounding box prediction has 5 components: $(x, y, w, h, \text{confidence})$. The (x, y) coordinates represent the center of the box; relative to the grid cell location (remember that, if the center of the box does not fall inside the grid cell, then this cell is not responsible for it). These coordinates are normalized to fall between 0 and 1. The (w, h) box dimensions are also normalized to $[0, 1]$, relative to the image size. Ceremoniously, we define confidence as $\text{Pr}(\text{Object}) * \text{IOU}(\text{pred}, \text{truth})$. If no object exists in that cell, the confidence score should be zero. Otherwise, we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth. Note that the confidence reflects the presence or absence of an object of any class. At this time, we understand the 5 components of the box prediction, remember that each grid cell makes B of those predictions, so there are in total $S \times S \times B \times 5$ outputs related to bounding box predictions.

It is also straightforward to predict the class probabilities, $\text{Pr}(\text{Class}(i)|\text{Object})$. This probability is conditioned on the grid cell having one object. In practice, it means that if no object is present on the grid cell, the loss function will not penalize it for a wrong class prediction. The network only predicts one set of class probabilities per cell, regardless of the number of boxes B . That makes $S \times S \times C$ class probabilities in total. Adding the class predictions to the output vector, we get $S \times S \times (B \times 5 + C)$ tensor as output. Once you understand how the predictions are encoded, the rest is easy. The network structure looks like a normal CNN, with convolutional and max pooling layers, followed by 2 fully connected layers in the end.

YOLO predicts multiple bounding boxes per grid cell. At training time we only want one bounding box predictor to be responsible for each object. We assign one predictor to be “responsible” for predicting an object based on which prediction has the highest current IOU with the ground truth. The Training process involved is as below;

- 🧩 Pretrain the first 20 convolutional layers using the ImageNet 1000-class competition dataset, using an input size of 224×224 .
- 🧩 Then, increase the input resolution to 448×448 .
- 🧩 Train the full network for about 135 epochs using a batch size of 64, momentum of 0.9 and decay of 0.0005.

- 📊 Learning rate schedule: For the first epochs, the learning rate was slowly raised from 0.001 to 0.01. Train for about 75 epochs and then start decreasing it.
- 📊 Use data augmentation with random scaling and translations, and randomly adjusting exposure and saturation.

CHAPTER 4

RECENT TRENDS IN COMPUTER VISION

4.1 INTRODUCTION

The current state of computer vision technology is powered by deep learning algorithms that utilize a distinctive kind of neural networks, Convolutional Neural Networks (CNNs), to make sense of images. These neural networks are trained to use thousands of sample images to assist the algorithm to comprehend and break down everything that is comprised of an image. CNNs scan images pixel by pixel which recognize patterns and memorize them. These networks also memorize the ideal output that they should offer for each input image or categorizes components of images by scanning characteristics like contours and colors. Afterward, this memory is practiced by the systems as the reference while scanning more images, and with every iteration, the AI system becomes better at providing the precise output. In several sectors computer vision technology is being used or tested, including healthcare, diagnosing diseases by assessing images received from CT scans and other medical procedures; Manufacturing, by leveraging this, manufacturers ensure manufacturing processes and finished products for non-conformance and imperfections; Security by using biometric analysis like retinal, facial recognition and fingerprint scanning to recognize individuals for security purposes.

Computer vision also is being leveraged in transportation to guide sovereign vehicles by detecting hurdles, people, and road signs along the way. As per the market research and insights, the automotive industry is one of the early adopters of computer vision systems and continues to lead the largest share of the computer vision market among other industries in the industrial vertical. Since computer vision swells its impact in the human world, there are numerous things to contemplate on how it will alter the way people see their lives and how they live it. With additional research on and fine-tuning of the technology, the future of computer vision will see it perform a wide array of functions. The technology will not only be easier to train but also be able to discern more from images than they perform now. Computer vision technologies will also be utilized alongside other technologies or other subsets of AI to fashion more compelling applications.

Image captioning applications, for instance, where it can be integrated with natural language generation (NLG) to interpret the objects in the surroundings for visually challenged people. In the development of artificial general intelligence (AGI) and artificial superintelligence (ASI), computer vision also can portray a significant role by providing them the ability to process information or even better than the human visual system. As a rising market, computer vision technology is firmly connected to visual and augmented reality (VR and AR). Indeed,

the market players in recent time have a keen interest in the amalgamation of VR and AR. And such attention is growing at a rapid pace which is being reflected in the introduction of many state-of-the-art technological products.

Computer Vision is a field of study that pursues to build practices to assist computers to see and comprehend the content of digital images like photographs and videos. It has been a subject of cumulative interest and rigorous research for decades now. The market for computer vision is anticipated to rise from US\$10.9 billion in 2019 to US\$17.4 billion by 2024, at a growing CAGR of 7.8%, according to a report. The market is majorly driven by an increasing need for quality inspection and automation, mounting demand for vision-guided robotic systems and application-specific computer vision systems. Computer vision is becoming more adroit at recognizing patterns from images than the human visual cognitive system. In the healthcare sector, for instance, computer vision-driven technologies have exceeded the pattern recognition capabilities of human physicians.

4.2 WHAT IS VISUAL QUESTION ANSWERING?

Visual Question Answering (VQA) is a new dataset containing open-ended questions about images. These questions require an understanding of vision, language and commonsense knowledge to answer. VQA is a recent problem in computer vision and natural language processing that has garnered a large amount of interest from the deep learning, computer vision, and natural language processing communities. In VQA, an algorithm needs to answer text-based questions about images. Since the release of the first VQA dataset in 2014, additional datasets have been released and many algorithms have been proposed.

It is a research area about building a computer system to answer questions presented in an image and a natural language. Because Visual Question Answering requires techniques involving image recognition and natural language processing, one major direction in research is on Deep Learning: using Convolutional Neural Network (CNN) for image recognition, using Recurrent Neural Network (RNN) for natural language processing, and then combining the results to deliver the final answer as shown in as an example in Figure 4.1.

Historically, building a system that can answer natural language questions about any image has been considered a very ambitious goal. As humans we can normally perform this task without major inconveniences, the development of a system with these capabilities has always seemed closer to science fiction than to the current possibilities of Artificial Intelligence (AI). However, with the advent of Deep Learning (DL), we have witnessed enormous research progress in Visual Question Answering (VQA), in such a way that systems capable of answering these questions are emerging with promising results. In a general way we can define a VQA system as an algorithm that takes as input an image and a natural language question about the image

and generates a natural language answer as the output. This is by nature a multi-discipline research problem. We need NLP for at least two reasons: to understand the question and to generate the answer. These are common problems in text-based Q&A, a well-studied problem in NLP. Given the following sentence:

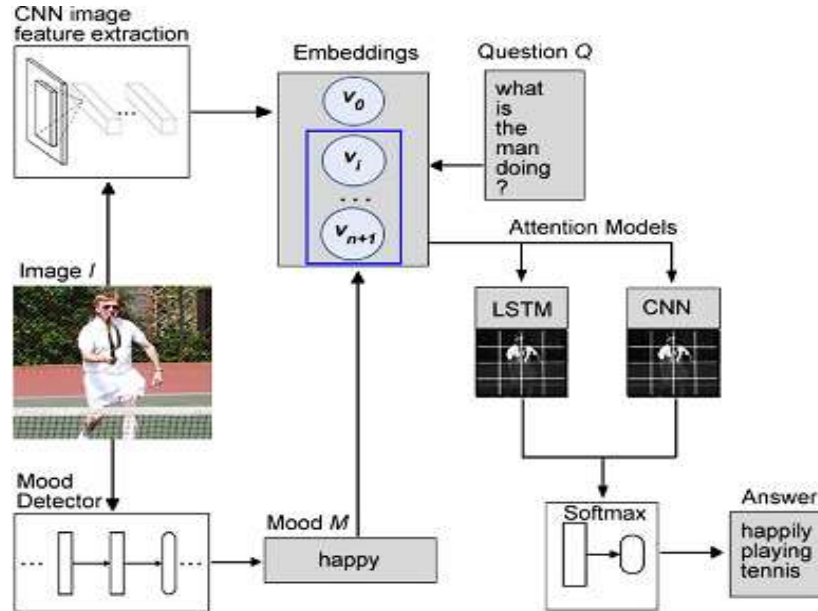


Fig. 4.1: A Visual Question Answering Model

“How many bridges are there in Paris?” A NLP Q&A system is typically going to:

- ✚ Classify or type the question: this is a “how many” question, so the response must be a number.
- ✚ Extract the object to count: bridges.
- ✚ Extract the context where the count must be performed: in this case, the city of Paris.

After the question has been analyzed, the system builds some kind of query and relies on a knowledge base to get the answer. This is far from trivial (e.g. there are at least 22 cities called Paris in the United States), but a lot of work has been done since the 1970s. The main difference in VQA is that the search and the reasoning part must be performed over the content of an image. So, to answer if there are any humans, the system must be able to detect objects. To say if it is raining, it needs to classify a scene. To answer who are the teams, the system needs world knowledge. Finally, to say which player has the ball, commonsense reasoning and, very likely, knowledge reasoning is necessary. Many of these tasks (object recognition, object detection, scene classification, etc.) have been addressed in the field of Computer Vision (CV),

with impressive results in the last few years. A good VQA system must be capable of solving a broad spectrum of typical NLP and computer vision tasks, as well as reasoning about image content. It is clearly a multi-discipline AI research problem, involving CV, NLP and Knowledge Representation & Reasoning (KR).

Available datasets: With many problems in NLP or CV, such as Machine Translation, Image Captioning or Entities Recognition, the availability of datasets is a key issue. They let, in combination with well-defined metrics, to fairly compare different systems, to compare them with human decisions and to measure how they perform in an absolute way, that is, to determine the empirical limitations of the state-of-the-art. The VQA field is so tricky that a good dataset should be large enough to capture the long range of possibilities within questions and image content in real world scenarios. Many of the datasets contain images from the Microsoft Common Objects in Context (COCO), a dataset with 328,000 images having 91 objects types that would be easily recognizable by a 4 year old, with a total of 2.5 million labeled instances.

The COCO dataset can simplify and accelerate the process of building a VQA dataset. However, it is not an easy task. For example, collecting varied, convenient and non-ambiguous questions is a great challenge. Besides the variety and precision problem, a good dataset also has to avoid biases that could be exploited. For example, given a dataset with only yes/no answers, where 90% of the answers are yes, a trivial most frequent class strategy will obtain an accuracy of 90%, without solving anything in terms of VQA. To get deeper into the VQA complexity, let's see some of the most important datasets released.

The DAQUAR dataset: The first significant VQA dataset was the DATaset for QUEStion Answering on Real-world images (DAQUAR). It contains 6794 training and 5674 test question-answer pairs, based on images from the NYU-Depth V2 Dataset. That means about 9 pairs per image on average. Although it is a great initiative, the NYU dataset contains only indoor scenes with, sometimes; lightning conditions that make it difficult to answer the questions. In fact, evaluation on humans shows an accuracy of 50.2%. The other drawback of the DAQUAR dataset is that its size makes it unsuitable for training and evaluating complex models.

The COCO-QA dataset: The COCO-QA dataset is significantly larger than DAQUAR. It contains 123,287 images coming from the COCO dataset, 78,736 training and 38,948 testing QA pairs. To create such a large amount of QA pairs, the authors used a NLP algorithm to automatically generate them from the COCO image captions. For example, for a given caption such as "Two chairs in a room", they would generate a question like "How many chairs are there?" It must be noted that all the answers are a single word. Although this is very clever, the obvious issue here is that the queries suffer from the inherent NLP limitations, so they are

sometimes bizarrely formulated or have grammatical errors. In some cases, they are simply incomprehensible: Another inconvenience is that the dataset only has four kinds of questions, not equally distributed: object (69.84%), color (16.59%), counting (7.47%) and location (6.10%).

The VQA dataset: Compared to other datasets, the VQA dataset is rather larger. In addition to 204,721 images from the COCO dataset, it includes 50,000 abstract cartoon images. There are three questions per image and ten answers per question that is over 760 K questions with about 10M answers. To achieve this, a team of Amazon Mechanical Turk workers generated the questions and another team wrote the answers. One interesting point is that for testing, they propose two kinds of answer modes: open-ended and multiple-choice. For the first mode, they propose the following metric:

Accuracy = $\min(\frac{\# \text{ humans that provided that answer}}{3}, 1)$ That means that an answer is considered 100% accurate if at least 3 workers provided that exact answer. For the multiple-choice mode, they create 18 candidate answers (correct and incorrect) per question:

- ✚ **The Correct Answer:** The most common answer given by the ten annotators.
- ✚ **Plausible Answers:** Three answers collected from annotators without looking at the image.
- ✚ **Popular Answers:** The top ten most popular answers in the dataset (e.g. “yes”, “no”, “2”, “1”, “white”, “3”, “red”, “blue”, “4”, “green” for real images)
- ✚ **Random Answers:** Randomly selected correct answers for other questions.

Despite the precautions taken in the design of the dataset (e.g. the inclusion of popular answers makes it more difficult to infer the type of question from the set of answers), we can observe some issues. Maybe the most striking one is that some questions are too subjective to have a single right answer. We cannot say that this man has children, but it is without doubt the most likely answer. However, there are questions such as “Would you like to fly in that?” over the image of an airplane, that do not have a correct answer. Finally, a major drawback regards what we could call trivial questions, which are questions that generally don’t need an image to have a good (or likely) answer. For example: “how many legs does the dog have?” or “what color are the trees?” where, although not always correct, four and green are the most frequent and obvious answers.

Current approaches: In general, we can outline the approaches in VQA as follows:

- ✚ Extract features from the question.
- ✚ Extract features from the image.
- ✚ Combine the features to generate an answer.

For text features, techniques such as Bag-Of-Words (BOW) or Long Short Term Memory (LSTM) encoders can be used. In the case of image features, CNNs pre-trained on ImageNet is the most frequent choice. Regarding the generation of the answer, the approaches usually model the problem as a classification task. Thus, the main difference between several approaches is how they combine the textual and image features. For example, they can simply combine them using concatenation and then feed a linear classifier. Or they can use Bayesian models to infer the underlying relationships between the feature distributions of the question, the image and the answer.

Baselines: As for many classification problems, a trivial baseline consists on always giving the most frequent answer to any question. Another trivial baseline is to pick up a random answer. For instance, the work by Antol et al. (2016) shows that always selecting the most popular answer from the top 1K answers on the VQA dataset (the answer is “yes”) leads to an accuracy of 29.72%. Leaving aside that this probably has to do with an undesired dataset bias; such a result illustrates the importance of having good baselines: they determine the minimal level of performance acceptable, and can also give a hint about the inherent complexity of the task and/or the dataset. A more sophisticated baseline, widely used in VQA, consists on training a linear classifier or a multilayer perceptron using vectors representing a combination of the features as input. This combination can be a simple concatenation or an element wise sum or product of the features. For example, the prior cited work experiments with two models:

- 🧩 A multi-layer perceptron (MLP) neural network classifier with two hidden layers and 1000 hidden units (dropout 0.5) in each layer with tanh non-linearity.
- 🧩 An LSTM model followed by a softmax layer to generate the answer.

In the first case, for the textual features a BOW approach is used, using the top 1,000 words in the questions and the 1,000 most popular words in the captions to compute them. For the image features they use the last hidden layer of VGGNet. As for the LSTM model, they use a one-hot encoding for the questions, and the same image features as above followed by a linear transformation to transform the image features to 1024 dimensions to match the LSTM encoding of the question. Question and image encodings are combined via element wise multiplication. The performances achieved by these baselines are very interesting.

For example, if the models are trained only on textual features, the accuracy is 48.09%, whereas if they are trained only on visual features it goes down to 28.13%. Their best model, a LSTM trained on both kinds of features, has an accuracy of 53.74%. The results on multiple-choice are better than open-answer and, as expected, all methods are sizably worse than human performance. Many variations in this framework can be implemented to obtain different baselines.

Approaches based on attention: The goal of the attention-based methods is to set the focus of the algorithm on the most relevant parts of the input. For example, if the question is “What color is the ball?” the image region containing the ball is more relevant than the others. In the same way, “color” and “ball” are more informative than the rest of the words.

The most common choice in VQA is to use spatial attention to generate region specific features to train CNNs. There are two common methods to obtain the spatial regions of an image. First, by projecting a grid over the image. After the grid has been applied, the relevance of each region is determined by the specific question. The other way is by proposing automatically generated bounding boxes. Given the proposed regions, we can use the question to determine the relevance of the features for each one, and to pick only those that are necessary to answer the question. These are only two techniques to incorporate attention to VQA systems, and many more can be found in the literature.

Bayesian approaches: The idea behind bayesian approaches is to model co-occurrence statistics of both the question and the image features, as a way of inferring relationships between questions and images. In Kafle and Kanan (2016), for example, the authors model the probability of image features given the question features and the type of the answer. They do so because they observe that given a question, the type of answer can be frequently predicted. For example, “How many players are in the image?” is a “how many” question, that needs a number as an answer. To model the probabilities they combine a Bayesian model with a discriminative model. Regarding the features, they use ResNet for the images and skip-thought vectors for text.

Evaluation metrics: It may seem obvious, but it is good to remember: the performance of any system (and consequently the state-of-the-art in a given domain) depends on the metrics with which it is evaluated. In VQA, the most straightforward metric we can use is classic accuracy. While this is a reasonable option for multiple-choice answer systems, when it comes to open-ended answers it tends to be very penalizing. If the ground truth answer is “oak tree”, does that mean that the answer “tree” is absolutely incorrect? If the question is “What animals appear in the image?”, and the image shows dogs, cats and rabbits, how incorrect the answer “cats and dogs” should be? These are very complex issues that have to be addressed in order to evaluate the different approaches as precisely as possible.

Real life applications: There are many potential applications for VQA. Probably the most direct application is to help blind and visually-impaired users. A VQA system could provide information about an image on the Web or any social media. Another obvious application is to integrate VQA into image retrieval systems. This could have a huge impact on social media or e-commerce. VQA can also be used with educational or recreational purposes. The VQA

Consortium has a very complete and useful website with information, resources and software on VQA. They hold the VQA Challenge as well as the VQA Challenge Workshop, and it is worthwhile to take a look at the tasks, talks and papers because they give a good hint about the future directions of the field. You can also try CloudCV, a very interesting online demo of a VQA system.

VQA is a recent field that requires the understanding of both text and vision. Since deep learning techniques are significantly improving NLP and CV results, we can reasonably expect that VQA is going to be more and more accurate in the coming years. As with many other tasks in IA, the built datasets and the defined metrics have somehow shaped the research work that has been done so far. The best way to evaluate a VQA system is still an open question, and it is very likely that new datasets and metrics will allow deepening and refining the notion of quality. How the phrasing of a question can bias a VQA system? There are still several remaining issues and ongoing discussions.

For example, is it good, appropriate or fair to use multiple-choice datasets, given that a VQA system can be right by chance? Is a system with excellent performance in multiple-choice datasets a good system in real life, where these are not available? What is the impact of the way a question is formulated in the response that is obtained? Although current performances are far from human decisions, the results are already exploitable and, in fact, very promising. As VQA is adopted by large public platforms, devices and tools, it is very likely that it will change the way we search and interact with data.

4.3 WHAT IS 3D OBJECT UNDERSTANDING?

Object and scene understanding involves figuring out, at the very least, what is in an image and where things are. Moreover, we want to know information about the scene and how objects in it are spatially related. The dominant paradigms treat this as primarily a pattern recognition problem that involves learning some filter-based representation of images that makes the detection and classification problem easier. In contrast, recognition often brings in 3D knowledge about objects in a variety of ways like;

-  Object modeling
 -  Object detection
 -  Object recognition
 -  Scene understanding
 -  Scene segmentation
 -  Humans interacting with objects
 -  Machine learning methods
 -  Statistical modeling of scene properties
-

Geometric models and reasoning

Scene understanding, in contrast to object recognition, attempts to analyze objects in context with respect to the 3D structure of the scene, its layout, and the spatial, functional, and semantic relationships between objects. Research in this area combines object detection/recognition with 3D reconstruction and spatial reasoning. The integrated analysis of low-level image features, together with high-level semantic and 3D object models, will enable robust scene understanding in complex and ambiguous emblems and will provide the foundation for further reasoning.

As humans, we influence the world through our bodies. We express our emotions through our facial expressions and body posture. We manipulate and change the world with our hands. For computers to be full partners with humans, they have to see us and understand our behavior. They have to recognize our facial expressions, our gestures, our movements and our actions. This means that we need robust algorithms and expressive representation that can capture human pose, motion, and behavior. There is a long history of research on this topic but, in Perceiving Systems, focus has always been on developing 3D representations of the body, extracting these from images and video, and using these as the basis for human behavior analysis.

Representing and extracting 3D body shape and pose has not been the dominant paradigm in the field but this is now changing. The outline of the SMPL body model helped change this. Skinned Multi-Person Linear Model (SMPL) is accurate, easy to use, compatible with game engines, differentiable, and is now widely used both in research and industry. It can be easily fit to image data "top down" or integrated into the end-to-end training of neural networks. In the last three years, SMPL has proven to fit image data and to train deep networks end-to-end to excerpt the full-body shape and pose from single images or video. This comprises the following methods, which provide foundational tools for capturing and analyzing human motion in natural settings;

-  SMPLify
-  Unite the People
-  Human Mesh Recovery (HMR)
-  Neural Body Fitting

But the body alone is not enough to capture human behavior; the face and hands convey critical information. Consequently, we have developed new models of the face and hands and have combined these with SMPL, creating a unified model with face, hands and body. We are often interested in the interaction between multiple people and between people and objects. Consequently, it has developed methods for detecting and tracking people in crowded scenes,

for recognizing hand-object interactions, and for tracking 3D human motion in the wild. The ultimate goal is to understand behavior. To do so we first want to capture it at scale. This means robustly and efficiently tracking human behavior in natural settings and relating that behavior to the 3D world around the person as shown in Figure 4.2.

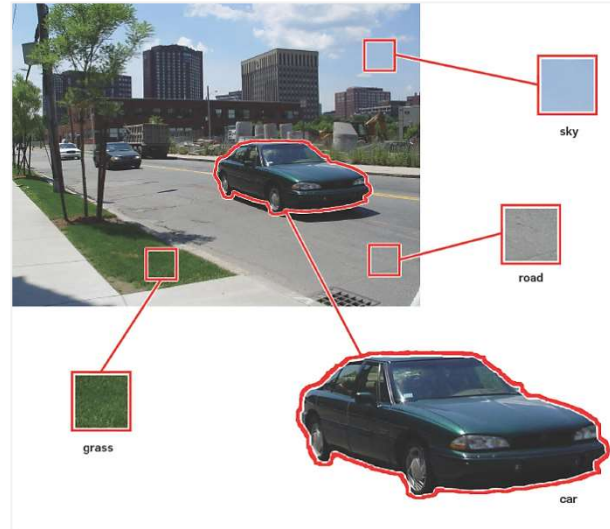


Fig. 4.2: 3D Object Understanding

For moving scenes, we compute the optical flow signifying the projection of the 3D motion field into the image. In doing so, we adventure the geometric structure of the problem to simplify it. If the scene is rigid and only the camera moves, then the optical flow is completely described by the depth of the scene and the camera motion. Real scenes, however, contain rigid structure and independently moving objects. To deal with this, we segment the scene into regions corresponding to the different types of motion. To do so we exploit different constraints that is both geometric and semantic. In the latter case, we know that certain objects like animals and cars can move independently while others, like buildings, cannot. Additionally objects like the road are typically planar and hence their motion is simply modeled. Thus, a semantic segmentation of the scene provides information about what motions may be present where.

4.4 WHAT IS OBJECT RECOGNITION WITH POINT CLOUD?

Recently, there are a lot of researches related to the object detection with LiDAR point cloud. The LiDAR point cloud inputs have the advantage of accurate depth information. Multi-view 3D networks (MV3D), a sensory-fusion framework that takes both LIDAR point cloud and RGB images as input and predicts oriented 3D bounding boxes. The network is composed of two subnetworks: one for 3D object proposal generation and another for multi-view feature

fusion. The network generates 3D candidate boxes efficiently from the bird's eye view representation of 3D point cloud. The deep fusion scheme combines region-wise features from multiple views and enable interactions between intermediate layers of different paths.

Apple invented a deep neural network configuration that can segment objects in point clouds obtained with a LIDAR sensor. Recently Apple joined the field of autonomous vehicles. Although we don't know a lot about their car, a lot of companies that are building autonomous vehicles use a so-called LIDAR to detect obstacles around them. The LIDAR emits a light-pulse and measures the time it takes for this pulse to return to its sensor. This gives a distance from the vehicle to nearby obstacles. Rotating around the sensor can give a distance to obstacles all around the sensor. Distances obtained with a LIDAR are stored in a so-called point cloud. After visualizing this collection of points, humans are good at detecting several types of objects in these point clouds such as humans, cars, and bicycles. Unfortunately, this remains a difficult task for computers.

Traditional methods rely on handcrafted features to make sense of this data. Examples of hand-crafted features are methods that segment the cloud into sub-clouds, or methods that separate the point cloud into surfaces. Another way to make sense of the LIDAR data is selecting a viewpoint and putting the image through existing computer-vision algorithms. A point cloud is a set of data points in space. Point clouds are generally produced by 3D scanners, which measure many points on the external surfaces of objects around them. The downside of these approaches is that designing these features is difficult, and it's difficult to design features which generalize well to all situations. Apple now created an end-to-end neural network to solve this problem. This approach does not rely on any hand-crafted features or other machine learning algorithms than neural networks.

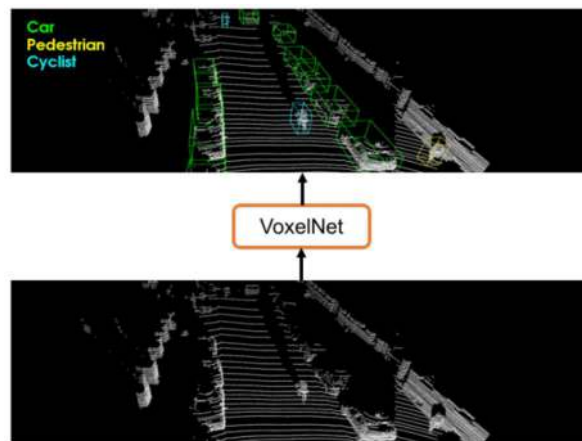


Fig. 4.3: Apples Feature Learning Network

The first part of their approach is the so-called "Feature learning network". Apple divides the world into so-called voxels (3D pixels). When detecting cars they make each voxel have a size of 2 meters high and 2.4 meters wide (so a car fits nicely into one voxel). In every voxel, they randomly select a subset of points (some voxels contain a lot of points, some only a few, this way each voxel contains the same amount of input for the neural network). They feed this subset of points through a neural network which creates a representation in 128-dimensional space. Doing this for each point in the world gives you a data structure you can feed through the same network architecture that you see in neural network approaches to computer vision. With several convolutional layers, the neural network projects its output on a probability map and a regression map. The probability map indicates for each voxel in the world if it contains an object or not. The regression map indicates for each voxel where in this voxel the object is.

Apple tested their approach on the KITTI Vision Benchmark. They compared their approach with other approaches that use similar data, but that DO use handcrafted features. It turns out that their approach has a better performance than all existing approaches, including approaches that DO use handcrafted features. With this research, Apple shows the approach they are taking in their autonomous vehicle project. This year it was released that Apple is using six out of a total of twelve LIDAR sensors on the top of their car.

4.5 WHAT ARE SEMANTIC AND INSTANCE SEGMENTATION?

Starting from AlexNet, high accuracy is obtained by convolutional neural network (CNN) for image classification, numerous CNN approaches are developed for other tasks such as object detection, semantic segmentation, and instance segmentation. DeepMask is the CNN approach for instance segmentation.

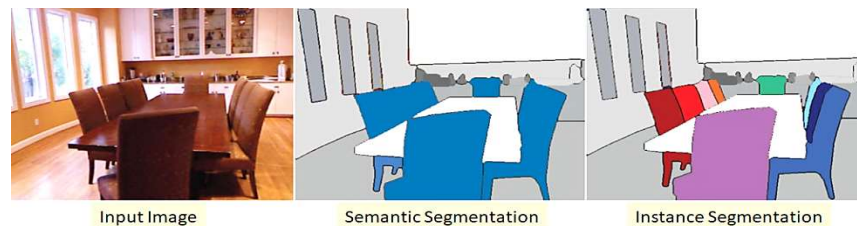


Fig. 4.4: Apples Feature Learning Network

-  **Image Classification:** Classify the main object category within an image.
-  **Object Detection:** Identify the object category and locate the position using a bounding box for every known object within an image.
-  **Semantic Segmentation:** Identify the object category of each pixel for every known object within an image. Labels are class-aware.

 **Instance Segmentation:** Identify each object instance of each pixel for every known object within an image. Labels are instance-aware.

Deep Mask: In 2015, it is one of the earliest deep learning using CNN for instance segmentation. It is worth to learn it to know the development of deep-learning-based instance segmentation. Since a region proposal can be generated based on the predicted segmentation mask, object detection task can also be performed. The model as shown in Figure 4.5, given the input image patch x , after feature extraction by VGGNet, The fully connected (FC) layers originated in VGGNet are removed. The last max pooling layer in VGGNet is also removed, thus the output before splitting into two paths are of the size of $1/16$ of input. For example as above, the input is 224×224 (3 is the number of channels in the input image, i.e. RGB), the output at the end of main branch is $(224/16) \times (224/16) = 14 \times 14$. (512 is the number of feature maps after convolution.)

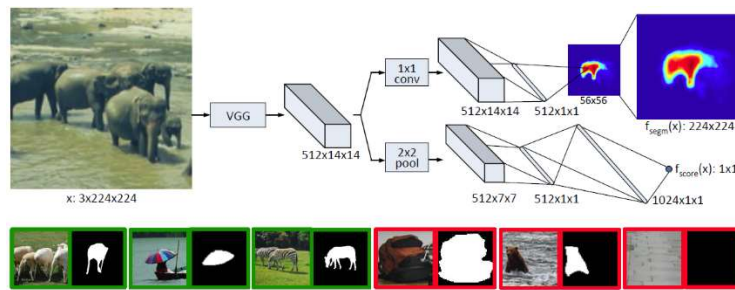


Fig. 4.5: Deep Mask Model Architecture (Top), Positive Samples (Green, Left Bottom), Negative Samples (Red, Right Bottom)

Left Bottom: Positive Samples

A label $y_k=1$ is given for k -th positive sample. To be a positive sample, two criteria need to be satisfied:

-  The patch contains an object roughly centered in the input patch.
-  The object is fully contained in the patch and in a given scale range.

When $y_k=1$, the ground truth mask m_k has positive values for the pixels which belong to the single object located in the centre of the image patch.

Right Bottom: Negative Samples

Otherwise, a label $y_k=-1$ is given for a negative sample even the object is partially present. When $y_k=-1$, the mask is not used.

There are two paths after VGGNet:

- ✚ The first path is to predict the class-agnostic segmentation mask, i.e. $f_{\text{segm}}(x)$.
- ✚ The second path is to assign a score corresponding to how likely the patch is to contain an object, i.e. $f_{\text{score}}(x)$.

Top, First Path: Predicting Segmentation Map

1×1 convolution is performed first without changing the number of feature maps; non-linear mapping without dimension reduction is done here. After that, two FC layers are performed. (It is notice that there is no ReLU in between these two FC layers!) Unlike in semantic segmentation, the network must output a mask for a single object even when multiple objects are present. (Just like the elephant in the centre of the input image.) Finally, a 56×56 segmentation map is generated. And a simple bilinear interpolation is to upsample the segmentation map to 224×224 .

Top, Second Path: Predicting Object Score

2×2 max pooling followed by two FC layers. Finally, one single value Predicted Object Score, $f_{\text{score}}(x)$, is obtained. Since positive samples are given based on the two criteria mentioned above, $f_{\text{score}}(x)$ is to predict whether the input image has satisfied these two criteria.

Loss Function

The network is trained to jointly learn the pixel-wise segmentation map $f_{\text{segm}}(x_k)$ at each location (i, j) and the predicted object score $f_{\text{score}}(x_k)$. The loss function is a sum of binary logistic regression losses, one for each location of the segmentation network $f_{\text{segm}}(x_k)$ and one for the object score $f_{\text{score}}(x_k)$. The first term implies that we only backpropagate the error over the segmentation path if $y_k=1$. If $y_k=-1$, i.e. the negative sample, the first term become 0 and will not contribute to the loss. Only the second term contributes the loss. For the sake of data balance, equal number of positive and negative samples is used. Batch size of 32 is used. Pretrained ImageNet model is used. There are 75M parameters in total. The model takes around 5 days to train on an Nvidia Tesla K40m.

During inference (testing), the model is applied densely at multiple locations with a stride of 16 pixels, and multiple scales from $1/4$ to 2 with a step size of square root of 2. This ensures that there is at least one tested image patch that fully contains each object in the image. Since the input test image is larger than the training input patch size, we need a corresponding 2D scoring map as an output rather than one single scoring value. An interleaving trick is used before the last max pooling layer for the scoring branch, i.e. the Fine Stride Max Pooling

proposed in OverFeat. To be brief, multiple max pooling is done on the feature map. A pixel shift is performed before each max pooling. 80,000 images and a total of nearly 500,000 segmented objects are used for training. And the first 5000 images of the MS COCO 2014 are used for validated. The inference time in MS COCO is 1.6s per image and in PASCAL VOC 2007 is 1.2s per image. Inference time can be further dropped by 30%, by parallelizing all scales in a single batch.



Fig. 4.6 (a): Deep Mask with $\text{IOU} < 0.5$ to the Ground Truth on Selected Images from COCO



Fig. 4.6 (b): Deep Mask with Highest IOU to the Ground Truth on Selected Images from COCO. Missed Objects (No Matching Proposals With $\text{IOU} > 0.5$) are Marked with a Red Outline.

Sharp Mask: Can a computer distinguish between the many objects in a photograph as effortlessly as the human eye? When humans look at an image, they can identify objects down to the last pixel. Over the past few years, progress in deep convolutional neural networks and the advent of ever more powerful computing architectures has led to machine vision systems rapidly increasing in their accuracy and abilities. We've witnessed massive advances in image classification (what is in the image?) as well as object detection (where are the objects?). See panels (a) and (b) in the image below (Figure 4.7). But this is just the beginning of understanding the most relevant visual content of any image or video. Recently we've have been designing techniques that identify and segment each and every object in an image, as in rightmost panel (c) of the image below, a key capability that will enable entirely new applications.

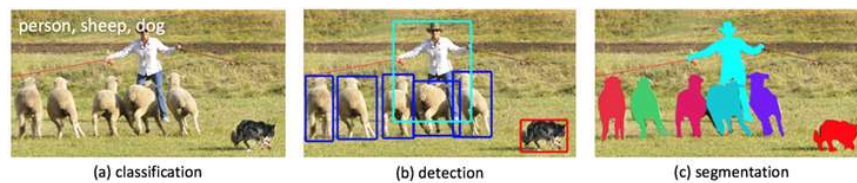


Fig. 4.7: Image Classification, Object Detection, Object Segmentation

The new algorithms driving our advances are the Deep Mask segmentation framework coupled with the Sharp Mask segment refinement module. Together, they have enabled machine vision systems to detect and precisely define every object in an image. The final stage of the recognition pipeline uses a specialized convolutional net, which is called as Multi Path Net; to label each object mask with the object type it contains (e.g. person, dog, sheep). Deep Mask, Sharp Mask as well as Multi Path Net help soon advance the field of machine vision.

Finding patterns in the pixels: Let's take a look at the building blocks of these algorithms. Glance at the first photo below (Figure 4.8), the one on the left. What do you see? A photographer operating his old-school camera, a grassy field and buildings in the background. And you probably notice countless other details as well. A machine sees none of this; an image is encoded as an array of numbers representing color values for each pixel, as in the second photo, the one on the right. So how do we enable machine vision to go from pixels to a deeper understanding of an image?

It's not an easy task, due to the near infinite variability of objects and scenes in real-world settings. Objects vary in their shapes and appearances, their sizes and positions, their textures and colors. Couple this with the inherent complexity of real scenes, variable backgrounds and lighting conditions, and the general richness of our world and you see can start to see how the problem can be so difficult for machines. Enter deep convolutional neural networks. Rather

than trying to programmatically define rule-based systems for object detection, deep nets are relatively simple architectures with tens of millions of parameters that are trained rather than designed. These networks automatically learn patterns from millions of annotated examples, and having seen a sufficient number of such examples start to generalize to novel images. Deep nets are mainly adapted at answering yes/no questions about an image, for example, does an image contain a sheep?

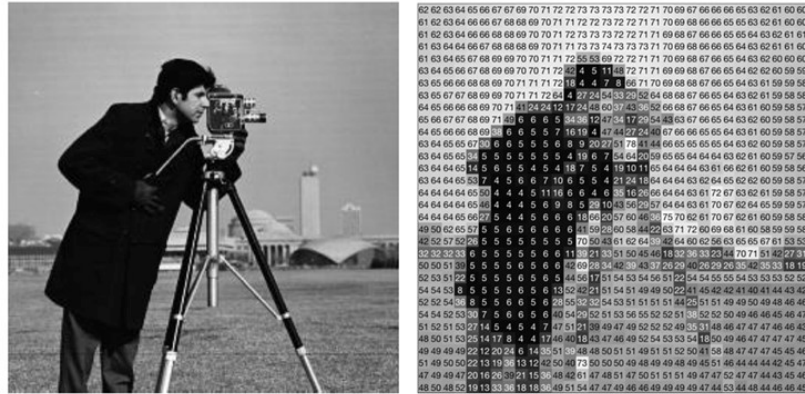


Fig. 4.8: Machine Vision to Deep Pixel Level Understanding an Image

Segmenting objects: So how do we employ deep networks for detection and segmentation? The technique we use in DeepMask is to think of segmentation as a very large number of binary classification problems. First, for every (overlapping) patch in an image we ask: Does this patch contain an object? Second, if the answer to the first question is yes for a given patch, then for every pixel in the patch we ask: Is that pixel part of the central object in the patch? We use deep networks to answer each yes/no question, and by cleverly designing our networks so that computation is shared for every patch and every pixel, we can quickly discover and segment all the objects in an image.

DeepMask employs a fairly traditional feedforward deep network design. In such networks, with progressively deeper network stages information is more abstract and semantically meaningful. For example, early layers in a deep net might capture edges and blobs, while upper layers tend to capture more semantic concepts such as the presence of an animal's face or limbs. By design, these upper-layer features are computed at a fairly low spatial resolution (for both computational reasons and in order to be invariant to small shifts in pixel locations). This presents a problem for mask prediction: The upper layer features can be used to predict masks that capture the general shape on an object but fail to precisely capture object boundaries. Which brings us to SharpMask? SharpMask enhances the output of DeepMask, generating higher-fidelity masks that more accurately delineate object boundaries.

While DeepMask predicts coarse masks in a feedforward pass through the network, SharpMask reverses the flow of information in a deep network and refines the predictions made by DeepMask by using features from progressively earlier layers in the network. Think of it this way: To capture general object shape, you have to have a high-level understanding of what you are looking at (DeepMask), but to accurately place the boundaries you need to look back at lower-level features all the way down to the pixels (SharpMask). In essence, it is aimed to make use of information from all layers of a network, with minimal additional overhead. Below are some examples outputs generated by DeepMask and refined by SharpMask (Figure 4.9). To keep the visualizations simple we only show predicted masks that best align to the objects actually present in the images (as annotated by humans). Note that the objects with red outlines are those annotated by humans but missed by DeepMask.

Classifying objects: DeepMask knows nothing about specific object types, so while it can delineate both a dog and a sheep, it can't tell them apart. Plus, DeepMask is not very selective and can generate masks for image regions that are not especially interesting. So how do we narrow down the pool of relevant masks and identify the objects that are actually present? As you might expect, we turn to deep neural networks once again. Given a mask generated by DeepMask, a separate deep net is trained to classify the object type of each mask (and “none” is a valid answer as well).

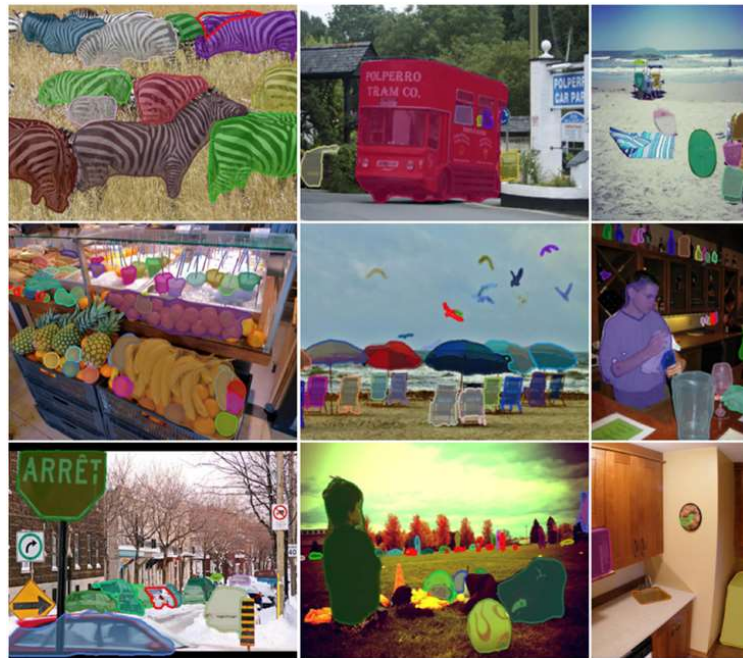


Fig. 4.9: Outputs Generated By Deepmask and Refined by Sharpmask

Here we are following the foundational paradigm called Region-CNN, or RCNN for short, pioneered by Ross Girshick. RCNN is a two-stage procedure where a first stage is used to draw attention to certain image regions, and in a second stage a deep net is used to identify the objects present. When developing RCNN, the first stage of processing available was fairly primitive. By using DeepMask as the first stage for RCNN and exploiting the power of deep networks we get a significant boost in detection accuracy and also gain the ability to segment objects.

To further boost performance, specialized network architecture to classify each mask (the second stage of RCNN) is focused. As we discussed, real-world photographs contains objects at multiple scales, in context and among clutter, and under frequent occlusion. Standard deep nets can have difficulty in such situations. To address this, we proposed a modified network that we call MultiPathNet. As its name implies, MultiPathNet allows information to flow along multiple paths through the net, enabling it to exploit information at multiple image scales and in surrounding image context. In summary, the object detection system follows a three stage procedure: DeepMask generates initial object masks, SharpMask refines these masks, and MultiPathNet detects the objects delineated by each mask.



Fig. 4.10: Objects Detected, Segmented and Classified by Sharpmask

There are wide ranging potential uses for visual recognition technology. Building off the present computer vision technology and empowering computers to recognize objects in photos,

for instance, it will be easier to search for specific images without an explicit tag on each photo. People with vision loss, too, will be able to understand what their friends share is in a photo because the system will be able to tell them, regardless of the caption posted alongside the image. Most recently it was demonstrated that this technology for blind users will assess photos and describe their content. Currently, visually impaired users browsing photos on Facebook only hear the name of the person who shared the photo, followed by the term “photo,” when they come upon an image in their News Feed. Instead we aim to offer richer descriptions, such as “Photo contains beach, trees, and three smiling people.”

Also, leveraging the segmentation technology, the goal is to enable even more immersive experiences that allow users to “see” a photo by swiping their finger across an image and having the system describe the content they're touching. As the technology move forward, you can imagine one day this image detection, segmentation, and identification capability applied to augmented reality in areas like commerce, health, or others.

4.6 WHAT IS TRANSFER LEARNING?

Transfer learning is a machine learning method where a model developed for a task is reused as the starting point for a model on a second task. It is a popular approach in deep learning where pre-trained models are used as the starting point on computer vision and natural language processing tasks given the vast compute and time resources required to develop neural network models on these problems and from the huge jumps in skill that they provide on related problems. Transfer learning is a machine learning technique where a model trained on one task is re-purposed on a second related task. “Transfer learning and domain adaptation refer to the situation where what has been learned in one setting is exploited to improve generalization in another setting.

Transfer learning is an optimization that allows rapid progress or improved performance when modeling the second task. Transfer learning is related to problems such as multi-task learning and concept drift and is not entirely an area of study for deep learning. Nevertheless, transfer learning is popular in deep learning given the enormous resources required to train deep learning models or the large and challenging datasets on which deep learning models are trained. Transfer learning only works in deep learning if the model features learned from the first task are general. This form of transfer learning used in deep learning is called inductive transfer. This is where the scope of possible models (model bias) is narrowed in a beneficial way by using a model fit on a different but related task. Humans have an inherent ability to transfer knowledge across tasks. What we acquire as knowledge while learning about one task, we utilize in the same way to solve related tasks. The more related the tasks, the easier it is for us to transfer, or cross-utilize our knowledge. Some simple examples would be,

- 📚 Know how to ride a motorbike (Learn how to ride a car)
- 📚 Know how to play classic piano (Learn how to play jazz piano)
- 📚 Know math and statistics (Learn machine learning)

In each of the above scenarios, we don't learn everything from scratch when we attempt to learn new aspects or topics. We transfer and leverage our knowledge from what we have learnt in the past! Conventional machine learning and deep learning algorithms, so far, have been traditionally designed to work in isolation. These algorithms are trained to solve specific tasks. The models have to be rebuilt from scratch once the feature-space distribution changes. Transfer learning is the idea of overcoming the isolated learning paradigm and utilizing knowledge acquired for one task to solve related ones. In this section, we will do a comprehensive coverage of the concepts, scope and real-world applications of transfer learning and even showcase some hands-on examples. The first thing to remember here is that, transfer learning, is not a new concept which is very specific to deep learning. There is a stark difference between the traditional approach of building and training machine learning models, and using a methodology following transfer learning principles.

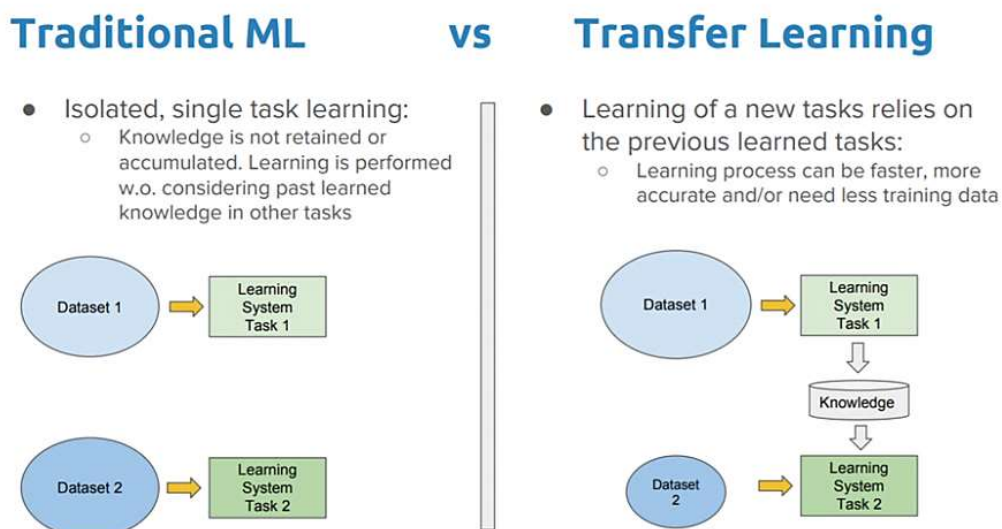


Fig. 4.11: Traditional Learning vs. Transfer Learning

Traditional learning is isolated and occurs purely based on specific tasks, datasets and training separate isolated models on them. No knowledge is retained which can be transferred from one model to another. In transfer learning, you can leverage knowledge (features, weights etc.) from earlier trained models for training newer models and even tackle problems like having less data for the newer task! Let's understand the preceding explanation with the help of an example.

Let's assume our task is to identify objects in images within a restricted domain of a restaurant. Let's mark this task in its defined scope as T1. Given the dataset for this task, we train a model and tune it to perform well (generalize) on unseen data points from the same domain (restaurant).

Traditional supervised ML algorithms break down when we do not have sufficient training examples for the required tasks in given domains. Suppose, we now must detect objects from images in a park or a café (say, task T2). Ideally, we should be able to apply the model trained for T1, but in reality, we face performance degradation and models that do not generalize well. This happens for a variety of reasons, which we can liberally and collectively term as the model's bias towards training data and domain.

Transfer learning should enable us to utilize knowledge from previously learned tasks and apply them to newer, related ones. If we have significantly more data for task T1, we may utilize its learning, and generalize this knowledge (features, weights) for task T2 (which has significantly less data). In the case of problems in the computer vision domain, certain low-level features, such as edges, shapes, corners and intensity, can be shared across tasks, and thus enable knowledge transfer among tasks! Also, as we have depicted in the earlier figure, knowledge from an existing task acts as an additional input when learning a new target task. Transfer learning, as we have seen so far, is having the ability to utilize existing knowledge from the source learner in the target task. During the process of transfer learning, the following three important questions must be answered:

-  **What to transfer:** This is the first and the most important step in the whole process. We try to seek answers about which part of the knowledge can be transferred from the source to the target in order to improve the recital of the target task. When trying to answer this question, we try to identify which portion of knowledge is source-specific and what is common between the source and the target.
-  **When to transfer:** There can be situations where transferring knowledge for the sake of it may make matters worse than refining anything, a negative transfer. We should aim at utilizing transfer learning to improve target task performance and not degrade them. We need to be careful about when to transfer and when not to.
-  **How to transfer:** Once what and when have been answered, we can proceed towards identifying ways of actually transferring the knowledge across domains/tasks. This involves changes to existing algorithms and different techniques. This should help us define the various scenarios where transfer learning can be applied and possible techniques.

There are different transfer learning strategies and techniques, which can be applied based on the domain, task at hand, and the availability of data. As Deep Learning continues to adapt and accelerate Transfer Learning and Meta Learning will grow as models become more commoditized, integrated and automated. The ability to use existing frameworks and models can be extended to new challenges and questions and in many cases these agents and models will train themselves and create efficiencies not even imagined in the recent past. These developments will help accelerate the potential of Machine Learning and Artificial Intelligence moving forward and increase the movement away from code bases and programmers to deep neural networks and other frameworks that write their own code and manage their own behaviors.

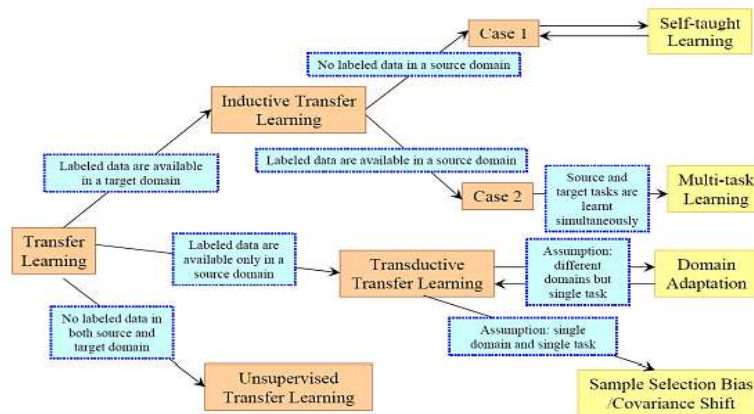


Fig. 4.12: Transfer Learning Strategies

Companies that are advanced in the space are already using layers of neural networks to run their business. These companies have built deep networks that in many cases are self-training and as data flows in learn, adapts and improve, decision making and operations seamless, friction-less and fluid. Some say Google for one is increasing relying on their collection of deep neural networks to run many of the tasks within their operations. The three transfer category outlines the different settings where transfer learning can be applied, and studied. Transfer learning methods can be classified based on the type of traditional ML algorithms involved:

- Inductive Transfer learning:** In this scenario, the source and target domains are the same, yet the source and target tasks are different from each other. The algorithms try to utilize the inductive biases of the source domain to help improve the target task. Depending upon whether the source domain contains labelled data or not, this can be further divided into two subcategories, similar to multitask learning and self-taught learning, respectively.

- ✚ **Unsupervised Transfer Learning:** This setting is similar to inductive transfer itself, with a focus on unsupervised tasks in the target domain. The source and target domains are similar, but the tasks are different. In this scenario, labelled data is unavailable in either of the domains.
- ✚ **Transductive Transfer Learning:** In this scenario, there are similarities between the source and target tasks, but the corresponding domains are different. In this setting, the source domain has a lot of labelled data, while the target domain has none. This can be further classified into subcategories, referring to settings where either the feature spaces are different or the marginal probabilities.

To answer the question of what to transfer across these categories, some of the following approaches can be applied:

- ✚ **Instance transfer:** Reusing knowledge from the source domain to the target task is usually an ideal scenario. In most cases, the source domain data cannot be reused directly. Rather, there are certain instances from the source domain that can be reused along with target data to improve results. Just in case of inductive transfer, modifications like AdaBoost by Dai help utilize training instances from the source domain for improvements in the target task.
- ✚ **Feature-representation transfer:** This approach aims to minimize domain divergence and reduce error rates by identifying good feature representations that can be utilized from the source to target domains. Depending upon the availability of labeled data, supervised or unsupervised methods may be applied for feature-representation-based transfers.
- ✚ **Parameter transfer:** This approach works on the assumption that the models for related tasks share some parameters or prior distribution of hyperparameters. Unlike multitask learning, where both the source and target tasks are learned simultaneously, for transfer learning; we may apply additional weightage to the loss of the target domain to improve overall performance.
- ✚ **Relational-knowledge transfer:** Unlike the preceding three approaches, the relational-knowledge transfer attempts to handle non-IID data, such as data that is not independent and identically distributed. In other words, data, where each data point has a relationship with other data points; for instance, social network data utilizes relational-knowledge-transfer techniques.

The following table clearly summarizes the relationship between different transfer learning strategies and what to transfer.

Table 4.2. Transfer Learning Strategies and What to Transfer

	Inductive Transfer Learning	Transductive Transfer Learning	Unsupervised Transfer Learning
Instance Transfer	✓	✓	
Feature Representation transfer	✓	✓	✓
Parameter transfer	✓		
Relational-knowledge transfer	✓		

Transfer Learning for Deep Learning: Deep learning models are representative of what is also known as *inductive learning*. The objective for inductive-learning algorithms is to deduce a mapping from a set of training examples. For instance, in cases of classification, the model learns mapping between input features and class labels. In order for such a learner to generalize well on unseen data, its algorithm works with a set of assumptions related to the distribution of the training data. These sets of assumptions are known as *inductive bias*. The inductive bias or assumptions can be characterized by multiple factors, such as the hypothesis space it restricts to and the search process through the hypothesis space. Thus, these biases impact how and what is learned by the model on the given task and domain. Inductive transfer techniques utilize the inductive biases of the source task to assist the target task. This can be done in different ways, such as by adjusting the inductive bias of the target task by limiting the model space, narrowing down the hypothesis space, or making adjustments to the search process itself with the help of knowledge from the source task. This process is depicted visually in Figure 4.13.

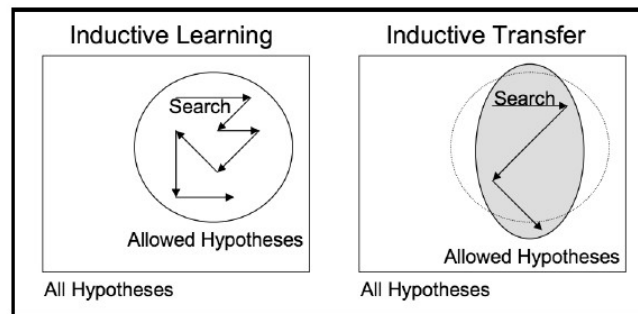


Fig. 4.13: Inductive Transfer

“Apart from inductive transfer, inductive-learning algorithms also utilize Bayesian and Hierarchical transfer techniques to assist with improvements in the learning and performance of the target task.

Deep Transfer Learning Strategies: Deep learning has made considerable progress in recent years. This has enabled us to tackle complex problems and yield amazing results. However, the

training time and the amount of data required for such deep learning systems are much more than that of traditional ML systems. There are various deep learning networks with state-of-the-art performance (at times as good as or even better than human performance) that have been radical and tested across domains such as computer vision and natural language processing (NLP).

In most cases, teams/people share the details of these networks for others to use. These pre-trained networks/models form the basis of transfer learning in the context of deep learning, or what we call 'deep transfer learning'. Let's look at the two most popular strategies for deep transfer learning.

(1) Off-the-shelf Pre-trained Models as Feature Extractors

Deep learning systems and models are layered architectures that learn different features at different layers (hierarchical representations of layered features). These layers are then finally connected to a last layer (usually a fully connected layer, in the case of supervised learning) to get the final output. This layered architecture allows us to utilize a pre-trained network (such as Inception V3 or VGG) without its final layer as a fixed feature extractor for other tasks.

The key idea here is to just leverage the pre-trained model's weighted layers to extract features but not to update the weights of the model's layers during training with new data for the new task. For instance, if we utilize AlexNet without its final classification layer, it will help us transform images from a new domain task into a 4096-dimensional vector based on its hidden states, thus enabling us to extract features from a new domain task, utilizing the knowledge from a source-domain task.

This is one of the most widely utilized methods of performing transfer learning using deep neural networks. Now a question might arise, how well do these pre-trained off-the-shelf features really work in practice with different tasks? It definitely seems to work really well in real-world tasks. The features from the pre-trained models consistently out-perform very specialized task-focused deep learning models.

(2) Fine Tuning Off-the-shelf Pre-trained Models

This is a more involved technique, where we do not just replace the final layer (for classification/regression), but we also selectively retrain some of the previous layers. Deep neural networks are highly configurable designs with various hyperparameters. As discussed earlier, the initial layers have been seen to capture generic features, while the later ones focus more on the specific task at hand.

An example depicted in the Figure 4.14 on a face-recognition problem, where initial lower layers of the network learn very generic features and the higher layers learn very task-specific features. Using this insight, we may freeze (fix weights) certain layers while retraining, or fine-tune the rest of them to suit our needs. In this case, we utilize the knowledge in terms of the overall architecture of the network and use its states as the starting point for our retraining step. This, in turn, helps us achieve better performance with less training time.

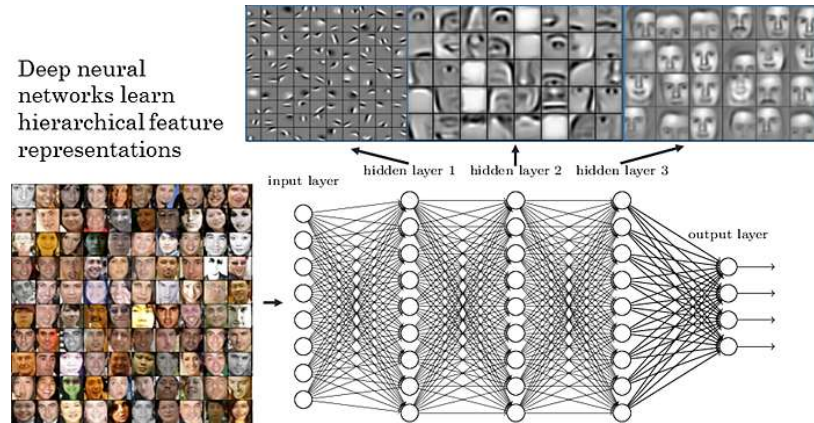


Fig. 4.14: Deep Neural Networks and Feature Representation

(3) Pre-trained Models

One of the fundamental requirements for transfer learning is the presence of models that perform well on source tasks. Luckily, the deep learning world believes in sharing. Many of the state-of-the-art deep learning designs have been openly shared by their respective teams. These span across different domains, such as computer vision and NLP, the two most popular domains for deep learning applications. Pre-trained models are typically shared in the form of the millions of parameters/weights the model realized while being trained to a stable state. Pre-trained models are available for everyone to use through different means. The famous deep learning Python library, keras, provides an interface to download some popular models. You can also access pre-trained models from the web since most of them have been open-sourced.

For computer vision, you can leverage some popular models including,

- 📁 VGG-16
- 📁 VGG-19
- 📁 Inception V3
- 📁 Xception
- 📁 ResNet-50

They definitely hold a lot of promise and it is sure they will be widely adopted pretty soon for real-world applications. For natural language processing tasks, things become more difficult due to the varied nature of NLP tasks. You can leverage word embedding models including,

- 📚 Word2Vec
- 📚 GloVe
- 📚 FastText
- 📚 Universal Sentence Encoder by Google
- 📚 Bidirectional Encoder Representations from Transformers (BERT) by Google

Types of Deep Transfer Learning: The literature on transfer learning has gone through a lot of iterations, and as mentioned at the start of this chapter, the terms associated with it has been used loosely and often interchangeably. Henceforth, it is sometimes confusing to differentiate between transfer learning, domain adaptation, and multi-task learning. Rest assured, these are all related and try to solve similar problems. In broad-spectrum, you should always think of transfer learning as a general concept or principle, where we will try to solve a target task using source task-domain knowledge.

(1) Domain Adaptation

Domain adaption is usually referred to in scenarios where the marginal probabilities between the source and target domains are different. There is an inherent shift or drift in the data distribution of the source and target domains that requires tweaks to transfer the learning. For instance, a corpus of movie reviews labeled as positive or negative would be different from a corpus of product-review sentiments. A classifier trained on movie-review sentiment would see a different distribution if utilized to classify product reviews. Therefore, domain adaptation techniques are utilized in transfer learning in these scenarios.

(2) Domain Confusion

We learned different transfer learning strategies and even discussed the three questions of what, when, and how to transfer knowledge from the source to the target. In particular, we discussed how feature-representation transfer can be useful. It is worth re-iterating that different layers in a deep learning network capture different sets of features. We can utilize this fact to learn domain-invariant features and improve their transferability across domains. Instead of allowing the model to learn any representation, we nudge the representations of both domains to be as similar as possible. This can be achieved by applying certain pre-processing steps directly to the representations themselves. The basic idea behind this technique is to add another objective to the source model to encourage similarity by confusing the domain itself, hence domain confusion.

(3) Multitask Learning

Multitask learning is a slightly different flavor of the transfer learning world. In the case of multitasks learning, several tasks are learned simultaneously without distinction between the source and targets. In this case, the learner receives information about multiple tasks at once, as compared to transfer learning, where the learner initially has no idea about the target task. This is depicted in the following Figure 4.15.

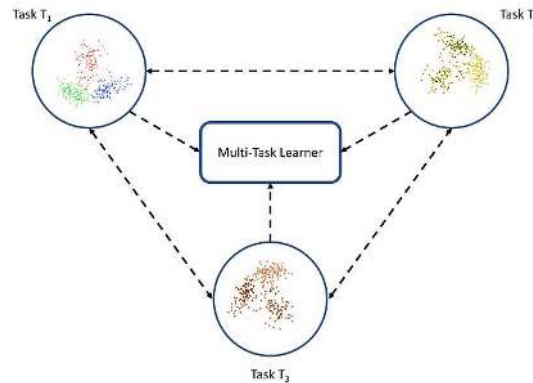


Fig. 4.15: Learner Receives Information from All Tasks Simultaneously

(4) One-shot Learning

Deep learning systems are data-hungry by nature, such that they need many training examples to learn the weights. This is one of the limiting aspects of deep neural networks, though such is not the case with human learning. For instance, once a child is shown what an apple looks like, they can easily identify a different variety of apple (with one or a few training examples); this is not the case with deep learning algorithms. One-shot learning is a variant of transfer learning, where we try to infer the required output based on just one or a few training examples. This is essentially helpful in real-world scenarios where it is not possible to have labeled data for every possible class (if it is a classification task), and in scenarios where new classes can be added often.

(5) Zero-shot Learning

Zero-shot learning is another extreme variant of transfer learning, which relies on no labelled examples to learn a task. This might sound incredible, especially when learning using examples is what most supervised learning algorithms are about. Zero-data learning or zero-shot learning methods make clever adjustments during the training stage itself to exploit additional information to understand unseen data. Zero-shot learning is a scenario where three variables

are learned, such as the traditional input variable, x , the traditional output variable, y , and the additional random variable that describes the task, T . The model is thus trained to learn the conditional probability distribution of $P(y | x, T)$. Zero-shot learning comes in handy in scenarios such as machine translation, where we may not even have labels in the target language.

Applications of Transfer Learning: Deep learning is definitely one of the specific categories of algorithms that have been utilized to reap the benefits of transfer learning very successfully. The following are a few examples:

- 📚 **Transfer learning for NLP:** Textual data presents all sorts of defies when it comes to ML and deep learning. These are frequently transformed or vectorized using different techniques. Embeddings, such as Word2vec and FastText, have been prepared using diverse training datasets. These are utilized in dissimilar tasks, such as sentiment analysis and document classification, by transferring the knowledge from the source tasks. Besides this, newer models like the Universal Sentence Encoder (USE) and Bidirectional Encoder Representations from Transformers (BERT) definitely present a myriad of possibilities for the future.
- 📚 **Transfer learning for Audio/Speech:** Similar to domains like NLP and Computer Vision, deep learning has been successfully used for tasks constructed on audio data. For instance, Automatic Speech Recognition (ASR) models developed for English have been successfully used to improve speech recognition performance for other languages. Also, automated-speaker identification is another example where transfer learning has greatly helped.
- 📚 **Transfer learning for Computer Vision:** Deep learning has been quite successfully utilized for various computer vision tasks, such as object recognition and identification, employing different CNN architectures in findings how the lower layers act as conventional computer-vision feature extractors, such as edge detectors, while the final layers work toward task-specific features.

Thus, these findings have helped in utilizing existing state-of-the-art models, such as VGG, AlexNet, and Inceptions, for target tasks, such as style transfer and face detection, that were different from what these models were trained for initially.

4.7 WHAT IS DEEP DOMAIN ADAPTATION?

During the last decade, the field of Computer vision has made huge progress. This progress is mostly due to the undeniable effectiveness of Convolutional Neural Networks (CNNs). CNNs allow for very precise predictions if trained with high-quality annotated training data. In a classification setting for instance, you would often use one of the standard network

architectures out there (ResNet, VGG etc.) and train it using your dataset. This would likely lead to very good performance. On the other hand, if you do not have huge manually annotated datasets for your particular problem at your disposal, CNNs also allow for making use of others who already trained a network for a similar problem by transfer learning. In this case, you would take a network that is pretrained on a large dataset and tune some of its upper layers using your own small annotated dataset. Both of these approaches assume that your training data (be it large or small) is representative of the underlying distribution.

However, if the inputs at test time differ significantly from the training data, the model might not perform very well. Let us assume, for instance, that you are an autonomous vehicle engineer and you want to segment the images captured by the car's camera in order to know what lies ahead (buildings, trees, other cars, pedestrians, traffic lights etc.). You have nice human-generated annotations for your NYC dataset and you train a large network using these annotations. You test your self-driving car on the streets of Manhattan and everything seems to work fine. Then you test the same system in Paris and suddenly things go very wrong. The car is not able to detect traffic lights anymore, the cars look very different (there are no yellow cabs in Paris) and the streets are not as straight anymore.

The reason your model did not perform very well in these scenarios is that the problem domain changed. In this particular case, the domain of the input data changed while the task domain (the labels) remained the same. On other occasions, you may want to use data from the same domain (drawn for the same underlying distribution) to accomplish a new task. Similarly, the input and task domains could differ at the same time. In these cases, domain adaptation comes to your rescue. Domain adaptation is a sub-discipline of machine learning which deals with scenarios in which a model trained on a source distribution is used in the context of a different (but related) target distribution. In general, domain adaptation uses labelled data in one or more source domains to solve new tasks in a target domain. The level of relatedness between the source and target domains hereby usually determines how successful the adaptation will be.

There are multiple approaches to domain adaptation. In “shallow” (not deep) domain adaptation, two methods are commonly used: reweighing the source samples and training on the reweighed samples and trying to learn a shared space to match the distributions of the source and target datasets. While these techniques may also be applicable in the context of deep learning, the deep features learned by Deep Neural Networks (DNNs) usually give rise to more transferable representations (generally learning highly transferable features in the lower layers while the transferability sharply decreases in higher layers), in Deep Domain Adaptation, we try to make use this property of DNNs.

Domain Adaptation Categories: Domain adaptation is defined to be a problem in which the task space is the same and the difference is solely in input domain divergence. Based on this

definition, domain adaptation can be either homogeneous (the input feature spaces are the same, but have different data distributions) or heterogeneous (the feature spaces and their dimensionalities differ). Domain adaptation may also occur in one step (one-step domain adaptation), or over multiple steps, traversing one or more domains in the process (multi-step domain adaptation). One-step domain adaptation is the most common type of domain adaptation.

Depending on the data you have available from the target domain, domain adaptation can further be classified into supervised (you do have labelled data from the target domain, albeit an amount which is too small for training a whole model), semi-supervised (you have both labelled and unlabelled data), and unsupervised (you do not have any labelled data from the target domain). How can we determine if a model trained in a source domain may be adapted to our target domain? It turns out that this question is not easy to answer and task relatedness is still a topic of active research. We can define two tasks to be similar if they use the same features to make a decision. Another possibility is to define two tasks as similar if their parameter vectors (i.e. classification boundaries) are close. On the other hand, two tasks are F-related if the data for both tasks can be generated from a fixed probability distribution using a set of transformations F .

Despite these theoretical deliberations, in practice it is probably necessary to try domain adaptation on your own datasets to see if you can obtain some benefit for your target task by using a model from the source task. Often, task relatedness can be determined by simple reasoning, such as images from different viewing angles or different lighting conditions, or, in the medical field, images from different devices and so on.

One-Step Domain Adaptation: There are three basic techniques for one-step domain adaptation:

- 🚦 Divergence-Based Domain Adaptation
- 🚦 Adversarial-Based Domain Adaptation Using Generative Models (Gans) or Using Domain-Confusion Loss
- 🚦 Reconstruction-Based Domain Adaptation Using Stacked Autoencoders (Saes) or Gans.

(1) Divergence-based Domain Adaptation

Divergence-based domain adaptation mechanize by minimizing some divergence criterion between the source and target data distributions, thus achieving a domain-invariant feature representation. If we find such a feature representation, the classifier will be able to perform equally well on both domains. This of course assumes that such a representation exists which

in turn assumes that the tasks are related in some way. The four most frequently used divergence measures are;

- ✚ Maximum Mean Discrepancy (MMD)
- ✚ Correlation Alignment (CORAL)
- ✚ Contrastive Domain Discrepancy (CCD)
- ✚ Wasserstein metric.

MMD is a hypothesis test which tests whether two samples are from the same distribution by comparing the means of the features after mapping them to a Reproducing Kernel Hilbert Space (RKHS). If the means are different, the distributions are likely different as well. This is often capable by using the kernel embedding trick and comparing the samples using a Gaussian kernel. The intuition here is that if two distributions are identical, the average (mean) similarity between samples from each distribution should be equal to the average similarity between mixed samples from both distributions. An example for using MMD in domain adaptation is, two-stream architecture maybe used with weights which are not shared but which lead to similar feature representations by using a combination of classification, regularization and domain discrepancy (MMD) loss, as in the figure below.

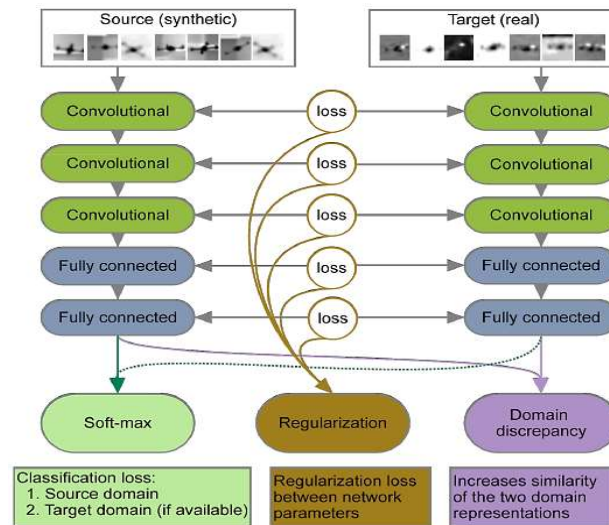


Fig. 4.16: Two-stream architecture

Hereby, the setting may either be supervised, semi-supervised or even unsupervised (no classification loss in target domain). CORAL is similar to MMD; however it tries to align the second-order statistics (correlations) of the source and target distributions instead of the means using a linear transformation. CORAL can be used in the context of deep learning by

constructing a differentiable CORAL loss using the Frobenius norm between the source and target covariance matrices.

CCD is also based on MMD but also makes use of label distributions by looking at conditional distributions. This makes sure that the joint domain features still retain predictivity w.r.t the labels. Minimizing CCD minimizes the intra-class discrepancy while maximizing the inter-class discrepancy. This requires both source and target domain labels. In order to get rid of this constraint, estimate missing target labels using clustering in an iterative procedure which jointly optimizes target labels and feature representations. Hereby target labels are found by clustering, whereupon CCD is minimized to adapt the features. Finally, feature and label distributions in the source and target domains may be aligned by considering the optimal transport problem and its consistent distance, the Wasserstein distance. The discrepancies between joint deep feature representations and labels perhaps minimized through optimal transport.

(2) Adversarial-based Domain Adaptation

This technique tries to achieve domain adaptation by using adversarial training. One approach is to generate synthetic target data which are somehow related to the source domain (e.g. by retaining labels) using Generative Adversarial Networks (GANs). These synthetic data are then used to train the target model. The CoGAN model tries to achieve this by using two generator/discriminator pairs for both the source and target distributions. Some weights of the generators and the discriminator are shared to learn a domain-invariant feature space. In this way, labeled target data can be generated that can further be used in a task such as classification. In another installation, try to learn a source/target converter network by using two discriminators: one for ensuring that the target data are real, the other to preserve relatedness between the source and target domains. The generator is hereby conditioned on the source data. This approach only requires unlabeled data in the target domain.

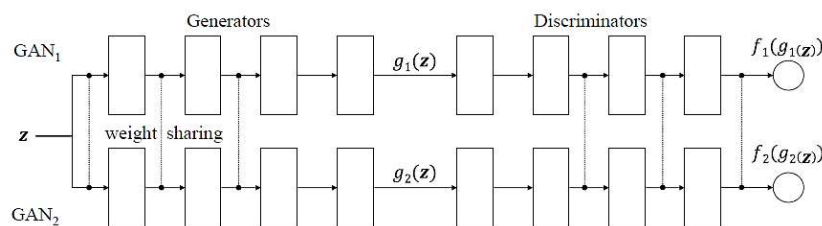


Fig. 4.17: CoGAN Architecture

We can also get rid of generators altogether and perform domain adaptation in one go if we use the so-called domain confusion loss in addition to the classification loss used for the current

task. The domain confusion loss is similar to the discriminator in GANs in that it tries to match the distributions of the source and target domains in order to “confuse” the high-level classification layers. Perhaps the most famous example of such a network is the Domain-Adversarial Neural Network (DANN). This system consists of two losses, the classification loss and the domain confusion loss. It contains a gradient reversal layer to match the feature distributions. By minimizing the classification loss for the source samples and the domain confusion loss for all samples (while maximizing the domain confusion loss for the feature extraction), this makes sure that the samples are mutually indistinguishable for the classifier.

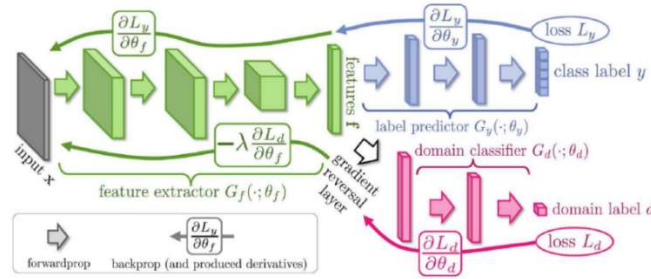


Fig. 4.18: Domain-Adversarial Neural Network Architecture

(3) Reconstruction-based Domain Adaptation

This approach uses an auxiliary reconstruction task to create a shared representation for each of the domains. Deep Reconstruction Classification Network (DRCN) attempts to solve these two tasks simultaneously: classification of the source data, and reconstruction of the unlabeled target data. This makes sure that the network learns not only to discriminate correctly but also preserves data about the target. The reconstruction pipeline learns to transform source images into images that resemble the target dataset, suggesting that a common representation is learned for both.

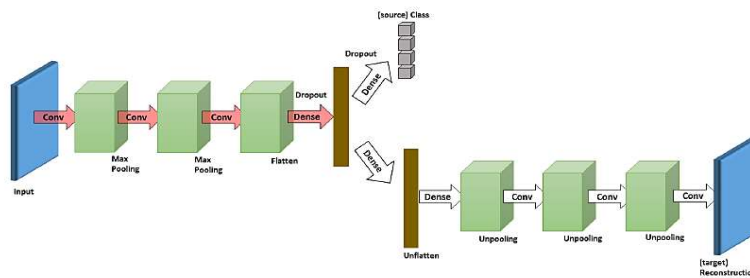


Fig. 4.19: DRCN architecture

Another possibility is to use so-called cycle GANs. Cycle GANs are stirred by the concept of dual learning in machine translation. This concept trains two opposite language translators (A-B, B-A) simultaneously. Feedback signals in the loop consist of the corresponding language models and the mutual BLEU scores. The same can be done with images using the im2im framework. Mapping from one image domain to another is learned without using any paired image samples. This is accomplished by separately training two GANs that generate the images in the two domains, respectively. To ensure consistency, a cycle consistency loss is introduced.

This makes sure that the transformation from one domain to another and back again results in roughly the same image as the input. Thus, the full loss of the two paired networks is a sum of the GAN losses of both discriminators and the cycle consistency loss. Finally, GANs can also be used in an encoder-decoder setting by conditioning their input on the image from another domain. Conditional GANs are used to translate images from one domain to another by conditioning the discriminator and the output of the generator on the input. This can be achieved using simple encoder-decoder design or alternatively using a U-Net architecture with skip-connections.

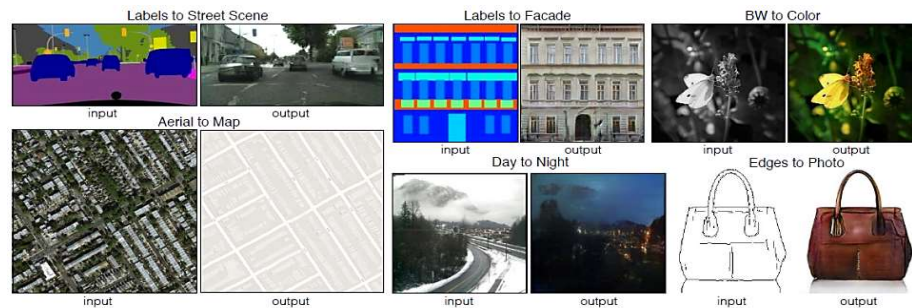


Fig. 4.20: Conditional GAN

Deep domain adaptation allows us to transfer the knowledge learned by a particular DNN on a source task to a new related target task. It has been successfully applied in tasks such as image classification or style transfer. In some sense, deep domain adaptation enables us to get closer to human-level recital in terms of the amount of training data required for a particular new computer vision task. Thus, progress in this area will be crucial to the entire field of Computer Vision and it will sooner or later lead us to effective and simple knowledge reuse across visual tasks.

4.8 WHAT IS SIMULTANEOUS LOCATION AND MAPPING?

Simultaneous localization and mapping (SLAM) is the synchronous location awareness and recording of the environment in a map of a computer, device, robot, drone or other autonomous

vehicle. SLAM is a key module in self-driving vehicles and other autonomous robots enabling awareness of where they are and the best routes to where they are going. By creating its own maps, SLAM enables quicker, more autonomous and adaptable response than pre-programmed routes. A number of emitters and sensors work together in sensor fusion with an AI for a single purpose in SLAM. A robot that uses SLAM, for example, employs various types of cameras and sensors such as radar, Lidar, ultrasonic and other tools to understand its environment. By better understanding its environment, a robot can more effectively map, navigate, avoid obstacles and adjust to changes. Highly accurate GPS modules have reduced the need for SLAM in some applications. High-precision GPS can almost entirely replace SLAM in some outdoor environments. GPS may suffer reduced performance or outages, and SLAM can fill in the gaps in navigation where more detail is needed and also take over in the case of these difficulties.

SLAM is the process of creating a map using a robot or unmanned vehicle that navigates that environment while using the map it generates. SLAM is technique behind robot mapping or robotic cartography. The robot or vehicle plots a course in an area, but at the same time, it also has to figure out where its own self is located in the place. The process of SLAM uses a complex array of computations, algorithms and sensory inputs to navigate around a previously unknown environment or to revise a map of a hitherto known environment. SLAM enables the remote creation of GIS data in situations where the environment is too dangerous or small for humans to map. SLAM is becoming a gradually important topic within the computer vision community, and is receiving particular interest from the augmented and virtual reality industries. With a variety of SLAM systems being made available, from both academia and industry, it is worth exploring what exactly is meant by SLAM. This section will give a brief introduction to what SLAM is (and what it isn't), what it's for, and why it's important, in the context of computer vision research and development, and augmented reality in particular.

'SLAM' is not a particular algorithm or piece of software, but rather it refers to the problem of trying to simultaneously localize (i.e. find the position/ orientation of) some sensor with respect to its surroundings, while at the same time mapping the structure of that environment. This can be done in a number of different ways, depending on the situation; so when I refer to a 'SLAM system', take that to mean 'a set of algorithms working to solve the simultaneous localization and mapping problem'. SLAM is not necessarily a computer vision problem only, and need not involve visual information at all: in fact much of the early research on SLAM involved ground based robots equipped with laser scanners. However, discussing mostly visual SLAM - where the primary mode of sensing is via a camera - since it is of most interest in the context of augmented reality, but many of the themes discussed can apply more generally. The requirement of recovering both the camera's position and the map, when neither is known to begin with, distinguishes the SLAM problem from other tasks.

For example, marker-based tracking is not SLAM, because the marker image (analogous to the map) is known beforehand. 3D reconstruction with a fixed camera rig is not SLAM either, because while the map (here the model of the object) is being recovered, the positions of the cameras are already known. The challenge in SLAM is to recover both camera pose and map structure while initially knowing neither. An essential division between SLAM and other apparently similar methods of recovering pose and structure is the requirement that it must operate in real time. This is a rather slippery concept, but in general it means that the processing done on each incoming camera image must be finished by the time the next one arrives, so that the pose of the camera is available immediately and not as a result of a post-processing stage. This distinguishes SLAM from techniques like structure from motion, where a set of unordered images are processed offline to recover the 3D structure of an environment, in a potentially (extremely) time consuming process. This can achieve impressive results (see here example), but crucially can't tell you where the camera is during acquisition.

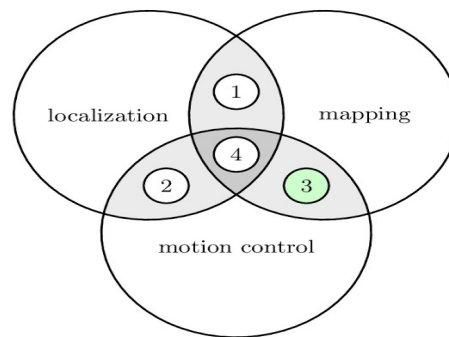


Fig. 4.21: Simultaneous Location and Mapping

A Brief History of SLAM: Research on the SLAM problem began within the robotics community, usually with wheeled robots traversing a flat ground plane. Typically this was done by combining sensor readings (such as from a laser scanner or sonar) with information about the control input (e.g. steering angle) and the measured robot state (e.g. counting wheel rotations). This may seem far removed from tracking a handheld camera freely moving in space, but embodies many of the core difficulties of SLAM, such as creating a consistent and accurate map, and making best use of multiple unreliable sources of information. More recently the use of visual sensors has become an important aspect of SLAM research, in part because an image provides a rich source of information about the structure of the environment (containing more information than a sonar ping for example). A large amount of research on visual SLAM used stereo cameras, or cameras alongside other sensors (such as accelerometers or GPS), but since around 2001 a number of works exhibited how SLAM could be done successfully using only a single camera (known as monocular visual SLAM).

This was crucial in making SLAM a more widely useful technology, since devices equipped with just a single camera, such as webcams and mobile phones are vastly more common and accessible than specialist sensing equipment. More recent work has demonstrated how monocular visual SLAM can be used to create large scale maps, how the maps can be automatically enhanced with meaningful 3D structures, and recover awfully detailed shapes in real time. SLAM is an active field of research within computer vision and new and improved techniques are always emerging. How SLAM Works? The majority of modern visual SLAM systems are based on tracking a set of points through successive camera frames, and using these tracks to triangulate their 3D position; while simultaneously using the estimated point locations to calculate the camera pose which could have observed them. It may seem that it is impossible to compute one without the other indeed from only a few points it is but by observing a sufficient number of points it is possible to solve for both structure and motion. Even with a single camera, by carefully combining measurements of points made over multiple frames it is possible to recover pose and structure with high accuracy, up to an unknown global scale factor.

At its heart, SLAM is an optimization problem, where the goal is to compute the best configuration of camera poses and point positions in order to minimize reprojection error (the difference between a point's tracked location and where it is expected to be given the camera pose estimate, over all points). The method of choice to solve this problem is called bundle adjustment, a nonlinear least squares algorithm which, given a suitable starting configuration, iteratively approaches the minimum error for the whole system. The problem with bundle adjustment is that it can be very time consuming to find the best solution, and the time taken grows quickly as the size of the map increases. With the advent of multi-core machines this is solved by separating the localization from the mapping. This means localization, the tracking of points in order to estimate the current camera pose can happen in real time on one thread, while the mapping thread can run bundle adjustment on the map in the background. On completion the mapping thread updates the map used for tracking, and in turn the tracker adds new observations to expand the map.

Thus can be viewed that tracking is continuous but the map is only optimized from time to time. Aside from details such as the method of tracking points, initialization of the map, robustness to incorrect matches and various strategies for optimizing the map more efficiently, the above describes the basic operation of a SLAM system. Beyond this, additional functionality is often incorporated in order to make SLAM more practical in the real world. For example, a crucial feature for larger scale mapping is loop closure, whereby the gradual accumulation of errors over time can be ameliorated by associating the current tracked location to an earlier part of the map, thereby enforcing extra constraints on the optimization problem to get an overall more correct map. Another essential technique is known as relocalization, which is able to cope with temporary poor tracking performance, which could otherwise cause

the system to fail completely. This allows the tracker to be restarted by finding which part of the previously visited map looks most like the current camera view.

This is demonstrated below using our SLAM system, where tracking fails (due to moving away from the mapped area before it can be expanded) and is recovered as soon as the map is seen again. This also solves the 'kidnapped camera' problem, where the camera view is blocked while being moved to a different location: tracking resumes successfully even after changing the camera's viewpoint.

Applications: As per the global robotics research, localizing a camera within an unknown environment is useful when exploring and navigating terrain for which no prior map (or GPS signal) is available, it is even being used to explore the surface of Mars. This ability to accurately localize a camera without a prior reference is also crucial to its use in augmented reality. This is how virtual content can be locked in place with respect to the real world: because the real world is what makes the map, and the camera which takes the images to be augmented has its position estimated constantly. As such, fast and accurate localization is vital (otherwise lag or drift in the rendered graphics would be apparent). The difference between SLAM and something like our marker tracker is that it is not necessary to have a pre-specified image target to create augmentations; nor is it necessary for any particular objects to remain in view. More sophisticated augmented reality uses are possible due to the fact that, as a necessary step in performing localization, a map is being built.

This enables applications beyond simply adding virtual content into the world coordinate frame, where virtual content can be aware of and react to real objects present in the scene. Rather than building up a map of a surrounding environment, a SLAM system can equally well be used to build a 3D reconstruction of an object, simply by moving a camera around it. For example, Slam System is used to show the process of creating a point-based 3D model of a complicated object, then to track it while deliberately ignoring the background. It is important to note that while SLAM enables a wide range of applications, there are some things for which it is not the right tool. SLAM systems work on the assumption that the camera moves through an unchanging scene, which makes it unsuitable for tasks such as person tracking and gesture recognition: both of these involve non-rigidly deforming objects and a non-static map, and while these tasks can indeed be tackled by computer vision, they are not 'mapping' problems. Also, vision tasks such as face recognition, image understanding and classification are not related to SLAM.

However, as shown above, a number of important tasks such as tracking moving cameras, augmented reality, map reconstruction, interactions between real and virtual objects, object tracking and 3D modelling can all be proficient using a SLAM system, and the availability of

such technology will lead to further developments and increased superiority in augmented reality applications. Just like humans, bots can't always rely on GPS, mainly when they operate indoors. And GPS isn't sufficiently accurate enough outdoors because precision within a few inches is required to move about safely. Instead they rely on what's known as simultaneous localization and mapping, or SLAM, to discover and map their surroundings. Using SLAM, robots build their own maps as they go. It lets them know their position by aligning the sensor data they collect with whatever sensor data they've already collected to build out a map for navigation. Sounds easy enough, but it's actually a multi-stage process that includes alignment of sensor data using a variety of algorithms well suited to the parallel processing abilities of GPUs. There are many forms of SLAM, which has been around since the 1980s. For the purpose of this section, we'll focus on its application within computer vision.



Fig. 4.22: Real and Virtual Objects

Sensor Data Alignment: Computers see a robot's position as simply a timestamp dot on a map or timeline. Robots continuously do split-second gathering of sensor data on their surroundings. Camera images are taken as many as 90 times a second for depth-image measurements. And LiDAR images, used for precise range measurements, are taken 20 times a second. When a robot moves, these data points help measure how far it's gone relative to its previous location and where it is located on a map.

Motion Estimation: In addition, what's known as wheel odometry takes into account the rotation of a robot's wheels to help measure how far it's travelled. Inertial measurement units are also used to gauge speed and acceleration as a way to track a robot's position. All of these sensor streams are taken into consideration in what's known as sensor fusion to get a better estimate of how a robot is moving. Kalman filter algorithms and particle filter algorithms, basically a bunch of sophisticated math that rely on sequential Monte Carlo methods can be used to fuse these sensor inputs.

Sensor Data Registration: Sensor data registration, or the measurement between data points, can happen between two measurements or between a measurement and a map. Using the NVIDIA Isaac SDK, developers can localize a robot with what's known as scan-to-map matching. Also in the SDK is an algorithm from NVIDIA researchers called HGMM, or Hierarchical Gaussian Mixture Model, that can align two point clouds (a large set of data points in space) taken from different points of view. Bayesian filters are applied to mathematically solve where the robot is located, using the continuous stream of sensor data and motion estimates.

GPUs for Split-Second Calculations: The mapping calculations described above happen 20-100 times a second, depending on the algorithms. This wouldn't be possible to perform in real time without the processing power of NVIDIA GPUs. Ideal for robotics, Jetson AGX Xavier delivers 32 teraops of GPU workstation-like performance in a compact package. The massive number crunching task of aligning point clouds or depth images can be done on NVIDIA GPUs as much as 20 times faster than with CPUs. Think of Jetson Nano as a similarly giant performance leap for the maker crowd and others.

Visual Odometry for Localization: Visual odometry is used to recover a robot's location and orientation using video as the only input. NVIDIA Isaac supports stereo visual odometer, two cameras that work in real time to help guide on the location, tapping into a minimum of 30 frames per second. It's available for use on all products powered by compact Jetson supercomputing modules. Using the stereo visual odometry capabilities that come standard in Isaac, robotics developers can accurately calculate a robot's location and use this for navigation. Visual odometry capabilities are packed into our Jetson Nano Developer Kit. (Visual odometry isn't part of Isaac for SLAM just yet.) Future development for Isaac on visual odometry will integrate it and elevate it to the level of SLAM. For now, SLAM is used as a check for map recovery of a robot's location and orientation to eliminate errors in navigation from inaccurate visual odometry results.

Map Building for Localization: Maps can be created in three different ways. One way is for mapping algorithms to be run on the Jetson device while somebody supervises and drives the robot manually. A second way is to have the Isaac application on the robot to stream data to the Isaac application running the mapping algorithms on a workstation. But a third, and recommended, method is to record the lidar scan and odometry data to a file using Isaac's handy recorder widget. That way mapping can be done offline using the logmapping application. This method allows tuning parameters of the mapping algorithms for optimized maps without driving a robot around again and again. For creating maps to localize and navigate, 2019.1 version of the NVIDIA Isaac SDK supports and uses Open Slam's Gmapping and Google's Cartographer algorithms. The modularity of Isaac enables users to also integrate

other third-party libraries of their choice or plug in their own implementations. Isaac feeds 2D range scan data that is obtained using a lidar or depth camera to these mapping algorithms. Isaac also supplies odometry information computed using wheel speeds, inertia measurement unit data and computer vision.

Occupancy Grid (for LiDAR SLAM): As a robot perceives its surroundings using LiDAR or cameras, Isaac creates an occupancy grid map of the robot's environment with the resolution determined by the user. This 2D "local map" provides information, whether each cell in the map is blocked or free, so that the robot can plan its navigation path accordingly. Well-constructed occupancy grids from Isaac are key to fast, natural and reliable obstacle-avoidance in the Isaac navigation stack.

How do SLAM Robots Navigate? SLAM is similar to a person trying to find his or her way around an unknown place. First, the person looks around to find familiar markers or signs. Once the person recognizes a familiar landmark, he or she can figure out where they are in relation to it. If the person does not recognize landmarks, he or she will be labeled as lost. However, the more that person observes the environment, the more landmarks the person will recognize and begin to build a mental image, or map, of that place. The person may have to navigate this certain environment several times before becoming familiar with a previously unknown place. In a related way, a SLAM robot tries to map an unknown environment while figuring out where it is at. The complexity comes from doing both these things at once. The robot needs to know its position before answering the question of what the atmosphere looks like.

The robot also has to figure out where it is at without the benefit of already having a map. The process of solving the problem begins with the robot or unmanned vehicle itself. The type of robot used must have an exceptional odometry performance. Odometry is the measure of how well the robot can estimate its own position. This is normally calculated by the robot using the position of its wheels. Something to keep in mind, however, is that there is normally a small margin of error with odometry readings. The robot might be off in its measurements by several centimeters. Consequently, the robot is not where it thinks it is in a given location. These errors must be taken into account in algorithms. Also, areas are often remapped to make up for this deficiency.

One requirement of SLAM is a range measurement device, the method for observing the environment around the robot. The most common form of measurement is a laser scanner such as LiDAR. Laser scanners are easy to use and very precise. However, they are also extremely expensive. There are other options, though. Sonar can be used, and this device is especially useful for mapping underwater environments. Imaging devices can also be used for SLAM.

These optical readers can come in 2D or even 3D formats. The measurement device used depends on several variables, including preferences, costs, and availability. Another key component in the SLAM process is getting data about the environmental surroundings of the robot. Just like a human, the robot uses landmarks to determine its location using its sensors, the laser, sonar, or whichever measuring device is used.

A robot will use different landmarks for different environments. However, there are certain requirements for landmarks used in SLAM. First of all, landmarks should be stationary. A robot cannot determine its own location if a nearby landmark is constantly moving. Additionally, landmarks should be unique and different from the surrounding environment. Landmarks also need to be plentiful and should be able to be viewed from many different angles. Once a robot has sensed a landmark, it can then determine its own location by extracting the sensory input and identifying the different landmarks. A method needs to be in place in order for the robot to do this. This landmark extraction can be done in a variety of ways from algorithms like Spike extraction to scan-matching. The important factor to remember is that the robot needs a way to identify a landmark. Robots can also use data from previously scanned landmarks and match them up with each other in order to determine its location. SLAM is the mapping of an environment using the continual interplay between the mapping device, the robot, and the location it is in.



Fig. 4.23: Robotic Mapping

4.9 WHAT IS AUGMENTED REALITY?

Augmented reality (AR) is a field of computer research which aims at supplementing reality by mixing computer-generated data and real world environments. When dealing with computer vision, the so-called computer-generated data are generally 3D models, text, videos that become virtual objects of the scene when displayed as they were really there and at the right time. Augmented reality has to be both structurally and contextually synchronized with the real world. The structural coherence is needed to deceive the human eyes and make virtual objects appear and behave as they would really exist in the scene.

The contextual coherence helps in providing selected information according to the user needs, without flooding him with plenty of undesired data. Computer generated data comprise also music, speech and all kind of multimedia. A usual example of AR is the match score showed in television broadcasts of football games. The real-world elements are the football field and the players while the virtual elements are the score numbers and the team flags, which are drawn over the image by computers in real time, stretched as they would appear if laid on the court.

Nowadays applications include but are not limited to:

- ✚ Support for complex tasks, i.e. assembly, maintenance, surgery etc., by inserting additional information into the field of view.
- ✚ Enhanced vision: labels or any text related to the objects/ places seen, rebuilt ruins, building or even landscape as seen in the past.
- ✚ Navigation devices, in buildings, outdoors, cars, airplanes.
- ✚ Visualization of architecture (virtual reconstruction of destroyed historic buildings as well as simulation of planned construction projects)
- ✚ Simulation, e.g. flight and driving simulators.
- ✚ Collaboration of distributed team conferences with real and virtual participants.
- ✚ Entertainment and education virtual objects in museums and exhibitions.

Most of the AR applications can be dramatically boosted using see-through display glasses or other special visual devices. For instance, driving assist applications, inside cars or airplanes, make up for this by deploying head-up displays integrated into the windshield. The computer vision challenges in the field of AR are the development of algorithms and systems that perform automatically and independently of the user motion and inside real environments.

Methodology: The most essential task of every augmented reality approach is pose estimation, explicitly the determination of the position and the orientation of the observer with respect to the environment. Every pose estimation algorithm relies on pattern recognition and matching. The system is usually trained with a set of patterns that model the appearance and/or the 3D structure of specific objects inside the environment. Those patterns can be fiducial markers expressly inserted into the scene (marker-based) or natural markers already present inside the environs (markerless). At run-time, the system attempts to track the visible patterns in the observer's field of view. Using the deformations affecting the recognized patterns the system burns an estimate of the position and the orientation of the observer with respect to the environment, called pose. Once the pose is known, computer graphic techniques allow virtual objects to be rendered consistently with the structure of the scene, so that they appear as they would if they truly exist in the environment.

The observer's pose and the identity of the recognized patterns are usually exploited to deliver location based services. The markerless AR system relies on planar patterns, i.e. objects, naturally present in the scene. Each pattern is encoded as a spatial cluster of natural local features, e.g. SIFT. The geometric transformation between training patterns and objects visible in a given view is considered representative of the motion of the observer with respect to the scene. Different analytical models can be employed to perform parameter estimation, these include homography, essential/fundamental matrix, explicit 3D roto-translation. Our method performs homography based estimation when dealing with uncalibrated cameras or varying intrinsics. On the other hand, explicit 3D estimation is employed when intrinsics are known and fixed. In specific circumstances mosaicing techniques can be leveraged to significantly enhance the accuracy with which the pose is estimated; besides this allows also for wider observer trajectory and improved estimation stability.

Experiments with two feature extraction and matching algorithms, namely SIFT and SURF, were conducted in order to assess their relative merits along two dimensions: achievable frame-rate and realism of the rendered contents (i.e. correct estimation of pose, stability, robustness to noise). The best augmentation illusion comes from the use of SIFT. Nevertheless, a solution based on the standard SIFT proposal is too slow to be used for real-time AR. A near real-time system can be obtained replacing SIFT with SURF, which is explicitly designed to offer a frame-rate speed-up, avoiding the initial image doubling pre-processing and working on sub-sampled (320x240) frames. However, this solution is not really able to provide a convincing illusion of augmentation, because of short spurious estimations and jitter. These errors are removed or mitigated exploiting temporal consistency across frames. A module based on Support Vector Machines used in e-regression mode provides a consistent smoothed pose, merging information from previous frames with the estimation performed on current frame.

Augmented reality is one of the hottest technologies of the present days. Of course, it is, AR is a staple of modern mobile apps. It allows consumers to virtually try on clothes and makeup, visualize furniture in their homes, catch Pokémon, turn themselves into dogs on Snapchat, and even at Facebook ads. AR involves superimposing sensory enhancements such as images or audio over what you actually see in the physical world. You probably knew that already, but you may not understand computer vision, where this begins. Computer vision is an interdisciplinary scientific field that refers to computers' understanding of digital videos and images. In other words, digitizing the human visual system.

Computer vision acquires, processes, analyzes and understands these images, the way your retina would, and extracts their data, turning them into numbers and symbols. This is how Facebook recognizes people when suggesting photo tags, how you're able to deposit a check with the Bank of America app, and how you can turn yourself into a taco on Snapchat. Let's

use the Snapchat filter as an example. When you use your camera, every frame is like a tiny image. Computer vision processes those images, in this case, your face. There are a few universal truths to the human face. Our eye regions are always darker than our upper cheeks, while the sockets are always darker than our foreheads. With that information, computers scan image data and detect contrasts between lighter and darker pixels. Those pixels lead to facial recognition, but only if they're facing forward. We'll get back there.



Fig. 4.24: Augmented Reality and Mobile Apps

Computer vision comes into play with AR because, in order for the technology to work, the computer must also understand the context of the physical world. Say you scan a Pizza Hut box to unlock AR content. Computer vision processes the position of the box, any images printed on the box and which direction it's facing, and what else is in the frame. Of course, the pizza box will have been embedded with a predefined marker. When the AR system recognizes that box, it captures the image with your smartphone camera. From there, it scans the image with sensors to identify exactly where to overlay the additional information on the box. Next, the AR solution overlays the content on that spot. This must align precisely over the original frame, which explains why Snapchat filters disappear when you move your head. Together, these form a whole image, a combination of the real world and AR data. There is also position-based AR, in which the app also utilizes GPS data and the mobile device's compass, accelerometer, and gyroscope. The latter two determine your direction and orientation, respectively.

Putting them in smartphone terms, the accelerometer is how your smartphone knows which direction you're facing when you look at a map, while the gyroscope allows you to change the device's orientation by rotating it. Position-based AR uses geolocation, rather than the embedded marker, to display the content. Instead of pointing your phone at the Pizza Hut box, you can point it at a sign outside the restaurant, which triggers additional information, such as

hours of operation or the menu. AR is live so this occurs every time the camera detects a new frame. This should theoretically occur in 16 milliseconds, as smartphones typically work at 60 frames per second. The AR frame is generally delayed by far more than that but it's also far faster than the human brain can compute. Either way, it's helpful to understand because AR is far more popular than its cousin, virtual reality ever was. In March, eMarketer forecasted that 42.9 million Americans, 13% of the population will use the technology at least once a month. Meanwhile, AR is expected to have whopping 60% more users. AR is also more practical. It doesn't require a whole new device that blocks out the rest of the world; it's already part of the device you use to block out the rest of the world.



Fig. 4.25: Augmented Reality and Computer Vision

While positional tracking will allow Augmented Reality (AR) and Virtual Reality (VR) headsets to see the world, computer vision will enable them to understand it. Computer vision is term encompassing a broad field that deals with how computers can be made to gain a high-level of indulgent from digital images or videos. Essentially, it is programming computers to 'see' in the way that the human visual system can. To look at an object and, based on its appearance, location, and surrounding setting, understand exactly what that object is. Allowing computers to see the world the way that humans do is not an easy task. True computer vision is only possible with AI, machine learning, and enormous sets of data not to mention a detail capturing camera and a vision processing unit (VPU) to help the machine sort through the image. The computer's camera needs a program to help it sort out the visible world. Currently, four primary methods are being used to accomplish this task: blob detection, scale space,

template matching, and edge detection. These methods can be combined to produce a holistic understanding as each has its strengths.

Blob Detection: Blob detection is used to determine various regions in an image. It determines these regions by looking at areas with color and brightness that contrast to their surrounding settings. For example, blob detection would have an easy time spotting a flourishing bush in the middle of a field since the bush would be a different color and vibrancy than the world around it. Blob detection would note that all regions of the bush looked similar and thus must be related, or at the very least similar. If there was a second bush in the field - blob detection would understand the two objects as separate but similar.

Scale Space: The scale space theory is built to process the multi-scale reality of image structures. Essentially, if no previous data is available to determine the appropriate scales for a given data set, then the logical approach for an uncommitted vision system is to show the input data at multiple scales to best understand its properties. The scale space theory breaks an image down into its most basic elements to better understand the relationship between points.

Template Matching: Template matching is a technique in digital image processing used to find small pieces of an image that match a chosen (template) picture. This technology is already used in facial recognition software. Each face has its own unique features and, after a large set of data, the program can easily determine which face belongs to which person. This is already visible in certain social networks like Facebook.

Edge Detection: Edge detection's goal is to identify key points in an image where the brightness drastically changes or ceases all together. These points of change are organized into a set of curved line segments called edges. Edge detection is often used to help determine the specific features of an object. Edges chattels can usually be characterized as viewpoint dependent or viewpoint independent, depending on if the edges will change with view.

All of these techniques come together to give computers solid programs with which to decipher the world. Machine learning will be crucial, as computers will need to be able to understand which information is the best at determining what an image actually is. An enormous amount of collected data taken from a large installed base will also greatly help refine the algorithms and increase their accuracy. This is happening today. Massive companies like Google are debuting computer vision programs that will improve with their user base. Google Lens will start narrow but will still likely process millions of images in its first year. Blippar is also rolling out a selective computer vision program, this one aimed at the automotive industry. Computer vision is vital for AR growth. It will allow enterprise solutions to more efficiently train their workforce and allow marketers to better target consumers, reducing the invasive nature of advertisements.

While not as immediate, the potential is also there for VR. VR headsets will soon have positional tracking, allowing users to freely move in the real world. With computer vision, VR programs could understand which objects could help enhance the experience and which are to be avoided at all costs. This small but notable improvement would be especially useful in the location-based (LBVRE) sector. In the 1970s when microcomputers evolved, people used to interact with computers using keyboard. Comparing it with the natural method of human interaction, which is primarily voice and hand gestures, using keyboard to interact with the machine creates a very big gap between humans and technology. And the output that we get from the machine was also text, which makes computers very less friendly to us. Then in 1980s came GUI, and Mouse. We began to see pictures and graphics, which is friendlier to us than reading simple text.

And with mouse, we started pointing things on the monitor just like we point real world things with our fingers. Computers began to understand interactions that are closer to our natural interactions and started giving graphical outputs that looks similar to real world. In the mid-2000s, Smartphones arrived! That entirely changed the way we interact with technology and became a mass medium after computers. With Touch Screens, people could just use their fingers for interaction. No more intermediate devices like mouses and keyboards between humans and computers. Thus, the gap between humans and technology has reduced considerably! Being able to touch something and drag. Humans love that way of interaction because it is very close to what nature taught us!

But still, we cannot interact with machines just we communicate with another human. Also, they cannot see and understand things just like what we do with our eyes. So, what's lacking for computers are Ears and Eyes? Umm, No. They have microphones and camera. They have the sense organs. Then what's missing? Well, A brain!

Giving machines the power to listen, see, perceive and take actions is what it takes to bring technology even closer to humans. Remember our mission statement? "Bringing Technology Closer to Humans" Thanks to Amazon, Google and Apple. Voice assistants are now getting popular. People use Siri to communicate with my iPhone when driving alone. And it is even nice that it communicates back to us using its voice. I don't have to look down to know where to turn. Though the technology is not yet great, it helps. It's evolving! AR give machines the power to understand what they see and overlay information on top of it in the real world. You may think who is going to hold a phone camera up all the time and see the world through a 5 inch screen when you have naturally gifted eyes with a great field of view! Well, you're correct. It's kinda limiting our own natural abilities. But wait. Don't be pessimistic. Remember smartphones as the most interesting revolution. Something bigger and more interesting is on the way.

Smart Glasses! Now go back to the limitation that we just discussed. You don't want to hold a 5 inch screen anymore. Your eyes will become the screen and the whole world in front of you will become the canvas! The promises of this new device are limitless and undoubtedly, Smart Glasses will change our lives. So, the smart glasses will see exactly the things that your eyes can see. It's not something like a third eye. But it can perceive more things that your brain cannot. And also, it can add more information on top of the visuals that your normal eyes see! Instead of (or along with) voice instructions, you can see a line drawn on the road itself to let you know where you should turn. See, you don't have to look down on your phone screen. The directions are right in front of you, drawn on the road! Seems so futuristic? Well, No. Hopefully, it won't take so many years. It's already happening. What it takes to build such a future, are the following things:

- ✚ Computers should identify the things that they 'see'. (For example, it should identify the road in order to draw the line)
- ✚ Augment an information/graphic on the real world and let it be persistent for everyone else to see. It should behave just like a physical object.
- ✚ Finally, a good-looking Smart Glass that people can wear and yet it will be Magic Leap to see the "revolutionary" device soon.

4.10 WHAT IS 3D RECONSTRUCTION?

3D reconstruction is to reconstruct the world around you. There are many ways to reconstruct the world around but it all reduces down to getting an actual depth map. A depth map is a picture where every pixel has depth information (instead of color information). It is normally represented like a grayscale picture. There are different ways to obtain a depth map and these depend on the sensor being used. A type of sensor could be a simple camera but it is conceivable to use others like LiDAR or infrared or a combination. The type of sensor will determine the accuracy of the depth map. In terms of accuracy it normally goes like this: LiDAR > Infrared > Cameras.

Depth maps can also be colorized to better visualize depth. Depending on the kind of sensor used there's more or fewer steps required to actually getting the depth map. The Kinect camera for example uses infrared sensors combined with RGB cameras and as such you get a depth map right away (because it is the information processed by the infrared sensor). But what if you don't have anything else but your phone camera? In this case you need to do stereo reconstruction. Stereo reconstruction uses the same principle your brain and eyes use to actually understand depth. The gist of it consists in looking at the same picture from different angles, look for the same thing in both pictures and infer depth from the difference in position. This is called stereo matching.

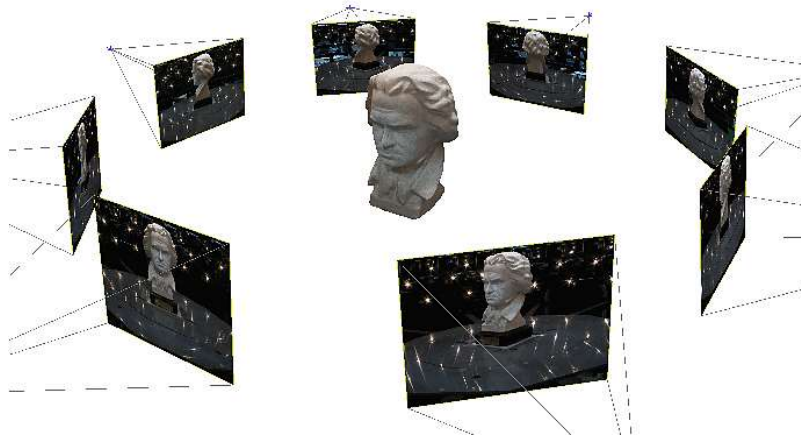


Fig. 4.26: Multi-View Stereo Matching

In order to do stereo matching it is important to have both pictures have the exact same characteristics. Put differently, both pictures shouldn't have any distortion. This is a problem because the lens in most cameras causes distortion. This means that in order to accurately do stereo matching one needs to know the optical centers and focal length of the camera. In most cases this information will be unknown (especially for your phone camera) and this is why stereo 3D reconstruction requires the following steps:

- 📷 **Camera calibration:** Use a bunch of images to infer the focal length and optical centers of your camera
- 📷 **Undistort images:** Get rid of lens distortion in the pictures used for reconstruction
- 📷 **Feature matching:** Look for similar features between both pictures and build a depth map
- 📷 **Reproject points:** Use depth map to reproject pixels into 3D space.
- 📷 **Build point cloud:** Generate a new file that contains points in 3D space for visualization.
- 📷 **Build mesh:** To get an actual 3D model.

Camera calibration needs to be executed once unless you change cameras. Other modules are required every time you take a new pair of pictures and that is pretty much it.

3D Reconstruction Models

3D Reconstruction models are models made using a process called 3D reconstruction. This is a process that involves capturing the appearance and shape of real objects. There are two foremost methods of creating 3D reconstruction models; active and passive methods. Active

methods are also known as range data methods, and these use a numerical approximation approach to reconstruct the 3D profile and build the model. Using this method involves active interference with the reconstructed object, by using either radiometric or mechanical rangefinders. A mechanical method would use depth gauges to measure distances to an object that is rotating on a turntable. Examples of radiometric methods used include ultrasound, microwaves, and moving light sources. The difference between active methods and passive methods is that passive methods do not interfere with the object. Instead, they use sensors to measure the radiance from the surface of the object and then use this for image understanding. If passive methods are used, it is possible to apply 3D reconstruction to a greater number of applications.

Two passive methods are widely used; monocular cues methods and binocular stereo vision. In the former category, shape-from-shading is widely used, says Science Direct. Shade information is collected from an object to learn about the depth of normal information on the object, and this allows the reconstruction of the object. Another form of a monocular cues method is photometric stereo, and this is simply a more sophisticated version of shape-from-shading. The third monocular cues method that is widely used is shape-from-texture. This is used to find the depth of normal information on the surface of an object by using hints from the distortion and perspective of 2D images. When binocular stereo vision is used, multiple images are collected to obtain the 3-dimensional geometric information about an object. Two cameras are used concurrently to collect the images from different angles, or one camera is used to take multiple photographs from different perspectives. An advantage of using binocular stereo vision methods over monocular cues methods is that this is a more direct approach.

3D Reconstruction Models and Virtual Reality

One application for 3D reconstruction models is virtual reality. 3D human face model reconstruction is a vital element of generating facial animations that are used in virtual reality. This poses some problems, such as creating a realistic human face and the collection of images with corresponding facial features. However, there is now a move towards using new algorithms based on binocular stereo vision methods is helping to create more realistic facial images for virtual reality.

3D Reconstruction Models and Medical Imaging

One of the most important applications for 3D reconstruction models is in the medical field. In medical imaging, the creation of reconstruction models is referred to as Iterative reconstruction. Iterative algorithms are used for certain medical imaging techniques to reconstruct either 2D or 3D images. Iterative algorithms are mathematical and scientific sets of instructions that are created by the repetition of a process that generates a sequence of outcomes. The main

advantage of using 3D reconstruction models is that they provide better imaging than some of the traditional methods of collecting medical images. This particularly applies to medical imaging that can suffer from interference along ray paths or where noise statistics are poor, thus creating unclear images. A further advantage is that it can give images from different angles. This then provides the medical specialists with a larger amount of data and may reveal something that would otherwise have been missed from a different angle.

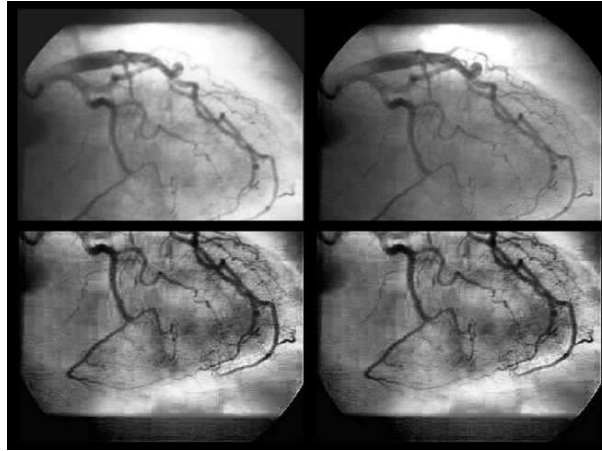


Fig. 4.27: 3D Reconstruction and Medical Imaging

Some forms of imaging are negatively affected by hardware limitations or by the fact that they take time. These are issues that the use of 3D reconstruction models can overcome as they are faster and more efficient than many other techniques. Finally, one type of imaging that has been significantly improved with the use of 3D reconstruction models is Magnetic Resonance Imaging (MRI). It can provide the extended modeling of physical processes and it allows medical professionals to use improved regularization techniques.

What about Models That Change Shape?

If a model changes shape over time, then the process of capturing images and creating models is called either non-rigid reconstruction or Spatio-temporal reconstruction. This takes reconstruction models one step further as these are 4D reconstructions. This is the process of capturing the shape and appearance of real objects that will gradually change over time. The 4D reconstruction models reflect this behavior as they will also change over time. This method is used in computer vision and computer graphics.

Although 3D reconstruction is a rather complicated process that has several challenges, it is a technology that has many applications. Currently, it is predominantly used to create realistic

imaging for virtual reality applications and for medical imaging. This technology is something that scientists are still trying to improve, but there have been big advances in this technology that have made it a better opportunity than many other techniques used in computer graphics, computer vision, and medical imaging.

Using minimal 2D images from a routine angiography, Vision's algorithm along with deep learning tracks the veins and constructs a route map of the arteries, from two different angles. Depth is calculated and the 3D structure of the vascular tree is restored to create a detailed 3D image of the heart. Measurements are calculated from the 3D reconstruction, enabling more accuracy when measuring the vein section and the narrowness severity. The sophisticated algorithms in image processing technology allow specialists to successfully reconstruct 3D models of the heart from 2D images, providing the client with improved diagnostic capabilities. This increases the efficiency of therapy, lowers risk to patients and physicians, and reduces costs.

Multi-View 3D Reconstruction

For a human, it is usually an easy task to get an idea of the 3D structure of any image. Due to the loss of one dimension in the projection process, the estimation of the true 3D geometry is difficult and a so called ill-posed problem, because usually infinitely many different 3D surfaces may produce the same set of images. The goal of multiview 3D reconstruction is to infer geometrical structure of a scene captured by a collection of images. Usually the camera position and internal parameters are assumed to be known or they can be estimated from the set of images.

By using multiple images, 3D information can be partially recovered by solving a pixel-wise connection problem. Since automatic correspondence estimation is usually ambiguous and incomplete further knowledge (prior knowledge) about the object is necessary. A typical prior is assuming that the object surface is smooth. It is focused on convex variational methods. The 3D reconstruction problem is formulated as an energy minimization problem. Due to the convexity of this energy, any (local) minimizer corresponds to the global minimum of this energy. Minimizers of this energy are found with iterative numerical optimization methods which evolve the surface gradually from the initial surface to best one with respect to energy functional.

As a further impact of the convexity, these techniques are independent of the initialization. The initial surface can be of any shape, for example a simple box. Considering a dynamic scene that change over time, 3D reconstruction can be applied to every time step independently. However, one can achieve temporally more consistent results by using the information from several time frames together, thus computing a spatio-temporal hyper-surface in 4D space.

3D Face Reconstruction

Disney Research and Carnegie Mellon University scientists have found that three computer vision methods commonly used to reconstruct 3-D scenes produce superior results in capturing facial details when they are performed simultaneously, rather than independently. Photometric stereo (PS), multi-view stereo (MVS) and optical flow (OF) are well-established methods for reconstructing 3-D images, each with its own strengths and weaknesses that often complement the others. By combining them into a single technique, called photogeometric scene flow (PGSF), the researchers were able to create synergies that improved the quality and detail of the resulting 3-D reconstructions. The quality of a 3-D model can make or break the perceived realism of an animation. That's particularly true for faces; people have a remarkably low threshold for inaccuracies in the appearance of facial features.

PGSF could prove extremely valuable because it can capture dynamically moving objects in high detail and accuracy. Also, found that they could obtain better results by solving the three difficult problems simultaneously. PS can capture the fine detail geometry of faces or other texture-less objects by photographing the object under different lighting conditions. The method is often used to enhance the detail of low-resolution geometry obtained by MVS, but requires a third technique, OF, to compensate for 3-D motion of the object over time. With each of these three steps, image misalignments and other errors can accumulate and lead to a loss of detail.

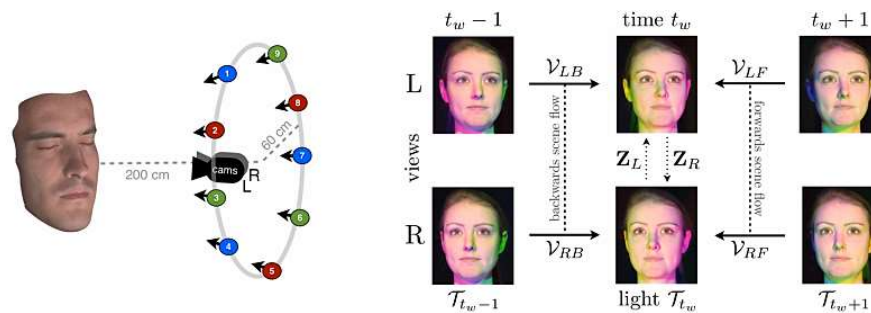


Fig. 4.28: Realistic 3D Face Reconstruction

The key to PGSF is the fact that PS not only benefits from, but also facilitates the computation of MVS and OF. The researchers found that facial details such as skin pores, eyes, brows, nostrils and lips that they obtained via PGSF were superior to those obtained using other state-of-the-art techniques. To achieve PGSF, the researchers created an acquisition setup consisting of two cameras and nine directional lights of three different colors. The lights are multiplexed in time and spectrally to sample the appearance of an actor's face within a very short interval—three video frames. This minimizes the need for motion compensation while also minimizing

self-shadowing, which can be tricky for 3-D reconstruction. The PGSF technique also can be applied to more complex acquisition setups with different numbers of cameras and light sources.

Video Frame 3D Reconstruction

The researchers from the University of Washington described a video frame 3D reconstruction. Using YouTube video, this program inevitably builds highly detailed 3D-models for each video frame. Given the complexity of the problem, this is quite an impressive result, since facial mimicry is very difficult to reproduce. It is important to catch the exact position of the eyes, bending eyebrows, wrinkles, etc. The slightest error in such 3D-model becomes visible in no time. The vast majority of other software for 3D-tracking and face digitization is based on the method of blend shapes, when object's shape changes and “flows” from one station to another. The method of smooth modifications has a big disadvantage, the loss of small details that are so important for face perception. On the other hand, although during the video frame reconstruction every single model is “separate” from each other, when playing it 30/60 frames per second the result should be more realistic than in the case of smooth modifications.

And even more: unlike other similar technologies, this one doesn't require human involvement in the test shooting. Instead, with the help of 3D optical flow method, the program uses a large archive of his or her photographs with different lighting and poses. This technology can be used in cinematography, 3D-printing, computer games and virtual reality; it may be used also to recreate the exact appearance and facial expressions of the deceased by their photos and videos. Under various lighting and with different facial expressions, it empowered realistic reconstruction of the face. For now, the creation of more realistic 3D-animation movies and characters' facial expressions in computer games seems the most direct application of this new technology.

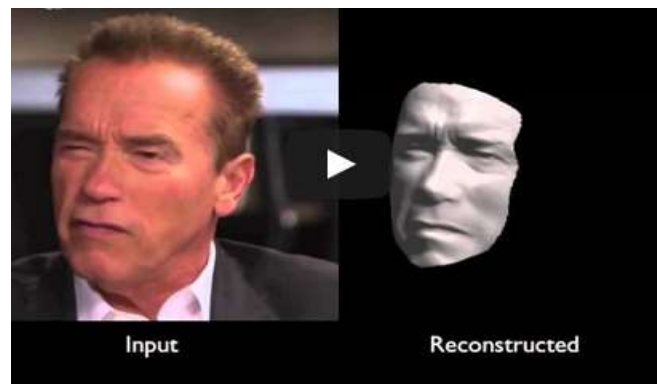


Fig. 4.29: Video Frame 3D Face Reconstruction

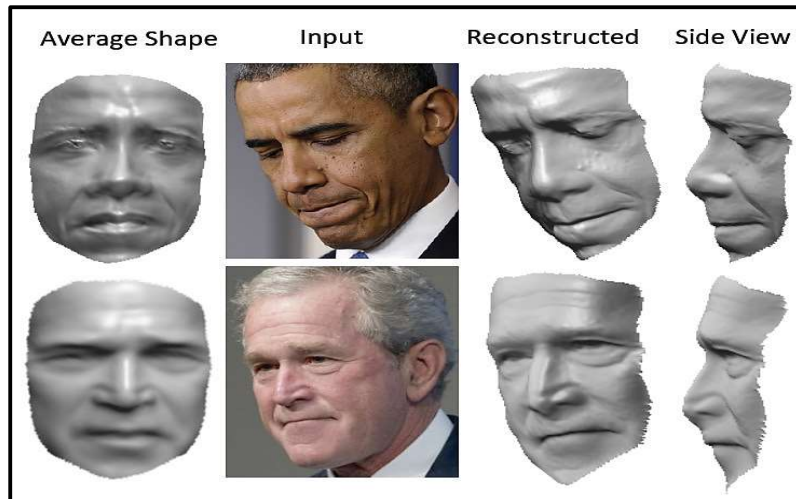


Fig. 4.30: 3D Face Reconstruction from Image

4.11 WHAT IS 3D MOTION TRACKING?

3D Motion Tracking, in its simplest form, is the process of tracking the movement of an object within a piece of footage. Once you've collected this track data from the selected point, you then apply it to another element or object. The results of applying this data are that your element or object now matches the movement of your footage. Essentially you can composite something into a scene that was never there. Below are just a few things motion tracking will help you with. From simple to complex compositions, tracking motion is a technique you must know.

- 🔲 Stabilize motion by utilizing tracking data.
- 🔲 Add elements such as text or solids to a composition.
- 🔲 Insert 3D objects into 2D footage.
- 🔲 Apply effects or color grading techniques.
- 🔲 Replace screens on a TV, Computer, or mobile device.

Motion tracking assists in tracking the movement of objects and shifting the sensed data to an application for advance processing. Motion tracking includes capturing the motions of objects matching with its stored motion template. This has a wide range of applications such as in military, sports, entertainment, medical applications, validation of computer vision and robotics. Furthermore, it is also used in film making and in video game development. In many areas, motion tracking is often called motion capture, whereas in film making and games, motion tracking is commonly called match moving. Motion tracking enhances human-computer interaction and plays a vital role in computer animation of a 3-D model. It provides real-time information, and the amount of animation data produced by motion tracking within a

given time is large. Motion tracking requires specific hardware and software programs to capture and process the data.

Video games often use motion tracking to animate characters in games like baseball, basketball or football. Movies use motion capture for effects. Motion tracking is also used in video tracking for locating a moving object using a camera. Video tracking can be used in surveillance applications to track anonymous user movement. For sophisticated motion tracking, specialized gear or clothing with embedded sensors has to be worn by the user. In such a case, the motion data is sensed by the sensors rather than captured from cameras. Motion tracking is not only used to track human motions it can also be used to track vehicular movements and other objects as well. At times, motion tracking, or match-moving, is the term used to describe the simulation of live-action camera moves and perspective inside compositing software such as After Effects, Combustion, Shake or Flame for 2D, and Boujou, PF Track, SynthEyes or Matchmover for 3D. It's commonly used to apply special effects to feature films and commercials, but it's becoming more common within type and character animation in motion graphics.

There are several types of motion tracking in both 2D and 3D. In the 2D environment your options range from tracking the X and Y-axis along with zooms and rotation on the Z-axis. You can also use motion tracking to stabilize a shaky or unstable shot. Motion tracking in 3D is for complex camera moves across all three axes. Three-dimensional tracks allow for dimensional objects created in 3D software, such as extruded type or character models. Though 3D tracks definitely have their advantages, they shouldn't be used unless necessary, because they add a new level of complexity and more work. Whether you're tracking in two or three dimensions the principles are the same; the only real variable is the software and method for setting it up.

The 3D camera tracker effect analyzes video sequences to extract camera motion and 3D scene data. The 3D camera motion allows you to correctly composite 3D elements over your 2D footage. With motion tracking, you can track the movement of an object and then apply the tracking data for that movement to another object such as another layer or an effect control point to create alignments in which images and effects follow the motion. You can also stabilize motion, in which case the tracking data is used to animate the tracked layer to compensate for movement of an object in that layer. You can link properties to tracking data using expressions, which opens up a wide variety of uses. After Effects tracks motion by matching image data from a selected area in a frame to image data in each succeeding frame.

You can apply the same tracking data to different layers or effects. You can also track multiple objects in the same layer. You set up, initiate, and apply motion tracking with the Tracker panel.

As with all properties, you can modify, animate, manage, and link tracking properties in the Timeline panel. You specify areas to track by setting track points in the Layer panel. Each track point contains a feature region, a search region, and an attach point. A set of track points is a tracker.

Feature region: The feature region defines the element in the layer to be tracked. The feature region should surround a distinct visual element, preferably one object in the real world. After Effects must be able to clearly identify the tracked feature throughout the duration of the track, despite changes in light, background, and angle.

Search region: The search region defines the area that After Effects will search to locate the tracked feature. The tracked feature needs to be distinct only within the search region, not within the entire frame. Confining the search to a small search region saves search time and makes the search process easier, but runs the risk of the tracked feature leaving the search region entirely between frames.

Attach point: The attach point designates the place of attachment for the target, the layer or effect control point to synchronize with the moving feature in the tracked layer.

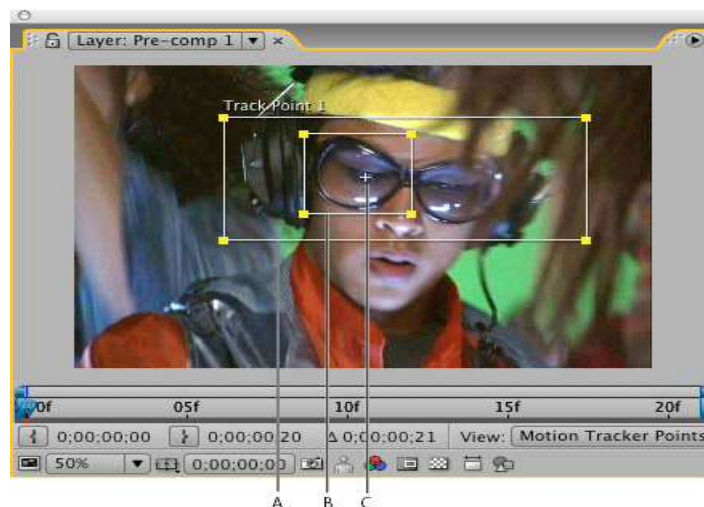


Fig. 4.31: 3D Motion Tracking in After Effects

There are many ways you can do motion tracking in After Effects, and the method and workflow you follow depend on the nature of your clip, and what you want to track.

Mask tracker: Use the mask tracker to draw masks around your object to track only certain objects in your scene.

Face tracker: Simple mask tracking lets you quickly apply effects only to a face, such as selective color correction or blurring a person's face, and more. However, with Face Tracking, you can track specific parts of the face such as pupils, mouth, and nose, allowing you to isolate and work on these facial features with greater detail. For example, change colors of the eyes or exaggerate mouth movements without frame-by-frame adjustments.

3D Camera tracker: Use the 3D camera tracker effect to analyze video sequences to extract camera motion and 3D scene data. You can then correctly composite 3D elements over your 2D footage.

Point tracker: You can track one or multiple reference features in a clip:

-  **One-point tracking:** Track a single reference pattern (a small area of pixels) in a movie clip to record position data.
-  **Two-point tracking:** Track two reference patterns in a movie clip and use the relationship between the two tracked points to record position, scale, and rotation data.
-  **Four-point tracking or Corner pin track:** Track four reference patterns in a movie clip to record position, scale, and rotation data. The four trackers analyze the relationship between four reference patterns, such as the corners of a picture frame or television monitor. This data is applied to each corner of an image or clip to “pin” the clip so that it appears locked in the picture frame or television monitor.
-  **Multiple-point tracking:** Track as many reference patterns in a clip as you like. You can manually add trackers within the Analyze Motion and Stabilize behaviors. When you apply a Track Points behavior from the Shape behaviors subcategory to a shape or mask, a tracker is automatically assigned to each shape control point.

Using the Point Tracker: Set up the shot: For motion tracking to go smoothly, you must have a good feature to track, preferably a distinctive object or region. For best results, prepare the object or region that you are tracking before you begin shooting. Because After Effects compares image data from one frame to the next to produce an accurate track, attaching high-contrast markers to the object or region lets After Effects more easily follow the motion from frame to frame. Lightweight, brightly colored balls (such as Ping-Pong balls) placed on the feature work well, in part because their appearance is the same from all angles.

The number of markers that you use corresponds to the number of points you are tracking. For example, if you're tracking four points using the Perspective Corner Pinning option, you'll track four features, to correspond to the four corners of the layer to attach. The more markers you add to your subject before shooting, the more features you'll have for tracking, but the more items you may have to remove later from the image with the Clone Stamp tool. You don't need to add a marker for each feature if a distinctive object or region is already at the

appropriate location. If you're tracking a large object or the set it such as for match-moving, you can get good results by using a grid of uniformly spaced triangles of a uniform size as tracking markers.

Add the appropriate number of track points: When you choose a mode from the Track Type menu in the Tracker panel, After Effects places the appropriate number of track points in the Layer panel for that mode. You can add more track points to track additional features with one tracker.

Select features to track, and place feature regions: Before you begin tracking, view the entire duration of the shot to determine the best features to track. What is clearly identifiable in the first frame may later blend into the background because the angle, lighting, or surrounding elements have changed. A tracked feature may disappear off the edge of the frame or be obscured by another element at some point in the scene. Though After Effects can extrapolate the motion of a feature, your chances for successful tracking are highest if you step through the entire shot to select the best candidates for tracking. A good tracked feature has these characteristics:

-  Visibility for the entire shot
-  A contrasting color from the surrounding area in the search region
-  A distinct shape within the search region
-  A consistent shape and color throughout the shot

Set the attach point offset: The attach point is where the target layer or effect control point will be placed. The default attach point position is in the center of the feature region. You can move the attach point to offset the position of the target relative to the position of the tracked feature by dragging the attach point in the Layer panel before tracking. For example, to animate a cloud above a person's head, position the feature region on the head and move the attach point above the head. If you left the attach point centered in the feature region, the cloud would be attached to that point and would obscure the head.

-  Attach point centered in feature region
-  Attach point centered in feature region
-  Attach point offset from feature region
-  Attach point offset from feature region

Adjust the feature region, search region, and tracking options: Place each feature region control tightly around its tracked feature, wholly enclosing the tracked feature, but including as little of the surrounding image as possible. The size and position of the search region depend on the movement of the feature you want to track. The search region must accommodate the

movement of the tracked feature, but only the frame-to-frame movement, not its movement throughout the shot. As After Effects locates the tracked feature in a frame, both the feature region and search region move to the new location. Thus, if the frame-to-frame movement of the tracked feature is gradual, then the search region needs to be only slightly larger than the feature region. If the feature changes position and direction quickly, then the search region needs to be big enough to encompass the largest position and direction change in any pair of frames. You can also set tracking options that determine such things as which color channels are compared to find a match to the feature region.

Analyze: You perform the actual motion tracking step by clicking one of the Analyze buttons in the Tracker panel. When tracking a tricky set of features, you may want to analyze a frame at a time.

Repeat as necessary: Due to the changing nature of an image in motion, automatic tracking is rarely perfect. In moving footage, the shape of a feature changes, along with the lighting and surrounding objects. Even with careful preparation, a feature generally changes during a shot and at some point no longer matches the original feature. If the change is too great, After Effects may not be able to track the feature, and the track point will wander or drift. When the analysis begins to fail, return to the frame where tracking was still accurate and repeat the adjust and analyze steps.

Apply Tracking Data: If you're using any Track Type setting other than Raw, you apply tracking data by clicking Apply, after making sure that the correct target is shown for Motion Target. You apply tracking data from a Raw tracking operation by copying keyframes from the trackers to other properties or by linking properties with expressions. You can also adjust the Attach Point or Attach Point Offset property after tracking in the Timeline panel, which can be useful when applying the same tracking data to multiple targets that you want to distribute around the tracked feature. If the layer that you're attaching has motion blur enabled, make sure that the Shutter Phase value is set to -1/2 times the Shutter Angle value. This combination of settings centers the motion blur on the attach point. Otherwise, the attached object may appear to lead or lag the object that it's attached to. You can apply the tracking data to a null object layer and parent the layer that you want to animate to the null object layer.

Track or stabilize motion with the point tracker: Tracking motion and stabilizing motion are essentially the same process, only with a different target and result. Use Track Motion to track motion and apply the results to a different layer or effect control point. Use Stabilize Motion to track motion and apply the results to the tracked layer to compensate for that motion (for example, to remove camera shake). To stabilize a layer, After Effects tracks the motion of a feature in the layer that should be stationary in the frame, and then uses the tracking data to

set keyframes to perform the opposite motion. You can stabilize to remove any combination of changes in position, rotation, and scale, while leaving desired motion unaffected. For example, if the camera is panning, deselect Position but select Scale and Rotation as the properties to stabilize.

When you select Rotation or Scale in the Tracker panel, you set two track points in the Layer panel. A line connects the attach points; an arrow points from the first attach point (the base) to the second. If possible, place the feature regions on opposite sides of the same object, or at least on objects that are the same distance from the camera. The farther apart the regions, the more accurate the calculations and the better the result. After Effects calculates rotation by measuring the change of angle of the line between the attach points. When you apply the tracking data to the target, After Effects creates keyframes for the Rotation property.

After Effects calculates scale by comparing the distance between attach points on each frame with the distance between the attach points on the start frame. When you apply the tracking data to the target, After Effects creates keyframes for the Scale property. When you track motion using either parallel or viewpoint corner-pinning, After Effects applies keyframes for the Corner Pin effect to the layer to scale and skew the target layer as necessary to fit the four-sided area defined by the feature regions. The feature regions should lie in a single plane in the real world, for example, on the side of a bus, on the same wall, or on the floor. The attach points should also all lie in a single plane, but not necessarily the same plane as the feature regions. For parallel corner-pinning only: To change which point is inactive, Alt-click (Windows) or Option-click (Mac OS) the feature region of the point to make inactive. (One point must remain inactive to keep the lines parallel.)

-  Select the layer to track in the Timeline panel.
-  Do one of the following:
 - Click Track Motion in the Tracker panel (or choose Animation > Track Motion), click Edit Target, and choose the target to apply the tracking data to.
 - Click Stabilize Motion in the Tracker panel (or choose Animation > Stabilize Motion). The target layer is the tracked (source) layer.
-  Select Position, Rotation, and/or Scale to specify what kinds of keyframes to generate for the target.
-  Move the current-time indicator to the frame from which to begin tracking.
-  Using the Selection tool, adjust the feature region, search region, and attach point for each track point.
-  In the Tracker panel, click either the Analyze Forward or Analyze Backward button to begin tracking.
-  If the tracking ceases to be accurate, click the Stop button, correct the problem as

described in Correct a motion track, and resume analysis.

- ✚ When you are satisfied with the position of the feature region and attach point throughout the track, click the Apply button to apply the motion to the specified target. After Effects creates keyframes for the target layer.
- ✚ When tracking position and applying this position data to a target, you can choose to apply only the x (horizontal) or y (vertical) component of motion. For example, you can apply the tracking data to the x axis to make a speech bubble (the motion target) remain at the top of the frame even when the actor (the motion source) moves downward.

X And Y (default) allows motion along both axes.

X Only restricts the motion target to horizontal movement.

Y Only restricts the motion target to vertical movement.

To bypass the Motion Tracker Apply Options dialog box and use the previous setting, hold Alt (Windows) or Option (Mac OS) as you click Apply. You can change the order of steps 1-3 by first selecting the property to which to apply the tracking data (Scale, Position, or Rotation) and then choosing Animation > Track This Property. After Effects prompts you to choose the layer to use as a motion source? When you stabilize a layer, the compensating motion may itself cause the layer to move too far in one direction, exposing the background in the composition or moving action out of the action-safe zone. You can correct this with a small change in scale for the layer. Find the frame where the problem is most severe, and then increase or decrease the scale of the layer until the problem is resolved. This technique adjusts the scale for the duration of the layer; you can also animate scale to correct this problem by zooming at different times.

Motion Tracking Controls: You set up, initiate, and apply motion tracking with the Tracker panel. The layer that contains the motion to track. Layers are available in the Motion Source menu if they have source footage items that can contain motion or if they are composition layers. You can precompose a layer to make it available in the Motion Source menu. The active tracker. You can modify settings for a tracker at any time by selecting the tracker from this menu. Set the tracking mode to use. The motion tracking itself is the same for each of these modes; they differ in the number of track points and how the tracking data is applied to the target:

- ✚ Stabilize tracks position, rotation, and/or scale to compensate for movement in the tracked (source) layer. When tracking position, this mode creates one track point and generates Anchor Point keyframes for the source layer. When tracking rotation, this mode creates two track points and produces Rotation keyframes for the source layer.

-  Transform tracks position, rotation, and/or scale to apply to another layer. When tracking position, this mode creates one track point on the tracked layer and sets Position keyframes for the target. When tracking rotation, this mode creates two track points on the tracked layer and sets Rotation keyframes for the target. When tracking scale, this mode creates two track points and produces Scale keyframes for the target.
-  Parallel Corner Pin tracks skew and rotation, but not perspective; parallel lines remain parallel, and relative distances are preserved. This mode uses three track points in the Layer panel—and calculates the position of the fourth—and sets keyframes for four corner points in a Corner Pin effect property group, which is added to the target. The four attach points mark the placement of the four corner points.
-  Perspective Corner Pin tracks skew, rotation, and perspective changes in the tracked layer. This mode uses four track points in the Layer panel and sets keyframes for four corner points in a Corner Pin effect property group, which is added to the target. The four attach points mark the placement of the four corner points. This option is useful for attaching an image to an opening door or the side of a bus that's turning a corner.
-  Raw tracks position only. Use Raw to generate tracking data that you won't apply using the Apply button. For example, you can copy and paste the keyframes for the Attach Point property to the Position property for a paint stroke; or, you can link effect properties for the Stereo Mixer effect to the x coordinate of the Attach Point property using expressions. Tracking data is stored on the tracked layer. The Edit Target button and the Apply button are not available with this tracking option. You can add track points to a tracker by choosing New Track Point from the Tracker panel menu.

Motion Target: The layer or effect control point that the tracking data is applied to. After Effects adds properties and keyframes to the target to move or stabilize it. Change the target by clicking Edit Target. No target is associated with a tracker if Raw is selected for Track Type. Analyze buttons and begin the frame-to-frame analysis of the track point in the footage:

-  **Analyze 1 Frame Backward:** Analyze the current frame by moving back to the previous frame.
-  **Analyze Backward:** Analyze from the current-time indicator backward to the beginning of the trimmed layer duration.
-  **Analyze Forward:** Analyze from the current-time indicator to the end of the trimmed layer duration.
-  **Analyze 1 Frame Forward:** Analyze the current frame by advancing to the next frame.

While analysis is in progress, the Analyze Backward and Analyze Forward buttons change to a Stop button, with which you can stop analysis when the track drifts or otherwise fails.

Restores the feature region, search region, and attach point to their default positions and deletes the tracking data from the currently selected track. Tracker control settings and keyframes already applied to the target layer remain unchanged. Sends the tracking data (in the form of keyframes) to the target layer or effect control point.

Motion Tracking Options: These settings apply to a tracker, a group of track points that is generated in one tracking session. You can modify these settings by clicking Options in the Tracker panel.

Track Name: The name for a tracker. You can also rename a tracker by selecting it in the Timeline panel and pressing Enter on the main keyboard (Windows) or Return (Mac OS).

Tracker Plug-in: The plug-in used to perform motion tracking for this tracker. By default, this option displays Built-in, the only tracking plug-in included with After Effects.

Channel: The components of the image data to use for comparison when searching for a match for the feature region. Select RGB if the tracked feature is a distinct color. Select Luminance if the tracked feature has a different brightness than the surrounding image (such as a burning candle carried through a room). Select Saturation if the tracked feature has a high concentration of color, surrounded by variations of the same color (such as a bright red scarf against a brick wall).

Process Before Match: Temporarily blurs or sharpens an image to improve tracking. Blur reduces noise in the footage. Usually a value of 2 to 3 pixels is enough to produce better tracks in grainy or noisy footage. Enhance exaggerates or refines the edges of an image and makes them easier to track. After Effects blurs or enhances the layer only for tracking. This blurring does not affect the motion source layer.

Track Fields: Temporarily doubles the frame rate of the composition and interpolates each field to a full frame to track motion in both fields of interlaced video.

Subpixel Positioning: When selected, keyframes are generated to a precision of a fraction of a pixel. When deselected, the tracker rounds off values to the nearest pixel for generated keyframes.

Adapt Feature On Every Frame: Causes After Effects to adapt the tracked feature for each frame. The image data that is searched for within the search region is the image data that was within the feature region in the previous frame, rather than the image data that was in the feature region at the beginning of analysis.

If Confidence is Below: Specifies the action to perform when the Confidence property value is below the percentage value that you specify. To determine an acceptable confidence threshold, track the motion and then examine the Confidence values for the track point in the Timeline panel for problematic frames. Specify a confidence value that is slightly larger than the largest confidence value for the problematic frames.

-  Select Continue Tracking to ignore the Confidence value. This behavior is the default behavior.
-  Select Stop Tracking to stop the motion tracking.
-  Select Extrapolate Motion to estimate the position of the feature region. Attach-point keyframes aren't created for low-confidence frames, and attach-point keyframes for the low-confidence frames from previous tracks are deleted.
-  Select Adapt Feature to use the original tracked feature until the confidence level falls below the specified threshold. At that point, After Effects adapts the tracked feature to be the contents of the feature region in the frame preceding the one that has low confidence and continues tracking. This option isn't available if Adapt Feature on Every Frame is selected in the Motion Tracker Options dialog box; enabling feature adaptiveness causes After Effects to adapt the feature region with every frame regardless of the confidence level.

Opens the Tracker Plug-in Options dialog box, which includes options for the AE Original Built-in Tracker. This command is available only if you choose to use the older After Effects tracker plug-in. To show or hide motion paths in the Layer panel, select or deselect the Display Motion Paths option in the panel menu of the Tracker panel. (The panel menu is the menu that you access by clicking the icon in the upper-right corner of a panel.) You can also use commands in this menu to add a new track point, reveal the current track in the Timeline panel, and toggle whether the feature region magnification is enabled.

Motion tracking properties in the Timeline panel: Each time you click Track Motion or Stabilize Motion in the Tracker panel (or choose Animation > Track Motion or Animation > Stabilize Motion), After Effects creates a new tracker for the layer in the Timeline panel. Each tracker contains track points, which are property groups that store the tracking data after tracking has been performed. After Effects groups trackers in the Motion Trackers property group for each layer in the Timeline panel. To show a tracker in the Timeline panel, select the tracker from the Current Track menu in the Tracker panel and press SS.

You can rename trackers and track points and modify and animate their property values in the Timeline panel just as you do for other layer properties and property groups. You must click Apply in the Tracker panel to apply the property changes to the target.

-  **Feature Center:** Position of the center of the feature region.
-  **Feature Size:** Width and height of the feature region.
-  **Search Offset:** Position of the center of the search region relative to the center of the feature region.
-  **Search Size:** Width and height of the search region.
-  **Confidence:** Property through which After Effects reports the amount of certainty regarding the match made for each frame. In general, Confidence is not a property that you modify.
-  **Attach Point:** Position assigned to the target layer or effect control point.
-  **Attach Point Offset:** Position of the attach point relative to the center of the feature region.

Adjust the track point: When you set up motion tracking, it's often necessary to refine your track point by adjusting the feature region, search region, and attach point. You can resize or move these items independently or in groups by dragging using the Selection tool. To help you define the area to be tracked, the image area within the feature region is magnified to 400% while you move the region.

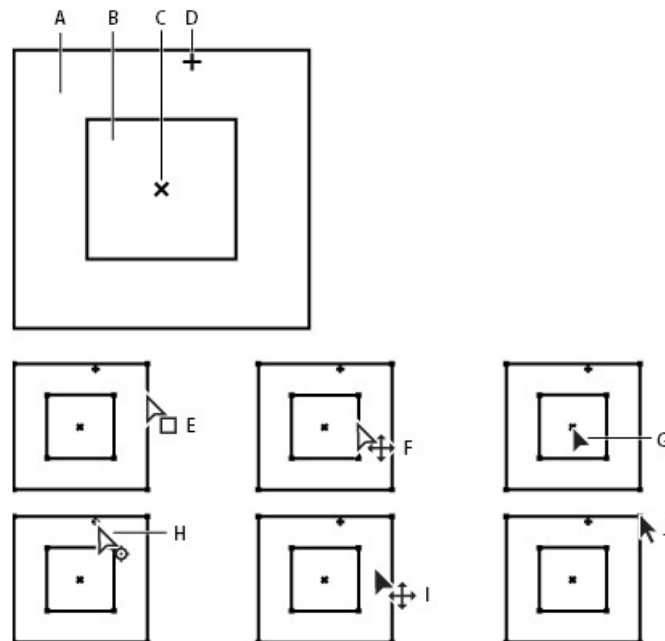


Fig. 4.32: Track point components and Selection tool pointer icons. A. Search region B. Feature region C. Keyframe marker D. Attach point E. Moves search region F. Moves both regions G. Moves entire track point H. Moves attach point I. Moves entire track point J. Resizes region

- ✚ To turn on or off feature region magnification, choose Magnify Feature When Dragging from the Tracker panel menu.
- ✚ To move the feature region, search region, and attach point together, drag inside the track point area (avoiding the region edges and the attach point), or press the Up, Down, Left, or Right Arrow key. Hold Shift while pressing an arrow key to move by an increment 10 times as large.
- ✚ To move only the feature and search regions together, drag the edge of the feature region, or Alt-drag (Windows) or Option-drag (Mac OS) with the Selection tool inside the feature or search region. You can also hold Alt (Windows) or Option (Mac OS) while pressing the Up, Down, Left, or Right Arrow key. Hold Alt+Shift (Windows) or Option+Shift (Mac OS) while pressing an arrow key to move by an increment 10 times as large.
- ✚ To move only the search region, drag the edge of the search region. Offset the search region center from the feature region center in the direction in which the tracked feature is traveling.
- ✚ To move only the attach point, drag the attach point.
- ✚ To resize the feature or search region, drag a corner handle.
- ✚ To make all of the sides of the region match the length of the longest side, and to resize the region relative to the original center point of the region, Shift-drag a corner handle.
- ✚ To make all of the sides of the region match the length of the longest side, and to resize the region relative to a particular corner handle, Ctrl+Shift-drag (Windows) or Command+Shift-drag (Mac OS) the opposite corner handle.
- ✚ To restrict the movement of the track point to the x (horizontal) or y (vertical) axis during tracking, resize the height or width of the search region to match the height or width of the feature region.

Apply tracking data to a new target: After you've tracked a motion source layer, you can apply the tracking data stored on that layer to any number of other target layers and effect control points. For example, you can apply the track to the position of a light bulb and to the effect control point of the Lens Flare effect.

- ✚ In the Tracker panel, choose the tracked layer from the Motion Source menu.
- ✚ Choose the track that contains the tracking data you want from the Current Track menu.
- ✚ Click Edit Target, and choose the target.
- ✚ In the Tracker panel, click the Apply button.

Correct A Motion Track: As an image moves in a shot, the lighting, surrounding objects, and angle of the object can all change, making the once-distinct feature no longer identifiable at the subpixel level. Also, if the search region is too small, the tracked feature may leave its bounds

from one frame to the next. Learning to choose a trackable feature takes time. Even with chary planning and practice, the feature region can drift away from the desired feature. Re-adjusting the feature and search regions, changing the tracking settings, and trying again is a standard part of automatic tracking. It's not necessary to get a single good track in one try. You may need to track the shot in sections, redefining the feature region in places where the feature changes and the region drifts.

You may even need to choose a different feature to track, one with movement that closely matches that of the feature to track, and use the attach point offset to place the target. After you've tracked motion, each track point has a motion path in the Layer panel that shows the position of the center of the feature region. You can fine-tune the keyframes of the motion path in the Layer panel as you would any other motion path. Modifying the motion path is most useful when you want to manually change the motion tracking data before applying it to a target. In some cases, it may be easier to manually modify the motion path created by the motion tracker than to get a perfect track.

Correct drifting by adjusting the feature and search regions:

-  Move the current-time indicator to the last well-tracked frame.
-  Alt-drag (Windows) or Option-drag (Mac OS) the feature and search regions only (not the attach point) to the correct location.
-  If you are correcting the track for one frame, go to step 4. If you are correcting the track for several contiguous frames, adjust the feature region and search region if necessary, and click Analyze. Watch the tracking to make sure that it is accurate. If the tracking is not accurate, then click the button again to stop tracking, adjust the feature region, and begin again.
-  When you are satisfied with the track, click Apply to apply the keyframes to the target layer or effect control point.

Correct drifting by modifying tracking settings:

-  Move the current-time indicator to the last well-tracked frame.
-  In the Tracker panel, click Options.
-  Change settings in the Motion Tracker dialog box as appropriate.
-  In the Tracker panel, click the Analyze Forward or the Analyze Backward button.
-  Watch the tracking to make sure that it is accurate. If the tracking is not accurate, then click the button again to stop tracking, adjust the settings, and begin again.
-  When you are satisfied with the track, click Apply to apply the keyframes to the target layer or effect control point.

Stabilize motion with the Warp Stabilizer VFX effect: You can stabilize motion with the Warp Stabilizer effect. It removes jitter caused by camera movement, making it possible to transform shaky, handheld footage into steady, smooth shots.

Warp Stabilizer VFX | After Effects CC

An upgrade to the Warp Stabilizer, VFX adds the ability to choose which objects within a scene get stabilized.

Stabilize with the Warp Stabilizer effect

-  To stabilize motion using the Warp Stabilizer effect, do the following: Select the layer you want to stabilize.
-  Do one of the following:

In After Effects CC: Go to the Effects & Presets panel > Distort and apply the Warp Stabilizer VFX to the layer. Right-click the footage item in the Timeline panel and choose Warp Stabilizer VFX.

After the effect is added to the layer, analysis of the footage begins immediately in the background. As analysis begins, the first of two banners displays in the Composition panel indicating that analysis is occurring. When analysis is complete, the second banner displays a message that stabilization is occurring. You are free to work with the footage or elsewhere in the project while these steps are occurring.

Warp Stabilizer VFX / settings:

Analyze: There is no need to press this button when you first apply Warp Stabilizer; it's pressed for you automatically. The Analyze button remains dimmed until some change takes place. For example, if you adjust a layer's In or Out points, or there is an upstream change to the layer source. Click the button to reanalyze the footage. Analysis does not take into account any masks or effects that are applied directly to the same layer. Pre-compose and place them in the upstream composition if you want them to be analyzed.

Cancel: Cancels an analysis in progress. During analysis, status information appears next to the Cancel button.

Stabilization: Stabilization settings allow for adjusting the stabilization process.

Result: Controls the intended result for the footage (Smooth or No Motion).

-  **Smooth motion (default):** Retains the original camera movement but makes it smoother. When selected, Smoothness is enabled to control how smooth the camera movement becomes.
-  **No Motion:** Attempts to remove all camera motion from the shot. When selected, the Crop Less <-> Smooth More function is disabled in the Advanced section. This setting is used for footage where at least a portion of the main subject remains within the frame for the entire range being analyzed.

Smoothness: Chooses how much the camera's original motion is stabilized. Lower values are closer to the camera's original motion while higher values are smoother. Values above 100 require more cropping of the image. Enabled when the Result is set to Smooth Motion.

Method: Specifies the most complex operation the Warp Stabilizer achieves on the footage to stabilize it:

-  Position Tracking is based on position data only and is the most basic way footage can be stabilized.
-  Position, Scale and Rotation Stabilization is based upon position, scale, and rotation data. If there are not enough areas to track, Warp Stabilizer chooses the previous type (Position).
-  **Perspective:** Uses a type of stabilization in which the entire frame is effectively corner-pinned. If there are not enough areas to track, Warp Stabilizer chooses the previous type (Position, Scale, and Rotation).
-  **Subspace Warp (default):** Attempts to warp various parts of the frame differently to stabilize the entire frame. If there are not enough areas to track, Warp Stabilizer chooses the previous type (Perspective).
-  The method in use on any given frame can change across the course of the clip based on the tracking accuracy.
-  In some cases, Subspace Warp can introduce unwanted warping, and Perspective can introduce unwanted keystoneing. You can prevent anomalies by choosing a simpler method.

Preserve Scale: When enabled, prevents the Warp Stabilizer from trying to adjust forward and backward camera movements with scale adjustments.

Borders: Borders settings adjust how borders (the moving edges) are treated for footage that is stabilized.

Framing: Controls how the edges appear in a stabilizing result. Framing can be set to one of the following:

-  **Stabilize Only:** Displays the entire frame, including the moving edges. Stabilize Only shows how much work is being done to stabilize the image. Using Stabilize Only allows you to crop the footage using other methods. When selected, the Auto-scale section and Crop Less <-> Smooth More property are disabled.
-  **Stabilize, Crop:** Crops the moving edges without scaling. Stabilize, Crop is identical to using Stabilize, Crop, Auto-scale, and setting Maximum Scale to 100%. With this option enabled, the Auto-scale section is disabled, but the Crop Less <-> Smooth More property is enabled.
-  **Stabilize, Crop, Auto-scale (default):** Crops the moving edges and scales up the image to refill the frame. The automatic scaling is controlled by various properties in the Auto-scale section.
-  **Stabilize, Synthesize Edges:** Fills in the blank space created by the moving edges with content from frames earlier and later in time (controlled by Synthesizes Input Range in the Advanced section). With this option, the Auto-scale section and Crop Less <-> Smooth More are disabled. It is possible for artifacts to appear when there is movement at the edge of the frame not related to camera movement.

Auto-scale: Displays the current auto-scale amount, and allows you to set limits on the amount of auto-scaling. Enable Auto-scale by setting framing to Stabilize, Crop, Auto-scale.

-  **Maximum Scale:** Limits the maximum amount a clip is scaled up for stabilization.
-  **Action-Safe Margin:** When non-zero, specifies a border around the edge of the image that you don't expect to be visible. Thus, auto-scale does not try to fill it.

Additional Scale: Scales up the clip with the same result as scaling using the Scale property under Transform, but avoids an extra resampling of the image.

Advanced:

Detailed Analysis: When set to on, makes the next Analysis phase do extra work to find elements to track. The resulting data (stored in the project as part of the effect) is much larger and slower with this option enabled.

Rolling Shutter Ripple: The stabilizer automatically removes the rippling associated with stabilized rolling shutter footage. Automatic Reduction is the default. Use Enhanced Reduction if the footage contains larger ripples. To use either method, set the Method to Subspace Warp or Perspective.

Crop Less <-> Smooth More: When cropping controls the trade-off between smoothness and scaling of the cropping rectangle as it moves over the stabilized image. Lower values are

smooth; however, more of the image is viewed. At 100%, the result is the same as the Stabilize Only option with manual cropping.

Synthesis Input Range (seconds): Used by Stabilize, Synthesize Edges framing, controls how far backward and forward in time the synthesis process goes to fill in any missing pixels.

Synthesis Edge Feather: Selects the amount of feather for the synthesized pieces. It is enabled only when using the Stabilize, Synthesize Edges framing. Use the feather control to smooth over edges where the synthesized pixels join up with the original frame.

Synthesis Edge Cropping: Trims off the edges of each frame before it is used to combine with other frames when using the Stabilize, Synthesize Edges framing option. Use the cropping controls to crop off bad edges that are common in analog video capture, or low quality optics. By default, all edges are set to zero pixels.

Objective: Determines the aim for the effect: for stabilizing, for temporary stabilization to perform visual effects work, or to composite a layer into a shaky scene. Choose an objective:

-  **Stabilize:** Default option for normal stabilization
-  **Reversible Stabilization and Reverse Stabilization:** Use these options to apply an effect to a region. Use two instances of the Warp Stabilizer VFX effect, one with Reversible Stabilization to steady a shaky object, and a duplicate instance with Reverse Stabilization to insert the shake back in, so that any affects you apply after Reversible Stabilization appear within the original scene.
-  **Apply Motion to Target and Apply Motion to Target:** Use these options to composite a layer into a shaky scene to apply the stabilized motion onto a different layer.

Target Layer: Choose a layer to which the stabilized motion is applied using the Apply Motion to Target or Apply Motion to Target over Original options.

Show Track Points: Determines if track points are displayed.

Track Point Size: Determines the size of the displayed track points

Auto-delete Points Across Time: When you delete track points in a alliance panel, corresponding track points on the same object, are deleted at other times on the layer. You do not need to manually delete the track points frame-by-frame.

Hide Warning Banner: Use when you don't want to reanalyze footage even though there is a warning banner indicating that it must be reanalyzed.

Warp Stabilizer workflow tips:

-  Apply Warp Stabilizer VFX.
-  While the Warp Stabilizer effect is analyzing your footage, you can adjust settings or work on a different part of your project.
-  Choose Stabilization > Result > No Motion if you want to completely remove all camera motion. Choose Stabilization > Result > Smooth Motion if you want to include some of the original camera movement in the shot.
-  If the result is good, you're done with stabilization. If not, do one or more of the following:
-  If the footage is too warped, or distorted, switch the Method to Position, Scale, and Rotation.
-  If there are occasional rippled distortions, and footage was shot with a rolling shutter camera, set Advanced > Rolling Shutter Ripple to Enhanced Reduction.
-  Try checking Advanced > Detailed Analysis.
-  If the result is too cropped, reduce either Smoothness or Crop Less <-> Smooth More. Crop Less <-> Smooth More is much more responsive, as it doesn't require a restabilize phase.
-  If you want to get a feel for how much work the stabilizer is actually doing, set the Framing to Stabilize Only.

When Framing is set to one of the cropping options and the cropping gets extreme, a red banner appears saying, "To avoid extreme cropping set Framing to Stabilize Only or adjust other parameters."

In this situation, you can either set Framing to Stabilize Only, or Stabilize, Synthesize Edges. Other options include reducing the value of Crop Less <-> Smooth More, or reducing Smoothness. Or, if you are satisfied with the results, enable the Hide Warning Banner option. Motion tracking has many more practices:

-  Combining elements filmed separately, such as adding video to the side of a moving city bus or a star to the end of a sweeping wand.
-  Animating a still image to match the motion of action footage, such as making a cartoon bumblebee sit on a swaying flower.
-  Animating effects to follow a moving element, such as making a moving ball glow.
-  Linking the position of a tracked object to other properties, such as making stereo audio pan from left to right as a car races across the screen.
-  Stabilizing footage to hold a moving object stationary in the frame to examine how a moving object changes over time, which can be useful in scientific imaging work.
-  Stabilizing footage to remove the jostling (camera shake) of a handheld camera.

4.12 WHAT IS MEDICAL IMAGE ANALYSIS?

Deep learning is providing exciting solutions for medical image analysis problems and is seen as a key method for computer vision applications. A clear understanding of the principles and methods of neural network and deep learning concepts, showing how the algorithms that integrate deep learning as a core component have been applied to medical image detection, segmentation and registration, and computer-aided analysis, using a wide variety of computer vision applications. Deep Learning for Medical Image Analysis is a great learning resource for academic and industry researchers in medical imaging analysis, on machine learning and deep learning for computer vision and medical image computing and analysis. Deep learning has the potential to revolutionize disease diagnosis and management by performing classification difficult for human experts and by promptly reviewing immense amounts of images.

Optical coherence tomography (OCT) is an imaging technique that uses coherent light to capture high-resolution images of biological tissues. OCT is heavily used by ophthalmologists to obtain high-resolution images of the eye retina. The retina of the eye functions much more like a film in a camera. OCT images can be used to diagnose many retina related eye diseases. In some cases, OCT alone may yield the diagnosis (e.g. macular hole). Yet, in other disorders, especially retinal vascular disorders, it may be helpful to order additional tests (e.g. fluorescein angiogram). Consider the dataset of Retinal Image obtained from OCT. Well, we have four classes of images CNV, DME, DRUSEN and NORMAL (Figure 4.33).

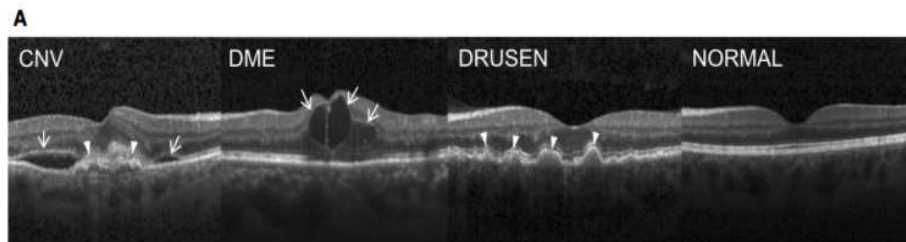


Fig. 4.33: OCT Retinal Image

- ✚ (Far left) Choroidal neovascularization (CNV) with a neovascular membrane (white arrowheads) and associated subretinal fluid (arrows).
- ✚ (Middle left) Diabetic macular edema (DME) with retinal-thickening-associated intraretinal fluid (arrows).
- ✚ (Middle right) Multiple drusen (arrowheads) present in early AMD.
- ✚ (Far right) A normal retina with preserved foveal contour and absence of any retinal fluid/edema.

To prepare the images for the network, we have to resize them to 224 x 224 and normalize each color channel by subtracting a mean value and dividing by a standard deviation. We will also augment the training data at this stage. These operations are done using the image transforms, which prepares the data for a neural network. This helps us to visualize the range of width and height of images in the training set. When we use the images in the pre-trained network, we'll have to reshape them to 224 x 224. This is the size of images and is therefore what the model expects. The images that are larger than this will be truncated while the smaller images will be interpolated.

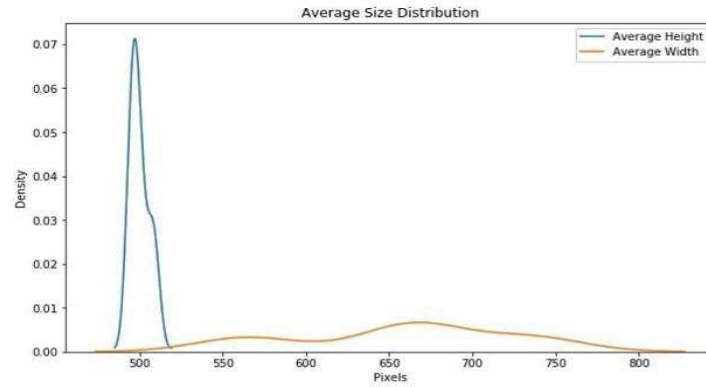


Fig. 4.34: Average Size Distribution

Data Augmentation: Due to the limited number of images, we can use image augmentation to artificially increase the number of images “seen” by the network. This means for training; we randomly resize and crop the images and also flip them horizontally. A different random transformation is applied to each epoch (while training), so the network effectively sees many different versions of the same image.

- ✚ The data is also converted to Torch Tensors before normalization.
- ✚ The validation and testing data are not augmented but are only resized and normalized.

Data Iterators: To avoid loading all of the data into memory at once, we use training DataLoaders.

- ✚ First, we create a dataset object from the image folders, and then we pass these to a DataLoader.
- ✚ At training time, the DataLoader will load the images from disk, apply the transformations, and yield a batch.
- ✚ To train and validation, we'll iterate through all the batches in the respective DataLoader.

One crucial aspect is to shuffle the data before passing it to the network. This means that the ordering of the image categories changes on each pass through the data (one pass through the data is one training epoch).

Using Pre-trained Model (VGG-16): The idea behind pre-training is the early convolutional layers of a CNN extract features that are relevant for many image recognition tasks. The later, fully-connected layers specialize in the specific dataset by learning higher-level features. Therefore, we can use the already trained convolutional layers while training only the fully-connected layers on the own dataset. Pre-trained networks have proven to be reasonably successful for a variety of tasks, and results in a significant reduction in training time and usually increase in recital. The classifier is the part of the model that will be trained. However, for the VGG, we'll only need to train the last few layers in the classifier and not even all of the fully connected layers.

- ✚ We freeze all of the existing layers in the network by setting requires_grad to False.
- ✚ To build the custom classifier, we use the nn.Sequential() module which allows us to specify each layer one after the other.
- ✚ The final output will be log probabilities which we can use in the Negative Log-Likelihood Loss (NLLL).
- ✚ Simply move the whole model onto the GPU.

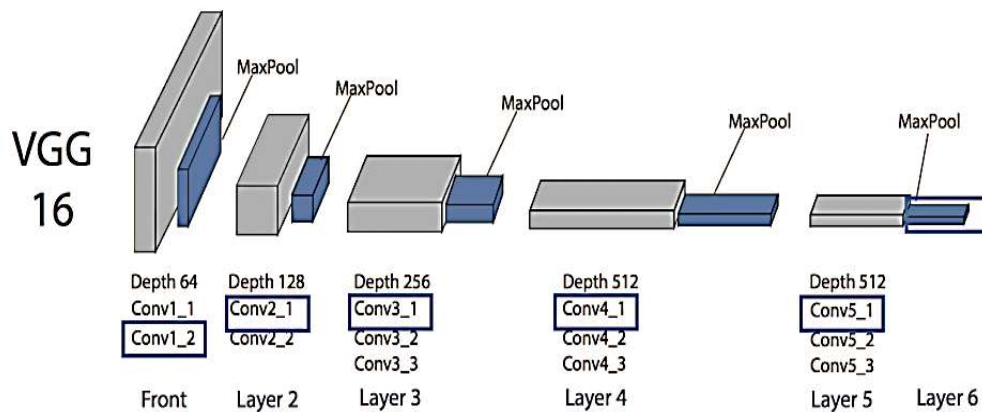


Fig. 4.35: VGG 16 Model

Mapping of Classes to Indexes: To keep track of the predictions made by the model, we create a mapping of classes to indexes and indexes to classes. This will let us know the actual class for a given prediction. So the complete model is pre-trained VGG + custom classifier. Well, it's a pretty long model and it's discussed in detail in the last chapter.

Training Loss and Optimizer:

Loss (criterion): Keeps track of the loss itself and the gradients of the loss with respect to the model parameters (weights)

Optimizer: Updates the parameters (weights) with the gradients.

- ✚ The loss is the negative log-likelihood and the optimizer is the Adam optimizer.
- ✚ The negative log-likelihood in PyTorch expects log probabilities so we need to pass it the raw output from the log softmax in the model's final layer.

Training: For training, we iterate through the train DataLoader, each time passing one batch through the model. We train for a set number of epochs or until early stopping kicks.

- ✚ After each batch, we calculate the loss criterion and then calculate the gradients of the loss with respect to the model parameters using auto differentiation and backpropagation to calculate the gradients.
- ✚ After calculating the gradients, we call `optimizer.step()` to update the model parameters with the gradients. This is done on every training batch so we are implementing stochastic gradient descent (or rather a version of it with momentum known as Adam).
- ✚ For each batch, we also compute the accuracy for monitoring and after the training loop has completed, we start the validation loop. This will be used to carry out early stopping.
- ✚ Early stopping halts the training when the validation loss has not decreased for a number of epochs. Each time the validation loss does decrease, the model weights are saved so we can later load in the best model.
- ✚ Early stopping is an effective method to prevent overfitting on the training data. If we continue training, the training loss will continue to decrease, but the validation loss will increase because the model is starting to memorize the training data. Early stopping prevents this from happening.
- ✚ Early stopping is implemented by iterating through the validation data at the end of each training epoch and calculating the loss. We use the complete validation data every time and record whether or not the loss has decreased. If it has not for a number of epochs, we stop training, retrieve the best weights, and return them. When in the validation loop, we make assured not to update the model parameters.

Training Results: We can inspect the training progress by looking at the history. As expected, the training loss decreases continually with epochs. There is not a massive amount of overfitting, likely because we were using Dropout. With the divergence in losses, there is not much more to gain from further training. The validation loss illustrates abnormal behavior due

to less number of validation images. As with the losses, the training truth increases while the validation accuracy plateaus in general. The model is able to attain higher accuracy right away, a sign that the convolution weights learned on Imagenet were able to easily transfer to the dataset.

Testing the Model: After the model has been trained to the point on no more improvement on the validation data, we need to test it on data it has never seen. For a final estimation of the model's performance, we need to use the holdout testing data. Here, we'll look at individual predictions along with loss and accuracy on the entire testing dataset.

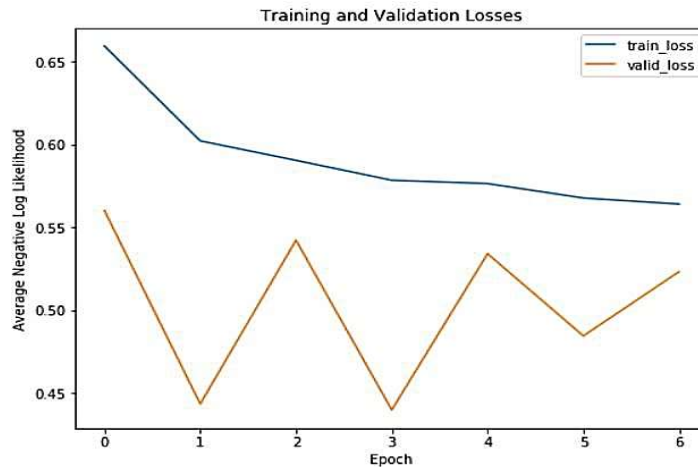


Fig. 4.36: Training and Validation Losses

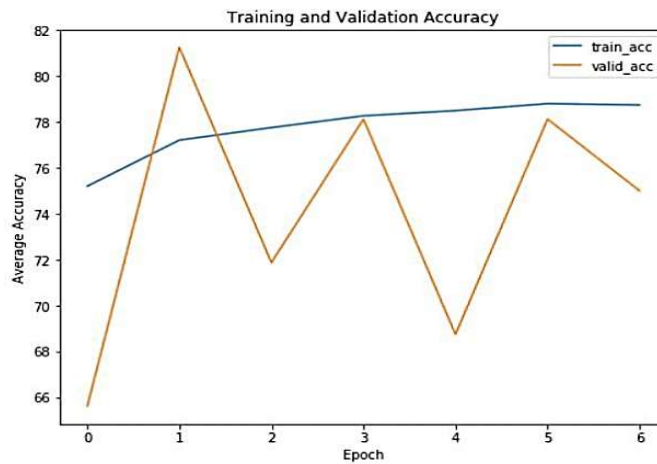


Fig. 4.37: Training and Validation Accuracy

Model Investigation: Although the model does well, there are likely steps to take which can make it even better. Often, the best way to figure out how to improve a model is to investigate its errors (note: this is also an effective self-improvement method.) Well, it seems like the model is working good on the test set. Let's try to get more intuitions about them. This function displays the picture along with the topk predictions from the model. The title over the image displays the true class.

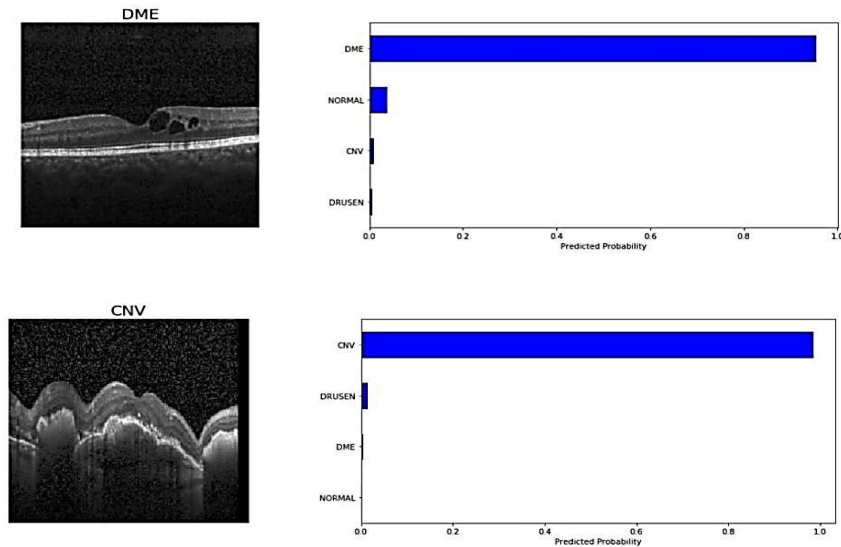


Fig. 4.38: Predicted Probability

Until only a few years ago, traditional computer vision techniques have provided excellent results to detection and segmentation task. More lately, with the advent of deep learning and neural networks in medical imaging, we obtain amazingly better results in all task, be it detection, segmentation, classification and the like. Let us consider the state-of-the-art in the newest model in medical image analysis as listed below.

Medical Segmentation: Deep learning is very active in all fields of medical image processing and computer vision applications. Besides all works in the domain of medical imaging for cardiology, ophthalmology, pulmonology and orthopedics, researchers have endowed to many other medical image analysis projects helping the clients to improve public health and save thousands of lives. These medical applications in computer vision help physicians perform early identification of major diseases in brain, kidney, prostate and many other organs.

Brain Tumor Segmentation: Tens of thousands of people (with thousands of children) die every year of primary cancerous tumors of the brain and spinal cord. Secondary tumors or brain

metastases only make these figures more dramatic. 3D segmentation of brain tumor has high clinical relevance for the estimation of the volume and spread of the tumor. Brain Tumor Segmentation constructs a probability map to localize the tumor and uses deformable models to obtain the tumor boundaries with zero level energy.

Prostate Segmentation: Prostate cancer is the second most common cancer among American men, with more than 200,000 new cases diagnosed every year and about 1 man in 7 diagnosed during his lifetime. Volume is a key indicator of the health of the prostate, revealing key information about the stage of the cancer, the probable prognosis and viable treatment. The rich experience of deep learning enables us to recommend an approach based on a semi-automatic prostate segmentation to give a precise estimate of the prostate volume.

Segmentation of bones and skeleton: A large proportion of the human skeleton is made of porous bone, which offers only low X-ray attenuation, resulting in data density equal to or only slightly higher than that of soft tissues. Bones segmentation and skeleton segmentation using image processing algorithms have become a valuable and indispensable process in many medical applications and have made possible a fast and reliable 3D observation of fractured bones. It's another successful medical application in computer vision.

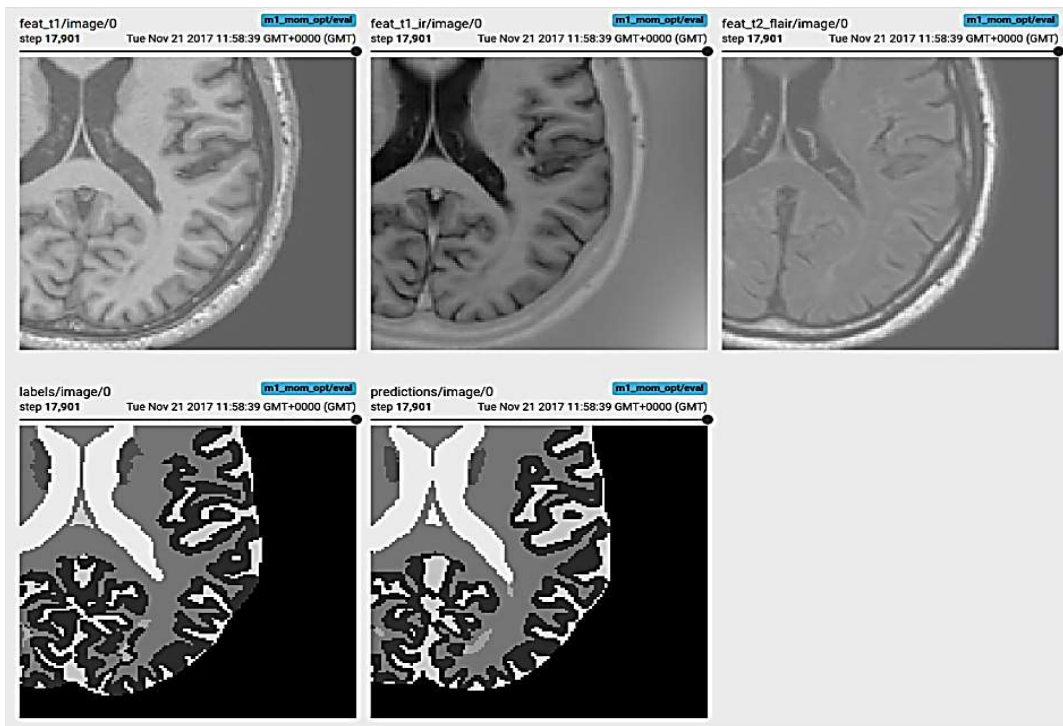


Fig. 4.39: Image Segmentation of Multi-Channel Brain MR Images

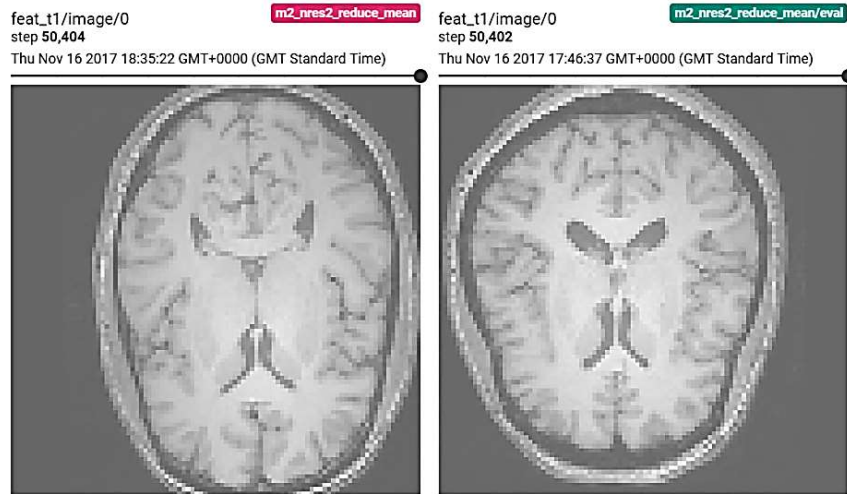


Fig. 4.40: Age Regression and Sex Classification from T1-Weighted Brain MR Images

Density Measurement of Cartilage Tissue: Interpretation of ultrasound images of cartilage is challenging since they display no obvious borders in the transition between tissues: the boundary between tissues can morph in both density and texture. Recent software can process these problematic ultrasound images and automatically measure the density of cartilage in the knee. The main benefits of this procedure are its non-invasive nature and the efficient and accurate measurement of the cartilage it provides.

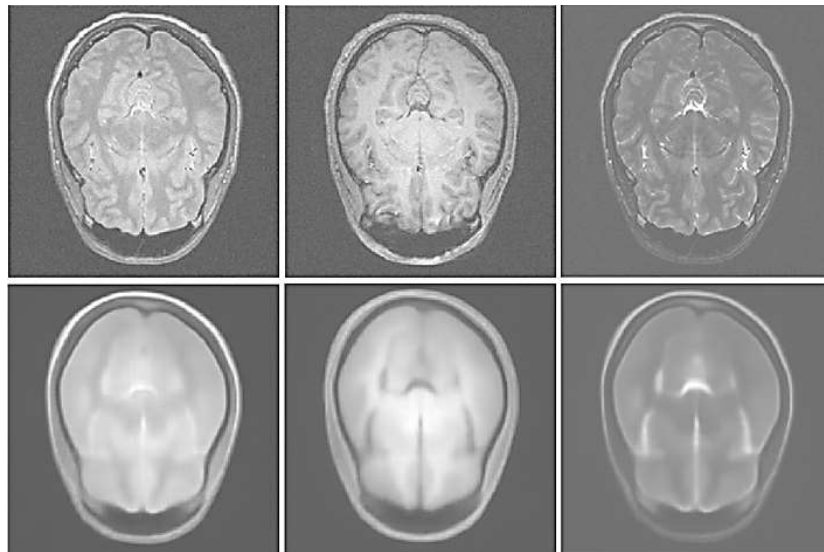


Fig. 4.41: Representation Learning on 3T Multi-Channel Brain MR Images

Kidney Segmentation with Deep Learning: The most vividly common kidney diseases are: kidney cancer, hitting 50,000 new patients every year only in the U.S.; and kidney failures, which leave the organ unable to remove wastes. Laparoscopic partial nephrectomy operations remove or reduce kidney tumors and some renal malfunctions. Computer vision help by provides semi-automatic and accurate kidney segmentation technique built on deep learning and neural networks to create a kidney model which would be specific for each patient. Below are few example results for your reference, the application of modern deep learning in advanced computer vision for medical image analysis.

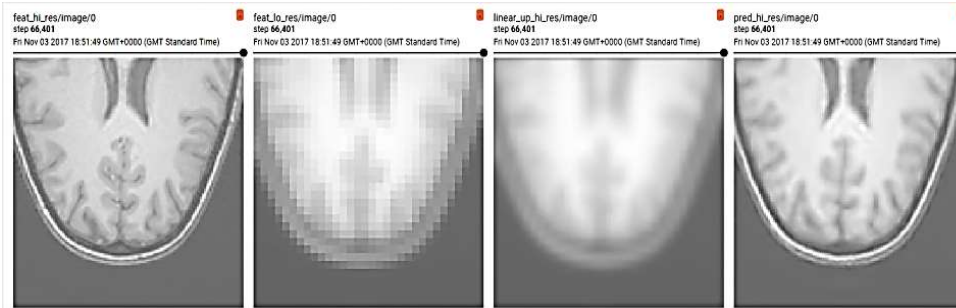


Fig. 4.42: Simple Image Super-Resolution on T1w Brain MR Images

CHAPTER 5

ADVANCED COMPUTER VISION TECHNIQUES

5.1 WHAT IS ADVANCED COMPUTER VISION?

Advanced Computer Vision includes fundamental and advanced domains in computer vision, covering topics from early vision to mid-level and high-level vision, including basics of machine learning and deep neural networks for vision. The powerful packages along with its clear syntax make state-of-the-art computer vision and machine learning accessible to developers with a variety of backgrounds in every field of human living and exploration, the tools and skills to utilize the latest and greatest algorithms in computer vision, making applications that weren't possible until contemporary years. Advanced Computer Vision Techniques are the in-depth look at latest methods adopted in computer vision and related fields. The chapter is designed to familiarize the current state of the art to provide a solid background in Computer Vision and Deep Learning for future generation to explore the further. The goal of Advanced Computer Vision is to expose a wide range of topics and trends that include and not limited to sequential learning, language and vision, deep reinforcement learning, sequential processing, embodied vision, image synthesis, and graph networks.

5.2 WHAT IS POSE ESTIMATION?

Pose estimation is a computer vision technique that predicts and tracks the location of a person or object. This is done by looking at a combination of the pose and the orientation of a given person/object. But why would you want to detect and track a person's or object's location? And how does pose estimation actually work? What are the different approaches, what are its potential benefits and limitations, and how might you use it in your business?

Pose estimation is a computer vision task that infers the pose of a person or object in an image or video. We can also think of pose estimation as the problem of determining the position and orientation of a camera relative to a given person or object. This is typically done by identifying, locating, and tracking a number of keypoints on a given object or person. For objects, this could be corners or other significant features. And for humans, these keypoints represent major joints like an elbow or knee. The goal of the machine learning models is to track these keypoints in images and videos.

Categories of Pose Estimation

When working with people, these keypoints represent major joints like elbows, knees, wrists, etc. This is referred to as human pose estimation. Humans fall into a particular category of

objects that are flexible. By bending our arms or legs, keypoints will be in different positions relative to others. Most inanimate objects are rigid. For instance, the corners of a brick are always the same distance apart regardless of the brick's orientation. Predicting the position of these objects is known as rigid pose estimation. There's also a key distinction to be made between 2D and 3D pose estimation. 2D pose estimation simply estimates the location of keypoints in 2D space relative to an image or video frame. The model estimates an X and Y coordinate for each keypoint. 3D pose estimation works to transform an object in a 2D image into a 3D object by adding a z-dimension to the prediction.

3D pose estimation allows us to predict the actual spatial positioning of a depicted person or object. As you might expect, 3D pose estimation is a more challenging problem for machine learners, given the complexity required in creating datasets and algorithms that take into account a variety of factors such as an image or video background scene, lighting conditions, and more. Finally, there is a distinction between detecting one or multiple objects in an image or video. These two approaches can be referred to as single and multi-pose estimation, and are fundamentally self-explanatory: Single pose estimation approaches detect and track one person or object, while multi-pose estimation approaches detect and track multiple people or objects.

Why Does Pose Estimation Matter?

With pose estimation, we're able to track an object or person (or multiple people) in real-world space at an incredibly granular level. This powerful capability opens up a wide range of possible applications. Pose estimation also differs from other common computer vision tasks in some important ways. A task like object detection also locates objects within an image. This localization, though, is typically coarse-grained, consisting of a bounding box encompassing the object. Pose estimation goes further, predicting the precise location of keypoints associated with the object. We can clearly envision the power of pose estimation by considering its application in automatically tracking human movement. From virtual sports coaches and AI-powered personal trainers to tracking movements on factory floors to ensure worker safety, pose estimation has the potential to create a new wave of automated tools designed to measure the precision of human movement. In addition to tracking human movement and activity, pose estimation opens up applications in a range of areas, such as:

-  Augmented reality
-  Animation
-  Gaming
-  Robotics

Of course, this isn't an exhaustive list, but it includes some of the primary ways in which pose estimation is shaping our future.

How Does Pose Estimation Work?

Now that we know a bit about what pose estimation is, the distinctions between different types of pose estimation, and what it can be used for, let's explore how it actually works in more depth. We'll look at several machine learning-based approaches to pose estimation and assess their advantages and limitations, focusing primarily on neural networks, which have become the state-of-the-art methods for pose estimation.

Primary Techniques for Pose Estimation

In general, deep learning architectures suitable for pose estimation are based on variations of convolutional neural networks (CNNs). There are two overarching approaches: a bottom-up approach, and a top-down approach. With a bottom-up approach, the model detects every instance of a particular keypoint (e.g. all left hands) in a given image and then attempts to assemble groups of keypoints into skeletons for distinct objects. A top-down approach is the inverse; the network first uses an object detector to draw a box around each instance of an object, and then estimates the keypoints within each cropped region. It's important to remember the distinction between 2D pose estimation and 3D pose estimation. The two tasks carry different data requirements, produce different outputs (2D pixel values vs. 3D spatial arrangement), and are generally used to solve different problems.

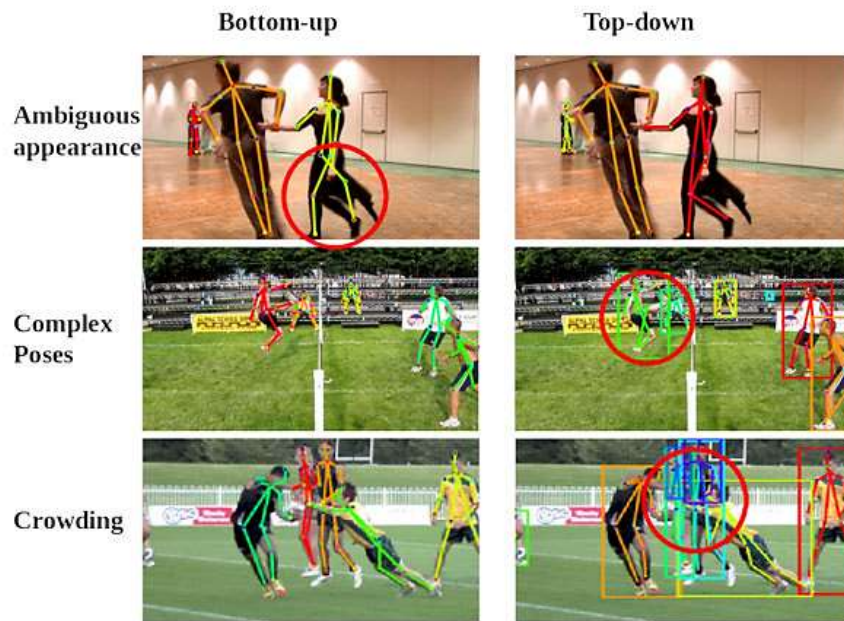


Fig. 5.1: Bottom-Up Approach and Top-Down Approach

Basic Structure

Deep learning models for pose estimation come in a few varieties related to the top-down and bottom-up approaches discussed above. Most begin with an encoder that accepts an image as input and extracts features using a series of narrowing convolution blocks. What comes after the encoder depends on the method of pose estimation. The most conceptually simple method uses a regressor to output final predictions of each keypoint location. The resulting model accepts an image as input and outputs X, Y, and potentially Z coordinates for each keypoint you're trying to predict. In practice, however, this architecture does not produce accurate results without additional refinement.

A slightly more complicated approach uses an encoder-decoder architecture. Instead of estimating keypoint coordinates directly, the encoder is fed into a decoder, which creates heatmaps representing the likelihood that a keypoint is found in a given region of an image. During post-processing, the exact coordinates of a keypoint are found by selecting heatmap locations with the highest keypoint likelihood. In the case of multi-pose estimation, a heatmap may contain multiple areas of high keypoint likelihood (e.g. multiple right hands in an image). In these cases, additional post-processing is required to assign each area to a specific object instance.

Top-down approaches also use an encoder-decoder architecture to output heatmaps, but they contain an additional step. An object detection module is placed between the encoder and decoder and is used to crop regions of an image likely to contain an object. Keypoint heatmaps are then predicted individually for each box. Rather than having a single heatmap containing the likely location of all left hands in an image, we get a series of bounding boxes that should only contain a single keypoint of each type. This approach makes it easy to assign keypoints to specific instances without a lot of post-processing.

Both types of architectures discussed thus far apply equally to 2D and 3D pose estimation. There are, however, a few options worth mentioning that are specific to 3D rigid pose estimation. Rather than predicting the coordinates of each keypoint in $[X, Y, Z]$ space, it's instead possible to predict the 6 degrees of freedom of an object and apply the transformation to a reference skeleton. For example, instead of trying to predict the 3D position of the top of a water bottle, you would instead predict the 3D coordinates of its center of mass and then predict the rotation of the bottle in an image relative to a reference coordinate system. This method of prediction is particularly useful for applications in AR, VR, or other apps that use 3D rendering software like Unity. It can, however, be hard to train accurate models.

Model Architecture Overview

There are too many specific neural network architectures to cover them all here, but there we'll highlight a few robust, reliable ones that make good places to start.

(1) Stacked-Hourglass Networks

Pure encoder-decoder networks take an image as input and output heatmaps for each keypoint. They are a good place to start if you're new to neural pose estimation. If you need to detect multiple poses, Mask-RCNN is a versatile architecture that predicts bounding boxes for objects in an image and then predicts poses within the regions of the image enclosed in the box.

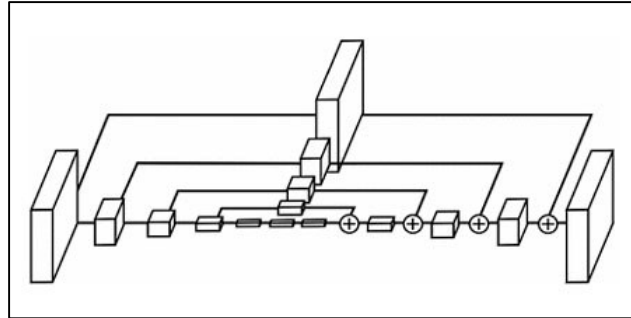


Fig. 5.2: Stacked Hourglass Networks for Human Pose Estimation

(2) Person Lab/Pose Net and Open Pose

Open Pose and Person Lab (also known as Pose Net) are variants of an encoder-decoder design with a twist. In addition to outputting heatmaps, the model also outputs refinements to heatmaps in the form of short, mid, and long-range offsets. Heatmaps identify broad regions of an image where a key point is likely to be found, while offsets guide out predictions within a region to a more accurate final prediction.

(3) Convolutional Pose Machines

Convolutional pose machines build on the encoder-decoder architecture by iteratively refining heatmap predictions using additional network layers and feature extraction. The final output is a single set of heatmaps, and post-processing involves identifying the pixels at which the heatmap probability is the highest for each keypoint.

How Pose Estimation Works On The Edge?

If your use case requires that pose estimation work in real-time, without internet connectivity, or on private data, you might be considering running your pose estimation model directly on an edge device like a mobile phone or IoT board. In those cases, you'll need to choose specific model designs to make sure everything runs smoothly on these lower power devices. Here are a few guidelines and tricks to ensure your models are ready for edge deployment:

- 🧩 Use Mobile Net-based architectures for your encoder. This design makes use of layer types like depth wise separable convolutions that require fewer parameters and less computation while still providing solid accuracy.
- 🧩 Add a width multiplier to your model so you can adjust the number of parameters in your network to meet your computation and memory constraints. The number of filters in a convolution layer, for example, greatly impacts the overall size of your model. Many papers and open-source implementations will treat this number as a fixed constant, but most of these models were never intended for mobile or edge use. Adding a parameter that multiplies the base number of filters by a constant fraction allows you to modulate the model architecture to fit the constraints of your device. For some tasks, you can create much, much smaller networks that perform just as well as large ones.
- 🧩 Shrink models with quantization, but beware of accuracy drops. Quantizing model weights can save a bunch of space, frequently reducing the size of a model by a factor of 4 or more. However, accuracy will suffer. Make sure you test quantized models strictly to determine if they meet your needs.
- 🧩 Input and output sizes can be smaller than you think! If you're designing a coaching app, it's tempting to think that your pose estimation model needs to be able to accept full resolution video frames as an input. In most cases, edge devices won't have nearly enough processing power to handle this. Instead, it's common to train pose estimation models to take images as small as possible to 224x224 pixels.

Use Cases and Applications

In this section, we'll provide an overview of real-world use cases for pose estimation. We've mentioned several of them in previous sections, but here we'll dive a bit deeper and explore the impact this computer vision practice can have across industries. Specifically, we'll examine how pose estimation is used in the following areas:

(1) Human Activity and Movement

One of the clearest areas in which pose estimation is pertinent is in tracking and measuring human movement. But tracking movement in and of itself isn't something you can put into production, generally speaking. With a little creative thinking, however, the applications resulting from tracking this movement are dynamic and far-reaching. Consider, as one example, an AI-powered personal trainer that works simply by pointing a camera at a person completing a workout, and having a human pose estimation model (trained on a number specific poses related to a workout regimen) indicate whether or not a given exercise has been completed properly. This kind of application could empower harmless and more inspirational home workout routines, while also increasing the accessibility and decreasing the costs associated with professional physical trainers. And given that pose estimation models can now

run on mobile devices and without internet access, this kind of application could easily extend the access of this kind of expertise to remote or otherwise hard-to-reach places. Using human pose estimation to track human movement could also power a number of other experiences, including but not limited to:

-  AI-powered sports coaches.
-  Workplace activity monitoring.
-  Crowd counting and tracking (e.g. for retail outlets measuring foot traffic)

(2) Augmented Reality Experiences

While not instantly evident, pose estimation also presents an opportunity to create more realistic and responsive augmented reality (AR) experiences. For much of this guide, we've focused on human pose estimation. But if you'll recall, we can locate and track objects with non-variable keypoints. From pieces of paper, to musical instruments, to...well, pretty much anything you can think of, rigid pose estimation allows us to determine a given object's primary keypoints and track them as they move through real-world spaces. How does this relate to AR? Well AR, in its essence, allows us to place digital objects in real-world scenes. This could be testing out a piece of furniture in your living room by placing a 3D rendering of it in the space, or trying on a pair of digitally-rendered shoes. So, where does pose estimation enter the picture? If we're able to locate and accurately track a physical object in real-world space, then we can also overlay a digital AR object onto the real object that's being tracked.

(3) Animation and Gaming

Traditionally, character animation has been a manual process that's relied on bulky and expensive motion capture systems. However, with the advent of deep learning approaches to pose estimation, there's the distinct potential that these systems can be streamlined and in many ways automated. Recent advances in both pose estimation and motion capture technology are making this shift possible, allowing for character animation that doesn't rely on markers or specialized suits, while still being able to capture motion in real-time. Similarly, capturing animations for immersive video game experiences can also potentially be automated by deep learning-based pose estimation. This kind of gaming experience was popularized with Microsoft's Kinect depth camera, and advances in gesture recognition promise to fulfill the real-time requirement that these systems demand.

(4) Robotics

Traditionally, industrial robotics has employed 2D vision systems to enable robots to perform their various tasks. However, this 2D approach presents a number of limitations. Namely,

computing the position to which a robot should move, given this 2D representation of space, requires intensive calibration processes, and become inflexible to environmental changes unless reprogrammed. With the advent of 3D pose estimation, however, the opportunity exists to create more responsive, flexible, and accurate robotics systems.

(5) Pose Estimation on Mobile

The benefits of using pose estimation aren't limited to applications that run on servers or in the cloud. In fact, pose estimation models can be made small and fast enough to run directly on mobile devices, opening up a wide range of possibilities: AI-powered sports coaches and personal trainers, highly-realistic and immersive AR practices, and crowd counting/tracking, to name a few. From brand loyalty, to user engagement and retention, and beyond, implementing pose estimation on-device has the potential to delight users in new and lasting ways, all while reducing cloud costs and keeping user data private. Fritz AI is the machine learning platform that makes it easy to teach devices how to see, hear, sense, and think. To learn more about how Fritz AI can help you build AI-powered sports coaches and personal trainers, immersive AR experiences, and more, check out our mobile Pose Estimation API.

Deep Learning and Pose Estimation

The classical approaches have its limitations and pose estimation has been greatly reshaped by CNNs. With the introduction of "DeepPose" on the human pose estimation began to shift from classic approaches to Deep Learning. Most of the recent pose estimation systems have universally adopted ConvNets as their main building block, largely replacing hand-crafted features and graphical models; this strategy has yielded drastic improvements on standard benchmarks.

(1) Deep Pose Human Pose Estimation via Deep Neural Networks

Deep Pose was the first major design that applied Deep Learning to Human pose estimation. It achieved SOTA performance and beat existing models. In this approach, pose estimation is formulated as a CNN-based regression problem towards body joints. They also use a cascade of such regressors to refine the pose estimates and get better estimates. One important thing this approach does is the reason about pose in a holistic fashion, i.e. even if certain joints are hidden, and they can be estimated if the pose is reasoned about holistically. CNNs naturally provide this sort of reasoning and exhibit strong results.

The model consisted of an AlexNet backend (7 layers) with an extra final layer that outputs $2k$ joint coordinates $(x_i, y_i) * 2$ for $i \in \{1, 2, \dots, k\}$ (where k is the number of joints). The model is trained using a L2 loss for regression. An interesting idea this model implements is refinement

of the predictions using cascaded regressors. Initial coarse pose is refined and a better estimate is achieved. Images are cropped around the predicted joint and fed to the next stage, in this way the subsequent pose regressors see higher resolution images and thus learn features for finer scales which ultimately leads to higher precision. Regressing to XY locations is difficult and adds learning complexity which weakens generalization and hence performs poorly in certain regions. Recent SOTA methods transform the problem to estimating K heatmaps of size $W_0 \times H_0, \{H_1, H_2, \dots, H_k\}$, where each heatmap H_k indicates the location confidence of the kth keypoint and K keypoints in total.

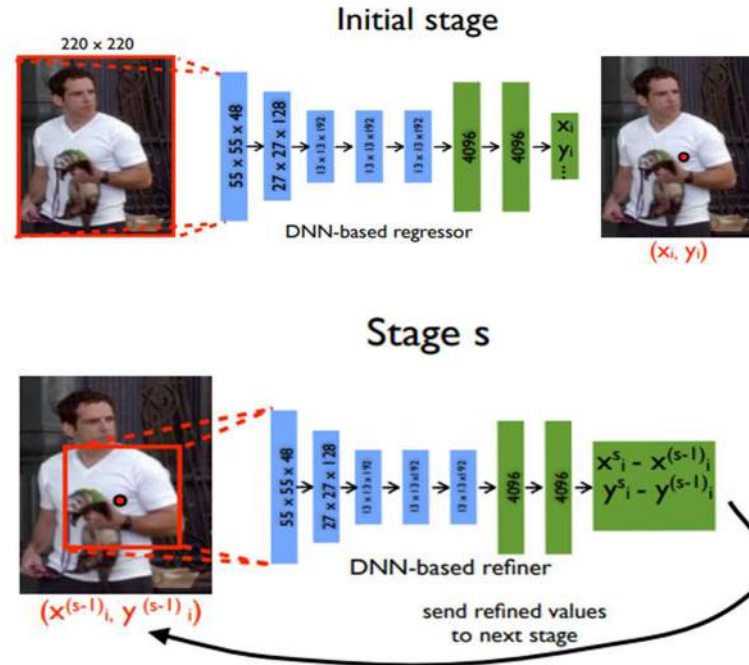


Fig. 5.3: Initial Stage and Stage S of DeepPose Human Pose Estimation

(2) Object Localization Using Convolutional Networks

This approach generates heatmaps by running an image through multiple resolution banks in parallel to simultaneously capture features at a variety of scales. The output is a discrete heatmap instead of continuous regression. A heatmap predicts the probability of the joint occurring at each pixel. This output model is very successful and a lot of research that followed predict heatmaps instead of direct regression. A multi-resolution CNN architecture (coarse heatmap model) is used to implement a sliding window detector to produce a coarse heatmap output. The main motivation is to recover the spatial accuracy lost due to pooling in the initial model. It is made possible by expending an additional ‘pose refinement’ ConvNet that refines

the localization result of the coarse heat-map. However, unlike a standard cascade of models, they reuse existing convolution features.

This not only reduces the number of trainable parameters in the cascade but also acts as a regularizer for the coarse heat-map model since the coarse and fine models are trained jointly. In essence, the model consists of the heat-map-based parts model for coarse localization, a module to sample and crop the convolution features at a specified (x,y) location for each joint, as well as an additional convolutional model for fine-tuning. A critical feature of this method is the joint use of a ConvNet and a graphical model. The graphical model learns typical spatial relationships between joints. The model is trained by minimizing the Mean Squared-Error (MSE) distance of our predicted heat-map to a target heat-map (The target is a 2D Gaussian of constant variance ($\sigma \approx 1.5$ pixels) centered at the ground-truth (x,y) joint location).

Heatmaps work better than direct joint regression. Joint use of a CNN and Graphical Model, however lacks structure modelling. The space of 2D human poses is highly structured because of body part proportions, left-right symmetries, interpenetration constraints, joint limits (e.g. elbows do not bend back) and physical connectivity (e.g. wrists are rigidly related to elbows), among others. Modelling this structure should make it easier to pinpoint the visible keypoints and make it possible to estimate the occluded ones.

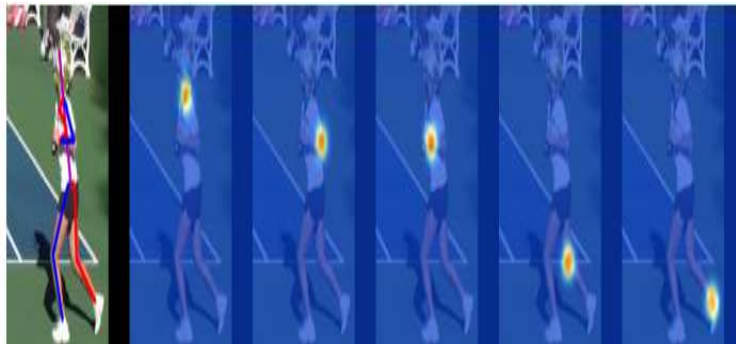


Fig. 5.4: Object Localization Using Convolutional Networks

(3) Convolutional Pose Machines

A pose machine consists of an image feature computation module followed by a prediction module. Convolutional Pose Machines are completely differentiable and their multi-stage architecture can be trained end to end. They provide a sequential prediction framework for learning rich implicit spatial models and work very well for Human pose. One of the main motivations is to learn long range spatial relationships and show that this can be achieved by using larger receptive fields.

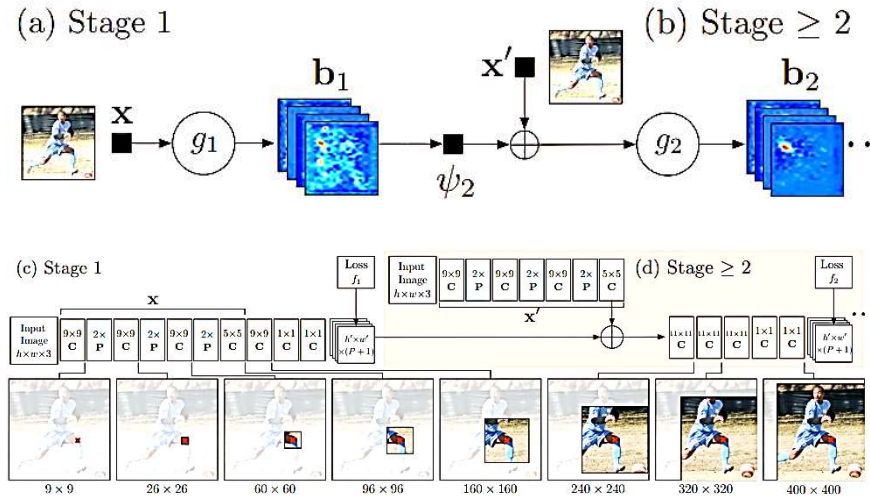


Fig. 5.5: Convolutional Pose Machines

$g_1()$ and $g_2()$ predict heatmaps (belief maps). Above is a high level view. Stage 1, the image feature computation module, and Stage 2 are the prediction module. Notice how the receptive fields increase in size? A CPM consists of > 2 Stages, and the number of stages is a hyperparameter. (Usually = 3). Stage 1 is fixed and stages > 2 are just repetitions of Stage 2. Stage 2 take heatmaps and image evidence as input. The input heatmaps add spatial context for the next stage. On a high level, the CPM refines the heatmaps through ensuing stages. The design use intermediate supervision after each stage to avoid the problem of vanishing gradients, which is a common problem for deep multi-stage networks.

5.3 WHAT IS GENERATIVE ADVERSARIAL NETWORKS?

Generative adversarial networks (GANs) are deep neural net architectures comprised of two nets, pitting one against the other (thus the “adversarial”). To understand GANs, you should know how generative algorithms work, and for that, contrasting them with discriminative algorithms is instructive. Discriminative algorithms endeavor to classify input data; that is, given the features of an instance of data, they predict a label or category to which that data belongs. For example, given all the words in an email (the data case), a discriminative algorithm could predict whether the message is spam or not spam. Spam is one of the labels, and the bag of words gathered from the email is the features that constitute the input data. When this problem is expressed mathematically, the label is called y and the features are called x .

The formulation $p(y|x)$ is used to mean “the probability of y given x ”, which in this case would translate to “the probability that an email is spam given the words it contains.” Thus,

discriminative algorithms map features to labels. They are concerned solely with that correlation. One way to think about generative algorithms is that they do the contradictory. Instead of predicting a label given certain features, they attempt to predict features given a certain label.

The question a generative algorithm tries to answer is: Assuming this email is spam, how likely are these features? While discriminative models care about the relation between y and x , generative models care about “how you get x .” They allow you to capture $p(x|y)$, the probability of x given y , or the probability of features given a label or category. (That said, generative algorithms can also be used as classifiers. It just so happens that they can do more than categorize input data.) Another way to think about it is to distinguish discriminative from generative like this:

- ✚ Discriminative models learn the boundary between classes
- ✚ Generative models model the distribution of individual classes

How GANs Work?

One neural network, called the generator, generates new data instances, while the other, the discriminator, evaluates them for authenticity; i.e. the discriminator decides whether each instance of data that it reviews belongs to the actual training dataset or not. Let’s say we’re trying to do something more banal than mimic the Mona Lisa. We’re going to generate hand-written numerals like those established in the Mixed National Institute of Standards and Technology MNIST (database of handwritten digits used for training image processing systems), which is taken from the real world. The goal of the discriminator, when shown an instance from the true MNIST dataset, is to recognize those that are authentic.

Meanwhile, the generator is creating new, synthetic images that it passes to the discriminator. It does so in the hopes that they, too, will be deemed authentic, even though they are fake. The goal of the generator is to generate passable hand-written digits: to lie without being caught. The goal of the discriminator is to identify images coming from the generator as fake.

Here are the steps a GAN takes:

- ✚ The generator takes in random numbers and returns an image.
- ✚ This generated image is fed into the discriminator alongside a stream of images taken from the actual, ground-truth dataset.
- ✚ The discriminator takes in both real and fake images and returns probabilities, a number between 0 and 1, with 1 representing a prediction of authenticity and 0 representing fake.

So you have a double feedback loop:

- ✚ The discriminator is in a feedback loop with the ground truth of the images, which we know.
- ✚ The generator is in a feedback loop with the discriminator.

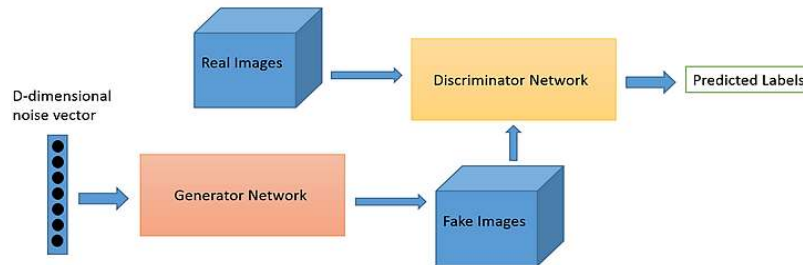


Fig. 5.6: Discriminator and Generator

You can think of a GAN as the opposition of a counterfeiter and a cop in a game of cat and mouse, where the counterfeiter is learning to pass false notes, and the cop is learning to detect them. Both are dynamic; i.e. the cop is in training, too (to extend the analogy, maybe the central bank is flagging bills that slipped through), and each side comes to learn the other's means in a constant escalation. For MNIST, the discriminator network is a typical convolutional network that can categorize the images fed to it, a binomial classifier labeling images as real or fake.

The generator is an inverse convolutional network, in a sense: While a standard convolutional classifier takes an image and downsamples it to produce a probability, the generator takes a vector of random noise and upsamples it to an image. The first throws away data through downsampling techniques like maxpooling, and the second generates new data. Both nets are trying to optimize a different and opposing objective function, or loss function, in a zero-sum game. This is basically an actor-critic model. As the discriminator changes its behavior, so does the generator, and vice versa. Their losses push against each other.

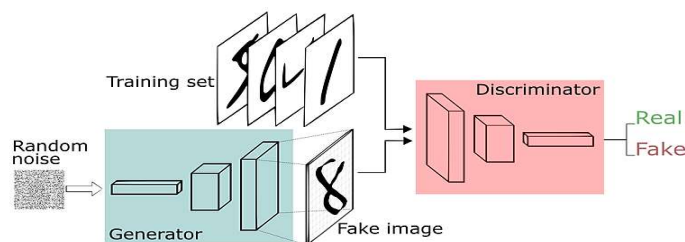


Fig. 5.7: Operation of GANs

GANs, Autoencoders and VAEs

It may be useful to compare generative adversarial networks to other neural networks, like autoencoders and variational autoencoders. Autoencoders encode input data as vectors. They engender a hidden, or compressed, portrayal of the raw data. They are useful in dimensionality reduction; that is, the vector serving as a hidden representation compresses the raw data into a smaller number of salient dimensions. Autoencoders can be paired with a so-called decoder, which allows you to reconstruct input data based on its hidden representation, much as you would with a restricted Boltzmann machine. Variational autoencoders are generative algorithm that adds an additional constraint to encoding the input data, namely that the hidden representations are normalized. Variational autoencoders are adept of both compressing data like an autoencoder and synthesizing data like a GAN. However, while GANs generate data in fine, granular detail, images generated by VAEs tend to be more blurred. Deep learning examples include both autoencoders and variational autoencoders.

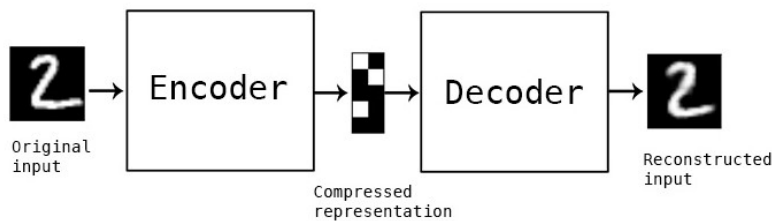


Fig. 5.8: Encoder and Decoder of Autoencoders

You can bucket generative algorithms into one of three types:

- ✚ Given a label, they predict the associated features (Naive Bayes)
- ✚ Given a hidden representation, they predict the associated features (VAE, GAN)
- ✚ Given some of the features, they predict the rest (inpainting, imputation)

Tips in Training a GAN

When you train the discriminator, hold the generator values constant; and when you train the generator, hold the discriminator constant. Each should train against a static adversary. For example, this gives the generator a better read on the gradient it must learn by. By the same token, pretraining the discriminator against MNIST before you start training the generator will establish a clearer gradient.

Each side of the GAN can overpower the other. If the discriminator is too good, it will return values so close to 0 or 1 that the generator will struggle to read the gradient. If the generator is

too good, it will persistently exploit weaknesses in the discriminator that lead to false negatives. This may be mitigated by the nets' respective learning rates. The two neural networks must have a similar "skill level." GANs take a long time to train. On a single GPU a GAN might take hours, and on a single CPU more than a day. While difficult to tune and therefore to use, GANs have stimulated a lot of interesting research and writing.

What Are Generative Models?

Suppose that we are interested in generating black and white square images of dogs with a size of n by n pixels. We can reshape each data as $N=n \times n$ dimensional vector (by stacking columns on top of each other) such that an image of dog can then be represented by a vector. However, it doesn't mean that all vectors represent a dog once shaped back to a square! So, we can say that the N dimensional vectors that effectively give something that look like a dog are distributed according to a very specific probability distribution over the entire N dimensional vector space (some points of that space are very likely to represent dogs whereas it is highly unlikely for some others). In the same spirit, there exists, over this N dimensional vector space, probability distributions for images of cats, birds and so on. Then, the problem of generating a new image of dog is equivalent to the problem of generating a new vector following the "dog probability distribution" over the N dimensional vector space. So we are, in fact, facing a problem of generating a random variable with respect to a specific probability distribution.

At this point, we can mention two important things. First the "dog probability distribution" we mentioned is a very complex distribution over a very large space. Second, even if we can assume the existence of such underlying distribution (there actually exists images that looks like dog and others that doesn't) we obviously don't know how to express explicitly this distribution. Both previous points make the process of generating random variables from this distribution pretty difficult. Let's then try to tackle these two problems in the following. Our first problem when trying to generate the new image of dog is that the "dog probability distribution" over the N dimensional vector space is a very complex one and we don't know how to directly generate complex random variables. However, as we know pretty well how to generate N uncorrelated uniform random variables, we could make use of the transform method.

To do so, we need to express the N dimensional random variable as the result of a very complex function applied to a simple N dimensional random variable! At this point, we can emphasize the fact that finding the transform function is not as candid as just taking the closed-form inverse of the Cumulative Distribution Function (that we obviously don't know) as we have done with inverse transforms method. The transform function can't be explicitly expressed and, then, we have to learn it from data.

As most of the time in these cases, very complex function naturally implies neural network modelling. Then and there, the idea is to model the transform function by a neural network that takes as input a simple N dimensional uniform random variable and that returns as output another N dimensional random variable that should follow, after training, the right “dog probability distribution”. Once the architecture of the network has been designed, we still need to train it. In the next two sections, we will discuss two ways to train these generative networks, including the idea of adversarial training behind GANs!

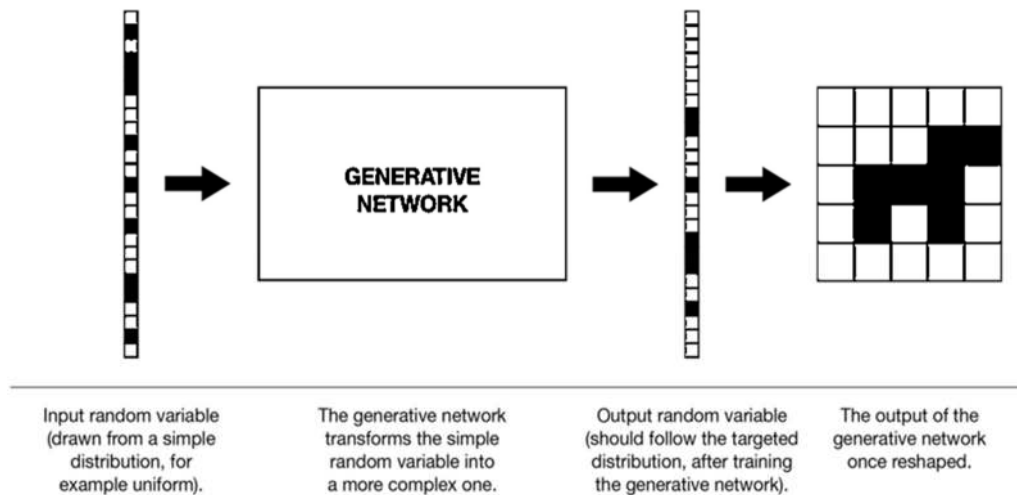


Fig. 5.9: What are Generative models?

The above Illustration is the notion of generative models using neural networks. Obviously, the dimensionality we are really talking about are much higher than represented here.

Generative Matching Networks

Training Generative Models

So far, we have shown that our problem of generating a new image of dog can be rephrased into a problem of generating a random vector in the N dimensional vector space that follows the “dog probability distribution” and we have suggested using a transform method, with a neural network to model the transform function. Now, we still need to train (optimize) the network to express the right transform function. To this purpose, we can suggest two different training methods: a direct one and an indirect one. The direct training method consists in matching the true and the generated probability distributions and backpropagating the difference (the error) through the network. This is the idea that rules Generative Matching Networks (GMNs).

For the indirect training method, we do not directly compare the true and generated distributions. In its place, we train the generative network by making these two distributions go through a downstream task chosen such that the optimization process of the generative network with respect to the downstream task will enforce the generated distribution to be close to the true distribution. This last idea is the one behind Generative Adversarial Networks (GANs) that we will present in the next section. But for now, let's start with the direct method and GMNs.

Comparing Two Probability Distributions Based On Samples

As mentioned, the idea of GMNs is to train the generative network by directly comparing the generated distribution to the true one. Nevertheless, we do not know how to express explicitly the true “dog probability distribution” and we can also say that the generated distribution is far too complex to be expressed explicitly. Thus, comparisons based on explicit expressions are not possible.

However, if we have a way to compare probability distributions based on samples, we can use it to train the network. Indeed, we have a sample of true data and we can, at each iteration of the training process, produce a sample of generated data. Although, in theory, any distance (or similarity measure) able to compare effectively two distributions based on samples can be used, we can mention in particular the Maximum Mean Discrepancy (MMD) approach. The MMD defines a distance between two probability distributions that can be computed (estimated) based on samples of these distributions.

Backpropagation of the Distribution Matching Error

Therefore, once we have defined a way to compare two distributions based on samples, we can define the training process of the generative network in GMNs. Given a random variable with uniform probability distribution as input, we want the probability distribution of the generated output to be the “dog probability distribution”. The idea of GMNs is then to optimize the network by repeating the following steps:

- ✚ Generate some uniform inputs.
- ✚ Make these inputs go through the network and collect the generated outputs
- ✚ Compare the true “dog probability distribution” and the generated one based on the available samples (for example compute the MMD distance between the sample of true dog images and the sample of generated ones)
- ✚ Use backpropagation to make one step of gradient descent to lower the distance (for example MMD) between true and generated distributions

As written above, when following these steps we are applying a gradient descent over the network with a loss function that is the distance between the true and the generated distributions at the current iteration.

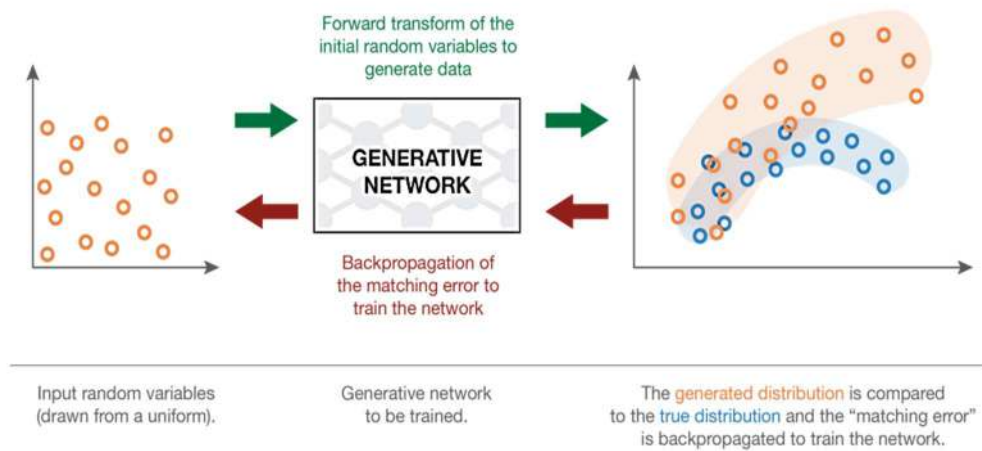


Fig. 5.10: Generative Matching Networks

Generative Matching Networks take simple random inputs, generate new data, and directly compare the distribution of the generated data to the distribution of the true data and backpropagate the matching error to train the network.

The “Indirect” Training Method (in GANs)

The “direct” approach presented above compare directly the generated distribution to the true one when training the generative network. The brilliant idea that rules GANs consists in replacing this direct comparison by an indirect one that takes the form of a downstream task over these two distributions. The training of the generative network is then done with respect to this task such that it forces the generated distribution to get closer and closer to the true distribution.

The downstream task of GANs is a discrimination task between true and generated samples. Or we could say a “non-discrimination” task as we want the discrimination to fail as much as possible. So, in a GAN architecture, we have a discriminator, that takes samples of true and generated data and that try to classify them as well as possible, and a generator that is trained to fool the discriminator as much as possible. Let’s see on a simple example why the direct and indirect approaches we mentioned should, in theory, lead to the same optimal generator.

The Ideal Case: Perfect Generator and Discriminator

In order to better understand why training a generator to fool a discriminator will lead to the same result as training directly the generator to match the target distribution, let's take a simple one dimensional example. We forget, for the moment, how both generator and discriminators are represented and consider them as abstract notions (that will be specified in the next subsection). Moreover, both are supposed “perfect” (with infinite capacities) in the sense that they are not constrained by any kind of (parameterized) model. Suppose that we have a true distribution, for example a one dimensional Gaussian, and that we want a generator that samples from this probability distribution. What we called “direct” training method would then consist in adjusting iteratively the generator (gradient descent iterations) to correct the measured difference/error amid true and generated distributions. Finally, assuming the optimization process perfect, we should end up with the generated distribution that matches exactly the true distribution.

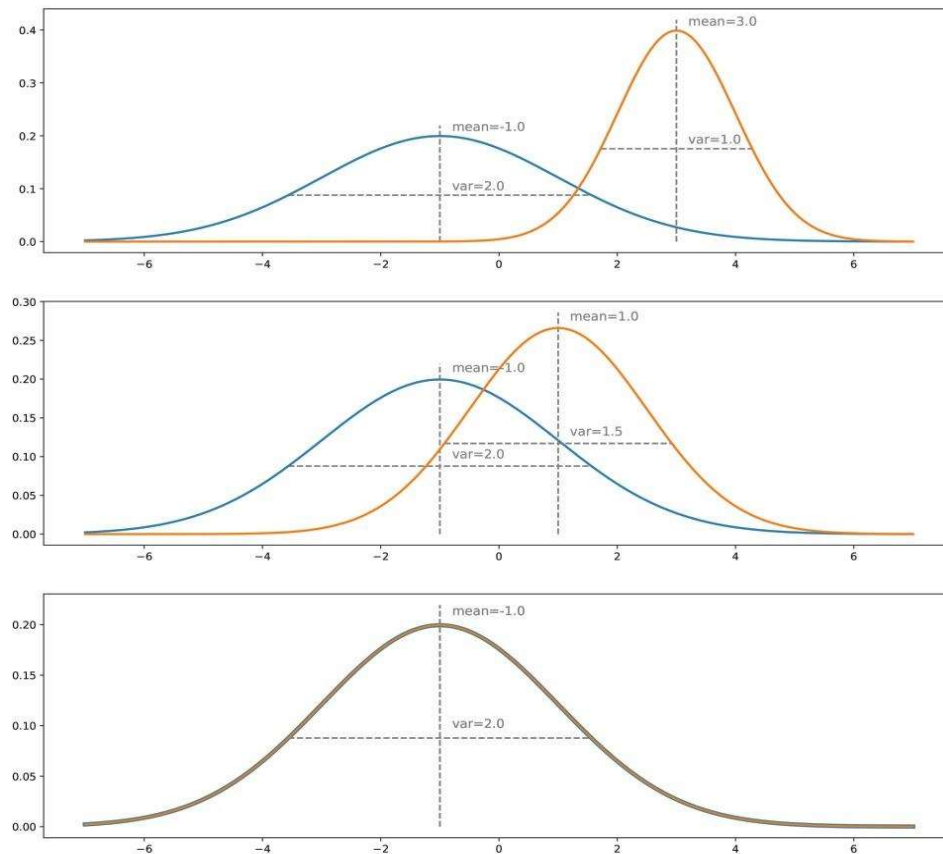


Fig. 5.11: Illustration of the Concept of Direct Matching Method.

The distribution in blue is the true one while the generated distribution is depicted in orange (Figure 5.11). Iteration by iteration, we compare the two distributions and adjust the networks weights through gradient descent steps. Here the comparison is done over the mean and the variance (similar to a truncated moments matching method). Notice that (obviously) this example is so simple that it doesn't require an iterative approach: the drive is only to illustrate the intuition given above.

For the “indirect” approach, we have to consider also a discriminator. We assume for now that this discriminator is a kind of oracle that knows exactly what the true and generated distribution is and that is able, based on this information, to predict a class (“true” or “generated”) for any given point. If the two distributions are far apart, the discriminator will be able to classify easily and with a high level of confidence most of the points we present to it. If we want to fool the discriminator, we have to bring the generated distribution close to the true one.

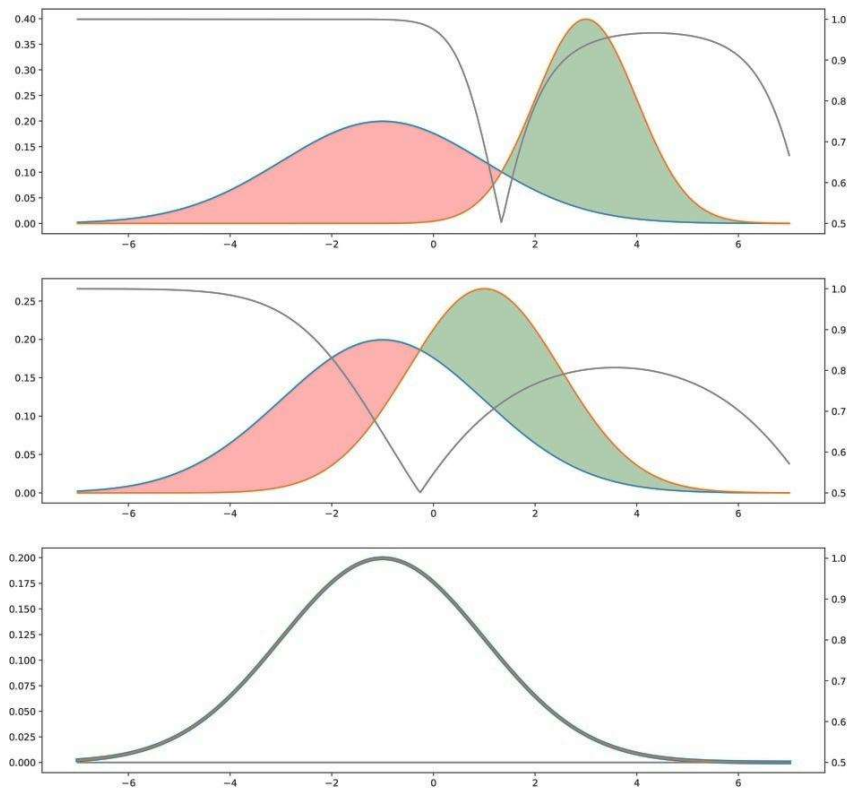


Fig. 5.12: Intuition for the Adversarial Method.

The discriminator will have the most difficult to predict the class when the two distributions will be equal in all points: in this case, for each point there are equal chances for it to be “true”

or “generated” and then the discriminator can’t do better than being true in one case out of two in average. The blue distribution is the true one, the orange is the generated one (Figure 5.12). In grey, with corresponding y-axis on the right, shows the probability to be true for the discriminator if it chooses the class with the higher density in each point (assuming “true” and “generated” data are in equal proportions). The closer the two distributions are, the more often the discriminator is wrong. When training, the goal is to “move the green area” (generated distribution is too high) towards the red area (generated distribution is too low).

At this point, it seems legit to wonder whether this indirect method is really a good idea. Indeed, it seems to be more complicated (we have to optimize the generator based on a downstream task instead of directly based on the distributions) and it requires a discriminator that we consider here as a given oracle but that is, in reality, neither known nor perfect. For the first point, the difficulty of directly comparing two probability distributions based on samples counterbalances the apparent higher complexity of indirect method. For the second point, it is obvious that the discriminator is not known. However, it can be learned!

The Approximation: Adversarial Neural Networks

Let’s now describe the specific form that takes the generator and the discriminator in the GANs architecture. The generator is a neural network that models a transform function. It takes as input a simple random variable and must return, once trained a random variable that follows the targeted distribution. As it is very complicated and unknown, we decide to model the discriminator with another neural network. This neural network models a discriminative function. It takes as input a point (in our dog example N dimensional vector) and returns as output the probability of this point to be a “true” one. Notice that the fact that we impose now a parameterized model to express both the generator and the discriminator (instead of the idealized versions in the previous subsection) has, in practice, not a huge impact on the theoretical argument/intuition given above: we just then work in some parameterized spaces instead of ideal full spaces and, so, the optimal points that we should reach in the ideal case can then be seen as “rounded” by the precision capacity of the parameterized models. Once defined, the two networks can then be trained jointly (at the same time) with opposite goals:

- 🚩 The goal of the generator is to fool the discriminator; accordingly the generative neural network is trained to maximize the final classification error (between true and generated data).
- 🚩 The goal of the discriminator is to detect fake generated data, so the discriminative neural network is trained to minimize the final classification error.

So, at each iteration of the training process, the weights of the generative network are updated in order to increase the classification error (error gradient ascent over the generator’s

parameters) whereas the weights of the discriminative network are updated so that to decrease this error (error gradient descent over the discriminator's parameters).

The generator takes simple random variables as inputs and generates new data. The discriminator takes “true” and “generated” data and tries to discriminate them, building a classifier. The goal of the generator is to fool the discriminator (increase the classification error by mixing up as much as possible generated data with true data) and the goal of the discriminator is to distinguish between true and generated data.

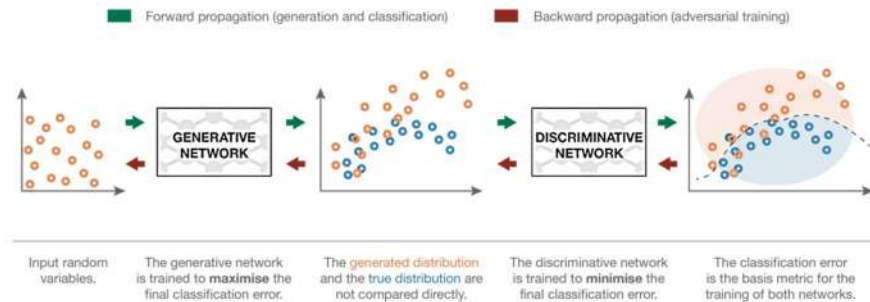


Fig. 5.13: Generative Adversarial Networks Representation

These opposite goals and the implied notion of adversarial training of the two networks explain the name of “adversarial networks”: both networks try to beat each other and, doing so, they are both becoming better and better. The rivalry between them makes these two networks “progress” with respect to their respective goals. From a game theory point of view, we can think of this setting as a minimax two-player game where the equilibrium state corresponds to the situation where the generator produces data from the exact targeted distribution and where the discriminator predicts “true” or “generated” with probability 1/2 for any point it receives.

Mathematical Details about GANs

Neural networks modelling mainly require defining two things: architecture and a loss function. We have already defined the architecture of Generative Adversarial Networks. It consists in two networks:

- Generative network $G(\cdot)$ that takes a random input z with density p_z and returns an output $x_g = G(z)$ that ought to follow (after training) the targeted probability distribution.
- Discriminative network $D(\cdot)$ that takes an input x that can be a “true” one (x_t , whose density is denoted p_t) or a “generated” one (x_g , whose density p_g is the density induced

by the density p_z going through G) and that returns the probability $D(x)$ of x to be a “true” data.

Let’s take now a closer look at the “theoretical” loss function of GANs. If we send to the discriminator “true” and “generated” data in the same proportions, the expected absolute error of the discriminator can then be expressed as;

$$\begin{aligned} E(G, D) &= \frac{1}{2} \mathbb{E}_{x \sim p_t} [1 - D(x)] + \frac{1}{2} \mathbb{E}_{z \sim p_z} [D(G(z))] \\ &= \frac{1}{2} (\mathbb{E}_{x \sim p_t} [1 - D(x)] + \mathbb{E}_{x \sim p_g} [D(x)]) \end{aligned}$$

The goal of the generator is to fool the discriminator whose goal is to be able to distinguish between true and generated data. So, when training the generator, we want to maximize this error while we try to minimize it for the discriminator. It gives us;

$$\max_G \left(\min_D E(G, D) \right)$$

For any given generator G (along with the induced probability density p_g), the best possible discriminator is the one that minimizes;

$$\mathbb{E}_{x \sim p_t} [1 - D(x)] + \mathbb{E}_{x \sim p_g} [D(x)] = \int_{\mathbb{R}} (1 - D(x)) p_t(x) + D(x) p_g(x) dx$$

In order to minimize (with respect to D) this integral, we can minimize the function inside the integral for each value of x . It then defines the best possible discriminator for a given generator;

$$\mathbb{1}_{(p_t(x) \geq p_g(x))}$$

(In fact, one of the best because x values such that $p_t(x) = p_g(x)$ could be handled in another way but it doesn’t matter for what follows). We then search G that maximizes;

$$\int_{\mathbb{R}} (1 - D_G^*(x)) p_t(x) + D_G^*(x) p_g(x) dx = \int_{\mathbb{R}} \min(p_t(x), p_g(x)) dx$$

Again, in order to maximize (with respect to G) this integral, we can maximize the function inside the integral for each value of x . As the density p_t is independent of the generator G , we can’t do better than setting G such that;

$$p_g(x) \geq p_t(x)$$

Of course, as p_g is a probability density that should integrate to 1, we necessarily have for the best G

$$p_g(x) = p_t(x)$$

So, we have shown that, in an ideal case with unlimited capacities generator and discriminator, the optimal point of the adversarial setting is such that the generator produces the same density as the true density and the discriminator can't do better than being true in one case out of two, just like the intuition told us. Finally, notice also that G maximizes

$$\frac{1}{2} \int_{\mathbb{R}} \min(p_t(x), p_g(x)) dx = \int_{\mathbb{R}} \frac{\min(p_t(x), p_g(x))}{p_t(x) + p_g(x)} \frac{p_t(x) + p_g(x)}{2} dx$$

Under this form, we better see that G wants to maximize the expected probability of the discriminator to be wrong.

Summary

The main takeaways of this section are:

- ✚ Computers can basically generate simple pseudo-random variables (for example they can generate variables that follow very closely a uniform distribution).
- ✚ There exists different ways to generate more complex random variables including the notion of “transform method” that consists in expressing a random variable as a function of some simpler random variable(s).
- ✚ In machine learning, the generative models try to generate data from a given (complex) probability distribution.
- ✚ Deep learning generative models are modelled as neural networks (very complex functions) that take as input a simple random variable and that return a random variable that follows the targeted distribution (“transform method” like).
- ✚ These generative networks can be trained “directly” (by comparing the distribution of generated data to the true distribution): this is the idea of Generative Matching Networks.
- ✚ These generative networks can also be trained “indirectly” (by trying to fool another network that is trained at the same time to distinguish “generated” data from “true” data): this is the idea of Generative Adversarial Networks.

The way to twist the loss function to go from a direct comparison to an indirect one is really something that can be very inspiring for further works in the deep learning area.

5.4 WHAT ARE STYLEGAN AND IMAGE SYNTHESIS?

Ever wondered what the 27th letter in the English alphabet might look like? Or how your appearance would be twenty years from now? Or perhaps how that super-grumpy professor of yours might look with a big, wide smile on his face? Thanks to machine learning, all this is not only possible, but relatively easy to do with the inference of a powerful neural network (rather than hours spent on Photoshop). The neural networks that make this possible are termed adversarial networks. Often described as one of the coolest concepts in machine learning, they are actually a set of more than one network (usually two) which are continually competing with each other (hence, adversarially), producing some interesting results along the way.

StyleGANs, a type of generative adversarial network that “enables intuitive, scale-specific control of image synthesis by learned, unsupervised separation of high-level attributes and stochastic variation”. Or, to put it plainly, StyleGANs switch up an image’s style. StyleGAN was originally an open-source project by NVIDIA to create a generative model that could output high-resolution human faces. The generator in a StyleGAN makes small adjustments to the “style” of the image at each convolution layer in order to manipulate the image features for that layer. Likewise, this new architecture is able to separate the high-level attributes (such as a person’s identity) from low-level attributes (such as their hairstyle) within an image. This separation is what allows the GAN to change some attributes without affecting others. For example, changing a person’s hairstyle in a given image. Before we dive into the specifics of how StyleGANs works, here’s a list of interesting implementations, just to give you a sense of what’s possible with these powerful neural networks:

-  Generating Game of Thrones characters
-  Generating new watch styles
-  Generating album covers
-  Generating Pokémon characters
-  Generating portraits (paintings)

How Does The StyleGAN Works?

Before diving into the changes made by the researchers to the GAN network architecture to build their StyleGAN, it’s important to note that these changes pertain only to the generator network, thus influencing the generative process only. There have been no changes to the discriminator or to the loss function, both of which remain the same as in a traditional GAN. The purpose of NVIDIA’s StyleGAN is to overcome the limitations of a traditional GAN, wherein control may not be possible for individual characteristics of data, such as facial features in photographs. The proposed model allows a user to tune hyperparameters in order to achieve

such control. Moreover, it allows for a factor of variability in generated images due to the addition of “styles” to images at each convolution layer.

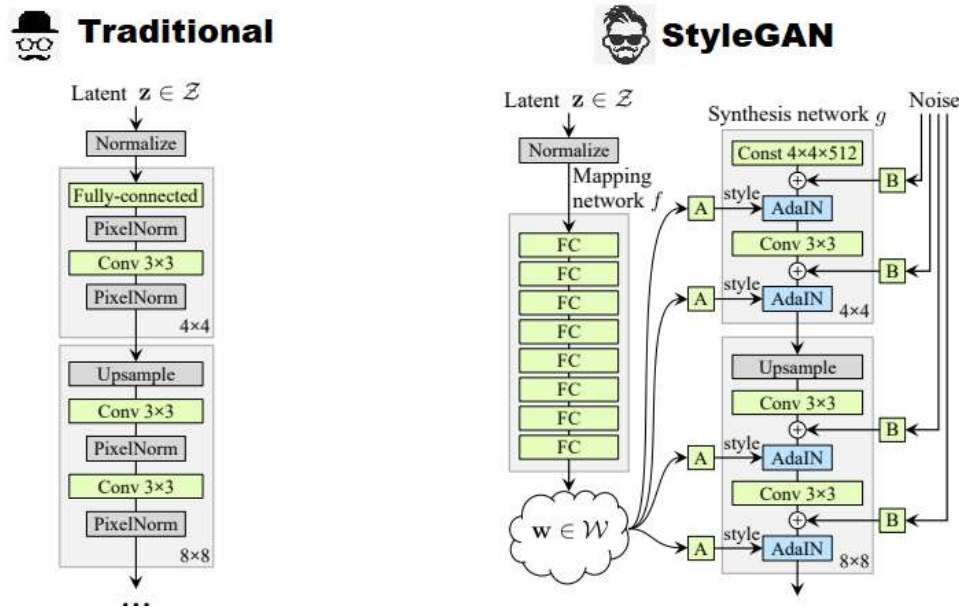


Fig. 5.14: GAN Generator vs. StyleGAN Generator

The model starts off by generating new images, starting from a very low resolution (something like 4x4) and eventually building its way up to a final resolution of 1024x1024, which actually provides enough detail for a visually appealing image. The main principle behind training the StyleGAN is this “progressive” method which was first used by NVIDIA in their ProGAN. It works by gradually increasing the resolution, thus ensuring that the network evolves slowly, initially learning a simple problem before progressing to learning more complex problems(or, in this case, images of a higher resolution).

This kind of training principle ensures stability and has been proven to minimize common problems associated with GANs such as mode collapse. It also makes certain that high level features are worked upon first before moving on to the finer details, reducing the likelihood of such features being generated wrong (which would have a more drastic effect on the final image than the other way around). StyleGANs use a similar principle, but instead of generating a single image they generate multiple ones, and this technique allows for styles or features to be dissociated from each other. Specifically, this method causes two images to be generated and then combined by taking low-level features from one and high-level features from the other. A mixing regularization technique is used by the generator, causing some percentage of both to appear in the output image.

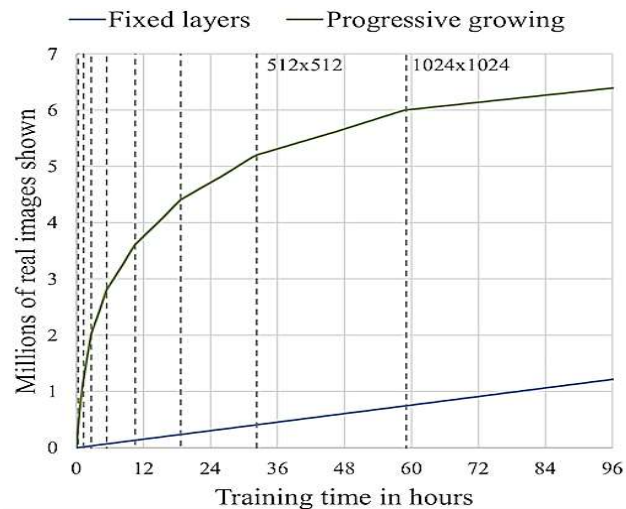


Fig. 5.15: Progressive Growth

At every convolution layer, different styles can be used to generate an image: coarse styles having a resolution between 4x4 to 8x8, middle styles with a resolution of 16x16 to 32x32, or fine styles with a resolution from 64x64 to 1024x1024. Coarse styles govern high-level features such as the subject's pose of in the image or the subject's hair, face shape, etc. Middle styles control aspects such as facial features.

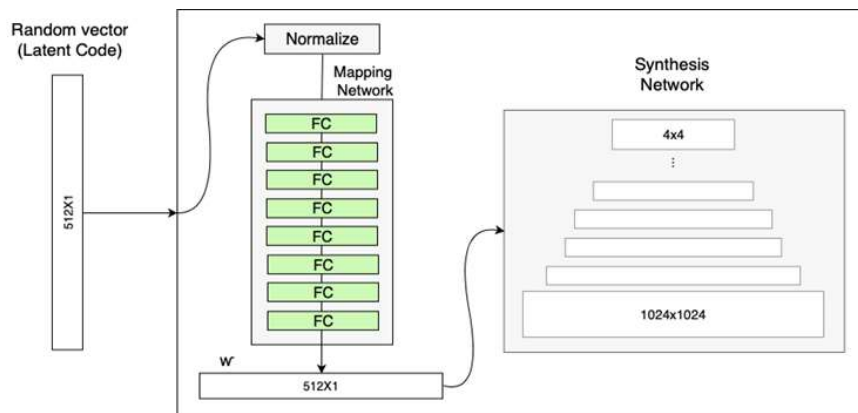


Fig. 5.16: Mapping Network

Lastly, fine styles cover details in the image such as color of the eyes or other microstructures. The Style GAN architecture also adds noise on a per-pixel basis after each convolution layer. This is done in order to create “stochastic variation” in the image. The researchers observe that adding noise in this way allows a localized style changes to be applied to “stochastic” aspects

of the image, such as wrinkles, freckles, skin pores, stubble, etc. In a traditional GAN, the generator network would obtain a random “latent” vector as its input, and using multiple transposed convolutions, would alter that vector into an image that would appear authentic to the discriminator. This latent vector can be thought of as a tensor representation of the image to the network.

The traditional GAN doesn’t allow for control over finer styling of the image because it follows its own distribution, as governed by its training with high-level attributes, and also because it gets influenced by the general “trend” of its dataset (say for example, a dominant hair color throughout the dataset). The most one could do is change the input image (the vector) and thus obtain a different result.

NVIDIA’s architecture includes an intermediate “latent space”, which can be thought of as being “detachable”. Input latent code can then be embedded into this space. The styles or features present in images are actually different forms of the same latent vector embedding, which is used to normalize the input of each convolutional layer. Additional noise is also fed to the network in order to assert greater control over the finer details. When generating the output image, the user switches between latent codes at a selected point in the network, thus leading to a mixing of styles.

Feature Disentanglement

The reason why traditional GANs have a problem with control of styles or features within the same image is owing to something called feature entanglement. As the name suggests, a GAN is not as capable of distinguishing these finer details as a human, thus leading the features to become “entangled” with each other to some extent within the GAN’s frame of perception. A good example would be “entanglement” between the features of hair color and gender. If the dataset used for training has a general trend of males having short hair and females having long hair, the neural network would learn that males can only have short hair and vice-versa for females. As a result, changing the latent vector to obtain long hair for the image of a male in the result would also end up changing the gender, leading to an image of a woman.

Using the intermediate latent space, the Style GAN architecture lets the user make small changes to the input vector in such a way that the output image is not altered dramatically. A “mapping network” is included that maps an input vector to another intermediate latent vector, which is then fed to the generator network. The researchers made use of a network with 8 layers for this purpose, whose input and output is both a vector with 512 dimensions. Here, they also make the case as to why these hyperparameters work best. They present two separate approaches to measure feature disentanglement:

- 📊 **Perceptual Path Length:** Calculate the difference amid VGG16 embeddings of images when interpolating between two random inputs. A drastic change indicates that multiple features have changed together and thus might be entangled.
- 📊 **Linear Separability:** Classify all inputs into binary classes, such as male and female. The better the classification, the more separable the features.

Here's a chance for you to get your hands dirty. Not only has NVIDIA made the whole project open-source, but they've also released a variety of resources, documentation, and pre-trained models for developers to play around with. A coder's paradise! StyleGAN was trained on the CelebA-HQ and FFHQ datasets for one week using 8 Tesla V100 GPUs. Its implementation is in TensorFlow and can be found in NVIDIA's GitHub repository, made available under the Creative Commons BY-NC 4.0 license.

This means you can practice, redistribute, and adapt the material for non-commercial purposes, as long as you give appropriate credit by citing the research paper and indicating any changes made. The researchers trained their model on the CelebA-HQ and Flickr-Faces-HQ datasets, both containing 1024x1024 resolution images. They strongly advise training with 8 GPUs in order to produce similar results. In addition to the faces datasets, the researchers also used their StyleGAN on three other datasets: the LSUN BEDROOMS, CARS, and CATS datasets.

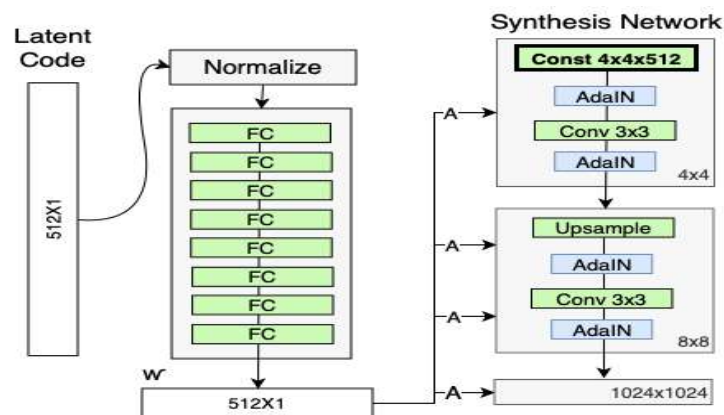


Fig. 5.17: Normalize vs. Synthesis Network

In the BEDROOM dataset, the coarse styles control the viewpoint of the camera, the middle styles select the particular furniture, and fine styles deal with colors and smaller details of materials. The same concept holds true for the other two datasets. The effects of stochastic variation can be observed in the fabrics in BEDROOM, backgrounds and headlamps in CARS, and fur, background, and the positioning of paws in the CATS dataset. The StyleGAN has been widely used by developers to tinker with image datasets, and many interesting results can be

found. From generating anime characters to creating brand-new fonts and alphabets in various languages, one could safely note that StyleGAN has been experimented with quite a lot. ThisPersonDoesNotExist.com also implements a StyleGAN to generate a fake high-res face every time the page is refreshed. It would also be worthwhile to take a look at some more recent findings and developments that use NVIDIA's research as a basis. This section summarizes the recent work relating to StyleGANs with a deep learning approach.

Semantic Image Synthesis with Spatially-Adaptive Normalization

Similarly, another method for photorealistic style transfer is Semantic Image Synthesis with Spatially-Adaptive Normalization (SPADE), or also called as spatially-adaptive normalization, acts as a layer for synthesizing images given an input semantic layout instead of directly feeding the semantic layout as input to a neural network. This input is then processed through stacks of convolution, normalization, and non-linearity layers. In many cases, traversing through this deep network of layers tends to "wash away" semantic information. NVIDIA's GauGAN is based on this approach. It creates photorealistic images from segmentation maps, which are labeled sketches that depict the layout of a scene. Users can use paintbrush and paint bucket tools to design their own landscapes with labels like river, rock, and cloud. A style transfer algorithm allows users to apply filters, change a daytime scene to sunset, or a photograph to a painting. Users can even upload their own filters to layer onto their masterpieces, or upload custom segmentation maps and landscape images as a foundation for their artwork.

Image2StyleGAN:

How to Embed Images into the StyleGAN Latent Space?

Yet another algorithm aims to embed a given image into the latent space of StyleGAN. Supposedly, this embedding enables semantic image editing operations that can be purposeful to existing photographs. Taking the StyleGAN trained on the FFHD dataset as an example, researchers were able to successfully demonstrate results for image morphing, style transfer, and expression transfer.

You can't synthesize a picture out of nothing, we assume; a picture had to be of someone. Sure a scammer could appropriate someone else's picture, but doing so is a risky strategy in a world with google reverse search and so forth. So we tend to trust pictures. A business profile with a picture visibly belongs to someone. A match on a dating site may turn out to be 10 pounds heavier or 10 years older than when a picture was taken, but if there's a picture, the person obviously exists. New adversarial machine learning algorithms allow people to rapidly generate synthetic 'photographs' of people who have never existed." StyleGAN is easily the most powerful GAN in existence.



Fig. 5.18: Morphing Between Two Embedded Images

With the ability to generate synthesized images from scratch in high resolution, some would dub its capabilities scary. Given that we live in an age where many security systems rely on measures such as facial recognition and images form a major part of all the data on the web, it's important for people to be aware of such technology and to not unquestioningly trust information just because it's in image form. On the one hand, such developments in machine learning make redundant specialized skills in the domain of image manipulation and engineering. On the other, they offer opportunities for developing skill sets specific to the machine learning domain, for which there is currently a huge demand. As ML development and incorporation grow more commonplace in every sector, we can expect many more milestones such as these soon. Stay tuned!

5.5 WHAT IS LANGUAGE AND VISION?

Language and Vision are two fundamental modalities through which human beings acquire knowledge about the world. We see and speak about things and events around us, and by doing so; we learn properties and relations about objects. These two modalities are quite interdependent and we constantly mix information we acquire through them. But, computational models of language and vision have been developing separately and the two research communities have for a long time been unaware of each other's work. Excitingly, through these parallel research lines, they have developed highly compatible representations of words and images, respectively. The importance of developing computational models of language and vision together has been highlighted by philosophers and cognitive scientists since the birth of the Artificial Intelligence paradigm. Only recently, however, the challenge has been empirically taken up by computational linguists and computer vision researchers.

In the last two decades, the availability of large amounts of text on the web has led to tremendous improvements in NLP research. Sophisticated textual search engines are now well consolidated and part of everybody's daily life. Images are the natural next challenge of the

digital society. The combination of language and vision is the winning horse for this new era. Driven by theoretical questions, we look at applications as the test bed for such models. The focus so far has been on the investigation of multimodal models combining linguistic and visual vector representations; cross-modal mapping from visual to language space; enhancement of visual recognizers through language models.

- ✚ Modeling the acquisition of cognitive core skills: Can multimodal models acquire core cognitive skills, e.g. numerosity? Is this a core skill also for the models, in other words does it facilitate learning other skills?
- ✚ Learning to see through interaction: Infants learn by interacting with the others and the world outside. Can model learn to ground language into vision through interaction?

How do you use verbal instructions to teach robots to target destinations, without access to a map? Robots that can navigate based on vision and perform tasks under instructions in ordinary language sounds like science fiction, but it's closer to reality than you might think. One of the big moves of AI is to have robots interact with the visual and physical world, especially via natural language. Vision-language navigation can enable many practical applications, for example, in-home domestic robots. Everyone, particularly the disabled, can ask the robot to perform some certain tasks, for example 'Go to the kitchen and bring a cup of water to me.' Vision-language navigation serves as a basic task to understand both visual scenes and natural language instructions, as well as perform physical actions to fulfill high-level jobs.

But what comes so naturally to people, navigating through a scene and performing tasks using real-time environmental cues and human concepts requires a flexible, hitherto elegant framework in which robots can learn to connect the data they pick up to the meaning of the instructions they receive. It's an iterative process, requiring reinforcement, feedback and adaptation. This breakthrough in language and vision research will enable robots to better assist humans in many daily and special routines, including home cleaning and maintenance, item finding and retrieving, remote control, assisting blind people, disaster relief, et cetera. It has the potential to influence millions of people and improve the life quality of humans, including freeing humans from tedious household tasks, so that we can make time for creative activities. Vision-language navigation for robots can be performed using any of the following;

- ✚ Model-based reinforcement learning
- ✚ Model-free reinforcement learning
- ✚ Self-imitation learning

Vision Language and Robots

Integrative language and vision software may enable an autonomous robot to interact with people in different environments and realize navigational goals. To accomplish this task, the

robot must operate safely in people's presence, encourage them to provide directions and use their information to find the goal. Graphic and language data can be integrated into its navigational process in order to locate a specific place or person. This robot could help self-driving cars communicate with passengers and pedestrians or could complete small-scale tasks in a business place such as delivering mail. The robot when received a task to locate a specific room, building or individual in a known or unknown location, will unite novel language and visual processing in order to navigate the environment, ask for directions, request doors to be opened or elevator buttons pushed and reach its goal.



Fig. 5.19: Vision Language Enabled Robot in Delivery Job

The researchers are developing high-level software to give the robot "common sense knowledge," the ability to cognize objects and atmospheres with human-level intuition, enabling it to identify navigational conventions. For example, the robot will integrate both spoken statements and physical gestures into its navigation route. This is where common sense knowledge comes in. The robot should know that odd and even numbered rooms sit across from each other in a hallway or that Room 317 should be on the building's third floor.

To develop the robot's common sense knowledge, the researches will ripen integrative natural language processing and computer vision software. Typically, natural language processing will enable the robot to communicate with people while the computer vision software will enable the robot to navigate its environment. Nevertheless, the researchers are advancing the software to inform each other as the robot moves. The robot needs to understand language in a visual context and vision in a language context. For example, while locating a specific person, the robot might receive information in a comment or physical gesture and must understand both within the context of its navigational goals. For instance, if the response is "Check for that

person in Room 300," the robot will need to process the statement in a visual context and identify what room it is currently in as well as the best route to reach Room 300. If the response is "That person is over there" with a physical cue, the robot will need to integrate the visual cue with the statement's meaning in order to identify a Person.

Interacting with humans is an unsolved problem in artificial intelligence, to help the robot to understand certain conventions it might run into or to anticipate that a dozen different responses could all have the same meaning. The technology could be adapted into self-driving cars, allowing the cars to ask for directions or passengers to request a specific destination, just like human drivers do. The researchers expect to send the robot on autonomous missions with growing complexity as the technology progresses. First, the robot will learn to navigate indoors on a single floor. Then, to move to other floors and buildings, it will ask people to operate the elevator or open doors for it. The researchers hope to progress to outdoor missions. "We expect this technology to be really big, because the industry of autonomous robots and self-driving cars is fetching bigger.

Visual and Textual Context

Online data, including the web and social media, presents an unprecedented visual and textual summary of human lives, events, and activities. The need is in designing scalable new machine learning algorithms that can make sense of this ever growing body of information, by automatically inferring the latent semantic correspondences between language, images, videos, and 3D models of the world. The ultimate goal is to achieve deep semantic integration at a very large scale. Given nearly unrestrained amounts of multimodal data, the aim is to build statistical models that can transform information in one modality to the other, for example to automatically summarize a visual scene or better find images that match complex similes. Crucial to this goal will be new methods for extracting and organizing commonsense knowledge about the visual world automatically, from large-scale data, and with minimal supervision. This work falls in the vision of future artificial intelligence; highly integrative research that spans disciplines to build a new generation of understanding systems.

Turing Test for Vision

The goal of language and vision research is to combine vision with aspects of language and social cognition to obtain complex knowledge about the environment. To obtain a full indulgent of visual scenes, computational models should be able to extract from the scene any eloquent information that a human observer can extract about actions, agents, goals, scenes and object configurations, social interactions, and more. We refer to this as the 'Turing test for vision,' i.e., the ability of a machine to use vision to answer a large and flexible set of queries about objects and agents in an image in a human-like manner. Queries might be about objects, their

parts, and spatial relations between objects, actions, goals, and interactions. Understanding queries and formulating answers requires interactions between vision and natural language.

Interpreting goals and interactions requires connections between vision and social cognition. Answering queries also requires task-dependent processing, i.e., different visual processes to achieve different goals. Turing Test for Vision is being able to use vision to answer a large and flexible set of queries about objects and agents in the image in a human-like manner. Queries can be, for example, about objects, their parts, spatial relations between objects, actions, goals, and interactions. Understanding queries and formulating answers requires interactions between vision and natural language. Interpreting goals and interactions requires connections between vision and social cognition. Answering queries also requires task-dependent processing, i.e., different visual processes to achieve different goals.

To achieve language and vision goals we need to develop novel methods of extracting meaningful information from images established on protracted interpretation and goal-directed processing. Semantic image interpretation often requires an extended process directed to specific objects and relations in a task-dependent manner, e.g., what is person X looking at or touching, or is object Y stable. This method will combine probabilistic inference with policy learning to generate a sequence of operations applied to the image. The first stage will construct in a bottom-up manner an initial interpretation of the scene, and the second will generate and apply an interpretation in a task dependent manner, in which different processes can be synthesized in response to different queries. The first component will produce rich hierarchical representations in a robust and invariant manner. The second stage, generating extended goal directed processing, will use policy learning related to Markov Decision Processes and reinforcement learning.

Close interactions with the Social Intelligence research thrust involve understanding actions, goals, and interactions among agents. Interactions with the Development of Intelligence research thrust involve incorporating useful structures and biases derived from human developmental cognition. Hierarchical object recognition connects with the Circuits for Intelligence research thrust for modeling cortical mechanisms of hierarchical representations and object recognition, and for involving the computational constraints with neuronal circuits. Computational theories will be used to design stimuli for testing neuronal responses to object configurations, agent-object interactions, and interactions among agents in Circuits for Intelligence, and for predicting, testing, and analyzing interactions among brain areas (Social Intelligence).

Visual aspects of social interactions connect with the Social Intelligence research thrust. The Vision and Language research thrust will focus on what vision can deliver and how, and the

Social Intelligence research thrust will focus on how representations of social knowledge incorporate visual information to make social inferences. All research in this direction will take part Theoretical Frameworks for Intelligence research thrust regarding invariant recognition and probabilistic modeling and inference. Visual recognition and language understanding are two challenging tasks in artificial intelligence. To study and acquire the skills to build machine learning and deep learning models that can reason about images and text is very much necessary.

Research at the intersection of vision and language includes generating image descriptions using natural language, visual question answering, image retrieval using complex text queries, learning from weakly supervised text, aligning images and text in large data collections, generating images from textual descriptions, video and language, and other related domains. On the technical side, models including bag-of-words, n-gram language models, neural language models, probabilistic graphical models (PGMs), recurrent neural networks (RNNs), long-short term memory networks (LSTMs), convolutional neural networks (Convnets), and memory networks are part and parcel of language and vision research.

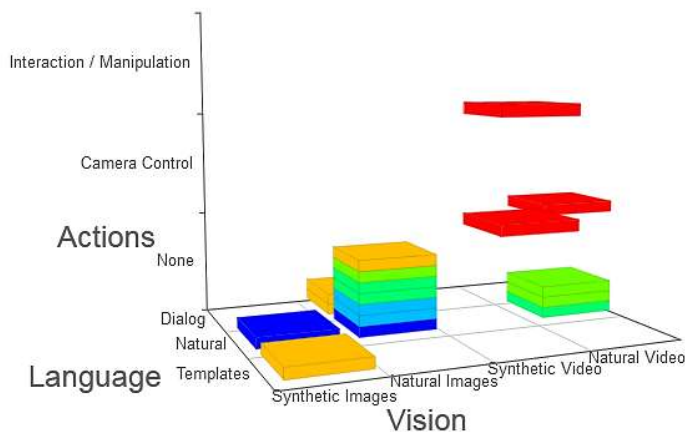


Fig. 5.20: Language, Vision, and Actions

Language and vision fetches its custom into the following applications;

(1) Human Language Learning: It is a computational framework for modeling language typology and understanding its role in second language acquisition. This method studies the cognitive and linguistic characteristics of cross-linguistic structure transfer by investigating the relations amid the speakers' native language properties and their usage patterns and mistakes in English.

(2) Hierarchical Motion Perception: Scenes filled with moving objects are often hierarchically organized, the motion of a migrating goose is nested within the flight pattern of its flock, and the motion of a car is nested within the traffic pattern of other cars on the road, the motion of body parts are nested in the motion of the body. Humans perceive hierarchical structure even in stimuli with two or three moving dots.

An influential theory of hierarchical motion perception holds that the visual system performs a “vector analysis” of moving objects, decomposing them into common and relative motions. However, this theory does not specify how to resolve ambiguity when a scene admits more than one vector analysis. This might be explored by utilizing a Bayesian theory of vector analysis. The theory takes a step towards understanding how moving scenes are parsed into objects.

(3) Multi-Sentence Event Recognition: Existing approaches to labeling images and videos with natural-language sentences generate either one sentence or a collection of unrelated sentences. Humans, however, produce a coherent set of sentences, which reference each other and describe the salient activities and relationships being shown. Understanding human intelligence can be devised with new approaches that can reproduce this behavior. The goal is to yield rich descriptions of sequences of events, static and changing spatial relationships, properties of objects, and other features.

(4) Grounded Language Acquisition and Answering: Children learn to tag what they see through visual observation while overhearing incomplete linguistic descriptions of events and properties of objects. Earlier progress on the problem of learning the meaning of words from visual observation is now extended in several ways. This prior work required that the metaphors be unambiguous, meaning that they have a single syntactic parse. Learning from ambiguous descriptions: simultaneously learning the meanings of the words while discovering the correct parse of the sentence to simultaneously acquire a lexicon and a parser through visual observation.

(5) Intentions and Goals in Human Interactions: Constructing detectors might determine gaze direction in 3D and to put on these detectors to model human-human and human-object interactions, to predict the intentions and goals of the agents by utilizing their direction of gaze, head pose, body pose, and the spatial relations between agents and objects.

(6) Minimal Visual Processing: To study human visual processing using image patches that are minimal recognizable configurations of the larger images. Although they represent only a small portion of the image, the majority of people can recognize these images, while any cropping or down-sampling severely hurts recognition performance.

5.6 WHAT ARE SMART MIRRORS?

A Smart Mirror is like any other Mirror but it is manufactured with a lot of Hardware-Technology like temperature sensors, LED for illumination, LCD Display for information, internet connection, touch capabilities, and a brain that is an internet of things Operating System. The first time you see a Smart Mirror in person you will be amazed on the beautiful technology it is, the mirrors have been with us for almost 200 hundred years and it has been a luxury piece of decoration, now add it all of this hardware technology and you have one of the most beautiful products with high potential. Imagine waking up, going to your bathroom to get ready to conquer the day and you look into your mirror and you receive all the information you need to start your day, like weather forecast, emails, calendar, estimated traffic to your destination and endless possibilities. All of this without moving a finger, that's the purpose of technology, to make our life easier and focus on what it's really important.

Besides our homes, Smart Mirrors can be used in various businesses like clothing shops, beauty shops, barber shops, kiosk mall, and many more. But it isn't all about the interactive "kiosk" type; it's more about the Artificial Intelligence it can bring to the world, telling you all the info you need depending on the place you are. Imagine going to your Beauty Shop to get a haircut, everything is normal, everything looks the same but they have smart mirrors installed instead of normal mirrors, now instead of waiting in your chair or look into old magazines you can browse a book of haircuts in the mirror in front of you, you can watch your favorite videos, read a book, watch movies or TV series right in the mirror in front of you, all of this sitting in your chair and with a remote control. This is a game changer for most of the industries that have mirrors installed and are a crucial part in their business.

Smart Mirrors and Computer Vision

A virtual or smart mirror is a two-way mirror with an electronic display behind the glass. The display can show the viewer different kinds of information in the form of widgets, such as weather, time, date, and news updates. Virtual mirrors may also be called magic mirrors, interactive mirrors, and digital mirrors. Virtual mirrors are usually fitted with computer vision technology, which is according to the British Machine Vision Association and Society for Pattern Recognition, is the science that aims to give machines or computers the ability to analyze visual patterns — much like how humans see. The technology uses algorithms to collect, analyze and make meaningful inferences from data from one or multiple images.

In retail fashion, virtual mirrors possibly will combine sensors, displays, and computer vision-equipped cameras to offer shoppers clothing references based on the data collected as well as fashion trends. The potential benefits of virtual mirrors in retail stores could include enhancement of brand experiences as well as an ease of sales associate workloads.



Fig. 5.21: Smart Mirrors

Research from Cisco suggests that these technologies also benefit shoppers as they can view several outfits without waiting in line for the fitting area. In this section, we explore how virtual mirrors and computer vision claim to offer new business use cases and how they could change the industry landscape. The technologies we discovered in this space seem to fit broadly into four major categories:

- 📱 Virtual Fitting Rooms
- 📱 Fashion Assistant
- 📱 In-Store Navigation
- 📱 Augmented Reality

(1) Virtual Fitting Rooms

Virtual mirrors simulate the process of trying on clothes in a retail store, but still offer the possibility of a clothing purchase. These technologies use a computer vision-enabled camera or scanner that captures the shopper's shape and size, which are then reflected as a virtual mannequin on the screen display to allow shoppers to see how the clothes will fit before deciding on a purchase.

Top Plus: Along with apparel, virtual mirrors can also be used in fitting and choosing eyewear. Top Plus is a Chengdu-based computer vision startup which offers Top Glasses. They say this is a software development kit for eyeglass and contact lens retailers who want to add a try-on experience to their eCommerce platforms or mobile apps. To use this mobile application, a retailer must first take a short video of the shopper's face, making sure to get shots of the front, right and left sides. As the computer vision algorithms capture the shape and measurements of the face on the recorded video, the application will display eyewear from the product catalog at the bottom of the screen that will fit the shopper's face. When the user taps a product, it will

be superimposed on the shopper's face. The user can keep swiping through and tapping the products until they select a product they like. The company claims that the application uses MagicBar, a camera module with depth-sensing and virtual simultaneous localization and mapping (VSLAM) algorithms.

The company says this enables tablets or mobile phones to sense the environment and build a 3D map of the face in real time. The company claims to serve Qualcomm, Sunny Group, Tmall, ThunderSoft, ZeroTech and Kedo.com as clients for its other computer vision technologies but did not reveal clients for its TopGlasses application. The company recently announced the release of its Vision of Me (Voome) software development kit, which developers can download without cost from Voome.cn. Voome is a face alignment and tracking algorithm engine that can be unified into gaming, live streaming, marketing and eCommerce applications. The company was recently invited to join the Nvidia Inception Program, which aims to support AI and deep learning startups in product development, prototyping, and deployment. It is not clear if the company joined the program.

Fitnect: Fitnect claims to be an augmented reality 3D fitting room platform that uses an external camera powered by Microsoft's Kinect AI software. The application features depth and infrared sensors that reside in Azure, Microsoft's AI Cloud service. Previously used in Xbox gaming technology, Kinect's fourth-generation version combines depth-sensing and AI abilities, which Fitnect says allows near and far objects to be captured and projected onto the display clearly. Fitnect is connected to a database of 3D images of clothes and accessories hosted on the Azure remote cloud servers. Shoppers can search and select clothes and accessories from a retailer's catalog with various hand gestures. They can also gesture in the direction of a specific item to it on virtually. If the user likes the look, Fitnect says shoppers can take a snapshot and share the image using the built-in social sharing tools. The system also features real-time cloth physics, which the company claims can show what the clothing item looks like when a user moves. It also allows full-body motion capture and can be embedded in websites using a browser plugin. The Fitnect mirrors have been used at events such as Elle Fashion Show last April 2018 as well as in Aggreko and Expo Noivas in Brazil. It is also installed in stores including Bilshirt and La Praille in Geneva.

Find Mine: Find Mine claims to offer virtual fitting rooms as part of an array of retail marketing solutions that include AI-driven eCommerce, mobile, in-store and personalized email services. The in-store virtual fitting room is rooted in the flagship "Complete the Look" recommendation technology, which the company claims creates complete outfits around one product from the curated catalog of clothing and accessories. For instance, if a female shopper walks into the fitting room wearing a coat. The virtual mirror reportedly recognizes the coat and recommends a matching pair of shoes, top, bag, jewelry, scarf and other items to

complement the look, according to FindMine. If the shopper says they like the look, they can tap on the mirror and associates will collect the items from inventory and deliver them to the shopper in the fitting room.

According to the company, image processing plays a key role in analyzing the products by color, pattern, titles, description, price, gender targets, and other parameters. Along with its virtual mirror, FindMine also offers a recommendation engine which retailers can integrate into their eCommerce site. Among the company's clients are brands such as Adidas and John Varvatos. A case study of recommendation engine use with the latter showed that deploying FindMine in the eCommerce site for a pilot phase resulted in a 74-percent increase in the average amount a customer spends per order on a website or mobile app.

The pilot also resulted in an 83-percent increase in conversation rate, 107-percent increase in time on site, and a 6.5-percent increase in revenue per customer. No other figures were provided in the deck. The company has not made other case studies available specifically about its virtual mirror but claims Adidas and Perry Ellis as among its clients. The company also recently partnered with Perry Ellis International to build an automated Complete-the-Look solution for its eCommerce site. According to FindMine, the partnership also tasks the company in building an application that will facilitate outfit creation and provide better customer experiences for Perry Ellis's brands such as Original Penguin, Callaway, Cubavera, and Laundry by Shelli Segal.

(2) In-Home Fashion Assistant

Echo Look: Another popular product is Amazon's Echo Look, a spinoff from the Echo home speaker that is equipped with the voice of Amazon's virtual assistant Alexa. Echo Look comes with the Style Check computer vision technology. Combined with Alexa's AI capabilities, the application then analyzes and judges if the outfit looks good on the wearer. Over time, Amazon says machine learning enables the device to compare two outfits and judge which looks better on the user. Amazon says the algorithm will continue to improve based on input from users and the Amazon style team. At the moment, the Echo Look speaker is for home use and is priced at \$200. It is not clear if Amazon intends to develop and market a version for retail businesses. There were no case studies available on retailers who might be using the product to market their own clothing. Because Amazon has made this product specifically for home shopping, we are unaware of retail stores using it for their specific clothing.

Sense MI: Sense My SA claims to offer a style and home assistant, virtual fitting room, and smart home speaker all in one device. The device, called Sense MI looks like a projector with a tablet screen. To activate the device, the user must look at the screen and unlock it. A computer-vision camera lock is also able to recognize and distinguish a specific user's face,

according to the company. At a user's home, SenseMySA claims its device projects images of outfits owned by the user, or wardrobes that can be purchased from the company's retail partners, onto the smart TV screen or a nearby wall. The clothes can also be projected on to an image of the user. To begin using the application for fitting, a user can take photos of their wardrobe with the device and store them in a gallery. The user will then be prompted to stand in front of the machine as it scans a full body photo. From there, the user can place their own clothes or a retailer's clothes on to their body in the photo.

The machine, which can connect to WiFi, is able to track the user's wardrobe and is capable of recommending outfit suggestions. The company says can also show a user different clothes based on an occasion, such as a wedding, or upcoming weather. The AI-powered virtual assistant is also embedded with voice recognition technology, which allows it to understand and respond to the user's navigation commands. The in-store retail version of this application projects clothing onto a larger virtual mirror where shoppers can choose from the store's catalog. The application recognizes swiping hand gestures so that the user waves his or her hand to view and choose other outfits. The company claims that the in-store virtual mirror's face recognition capabilities will save the shopper's likeness to remember users for the next time they shop. The company also claims its system saves the shopper's try-on and shopping history to remember the specific shopper's preferences. To date, SenseMi has been installed at the Sharjah Tourism Booth in Gitex, Carrefour Hypermarkets, Arabic Kandora Store in Dubai, Abu Dhabi Culture and Arts Gallery, and Avirate Fashion Stores. No case studies were available on the product's use.

(3) In-Store Navigation

These applications claim that studying customer behavior helps retailers understand the various factors that influence the buying decision. Customer behavior could include where in the store they go, the route they take, how much time they spend inside, among others. Understanding the behavior enables store owners to respond to customer needs, and potentially increases the chance of customers buying a product.

Lowe's: Lowe's uses augmented reality combined with geolocation technology for their mobile In-Store Navigation app; the in-house created application is powered by Google Tango, a computer vision technology that uses mobile devices to determine the user's position relative to their surroundings. This technology is used to create applications for indoor navigation, 3D mapping, physical space measurement, environmental recognition, augmented reality, and windows into a virtual world. To use the navigation software, the customer must download Tango app on their smartphone and create a list of the items to buy in the app. The app makes available product reviews and information, enabling customers to make informed purchasing

decisions. From there, the application will direct the customer to the products inside the store, following turn-by-turn directions from the application. The application allows store associates to spend more time advising customers on home improvement projects.

Walmart: Another retailer that is using computer vision in-store is Walmart, which recently announced that it is testing shelf-scanning robots developed by Bossa Nova Robotics in 50 of its stores. The robots are equipped with computer vision to navigate the aisles inside the stores. As the robot moves along the aisles, using computer vision to scan the shelves and capture images as a way of collecting data, which the algorithms will analyze and interpret and determine out-of-stock items, missed price changes on products, and missing labels: In a case study, Walmart describes the in-store robots as standing about two-feet tall and fitted with six-foot arms that can be extended above the robot's body. The arm is fitted with computer-vision cameras that capture and detect data about our products and labels on the shelves. In addition, the robot has light detection and ranging (LiDAR) sensors, a remote sensing method that uses laser light to measure distances to the earth. This technology enables the robot to be aware of its surroundings, allowing it to navigate past poles, customers, associates, and other obstacles that might be on the aisles.

The data from the robot is compressed and processed to analyze the image for discrepancies against data it has been trained with. The robot then feeds this data back to store associates, who will take action to restock shelves, provide correct labels and prices, or do the needed action. In the study, Walmart claims that the robot will serve as the eyes of the employees and give them more time to care for customers. As of March 2018, Walmart reports that the robots have covered more than 620 miles in-store without mishap, and identified tens of thousands of out of stocks. According to the company, the data collected by the robots will be integrated to train other AI-driven store applications, to reduce the time spent by store employees on manual, repetitive tasks, and enable them to focus on helping customers.

(4) Measuring Foot Traffic

Density: Density developed a camera that measures the number of people and manages entrances and exits through a door. Density claims it uses a mix of computer vision and deep learning algorithms and depth-sensing technology to anonymously measure the number of people who enter and exit through a door. The company says its algorithms filter the noise from the surroundings such as chatter, squeaking doors, or footsteps while capturing the images. The company explains that the quality of the images captured is rendered in gray to maintain people's privacy. The link below shows a sample of the images captured by the camera. This image is also featured on the company website. Unlike a security camera, the sensors can't determine the gender or ethnicity of the people it tracks.

Because of this, the Density claims its hardware can be installed in places where a video camera is not allowed such as public bathrooms, churches, secure office spaces, schools, and dressing rooms, the company says. The technology can handle complex human behavior such as group collisions, bi-directional movement, lines and lingering. Retailers can integrate the application into their pre-existing software solutions, the company website claims. No case studies are available but the company says its customers include homeless shelters, Ivy League colleges, theme parks realtors, hospices, businesses examining the use of office space. Density was also piloted by a startup mobile app company called Work from, The Portland-based startup, which aimed to help remote workers find coffee shops with Wi-Fi, used Density to determine when seating might be available based on door traffic in partnering coffee shop locations. It is not clear if the pilot was successful. The hardware can be installed above a door or plugged into a wall and tracks movement frame-by-frame with two infrared beams that bounce off of the floor. Use of the hardware is free, but the company says it sells the data collected. The data is sent to a cloud-hosted server, where it is analyzed. The company has raised \$16.2 million in three rounds of funding; the most recent of which totaled \$12 million in June 2018.

(5) Computer Vision for Play

Playfusion: Playfusion created a computer vision-based game for the former Toys R Us, which announced the AR application at its stores in October 2017. Augmented reality (AR) in computer vision is the process by which computer-generated content is overlaid onto images or video streams. Users could download the game, called Play Chaser on their smartphone or tablets. The players then use the device to scan signages throughout the store to activate different AR games or activities. Playfusion's game technology features simultaneous localization and mapping, which plots the environment in relation to the position of a camera that scans that environment. It also uses object tracking, a technology that follows a moving object such as an animated character, with the aid of the computer vision system. Combined with image recognition, these technologies provide the basis of an augmented reality experience. Playfusion is responsible for AR games such as Lightseekers and Warhammer Age of Sigmar.

In the clusters that we discussed above, the aim of the computer vision and deep learning technologies seems to be to enhance the shopping experience by providing convenience or increasing engagement. Virtual mirrors, such as Fitnect, claim to expedite the process of fitting clothes, which could also speed up purchase decisions. In the case of in-store robots at Lowe's and Walmart, the artificial intelligence privileges to help customers navigate through the aisles or ensure shelves are filled with correctly priced and labeled products. While most retail stores use mirrors to help shoppers choose clothing and fashion accessories, the technology can be used in other different ways. For instance, Lululemon reportedly uses virtual mirrors to offer

information about the community activities. Accordingly, flagship stores in Manhattan and Vancouver use the mirrors as a directory to display community events such as film screenings, places to exercise, and information about healthy living.

5.7 WHAT IS MERGED REALITY?

Merged Reality is the combining of real and virtual worlds to allow physical and digital to co-exist and interact in real time. Augmented reality is a form of merged reality, allowing digital layers to be presented on top of physical layers, often using some kind of display technology like a headset, video screens, tablet or a user's mobile phone. Its history is hard to pin down, but some cite the first fully immersive mixed reality system as the 'Virtual Fixtures' platform developed by the US Air Force in 1992 to allow human users to control robots in real-world environments. These included real physical objects and 3D virtual overlays called "fixtures". The virtual sensory overlays could be used to show physically realistic structures which when shown to the user would be perceived as fully present in the real workspace environment.

Because 3D graphics were too slow in the early 1990s to present a photorealistic and spatially-registered augmented reality, Virtual Fixtures used two real physical robots, controlled by a full upper-body exoskeleton worn by the user. To create the immersive experience for the user, a unique optics configuration was employed that involved a pair of binocular magnifiers aligned so that the user's view of the robot arms were brought forward so as to appear registered in the exact location of the user's real physical arms. With virtual reality whilst the likes of Oculus and Magic Leap are working hard to shift units, the truth remains that the hardware itself is still bulky, can be cumbersome and blocks the user's eyes and social interaction. Apple still remains noticeably absent from the VR market. Its consumer adoption has mainly been for gamers looking for a more immersive experience.

We can think of a number of organizations who have tried virtual reality (using Google Glass or equivalent) but in field service organizations the tipping point is most often at the point of balance between the cost and adoption rate of the hardware and managing the artificiality of the VR headset itself. The level of fidelity and context sensitive modelling that is often required can make actual deployment at scale cost prohibitive.

We see the path for augmented reality and mixed reality as somewhat different. The truth is it's all about numbers. Whilst there are an estimated 82 million VR units in the world, assuming augmented reality capability is currently built out from the smartphone, AR units could hit over three billion. That's an awful lot more consumers and business that already have the ability to access augmented reality in their pockets! While current 4G networks can power current AR applications, more advanced and pervasive adoption of AR will require the greater speed and bandwidth of 5G networks and be a business case in its own right.

The Ease of Merged Reality

Merged reality gets round a lot of the challenges of virtual reality (requiring headsets, hardware and 3D modelling) and augmented reality (at times limited in its business application). Done right, Merged Reality can allow service organizations to fix customer problems remotely, by being able to see the problem their customer or field service technician is having, and virtually reaching out and touching the problem, as though they are standing right next to them.

Merged reality technology can be delivered from the mobile device using real-time video calls. This ability to be “hands-on” from a remote location accelerates and simplifies the way knowledge is exchanged and problems are solved. Merged reality adds missing visual cues, gestures, and non-verbal communication methods to any session. It can allow the hand of the expert giving help to appear on top of a colleague's image. Telestrating, freezing images, using hand gestures and even adding real objects into the scene can get the point across in an easy and intuitive manner.

How Can Merged Reality Help?

Typically service organizations we work with, want to make their field operation better, more efficient, more cost effective, more productive and with better knowledge sharing between staff. Merged reality fits well with these goals. More and more companies are looking at merged reality to help them reduce costs, improve productivity, maximize product uptime, and drive stickiness between their company and their customers. Using existing mobile devices (iOS, Android) or a web-browser, experts can now visually collaborate as though they were working side-by-side.

Merged reality blends two real-time video streams, that of a remote expert and an onsite expert, into a collaborative environment. Research indicates that nonverbal cues are 430% more effective than verbal cues and those nonverbal cues make verbal/nonverbal communication combination up to 10 times more active. Instant, accurate communication augmented by mobile merged reality and virtual presence can have a dramatic impact on service operations and proactively, efficiently and effectively attack hard costs that plague the efficiency and effectiveness of operations and customer interactions.

There are many customer success stories from merged reality. For example a large field service team who were able to fix a problem remotely leading to saving of £6,000 in parts and labor. A manufacturing company in the UK was able to reduce field service costs by £11,500 in the first month of using merged reality. A service organization was able to save between 5-10 truck rolls per week (at an average cost of £90 per truck roll), leading to monthly savings of up to £3,700 per month.

Welcome To Merged Reality

It is a new technique of experiencing physical and virtual interactions and environs through a suite of next-generation sensing and digitizing skills. Merged reality delivers virtual-world experiences more dynamically and naturally than ever before and makes experiences impossible in the real world now possible. Pick up your real-world tennis racket in your living room and step virtually onto the court at Wimbledon. In doing so, forgo the cost and creep factor of installing high-end sensors in every corner of your living room normally needed to scan your environment. Instead just lean fully into practicing your back swing bringing both your hand and your racket into the virtual field of play. Surge backward or forward, reach up or down, sway to the left or right, enjoy fully 6 degrees of total freedom in your movement.

And take that virtual swing, without hitting that very real-world, and very expensive, table lamp several feet away from you. Be the ultimate concert master, fully unplugged. Cut the cord and choose your own amazing musical experience. Play a virtual piano with one hand and a cello with your other hand. That's right: play two instruments at the same time, an impossible real-world feat by even the most talented musicians. Reinvent the world around you. Go from where you aren't to where you could never be. Do so without cumbersome controllers, but with your own two hands in the field of your virtual vision. Plan your virtual visit to the Sistine Chapel while never leaving the office.

Get up from your desk confidentially immersing yourself in the digitized splendor of Michelangelo, looking upward to get a better view without walking into that real-world and potentially painful, conference table right in front of you. In other words, zone out in virtual spaces, without stumbling in the real world.



Fig. 5.22: Merged Reality

Experience a sporting event, a concert or a film scene from any point of view and from any position. Walk onto the field and right into the goalie box, or join the huddle, looking over the quarterback's shoulder to see the calls before they're played. Stand on the stage with your favorite band and feel the love of the fans from their point of view. Choose your experience and navigate it. It's all up to you. The virtual world doesn't need to be virtual; it can be real. All of this and even more is possible with merged reality. And here's why: Merged reality hinges on five technological advances soon to be available to developers, makers and inventors.

(1) Six Degrees of Mobility: Merged reality dramatically increases freedom in movement in 3-D virtual spaces with real-world awareness. Said another way: It's about sensing technologies to help make sure that while you're experiencing your virtual world, you're not colliding into real-world stuff.

(2) Integrated Tracking: Merged reality reduces the need for elaborate and costly sets of external sensors that translate real-world environments into digital representations. It does so by attaching sensing technologies, such as Intel RealSense cameras, to your headset and other smart and connected devices. The result will be even greater proliferation, availability and portability of VR and AR experiences.

(3) More Natural Manipulation: Merged reality is about more natural ways of interacting with and manipulating virtual environments by liberating you from the controllers and the nun chucks of today's VR systems by immersing your hands, your real-life hands into your simulated experiences thanks to readily available new sensing technologies.

(4) Go Untethered: Merged reality "cuts the VR cord," allowing you to go fully untethered from consoles and controllers. Sure, the most aggressive, high-end gamers who value the low latency of tethered systems will continue to enjoy those systems, and their systems will continue to advance and improve. But for the rest of us, we now have the choice to experience our virtual worlds across larger spaces without pesky cords. No more being jolted out of your VR experience because you have reached the end of your cord.

(5) Digitized Real-World Content: Merged reality goes beyond computer-generated content. It's about bringing the real physical world into the digital world real time and reimagining it with computing power. Rather than a single point of view, advancements like Intel's Replay 360-degree expertise use encoded video and advanced composition algorithms captured from an array of cameras to digitize whole playing fields and venues from any position, from any point of view, and with an enhanced ability to interact. This is a game changer for the entire category of virtual and augmented reality. You choose the experience, and you get to navigate real-world content in new ways.

To provide tomorrow's truthfully immersive compute experiences today; developers, makers and inventors need these more advanced hardware and software technologies of merged reality. That's why Intel is partnering with other leading technology companies to share technologies such as Intel's RealSense, Replay, and the first of its kind Project Alloy Head-Mounted Device (HMD). When it comes down to it, why would you choose to limit your reality, when the new reality is that you can have it all? With increased mobility, integrated tracking and highly sensing human motion capabilities, merged reality will profoundly impact the way we work, how we are entertained and how we communicate.

Unveiled during the opening keynote of the Intel Developer Forum in San Francisco on Tuesday, August 16, 2016, Project Alloy is an all-in-one virtual reality solution leveraging Intel RealSense technology. Project Alloy will be offered as an open platform in 2017. So you see, regardless of whether you're seeking to experience augmented reality or virtual reality, your freedom in the world of your choice depends on how you merge your realities and the technologies that enable you to cross over from the physical to the virtual, and the virtual to the physical. We're rapidly moving toward a world where the boundary between the digital and the physical is eroding, and merging in new exciting ways.

How Will It Affect Your Future?

Two new examples of "merged reality" expertise have been making waves, but what does it actually mean? According to Intel and Microsoft, who, at the Intel Developer Forum in San Francisco, discussed two new advances in merged reality. The first is Project Alloy, a wireless headset that allows you to bring real objects into the virtual world using 3D cameras. The second is Windows Holographic Shell, and it does the reverse, allowing virtual computer screens, objects, and people into the real world. Mixed reality is when the real and virtual world is merged and objects from both can interact. So, for example, if you were playing a virtual video game, you could pick up your nearby real-life water bottle and hit Dr.Eggman over the head with it. Specifically, Intel's Project Alloy uses mixed reality to allow people to use their real hands in the virtual world.

Before this point, users had to hold controllers that represented their hands. Now, the headset's RealSense cameras will detect your finger movements and allow you to manipulate virtual objects. It is quite an exciting development, the headset will also allow you to see your friends in the virtual world, and keep the confines of the virtual world in the real world space around you. In short, this means less walking-into-walls than in a traditional virtual world. The other development, Windows Holographic, uses the HoloLens headset to let you interact with holographic screens that display things like your emails and calendars. The HoloLens is also one single operating system for virtual reality, mixed reality, and augmented reality. Virtual

reality is the umbrella term for all these magical artificial worlds that are popping up around us. Most commonly, users plonk on a headset and get a 360° view of an immersive fake world. Naturally, it's thus far predominantly been used for video games.

Augmented reality, of which the most famous example is Pokemon Go, is when virtual stuff is added to your real world environment. The virtual reality element, i.e. Pikachu is overlaid onto the real world. This has been used for sporting events and tourism, as information about the game or place can be overlaid as you look around. Although this is similar to merged reality, Microsoft and Intel's new projects go further as they do not require sensors and are wireless, so more seamlessly merge the real and virtual worlds. Windows Holographic was unveiled last year, but the latest announcement revealed that it will be soaring into any Windows 10 user's life next year. Project Alloy's consumer release date is set for 2018, and for the time being, there is no word on the price. Although these are big advances, it could take years for the technology to become commonplace. In practice, although it sounds cool, there might not be much demand or it may be considered socially unacceptable (remember Google Glass).

Theoretically in a few years, you could attend a sporting event or concert without leaving your home. Ok, sure, TV has let you do this for a fair few decades, but Intel's CEO Brian Krzanich explains the difference in a Walk onto the field and right into the goalie box, or joins the huddle, looking over the quarterback's shoulder to see the calls before they're played. Stand on the stage with your favorite band and feel the love of the fans from their point of view. Choose your experience and navigate it. It's all up to you. The virtual world doesn't need to be virtual, it can be real. Krzanich also theorizes that you could practice tennis without visiting a court, go on a virtual holiday, and for some reason, if you were so inclined, play the piano and cello at the same time.

Intel's announcement also features a man enjoying a virtual party under the premise: "What if virtual reality felt less virtual and more real?" The idea is that people could come together to party while physically being in different parts of the world, and that you could physically walk around the party without the need for cords and controls. Sounds great, right? Except then they demonstrate that you can pause the party and throw a waiter in a pool. For gamers, mixed reality will also potentially mean a better experience, whereby users can be completely immersed in fictional worlds by using their own hands and body instead of controllers. In the world of work, Windows Holographic could let you have meetings with people where you're physically remote but virtually in the same place as shown in the below illustrations;

(1) Socializing in Virtual Reality: Another unique aspect of video viewing in VR is the ability to combine it with social networking. Today, it's possible to watch media with friends in VR

application rooms like Bigscreen and AltspaceVR. The money here is going to be in the concert industry, can watch his favorite band with integration between a fully immersive VR experience and 360 degree streaming camera, with hundreds of others from around the globe. Live streaming of concerts and sporting events could potentially be a huge application for VR in the media space. Sixty percent of early adopters expect that interactive music concerts and events in VR will be mainstream in the next three years. Ironically, there are simultaneous positives and negatives to VR. On one hand we meet a lot more people in the virtual environment, while on the other we may be isolated from people in our physical environment due to the nature of the current VR headsets. While 51 percent of early adopters believe video viewing in VR will be popular since it can combine social networking with video, 38 percent also believe video viewing in VR will be inspiring as it can only be done alone.

(2) Virtual Classrooms: It is supposed that VR and AR could drastically transform education and learning. Sixty-one percent of early adopters believe that VR could be used for education and social purposes. In fact, 49 percent of consumers who are planning to use a headset, as well as 25 percent of the laggards also believe so. Imitating physical classrooms, textbooks, field visits/experiments, and learning through demonstrations and interactivity are some of the ways that VR could transform the field of education and learning. A quarter of early adopters believe that, within a year, students will be able to gain practical experiences without actually leaving the classroom. Laggards are open to using VR for education and learning. In the context of education, they are mostly open to specialized and technical trainings.

(3) AR-Enabled Adventure: The other domains are not too far behind. Travel and tourism could go through another transformation. Twenty-five percent of early adopters believe that in a year from now, we will be exploring destinations through AR-enabled information and maps overlaid onto our physical environment. VR will also give rise to a new concept of tourism and adventure; 23 percent of early adopters believe that within a year we will go on adventure trips virtually with others. Consumers expect VR and AR to have an impact on daily activities and experiences across other domains as well. Twenty-six percent of early adopters feel that within a year we will be shopping virtually for large items like furniture, and even experiencing them in our homes before purchase.

(4) Stationary Motion Sickness: VR can leave users feeling sick because of latency issues. The delay between the movement in the virtual world and actual movement in the user's physical body may leave them feeling nauseous. As per the 10 Hot Consumer Trends of 2018, 1 in 3 consumers wanted motion sickness pills for use with VR and AR for this reason. Almost a fifth of the early adopters find sickness to be a problem in VR. Users also complain of a poor visual experience due to low resolution. Hence VR can be physically draining and restricting for users in its current state. Non-users also have similar concerns with VR; though their biggest

turn-off is that it is expensive. Consumers also expect VR to integrate haptic feedback (that is the sense of touch), more than half of early adopters believe that haptic feedback is an important element that is missing from VR today and must be added in the future. While 60 percent think this will enable new possibilities in video, simulations, gaming and shopping, another 56 percent believe that adding haptic feedback will make VR more valuable for specialist industries, such as healthcare and therapy, and for training.

(5) Smartphone-Enabled VR Headsets: While the smartphone versions of VR headsets provide mobility and are easier to set up, they have their own challenges more than a third of mobile VR users complain that the smartphone heats up and consumes a lot of battery power. Since you look awkward when using the headset and cannot walk freely, due to being cut off from your surroundings, as many as 68 percent of mobile VR users only use their headsets at home. A quarter of mobile VR users have in fact reduced their overall usage of VR since they started using it.

Do We Want A Virtual Society?

These are only the physiological challenges associated with VR, but could VR also have a negative impact on society? Almost a third of early adopters agree that VR is socially risky and people may start disliking their physical reality and social relationships. VR, by virtue of being realistic and immersive, could also be used negatively to manipulate the human brain. VR could have a negative influence as well because you could control or manipulate someone's thoughts with something immersive like this, like by showing them some negative visuals. For instance, a violent video game could start feeling like murder something that VR developers are already showing concern over. But when the technological challenges are overcome and headsets are no longer bulky and restricting, so that consumers can transition between the virtual and physical worlds effortlessly, the social risks of escapism and individualism may cease to exist. In such a world, the users wouldn't necessarily know if they are in the virtual or physical world. VR today is fraught with challenges.

In its current state, VR only mixes reality but does not merge with it. The consumer research indicates that one way of overcoming this paradox, and moving towards a merged reality, is to integrate VR within AR. Consumers perceive AR to avoid a number of issues that hold VR back from becoming mainstream, especially when it comes to everyday practicality they say they can still interact with the real world, with their friends and family, rather than locking themselves away in a virtual space. It is for these reasons that early adopters of VR also believe that AR has greater potential to go mainstream. However, these early adopters also believe that as technology evolves, AR headsets will subsume VR as part of the experience. An AR experience does not necessarily require a headset.

The AR game Pokémon Go merely uses the smartphone screen and camera to deliver the experience. Likewise, Snapchat Lenses uses the camera to overlay features and expressions on the user's face. Facebook recently announced that they will be developing smartphone cameras as the first mainstream augmented reality platform. Similarly, when VR becomes part of the AR experience, our interviews indicate people expect a pair of AR glasses with in-built VR capabilities rather than a bulky headset that blocks off the real world. They will then be able to enjoy a movie on a VR screen and (see and) eat their popcorn as well.

5G Expectations

Realities will not merge if the user is tethered to a computer or cut off from physical reality. Early adopters of VR expect next-generation networks like 5G to play a central role here. Thirty-six percent have expectations on 5G to provide VR mobility through a stable, fast and high-bandwidth network. Thirty percent of early adopters also expect 5G to enable tethered headsets to become wireless. Shared experiences, social video viewing, tourism and live-streaming are all everyday social activities that early adopters expect to hit mainstream in VR within the next one to three years. But social VR in this sense could be a resource hog and may require a network with less lag and higher bandwidth. Low latency is crucial to making social VR work, as it is incredibly difficult to maintain a conversation if there are delays in responses and reactions. Thirty one percent of early adopters expect 5G to enable more shared experiences through a network with less lag.

Improving Merged Reality

The other challenge today is whether our bodies will accept virtual experiences given the nausea and sickness that it could cause. In the qualitative tests, a few were almost immune to nausea, and a few would always feel nauseous. But for the majority, the amount of lag determines if they feel fine or feel sick. A network that minimizes lag in responses could address this issue; believe more than a quarter of the early adopters surveyed. Almost a third of early adopters also believe that next-generation networks like 5G could improve the efficiency of connected devices (such as smartphones) by consuming less battery power, while a similar ratio believes that 5G could make VR and AR more accessible through cheaper data plans. Increased use of AR and VR will move us towards a merged reality where users no longer separate between what is augmented, virtual and physical.

“Merged reality will allow us to transition from an internet of information to an internet of 'realities'. In a virtual reality setup, a user is totally unknown to the real world and puts a headset that displays a virtual world only. In augmented reality world, virtual objects are placed on top of the real world as a layer. Mixed reality is a kind of augmented reality where objects from both, virtual and real worlds are merged and interact.

5.8 WHAT ARE VIDEO ANALYTICS?

Video analytics software was created to help review the growing hours of surveillance video that a security guard or system manager may never have time to watch, your video surveillance system is only as useful as the incidents you can actually capture and watch, and video analytics will help you find them. Using video analytics makes your surveillance system more efficient, reduces the workload on security and management staff, and helps you capture the full value of security video by making your IP camera system more intelligent in its work. Video analytics software for security cameras is available in several forms: installed on your camera, on your NVR, or as 3rd party software you buy. Each version will do the same thing, however - monitor your videos to search for and alert you to activity.

Each video analytics solution will work a bit differently depending on the manufacturer and application. They all work in the same basic way, however when setting up the software you set up parameters to the activity the software is looking for, set up the alert notification system, and when the software detects something that meets its search criteria it alerts you. For example, many businesses use surveillance systems to detect motion in their store after hours. You can set your system to motion detection during the hours your business is closed, so if it detects motion it will send you an email a quick response that may help you react quickly to a break-in or accident.

The old idea of a video surveillance system is of a security guard sitting in a booth watching the security camera feed live, hoping to catch suspicious activity. This model relies on having a live person watching and reviewing all your video, however, which is not practical or efficient. Different security guards may have differing levels of focus or different ideas of suspicious activity. Video management software changes this system by using software to monitor your video feed around the clock, alerting you to activity so you only need to watch the cameras when something happens. This will help you best utilize your surveillance system, saving you time and effort. Video analytics can be used for:

-  Motion detection and traffic control.
-  Facial recognition and license plate reading.
-  People counting and dwell time monitoring for retail stores.
-  Recognizing long lines at checkout and sending alerts.

Video analytics software comes with a wide range of features, making them easy to fit into any surveillance system. Choose from software that supports every system, from the smallest systems to large multi-campus systems. A variety of leading manufacturers create video analytics software, including Milestone. Video Analytics uses algorithms to differentiate between object types and identify certain behavior or action in real time, providing alerts and

insights to users. When it comes to video analytics, experience is the real differentiator. The success of video analytics relies on expert utilization.

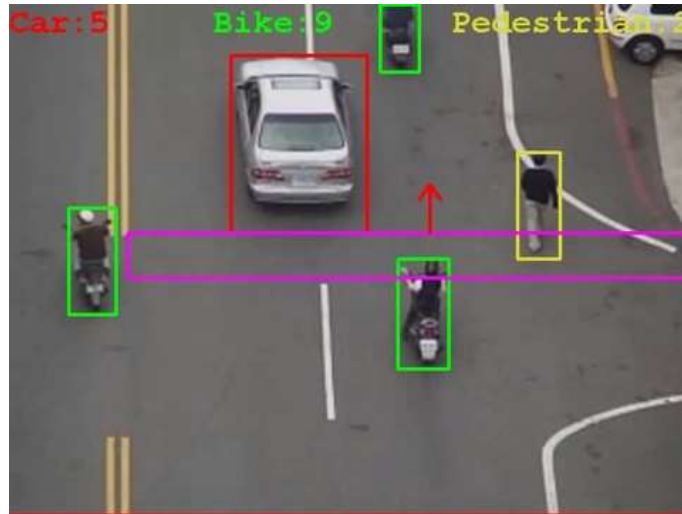


Fig. 5.23: Video Analytics in Traffic Analysis

(1) Perimeter Intrusion Detection System (PIDS): Your perimeter is often the first line of defense in keeping intruders out. In environments such as airports, seaports, and railways, where the perimeter can extend for miles it can be a security weak point. PIDS works with or without a physical fence, and can also be a part of a smart fence outline. With this system, you'll be forewarned to intrusions anywhere along your perimeter in real time, allowing you to apprehend intruders and mitigate menace.

(2) Crowd Management and People Count: Some of the world's busiest mass transit hubs to use video analytics to ensure security, safety, and a positive customer experience. Using the crowd management and people count video analytics applications will able you to achieve a measurable accuracy rate of over 90%.

(3) Counter-Flow Detection: Walking against the flow in certain locations, such as airport security checkpoints and gates, can be a sign of something much bigger. It can potentially result in terminal shutdowns, along with significant financial and security consequences. Counter-Flow Detection forewarns operators to individuals breaking airport and security regulations by walking in an unauthorized direction.

(4) Suspect Search: Suspect Search software helps you locate and track specific people in near real time, online or offline, dramatically expediting forensic video searches. When looking for

someone either because they are in danger or causing it, time is always of the essence. Reduce the time it takes you to find and track an individual from hours or days to seconds and minutes.

Video Analytics uses metadata to add sense and structure to video footage. This enables cameras to understand what they're seeing and alert if there are threats the moment they happen. It also makes it possible to retrieve the right footage from hours of stored video instantly and analyze the scene by providing different statistics so an operator can act faster and more efficiently to potential threats. Video Analytics is an important enabler of any strategy to substantially improve levels of security, as well as offering clear business advantages beyond security. Because they can provide all kinds of statistics in the form of metadata, cameras featuring built-in Video Analytics also enable a scene to be analyzed for reasons that go far beyond security.

Making it possible to make smarter business decisions, reduce costs and increase efficiency with a return on investment that can be measured through tangible business results. There is no 'one size fits all' when it comes to protecting people and property. That's why the next step with Camera Trainer. The more a video security camera can know and learn, the more accurately and application-specifically it can perform. With Video Analytics, the focus is always on making video surveillance more relevant. Consequently, the solutions also come with a range of advanced features such as:

(5) Intelligent Tracking: The Intelligent Tracking feature automatically zooms in on and follows a selected object, so you never lose track of a potential security threat, or object of interest. Tracking is also optimized to detect objects above the surface of water for use in harbor and canal surveillance, and coastline protection (Ship tracking).

(6) Video Analytics While Moving: Unlike traditional Pan-Tilt-Zoom cameras, modern cameras feature Video Analytics on the move. Intelligent tracking automatically follows a moving object and dynamically adjusts the field of view to capture optimal images, even while the camera is panning, tilting or zooming.

(7) Metadata Fusion: For complete situational awareness, metadata from the MIC IP fusion 9000i camera's built-in optical and thermal imager can be fused together, regardless of which video stream is being watched. If an event is detected but is not necessarily visible in the video stream you are watching, an alarm is triggered and you can click an overlay box to see video data related to that event. This gives you the advantage of being able to focus on 'invisible' things that require attention.

(8) Video-Based Fire Detection: Video-based fire detection system is a genuine life saver. Its unique physical detection model differentiates between disturbances and real fires with greater

accuracy. The proven camera technology and intelligent video analytics analyze footage directly in the camera, eliminating the cost of installing additional processing units.

(9) Forensic Search: Unlike other solutions that only offer searches via recorded alarms, forensic search enables you to search through material while continually changing search criteria. There's no need to set them up as initial alarm criteria, you can add new criteria at any time, giving you complete flexibility in the way you search for evidence.

(10) Tamper Detection: As you'd expect from the robust security systems, built-in tamper detection generates alarms when cameras are hooded or marked, blinded, de-focused or repositioned. With Video Analytics you can customize your tamper alarm to the exact level you demand.

(11) Geolocation: Another innovative capability is the geolocation feature. Because Video Analytics automatically generates information on an object's location, speed and trajectory as part of the metadata, you can identify the real world location of any object that is being detected and tracked. This can then be visualized in the camera's global positioning system (GPS), or in a local map coordinate system, to give you a clearer picture of any potential security threat. If technology is too complex to set up, manage and interpret, you may not capture the data you need to protect people and property, and the insights to drive business advantages.

Video Analytics: The What's and the How's Video Analytics uses mathematical algorithms to monitor, analyze and cope large volumes of video. It digitally analyzes video inputs; transforming them into intelligent data which help in taking decisions. Video Analytics can be real-time configured to track and provide alerts to specific incidents as they happen or post event retrospectively searching for incidents that have already occurred. Intelligent Video Analytics (IVA) applications can run at the centre (on servers or DVRs at the central monitoring station), built into cameras or as a combination of both. The 'IVA' is ideal to locate live analytics.

Central real-time processing can run out of steam based on the number of cameras in the network, processing power and the network bandwidth; whereas in the 'IVA', every camera has dedicated processing. With limited bandwidth on their networks can opt for an analytics, so that only information on suspicious incidents gets sent through the network; and hence, doesn't use up network bandwidth. The optimal place to locate post-processing analytics is obviously on a central server, so that recorded video can be searched many times, with different parameters. Using analytics a user can search large amounts of recorded video for possible events and then verify the results. End user companies with existing infrastructure, who want to upgrade by adding analytics to some of their existing cameras, may find integrating analytics to the central server a cost-effective and flexible solution.

Applications

Video surveillance analytics are finding numerous applications in both the public and private sectors; in government, retail, transport and financial services. These applications protect people and assets against harm and damage, ideally before events occur. Systems enabled with video analytics work on two key concepts: motion detection and pattern recognition.

Motion Detection: By examining each pixel in the frame, the video analytics software is able to pick up even the slightest movement.

Pattern Recognition: Video analytics help distinguish objects within a video frame. Specific patterns/objects can be programmed, which will be recognized within the frame. Should any change happen, i.e. object is moved, goes missing, or new object added; the software immediately recognizes it and sends out an alert.

Typical applications of Video Analytics in security and surveillance include:

-  License Plate Recognition
-  Security Access Point Monitoring
-  Perimeter Protection / Intrusion Detection
-  Abandoned Object
-  Object Removal
-  Camera Tampering

Video analytics is also gaining popularity in the retail segment for market analysis. A lot of the analytics information can be valuable for marketing teams in companies. These include:

-  Footfalls (People counting)
-  Gender counting
-  Display/shelf space effectiveness and
-  Forensics analysis of data to determine trends

Benefits of Video Analytics

Controlling and managing a video surveillance system is difficult; especially if you are working with a large number of cameras. Keeping track of everything that is going on is a hassle and requires a lot of manpower. This is not the case with video analytics. Video analytics use detailed and sophisticated algorithms to examine digital video feeds. Video images are examined pixel by pixel, missing absolutely nothing. Analytics filters can be intelligently tailored to meet specific security or business needs.

The Filters/Algorithms:

-  Direction filters with independent adjustment of acceptable direction and angle
-  Dwell filter with variable time threshold.
-  Speed filter with upper and lower speed thresholds
-  Object filters to filter out birds, small animals, blowing trash and other such clutter

Limitations of Video Analytics

While video analytics holds great promise, the jury is still out on the efficacy of the technology in the real world. The key thing to note here is that video analytics technology is still in its infancy. Current video analytics solutions do work, but in a constrained environment. For example, video analytics if secondhand for perimeter security or vehicle license plate recognition or unattended baggage in a public space, will work; if weather conditions are not extreme, the number of vehicles is not very high, or in a not so crowded location, respectively. On the other hand, if a user wishes to categorize unattended baggage in a busy railway station at peak hours, he will surely be disappointed; as the system will end up generating too many false alerts. Thus, keep in mind that the success of a video analytics implementation will be determined by the use it is put to and in situ conditions.

Explosive trends in the video analytics market have really caught the eye of business intelligent departments. This little known software is quickly picking up speed for an expansive amount of applications. From security to public safety to crowd management, video analytics have begun to see a big boom on the business world. Video analytics, or intelligent video analytics, is software that is used to monitor video streams in near real-time. While monitoring the videos, the software identifies attributes, events or patterns of specific behavior via video analysis of monitored environments. Video analysis software also generates automatic alerts and can facilitate forensic analysis of historical data to identify trends, patterns and incidents. The software enables its users to analyze, organize and share any insight gained from the data to make smarter, better decisions. It can promote enhanced coordination across and within agencies and organizations. Its applications are widespread, including monitoring vehicle patterns or violations of traffic laws, or people entering restricted areas during defined time frames. The data can then be sorted by time and date or over an extended time period to create a trend analysis.

Technical Aspects

A simple function of video analytics is motion detection with a fixed background. More technical functions can include ego motion estimation (the 3D motion of a camera within an environment) and video tracking. From here can come identification (such as a specific vehicle,

worst case scenario one in an Amber Alert) and behavior analysis. The technology used to accomplish video analytics includes sensors, cameras, image compression techniques and connectivity techniques. Video analytics come with many benefits to businesses especially in the security and public safety realms. It is often used to enhance these two sectors with comprehensive intelligence, security and investigative capabilities. The data derived from video analytics is designed to produce actionable intelligence to help enable city or infrastructure planners, law enforcement, or even managers and administrators to understand and respond to current situations. Other industries that have used video analytics applications also include education, transportation, banking and financial institutions, casinos, and gaming, and government industries.

Non-Security Reasons

There is also an increasing demand for video analytics for non-security reasons. One such reason that has really begun to catch a hold is counting and crowd management applications. This application can help end-users to measure the flow of people at certain exit and entry points within a building, facility or door all within real-time or during periodic reporting. Industries such as retail and entertainment have begun to use it for these applications. It helps in monitoring dwell time in retail stores and also people counting for concerts and more. Video analytics, however, aren't just meant for the business world. Other beneficial applications can include flame and smoke detection and even home automation.

In the Global Video Analytics Market Report of 2016, Research and Markets estimated the video analytics market to grow from the USD 1.69 billion of 2016 to USD 4.23 billion by 2021 (a nearly 21 percent growth rate). The sudden increase in popularity of the video analytics market comes from an increase in security threats and the need for intelligent surveillance.

The market is also driven by the increased use of cloud computing, the growing demand for automation, the need for infrastructural safety and for securing computing environment. Technological advances will continue to also affect market growth significantly. Especially as certain aspects such as ease of installation, HD quality video recording, enhanced user friendliness and biometric recognition technology are enhanced.

At the moment, North America is projected to have the largest hold in the market share from 2016 to 2021 as there has been a large adoption of video surveillance systems. APAC however, is expected to be the fastest-growing region in global intelligent video analytics market. This is due to the fact that this region has an increasing adoption of new technologies such as analytics, cloud, and also the development of smart city projects in countries such as Singapore, China, and India.

5.9 WHAT ARE GRAPH NEURAL NETWORKS?

Graphs and their study have received a lot of attention since ages due to their ability of representing the real world in a fashion that can be analyzed objectively. Indeed, graphs can be used to represent a lot of useful, real world datasets such as social networks, web link data, molecular structures, geographical maps, etc. Apart from these cases which have a natural structure to them, non-structured data such as images and text can also be modelled in the form of graphs in order to perform graph analysis on them. Due to the expressiveness of graphs and a tremendous increase in the available computational power in recent times, a good amount of attention has been directed towards the machine learning way of analyzing graphs, i.e. Graph Neural Networks.

Graph neural networks (GNNs) are connectionist models that capture the dependence of graphs via message passing between the nodes of graphs. They are extensions of the neural network model to capture the data represented as graphs. However, unlike the standard neural nets, GNNs maintain state information to capture the neighborhood properties of the nodes. These states of the nodes of a graph can then be used to produce output labels such as a classification of the node or an arbitrary function value computed by the node. The network endeavors to learn these encodings (hv) for each of the nodes through a mutual sharing of data among the nodes' neighbors in an iterative manner until convergence.

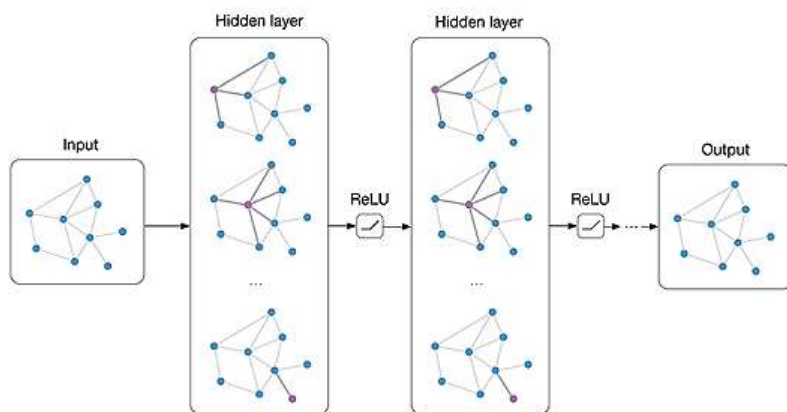


Fig. 5.24: Graph Neural Networks (GNNs)

Given the wide range of domains in which GNNs can be applied, most of them could either be classified as structural applications or as non-structural applications based on the type of data. The structural case represents data that has an inherent structure to it such as social interaction networks or molecular networks. Consequently, the non-structural scenarios represent cases where the structure in the data is not explicit such as text and images. In these non-structural

scenarios then, the general approach is to try to transform the data to a structured format and then apply GNN. There are a few other applications as well that do not fall into the purview of either of the classes mentioned above. We will discuss some interesting applications from each of these categories here.

Physical Systems

The human brain is said to aid its reasoning process through the creation of graphs learned from daily experiences of the real world. Hence great importance is being assigned to the modelling of real-world entities and their interactions as graphs. This is deemed as a first steps towards human-like artificial intelligence. By considering objects as nodes and the relations or interactions between them as edges, simple physical systems can be modelled as graphs and can be effectively analyzed using GNN.

Interaction networks can be trained to reason about the interactions of objects in a complex physical system. It can make predictions and inferences about various system properties in domains such as collision dynamics (rigid and non-rigid). It simulates these systems using object and relation centric reasoning using deep neural networks on graphs. These GNNs have been shown to predict the trajectories of different objects thousands of time steps into the future.

Visual interaction networks make use of the above interaction network to go a step further and learn about the dynamics of a system from just its raw visual observation or to put it simply, with as little as six video frames of the system in action. Apart from predicting the future trajectories of a range of physical systems just like its parent network, this model can also infer the mass and future states of unknown objects from their influence on the visible ones! This is achieved through the co-training of a perceptual front-end convolutional neural network that parses a visual scene to provide a set of object node representations to a back-end interaction network.

Chemistry

The nano-scale molecules have an inherent graph like structure with the ions or the atoms being the nodes and the bonds between them, edges. GNNs can be applied in both scenarios: learning about existing molecular structures as well as discovering new chemical structures. This has had a significant impact in computer aided drug design. Molecular fingerprints are feature vectors that represent molecules. Machine learning models to predict the properties of a new molecule by learning from example molecules use fixed length fingerprints as inputs. GNNs can replace the traditional means that give a fixed encoding of the molecule to allow the generation of differentiable fingerprints adapted to the task for which they are required.

Furthermore, these molecular fingerprints learned by the GNN from the graph structure of the molecule need not be as large as the fixed fingerprints which must encode all possible substructures in a single feature vector. The differentiable fingerprints can be optimized to encode only related features, reducing downstream computation and regularization requirements.

Image classification

Image classification, a classic computer vision problem, has outstanding solutions from a number of state-of-the-art machine learning mechanisms, the most popular being convolutional neural networks (CNN). GNN, which drive their motivation out of CNN, have also been applied in this domain. Most of these models, including GNN, do provide attractive results when given a huge training set of labelled classes. The focus now is towards getting these models to perform well on zero-shot and few-shot learning tasks. Zero shot learning (ZSL) represents in trying to learn, to recognize classes that the model has not encountered in its training. ZSL recognition relies on the existence of a labelled training set of seen classes and the knowledge about how each unseen class is semantically related to the seen ones.

One approach is to leverage structural information, in the form of graphs, in ZSL image classification. GNN, consequently, appear quite appealing in this respect. Knowledge graphs can provide the necessary information to guide the ZSL task. Different approaches differ in the kind of information they represent in the knowledge graph. These graphs may be based on the similarities between the images themselves or those of the objects in the images extracted through object detection. The graphs may also incorporate semantic information from word embeddings of the category labels of the images. GNNs can then be applied to this structured data to aid the ZSL image classification-recognition task.

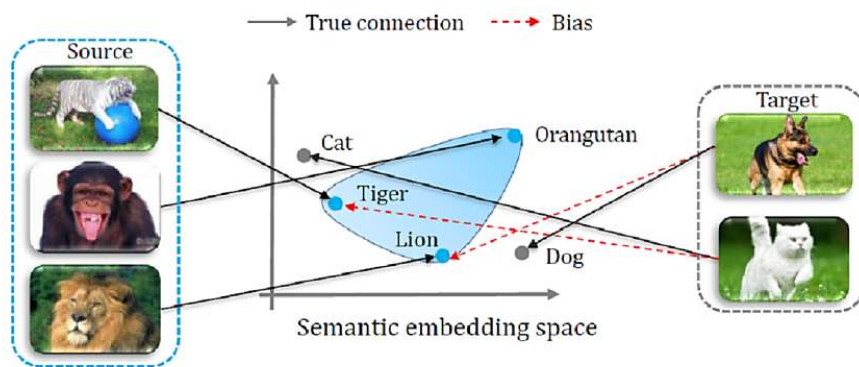


Fig. 5.25: ZSL Image Classification-Recognition Task

Text Classification

Like images, the relation within text is also not explicit, i.e. text cannot be considered structured data to which GNN can be applied directly. However, there are ways to convert a text document into structured data such as a graph of words or of sentences and then use graph convolution to convolve the word graphs. Another approach to structure the text data is to use Sentence LSTM which views an entire sentence as a single state (node) consisting of sub-states i.e. words. Document citation relations can also be used to construct graphs with documents as the nodes. Text GNNs can then be used to learn embeddings for words and documents. These approaches can be used for various NLP tasks such as text classification, sequence labelling, machine translation, relation and event extraction. One interesting area of application is reading comprehension; given a text passage, the model must be able to answer questions based on it by consolidating information in the passage. Reading comprehension is one of the complex reasoning tasks performed by humans; the answer to a question may not be located in any single part of the extract but may need global inferencing. Representing the passage in the form of a graph using a Sentence LSTM helps in better connecting the global evidences.

Combinatorial Optimization

Combinatorial optimization problems over graphs are a set of NP-hard problems. Some of these can be solved by heuristic methods. In recent times, attempt is being made to solve them using deep neural networks. Consequently, GNNs are also being leveraged to operate over these graphs structured datasets.

Travelling Salesman Problem

Interestingly, a general GNN based framework can be applied to a number of optimization problems over graphs such the minimum vertex cover problem, maximum cut, the travelling salesman problem, minimum spanning tree, etc. A few approaches combine GNNs with reinforcement learning to iteratively learn a solution given an input graph. One of the instances in which GNNs outperform traditional methods is the Quadratic Assignment Problem. It aims to measure the similarity of two graphs. The comparison of the graphs using GNN-learned node embeddings offers better performance than standard relaxation-based techniques.

Graph Neural Networks are increasingly gaining popularity, given their expressive power and explicit representation of graphical data. Hence, they have a wide range of applications in domains that can harness graph structures out of their data. Presented above is just the tip of the iceberg. As newer architectures continue to crop up, GNNs will continue to foray into diverse domains. Graph Neural Networks have now evolved into Graph Convolution Networks which, as the name recommends, are inspired by Convolution Neural Networks. These are

much more proficient and powerful and form the baseline for other complex Graph Neural Network architectures such as;

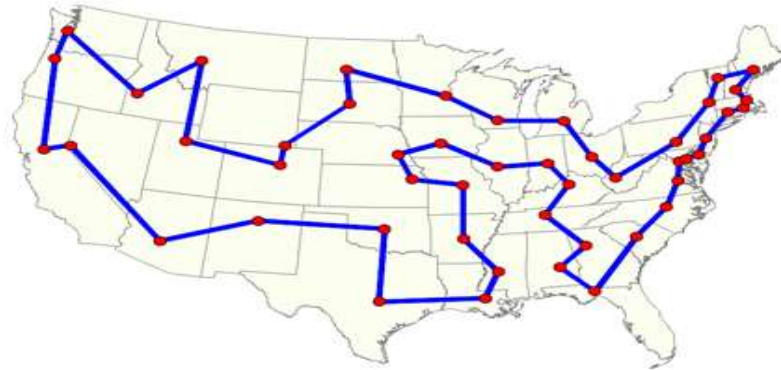


Fig. 5.26: Travelling Salesman Problem

- ✚ Graph Attention Networks
- ✚ Graph Auto-Encoders
- ✚ Graph Generative Networks
- ✚ Graph Spatio-temporal Networks

Graph Neural Networks and Computer Vision

Graph Neural Network is a type of Neural Network which directly operates on the Graph structure. A typical application of GNN is node classification. Essentially, every node in the graph is associated with a label, and we want to predict the label of the nodes without ground-truth. You know how a classical neural network works, right? We have some C-dimensional features X as the input to the net. Using the running MNIST example, X will be $C=784$ dimensional pixel features (i.e. a “flattened” image). These features get multiplied by $C \times F$ dimensional weights W that we update during training to get the output closer to what we expect. The result can be directly used to solve the task (e.g. in case of regression) or can be further fed to some nonlinearity (activation), like ReLU, or other differentiable (or more precisely, sub-differentiable) functions to form a multi-layer network. In general, the output of some layer l is:

$$X^{(l+1)} = X^{(l)} W^{(l)} \quad (1)$$

Fully-connected layer with learnable weights W . “Fully-connected” means that each output value in $X^{(l+1)}$ depends on, or “connected to”, all inputs $X^{(l)}$. Typically, although not always, we add a bias term to the output. The signal in MNIST is so strong; that you can get an accuracy

of 91% by just using the formula above and the Cross Entropy loss without any nonlinearity. Such model is called multinomial (or multiclass, since we have 10 classes of digits) logistic regression. Now, how do we transform our vanilla neural network to a graph neural network? As you already know, the core idea behind GNNs is aggregation over “neighbors”. Here, it is important to understand that in many cases, it is actually you who specifies “neighbors”.

Let’s consider a simple case first, when you are given some graph. For example, this can be a fragment (subgraph) of a social network with 5 persons and an edge between a pair of nodes denotes if two people are friends (or at least one of them think so). An adjacency matrix (usually denoted as A) in the figure below on the right is a way to represent these edges in a matrix form, suitable for our deep learning frameworks. Yellow cells in the matrix represent the edge and blue, the absence of the edge.

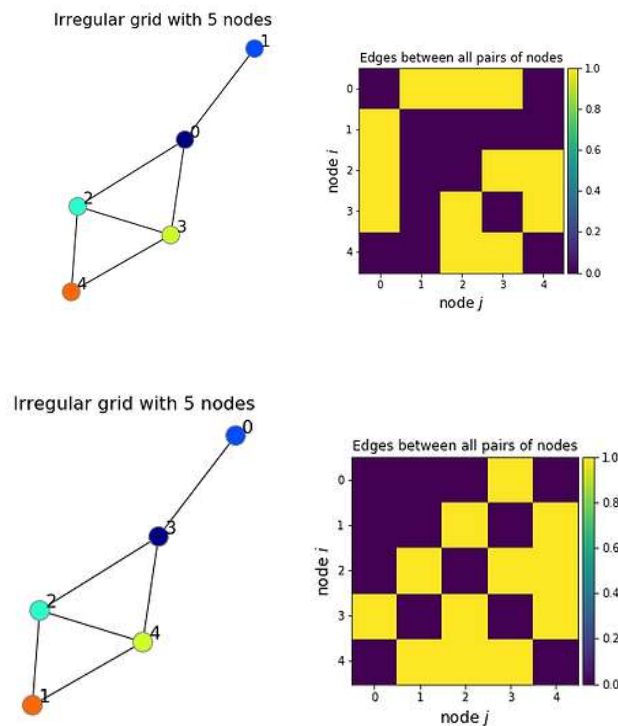


Fig. 5.27: Example of a graph and its adjacency matrix. The order of nodes we defined in both cases is random, while the graph is still the same.

Now, let’s create an adjacency matrix A for our MNIST example based on coordinates of pixels. This is a typical, but not the only, way to define an adjacency matrix for visual tasks. This adjacency matrix is our prior, or our inductive bias, we impose on the model based on our

intuition that nearby pixels should be connected and remote pixels shouldn't or should have very thin edge (edge of a small value). This is motivated by observations that in natural images nearby pixels often correspond to the same object or objects that interact frequently (the locality principle), so it makes a lot of sense to connect such pixels. So, now instead of having just features X , we have some fancy matrix A with values in the range $[0,1]$. It's important to note that once we know that our input is a graph, we assume that there is no canonical order of nodes that will be consistent across all other graphs in the dataset. In terms of images, it means that pixels are assumed to be randomly shuffled. Finding the canonical order of nodes is combinatorically unsolvable in practice. Even though for MNIST we technically can trickster by knowing this order (because data are originally from a regular grid), it's not going to work on actual graph datasets.

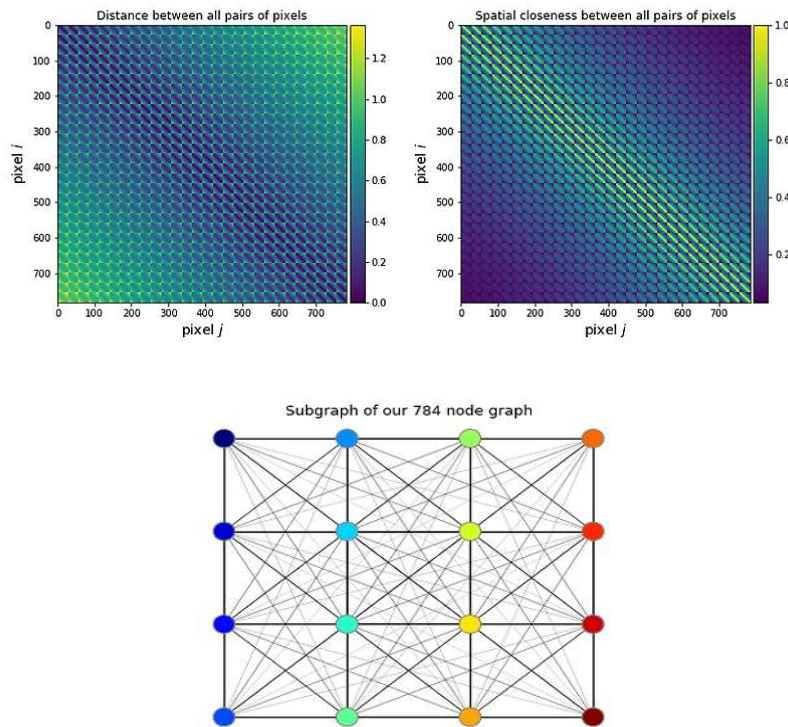


Fig. 5.28: Adjacency matrix ($N \times N$) in the form of distances (left) and closeness (right) between all pairs of nodes (bottom). A subgraph with 16 neighboring pixels corresponding to the adjacency matrix in the middle. Since it's a complete subgraph, it's also called a "clique".

Remember that our matrix of features X has N rows and C columns. So, in terms of graphs, each row corresponds to one node and C is the dimensionality of node features. But now the

problem is that we don't know the order of nodes, so we don't know in which row to put features of a particular node. If we just pretend to ignore this problem and feed X directly to an MLP as we did before, the effect will be the same as feeding images with randomly shuffled pixels with independent (yet the same for each epoch) shuffling for each image! Surprisingly, a neural network can in principle still fit such random data; however test performance will be close to random prediction. One of the solutions is to simply use the adjacency matrix A ; we created before, in the following way:

$$X^{(l+1)} = \mathcal{A}X^{(l)}W^{(l)} \quad (2)$$

Graph neural layer with adjacency matrix A , input/output features X and learnable weights W . We just need to make sure that row i in A corresponds to features of node in row i of X . Here, using $\mathcal{A}$ instead of plain A , because often you want to normalize A . If $\mathcal{A}=A$, the matrix multiplication $\mathcal{A}X^{(l)}$ will be equivalent to summing features of neighbors, which turned out to be useful in many tasks. Most commonly, you normalize it so that $\mathcal{A}X^{(l)}$ averages features of neighbors, i.e. $\mathcal{A}=A/\Sigma_i A_i$. After running the code, you may notice that the classification accuracy is actually about the same. What's the problem? Aren't graph networks supposed to work better? Well, they are, in many cases. But not in this one, because the $\mathcal{A}X^{(l)}$ operator we added is actually nothing else, but a Gaussian filter:

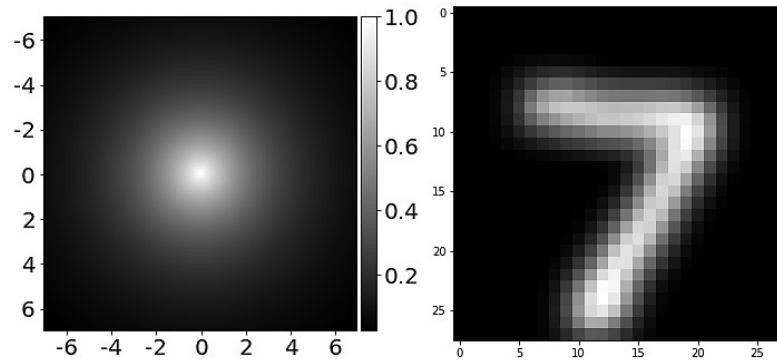


Fig. 5.29: 2D visualization of a filter used in a graph neural network and its effect on the image.

So, our graph neural network turned out to be equivalent to a convolutional neural network with a single Gaussian filter, that we never update during training, followed by the fully-connected layer. This filter basically blurs/smooths the image, which is not a particularly useful thing to do (see the image above on the right). However, this is the simplest variant of a graph neural network, which nevertheless works great on graph-structured data. To make GNNs work

better on regular graphs, like images, we need to apply a bunch of tricks. For example, instead of using a predefined Gaussian filter, we can learn to predict an edge between any pair of pixels by using a differentiable function. This idea is similar to Dynamic Filter Networks and Edge-conditioned Graph Networks. GNNs are a very lithe and interesting family of neural networks that can be applied to really complex data.

As always, such flexibility must come at a certain cost. In case of GNNs it is the difficulty of regularizing the model by defining such operators as convolution. Research in that direction is advancing quite fast, so that GNNs will see application in increasingly wider areas of machine learning and computer vision. Recently, Graph Neural Network has gained growing popularity in various domains, including social network, knowledge graph, recommender system, and even life science. The power of GNN in modeling the dependencies between nodes in a graph enables the breakthrough in the research area related to graph analysis. Let us consider two more advanced algorithms, Deep Walk and Graph Sage.

Deep Walk

Deep Walk is the first algorithm proposing node embedding learned in an unsupervised manner. It highly resembles word embedding in terms of the training process. The motivation is that the distribution of both nodes in a graph and words in a corpus follow a power law as in the following figure:

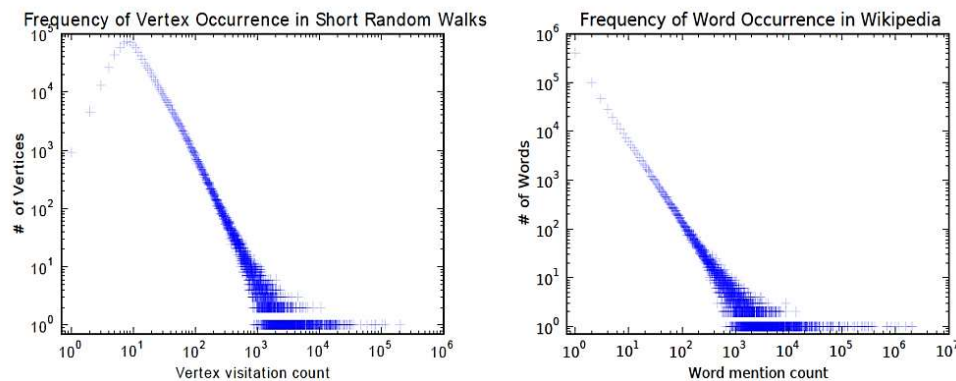


Fig. 5.30: Distribution of Nodes

The algorithm contains two steps:

1. Perform random walks on nodes in a graph to generate node sequences.
2. Run skip-gram to learn the embedding of each node based on the node sequences generated in step 1.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{k=1}^K e^{x_k}}$$

At each time step of the random walk, the next node is sampled uniformly from the neighbor of the previous node. Each sequence is then truncated into sub-sequences of length $2|w| + 1$, where w denotes the window size in skip-gram. Hierarchical softmax is applied to address the computational cost of softmax due to the huge number of nodes. To compute the softmax value of each of the individual output element, we must compute all the e^{x_k} for all the element k . Therefore, the computation time is $O(|V|)$ for the original softmax, where V denotes the set of vertices in the graph.

Hierarchical softmax utilizes a binary tree to deal with the problem. In this binary tree, all the leaves ($v_1, v_2, \dots, v_8$ in the graph) are the vertices in the graph. In each of the inner node, there is a binary classifier to decide which path to choose. To compute the probability of a given vertex v_k , one simply compute the probability of each of the sub-path along the path from the root node to the leaf v_k . Since the probability of each node's children sums to 1, the property that the sum of the probability of all the vertices equals 1 still holds in the hierarchical softmax. The computation time for an element is now reduced to $O(\log|V|)$ as the longest path for a binary tree is bounded by $O(\log(n))$ where n is the number of leaves.

After a DeepWalk GNN is trained, the model has learned a good depiction of each node as shown in the following figure. Different colors indicate different labels in the input graph. We can see that in the output graph (embedding with 2 dimensions); nodes having the same labels are clustered together, while most nodes with different labels are separated properly. But, the main issue with DeepWalk is that it lacks the ability of generalization. Whenever a new node comes in, it has to re-train the model in order to represent this node (transductive). Thus, such GNN is not suitable for dynamic graphs where the nodes in the graphs are ever-changing.

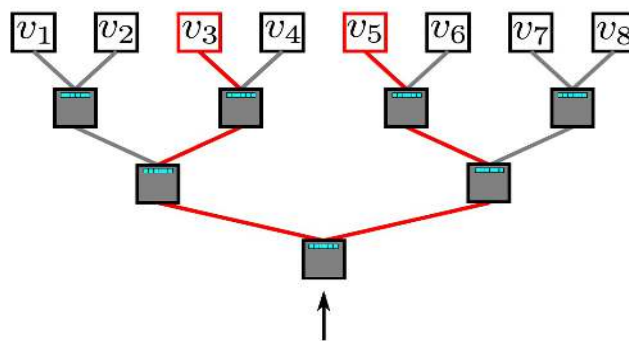


Fig. 5.31: DeepWalk GNN Model

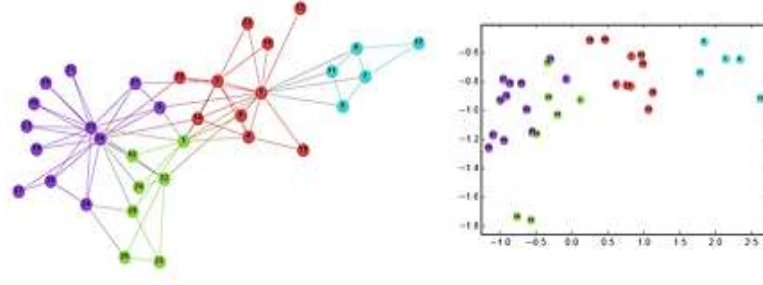


Fig. 5.32: Input (Left) and Output (Right) Representation of Deep Walk

Graph Sage

Graph Sage provides a solution to address the aforementioned problem, learning the embedding for each node in an inductive way. Specifically, each node is represented by the aggregation of its neighborhood. Thus, even if a new node unseen during training time appears in the graph, it can still be properly represented by its neighboring nodes.

The outer loop indicates the number of update iteration, while h^k_v denotes the latent vector of node v at update iteration k . At each update iteration, h^k_v is updated based on an aggregation function, the latent vectors of v and v 's neighborhood in the previous iteration, and a weight matrix W^k . Graph Sage proposes three aggregation functions;

Below is the algorithm of Graph Sage:

Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{x_v, \forall v \in \mathcal{V}\}$; depth K ; weight matrices $W^k, \forall k \in \{1, \dots, K\}$; non-linearity σ ; differentiable aggregator functions $AGGREGATE_k, \forall k \in \{1, \dots, K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector representations z_v for all $v \in \mathcal{V}$

```

1  $h_v^0 \leftarrow x_v, \forall v \in \mathcal{V}$ ;
2 for  $k = 1 \dots K$  do
3   for  $v \in \mathcal{V}$  do
4      $h_{\mathcal{N}(v)}^k \leftarrow AGGREGATE_k(\{h_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ ;
5      $h_v^k \leftarrow \sigma(W^k \cdot \text{CONCAT}(h_v^{k-1}, h_{\mathcal{N}(v)}^k))$ 
6   end
7    $h_v^k \leftarrow h_v^k / \|h_v^k\|_2, \forall v \in \mathcal{V}$ 
8 end
9  $z_v \leftarrow h_v^K, \forall v \in \mathcal{V}$ 

```

(1) Mean aggregator:

The mean aggregator takes the average of the latent vectors of a node and its entire neighborhood.

$$\mathbf{h}_v^k \leftarrow \sigma(\mathbf{W} \cdot \text{MEAN}(\{\mathbf{h}_v^{k-1}\} \cup \{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\}))$$

It removes the concatenation operation at line 5 in the above pseudo code. This operation can be viewed as a “skip-connection”, which proved to largely improve the performance of the model.

(2) LSTM aggregator:

Since the nodes in the graph don't have any order, they assign the order randomly by permuting these nodes.

(3) Pooling aggregator:

This operator makes an element-wise pooling function on the neighboring set. Below shows an example of max-pooling:

$$\text{AGGREGATE}_k^{\text{pool}} = \max(\{\sigma(\mathbf{W}_{\text{pool}} \mathbf{h}_{u_i}^k + \mathbf{b}), \forall u_i \in \mathcal{N}(v)\})$$

Which can be replaced with mean-pooling or any other symmetric pooling function? It points out that pooling aggregator performs the best, while mean-pooling and max-pooling aggregator has similar performance. The paper uses max-pooling as the default aggregation function.

The loss function is defined as the following:

$$J_{\mathcal{G}}(\mathbf{z}_u) = -\log(\sigma(\mathbf{z}_u^T \mathbf{z}_v)) - Q \cdot \mathbb{E}_{v_n \sim P_n(v)} \log(\sigma(-\mathbf{z}_u^T \mathbf{z}_{v_n}))$$

Where u and v co-occur in a fixed-length random walk, while v_n are the negative samples that don't co-occur with u . Such loss function encourages nodes closer to have comparable embedding, while those far apart to be separated in the projected space. Via this approach, the nodes will gain more and more information about their neighborhoods.

Graph Sage enables representable embedding to be generated for unseen nodes by aggregating its nearby nodes. It allows node embedding to be applied to domains involving dynamic graph, where the structure of the graph is ever-changing. Pinterest, for example, has adopted an extended version of Graph Sage, Pin Sage, as the core of their content discovery system.

The power of GNN in modeling complex graph structures is truly astonishing. In view of its effectiveness, in the near future, GNN will play an important role in AI's development.

5.10 WHAT ARE VISUAL AND FACIAL RECOGNITION?

One of the most important applications of computer vision is face recognition and face detection. For many internet companies like Facebook and Google, face detection and recognition is one of the most important modules in artificial intelligence systems. Facebook has also started training its face and image recognition models on hashtags. Earlier this year, Apple introduced face ID for its iPhone X.

They have developed proprietary technology and knowledge bases that have excelled in detecting faces. Last year even Samsung launched its device named Galaxy Note7, the first gadget to feature Iris recognition for popular devices. Face detection is also doing wonders for companies working in the biometric identification systems.

They have applications in the sectors that range from banking, healthcare to travel and retail. Reportedly, the face recognition market was around \$3.85 billion in 2017 and will most probably grow to \$9.78 billion by 2023 at a CAGR of 16.81% during this period. The report also says that the Asia Pacific is the fastest growing region during the forecasted period. The most familiar face recognition APIs are;

(1) Google Cloud Vision

Face recognition platforms tuned into cloud engines have a massive advantage. If your application runs on Google cloud engine it will be very easy to integrate Google Cloud Vision into your product or application. It boasts many pre-trained models and API outlets to become a power tool for many computer vision programmers. There is an added benefit of availability of AutoML Vision that can help programmes to train customer vision models. Google Cloud Vision has the power to detect multiple faces within an image along with major attributes like emotional state and usage of headband by the person. Google is also reportedly working on adding a facial recognition feature to the suite of tools.

(2) IBM Watson Visual Recognition

The IBM Watson Visual Recognition is a very powerful and trusted industry scale application which can be used in many computer vision applications. It has the power to accurately and quickly tag, classify and train on vision data/datasets using machine learning. In the early part due to some developments, IBM disabled features that could help programmers to identify the name and type of the person given the image but the engine still remains a very powerful place to train state of the art facial recognition and detection applications. Currently, the supported image formats are .gif, .jpg, .png and .tif with a maximum size of 10 MB with a recommended pixel density of 32 x 32 per inch.

(3) Microsoft Face API

Microsoft boasts a top AI team with experts from computer vision domains who have built strong cloud centric intelligent applications. Microsoft Face API helps the developers to detect similar faces and compare them, organize face images into similar groups to cluster them. Face API also empowers developers to identify previously tagged people in images that can lead to powerful security and internet applications. The Face API by Microsoft provides Face Verification as a service which can be used to check the likelihood of two different faces to be the same person and return a score. Face detection service from the API has the power to detect one or more human faces in an image and get a face rectangle for the face with 27 landmarks for a single face. Additional features in Microsoft Face API include emotion recognition for faces that can detect emotions such as anger, contempt, disgust, fear, happiness among other things.

(4) Amazon Rekognition

Amazon Rekognition backed by AWS cloud services is another giant in the face recognition space. Amazon Rekognition is the easiest way to add features relating to image or video processing to your application, especially if you are running your application on AWS cloud. The service can identify faces, people and activities among many other things once given image content.

Amazon Rekognition is very popular for its facial analysis service. It has a fairly accurate facial recognition engine that works on many kinds of images and videos also. The developer can use this service to build application features that can verify users, count the number of people and build public safety applications. AWS being the biggest cloud service in the world leverages humongous amounts of data to improve its machine learning models. The service has additional benefits like having the option to run processes in Batch mode & real-time mode.

(5) Kairos

Kairos is a company specializing in face recognition and detection. Unlike other APIs which are a suite of tools and services rolled into one, Kairos exclusively delivers face recognition solutions and hence is one of the best Face AI solutions in the world. Many of the customers prefer to host Kairos API on their own servers and hence being safer and being in control of their data. This feature also gives an edge to Kairos over the competition. Kairos has countless face analysis features such as face detection, identification and verification. Other services include emotion detection and detection of facial features which can detect specific parts such as eyes, nose and others.

Visual Recognition And Advertising

Ad relevancy is key to moving a customer down their purchase path. The consumer profiles we build on each individual user provide insight into their needs, brand biases, social stances and much, much more. When done correctly, we are able to better understand a prospect, allowing us to place a relevant ad in front of a person who is actively shopping for our brands or products. To be even more successful, we need to go beyond what that individual wants at that specific moment. Behavioral economics has shown that consistent touch points over a period of time allow us to influence potential customers when they are activated to make a purchase decision at a later date. Being there throughout the funnel, not just at the bottom of the funnel, is key to increasing your pool of qualified customers. One way to get a more complete picture of a customer is to perform textual and sentiment analysis on the words they share on social media.

The technology to do this kind of thing has been around for a while, which is why innovators are taking such analyses to the next level. Nowadays, visual recognition software is facilitating to develop a deeper understanding of consumers in a similar way. (Note: The industry doesn't seem to have settled on a single name for this technology. Besides visual recognition, it's also known as computer vision, image recognition and image analysis.) The images and videos a person shares or likes via social media provide insights not only into what products they like, but can also give marketers a window into many other elements of a potential customer's lifestyle. Powered by machine learning, algorithms can quickly identify unique elements of images that can help power the marketing machine that determines what ad should be delivered to whom for the best result.

It seems likely, though, that visual recognition will eventually become a part of the digital landscape in some form and, even without faces, it offers a lot of potential opportunities. Within many images users upload onto social media there are hundreds, if not thousands of elements that can now be identified through visual recognition software. It may seem like a picture of you and your friends at a bar, but it is so much more. Of course, standard elements like location, tagged individuals and the hashtag used are all up for grabs. But visual recognition analyzes everything within an image.

It can tell if you are drinking whiskey or vodka based on the color of the liquor in your glass. It can determine the fabric of your shirt and the brand of your jeans. It can read the sign in the background and the name of the restaurant from the menu sitting on your table. This type of information provides deep insight into which we are as consumers and is extremely valuable to advertisers. For example, if a picture someone shares on social media includes them holding a glass of high-end whiskey while also wearing a Gucci belt, chances are they are a good target

for luxury retail goods. Compare that with someone holding a low-end beer brand and a shirt with an Old Navy logo on it.

By looking back through the various images shared on an individual's profiles, the deep-learning algorithm can determine if luxury goods are a common part of this person's lifestyle. Affinities toward brands and/or products can be used in real time to purchase programmatic inventory across display, video and even addressable TV. Being able to analyze images to this level enables us to go from targeting people by what companies they "like" or "follow" on social media to targeting them based on the products and brands they actually buy and use, leading to more relevant ads and ultimately a better-targeted customer.

Advances In Visual Recognition

Many companies such as IBM, Apple, Amazon, Google and Facebook endure to push the envelope when it comes to image recognition. Google's advances in image search continue to evolve, while Facebook can spot almost anyone in a crowd of an uploaded image. IBM continues to evolve within its Watson platform, using machine learning to identify various elements within the image. IBM is going beyond the people in the picture. "Visual recognition capabilities allow for the identification of scenes, objects, faces, colors, food, text and much more.

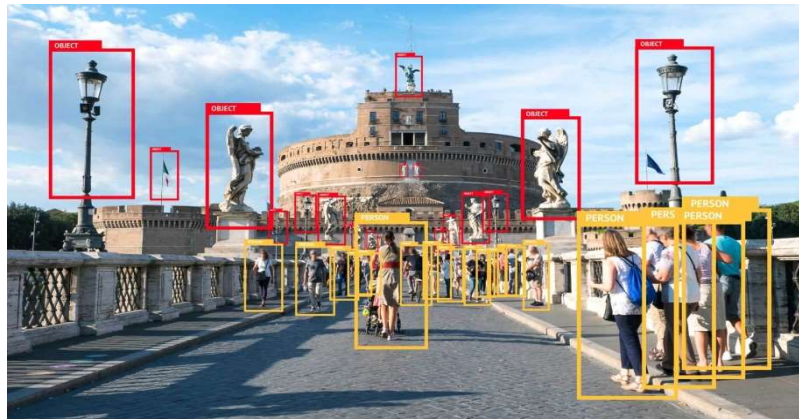


Fig. 5.33: Visual Recognition (Yellow box-Person and Red box-Object)

And with the use of APIs, these technologies are ready to be tapped and integrated into any number of martech tools. Amazon is aggressively marketing its Rekognition service, which makes powerful image and video analysis available. Google, too, has made available its Cloud Video Intelligence and Vision APIs along with the Cloud AutoML Vision machine learning service. Additionally, some social media analytics players like Brandwatch, Crimson Hexagon

and Netbase are already touting their own capabilities. As quickly as it is advancing, image or visual recognition software faces a challenge on the consumer side, where many fear the privacy consequences of having such a technology widely deployed. Think back to when Apple released Face ID, a biometric security measure with its iPhone X, and how news about its inaccuracy and general “creepiness” dominated the tech headline. And with the more recent privacy discussion sparked by the General Data Protection Regulation (GDPR) pushing forward in Europe, companies like Facebook are constantly having their innovation in facial recognition questioned and stymied there due to associated privacy concerns.

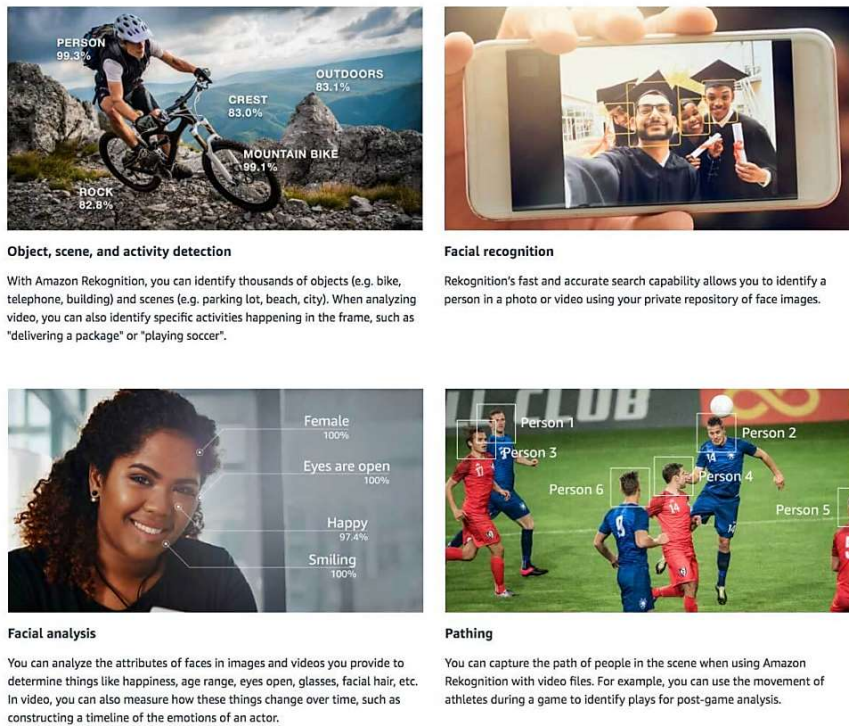


Fig. 5.34: Visual Recognition and Exemplary

Recognizing objects and patterns in images and video used to be a complicated task that was only available to Data Scientists and Software Engineers familiar with Machine Learning. The barrier to entry in this field has drastically reduced in the last few years due to advances in open source libraries and availability of vast amounts of data. IBM particularly has made it very simple to distinguish objects in images with their Watson Visual Recognition service. Visual understanding goes well further than object recognition. With one glance at an image, we can effortlessly imagine the world beyond the pixels: for instance, we can infer people’s actions, goals, and mental states.

They offer a couple of in-built models

- 📌 **General model:** Default classification from thousands of classes
- 📌 **Face model:** Facial analysis with age and gender
- 📌 **Explicit model:** Whether an image is inappropriate for general use
- 📌 **Food model:** Specifically for images of food items
- 📌 **Text model:** Text extraction from natural scene images

Additionally, they offer a very intuitive interface to build and train custom models. You only need a few things to get started.

- 📌 IBM Cloud account to log into the Watson Dataplatform.
- 📌 At least two categories for classification with at least 10 positive images for each category.
- 📌 It is advisable to have negative samples as well that do not belong to any of the other target categories.

Face recognition

Face recognition is the problem of identifying and verifying people in a photograph by their face. It is a task that is trivially performed by humans, even under varying light and when faces are changed by age or obstructed with accessories and facial hair. Nevertheless, it is remained a challenging computer vision problem for decades until recently. Deep learning methods are able to leverage very large datasets of faces and learn rich and compact representations of faces, allowing modern models to first perform as-well and later to outperform the face recognition capabilities of humans. There is often a need to automatically recognize the people in a photograph. There are many reasons why we might want to automatically recognize a person in a photograph.

For example:

- 📌 We may want to restrict access to a resource to one person, called face authentication.
- 📌 We may want to confirm that the person matches their ID, called face verification.
- 📌 We may want to assign a name to a face, called face identification.

Generally, we refer to this as the problem of automatic “face recognition” and it may apply to both still photographs or faces in streams of video. Humans can perform this task very easily. We can find the faces in an image and comment as to who the people are, if they are known. We can do this very well, such as when the people have aged, are wearing sunglasses, have different colored hair, are looking in different directions, and so on. We can do this so well that

we find faces where there aren't any, such as in clouds. Nevertheless, this remains a hard problem to perform automatically with software, even after 60 or more years of research. Until perhaps very recently. For example, recognition of face images acquired in an outdoor environment with changes in illumination and/or pose remains a largely unsolved problem. In other words, current systems are still far away from the capability of the human perception system.

Process of Automatic Face Recognition

Face recognition is the problem of identifying or verifying faces in a photograph. A general statement of the problem of machine recognition of faces can be formulated as follows: given still or video images of a scene, identify or verify one or more persons in the scene using a stored database of faces. Face recognition is often described as a process that first involves four steps; they are: face detection, face alignment, feature extraction, and finally face recognition.

-  **Face Detection:** Locate one or more faces in the image and mark with a bounding box.
-  **Face Alignment:** Normalize the face to be consistent with the database, such as geometry and photometrics.
-  **Feature Extraction:** Extract features from the face that can be used for the recognition task.
-  **Face Recognition:** Perform matching of the face against one or more known faces in a prepared database.

A given system may have a separate module or program for each step, which was traditionally the case, or may combine some or all of the steps into a single process.

Face Detection Task: Face detection is the non-trivial first step in face recognition. It is a problem of object recognition that requires that both the location of each face in a photograph is identified (e.g. the position) and the extent of the face is localized (e.g. with a bounding box). Object recognition itself is a challenging problem, although in this case, it is similar as there is only one type of object, e.g. faces, to be localized, although faces can vary wildly. The human face is a dynamic object and has a high degree of variability in its appearance, which makes face detection a difficult problem in computer vision.

Further, because it is the first step in a broader face recognition system, face detection must be robust. For example, a face cannot be recognized if it cannot first be detected. That means faces must be detected with all manner of orientations, angles, light levels, hairstyles, hats, glasses, facial hair, makeup, ages, and so on. As a visual front-end processor, a face detection system should also be able to achieve the task regardless of illumination, orientation, and camera distance. The taxonomy of face detection methods can be broadly divided into two main groups:

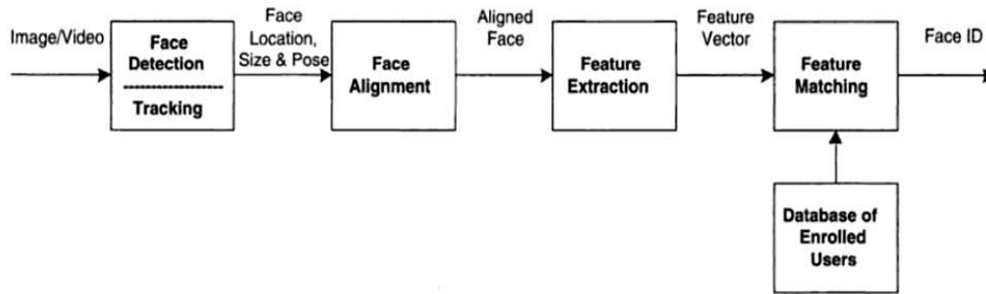


Fig. 5.35: Overview of the Steps in a Face Recognition Process

- 📌 Feature-Based.
- 📌 Image-Based.

The feature-based face detection uses hand-crafted filters that search for and locate faces in photographs based on a deep knowledge of the domain. They can be very fast and very effective when the filters match, although they can fail dramatically when they don't, e.g. making them somewhat fragile. Make explicit use of face knowledge and follow the classical face detection methodology in which low level features are derived prior to knowledge-based analysis. The apparent properties of the face such as skin color and face geometry are exploited at different system levels. Alternately, image-based face detection is holistic and learns how to automatically locate and extract faces from the entire image. Neural networks fit into this class of methods. Address faces detection as a general recognition problem.

Image-based representations of faces, for example in 2D intensity arrays, are directly classified into a face group using training algorithms without feature derivation and analysis. These relatively new techniques incorporate face knowledge implicitly into the system through mapping and training schemes. Perhaps the most dominant method for face detection called the detector cascade or simply "cascade." The detector, called detector cascade, consists of a sequence of simple-to-complex face classifiers and has attracted extensive research efforts. Moreover, detector cascade has been deployed in many commercial products such as smartphones and digital cameras. While cascade detectors can accurately find visible upright faces, they often fail to detect faces from different angles, e.g. side view or partially occluded faces.

Face Recognition Tasks: The task of face recognition is broad and can be tailored to the specific needs of a prediction problem. The three face recognition tasks are:

- 📌 **Face Matching:** Find the best match for a given face.

- 📌 **Face Similarity:** Find faces that are most similar to a given face.
- 📌 **Face Transformation:** Generate new faces that are similar to a given face.

Matching requires that the candidate matching face image be in some set of face images selected by the system. Similarity detection requires in addition to matching that images of faces be found which are similar to a recalled face this requires that the similarity measure used by the recognition system closely match the similarity measures used by humans. Transformation applications require that new images created by the system be similar to human recollections of a face. The two main modes for face recognition are

- 📌 **Face Verification:** A one-to-one mapping of a given face against a known identity (e.g. is this the person?).
- 📌 **Face Identification:** A one-to-many mapping for a given face beside a database of known faces (e.g. who is this person?).

A face recognition system is expected to identify faces present in images and videos automatically. It can operate in either or both of two modes: face verification (or authentication), and face identification (or recognition). We can describe the problem of face recognition as a supervised predictive modeling task trained on samples with inputs and outputs. In all tasks, the input is a photo that contains at least one face, most likely a detected face that may also have been aligned.

The output varies based on the type of prediction required for the task; for example:

- 📌 It may then be a binary class label or binary class probability in the case of a face verification task.
- 📌 It may be a categorical class label or set of probabilities for a face identification task.
- 📌 It may be a similarity metric in the case of a similarity type task.

Deep Learning for Face Recognition

Face recognition has remained an active area of research in computer vision. Perhaps one of the more widely known and adopted “machine learning” methods for face recognition was “Eigenfaces,” was a milestone as it achieved impressive results and demonstrated the capability of simple holistic approaches. Face images are projected onto a feature space (“face space”) that best encodes the variation among known face images. The face space is defined by the “eigenfaces”, which are the eigenvectors of the set of faces; they do not necessarily correspond to isolated features such as eyes, ears, and noses. The holistic methods dominated the face recognition community in the 1990s. In the early 2000s, handcrafted local descriptors became popular, and the local feature learning approaches were introduced in the late 2000s.

Shallow learning methods performance steadily improves from around 60% to 90%, while deep learning boosts the performance to 99.80% in just three years. Given the breakthrough of AlexNet in 2012 for the simpler problem of image classification, there was a flurry of research on deep learning methods for face recognition. Capabilities quickly realized near-human-level performance, then exceeded human-level performance on a standard face recognition dataset within a three year period, which is an astounding rate of improvement given the prior decades of effort. There are perhaps four milestone systems on deep learning for face recognition that drove these innovations; they are: DeepFace, the DeepID series of systems, VGGFace, and FaceNet. Let's briefly touch on each.

DeepFace is a system based on deep convolutional neural networks from Facebook AI Research and Tel Aviv. It was described in the 2014 titled "DeepFace: Closing the Gap to Human-Level Performance in Face Verification." It was conceivably the first major leap forward using deep learning for face recognition, achieving near human-level performance on a standard benchmark dataset. This method reaches an accuracy of 97.35% on the Labeled Faces in the Wild (LFW) dataset, reducing the error of the current state of the art by more than 27%, closely approaching human-level performance.

The DeepID, or "Deep hidden IDentity features," is a series of systems (e.g. DeepID, DeepID2, etc.), first described in 2014 titled "Deep Learning Face Representation from Predicting 10,000 Classes." This system was first described much like DeepFace, although was expanded in subsequently to support both identification and verification tasks by training via contrastive loss. The key challenge of face recognition is to develop effective feature representations for reducing intra-personal variations while enlarging inter-personal differences. The face identification task increases the inter-personal variations by drawing DeepID2 features extracted from different identities apart, while the face verification task reduces the intra-personal variations by pulling DeepID2 features extracted from the same identity together, both of which are essential to face recognition. The DeepID systems were among the first deep learning models to achieve better-than-human performance on the task, e.g. DeepID2 achieved 99.15% on the Labeled Faces in the Wild (LFW) dataset, which is better-than-human performance of 97.53%. Following systems such as FaceNet and VGGFace improved upon these results.

FaceNet was described by Google in their 2015 titled "FaceNet: A Unified Embedding for Face Recognition and Clustering." This system achieved then state-of-the-art results and presented an innovation called 'triplet loss' that allowed images to be encoded efficiently as feature vectors that allowed rapid similarity calculation and matching distance calculations. FaceNet, which directly learns a mapping from face images to a compact Euclidean space where distances directly correspond to a measure of face similarity. The method uses a deep

convolutional network trained to directly optimize the embedding itself, rather than an intermediate bottleneck layer as in previous deep learning approaches. To train, it uses triplets of roughly aligned matching/non-matching face patches generated using a novel online triplet mining method.

The VGGFace was developed by the Visual Geometry Group (VGG) at Oxford and was described in 2015 titled “Deep Face Recognition.” In addition to a better-tuned model, the focus of this work was on how to collect a very large training dataset and use this to train a very deep CNN model for face recognition that allowed them to achieve then state-of-the-art results on standard datasets. This showed how a very large scale dataset (2.6M images, over 2.6K people) can be assembled by a combination of automation and human in the loop. Although these may be the key early milestones in the field of deep learning for computer vision, progress has continued, with much innovation focused on loss functions to effectively train the models.

5.11 WHAT IS DATA EXTRACTION AND ANALYSIS?

Computer vision is one such AI technology that has seen exponential breakthrough over the last few years. Today, the technology is being realized in many forms and to solve various problems across different industries. Besides its very popular application in facial recognition and driverless cars, one of the attractive uses of computer vision is data extraction from images and PDFs. The use of this technology in life sciences and healthcare can be used to fast-track the analysis of patient data, research data from publications and theses, and information about clinical trials. The data ocean of images and PDFs is almost impossible to be analyzed; information extracted from it, and segregates it properly. Though, this is where computer vision comes in to save tons of time, money, and manpower.

How Can Computer Vision Help In Data Extraction?

In the healthcare industry, computer vision can play a significant role. How? By analyzing medical imagery and research PDFs for various purposes, such as in order to facilitate faster and much more efficient drug development and patient care. Medical imagery is one of the most vital components that facilitate medical findings. With computer vision, scientists can accelerate their research and find new solutions by analyzing unfrequented patterns by using AI algorithms for image registration, segmentation, annotation, and multimodal image fusion. Data extraction can be carried out from two types of healthcare data. The first is the patients’ data such as X-rays, MRIs, CT scans, etc.

It is estimated that the amount of this data is growing at 300% each year. This medical data is used to train computer vision algorithms to drive faster insights for patient care and for

discovery. New cures are being realized with the help of AI and hidden patterns are being revealed that are enabling healthcare professionals to find more causes as well as signals of various diseases and disorders. On the other hand, healthcare data also includes medical publications, thesis, and dissertations, and information about a number of clinical trials and research is taking place in life sciences. This data is usually available in the form of PDFs. These scanned text images cannot be manually read for every important information. Manually it will take years to analyze and extract information from all the PDFs available in the life science repository. This is one of the prime reasons why the visualization and analysis of healthcare big data with the convention of computer vision is necessary. However, it is also difficult. With a lot of information still in the form of PDFs, the data becomes inaccessible as these formats cannot be processed easily.

Challenges of Data Extraction from PDFs

Data that can be used by researchers is available on the web in various formats. It becomes nearly impossible to extract all the relevant information from PDFs and Images. Although computer vision combined with other AI techniques can handle vast amounts of data, the challenges of extracting data from PDFs or text images go hand in hand. Some of these include single column vs. double column formats, variation in layout, the inclusion of graphs, pie charts, and footnotes, multiple fonts, colors, and font sizes. First of all, it is important to understand that, here, for the purpose of computer vision PDFs are considered image data and not documents.

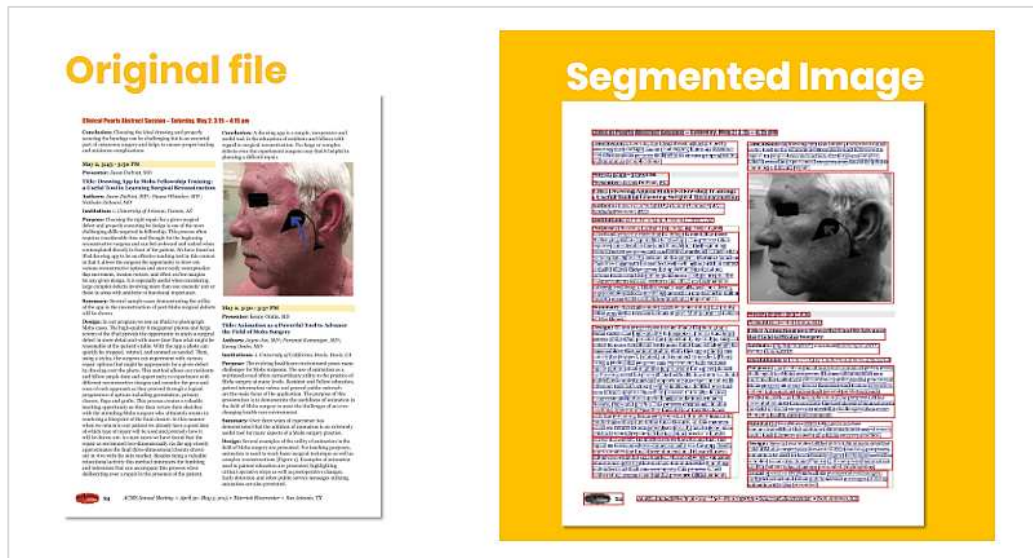


Fig. 5.36: Data Extraction from PDFs

PDFs are basically scanned images! Due to this reason, it is hard for the computer to understand the text on PDFs. A lot of times, PDF text extraction follow the removal of footnotes, which may be important for the document, or missing important characters, exclusion of graphs etc. To overcome this problem, the use of Optical Character recognition is required to differentiate between symbols, formats, fonts, etc. in order to read these scanned images efficiently and extract data from them.

Compared to standard Optical Character Recognition Software, which can only extract the words, it is possible to extract information along with the context from where the text originated. For example, the computer vision technologies can identify the sections from which text is extracted, such as an abstract, introduction, results or discussion section. This enables our AI algorithms to rank the strength of relationships found within an article, a relationship located within the results or discussion section is more likely to be a novel relationship than one mentioned in the introduction.

Why Is It Important To Extract PDF Data?

The first and foremost reason why it is important to extract PDFs data, especially for the healthcare industry is that a lot of important information, research theses, etc. is available in PDF form. It is not possible to go through every PDF and manually take notes or copying useful data for future use. Manual data entry tasks require lots of time, patience, manpower and money. As we know the big data of images and PDFs are already exploding, it is important to use this data effectively. For this purpose, a tool facilitated with AI and computer vision can be very helpful to quickly extract data without any human error. Once the four essentials of image analysis, namely classification, indexing, retrieval, and management are done, it can be used. Within a webpage or publication, there will be multiple of elements that may be in table or graph formats.

Understanding what is a table vs. what is a paragraph is important as the text within each will have different context. In the example displayed (Figure 5.37), text pertaining to the Human EGFR gene has been extracted, and most notably associated with EGFR. This “association” is the underlying context of all of the data on the page shown. Advanced computer vision uses the technology combined with enhanced OCR, interprets layouts and text with much higher precision and offers a great solution to this problem. By leveraging computer vision and combining it with Natural Language Processing, relevant information is processed and extracted from formats such as PDFs and Images. Medical image analysis now offers a secure solution; it relies on seamless updatability and real-time data availability related to medical data, research, clinical trials for drug advance, medical theses, new publications etc. extracted from PDF data to effectively empower new medical research and drug development.

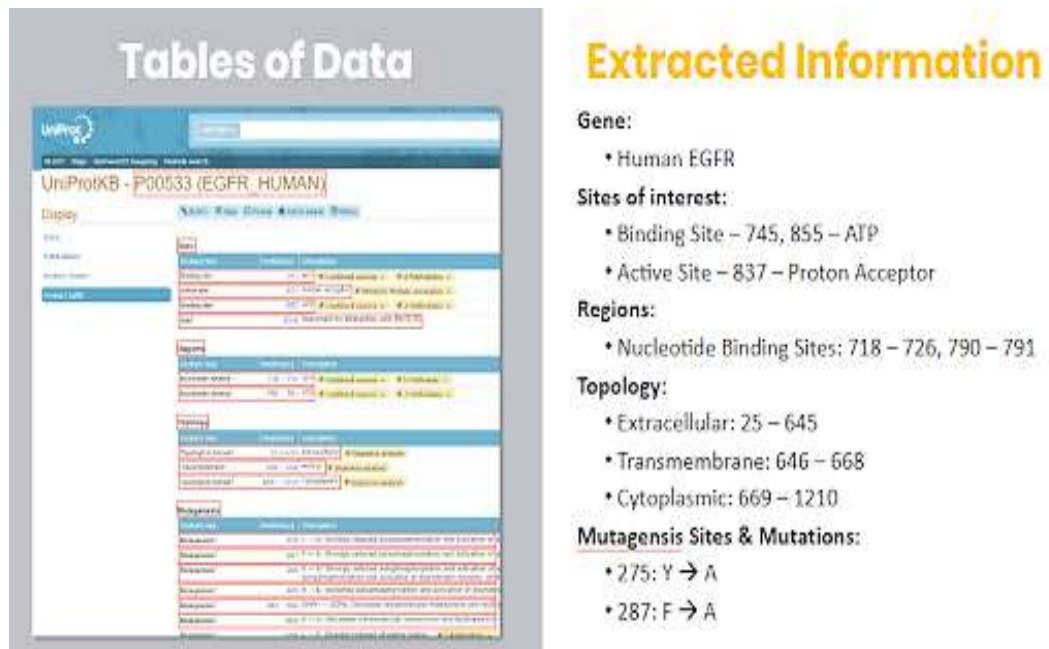


Fig. 5.37: Data Extraction Exemplary from PDFs

Future Potential of Computer Vision in Healthcare

The scope of artificial intelligence and machine learning algorithms in healthcare has always been great. It is predicted to effectively reduce hospitalizations, discover new cures, accelerate drug development, and drive better insights into patient care. Facilitating research through technologies such as artificial intelligence, machine learning, and computer vision has become even more important in recent years. Since 2017, more and more healthcare providers are focusing on computer-aided diagnosis, image-guided therapy, and other computer vision usage. The need for research acceleration for more discoveries and faster drug development is growing and also becoming possible with the use of technology.

As more healthcare companies invest in software that uses AI to empower their data analytics team, research in healthcare will boost and more quality improvements will be seen in patient care in the coming years. The healthcare industry can benefit tremendously with the use of computer vision for more accurate diagnostics, quicker drug development, and better research. Computer vision can be used to analyze images in high detail and with much more accuracy than human vision is capable of. This will save both time and energy put in by data analytics teams which spend more than 75% time in collecting and processing data. Likewise, it will also cut costs and accelerate decision making, accommodating the healthcare industry with colossal improvements.

Multi-Task Network for Text Extraction

MaskRCNN is a case of a multi-task network: with a single input (image), the model must predict multiple kinds of outputs. Specifically, MaskRCNN is split into three heads, where one of the heads is anxious about proposing bounding boxes that likely contain objects of interest, another head is concerned with classifying which type of object is contained within each box, and the final head identifies a pixel-wise bitmask for estimating the segmentation in each box. Importantly, all three of the heads are relying upon a shared representation that is calculated from a deep convolutional backbone model such as ResNet or similar. These shared representation is important in multi-task learning, and allows each of the heads to back-propagate their respective errors and update this backbone representation. The overall effect of this is that each head actually becomes more accurate than if they were trained as separate models.

Taking inspiration from models like MaskRCNN, multi-task network for solving both text localization and text recognition is designed. Similar to previous approaches, the model entails a convolutional backbone for extracting image features. Both together ResNet and Densely Connected Convolutional Networks (DenseNet) are evaluated, from which it is evident that DenseNet leads to higher accuracy. Additionally, the output of the convolutional stack is then input into a Feature Pyramid Network which helps to combine high spatial resolution information from early in the stack with low-resolution but rich semantic detail from deeper in the stack. These form the basis of the convolutional backbone which is then sent to the model heads. Similar to recent object detection models, the text localization head is comprised of a two-stage mechanism with a Region Proposal Network followed by a Bounding Box Regression network.

The output of the latter component is a set of predicted boxes (RoIs) that might contain text. The second head of the model is the classification component which is tasked with estimating the class of object contained inside of each RoI, in this case, a simple binary classification (text vs. background). Finally, the text recognition head, which takes as input the feature maps from the convolutional backbone and the RoI coordinates generated from the text localization head.

This text recognition head must, for each RoI, produce a predicted sequence corresponding to the text inside each box and uses a CTC loss for training. The text recognition head is a primary point of departure for the model as compared to object detection methods, so some additional detail is advisable. In object detection, feature maps are extracted from multiple levels of the convolutional backbone and are pooled to a fixed representation through a mechanism known as RoIPool (or RoIAlign). In OCR, we must retain high spatial resolution information, so we extract features only from early blocks of the convolutional backbone.

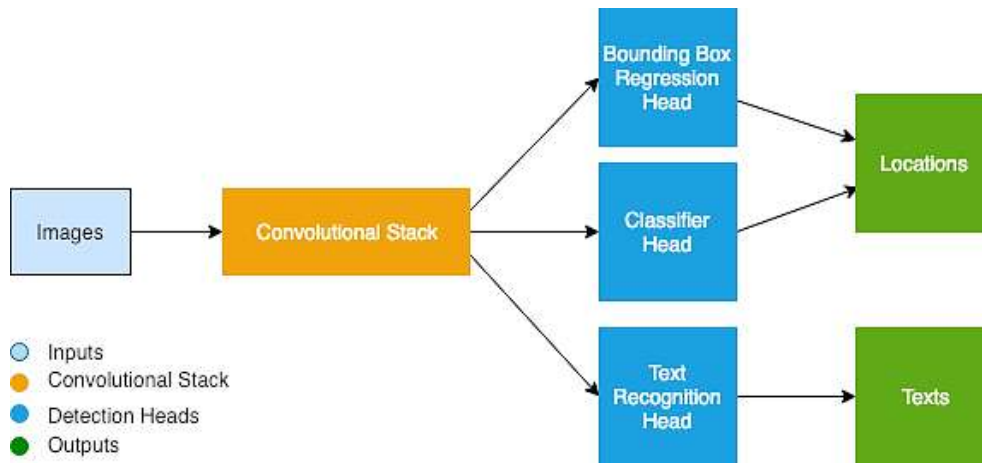


Fig. 5.38: Multi-Task Model for Text Extraction

Moreover, the model relies on a new pooling mechanism that allows objects of diverse aspect ratios to be signified without compressing long sequences or stretching short ones. Also, one is presented with vast possibilities for how to implement the text recognition head. A simple approach might be to crop the input image as identified by the RoIs from the Bound Box Regression head and then process this cropped image through an RNN architecture. The limitation of such an approach is that we do not reuse image feature representations between the heads, which would require the recognition head to do more computation on its own. Instead, our multi-task network proceeds by using the identified RoIs and then fetching the relevant representations for each region from the convolutional backbone. Each RoI's feature map is then transformed into a fixed shape as described above and the text recognition can proceed.

A typical method for sequence classification and sequence labelling tasks would be to use an RNN design of some kind where we sequentially process from “left to right” across the RoI's approximately 200 horizontal spatial steps and try to predict an output label for each location in the RoI feature map. However, it is found that RNNs perform quite poorly for text recognition here. This is likely due to the fact that we don't really need to account for long-range correlations in this sequence in order to decode the text. Instead, we only need to look at a few “feature columns” at a time in order to get a sense of which character is being represented. For this reason, we find it most useful to use simple convolutional methods here with short-range kernel widths.

At every single spatial step, the output of convolutions is used to predict an output letter, and then the overall sequence is collapsed through the CTC layer to output the final sequence for

the ROI. In order to train the model described here, we would need a large number of labelled images. In lieu of tagging and generating these manually, we instead chose to develop synthetic training documents. With enough variability in fonts, sizes, colors, distractions objects, and so on, the synthetic data should result in a model which is able to perform well on real-world images. Consequently, text segments are identified apart from background pixels and other image objects. This model is able to accurately identify text object which occur with a large variety of aspect ratios, fonts, font sizes, and colors.

Computer vision has brought impressive advances to numerous areas including medical diagnostics, robotics, and autonomous driving. In this section, we've discussed use cases in document analysis. Relying upon recent work in object detection, multi-task network is able to concurrently localize and recognize text blocks in arbitrarily complex documents. This is a promising approach for increasing efficiency of document processing pipelines and sheds light on the power of machine learning to transform the future of work in this arena. Similarly, Image analytics is the automatic algorithmic extraction and logical analysis of information found in image. With an explosion of image data, which makes up about 80 percent of all unstructured big data, there is a growing need of analytical systems to infer images, which is unstructured data to machine readable format. Case in point, the use of bar codes and QR codes are simple and popular examples of image analytics that we encounter in our day-to-day lives.

Data extraction from documents, or document analysis, has become one of the most highly sought after capabilities from businesses. They are looking to the tech industry and the world of AI to solve this, now more than ever. It's been something that we have been working on very closely with various use cases across different industries. Whenever presenting a solution, the most common question we get is if others can use "that solution" to extract data from their client's documents to optimize their business processes. The answer is maybe, but there is no one size fits all. We are going to share the different aspects that you need to consider when looking at a document data extraction use case and share some real examples.

First, some clarity on the technical capability versus the business value. The tech capability is making sense of and extracting data from documents to be used in some way. Some examples are making a document more searchable with tagged metadata, automatically classifying and routing it to the right place, or automating business application by feeding it critical data from those documents. The business value is it helps automate and scale the business process, saving you a lot of time, effort and money.

When looking at any document understanding or document extraction use case, the first thing you need to do is understand the types of documents you are dealing with. Are these unstructured documents such as emails or news articles? Are these structured documents such

as application forms or shipping labels? Do they contain tables of data? What is the file format, are these PDFs, Word documents, JPEGs, PNGs, etc.? If PDFs, are these digital or scanned? If they are structured forms as digital PDFs or Word documents, is there a template being used, is there a finite set of formats or is there a large or infinite number formats? If they are scanned PDFs or image files, is the image clear, with the same orientation, and same quality? There are many important differences and permutations within documents and the above are some of the most critical aspects to look for. The approach you take to implementing a robust solution will depend on those variables. Again, it is not a one size fits all. Let's look at a few examples.

Note: these solutions are examples, based on real implementations, and all will depend on the nature of your data. You can also use combinations of the solutions below to achieve your goals.

(1) E-mail

There are many use cases where email communication is analyzed for certain things. This could be anything, auto damage claims, service desk inquiries, bullying, inappropriate behavior, etc. Let's take "claims" as the example where you need to automate the claims process by helping your customers file claims more quickly, using AI to identify everything you need to automate the process. In your support organization's incoming emails from your customers, you need to detect things like type of claim (e.g. auto, home) accident date, type of auto damage (e.g. rear bumper is dented, passenger side door was smashed), injury sustained (e.g. left arm broken, extreme neck and back pain).

A solution for this type of use case would be to train a deep learning model to learn and understand the claims domain. You can use the IBM Watson Natural Language Classifier service (or IBM Watson Assistant) to classify each email to understand the intent. You can then use IBM Watson Knowledge Studio to train a ML model, and deploy it to either IBM Watson Discovery or the IBM Watson Natural Language Understanding service, to analyze and extract specific metadata around claims. You can also use Watson Tone Analyzer to identify the emotion of the customer (e.g. angry or frustrated) to prioritize your email actions. These are key differentiating products in the AI market offered on the IBM Watson platform. Based on all these Watson AI-powered insights, you are able to automate your claims process by assisting customer service agents working these claims or automatically filing claims for customers.

(2) Form Automation

In this use case, you're a financial organization and need to validate client financial documents in your audit and underwriting processes. You look to validate data such as personal and

financial information. These are digital PDF, structured documents with a known set of templates provided by your financial organization. A solution for this would entail using IBM Watson's Natural Language Classifier service (or IBM Watson Assistant) to first classify the type of document that you are analyzing. Once you know what type of document you are analyzing, then you know what type of information that you are looking for. Next, you need to understand the layout of the document. This can be achieved a number of different ways such as creating a custom machine learning model leveraging IBM Watson Studio and use coordinate-based annotation to do this. You can also use IBM's Business Automation Content Analyzer which is an offering that can be trained to understand the structure of your document. Once you understand the layout of the document, you can leverage IBM Watson's Visual Recognition service to detect objects like checkboxes and validate if a signature has been provided.

(3) Invoice Management

In this use case, you are a shipping department that is trying to validate invoices or purchase orders from various different vendors, all with different formats and templates. These are primarily scanned images (converted to PDFs) and do not have perfect orientation or cosmetics. Your technical goal is to extract and validate buyer info, shipper info, and item information such as type of item, quantity, price etc. to ensure that the right items have been billed, shipped and received. A solution for this would be to leverage the IBM Watson Compare and Comply service's "invoice understanding" model. This is a pre-trained model geared towards understanding various types of invoices and billing documents to extract key/value pairs of data and understand table structures.

This accounts for many different variations of documents where the formats are not all known. As pointed out some of the most common use cases and solutions, there are many other variations that can be achieved leveraging the IBM Watson and Cloud Platform. Many organizations look to optimize their business process using Watson AI and automation to increase their efficiency and scalability. So next time you see a data extraction solution and wonder if "it" can work for you, you'll be better positioned and aware of the key aspects to look for to help you answer that question.

Most industries working with large amounts of data have recognized the value of deep learning technology. By gleaning insights from this data, often in real time, organizations are able to work more efficiently or gain an advantage over competitors.

Financial services: Banks and other businesses in the financial industry use machine learning technology for two key purposes: to identify important insights in data, and prevent fraud. The insights can identify investment opportunities, or help investors know when to trade. Data

mining can also identify clients with high-risk profiles, or use cyber surveillance to pinpoint warning signs of fraud.

Government: Government agencies such as public safety and utilities have a particular need for machine learning since they have multiple sources of data that can be mined for insights. Analyzing sensor data, for example, identifies ways to increase efficiency and save money. Machine learning can also help detect fraud and minimize identity theft.

Health care: Machine learning is a fast-growing trend in the health care industry, thanks to the advent of wearable devices and sensors that can use data to assess a patient's health in real time. The technology can also help medical experts analyze data to identify trends or red flags that may lead to improved diagnoses and treatment.

Retail: Websites recommending items you might like based on previous purchases are using machine learning to analyze your buying history. Retailers rely on machine learning to capture data, analyze it and use it to personalize a shopping experience, implement a marketing campaign, price optimization, merchandise supply planning, and for customer insights.

Oil and gas: Finding new energy sources. Analyzing minerals in the ground. Predicting refinery sensor failure. Streamlining oil distribution to make it more efficient and cost-effective. The number of machine learning use cases for this industry is vast and still expanding.

Transportation: Analyzing data to identify patterns and trends is key to the transportation industry, which relies on making routes more efficient and predicting potential problems to increase profitability. The data analysis and modeling aspects of machine learning are important tools to delivery companies, public transportation and other transportation organizations.

Automating Data Extraction for Predictive Analytics

A deep learning model could replace the tedious process of manually extracting data for predictive analytics tools. Deep learning algorithm can automate the process of annotating training datasets for predictive analytics tools, a promising advancement as certain datasets grow increasingly large.

Developed by Massachusetts Institute of Technology (MIT), the model automatically learns features predictive of vocal cord disorders, and could help develop methods to prevent, diagnose, and treat the disorder. Training datasets typically consist of many sick and healthy subjects, but with fairly little data for each subject. Experts must find those aspects or features in the datasets that will be important for making predictions.

Dig Deeper

This process can be tedious and expensive, and it has become all the more challenging with the rise of wearable devices. Experts can now monitor patients' biometrics over long periods, including their sleeping patterns or voice activity. After just one week of monitoring, experts could have billions of data samples for each subject. "It's becoming increasingly easy to collect long time-series datasets. But you have physicians that need to apply their knowledge to labelling the dataset.

We want to remove that manual part for the experts and offload all feature engineering to machine-learning model. The group built a machine learning model to automate this process, focusing on predictive features of vocal cord disorders. The features came from a dataset of about 100 subjects, each with about a week's worth of voice-monitoring data and several billion samples. Typically, experts would need to manually identify features that may be useful for a model to detect various diseases or conditions. This helps prevent overfitting, a mutual machine learning problem in healthcare that occurs when a model memorizes subject data instead of learning just the clinically relevant features. When tested, those models often fail to distinguish similar patterns in previously unseen subjects. Instead of learning features that are clinically significant, a model sees patterns and predicts. So, it's just memorizing patterns of subjects. Then, when it sees data from Andrew, which has a new vocal usage pattern, it can't figure out if those patterns match a classification.

Researchers had to find a way to prevent overfitting while automating manual feature extraction. The team forced the model to learn features without subject information, which meant capturing all moments when subjects speak, as well as the intensity of their voices. In several trials, the model automatically extracted features from this data and used these features to classify, with high accuracy, patients with and without vocal cord nodules. The algorithm performed as accurately as state-of-the-art models that require manual feature engineering. The model was also able to do this without a large set of hand-labeled data. This research adds to MIT's efforts to improve machine learning algorithms and accelerate their adoption in healthcare. A team from the institution developed an automated system that can gather more data from images used to train deep learning models. The system can be used to improve the training of machine learning models, helping them to better detect anatomical structures in new scans.

"We're hoping this will make image segmentation more accessible in realistic situations where you don't have a lot of training data. In dip deeper approach, you can learn to mimic the variations in unlabelled scans to intelligently synthesize a large dataset to train your network. While the feature extraction tool can be adapted to learn patterns in any disease or condition, it

has significant implications for identifying vocal cord disorders and alerting people to potentially damaging vocal behaviors, the team said. Going forward, the researchers want to monitor how different treatments, including surgery and vocal therapy, impact vocal behavior. The group also expects to use a similar method on electrocardiogram data, which is used to track muscular functions of the heart. In order to understand exactly how valuable computer vision can be in gathering web data, you first need to understand what makes it unique that is to say, what separates it from general AI. Artificial Intelligence is simply the use of computer systems to perform tasks and functions that usually require human intelligence. In other words, “getting machines to think and act like humans.” Computer vision, on the other hand, describes the ability of machines to process and understand visual data; automating the type of tasks the human eye can do. “Advanced Computer Vision is Deep Artificial Intelligence applied to the visual world.

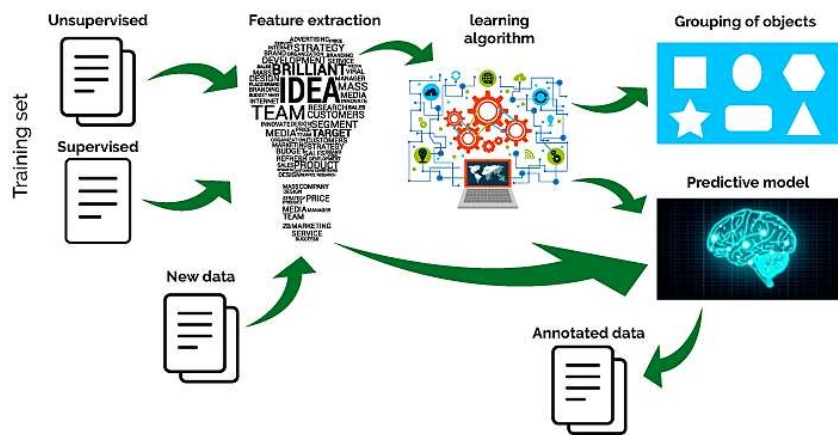


Fig. 5.39: Data Extraction for Predictive Analytics

5.12 WHAT IS DETECTION AND RECOGNITION?

Computer Vision is also composed of innumerable aspects such as image recognition, object detection, image generation, image super-resolution and more. Object detection is probably the most profound aspect of computer vision due the number practical use cases. In this section, we will briefly introduce the concept of modern object detection, challenges faced by software developers, the solution my team has provided as well as code tutorials to perform high performance object detection.

Object detection refers to the capability of computer and software systems to locate objects in an image/scene and identify each object. Object detection has been widely used for face detection, vehicle detection, pedestrian counting, web images, security systems and driverless

cars. There are many ways object detection can be used as well in many fields of practice. Like every other computer technology, a wide range of creative and amazing uses of object detection will definitely come from the efforts of computer programmers and software developers. Getting to use modern object detection methods in applications and systems, as well as building new applications based on these methods is not a straight forward task. Early implementations of object detection involved the use of classical algorithms, like the ones supported in OpenCV, the popular computer vision library. However, these classical algorithms could not achieve enough performance to work under different conditions.

The breakthrough and rapid adoption of deep learning in 2012 brought into existence modern and highly accurate object detection algorithms and methods such as R-CNN, Fast-RCNN, Faster-RCNN, RetinaNet and fast yet highly accurate ones like SSD and YOLO. Using these methods and algorithms, based on deep learning which is also based on machine learning require lots of mathematical and deep learning frameworks understanding. There are millions of expert computer programmers and software developers that want to integrate and create new products that uses object detection. But this technology is kept out of their reach due to the extra and complicated path to understanding and making practical use of it. There's no shortage of interesting problems in computer vision, from simple image classification to 3D-pose estimation. One of the problems we're most interested in and have worked on a bunch is object detection.

Like many other computer vision problems, there still isn't an obvious or even "best" way to approach object detection problems, meaning there's still much room for improvement.

Image Classification



Fig. 5.40: ImageNet Classification with Deep CNNs

Probably the most well-known problem in computer vision. It consists of classifying an image into one of many different categories. One of the most popular datasets used in academia is ImageNet, composed of millions of classified images, (partially) utilized in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). In recent years classification models have surpassed human performance and it has been considered practically solved. While there are plenty of challenges to image classification, there are also plenty of write-ups on how it's usually solved and which the remaining challenges are.

Localization

Similar to classification, localization finds the location of a single object inside the image. Localization can be used for lots of useful real-life problems. For example, smart cropping (knowing where to crop images based on where the object is located), or even regular object extraction for further processing using different techniques. It can be combined with classification for not only locating the object but categorizing it into one of many possible categories.

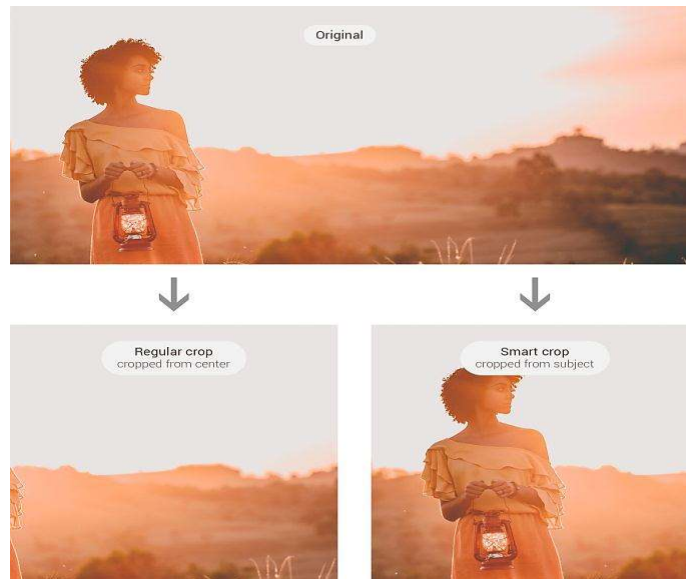


Fig. 5.41: Regular Crop and Smart Crop

Instance Segmentation

Going one step further from object detection we would want to not only find objects inside an image, but find a pixel by pixel mask of each of the detected objects. We refer this problem as instance or object segmentation.

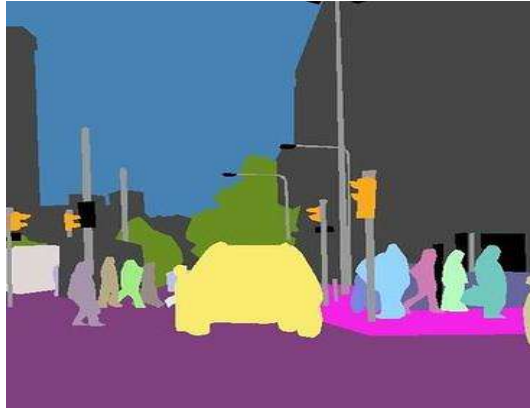


Fig. 5.42: Instance Segmentation

Object Detection

Iterating over the problem of localization plus classification we end up with the need for detecting and classifying multiple objects at the same time. Object detection is the problem of finding and classifying a variable number of objects on an image. The important difference is the “variable” part. In contrast with problems like classification, the output of object detection is variable in length, since the number of objects detected may change from image to image.

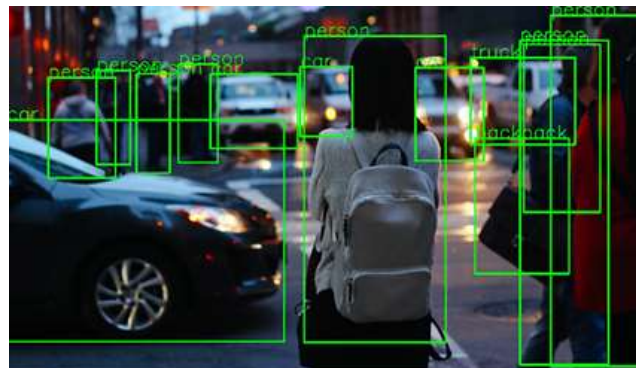


Fig. 5.43: Object Detection

Practical Uses of Object Detection

In applying state of the art machine learning to solve business problems, so even though we love all the crazy machine learning research problems, at the end of the day we end up worrying a lot more about the applications. Even though object detection is somewhat still of a new tool in the industry, there are already many useful and exciting applications using it.

(1) Face Detection

Since the mid-2000s some point and shoot cameras started to come with the feature of detecting faces for a more efficient auto-focus. While it's a narrower type of object detection, the methods used apply to other types of objects as well.

(2) Counting

One simple but often ignored use of object detection is counting. The ability to count people, cars, flowers, and even microorganisms, is a real world need that is broadly required for different types of systems using images. Recently with the ongoing surge of video surveillance devices, there's a bigger than ever opportunity to turn that raw information into structured data using computer vision.

(3) Visual Search Engine

Finally, one use case we're fond of is the visual search engine of Pinterest. They use object detection as part of the pipeline for indexing different parts of the image. This way when searching for a specific purse, you can find instances of purses similar to the one you want in a different context. This is much more powerful than just finding similar images, like Google Image's reverse search engine does.

(4) Aerial Image Analysis

In the age of cheap drones and (close to) affordable satellite launches, there has never been that much data of our world from above. There are already companies using satellite imagery from companies like Planet and Descartes Labs, applying object detection to count cars, trees and ships. This has resulted in high quality data, which was impossible (or extremely expensive) to get before, now reaching a broader audience. Some companies are using drone footage for automatic inspections on hard to reach places (e.g. Better View) or using object detection for general purpose analysis (e.g. Tensor Flight). On top of this, some companies add automatic detection and location of problems without the need for human intervention.

Problems and Challenges with Object Detection

Let's start getting deeper into which are the main issues of object detection.

(1) Variable Number of Objects

We already mentioned the part about a variable number of objects, but we omitted why it's a problem at all. When training machine learning models, you usually need to represent data into

fixed-sized vectors. Since the number of objects in the image is not known beforehand, we would not know the correct number of outputs. Because of this, some post-processing is required, which adds complexity to the model. Historically, the variable number of outputs has been tackled using a sliding window based approach, generating the fixed-sized features of that window for all the different positions of it. After getting all predictions, some are discarded and some are merged to get the final result.

(2) Sizing

Another big challenge is the different conceivable sizes of objects. When doing simple classification, you expect and want to classify objects that cover most of the image. On the other hand, some of the objects you may want to find could be as small as a dozen pixels (or a small percentage of the original image). Traditionally this has been solved with using sliding windows of different sizes, which is simple but very inefficient.

(3) Modeling

A third challenge is solving two problems at the same time. How do we combine the two different types of requirements: location and classification into, ideally, a single model? Before diving into deep learning and how to tackle these challenges, let's do a quick run-up of the classical methods.

Classical Approach

Although there have been many different types of methods throughout the years, we want to focus on the two most popular ones (which are still widely used). The first one is the Viola-Jones framework proposed in 2001 by Paul Viola and Michael Jones in the paper Robust Real-time Object Detection. The approach is fast and relatively simple, so much that it's the algorithm implemented in point-and-shoot cameras which allows real-time face detection with little processing power. We won't go into details on how it works and how to train it, but at the high level, it works by generating different (possibly thousands) simple binary classifiers using Haar features. These classifiers are assessed with a multi-scale sliding window in cascade and dropped early in case of a negative classification. Another traditional and similar method is using Histogram of Oriented Gradients (HOG) features and Support Vector Machine (SVM) for classification. It still requires a multi-scale sliding window, and even though it's superior to Viola-Jones, it's much slower.

Deep Learning Approach

It's not news that deep learning has been a real game changer in machine learning, especially in computer vision. In a similar way that deep learning models have crushed classical models

on the task of image classification, deep learning models are now state of the art in object detection as well. Now that you probably have a better intuition on what the challenges are and how to tackle them, we will do an overview on how the deep learning approach has evolved in the last couple of years. Since most of the deep learning algorithms have been dealt in depth in the earlier chapters, here we present the lighter notes.

(1) Over Feat

One of the first advances in using deep learning for object detection was Over Feat from NYU published in 2013. They proposed a multi-scale sliding window algorithm using Convolutional Neural Networks (CNNs).

(2) R-CNN

Quickly after Over Feat, Regions with CNN features or R-CNN from Ross Girshick, et al. at the UC Berkeley was published which boasted an almost 50% improvement on the object detection challenge. What they proposed was a three stage approach:

-  Extract possible objects using a region proposal method (the most popular one being Selective Search).
-  Extract features from each region using a CNN.
-  Classify each region with SVMs.

While it achieved great results, the training had lots of problems. To train it you first had to generate proposals for the training dataset, apply the CNN feature extraction to every single one (which usually takes over 200GB for the Pascal 2012 train dataset) and then finally train the SVM classifiers.

(3) Fast R-CNN

This approach quickly evolved into a purer deep learning one, when a year later Ross Girshick published Fast R-CNN. Similar to R-CNN, it used Selective Search to generate object proposals, but instead of extracting all of them independently and using SVM classifiers, it applied the CNN on the complete image and then used both Region of Interest (RoI) Pooling on the feature map with a final feed forward network for classification and regression. Not only was this approach faster, but having the RoI Pooling layer and the fully connected layers allowed the model to be end-to-end differentiable and easier to train. The biggest downside was that the model still relied on Selective Search (or any other region proposal algorithm), which became the bottleneck when using it for inference.

(4) YOLO

You Only Look Once: Unified, Real-Time Object Detection (YOLO) paper published by Joseph Redmon (with Girshick appearing as one of the co-authors). YOLO proposed a simple convolutional neural network approach which has both great results and high speed, allowing for the first time real time object detection.

(5) Faster R-CNN

Subsequently, Faster R-CNN authored by Shaoqing Ren (also co-authored by Girshick, now at Facebook Research), the third iteration of the R-CNN series. Faster R-CNN added what they called a Region Proposal Network (RPN), in an attempt to get rid of the Selective Search algorithm and make the model completely trainable end-to-end. We won't go into details on what the RPNs do, but in abstract it has the task to output objects based on an "objectness" score. These objects are used by the RoI Pooling and fully connected layers for classification.

(6) SSD and R-FCN

Finally, there are two notable papers, Single Shot Detector (SSD) which takes on YOLO by using multiple sized convolutional feature maps achieving better results and speed, and Region-based Fully Convolutional Networks (R-FCN) which takes the architecture of Faster R-CNN but with only convolutional networks.

Importance of Datasets

Datasets play a very important (and sometimes underrated) role in research. Every time a new dataset is released, papers are released, and new models are compared and often improved upon, pushing the limits of what's possible. Unfortunately, there aren't enough datasets for object detection. Data is harder (and more expensive) to generate, companies probably don't feel like freely giving away their investment, and universities do not have that many resources.

ISBN: 978-93-48116-19-2

Author Details



Dr. Jagadeesh Kumar, Professor in the Department of Robotics and AI, NIMS University Rajasthan, is a renowned Scientist, Researcher, and Academician. He received his educational credentials in Engineering, Technology, and Management from the Top Universities and a notable Alumnus of Harvard University, United States and University of Oxford, United Kingdom. He authored more than 100 journal, conference, and magazine articles indexed in Scopus, Web of Science and SCI. Also, he is an author of more than 50 international books.

Xoffencer International Publication
838- Laxmi Colony. Dabra,
Gwalior, Madhya Pradesh, 475110
www.xoffencerpublication.in



MRP: ₹ 550/-

